

한 점에서 반사된 것은 다음 식으로 주어진다:

$$c_d = k_d \cdot l_d \otimes m_d = k_d D \quad (\text{식 7.1})$$

Incoming Light

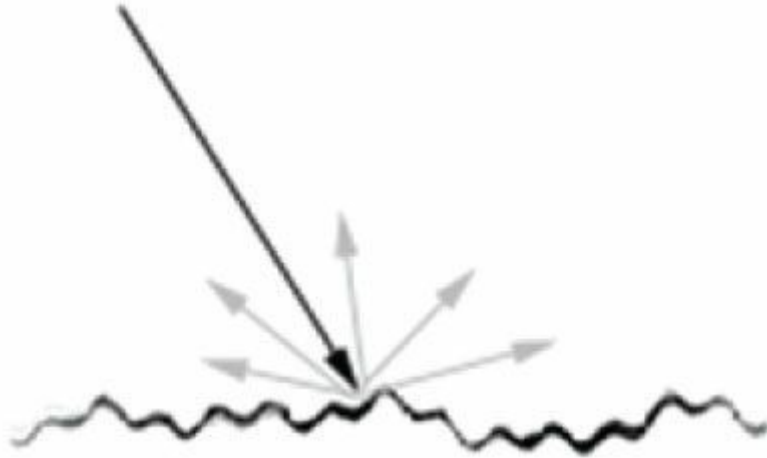


그림 7.10. 확산 표면에 입사하는 빛은 무작위 방향으로 산란됩니다. 이는 표면이 미시적 수준에서 거칠다는 개념을 반영합니다.

7.5 주변 조명

앞서 언급했듯이, 우리의 조명 모델은 장면 내 다른 물체에 반사된 간접광을 고려하지 않습니다. 그러나 현실 세계에서 우리가 보는 빛의 상당 부분은 간접광입니다. 예를 들어, 방과 연결된 복도는 방 안의 광원과 직접적인 시야선에 있지 않을 수 있지만, 빛이 방 벽에 반사되어 일부가 복도로 들어와 약간 밝게 만들 수 있습니다. 두 번째 예로, 책상 위에 찻주전자가 놓인 방에 앉아 있고 방 안에 하나의 광원이 있다고 가정해 보겠습니다. 찻주전자의 한쪽 면만 광원의 직선 시야에 있지만, 그림에도 찻주전자의 뒷면이 칠흑같이 어둡지는 않을 것입니다. 이는 일부 빛이 방 안의 벽이나 다른 물체에 산란되어 결국 찻주전자의 뒷면에 닿기 때문입니다.

이러한 간접광을 어느 정도 활용하기 위해 조명 방정식에 주변광 항을 도입합니다:

$$A = l_a \otimes m_a$$

색상 l_a 는 표면이 광원에서 받는 간접(주변)광의 총량을 지정합니다. 주변 재료 색상 m_a 는 표면이 반사하고 흡수하는 주변광의 양을 지정합니다. 주변광은 단순히 물체를 균일하게 약간 밝게 할 뿐이며, 실제 물리적 계산은 전혀 없습니다. 간접광은 장면 주변을 여러 번 산란 및 반사되어 여러 번 산란 및 반사되어 모든 방향에서 균일하게 물체에 도달한다는 개념입니다.

주변광 항과 확산광 항을 결합하면 새로운 조명 방정식은 다음과 같습니다:

$$\begin{aligned} LitColor &= l_a \otimes m_a + k_d \cdot l_d \otimes m_d \\ &= A + k_d D \end{aligned} \quad (\text{eq. 7.2})$$

7.6 반사광 조명

그림 7.11과 같은 매끄러운 표면을 생각해 보자. 빛이 이러한 표면에 닿으면 광선은 *반사 원뿔*을 통해 일반적인 방향으로 날카롭게 반사된다. 이를 *정반사*(specular reflection)라고 한다. 확산광과 달리 정반사광은 특정 방향으로 반사되기 때문에 눈으로 들어오지 않을 수 있다. 정반사광 계산은 시점에 따라 달라진다. 즉, 눈이 장면 내에서 움직일수록 받는 정반사광의 양이 변한다는 의미이다.

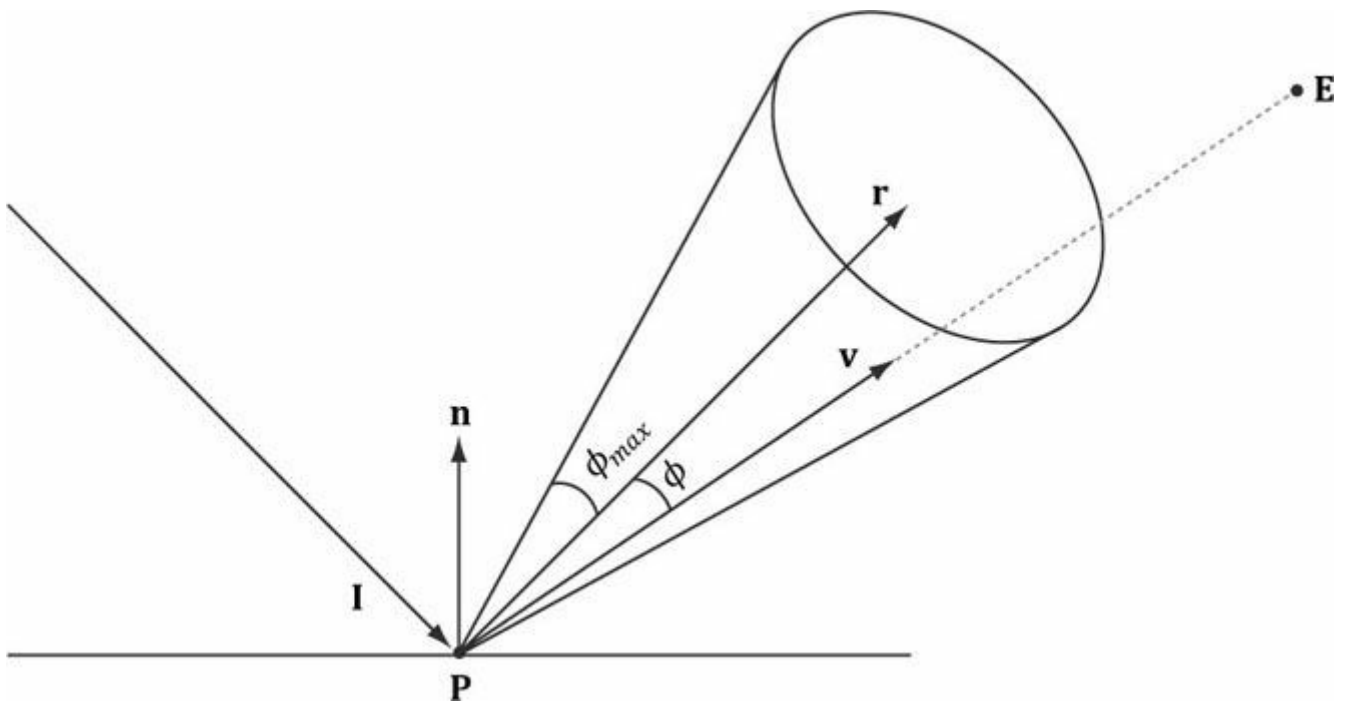


그림 7.11. 입사 광선은 $\mathbf{l}$ 로 표시된다. 정반사광은 모든 방향으로 산란되지 않고, 대신 우리가 매개변수로 크기를 제어할 수 있는 일반적인 반사 원뿔을 통해 반사된다. $\mathbf{v}$ 가 원뿔 내에 있으면 눈은 정반사광을 수신하지만, 그렇지 않으면 수신하지 않는다. $\mathbf{v}$ 가 반사 벡터 $\mathbf{r}$ 과 가까울수록 눈이 수신하는 정반사광은 더 많아진다.

경광이 반사되는 원뿔은 반사 벡터 $\mathbf{r}$ 에 대한 각도 ϕ_{max} 로 정의됩니다. 직관적으로, 반사 벡터 $\mathbf{r}$ 과 시점 벡터 $\mathbf{v}$ (view vector) 사이의 각도 ϕ 에 따라 경광 강도를 변화시키는 것이 합리적입니다.

$$\mathbf{V} = \frac{\mathbf{E} - \mathbf{P}}{\|\mathbf{E} - \mathbf{P}\|} \quad (\text{즉, 표면 점 } \mathbf{P} \text{에서 눈 위치 } \mathbf{E} \text{까지의 단위 벡터}) \text{를 다음과 같이 정의한다: 우리는}$$

반사광 강도는 $\phi = 0$ 일 때 최대가 되며, ϕ 가 ϕ_{max} 에 접근함에 따라 부드럽게 0으로 감소한다고 규정합니다. 이를 수학적으로 모델링하기 위해 람베르트 코사인 법칙에서 사용된 함수를 수정합니다. 그림 7.12는 $p \geq 0$ 에 대한 코사인 함수의 그래프를 보여줍니다.

1. 본질적으로, 서로 다른 p 를 선택함으로써 우리는 빛의 강도가 0으로 떨어지는 원뿔 각도 ϕ_{max} 를 간접적으로 제어합니다. 매개변수 p 는 표면의 광택도를 제어하는 데 사용될 수 있습니다. 즉, 고광택 표면은 덜 반짝이는 표면보다 반사 원뿔이 더 작아집니다(빛이 더 날카롭게 반사됨). 따라서 무광 표면보다 광택 표면에 더 큰 p 를 사용하게 됩니다.

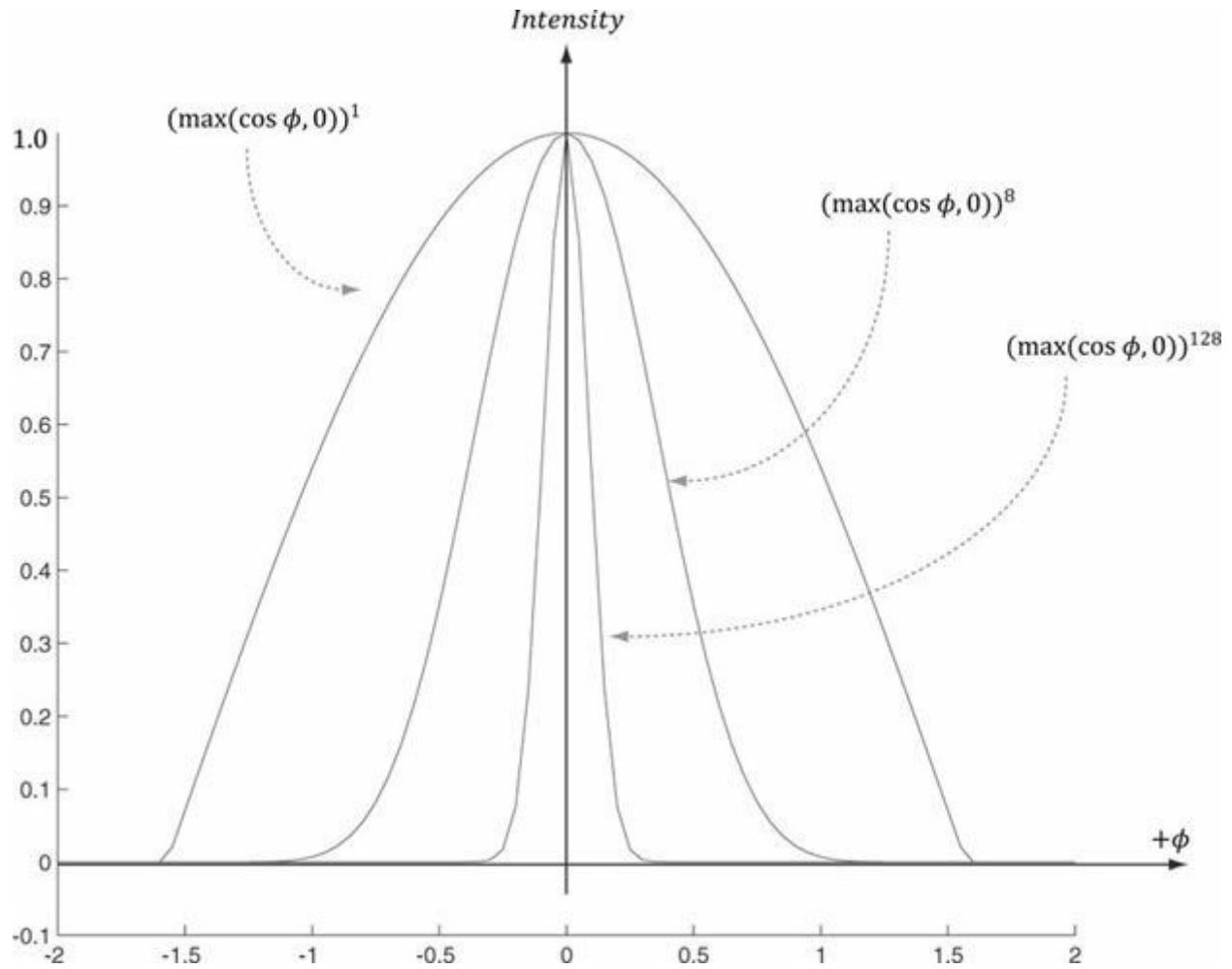


그림 7.12. $p \geq 1$ 에 대해 다른 지수를 가진 코사인 함수의 그래프.

$\mathbf{v}$ 와 $\mathbf{r}$ 이 단위 벡터이므로 $\cos(\phi) = \mathbf{v} \cdot \mathbf{r}$ 임을 유의하십시오.

눈에 도달하는 점으로부터 반사된 정반사광의 양은 다음 식으로 주어집니다:

$$\begin{aligned} \mathbf{c}_s &= k_s \cdot \mathbf{l}_s \otimes \mathbf{m}_s \\ &= k_s \mathbf{S} \end{aligned}$$

where

$$k_s = \begin{cases} \max(\mathbf{v} \cdot \mathbf{r}, 0)^p, & \mathbf{L} \cdot \mathbf{n} > 0 \\ 0, & \mathbf{L} \cdot \mathbf{n} \leq 0 \end{cases}$$

색상 $\mathbf{l}_s$ 는 광원이 방출하는 정반사광의 양을 지정합니다. 정반사 재료 색상 $\mathbf{m}_s$ 는 표면이 반사하는 입사 정반사광의 양을 지정합니다. 계수 $k_{(s)}$ 는 $\mathbf{r}$ 과 $\mathbf{v}$ 사이의 각도에 따라 반사광의 강도를 조정합니다. 그림 7.13은 표면이 확산광을 전혀 받지 않더라도($\mathbf{L} \cdot \mathbf{n} < 0$) 반사광을 받을 수 있음을 보여줍니다. 그러나 표면이 확산광을 전혀 받지 않는다면 반사광을 받는 것은 의미가 없으므로, 이 경우 $k_s = 0$ 으로 설정합니다.

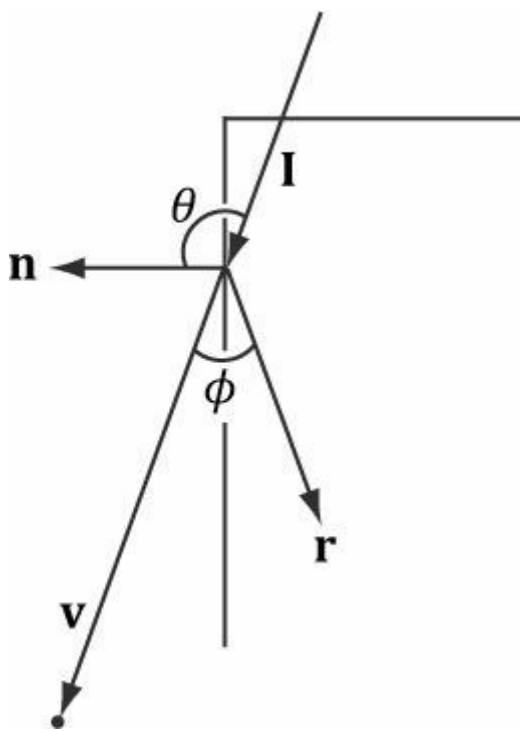


그림 7.13. 빛이 표면의 뒷면에 부딪혀도 높은 정반사광을 감지할 수 있다. 이는 잘못된 것이므로, 이러한 상황을 감지하여 해당 경우 $k_s=0$ 으로 설정해야 한다.

Note: 반사 전력 p 는 항상 1 이상이어야 합니다.

새로운 조명 모델은 다음과 같다:

$$\begin{aligned} LitColor &= \mathbf{l}_a \otimes \mathbf{m}_a + k_d \cdot \mathbf{l}_d \otimes \mathbf{m}_d + k_s \cdot \mathbf{l}_s \otimes \mathbf{m}_s \\ &= \mathbf{A} + k_d \mathbf{D} + k_s \mathbf{S} \end{aligned}$$

$$k_d = \max(\mathbf{L} \cdot \mathbf{n}, 0)$$

$$k_s = \begin{cases} \max(\mathbf{v} \cdot \mathbf{r}, 0)^p, & \mathbf{L} \cdot \mathbf{n} > 0 \\ 0, & \mathbf{L} \cdot \mathbf{n} \leq 0 \end{cases} \quad (\text{방정식 7.3})$$

Note: 반사 벡터는 다음과 같이 주어집니다: $\mathbf{r} = \mathbf{l} - 2(\mathbf{n} \cdot \mathbf{l})\mathbf{n}$; 그림 7.14 참조 (n 이 단위 벡터라고 가정합니다.) 그러나 실제로는 HLSL 내장 함수 `reflect`를 사용하여 셰이더 프로그램에서 $\mathbf{r}$ 을 계산할 수 있습니다.

입사 벡터 $\mathbf{l}$ 는 들어오는 빛의 방향(즉, 빛 벡터 $\mathbf{L}$ 의 반대 방향)임을 관찰하십시오.

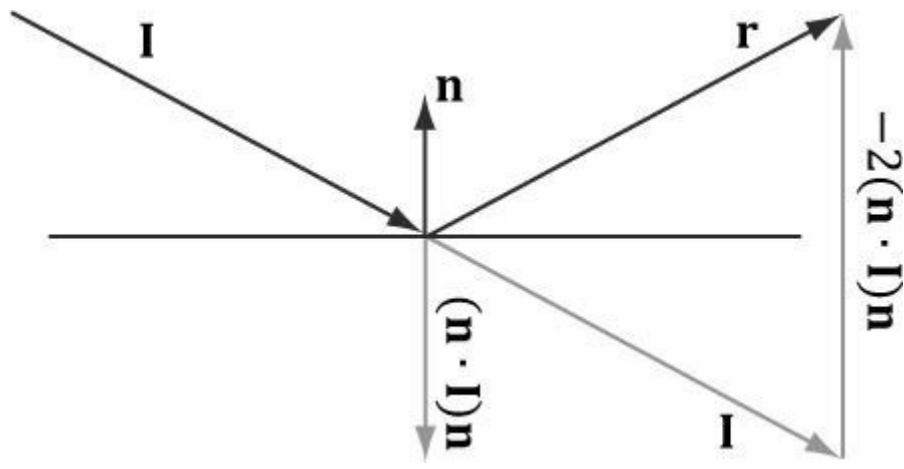


그림 7.14. 반사의 기하학.

7.7 간단한 요약

우리 모델에서 광원은 세 가지 다른 종류의 빛을 방출합니다:

1. 주변광: 간접 조명을 모델링하기 위한 것입니다.
2. 확산광: 상대적으로 거친 표면의 직접 조명을 모델링합니다.
3. 반사광: 비교적 매끄러운 표면의 직접 조명을 모델링합니다. 이에 따라 표면 점에는 다음과 같은 재료 특성이 연관됩니다:

1. 주변 재료: 표면이 반사하고 흡수하는 주변광의 양.
2. 확산 재료: 표면이 반사하고 흡수하는 확산광의 양.
3. 반사 재료: 표면이 반사하고 흡수하는 반사광의 양.
4. 반사 지수: 반사광 계산에 사용되는 지수로, 반사 원뿔의 크기를 제어하여 표면의 광택도를 결정합니다. 원뿔이 작을수록 표면은 더 매끄럽고 반짝입니다.

조명을 이렇게 세 가지 요소로 나누는 이유는 유연성을 위해서입니다. 아티스트는 원하는 결과를 얻기 위해 여러 가지 자유도를 조정할 수 있습니다. [그림 7.15](#)는 이 세 가지 요소가 함께 작동하는 방식을 보여줍니다.

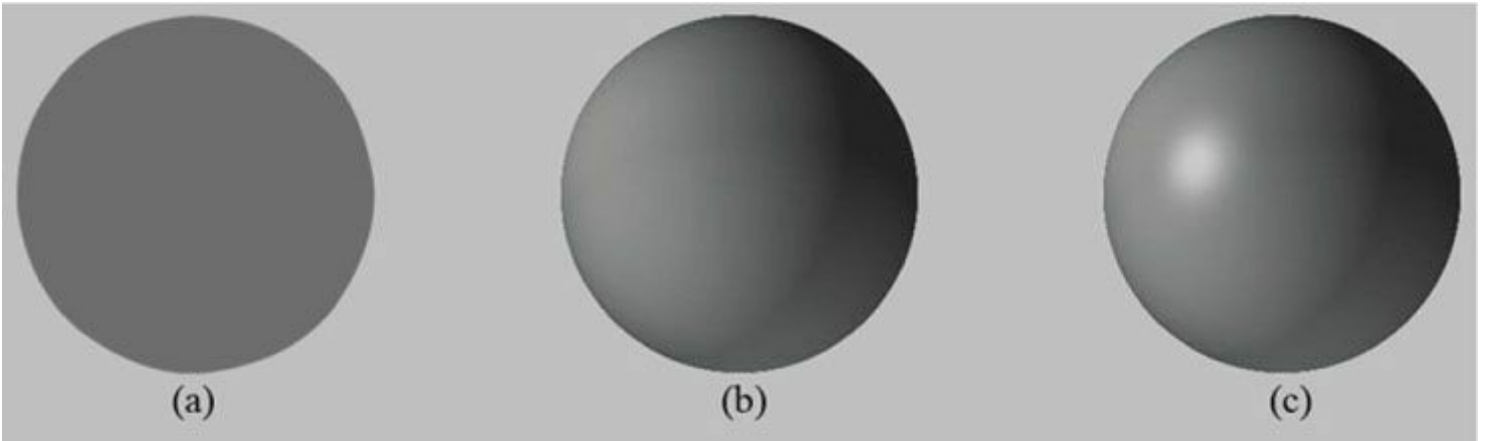


그림 7.15. (a) 주변광만으로 채색된 구체. 주변광은 구체를 균일하게 밝게 합니다. (b) 주변광과 확산광이 결합된 상태. 램버트 코사인 법칙에 따라 밝은 부분에서 어두운 부분으로 부드러운 전환이 이루어집니다. (c) 주변광, 확산광, 반사광이 모두 적용된 상태. 반사광은 스페큘러 하이라이트를 생성합니다.

7.8 재료 지정

재료 값은 어떻게 지정해야 할까요? 재료 값은 표면 전체에 걸쳐 달라질 수 있습니다. 즉, 표면의 서로 다른 지점마다 다른 재료 값을 가질 수 있습니다([그림 7.16](#) 참조). 예를 들어, 차체 프레임, 창문, 조명, 타이어가 빛을 다르게 반사하고 흡수하는 자동차 모델을 생각해 보면, 재료 값은 자동차 표면 전체에 걸쳐 달라져야 합니다.

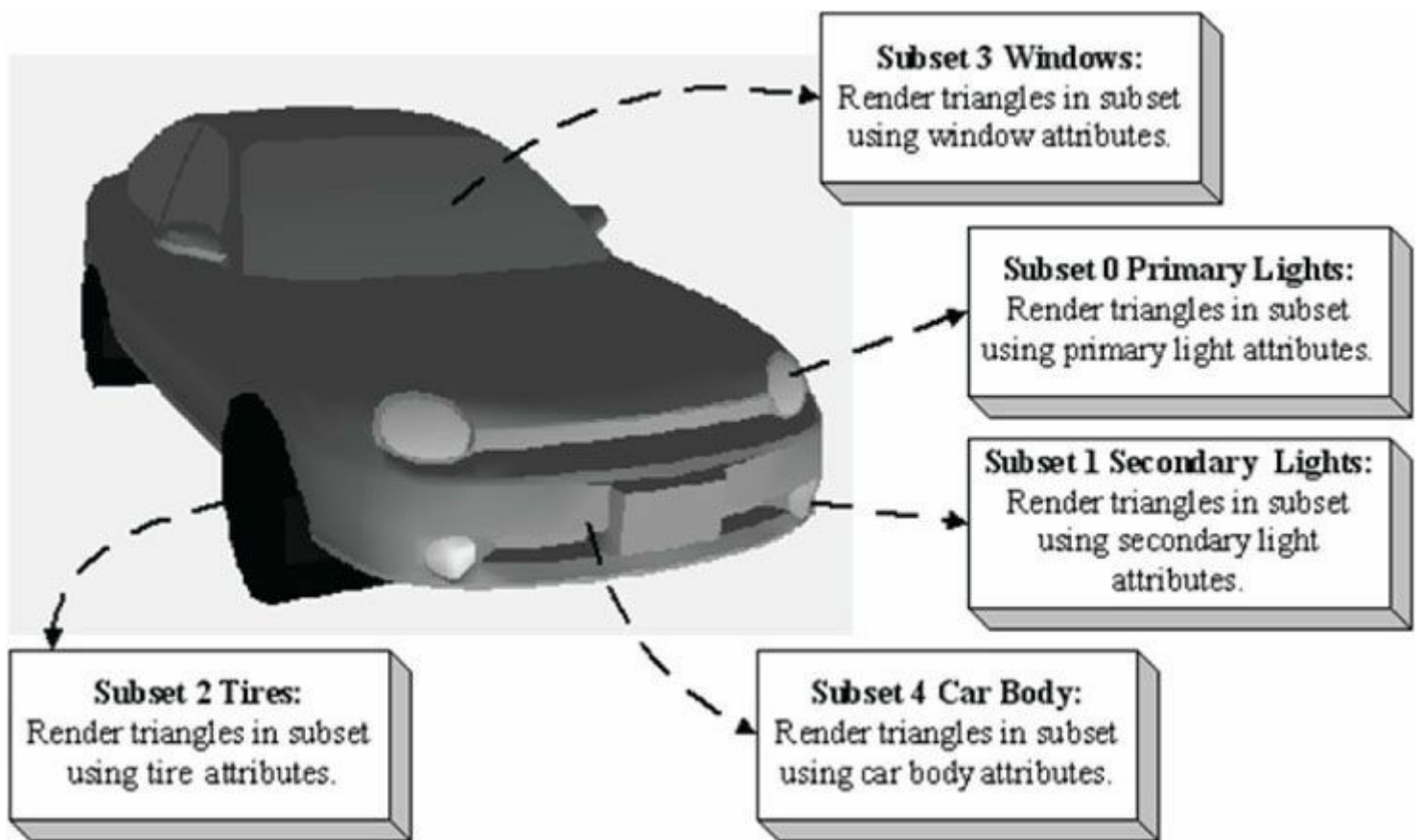


그림 7.16. 다섯 개의 재료 속성 그룹으로 분할된 자동차 메쉬.

이러한 변형을 대략적으로 모델링하기 위한 한 가지 해결책은 정점별로 재료 값을 지정하는 것입니다. 이렇게 정점별 재료는 래스터화 과정에서 삼각형 전체에 걸쳐 보관되어 삼각형 메쉬 표면의 각 점에 대한 재료 값을 제공합니다. 그러나 6장의 "언덕" 데모에서 보았듯이, 정점별 색상만으로는 미세한 디테일을 사실적으로 모델링하기에는 여전히 너무 거칠습니다. 게다가 정점별 색상은 정점 구조에 추가 데이터를 부담시키며, 이를 위한 도구가 필요합니다. 대신 널리 쓰이는 해결책은 텍스처 매핑인데, 이는 다음 장에서 다루기로 하겠습니다. 그동안 우리는 드로우 콜 주기마다 머티리얼 변경을 허용합니다. 즉, 머티리얼 값을 상수 버퍼의 멤버로 설정하면, 이후 드로우되는 모든 지오메트리는 드로우 콜 사이에서 변경될 때까지 해당 머티리얼을 사용합니다. 다음 의사 코드는 자동차를 드로우하는 방법을 보여줍니다:

기본 조명 머티리얼을 상수 버퍼에 설정 기본 조명 지오메트리 그리기

보조 조명 머티리얼을 상수 버퍼에 설정 보조 조명 지오메트리 그리기

타이어 재질을 상수 버퍼에 설정 타이어 지오메트리 그리기

창문 재질을 상수 버퍼에 설정 창문 지오메트리 그리기

차체 재질을 상수 버퍼에 설정 차체 지오메트리 그리기

우리의 머티리얼 구조는 다음과 같으며 *LightHelper.h*에 정의되어 있습니다:

```
struct Material
{
    Material() { ZeroMemory(this, sizeof(this)); }

    XMFLOAT4 Ambient;
    XMFLOAT4 Diffuse;
    XMFLOAT4 Specular; // w = SpecPower XMFLOAT4 Reflect;
```

};

반사 멤버는 당분간 무시하세요. 나중에 거울 같은 반사를 모델링할 때 표면이 거울처럼 행동하는 정도를 지정해야 할 때 사용됩니다. 또한, 스페큘러 파워 지수 p 를 스페큘러 재료 색상의 네 번째 구성 요소에 포함시킨다는 점에 유의하십시오. 이는 알파 구성 요소가 조명에 필요하지 않으므로, 빈 슬롯을 유용한 것을 저장하는 데 사용할 수 있기 때문입니다. 확산 재료의 알파 구성 요소도 후속 장에서 알파 블렌딩에 사용될 것입니다.

마지막으로, 삼각형 메쉬 표면의 각 점에서 법선 벡터가 필요하다는 점을 상기시킵니다. 이를 통해

삼각형 메쉬 표면의 각 지점에 빛이 입사하는 각도를 결정하기 위함입니다(람버트 코사인 법칙 적용). 삼각형 메쉬 표면의 각 지점에서 법선 벡터 근사값을 얻기 위해, 우리는 정점 수준에서 법선을 지정합니다. 이 정점 법선들은 래스터화 과정에서 삼각형 전체에 걸쳐 보간될 것입니다.

지금까지 우리는 빛의 구성 요소에 대해 논의했지만, 특정 종류의 광원에 대해서는 다루지 않았습니다. 다음 세 개의 섹션에서는 평행광, 점광원, 스포트라이트 구현 방법을 설명합니다.

7.9 평행광

평행광(또는 방향광)은 매우 먼 거리에 있는 광원을 근사화합니다. 따라서 모든 입사 광선을 서로 평행하다고 가정할 수 있습니다(그림 7.17). 평행 광원은 광선이 이동하는 방향을 지정하는 벡터로 정의됩니다. 광선이 평행하므로 모두 동일한 방향 벡터를 사용합니다. 이 광 벡터는 광선이 이동하는 방향과 반대 방향을 향합니다. 실제 방향성 광원의 대표적인 예는 태양입니다(그림 7.18). 방향성 광원의 방정식은 정확히 방정식 7.3과 같습니다.

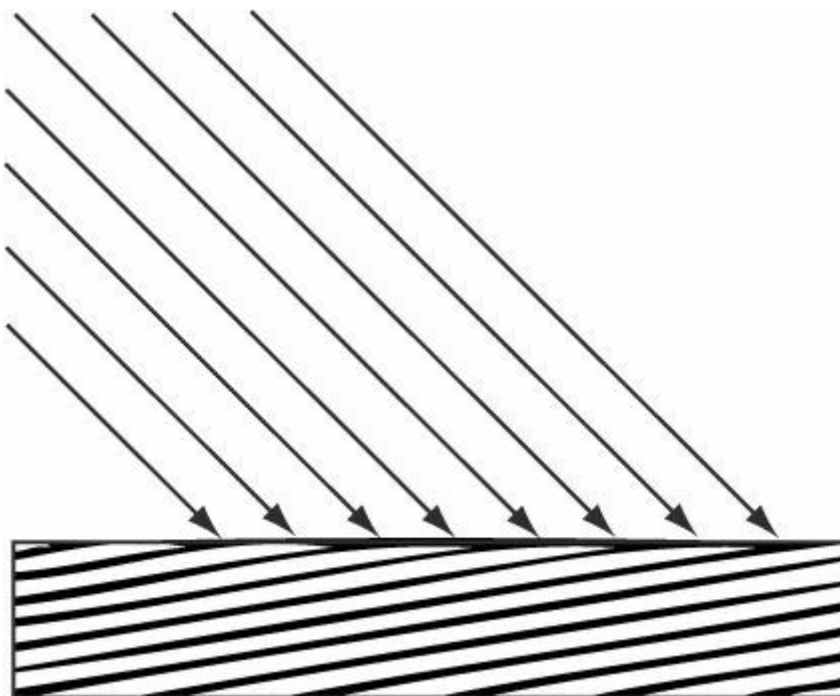


그림 7.17. 평행 광선이 표면에 입사하는 모습.



그림 7.18. 그림은 실제 크기를 나타내지 않지만, 지구 표면의 작은 영역을 선택하면 그 영역에 입사하는 광선들은 대략 평행하다.

7.10 점광원

점광원의 좋은 물리적 예는 전구이다; 이는 모든 방향으로 구면적으로 복사한다(그림 7.19). 특히, 임의의 점 P에 대해, 점광원 위치 Q에서 출발하여 해당 점을 향하는 광선이 존재한다. 일반적으로, 우리는 광벡터가 반대 방향, 즉 점 P에서 **점광원 Q로의** 방향으로 향하도록 정의한다:

$$\mathbf{L} = \frac{\mathbf{Q} - \mathbf{P}}{\|\mathbf{Q} - \mathbf{P}\|}$$

본질적으로, 점광원과 평행광원의 유일한 차이는 광선 벡터의 계산 방식에 있습니다. 점광원의 경우 광선 벡터가 점마다 달라지지만, 평행광원은 일정한 값을 유지합니다.

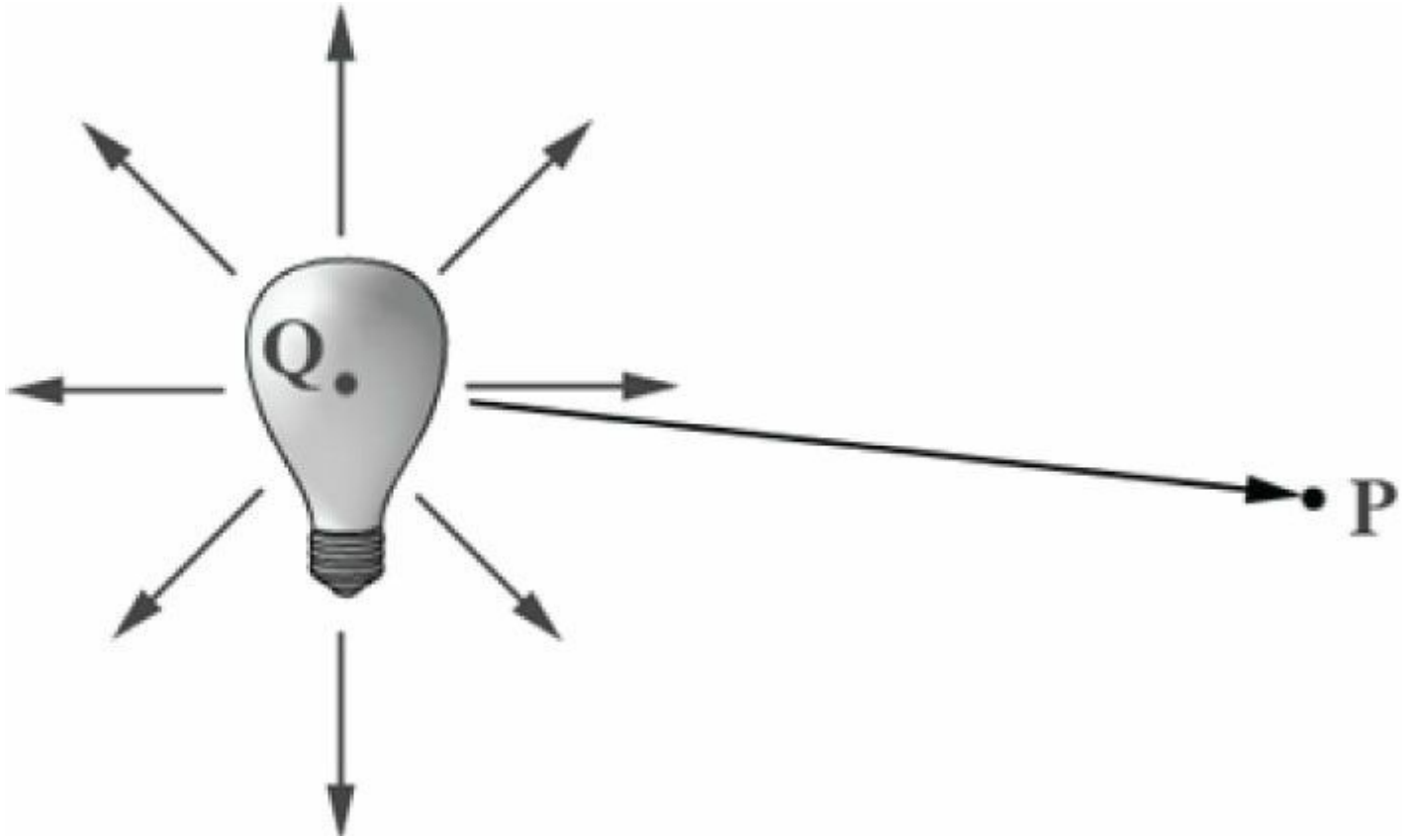


그림 7.19. 점광원은 모든 방향으로 빛을 방출합니다. 특히 임의의 점 P에 대해, 점광원 Q에서 P를 향하는 광선이 존재합니다.

7.10.1 감쇠

물리적으로 빛의 강도는 거리의 제곱에 반비례하는 법칙에 따라 약해집니다. 즉, 광원에서 거리 d만큼 떨어진 지점에서의 빛 강도는 다음과 같이 주어집니다:

$$I(d) = \frac{I_0}{d^2}$$

여기서 I_0 은 광원으로부터 거리 $d = 1$ 지점의 광도입니다. 그러나 이 공식은 항상 미적으로 만족스러운 결과를 제공하지는 않습니다. 따라서 물리적 정확성에 집착하기보다는, 아티스트/프로그래머가 제어할 수 있는 매개변수를 제공하는 보다 일반적인 함수를 사용합니다(즉, 아티스트/프로그래머는 결과에 만족할 때까지 다양한 매개변수 값을 실험합니다). 광도를 조정하는 데 일반적으로 사용되는 공식은 다음과 같습니다:

$$I(d) = \frac{I_0}{a_0 + a_1 d + a_2 d^2}$$

a_0, a_1, a_2 를 **감쇠 매개변수**라고 부르며, 이는 아티스트나 프로그래머가 제공해야 합니다. 예를 들어, 빛의 강도가 거리의 역수에 비례하여 약해지도록 하려면 $a_0 = 0, a_1 = 1, a_2 = 0$ 으로 설정합니다. 실제 역제곱 법칙을 적용하려면

을 원한다면 $a_0=0, a_1=0, a_2=1$ 로 설정하십시오.
감쇠를 조명 방정식에 통합하면 다음과 같습니다:

$$LitColor = \mathbf{A} + \frac{k_d \mathbf{D} + k_s \mathbf{S}}{a_0 + a_1 d + a_2 d^2}$$

감쇠는 주변광 향에 영향을 미치지 않는데, 이는 주변광 향이 여러 번 반사된 간접광을 모델링하기 위해 사용되기 때문입니다.

7.10.2 범위

점광원의 경우 추가적인 *범위* 매개변수를 포함합니다. 광원으로부터의 거리가 이 범위를 초과하는 점은 해당 광원으로부터 빛을 전혀 받지 않습니다. 이 매개변수는 특정 영역에 빛을 국한시키는 데 유용합니다. 감쇠 매개변수가 거리와 함께 빛의 강도를 약화시키더라도, 광원의 최대 범위를 명시적으로 정의할 수 있다는 점은 여전히 유용합니다. 범위 매개변수는 셰이더 최적화에도 유용합니다. 곧 보게 되겠지만, 셰이더 코드에서 점이 범위 밖에 있으면 동적 분기를 통해 조기 반환하고 조명 계산을 건너뛸 수 있습니다. 범위 매개변수는 매우 먼 거리의 광원을 모델링하는 평행광에는 영향을 미치지 않습니다.

7.11 스포트라이트

스포트라이트의 좋은 물리적 예는 손전등입니다. 기본적으로 스포트라이트는 위치 Q를 가지며 방향 d를 향하고 원뿔을 통해 빛을 방출합니다(그림 7.20 참조).

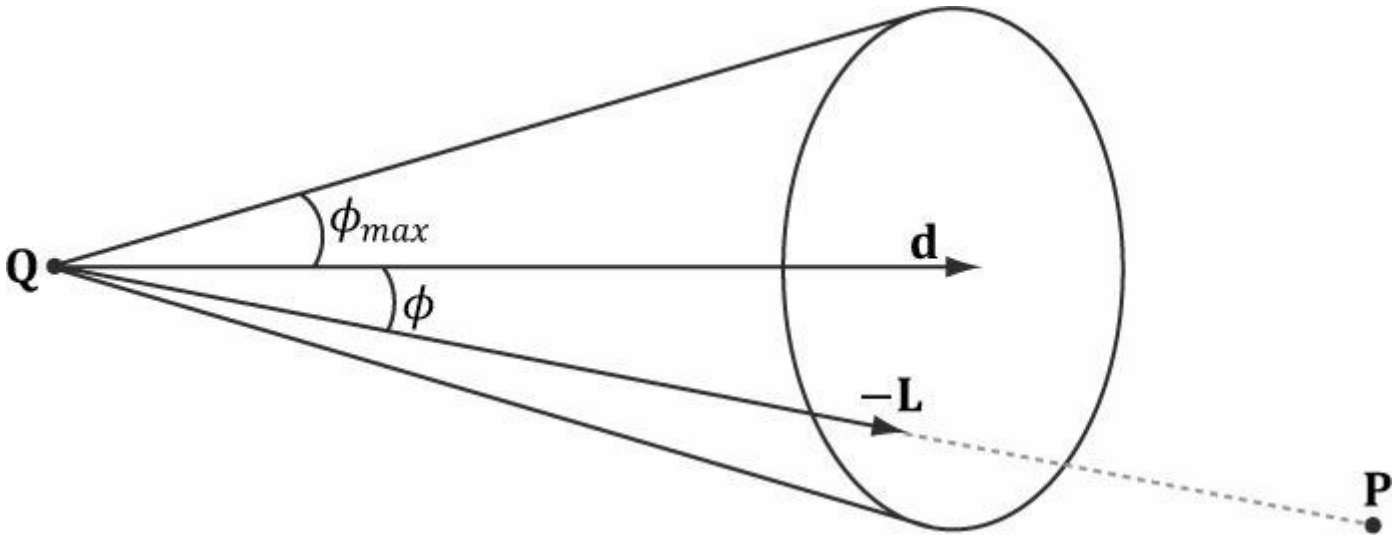


그림 7.20. 스포트라이트는 위치 Q에 있으며, 방향 d를 향하고, 각도 ϕ_{max} 의 원뿔을 통해 빛을 방출한다.

스포트라이트를 구현하기 위해, 우리는 점광원과 동일한 방식으로 시작합니다: 빛 벡터는 다음과 같이 주어집니다:

$$\mathbf{L} = \frac{\mathbf{Q} - \mathbf{P}}{\|\mathbf{Q} - \mathbf{P}\|}$$

여기서 P는 조명을 받는 점의 위치이고 Q는 스포트라이트의 위치이다. 그림 7.20에서 관찰하듯이, P가 스포트라이트의 원뿔 안쪽에 위치하여 빛을 받는 것은 오직 -L과 d 사이의 각도 ϕ 가 원뿔 각도 ϕ_{max} 보다 작을 때에만 성립합니다. 또한 스포트라이트 원뿔 내의 모든 빛이 동일한 강도를 가져서는 안 되며, 원뿔 중심부의 빛이 가장 강해야 합니다

강해야 하며, $\phi \neq 0$ 에서 ϕ_{max} 로 증가함에 따라 광도는 점차 0으로 감소해야 합니다.

그렇다면 ϕ 의 함수로서 강도 감쇠를 어떻게 제어할 수 있을까요? 또한 스포트라이트의 원뿔 크기는 어떻게 조절할 수 있을까요? 음, 우리는 반사율의 스펙큘러 원뿔에서 했던 것과 동일한 방법을 적용할 수 있습니다. 즉, 다음 함수를 사용합니다:

$$k_{spot}(\phi) = \max(\cos\phi, 0)^s = \max(-\mathbf{L} \cdot \mathbf{d}, 0)^s$$

이 함수의 그래프는 그림 7.12를 참조하십시오. 보시다시피, ϕ 가 증가함에 따라 강도가 부드럽게 감소하는데, 이는 우리가 원하는 특성 중 하나입니다. 또한 지수 s를 변경함으로써 ϕ_{max} (강도가 0으로 떨어지는 각도)를 간접적으로 제어할 수 있습니다. 즉, s를 변화시켜 스포트라이트 원뿔을 축소하거나 확장할 수 있습니다. 예를 들어, s = 8로 설정하면 원뿔의 반각은 약 45°가 됩니다.

따라서 스포트라이트 방정식은 점광원 방정식과 유사하지만, 스포트라이트 요인에 곱하여 점광원이 스포트라이트 원뿔에 대해 점의 위치에 따라 광도를 조정한다는 점이 다릅니다:

$$LitColor = k_{spot} \left(\mathbf{A} + \frac{k_d \mathbf{D} + k_s \mathbf{S}}{a_0 + a_1 d + a_2 d^2} \right) \quad (\text{eq. 7.5})$$

방정식 7.4와 7.5를 비교해 보면, 스포트라이트가 포인트 라이트보다 비용이 더 많이 든다는 것을 알 수 있습니다. k_{spot} 계수를 계산하여 곱해야 하기 때문입니다. 마찬가지로, 방정식 7.3과 7.4를 비교해 보면, 거리 d 를 계산해야 하고(거리는 제곱근 연산을 포함하기 때문에 실제로 상당히 비용이 많이 듭니다), 감쇠 표현식으로 나누어야 하기 때문에 포인트 라이트가 방향성 라이트보다 비용이 더 많이 든다는 것을 알 수 있습니다. 요약하자면, 방향성 라이트가 가장 비용이 적게 드는 광원이고, 그 다음이 포인트 라이트입니다. 스포트라이트는 가장 비용이 많이 드는 광원입니다.

7.12 구현

7.12.1 조명 구조

`LightHelper.h`에서 지원하는 세 가지 유형의 조명을 표현하기 위해 다음과 같은 구조체를 정의합니다.

```
struct Directional Light
{
    DirectionalLight() { ZeroMemory(this, sizeof(this)); }

    XMFLOAT4 Ambient; XMFLOAT4 Diffuse;
    XMFLOAT4 Specular; XMFLOAT3 Direction;
    float Pad; // 마지막 부동소수점 값을 패딩하여
               // 원하는 경우 라이트 배열로 사용할 수 있도록.
};

struct Point Light
{
    PointLight() { ZeroMemory(this, sizeof(this)); }

    XMFLOAT4 Ambient;
    XMFLOAT4 Diffuse;
    XMFLOAT4 Specular;

    // 4차원 벡터로 압축: (위치, 범위) XMFLOAT3 Position;
    float Range;

    // 4차원 벡터로 압축: (A0, A1, A2, 패딩) XMFLOAT3 Att;
    float Pad; // 마지막 부동 소수점 값에 패딩을 추가하여
               // 필요 시 조명 배열을 설정할 수 있도록.
};

struct SpotLight
{
    SpotLight() { ZeroMemory(this, sizeof(this)); }

    XMFLOAT4 Ambient;
    XMFLOAT4 Diffuse;
    XMFLOAT4 Specular;
```

```
// 4차원 벡터로 압축: (위치, 범위) XMFLOAT3 Position;

float Range;

// 4차원 벡터로 압축: (방향, 스폿) XMFLOAT3 Direction;

float Spot;

// 4차원 벡터로 압축: (Att, Pad) XMFLOAT3 Att;

float Pad; // 마지막 부동소수점값에 패딩을 넣어
// 원하는 경우 조명 배열을 설정할 수 있도록.

};
```

- 1. 앰비언트:** 광원이 방출하는 주변광의 양.
- 2. 확산:** 광원이 방출하는 확산광의 양.
- 3. 반사광:** 광원이 방출하는 반사광의 양.
- 4. 방향:** 빛의 방향.
- 5. 위치:** 빛의 위치.
- 6. 범위:** 빛의 범위. 광원으로부터의 거리가 범위를 초과하는 지점은 조명되지 않습니다.
- 7. 감쇠:** 빛의 강도가 거리에 따라 어떻게 감소하는지를 제어하는 세 가지 감쇠 상수(a_0 , a_1 , a_2)를 형식으로 저장합니다.
- 8. Spot:** 스포트라이트 계산에서 스포트라이트 원뿔을 제어하기 위해 사용되는 지수.

다음 섹션에서 "패드" 변수와 "패킹" 형식의 필요성에 대해 논의하겠습니다.

LightHelper.fx 파일은 이를 반영하는 구조체를 정의합니다.

```
struct DirectionalLight
{
    float4 Ambient; float4
    Diffuse; float4 Specular;
    float3 Direction; float
    pad;
};

struct PointLight
{
    float4 Ambient; float4
    Diffuse; float4 Specular;

    float3 Position; float
    Range;

    float3 Att; float
    pad;
};

struct SpotLight
{
    float4 Ambient; float4
    Diffuse; float4 Specular;

    float3 위치; float 범위;

    float3 Direction; float
    Spot;
};
```

```
float3 Att; float
pad;

};
```

7.12.2 구조체 패킹

이전 섹션에서는 HLSL 구조체를 정의했으며, 이를 다음과 같이 상수 버퍼에 인스턴스화합니다:

```
cbuffer cbPerFrame

{
    DirectionalLight gDirLight; PointLight
    gPointLight; SpotLight gSpotLight;
    float3 gEyePosW;
};
```

애플리케이션 레벨에서는 미러링 구조체를 인스턴스화합니다. 각 데이터 멤버를 개별적으로 설정하기보다는, 한 번의 호출로 효과 변수에 조명 인스턴스를 설정하고 싶습니다. 구조체 인스턴스는 다음 함수를 사용하여 효과 변수 인스턴스에 설정할 수 있습니다:

```
ID3DX11EffectVariable::SetRawValue(void *pData, UINT Offset, UINT Count);

// 호출 예시:
DirectionalLight mDirLight;
mfxDirLight->SetRawValue(&mDirLight, 0, sizeof(mDirLight));
```

그러나 이 함수는 단순히 원시 바이트를 복사하기 때문에 주의하지 않으면 찾기 어려운 오류가 발생할 수 있습니다. 특히 주의해야 할 점은 C++이 HLSL과 동일한 패킹 규칙을 따르지 않는다는 것입니다.

HLSL에서는 구조체 패딩이 발생하여 요소가 4차원 벡터로 패킹되며, 단일 요소가 두 개의 4차원 벡터로 분할할 수 없습니다. 다음 예시를 고려하십시오:

```
//HLSL
struct S
{
    float3 Pos; float3
    Dir;
};
```

데이터를 4차원 벡터로 압축해야 한다면 다음과 같이 할 수 있다고 생각할 수 있습니다:

벡터 1: (Pos.x, Pos.y, Pos.z, Dir.x) 벡터 2: (Dir.y, Dir.z, 빈 자리, 빈 자리)

그러나 이 방식은 dir 요소를 두 개의 4차원 벡터에 걸쳐 배치하게 되는데, 이는 HLSL 규칙에서 허용되지 않습니다—요소가 4차원 벡터 경계를 가로지르는 것은 허용되지 않습니다. 따라서 다음과 같이 패킹해야 합니다:

벡터 1: (Pos.x, Pos.y, Pos.z, 빈 자리) 벡터 2: (Dir.x, Dir.y, Dir.z, 빈 자리)

이제 우리의 미러링 C++ 구조체가 다음과 같이 정의되었다고 가정해 보겠습니다:

```
// C++
struct S
{
    XMFLOAT3 Pos;
    XMFLOAT3 Dir;
};
```

이러한 패킹 규칙을 무시하고 무턱대고 바이트를 호출하여 복사했다면, 결국 잘못된 첫 번째 상황이 발생했을 것입니다:

벡터 1: (Pos.x, Pos.y, Pos.z, Dir.x) 벡터 2: (Dir.y, Dir.z, 빈 자리, 빈 자리)

따라서 HLSL 패킹 규칙에 따라 요소들이 HLSL 구조체로 올바르게 복사되도록 C++ 구조체를 정의해야 합니다. 이를 위해 "패드" 변수를 사용합니다. HLSL이 어떻게 패킹되는지 몇 가지 예시를 더 살펴보겠습니다.

다음과 같은 구조체가 있다고 가정해 보겠습니다:

```
struct S
{
    float3 v;
    float s;
    float2 p;
    float3 q;
};
```

이 구조체는 패딩 처리되며 데이터는 다음과 같이 세 개의 4차원 벡터로 압축됩니다:

벡터 1: (v.x, v.y, v.z, s)
벡터 2: (p.x, p.y, 빈 자리, 빈 자리) 벡터 3: (q.x, q.y, q.z, 빈 자리)

여기서 스칼라 s를 첫 번째 벡터의 네 번째 성분에 넣을 수 있습니다. 그러나 q를 벡터 2의 남은 슬롯에 모두 넣을 수 없으므로 q는 자체 벡터를 가져야 합니다.

마지막 예시인 구조체:

```
struct S
{
    float2 u;
    float2 v;
    float a0;
    float a1;
    float a2;
};
```

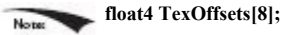
다음과 같이 패딩 및 패킹됩니다:

벡터 1: (u.x, u.y, v.x, v.y) 벡터 2: (a0, a1, a2, 빈 공간)

배열은 다르게 처리됩니다. SDK 문서에 따르면 "배열의 모든 요소는 4개의 구성 요소로 이루어진 벡터에 저장됩니다." 예를 들어, float2 배열이 있다면:

```
float2 TexOffsets[8];
```

위 예시에서 암시하듯 두 개의 float2 요소가 하나의 float4 슬롯에 압축될 것이라고 가정할 수 있습니다. 그러나 배열은 예외이며, 위 코드는 다음과 동일합니다:



```
float4 TexOffsets[8];
```

따라서 C++ 코드에서는 제대로 작동하도록 8개의 XMFLOAT2 배열이 아닌 8개의 XMFLOAT4 배열을 설정해야 합니다. 각 요소는 실제로 float2 배열만 필요했기 때문에 저장 공간에서 두 개의 부동 소수점 연산자를 낭비합니다. SDK 문서에서는 캐스팅과 추가 주소 계산 지침을 사용하여 메모리 효율성을 높일 수 있다고 지적합니다.

```
float4 array[4];

static float2 aggressivePack Array[8] = (float2[8])array;
```

7.12.3 방향성 라이트 구현

다음 HLSL 함수는 재료, 방향광원, 표면 법선, 그리고 조명받는 표면 점에서 시선까지의 단위 벡터를 입력으로 받아 점의 조명된 색상을 출력합니다:

```
// LightHelper.fx에서 정의됨.

// 방정식 7.3
void ComputeDirectionalLight(Material mat, DirectionalLight L, float3 normal, float3
    toEye,
    out float4 ambient, out
    float4 diffuse, out float4
    spec)
{

    // 출력 초기화.
    ambient = float4(0.0f, 0.0f, 0.0f, 0.0f); diffuse =
    float4(0.0f, 0.0f, 0.0f, 0.0f); spec = float4(0.0f, 0.0f,
    0.0f, 0.0f);

    // 광선 벡터는 광선이 이동하는 방향과 반대 방향을 향합니다. float3 lightVec = -L.Direction;

    // 주변광 항 추가.
    ambient = mat.Ambient * L.Ambient;

    // 표면이 빛의 시선 내에 있을 경우 확산 및 반사 항 추가
    // 빛의 시선 내에 있는 경우.

    float diffuseFactor = dot(lightVec, normal);

    // 동적 분기 방지 위해 평탄화. [flatten]
    if(diffuseFactor > 0.0f)
    {
        float3 v = reflect(-lightVec, normal);
        float specFactor = pow(max(dot(v, toEye), 0.0f), mat.Specular.w);

        diffuse = diffuseFactor * mat.Diffuse * L.Diffuse; spec = specFactor *
        mat.Specular * L.Specular;
    }
}
```

다음과 같은 내장 HLSL 함수가 사용되었습니다: **dot**, **reflect**, **pow**, **max**. 이들은 각각 벡터 내적 함수, 벡터 반사 함수, 지수 함수, 최대 함수입니다. 대부분의 HLSL 내장 함수에 대한 설명은 부록 B에서 확인할 수 있으며, 다른 HLSL 구문에 대한 간단한 입문서도 함께 수록되어 있습니다. 다만 한 가지 주의할 점은 두 벡터가 연산자 *****로 곱해질 때, 곱셈은 성분별로 수행된다는 것입니다.

 PC 환경에서는 HLSL 함수가 항상 인라인 처리되므로, 함수 호출이나 매개변수 전달에 따른 성능 오버헤드가 발생하지 않습니다.

7.12.4 포인트 라이트 구현

다음 HLSL 함수는 재료, 점광원, 표면 위치, 표면 법선, 그리고 조명받는 표면 점에서 시선까지의 단위 벡터를 입력으로 받아 점의 조명된 색상을 출력합니다:

```
// LightHelper.fx에서 정의됨.

// 방정식 7.4
void ComputePointLight(Material mat, PointLight L, float3 pos, float3 normal, float3
    toEye,
    out float4 ambient, out float4 diffuse, out float4 spec)
{

    // 출력 초기화.
    ambient = float4(0.0f, 0.0f, 0.0f, 0.0f); diffuse =
    float4(0.0f, 0.0f, 0.0f, 0.0f); spec = float4(0.0f, 0.0f,
    0.0f, 0.0f);
```

```

// 표면에서 광원까지의 벡터. float3 lightVec = L.Position - pos;

// 표면에서 광원까지의 거리. float d =
length(lightVec);

// 범위 테스트. if(d >
L.Range)
    return;

// 광원 벡터 정규화. lightVec /= d;

// 환경광 항.
ambient = mat.Ambient * L.Ambient;

// 표면이 빛의 시선 내에 있을 경우 확산 및 반사 항 추가

// 빛의 시선 내에 있는 경우.
float diffuseFactor = dot(lightVec, normal);

// 동적 분기 방지 위해 평탄화. [flatten]
if(diffuseFactor > 0.0f)
{
    float3 v = reflect(-lightVec, normal);
    float specFactor = pow(max(dot(v, toEye), 0.0f), mat.Specular.w);

    diffuse = diffuseFactor * mat.Diffuse * L.Diffuse; spec = specFactor *
mat.Specular * L.Specular;
}

// 감쇠
float att = 1.0f / dot(L.Att, float3(1.0f, d, d*d));

diffuse *= att; spec
    *= att;
}

```

7.12.5 스포트라이트 구현

다음 HLSL 함수는 재료, 스포트라이트 광원, 표면 위치, 표면 법선, 그리고 조명받는 표면 점에서 시선까지의 단위 벡터를 입력으로 받아 점의 조명된 색상을 출력합니다:

```

// LightHelper.fx에서 정의됨.

// 방정식 7.5
void ComputeSpotLight(Material mat, SpotLight L, float3 pos, float3
normal, float3 toEye,
out float4 ambient, out float4 diffuse, out float4 spec)
{

// 출력 초기화.
ambient = float4(0.0f, 0.0f, 0.0f, 0.0f); diffuse =
float4(0.0f, 0.0f, 0.0f, 0.0f); spec = float4(0.0f, 0.0f,
0.0f, 0.0f);

// 표면에서 광원까지의 벡터. float3 lightVec = L.Position - pos;

// 표면에서 광원까지의 거리. float d =
length(lightVec);

// 범위 테스트.

```

```

if( d > L.Range ) return;

// 광원 벡터 정규화. lightVec /= d;

// 환경광 항.
ambient = mat.Ambient * L.Ambient;

// 표면이 투과 상태인 경우 확산 및 반사 항 추가
// 빛의 시선 방향.

float diffuseFactor = dot(lightVec, normal);

// 동적 분기 방지 위해 평탄화. [flatten]
if(diffuseFactor > 0.0f)
{
    float3 v = reflect(-lightVec, normal);
    float specFactor = pow(max(dot(v, toEye), 0.0f), mat.Specular.w);

    diffuse = diffuseFactor * mat.Diffuse * L.Diffuse; spec = specFactor *
    mat.Specular * L.Specular;
}

// 스포트라이트 계수로 스케일링하고 감쇠.
float spot = pow(max(dot(-lightVec, L.Direction), 0.0f), L.Spot);

// 스포트라이트 계수로 스케일링하고 감쇠. float att = spot /
dot(L.Att, float3(1.0f, d, d*d));
ambient *= spot;
diffuse *= att; spec
    *= att;
}

```

7.13 조명 데모

첫 번째 조명 데모에서는 방향광, 점광, 스포트라이트 세 가지 조명을 동시에 활성화합니다. 방향광은 고정된 상태를 유지하고, 점광은 지형을 중심으로 회전하며, 스포트라이트는 카메라와 함께 이동하면서 카메라가 향하는 방향을 비춥니다. 이 조명 데모는 이전 장의 "파도" 데모를 기반으로 합니다. 효과 파일은 다음 섹션에 제시되며, §7.10에서 정의된 구조체와 함수를 활용합니다.

7.13.1 효과 파일

```

#include "LightHelper.fx" cbuffer

cbPerFrame
{
    DirectionalLight gDirLight; PointLight
    gPointLight; SpotLight gSpotLight;
    float3 gEyePosW;

};

cbuffer cbPerObject
{
    float4x4 gWorld;
    float4x4 gWorldInvTranspose; float4x4
    gWorldViewProj;

```

```

    Material gMaterial;
};

struct VertexIn
{
    float3 PosL : POSITION; float3 NormalL
    : NORMAL;
};

struct VertexOut
{
    float4 PosH : SV_POSITION; float3
    PosW : POSITION; float3 NormalW :
    NORMAL;
};

VertexOut VS(VertexIn vin)
{
    VertexOut vout;

    // 월드 공간으로 변환.
    vout.PosW = mul(float4(vin.PosL, 1.0f), gWorld).xyz; vout.NormalW = mul(vin.NormalL,
(float3x3)gWorldInvTranspose);

    // 동차 클립 공간으로 변환.
    vout.PosH = mul(float4(vin.PosL, 1.0f), gWorldViewProj);

    return vout;
}

float4 PS(VertexOut pin) : SV_Target
{
    // 보간된 법선은 비정규화될 수 있으므로 정규화합니다. pin.NormalW =
    normalize(pin.NormalW);

    float3 toEyeW = normalize(gEyePosW - pin.PosW);

    // 합계를 0으로 시작합니다.
    float4 ambient = float4(0.0f, 0.0f, 0.0f, 0.0f); float4 diffuse =
    float4(0.0f, 0.0f, 0.0f, 0.0f); float4 spec = float4(0.0f, 0.0f, 0.0f,
    0.0f);

    // 각 광원의 빛 기여도를 합산합니다. float4 A, D, S;

    ComputeDirectionalLight(gMaterial, gDirLight, pin.NormalW, toEyeW,
    A, D, S);
    ambient += A;
    diffuse += D; spec
    += S;

    ComputePointLight(gMaterial, gPointLight, pin.PosW, pin.NormalW,
    toEyeW, A, D, S);
    ambient += A;
    diffuse += D; spec
    += S;

    ComputeSpotLight(gMaterial, gSpotLight, pin.PosW, pin.NormalW,
    toEyeW, A, D, S);
    ambient += A;
    diffuse += D; spec
    += S;

```

```

float4 litColor = ambient + diffuse + spec;

// 일반적인 경우, 확산 재질에서 알파를 가져옴. litColor.a =
gMaterial.Diffuse.a;

return litColor;
}

technique11 LightTech
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS()));
        SetGeometryShader(NULL); SetPixelShader(CompileShader(ps_5_0,
        PS()));
    }
}

```

7.13.2 C++ 애플리케이션 코드

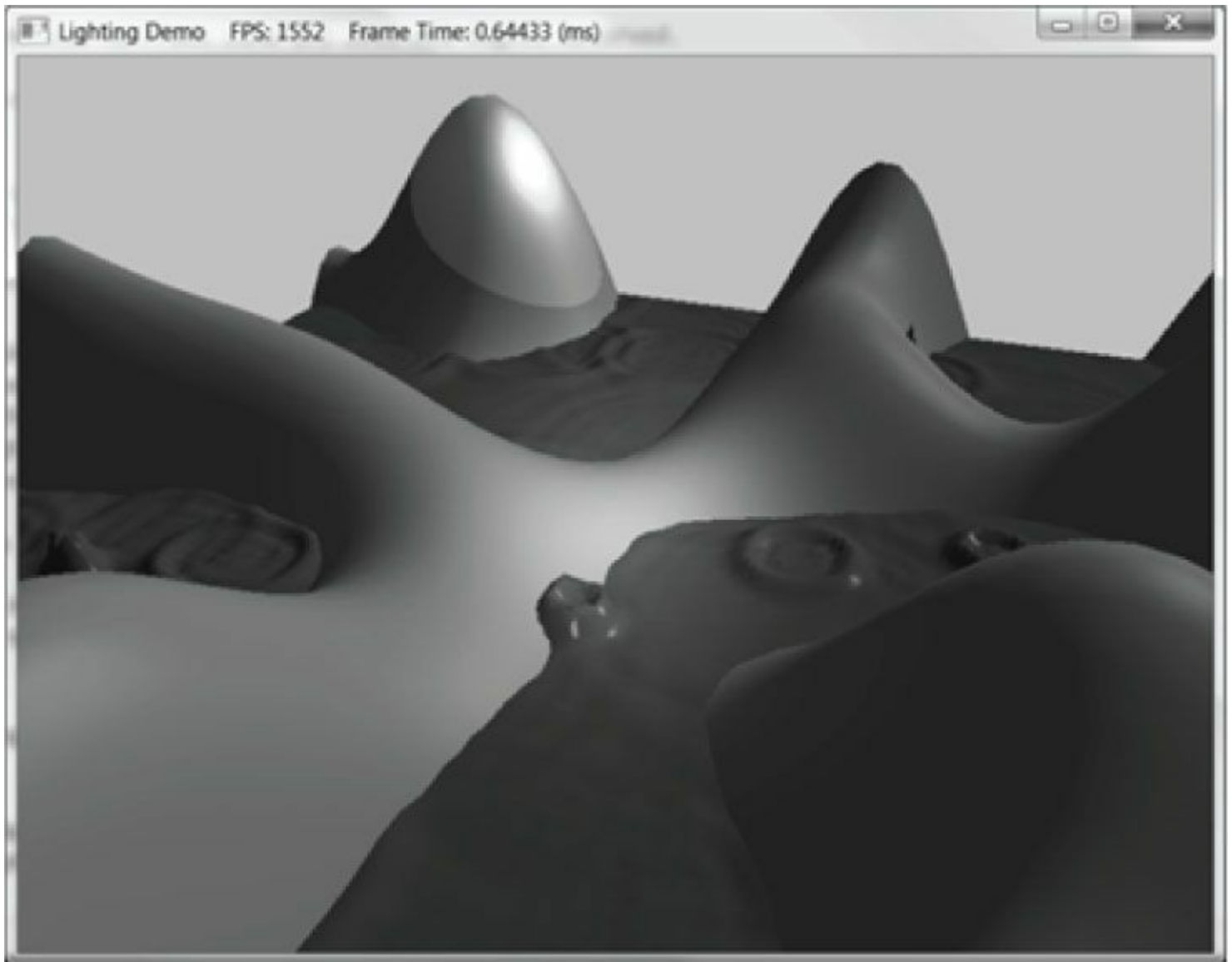


그림 7.21. "Lighting" 데모 스크린샷.

조명 계산에는 표면 법선이 필요합니다. 우리는 정점 수준에서 법선을 정의합니다. 이 법선들은 삼각형의 픽셀 전체에 걸쳐 보관되어 픽셀 단위로 조명 계산을 수행할 수 있게 합니다. 또한 더 이상 정점 색상을 지정하지 않습니다. 대신 표면 색상은 각 픽셀에 조명 방정식을 적용하여 생성됩니다. 우리의 입력 레이아웃 설명은 다음과 같습니다:

```
D3D11_INPUT_ELEMENT_DESC vertexDesc[ =
```

```
{
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
        D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"COLOR", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 0, 12,
        D3D11_INPUT_PER_VERTEX_DATA, 0}
};
```

애플리케이션 클래스에서 세 개의 라이트와 두 개의 머티리얼을 정의합니다.

```
DirectionalLight mDirLight; PointLight
mPointLight; SpotLight
mSpotLight; Material mLandMat;
Material mWavesMat;
```

이들은 생성자에서 초기화됩니다:

```
LightingApp::LightingApp(HINSTANCE hInstance)
{
    /* ...관련 없는 코드 생략... */

    // 방향광.
    mDirLight.Ambient = XMFLOAT4(0.2f, 0.2f, 0.2f, 1.0f); mDirLight.Diffuse = XMFLOAT4(0.5f,
    0.5f, 0.5f, 1.0f); mDirLight.Specular = XMFLOAT4(0.5f, 0.5f, 0.5f, 1.0f); mDirLight.Direction =
    XMFLOAT3(0.57735f, -0.57735f, 0.57735f);

    // 포인트 라이트—애니메이션을 위해 매 프레임마다 위치가 변경됨

    // UpdateScene 함수 내에서.
    mPointLight.Ambient = XMFLOAT4(0.3f, 0.3f, 0.3f, 1.0f); mPointLight.Diffuse =
    XMFLOAT4(0.7f, 0.7f, 0.7f, 1.0f); mPointLight.Specular = XMFLOAT4(0.7f, 0.7f,
    0.7f, 1.0f); mPointLight.Att      = XMFLOAT3(0.0f, 0.1f, 0.0f); mPointLight.Range
    = 25.0f;

    // 스포트라이트—매 프레임마다 위치와 방향이 변경되어

    // UpdateScene 함수에서 애니메이션 처리.
    mSpotLight.Ambient = XMFLOAT4(0.0f, 0.0f, 0.0f, 1.0f); mSpotLight.Diffuse =
    XMFLOAT4(1.0f, 1.0f, 0.0f, 1.0f); mSpotLight.Specular = XMFLOAT4(1.0f, 1.0f,
    1.0f, 1.0f); mSpotLight.Att      = XMFLOAT3(1.0f, 0.0f, 0.0f); mSpotLight.Spot
    = 96.0f;

    mSpotLight.Range      = 10000.0f;

    mLandMat.Ambient = XMFLOAT4(0.48f, 0.77f, 0.46f, 1.0f); mLandMat.Diffuse =
    XMFLOAT4(0.48f, 0.77f, 0.46f, 1.0f); mLandMat.Specular = XMFLOAT4(0.2f, 0.2f, 0.2f, 16.0f);

    mWavesMat.Ambient = XMFLOAT4(0.137f, 0.42f, 0.556f, 1.0f); mWavesMat.Diffuse =
    XMFLOAT4(0.137f, 0.42f, 0.556f, 1.0f); mWavesMat.Specular = XMFLOAT4(0.8f, 0.8f, 0.8f,
    96.0f);
}
```

여러 개의 조명을 사용할 때는 장면이 과도하게 밝아지지 않도록 주의해야 합니다. 따라서 적절한 균형을 찾기 위해 주변광, 확산광, 반사광 값을 실험해 보아야 합니다. 감쇠와 범위 설정도 실험이 필요합니다. 스포트라이트가 노란색 확산광을 발산한다는 점을 유의하세요. 색상 조명을 사용하는 것을 종종 잊기 쉽지만, 다양한 효과를 연출하는 데 활용할 수 있습니다. 예를 들어, 방향광으로 햇빛을 표현할 때 해가 밝은 주황색으로 지고 있다면, 빛 색상을 주황색으로 조정하여 장면의 오브젝트에 주황빛을 입힐 수 있습니다. 미묘한 푸른빛은 외계 우주선을 탐험할 때 효과적이며, 붉은 빛은 비상 상황을 전달할 수 있습니다.

앞서 언급한 대로 포인트 라이트와 스포트라이트는 애니메이션 처리됩니다. 이는 UpdateScene 메서드에서 수행됩니다:

```
void LightingApp::UpdateScene(float dt)
{
```

```
/* ...관련 없는 코드 생략... */
```

```
// 구좌표를 직교좌표로 변환
```

```
float x = mRadius*sinf(mPhi)*cosf(mTheta); float z =  
mRadius*sinf(mPhi)*sinf(mTheta); float y =  
mRadius*cosf(mPhi);
```

```
mEyePosW = XMFLOAT3(x, y, z);
```

```
// 뷰 행렬 생성.
```

```
XMVECTOR pos = XMVectorSet(x, y, z, 1.0f); XMVECTOR  
target = XMVectorZero();
```

```
XMVECTOR up = XMVectorSet(0.0f, 1.0f, 0.0f, 0.0f);
```

```
/* ...관련 없는 코드 생략... */
```

```
// 지표면 위의 원형 조명.
```

```
mPointLight.Position.x = 70.0f*cosf(0.2f*mTimer.TotalTime()); mPointLight.Position.z =  
70.0f*sinf(0.2f*mTimer.TotalTime()); mPointLight.Position.y = MathHelper::Max(  
GetHillHeight(mPointLight.Position.x, mPointLight.Position.z), -3.0f) + 10.0f;
```

```
// 스포트라이트는 카메라 위치를 취하고 카메라가 바라보는 방향으로 조준됩니다.
```

```
// 카메라가 바라보는 방향과 동일합니다. 이렇게 하면
```

```
// 마치 손전등을 들고 있는 것처럼 보입니다. mSpotLight.Position  
= mEyePosW; XMStoreFloat3(&mSpotLight.Direction,  
XMVector3Normalize(target - pos));
```

```
}
```

포인트 라이트는 기본적으로 xz 평면에서 원형 궤적을 따라 이동하지만, 항상 육지나 물 위를 지나갑니다. 스포트라이트는 카메라 시점 위치에 배치되며 카메라가 바라보는 방향으로 조준됩니다. 이로 인해 시청자가 손전등처럼 빛을 들고 있는 것처럼 보이게 합니다.

마지막으로, 렌더링 전에 조명과 재질을 효과에 설정합니다:

```
void LightingApp::DrawScene()  
{
```

```
/* ...관련 없는 코드 생략... */
```

```
// 프레임별 상수 설정.
```

```
mfxDirLight->SetRawValue(&mDirLight, 0, sizeof(mDirLight));  
mfxPointLight->SetRawValue(&mPointLight, 0, sizeof(mPointLight));  
mfxSpotLight->SetRawValue(&mSpotLight, 0, sizeof(mSpotLight));  
mfxEyePosW->SetRawValue(&mEyePosW, 0, sizeof(mEyePosW));
```

```
/* ...관련 없는 코드 생략... */
```

```
// 지형 재질 설정 (재질은 오브젝트마다 다름).
```

```
mfxMaterial->SetRawValue(&mLandMat, 0, sizeof(mLandMat));
```

```
/* ...지형 렌더링... */
```

```
// 파도 재질 설정 (객체마다 재질이 다름).
```

```
mfxMaterial->SetRawValue(&mWavesMat, 0, sizeof(mWaves Mat));
```

```
/* ...파도 렌더링... */
```

```
}
```

7.13.3 법선 계산

우리의 지형 표면이 함수 $y = f(x, z)$ 로 주어지기 때문에,

표면상의 각 점에 대해, 편미분을 취하여 +x-방향과 +z-방향의 두 접선 벡터를 형성합니다:

$$\mathbf{T_x} = \left(1, \frac{\partial f}{\partial x}, 0\right)$$
$$\mathbf{T_z} = \left(0, \frac{\partial f}{\partial z}, 1\right)$$

이 두 벡터는 표면 점의 접평면에 속합니다. 이들의 외적은 법선 벡터를 줍니다:

$$\mathbf{n} = \mathbf{T_z} \times \mathbf{T_x} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ 0 & \frac{\partial f}{\partial z} & 1 \\ 1 & \frac{\partial f}{\partial x} & 0 \end{vmatrix}$$
$$= \left(\begin{vmatrix} \frac{\partial f}{\partial z} & 1 \\ \frac{\partial f}{\partial x} & 0 \end{vmatrix}, -\begin{vmatrix} 0 & 1 \\ 1 & 0 \end{vmatrix}, \begin{vmatrix} 0 & \frac{\partial f}{\partial z} \\ 1 & \frac{\partial f}{\partial x} \end{vmatrix} \right)$$
$$= \left(-\frac{\partial f}{\partial x}, 1, -\frac{\partial f}{\partial z} \right)$$

지형 메쉬 생성에 사용한 함수는 다음과 같습니다:

$$f(x, z) = 0.3 \cdot z \cdot \sin(0.1x) + 0.3x \cdot \cos(0.1z)$$

편미분은 다음과 같습니다:

$$\frac{\partial f}{\partial x} = 0.03z \cdot \cos(0.1x) + 0.3 \cos(0.1z)$$
$$\frac{\partial f}{\partial z} = 0.3 \sin(0.1x) - 0.03x \cdot \sin(0.1z)$$

따라서 표면 점 (x, f(x, z), z)에서의 표면 법선은 다음과 같이 주어집니다:

$$\mathbf{n}(x, z) = \left(-\frac{\partial f}{\partial x}, 1, -\frac{\partial f}{\partial z} \right) = \begin{bmatrix} -0.03z \cdot \cos(0.1x) - 0.3 \cos(0.1z) \\ 1 \\ -0.3 \sin(0.1x) + 0.03x \cdot \sin(0.1z) \end{bmatrix}^T$$

이 표면 법선은 단위 길이가 아니므로, 조명 계산 전에 정규화해야 합니다.

특히, 각 정점 위치에서 이전의 법선 계산을 수행하여 정점 법선을 구합니다:

```
XMFLOAT3 LightingApp::GetHillNormal(float x, float z)const
{
```

```
// n = (-df/dx, 1, -df/dz) XMFLOAT3 n(
-0.03f*z*cosf(0.1f*x) - 0.3f*cosf(0.1f*z), 1.0f,
-0.3f*sinf(0.1f*x) + 0.03f*x*sinf(0.1f*z));
```

```
XMVECTOR unitNormal = XMVector3Normalize(XMLoadFloat3(&n)); XMStoreFloat3(&n, unitNormal);
```

```
return n;
```

수면 법선 벡터는 유사한 방식으로 처리되나, 물에 대한 공식이 존재하지 않습니다. 그러나 각 정점에서의 접선 벡터는 유한 차분법(Lengyel02 또는 수치 해석 서적 참조)을 통해 근사화할 수 있습니다.

미적분학이 녹슬었다 해도 걱정하지 마십시오. 이 책에서는 미적분학이 큰 역할을 하지 않을 것입니다. 현재 유용한 이유는 우리가  에서 수학적 표면을 이용해 기하학적 구조를 생성하고 있기 때문입니다. 이를 통해 흥미로운 물체를 그릴 수 있게 됩니다.

결국에는 3D 모델링 프로그램에서 내보낸 파일로부터 3D 메시를 불러올 것입니다.

7.14 LIT SKULL DEMO

"Lit Skull" 데모([그림 7.22](#))는 Basic.fx라는 효과를 정의하며, 이는 일반적으로 XNA 4.0의 BasicEffect라는 효과 클래스에서 영감을 받았습니다. 앞으로 몇 장에 걸쳐 더 많은 것을 배우면서 이 효과에 기능을 추가할 것입니다. 이 효과가 완성되면 기본 렌더링 효과로 사용되며, 조명, 텍스처링, 알파 테스트, 그림자, 안개 등 일반적이고 기본적인 작업을 지원할 것입니다. 더 특수한 효과들은 다른 효과들을 통해 구현될 것입니다.

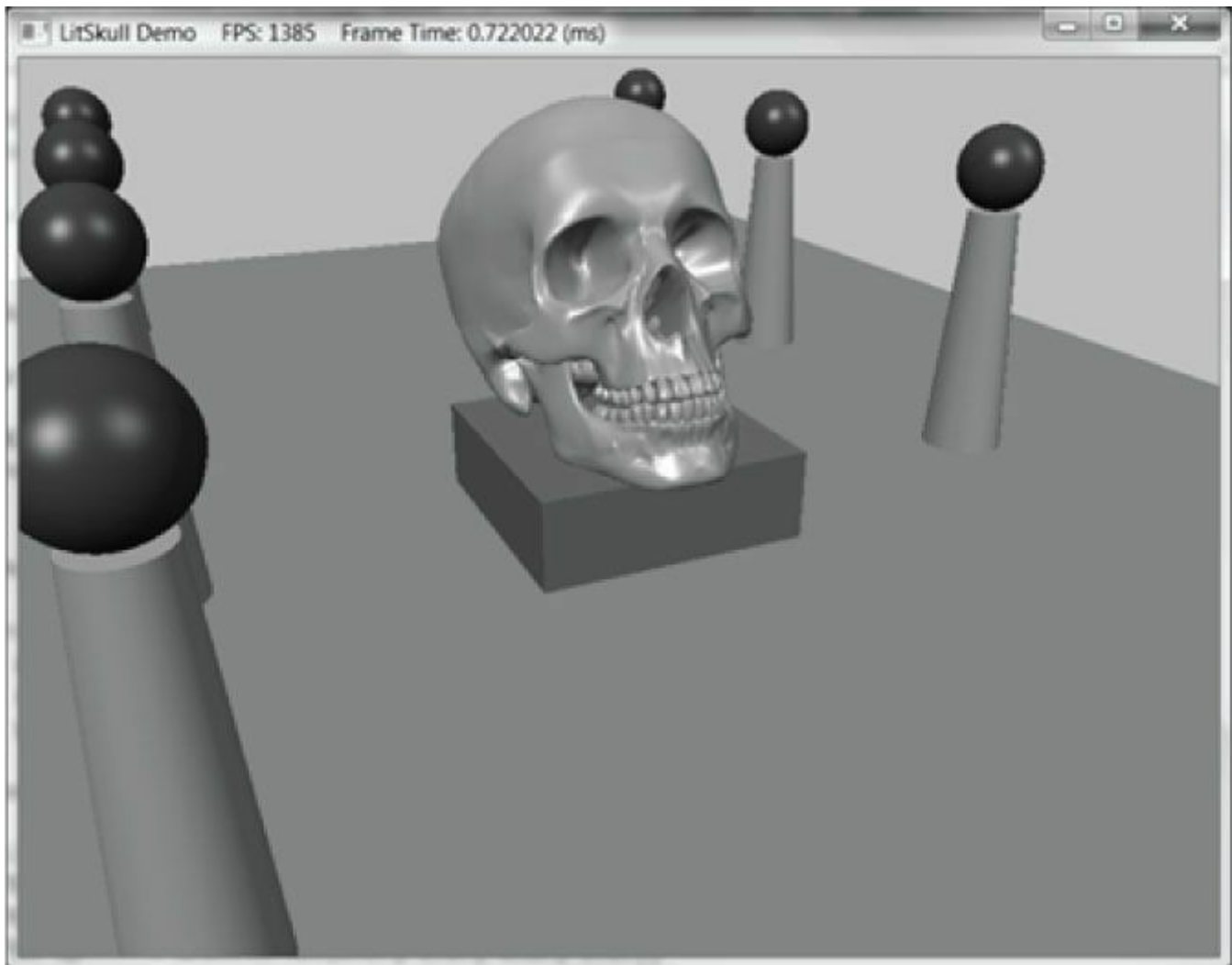


그림 7.22. "Lit Skull" 데모의 스크린샷.

기본 효과에 적용한 한 가지 제한 사항은 방향광을 최대 3개까지만 지원한다는 점입니다. 이는 꼭 필요한 제약은 아니지만, 데모 애플리케이션의 경우 일부 관리 작업을 단순화하는 효과가 있습니다. 특히 효과 프레임워크가 생성해야 하는 셰이더 변형의 수를 줄여줍니다. 또한 많은 수의 광원을 지원하면 비용이 매우 높아지기 시작합니다. 이 책의 제3부에서는 다수의 광원을 효율적으로 처리하고, 포인트 라이트와 스팟 라이트를 더 균일하게 처리할 수 있는 대체 렌더링 방식인 *디퍼드 렌더링(deferred rendering)*을 소개합니다.

기본 효과는 3점 조명 시스템 구현을 위해 세 개의 방향광을 지원합니다. 3점 조명은 조명 시스템은 영화와 사진 촬영에서 단일 광원보다 더 나은 조명을 얻기 위해 흔히 사용되며, *키 라이트*라고 불리는 주광원, 일반적으로 키 라이트에서 측면 방향으로 비추는 보조광인 *필 라이트*, 그리고 *백라이트*로 구성됩니다.

"Lit Skull" 데모에서 사용자는 '0', '1', '2', '3' 키를 눌러 각각 0, 1, 2, 3개의 조명을 활성화할 수 있습니다. 이전 데모와 비교하여 "Lit Skull" 데모의 또 다른 변경점은 ID3DX11Effect 포인터를 해당 효과 변수들을 하나의 클래스로 통합합니다. 이는 효과 변수들이 그 효과가 발생한 효과와 밀접하게 연관되어 있기 때문에 타당합니다. 프로그램 규모가 커짐에 따라 렌더링 코드를 여러 클래스로 분할하게 됩니다. 각 클래스는 구현된 효과에 접근할 수 있어야 합니다. 초기 접근 방식으로는 각 클래스가 효과를 별도로 로드하는 것이지만, 이는 효과 중복으로 비효율적입니다. 단일 렌더링 효과가 애플리케이션 전체에서 지오메트리 렌더링에 공유될 수 있으므로, 구현된 효과에 이전에 구현한 효과에 접근해야 할 수 있습니다. 초기 접근 방식으로는 각 클래스가 효과를 개별적으로 로드하는 방법이 있지만, 이는 중복된 효과를 생성하여 비효율적입니다. 단일 렌더링 효과는 애플리케이션 전반에 걸쳐 지오메트리 렌더링에 공유될 수 있으므로, 각 효과 래퍼 클래스의 단일 인스턴스를 생성하고 이를 Effects 클래스의 공용 정적 변수로 저장하여 전역적으로 노출합니다. 이를 통해 애플리케이션의 서로 다른 부분이 동일한 효과 파일에서 효과를 중복 생성하는 것을 방지합니다. 이 구조는 데모 프로젝트나 중간 규모 프로젝트에는 효과 수가 적기 때문에 간단하면서도 충분합니다. 대규모 프로젝트에는 보다 정교한 관리가 필요할 수 있습니다. 효과 래퍼 클래스와 Effects 클래스는 *Effects.h/.cpp*에 구현됩니다.

이 책 전체에 걸쳐 이러한 파일들에 추가할 예정이므로, 우리는 이들을 *공통 디렉터리*에 넣지 않습니다. 이펙트와 마찬가지로, 우리는 소수의 서로 다른 버텍스 포맷만을 가질 것이며, 중복된 입력 레이아웃이 생성되는 것을 피하고 싶습니다 생성되는 것을 피하고자 합니다(즉, 동일한 입력 레이아웃 설명으로부터 생성되는 입력 레이아웃). *Vertex.h/.cpp*에서 우리는 정점 형식 구조체와 입력 레이아웃 설명을 정의하고, 각 입력 레이아웃의 단일 인스턴스를 생성하여 InputLayouts 클래스 **내** 공용 정적 변수로 저장함으로써 전역적으로 노출합니다. 책 전체에 걸쳐 *Vertex.h/.cpp*에 추가할 예정이므로, 이를 *Common* 디렉터리에 포함시키지 않습니다.

렌더 상태 객체도 유사하게 처리할 수 있습니다. 프로그램은 소수의 서로 다른 렌더 상태 객체만 사용하며, 렌더 상태 객체는 코드베이스 전반에서 재사용될 수 있습니다. 따라서 향후 데모에서는 필요한 모든 렌더링 상태 객체를 정의하고 전역적으로 접근할 수 있도록 하는 *RenderState.h/.cpp* 파일을 생성할 것입니다. 독자는 앞으로 진행될 구조 변경에 익숙해질 때까지 "Lit Skull" 데모를 연구해야 합니다.

7.15 요약

1. 조명 처리에서는 더 이상 버텍스별 색상을 지정하지 않고, 대신 장면 조명과 버텍스별 재질을 정의합니다. 머티리얼은 빛이 객체의 표면과 상호작용하는 방식을 결정하는 속성으로 생각할 수 있습니다. 버텍스별 머티리얼은 삼각형 면 전체에 보관되어 삼각형 메시의 각 표면 점에서 머티리얼 값을 얻습니다. 이후 조명 방정식은 빛과 표면 머티리얼 간의 상호작용을 기반으로 눈이 보는 표면 색상을 계산합니다. 표면 법선과 눈 위치와 같은 다른 매개변수도 관련됩니다.
2. 표면 법선은 표면 상의 한 점에 대한 접평면에 수직인 단위 벡터입니다. 표면 법선은 표면 상의 한 점이 "향하는" 방향을 결정합니다. 조명 계산을 위해서는 삼각형 메시 표면의 각 지점에서 표면 법선을 필요로 합니다. 이를 통해 빛이 메시 표면의 해당 지점에 도달하는 각도를 결정할 수 있기 때문입니다. 표면 법선을 얻기 위해, 우리는 정점 위치에서만 표면 법선을 지정합니다(소위 정점 법선). 그런 다음 삼각형 메시 표면의 각 점에서 표면 법선 근사값을 얻기 위해 래스터화 과정에서 이 정점 법선들이 삼각형 전체에 걸쳐 보관됩니다. 임의의 삼각형 메시의 경우 정점 법선은 일반적으로 '법선 평균화' 기법을 통해 근사됩니다. 행렬 A가 점과 벡터(법선이 아닌 벡터)를 변환하는 데 사용된다면, 표면 법선을 변환하는 데는 $(A^{-1})^T$ 를 사용해야 합니다.
3. 평행(방향성) 광원은 매우 먼 거리에 있는 광원을 근사화한 것입니다. 따라서 모든 입사 광선을 서로 평행하다고 가정할 수 있습니다. 방향성 광원의 실제 예로는 지구에 대한 태양을 들 수 있습니다. 점광원은 모든 방향으로 *빛*을 방출합니다. 점광원의 실제 예로는 전구를 들 수 있습니다. 스포트라이트는 원뿔 모양으로 빛을 방출합니다. 스포트라이트의 실제 예로는 손전등을 들 수 있습니다.
4. 주변광은 장면 내에서 여러 번 산란되고 반사되어 모든 방향에서 균일하게 물체에 도달함으로써 표면을 고르게 밝게 하는 간접광을 모델링합니다. 확산광은 특정 방향으로 이동하며 표면에 닿으면 모든 방향으로 균일하게 반사됩니다. 확산광은 거칠거나 무광택 표면을 모델링하는 데 사용해야 합니다. 반사광은 특정 방향으로 이동하며, 표면에 닿으면 한 방향으로 날카롭게 반사되어 특정 각도에서만 보이는 밝은 광택을 생성합니다. 반사광은 매끄럽고 광택이 나는 표면을 모델링할 때 사용해야 합니다.

7.16 연습 문제

1. 이 장의 조명 데모를 수정하여 방향광은 빨간색만, 점광은 녹색만, 스포트라이트는 파란색만 발산하도록 하십시오. 색상별 조명을 활용하면 게임 분위기를 다르게 연출하는 데 유용합니다. 예를 들어, 빨간색 조명은 비상 상황을 나타내는 데 사용될 수 있습니다.
2. 이 장의 조명 데모를 수정하여 *표면의* "반짝임"을 제어하는 스페큘러 파워 재료 구성 요소를 변경하십시오. $p = 8, p = 32, p = 64, p = 128, p = 256, p = 512$ 를 시도해 보십시오.
3. 툰 라이팅의 특징 중 하나는 그림 7.23에서 볼 수 있듯이 한 색조에서 다음 색조로의 갑작스러운 전환(부드러운 전환과 대비됨)입니다. 이는 k_d 와 k_s 를 일반적인 방식으로 *계산한* 후, 픽셀 셰이더에서 사용하기 전에 다음과 같은 이산 함수로 변환하여 구현할 수 있습니다:

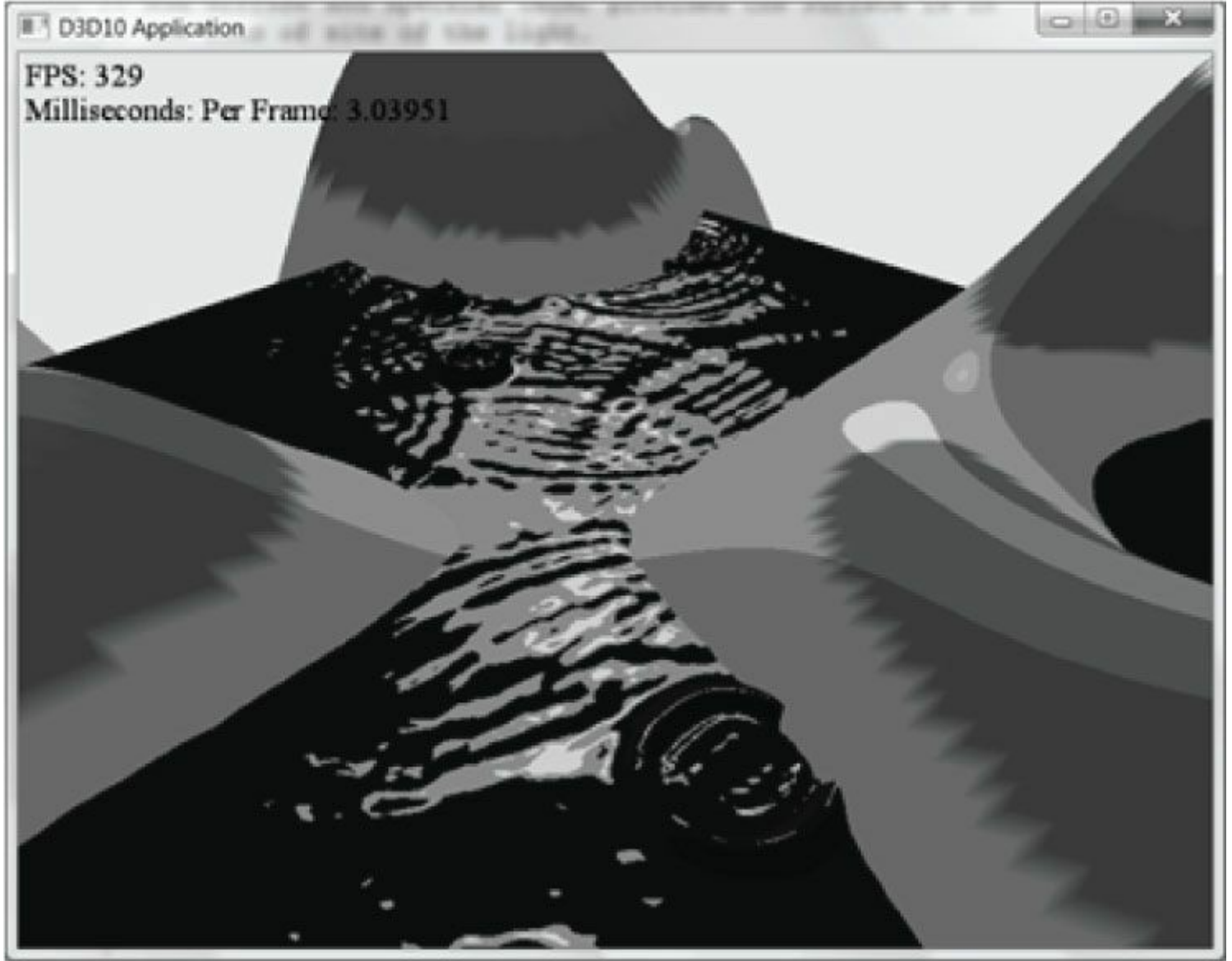


그림 7.23. 툰 셰이더 스크린샷.

$$k'_d = f(k_d) = \begin{cases} 0.4 & \text{if } -\infty < k_d \leq 0.0 \\ 0.6 & \text{if } 0.0 < k_d \leq 0.5 \\ 1.0 & \text{if } 0.5 < k_d \leq 1.0 \end{cases}$$

$$k'_s = g(k_s) = \begin{cases} 0.0 & \text{if } 0.0 \leq k_s \leq 0.1 \\ 0.5 & \text{if } 0.1 < k_s \leq 0.8 \\ 0.8 & \text{if } 0.8 < k_s \leq 1.0 \end{cases}$$

이 장의 조명 데모를 수정하여 이러한 종류의 투온 셰이딩을 사용하도록 하십시오. (참고: 이전의 함수 f 와 g 는 시작을 위한 샘플 함수에 불과하며, 원하는 결과를 얻을 때까지 조정할 수 있습니다.)

- 4. 이 장의 조명 데모를 수정하여 사용자의 키보드 입력에 따라 스포트라이트의 원뿔 각도를 증가 또는 감소시킬 수 있도록 하십시오.

제8장 텍스처링

데모가 점점 흥미로워지고 있지만, 실제 물체는 일반적으로 버텍스별 색상으로는 표현하기 어려운 세부 사항을 가지고 있습니다. **텍스처 매핑**은 이미지 데이터를 삼각형에 매핑하는 기술로, 이를 통해 장면의 세부 사항과 사실감을 크게 향상시킬 수 있습니다. 예를 들어, 정육면체를 만들고 각 면에 상자 텍스처를 매핑하여 상자로 변환할 수 있습니다([그림 8.1](#)).

목표:

1. 삼각형에 매핑될 텍스처의 특정 부분을 지정하는 방법을 학습합니다.
2. 텍스처를 생성하고 활성화하는 방법을 알아봅니다.
3. 텍스처를 필터링하여 더 부드러운 이미지를 만드는 방법을 알아봅니다.
4. 주소 모드를 사용하여 텍스처를 여러 번 타일링하는 방법을 알아봅니다.
5. 여러 텍스처를 결합하여 새로운 텍스처와 특수 효과를 생성하는 방법을 알아봅니다.
6. 텍스처 애니메이션을 통해 기본적인 효과를 생성하는 방법을 학습합니다.

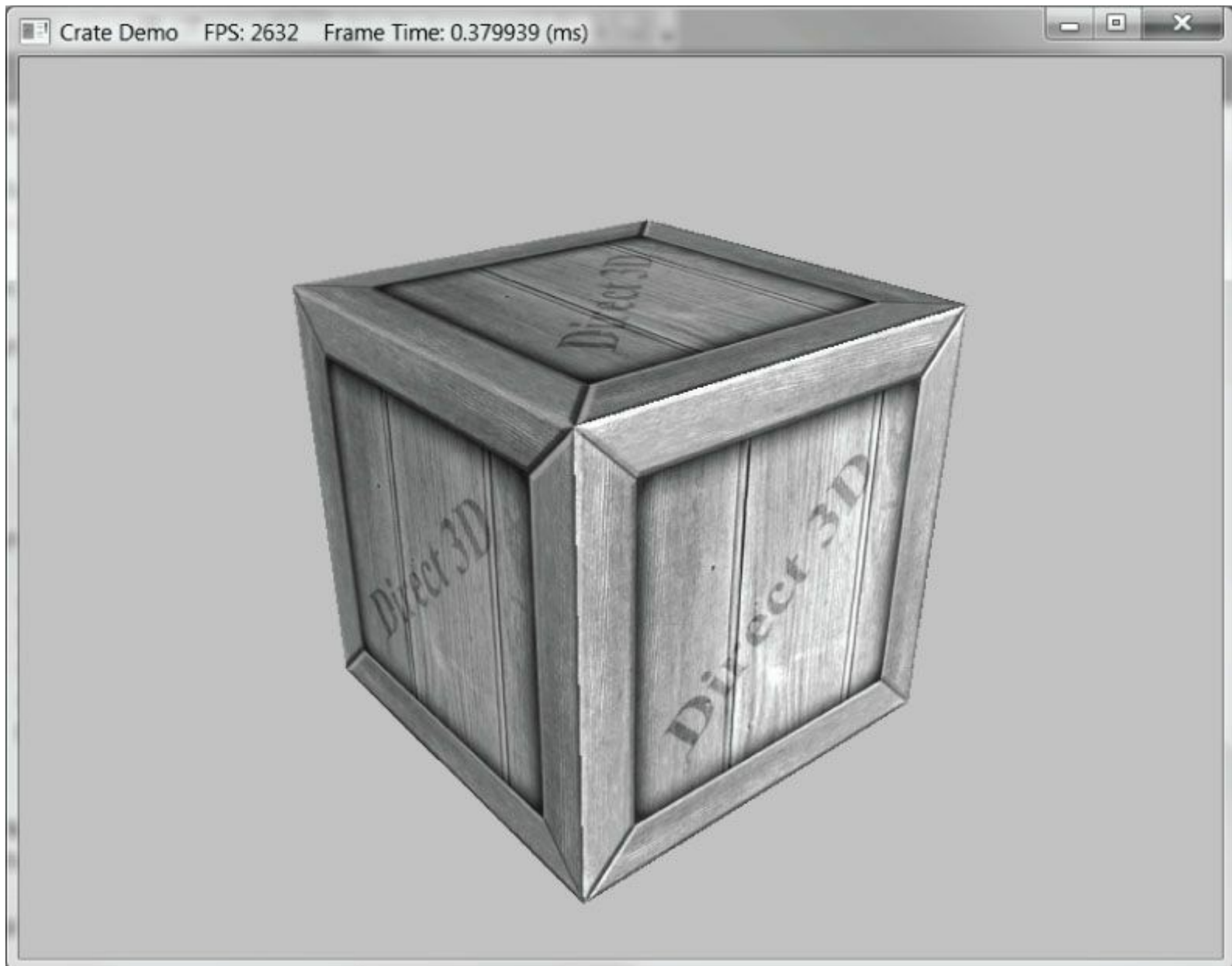


그림 8.1. Crate 데모는 크레이트 텍스처를 적용한 큐브를 생성합니다.

8.1 텍스처 및 리소스 요약

4장에서부터 이미 텍스처를 사용해 왔음을 상기하십시오. 특히, 깊이 버퍼와 백 버퍼는 ID3D11Texture2D 인터페이스로 표현되는 2차원 텍스처 객체입니다.

ID3D11Texture2D 인터페이스로 표현됩니다. 쉽게 참고할 수 있도록, 이 첫 번째 섹션에서는 4장에서 이미 다룬 텍스처 관련 내용을 대부분 복습합니다.

2D 텍스처는 데이터 요소들의 행렬입니다. 2D 텍스처의 한 가지 용도는 2D 이미지 데이터를 저장하는 것으로, 텍스처의 각 요소는 픽셀의 색상을 저장합니다. 그러나 이것이 유일한 용도는 아닙니다. 예를 들어 노멀 매핑이라는 고급 기법에서는 텍스처의 각 요소가 색상 대신 3차원 벡터를 저장합니다. 따라서 텍스처를 이미지 데이터 저장소로 생각하는 것이 일반적이지만, 실제로는 그보다 더 범용적인 용도로 사용됩니다. 1차원 텍스처(ID3D11Texture1D)는 데이터 요소의 1차원 배열과 같으며, 3차원 텍스처(ID3D11Texture3D)는 데이터 요소의 3차원 배열과 같습니다. 1차원, 2차원, 3차원 텍스처 인터페이스는 모두

ID3D11Resource를 상속합니다.

이 장 후반부에서 논의될 것처럼, 텍스처는 단순한 데이터 배열 이상입니다. 미프맵 레벨을 가질 수 있으며 GPU는 필터 적용 및 멀티샘플링과 같은 특수 작업을 수행할 수 있습니다. 그러나 텍스처는 임의의 데이터 체크가 아닙니다. DXGI_FORMAT 열거형 유형으로 설명되는 특정 종류의 데이터 형식만 저장할 수 있습니다. 몇 가지 예시 형식은 다음과 같습니다:

- 1. DXGI_FORMAT_R32G32B32_FLOAT: 각 요소는 32비트 부동 소수점 성분 3개를 가집니다.
- 2. DXGI_FORMAT_R16G16B16A16_UNORM: 각 요소는 [0, 1] 범위에 매핑된 네 개의 16비트 구성 요소를 가집니다.
- 3. DXGI_FORMAT_R32G32_UINT: 각 요소는 두 개의 32비트 부호 없는 정수 구성 요소를 가집니다.
- 4. DXGI_FORMAT_R8G8B8A8_UNORM: 각 요소는 [0, 1] 범위에 매핑된 4개의 8비트 부호 없는 구성 요소를 가집니다.
- 5. DXGI_FORMAT_R8G8B8A8_SNORM: 각 요소는 [-1, 1] 범위에 매핑된 4개의 8비트 부호 있는 구성 요소를 가지고 있습니다.
- 6. DXGI_FORMAT_R8G8B8A8_SINT: 각 요소는 [-128, 127] 범위에 매핑된 4개의 8비트 부호 있는 정수 구성 요소를 가집니다.
- 7. DXGI_FORMAT_R8G8B8A8_UINT: 각 요소는 [0, 255] 범위에 매핑된 네 개의 8비트 부호 없는 정수 구성 요소를 가집니다.

R, G, B, A 문자는 각각 빨강(red), 녹색(green), 파랑(blue), 알파(alpha)를 나타냅니다. 그러나 앞서 언급했듯이 텍스처는 색상 정보를 저장할 필요가 없습니다. 예를 들어,

DXGI_FORMAT_R32G32B32_FLOAT

는 세 개의 부동 소수점 구성 요소를 가지므로 부동 소수점 좌표를 가진 3차원 벡터(색상 벡터일 필요 없음)를 저장할 수 있습니다. 또한 텍스처가 렌더링 파이프라인에 바인딩될 때 메모리를 예약한 후 데이터를 재해석하는 방식(일종의 형 변환)을 나중에 지정하는 *무형식* 형식도 있습니다. 예를 들어, 다음 무형식 형식은 4개의 8비트 구성 요소를 가진 요소를 예약하지만 데이터 유형(예: 정수, 부동 소수점, 부호 없는 정수)은 지정하지 않습니다:

DXGI_FORMAT_R8G8B8A8_TYPELESS

텍스처는 렌더링 파이프라인의 서로 다른 단계에 바인딩될 수 있습니다. 일반적인 예로는 텍스처를 렌더 타깃(즉, Direct3D가 텍스처에 그리는 경우)과 셰이더 리소스(즉, 셰이더에서 텍스처를 샘플링하는 경우)로 사용하는 것입니다. 이러한 두 가지 목적으로 생성된 텍스처 리소스에는 다음과 같은 바인딩 플래그가 지정됩니다:

D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE

텍스처가 바인딩될 두 파이프라인 단계를 나타냅니다. 실제로 리소스는 파이프라인 단계에 직접 바인딩되지 않으며, 대신 관련 리소스 뷰가 서로 다른 파이프라인 단계에 바인딩됩니다. 텍스처를 사용하는 각 방식에 대해 Direct3D는 초기화 시점에 해당 텍스처의 *리소스 뷰*를 생성할 것을 요구합니다. 이는 주로 효율성을 위한 것으로, SDK 문서에서도 지적하듯 "이를 통해 런타임 및 드라이버에서의 유효성 검사 및 매핑이 뷰 생성 시점에 수행되어 바인딩 시점의 타입 검사를 최소화할 수 있습니다." 따라서 텍스처를 렌더 타깃 및 셰이더 리소스로 사용하는 예시의 경우, 렌더 타깃 뷰(ID3D11RenderTargetView)와 셰이더 리소스 뷰(ID3D11ShaderResourceView)라는 두 개의 뷰를 생성해야 합니다. 리소스 뷰는 기본적으로 두 가지 역할을 수행합니다: Direct3D에 리소스의 사용 방식(즉, 파이프라인의 어느 단계에 바인딩할지)을 알리고, 리소스 형식이 생성 시 무형식(typless)으로 지정된 경우 뷰 생성 시 해당 형식을 명시해야 합니다. 따라서 타입리스 형식을 사용하면 텍스처의 요소가 한 파이프라인 단계에서는 부동 소수점 값으로, 다른 단계에서는 정수 값으로 볼 수 있습니다. 이는 본질적으로 데이터의 재해석 캐스트에 해당합니다.

August SDK 문서에는 다음과 같이 명시되어 있습니다. "완전한 타입 지정 리소스를 생성하면 해당 리소스가 생성된 형식()으로 제한됩니다. 이를 통해 런타임은 액세스를 최적화할 수 있습니다[...]". 따라서 정말 필요한 경우에만 타입이 지정되지 않은 리소스를 생성해야 합니다. 그렇지 않은 경우 완전히 타입이 지정된 리소스를 생성하십시오. 필요할 때만 생성해야 하며, 그렇지 않은 경우 완전히 타입화된 리소스를 생성해야 합니다.

특정 리소스에 대한 뷰를 생성하려면 해당 리소스를 생성할 때 특정 바인드 플래그를 지정해야 합니다. 예를 들어, 리소스를 D3D11_BIND_SHADER_RESOURCE 바인드 플래그(텍스처가 셰이더 파이프라인에

깊이/스텐실 버퍼로 파이프라인에 바인딩될 것임을 나타냄), 해당 리소스에 대한 **ID3D11ShaderResourceView**를 생성할 수 없습니다. 시도할 경우 다음과 같은 Direct3D 디버그 오류가 발생합니다:

D3D11: ERROR: ID3D11Device::CreateShaderResourceView: D3D11_BIND_SHADER_RESOURCE 바인드 플래그를 지정하지 않은 리소스에 대해서는 셰이더 리소스 뷰를 생성할 수 없습니다.

이 장에서는 텍스처를 셰이더 리소스로 바인딩하는 데만 초점을 맞출 것입니다. 이를 통해 픽셀 셰이더가 텍스처를 샘플링하고 이를 활용하여 픽셀에 색상을 부여할 수 있습니다.

8.2 텍스처 좌표

Direct3D는 이미지에 대해 수평으로 진행하는 u축과 수직으로 진행하는 v축으로 구성된 텍스처 좌표계를 사용합니다. $0 \leq u, v \leq 1$ 인 좌표 (u, v)는 *텍셀0/라고* 불리는 텍스처 상의 요소를 식별합니다. v축은 "아래" 방향으로 양의 값을 가짐에 유의하십시오([그림 8.2](#) 참조). 또한 정규화된 좌표 구간 [0, 1]이 사용되는 이유는 Direct3D가 작업할 수 있는 차원 독립적 범위를 제공하기 때문입니다. 예를 들어, (0.5, 0.5)는 실제 텍스처 크기가 256×256 , 512×1024 , 2048×2048 픽셀 중 어느 것이든 항상 중앙 텍셀을 지정합니다. 마찬가지로 (0.25, 0.75)는 가로 방향으로 전체 너비의 1/4 지점, 세로 방향으로 전체 높이의 3/4 지점을 가리킵니다. 현재 텍스처 좌표는 항상 [0, 1] 범위 내에 있지만, 이 범위를 벗어날 경우 발생할 수 있는 현상에 대해서는 후술하겠습니다.

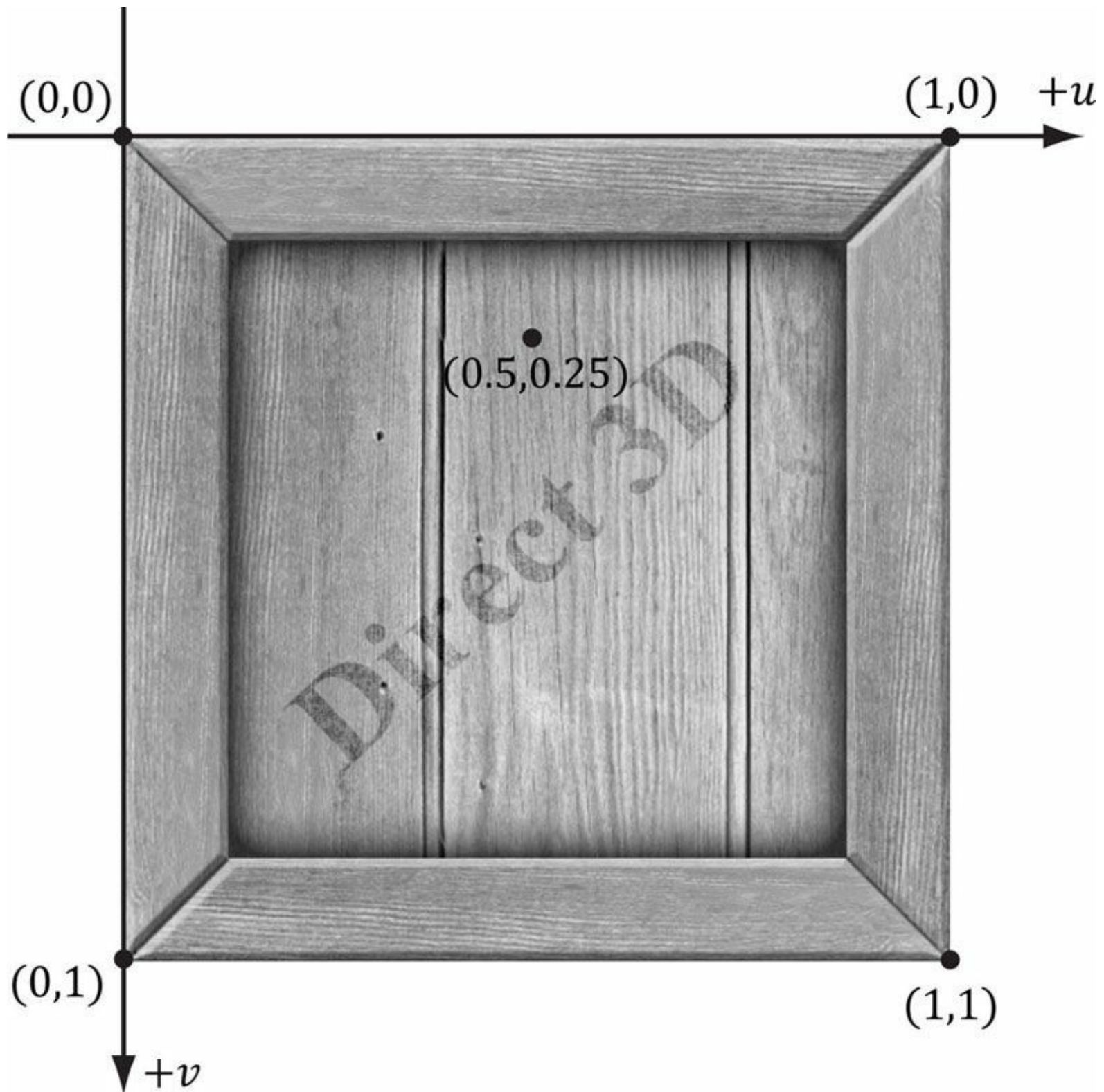


그림 8.2. 텍스처 좌표계(때로는 텍스처 공간이라고도 함).

각 3차원 삼각형에 대해, 해당 3차원 삼각형에 매핑될 텍스처 상의 대응 삼각형을 정의하고자 한다(그림 8.3 참조). p_0, p_1, p_2 를 각각 텍스처 좌표 $q_0, q_1, q_{(2)}$ 를 가지는 3차원 삼각형의 꼭짓점으로 하자. 3차원 삼각형 상의 임의의 점 (x, y, z) 에 대해, 그 텍스처 좌표 (u, v) 는 동일한 s, t 매개변수를 사용하여 3차원 삼각형 전체에 걸쳐 정점 텍스처 좌표를 선형 보간함으로써 구해집니다. 즉, 만약

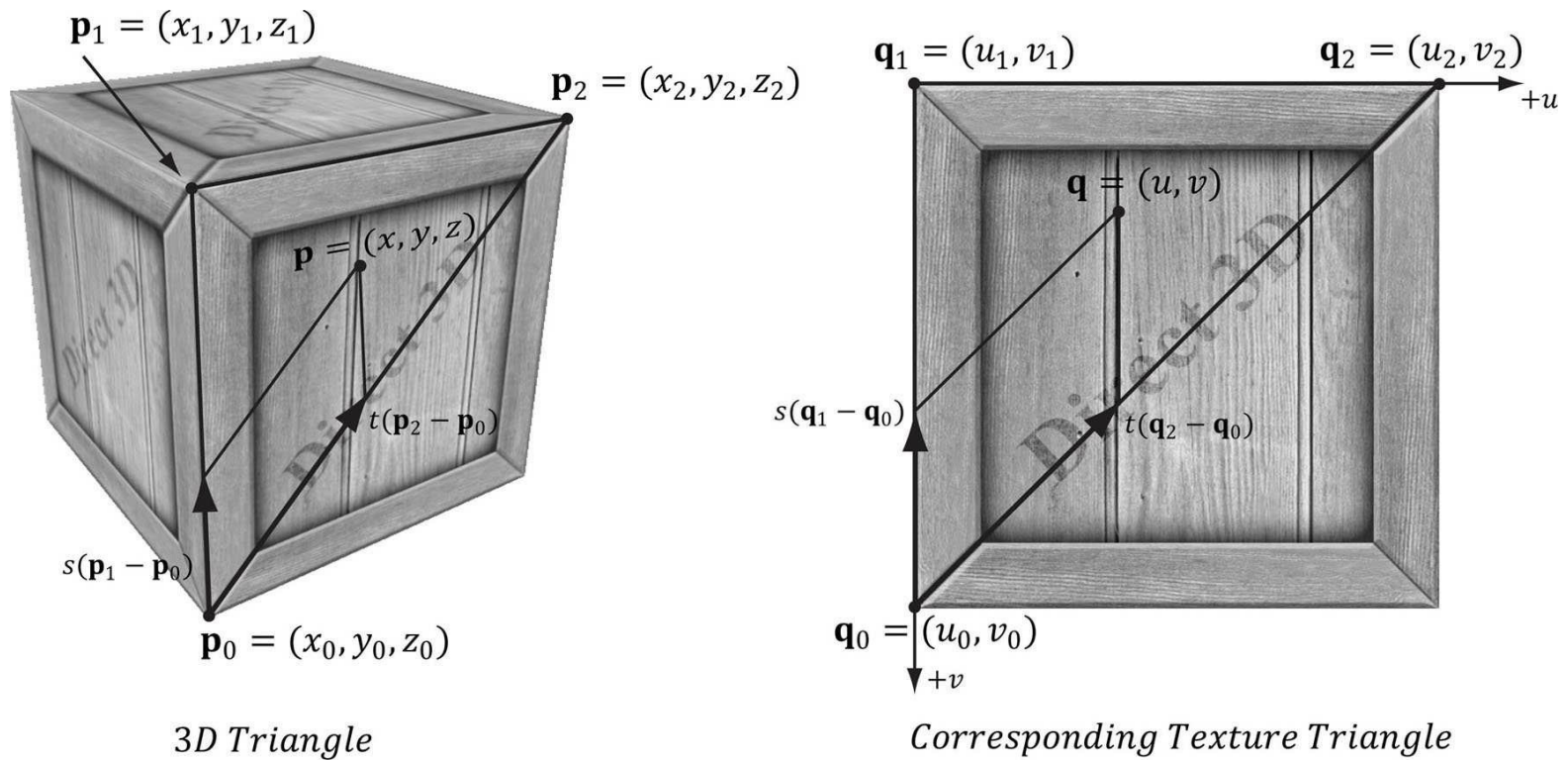


그림 8.3. 왼쪽은 3차원 공간의 삼각형이며, 오른쪽은 3차원 삼각형에 매핑될 텍스처 위에 정의된 2차원 삼각형이다.

$$(x, y, z) = \mathbf{p} = \mathbf{p}_0 + s(\mathbf{p}_1 - \mathbf{p}_0) + t(\mathbf{p}_2 - \mathbf{p}_0)$$

$s \geq 0, t \geq 0, s + t \leq 1$ 일 때,

$$(u, v) = \mathbf{q} = \mathbf{q}_0 + s(\mathbf{q}_1 - \mathbf{q}_0) + t(\mathbf{q}_2 - \mathbf{q}_0)$$

이렇게 하면 삼각형의 모든 점에 대응하는 텍스처 좌표가 부여됩니다.

이를 구현하기 위해 버텍스 구조를 다시 수정하고 텍스처 상의 점을 식별하는 텍스처 좌표 쌍을 추가합니다.

이제 모든 3D 정점에는 대응하는 2D 텍스처 정점이 존재합니다. 따라서 세 개의 정점으로 정의된 모든 3D 삼각형은 텍스처 공간에서도 2D 삼각형을 정의합니다(즉, 모든 3D 삼각형에 대해 2D 텍스처 삼각형을 연관시켰습니다).

// 기본 32비트 버텍스 구조체. struct Basic32

```
{
    XMFLOAT3 Pos;
    XMFLOAT3 Normal;
    XMFLOAT2 Tex;
};
const D3D11_INPUT_ELEMENT_DESC InputLayoutDesc::Basic32[3] =
{
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
     D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12,
     D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 24,
     D3D11_INPUT_PER_VERTEX_DATA, 0}
};
```

2D 텍스처 삼각형과 3D 삼각형이 크게 다른 "이상한" 텍스처 매핑을 생성할 수 있습니다. 따라서 2D 텍스처를 3D 삼각형에 매핑할 때 많은 늘어나고 왜곡이 발생하여 결과가 보기 좋지 않게 됩니다. 예를 들어, 예각 삼각형을 직각 삼각형에 매핑하려면 늘려야 합니다. 일반적으로 텍스처 왜곡은 텍스처 아티스트가 왜곡된 모습을 의도하지 않는 한 최소화해야 합니다.

그림 8.3에서 볼 수 있듯이, 우리는 전체 텍스처 이미지를 큐브의 각 면에 매핑합니다. 이는 반드시 필요한 것은 아닙니다. 텍스처의 일부만 지오메트리에 매핑할 수도 있습니다. 실제로 하나의 큰 텍스처 맵(텍스처 아틀라스라고 함)에 서로 관련 없는 여러 이미지를 배치하고, 이를 여러 다른 오브젝트에 사용할 수 있습니다(그림 8.4). 텍스처 좌표는 삼각형에 매핑되는 텍스처의 어느 부분이 될지를 결정합니다.

삼각형에 매핑될지 결정합니다.

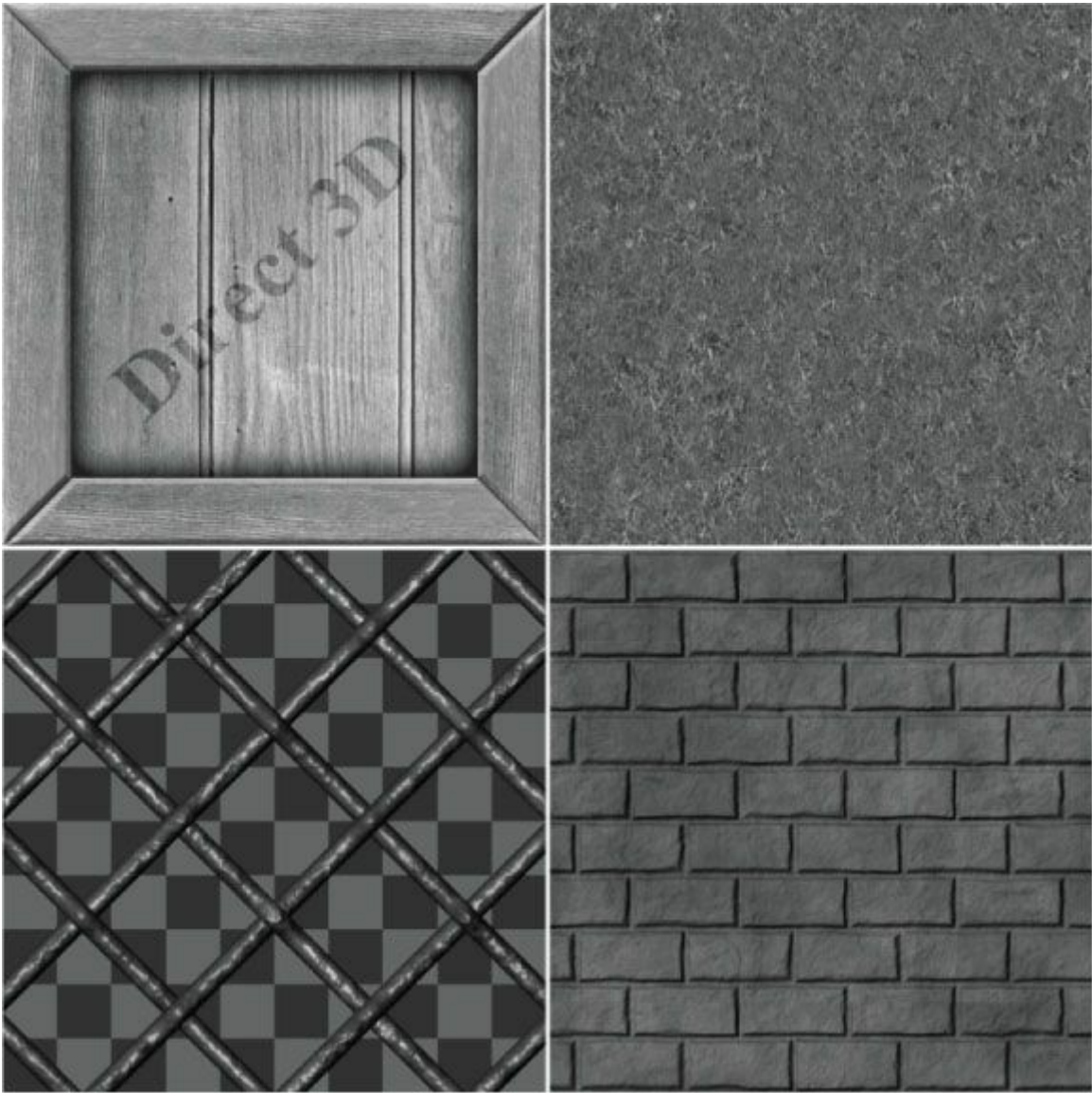


그림 8.4. 하나의 큰 텍스처에 네 개의 서브텍스처를 저장한 텍스처 아틀라스. 각 버텍스의 텍스처 좌표는 원하는 텍스처 부분이 지오메트리에 매핑되도록 설정됩니다.

8.3 텍스처 생성 및 활성화

텍스처 데이터는 일반적으로 디스크에 저장된 이미지 파일에서 읽혀져 ID3D11Texture2D 객체로 로드됩니다(D3DX11CreateTextureFromFile 참조). 그러나 텍스처 리소스는 렌더링 파이프라인에 직접 바인딩되지 않습니다. 대신 텍스처에 대한 셰이더 리소스 뷰(ID3D11ShaderResourceView)를 생성한 후, 이 뷰를 파이프라인에 바인딩합니다. 따라서 두 단계가 필요합니다:

1. 디스크에 저장된 이미지 파일로부터 ID3D11Texture2D 객체를 생성하려면 D3DX11CreateTextureFromFile를 호출하십시오.
2. 텍스처에 대응하는 셰이더 리소스 뷰를 생성하려면 ID3D11Device::CreateShaderResourceView를 호출하십시오. 이 두 단계는 다음 D3DX 함수를 사용하여 한 번에 수행할 수 있습니다:

```
HRESULT D3DX11CreateShaderResourceViewFromFile( ID3D11Device
    *pDevice,
    LPCTSTR pSrcFile, D3DX11_IMAGE_LOAD_INFO
    *pLoadInfo, ID3D11ThreadPump *pPump,
    ID3D11ShaderResourceView **ppShaderResourceView, HRESULT *pHResult
);
```

1. **pDevice**: 텍스처를 생성할 D3D 장치에 대한 포인터.
2. **pSrcFile**: 로드할 이미지의 파일명.
3. **pLoadInfo**: 선택적 이미지 정보; 소스 이미지의 정보를 사용하려면 null을 지정합니다. 예를 들어, 여기에 null을 지정하면 소스 이미지 크기가 텍스처 크기로 사용되며, 전체 맵 체인이 생성됩니다 (§8.4.2). 이는 일반적으로 항상 원하는 것이며 좋은 기본 선택입니다.
4. **pPump**: 리소스 로딩을 위한 새 스레드를 생성하는 데 사용됩니다. 메인 스레드에서 리소스를 로드하려면 null을 지정합니다. 이 책에서는 항상 null을 지정할 것입니다.
5. **ppShaderResourceView**: 파일에서 로드된 텍스처에 대해 생성된 셰이더 리소스 뷰에 대한 포인터를 반환합니다.
6. **pHResult**: **pPump**에 null이 지정된 경우 null을 지정합니다.

이 함수는 다음 이미지 형식 중 하나를 로드할 수 있습니다: BMP, JPG, PNG, DDS, TIFF, GIF 및 WMP (

D3DX11_IMAGE_FILE_FORMAT 참조).

 때로는 텍스처와 해당 셰이더 리소스 뷰를 서로 바꿔 쓸 수 있는 것으로 언급하기도 합니다. 예를 들어, 실제로는 뷰를 바인딩하고 있음에도 텍스처를 파이프라인에 바인딩한다고 표현할 수 있습니다.

예를 들어, *WoodCreate01.dds*라는 이미지에서 텍스처를 생성하려면 다음과 같이 작성합니다:

```
ID3D11ShaderResourceView* mDiffuseMapSRV;
HR(D3DX11CreateShaderResourceViewFromFile(md3dDevice,
    L"WoodCrate01.dds", 0, 0, &mDiffuseMapSRV, 0));
```

텍스처가 로드되면 픽셀 셰이더에서 사용할 수 있도록 효과 변수에 할당해야 합니다.

fx 파일 내의 2D 텍스처 객체는 **Texture2D** 타입으로 표현됩니다. 예를 들어, 효과 파일에서 텍스처 변수를 다음과 같이 선언합니다:

```
// 비수치 값은 cbuffer에 추가할 수 없습니다. Texture2D gDiffuseMap;
```

주석에서 언급된 대로 텍스처 객체는 상수 버퍼 외부로 배치됩니다. 효과의 **Texture2D** 포인터는 다음과 같이 얻을 수 있습니다:

다음과 같이 C++ 애플리케이션 코드에서 객체(셰이더 리소스 변수)를 참조합니다:

```
ID3DX11EffectShaderResourceVariable* DiffuseMap;
fxDiffuseMap = mFX->GetVariableByName("gDiffuseMap")->AsShaderResource();
```

이펙트의 **Texture2D** 객체에 대한 포인터를 획득한 후에는 다음과 같이 C++ 인터페이스를 통해 업데이트할 수 있습니다:

```
// C++ 텍스처 리소스 뷰를 효과 텍스처 변수로 설정합니다. fxDiffuseMap-
>SetResource(mDiffuseMapSRV);
```

다른 효과 변수와 마찬가지로, 드로우 호출 사이에 변경이 필요한 경우 **Apply**를 호출해야 합니다:

```
// 크레이트 텍스처 설정
fxDiffuseMap->SetResource(mCrateMapSRV); pass->Apply(0,
md3dImmediateContext); DrawCrate();
```

```
// 잔디 텍스처 설정
fxDiffuseMap->SetResource(mGrassMapSRV); pass->Apply(0,
md3dImmediateContext); DrawGrass();
```

```
// 벽돌 텍스처 설정
fxDiffuseMap->SetResource(mBrick MapSRV); pass->Apply(0,
md3dImmediateContext); DrawBricks();
```

텍스처 아틀라스는 하나의 드로우 콜로 더 많은 지오메트리를 렌더링할 수 있게 하여 성능을 향상시킬 수 있습니다. 예를 들어, [그림 8.3](#)과 같이 크레이트, 잔디, 벽돌 텍스처를 포함하는 텍스처 아틀라스를 사용한다고 가정해 보겠습니다. 그런 다음 각 오브젝트의 텍스처 좌표를 해당 서브텍스처에 맞게 조정함으로써 하나의 드로우 콜로 지오메트리를 렌더링할 수 있습니다(오브젝트별로 변경해야 할 다른 매개변수가 없다고 가정할 때):

```
// 텍스처 아틀라스 설정
```



```
fxDiffuseMap->SetResource(mAtlasSRV); pass->Apply(0,
md3dImmediateContext); DrawCrateGrassAndBricks();
```

드로우 콜에는 오버헤드가 발생하므로 이와 같은 기법으로 드로우 콜 수를 최소화하는 것이 바람직합니다.

텍스처 리소스는 실제로 모든 셰이더(버텍스, 지오메트리 또는 픽셀 셰이더)에서 사용할 수 있습니다. 현재로서는 픽셀 셰이더에서만  할 예정입니다. 앞서 언급했듯이 텍스처는 본질적으로 특수한 배열이므로,

배열 데이터가 버텍스 및 지오메트리 셰이더 프로그램에서도 유용할 수 있다는 점을 쉽게 예상할 수 있습니다.

8.4 필터

8.4.1 확대

텍스처 맵의 요소들은 연속적인 이미지에서 추출된 개별 색상 샘플로 간주되어야 하며, 면적을 가진 직사각형으로 생각해서는 안 됩니다. 따라서 질문은 다음과 같습니다: 텍셀 점 중 어느 하나와도 일치하지 않는 텍스처 좌표 (u, v)가 있다면 어떻게 될까요? 이는 다음과 같은 상황에서 발생할 수 있습니다. 플레이어가 장면 속 벽을 확대하여 벽이 화면 전체를 덮도록 한다고 가정해 보자. 예를 들어 모니터 해상도가 1024×1024 이고 벽의 텍스처 해상도가 256×256 이라고 가정하자. 이는 텍스처 *확대*를 보여준다—소수의 텍셀로 많은 픽셀을 덮으려는 시도이다. 이 예시에서 각 텍셀 점 사이에는 네 개의 픽셀이 위치한다. 삼각형 전체에 걸쳐 정점 텍스처 좌표가 보간될 때 각 픽셀에는 고유한 텍스처 좌표 쌍이 할당됩니다. 따라서 텍셀 점과 일치하지 않는 텍스처 좌표를 가진 픽셀이 존재하게 됩니다. 텍셀의 색상을 기반으로 보간을 통해 텍셀 사이의 색상을 근사할 수 있습니다. 그래픽스 하드웨어가 지원하는 보간 방법에는 상수 보간과 선형 보간 두 가지가 있습니다. 실제로는 거의 항상 선형 보간이 사용됩니다.

그림 8.5는 이러한 방법을 1차원에서 설명합니다: 256개의 샘플과 보간된 텍스처 좌표

$u = 0.126484375$ 입니다. 이 정규화된 텍스처 좌표는 $0.126484375 \times 256 = 32.38$ 텍셀을 가리킵니다. 물론 이 값은 두 텍셀 샘플 사이에 위치하므로, 이를 근사화하기 위해 보간법을 사용해야 합니다.

2차원 선형 보간법은 이차원 보간법(bilinear interpolation)이라 불리며 그림 8.6에 설명되어 있습니다. 두 텍스처 좌표 사이의 한 쌍이 주어지면

네 개의 텍셀에 대해, u 방향으로 두 번의 1차원 선형 보간을 수행한 후 v 방향으로 한 번의 1차원 보간을 수행합니다.

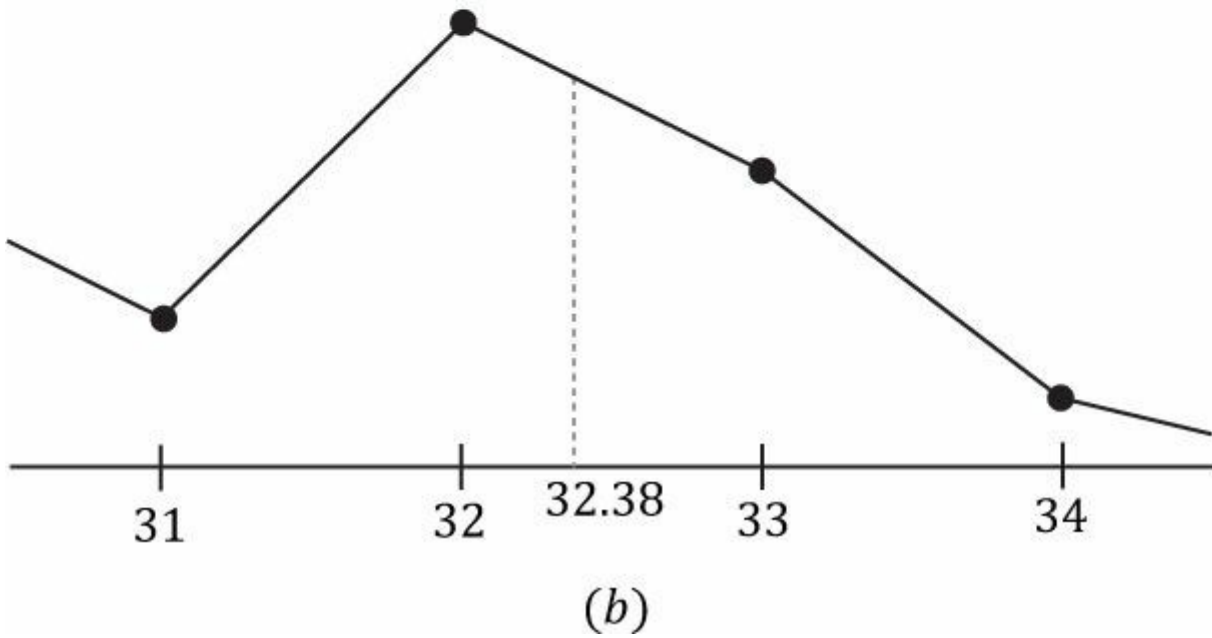
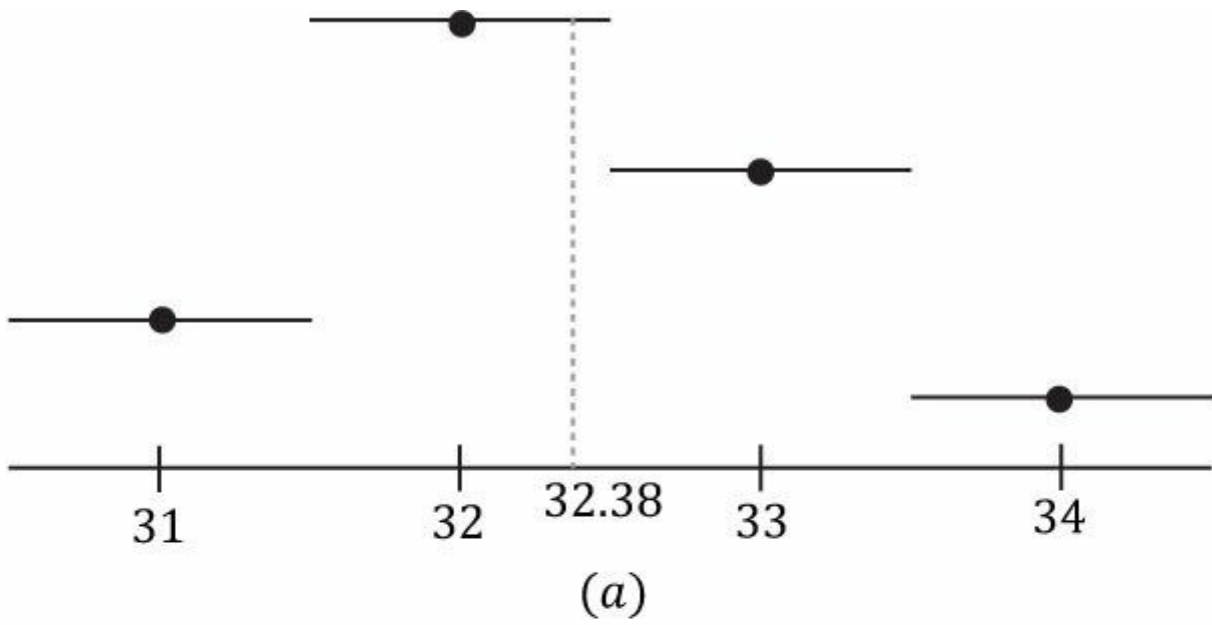


그림 8.5. (a) 텍셀 점들을 주어, 텍셀 점들 사이의 값을 근사하기 위해 부분적으로 일정한 함수를 구성합니다. 이는 가장 가까운 텍셀 점의 값을 사용하기 때문에 때때로 '가장 가까운 점 샘플링'이라고도 불립니다. (b) 텍셀 점들을 주어, 텍셀 점들 사이의 값을 근사하기 위해 부분적으로 선형인 함수를 구성합니다.

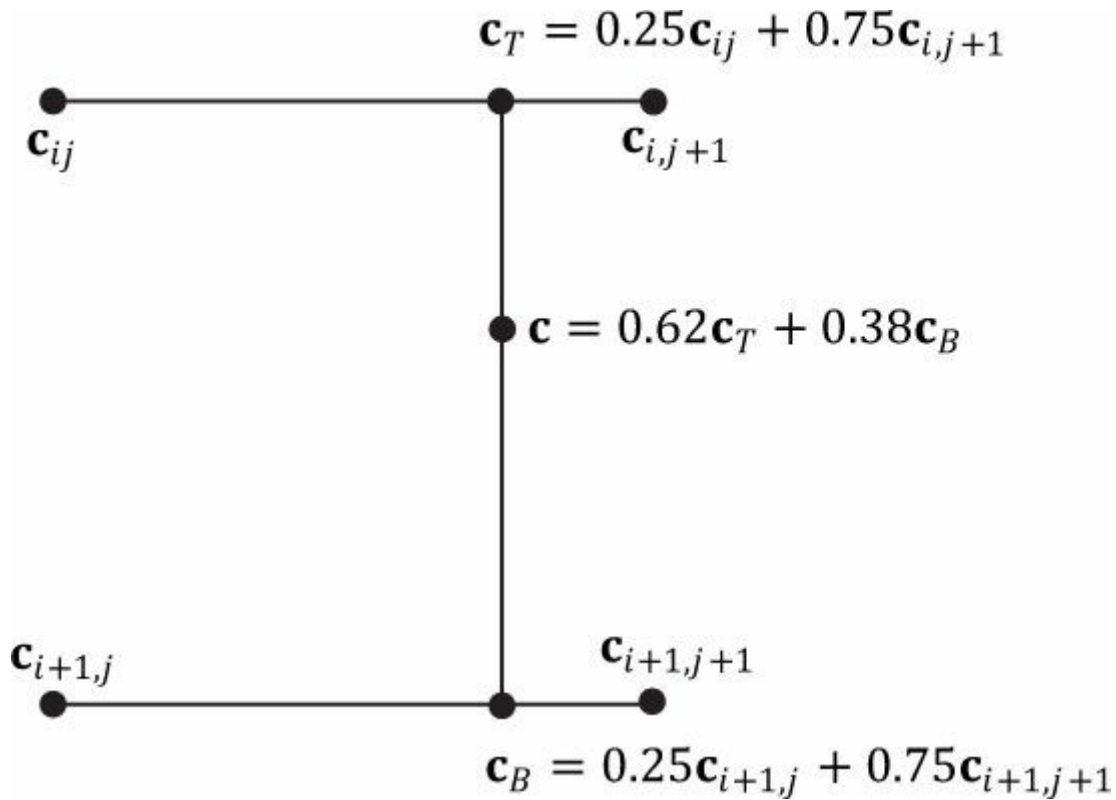


그림 8.6. 여기에는 네 개의 텍셀 점 c_{ij} , $c_{i,j+1}$, $c_{i+1,j}$, $c_{i+1,j+1}$ 이 있습니다. 우리는 보간법을 사용하여 이 네 텍셀 점 사이에 위치한 c 의 색상을 근사하고자 합니다. 이 예시에서 c 는 c_{ij} 의 오른쪽 0.75 단위, 아래 0.38 단위에 위치합니다. 먼저 상단 두 색상 사이에서 1차원 선형 보간을 수행하여 c_T 를 구합니다. 마찬가지로 하단 두 색상 사이에서 1차원 선형 보간을 수행하여 c_B 를 구합니다. 마지막으로 c_T 와 c_B 사이에서 1차원 선형 보간을 수행하여 c 를 구합니다.

그림 8.7은 상수 보간과 선형 보간의 차이를 보여줍니다. 보시다시피 상수 보간은 블록 모양의 이미지를 생성하는 특성이 있습니다. 선형 보간은 더 부드럽지만, 보간을 통해 유도된 데이터 대신 실제 데이터(예: 더 높은 해상도의 텍스처)를 사용했을 때만큼 좋게 보이지는 않습니다.

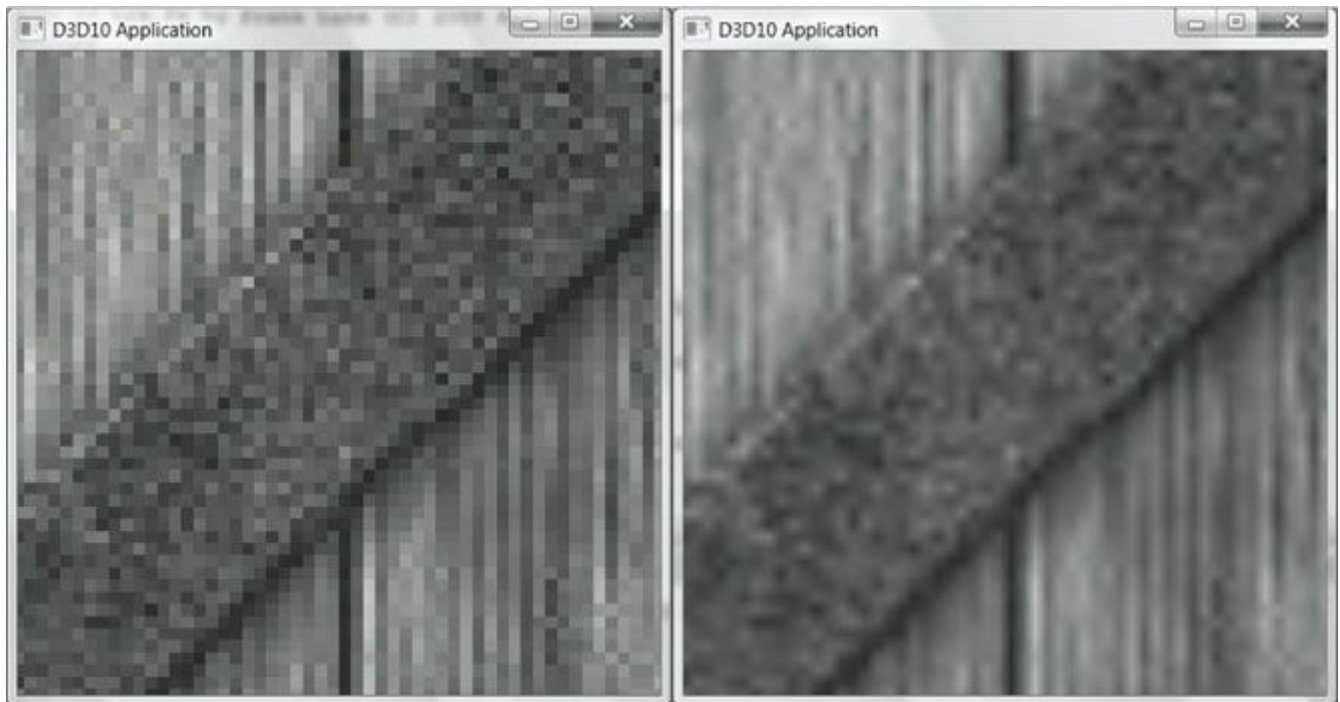


그림 8.7. 상자 질감이 적용된 정육면체를 확대하여 확대 효과를 보여줍니다. 왼쪽은 상수 보간법을 사용한 결과로, 블록 모양의 외관이 나타납니다. 이는 보간 함수가 불연속성을 가지기 때문에(그림 8.5a) 변화가 부드럽지 않고 갑작스럽게 나타나는 것이 합리적입니다. 오른쪽은 선형 필터링을 사용한 결과로, 보간 함수의 연속성 덕분에 더 부드러운 이미지가 생성됩니다.

이 논의에서 주목할 점은 가상 시점이 자유롭게 이동하며 탐색하는 인터랙티브 3D 프로그램에서 확대 효과를 완전히 피할 방법이 없다는 것입니다. 일정 거리에서는 텍스처가 훌륭해 보이지만, 시점이 너무 가까이 접근하면 텍스처가 깨지기 시작합니다. 일부 게임은 과도한 확대를 방지하기 위해 가상 시점이 표면에 접근할 수 있는 거리를 제한합니다. 고해상도 텍스처를 사용하는 것이 도움이 될 수 있습니다.

텍스처링의 맥락에서, 텍셀 사이의 텍스처 좌표에 대한 텍스처 값을 찾기 위해 상수 보간법을 사용하는 것을 포인트 필터링이라고도 합니다. 그리고 텍셀 사이의 텍스처 좌표에 대한 텍스처 값을 찾기 위해 선형 보간법을 사용하는 것을 선형 필터링이라고도 합니다. 포인트 및 선형 필터링은 Direct3D가 사용하는 용어입니다.

8.4.2 미니피케이션

축소(minification)는 확대(magnification)의 반대 개념입니다. 축소에서는 너무 많은 텍셀이 너무 적은 픽셀에 매핑됩니다. 예를 들어, 256×256 텍스처가 매핑된 벽이 있는 상황을 생각해 보십시오. 벽을 바라보는 시선이 계속 뒤로 물러나면서 벽이 점점 작아져 결국 화면에서 64×64 픽셀만 차지하게 됩니다. 이제 256×256 텍셀이 64×64 화면 픽셀에 매핑됩니다. 이 상황에서도 픽셀의 텍스처 좌표는 일반적으로 텍스처 맵의 어떤 텍셀과도 일치하지 않으므로, 미니어처화 경우에도 상수 보간 및 선형 보간 필터가 여전히 적용됩니다. 그러나 축소화에는 더 많은 작업이 가능합니다. 직관적으로 256×256 텍셀을 64×64 로 줄이기 위해 일종의 평균 다운샘플링을 수행해야 합니다. mipmapping 기법은 추가 메모리 비용을 대가로 이를 효율적으로 근사화합니다. 초기화 시점(또는 에셋 생성 시점)에 이미지를 다운샘플링하여 더 작은 버전의 텍스처를 생성함으로써 mipmap 체인을 만듭니다(그림 8.8 참조). 따라서 mipmap 크기에 대한 평균화 작업은 미리 계산됩니다. 런타임에는 그래픽스 하드웨어가 프로그래머가 지정한 mipmap 설정에 따라 두 가지 다른 작업을 수행합니다:

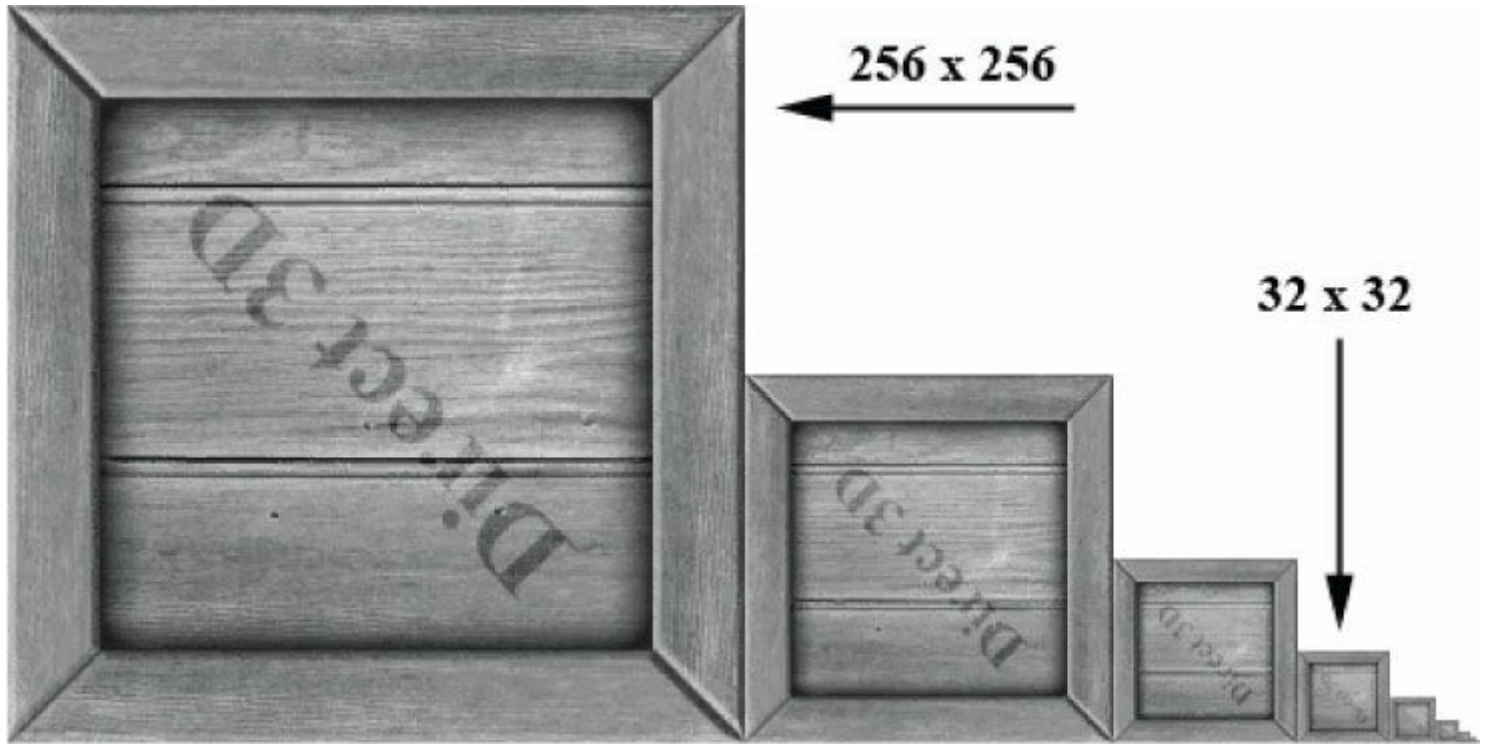


그림 8.8. mipmap 체인; 각 연속적인 mipmap은 이전 디테일 레벨의 mipmap보다 각 차원에서 크기가 절반이며, 1×1 까지 감소한다.

1. 텍스처링을 위해 투영된 화면 기하학적 해상도와 가장 잘 일치하는 mipmap 레벨을 선택하여 사용하십시오. 필요에 따라 상수 또는 선형 보간을 적용합니다. 이는 상수 보간과 유사하기 때문에 mipmap에 대한 *포인트 필터링*이라고 불립니다—가장 가까운 mipmap 레벨을 선택하여 텍스처링에 사용하는 방식입니다.
2. 텍스처링을 위해 투영된 화면 기하 구조 해상도와 가장 잘 일치하는 가장 가까운 두 mipmap 레벨을 선택합니다(하나는 화면 기하 구조 해상도보다 크고, 다른 하나는 작을 것입니다). 다음으로, 이 두 mipmap 레벨 모두에 상수 필터링 또는 선형 필터링을 적용하여 각각에 대한 텍스처 색상을 생성합니다. 마지막으로, 이 두 텍스처 색상 결과 사이를 보간합니다. 이는 가장 가까운 두 mipmap 레벨 사이를 선형 보간하는 것과 유사하기 때문에 mipmap에 대한 *선형 필터링*이라고 합니다.

mipmap 체인에서 최적의 텍스처 디테일 레벨을 선택함으로써, 디테일 축소량이 크게 감소됩니다.

8.4.2.1 mipmap 생성

mipmap 레벨은 아티스트가 직접 생성하거나 필터링 알고리즘을 통해 생성할 수 있습니다.

일부 이미지 파일 형식(예: .DDS(DirectDraw Surface format))은 mipmap 레벨을 데이터 파일에 직접 저장할 수 있습니다. 이 경우 데이터를 읽기만 하면 됩니다. 런타임에 알고리즘으로 mipmap 레벨을 계산할 필요가 없습니다. DirectX 텍스처 도구를 사용하면 텍스처에 대한 mipmap 체인을 생성하고 이를 .DDS 파일로 내보낼 수 있습니다. 이미지 파일에 완전한 mipmap 체인이 포함되어 있지 않은 경우, `D3DX11CreateShaderResourceViewFromFile` 또는 `D3DX11CreateTextureFromFile` 함수는 지정된 필터링 알고리즘을 사용하여 mipmap 체인을 생성합니다(SDK 문서의 `D3DX11_IMAGE_LOAD_INFO`, 특히 `MipFilter` 데이터 멤버 참조). 따라서 mipmapping은 본질적으로 자동으로 이루어진다고 볼 수 있습니다. 소스 파일에 mipmap 체인이 포함되어 있지 않은 경우 `D3DX11` 함수가 자동으로 생성해 줍니다. 또한 mipmapping이 활성화된 상태라면 하드웨어가 런타임에 사용할 적절한

미프맵 레벨을 선택합니다.

Note: 일반적인 필터링 알고리즘은 다운샘플링 시 원하는 디테일을 보존하지 못하는 경우가 있습니다. 예를 들어, [그림 8.8](#)에서 상자에 쓰여진 "Direct 3D" 텍스트는 낮은 mip 레벨에서 흐려져 사라집니다.

용납할 수 없다면, 아티스트는 중요한 디테일을 유지하기 위해 mip 레벨을 수동으로 생성하거나 조정할 수 있습니다.

8.4.3 이방성 필터링

사용 가능한 또 다른 필터 유형은 *이방성 필터링*입니다. 이 필터는 다각형의 법선 벡터와 카메라의 시선 벡터 사이 각도가 넓을 때 발생하는 왜곡을 완화하는 데 도움이 됩니다(예: 다각형이 뷰 윈도우에 직각일 때). 이 필터는 가장 비용이 많이 들지만 왜곡 아티팩트를 수정하는 데 그 비용을 감수할 가치가 있습니다. [그림 8.9](#)는 이방성 필터링과 선형 필터링을 비교한 스크린샷을 보여줍니다.



그림 8.9. 상자의 상단면은 뷰 윈도우에 거의 직각을 이룬다. (왼쪽) 선형 필터링을 사용하면 상자의 상단이 심하게 흐려진다. (오른쪽) 이 각도에서 상자의 상단면을 렌더링하는 데 이방성 필터링이 더 효과적이다.

8.5 텍스처 샘플링

텍스처2D 객체가 이펙트 파일 내 텍스처를 나타낸다는 점을 살펴보았습니다. 그러나 텍스처와 연관된 또 다른 객체인 **샘플러 상태** 객체(또는 샘플러)가 존재합니다. 샘플러 객체는 텍스처에 사용할 필터를 정의하는 곳입니다. 다음은 몇 가지 예시입니다:

```
// 축소, 확대 및 mip매핑에 선형 필터링 사용 SamplerState mySampler0
{
    Filter = MIN_MAG_MIP_LINEAR;
};
```

```
// 축소에는 선형 필터링, 확대에는 점 필터링,
// 그리고 mip매핑에는 포인트 필터링을 사용합니다.
SamplerState mySampler1
{
    필터 = MIN_LINEAR_MAG_MIP_POINT;
};
```

```
// 축소에는 포인트 필터링, 확대에는 선형 필터링,
// 그리고 mip매핑에는 포인트 필터링을 사용합니다.
SamplerState mySampler2
{
    Filter = MIN_POINT_MAG_LINEAR_MIP_POINT;
};
```

```
// 축소, 확대,
// 및 mip매핑에 이방성 보간을 사용합니다.
SamplerState mySampler3
{
    Filter = ANISOTROPIC;
    MaxAnisotropy = 4;
};
```

이방성 필터링의 경우 최대 이방성 값(1~16 사이의 숫자)을 반드시 지정해야 합니다. 값이 클수록 비용은 증가하지만 더 나은 결과를 얻을 수 있습니다. 다른 가능한 조합은 이 예시들을 통해 파악하거나 SDK 문서에서 **D3D11_FILTER** 열거형 유형을 참조하세요. 본 장 후반부에서 다른 속성들이 샘플러와 연관되어 있지만, 현재로서는 첫 번째 데모에 필요한 설정은 여기까지입니다.

이제 픽셀 셰이더에서 픽셀에 대한 텍스처 좌표 쌍이 주어지면, 실제로 다음과 같은 구문을 사용하여 텍스처를 샘플링합니다:

// 비숫자 값은 cbuffer에 추가할 수 없습니다. Texture2D gDiffuseMap;

```
SamplerState samAnisotropic
{
    Filter = ANISOTROPIC;
    MaxAnisotropy = 4;
};

struct VertexOut
{
    float4 PosH : SV_POSITION; float3 PosW :
        POSITION; float3 NormalW :
        NORMAL;
    float2 텍스처 : TEXCOORD;
};

float4 PS(VertexOut pin, uniform int gLightCount) : SV_Target
{
    float4 texColor = gDiffuseMap.Sample(samAnisotropic, pin.Tex);
    ...
}
```

보시다시피, 텍스처를 샘플링하기 위해 **Texture2D::Sample** 메서드를 사용합니다. 첫 번째 매개변수로 **SamplerState** 객체를 전달하고, 두 번째 매개변수로 픽셀의 (u, v) 텍스처 좌표를 전달합니다. 이 메서드는 **SamplerState** 객체에 지정된 필터링 방법을 사용하여 지정된 (u, v) 지점에서 텍스처 맵의 보간된 색상을 반환합니다.

*HLSL 유형 **SamplerState**는 **ID3D11SamplerState** 인터페이스를 반영합니다. 샘플러 상태는 애플리케이션 수준()에서도 설정할 수 있습니다. 레벨에서도 설정할 수 있습니다. 에서 **ID3DX11EffectSamplerVariable::SetSampler**를 사용하여 설정할 수도 있습니다. **D3D11_SAMPLER_DESC** 및 **ID3D11Device::CreateSamplerState**를 참조하십시오. 렌더 상태와 마찬가지로 샘플러 상태는 초기화 시점에 생성해야 합니다.*

8.6 텍스처와 머티리얼

텍스처를 재료/조명 시스템에 통합하기 위해, 텍스처 색상을 주변광 및 확산광 향으로 변조하는 것은 일반적이지만, 반사광 향으로는 변조하지 않습니다(이는 흔히 "낮은 추가로 변조"라고 불림):

```
// 낮게 추가로 변조.
litColor = texColor*(ambient + diffuse) + spec;
```

이러한 수정 방식은 픽셀 단위의 앰비언트 및 디퓨즈 머티리얼 값을 제공하며, 이는 일반적으로 하나의 삼각형을 여러 텍셀이 커버하기 때문에 오브젝트 단위 머티리얼보다 더 높은 해상도를 제공합니다. 즉, 각 픽셀은 보간된 텍스처 좌표(u, v)를 얻게 되며, 이 텍스처 좌표는 해당 픽셀의 머티리얼 표현에 기여하는 색상을 얻기 위해 텍스처를 샘플링하는 데 사용됩니다.

8.7 크레이트 데모

이제 큐브에 크레이트 텍스처를 추가하는 주요 포인트를 살펴보겠습니다([그림 8.1](#) 참조).

8.7.1 텍스처 좌표 지정

GeometryGenerator::CreateBox는 텍스처 이미지가 큐브의 각 면에 매핑되도록 텍스처 좌표를 생성합니다. 간결함을 위해 앞면, 뒷면, 윗면의 정점 정의만 보여줍니다. 또한 생략된 부분을 유의하십시오.

버텍스 생성자에서 법선 벡터와 접선 벡터의 좌표 (텍스처 좌표는 굵게 표시됨).

```
void GeometryGenerator::CreateBox(float width, float height, float depth, MeshData& meshData)
{
    Vertex v[24];

    float w2 = 0.5f*width; float h2 = 0.5f*height; float d2 = 0.5f*depth;

    // 정면 면의 정점 데이터 채우기.
    v[0] = Vertex(-w2, -h2, -d2, ..., 0.0f, 1.0f);
    v[1] = Vertex(-w2, +h2, -d2, ..., 0.0f, 0.0f);
    v[2] = Vertex(+w2, +h2, -d2, ..., 1.0f, 0.0f);
    v[3] = Vertex(+w2, -h2, -d2, ..., 1.0f, 1.0f);

    // 뒷면 정점 데이터를 채웁니다.
    v[4] = Vertex(-w2, -h2, +d2, ..., 1.0f, 1.0f);
    v[5] = Vertex(+w2, -h2, +d2, ..., 0.0f, 1.0f);
    v[6] = Vertex(+w2, +h2, +d2, ..., 0.0f, 0.0f);
    v[7] = Vertex(-w2, +h2, +d2, ..., 1.0f, 0.0f);

    // 상단 면 정점 데이터 입력.
    v[8] = Vertex(-w2, +h2, -d2, ..., 0.0f, 1.0f);
    v[9] = Vertex(-w2, +h2, +d2, ..., 0.0f, 0.0f);
    v[10] = Vertex(+w2, +h2, +d2, ..., 1.0f, 0.0f);
    v[11] = Vertex(+w2, +h2, -d2, ..., 1.0f, 1.0f);
```

텍스처 좌표가 왜 이런 식으로 지정되는지 이해하는 데 도움이 필요하다면 [그림 8.3](#)을 다시 참조하십시오.

8.7.2 텍스처 생성

초기화 시점에 파일(기술적으로는 텍스처에 대한 셰이더 리소스 뷰)로부터 텍스처를 다음과 같이 생성합니다:

```
// CrateApp 데이터 멤버 ID3D11ShaderResourceView*
mDiffuseMapSRV;

bool CrateApp::Init()
{
    if(!D3DApp::Init()) return false;

    // 입력 레이아웃이 셰이더 시그니처에 의존하므로 먼저 이펙트를 초기화해야 함

    // 셰이더 시그니처에 의존합니다.
    Effects::InitAll(md3dDevice);
    InputLayouts::InitAll(md3dDevice);

    HR(D3DX11Create ShaderResource ViewFromFile(md3dDevice, L"Textures/WoodCrate
01.dds", 0, 0, &mDiffuse MapSRV, 0));

    BuildGeometryBuffers(); return true;
}
```

8.7.3 텍스처 설정

텍스처 데이터는 일반적으로 픽셀 셰이더에서 액세스됩니다. 픽셀 셰이더가 이를 액세스하려면 .fx 파일에서 텍스처 뷰(**ID3D11ShaderResourceView**)를 Texture2D 객체로 설정해야 합니다. 이는 다음과 같이 수행됩니다:

```
// BasicEffect의 멤버. ID3DX11EffectShaderResourceVariable* DiffuseMap;
```

```
// 이펙트 파일 변수에 대한 포인터 획득.
DiffuseMap = mFX->GetVariableByName("gDiffuseMap")->AsShaderResource();

void BasicEffect::SetDiffuseMap(ID3D11ShaderResourceView* tex)
{
    DiffuseMap->SetResource(tex);
}

// [.FX 코드]

// 이펙트 파일 텍스처 변수. Texture2D
gDiffuseMap;
```

8.7.4 업데이트된 기본 효과

아래는 텍스처링을 지원하는 수정된 *Basic.fx* 파일입니다(텍스처링 코드는 굵게 표시됨):

```
//=====

// Frank Luna의 Basic.fx (C) 2011 All Rights Reserved.
//
// 현재 변환, 조명,
// 텍스처링을 지원하는 기본 효과입니다.
//=====

#include "LightHelper.fx" cbuffer

cbPerFrame
{
    DirectionalLight gDirLights[3]; float3
    gEyePosW;

    float gFogStart; float
    gFogRange; float4
    gFogColor;
};

cbuffer cbPerObject
{
    float4x4 gWorld;
    float4x4 gWorldInvTranspose; float4x4
    gWorldViewProj; float4x4 gTexTransform;
    Material gMaterial;
};

// 비수치 값은 cbuffer에 추가할 수 없습니다. Texture 2D gDiffuse Map;

샘플러 상태 samAnisotropic
{
    Filter = ANISOTROPIC;
    MaxAnisotropy = 4;

    AddressU = WRAP; AddressV
    = WRAP;
};

struct VertexIn
{
    float3 PosL : POSITION; float3
    NormalL : NORMAL; float2 Tex :
    TEXCOORD;
};
```

```

struct VertexOut
{
    float4 PosH      : SV_POSITION; float3
    PosW            : POSITION; float3
    NormalW : NORMAL; float2 Tex      :
    TEXCOORD;
};

VertexOut VS(VertexIn vin)
{
    VertexOut vout;

    // 월드 공간으로 변환.
    vout.PosW = mul(float4(vin.PosL, 1.0f), gWorld).xyz; vout.NormalW = mul(vin.NormalL,
(float3x3)gWorldInvTranspose);

    // 동차 클립 공간으로 변환.
    vout.PosH = mul(float4(vin.PosL, 1.0f), gWorldViewProj);

    // 삼각형 간 보간을 위한 정점 속성 출력. vout.Tex = mul(float4(vin.Tex, 0.0f, 1.0f),
gTexTransform).xy;

    return vout;
}

float4 PS(VertexOut pin, uniform int gLightCount, uniform bool gUse Texture) : SV_Target
{
    // 보간된 법선은 비정규화될 수 있으므로 정규화합니다. pin.NormalW =
    normalize(pin.NormalW);

    // toEye 벡터는 라이팅에 사용됩니다. float3 toEye =
    gEyePosW - pin.PosW;

    // 이 표면 점에서 눈까지의 거리를 캐시합니다. float distToEye = length(toEye);

    // 정규화합니다.
    toEye /= distToEye;

    // 기본값으로 곱셈 항등원 사용. float4 texColor =
    float4(1, 1, 1, 1); if(gUseTexture)
    {
        // 텍스처 샘플링.
        텍스처 색상 = gDiffuseMap.샘플(samAnisotropic, pin.Tex);
    }

    //
    // 조명.
    //

    float4 litColor = texColor; if(gLightCount > 0)
    {
        // 합계를 0으로 시작합니다.
        float4 ambient = float4(0.0f, 0.0f, 0.0f, 0.0f); float4 diffuse =
        float4(0.0f, 0.0f, 0.0f, 0.0f); float4 spec = float4(0.0f, 0.0f, 0.0f,
        0.0f);

        // 각 광원의 빛 기여도를 합산합니다. [unroll]
        for(int i = 0; i < gLightCount; ++i)
        {

```

```

        float4 A, D, S; ComputeDirectionalLight(gMaterial, gDirLights[i],
pin.NormalW, toEye,
        A, D, S);
        ambient += A;
        diffuse += D; spec +=
        S;
    }
    // 후기 합성으로 변조.
    litColor = texColor*(ambient + diffuse) + spec;
}

// 디퓨즈 머티리얼과 텍스처에서 알파를 가져오는 일반적인 방법. litColor.a = gMaterial.Diffuse.a *
texColor.a;

return litColor;
}

technique11 Light1
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS())); SetGeometryShader(NULL);
        SetPixelShader(CompileShader(ps_5_0, PS(1, false)));
    }
}

technique11 Light2
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS())); SetGeometryShader(NULL);
        SetPixelShader(CompileShader(ps_5_0, PS(2, false)));
    }
}

technique11 Light3
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS())); SetGeometryShader(NULL);
        SetPixelShader(CompileShader(ps_5_0, PS(3, false)));
    }
}

technique11 Light0Tex
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS())); SetGeometryShader(NULL);
        SetPixelShader(CompileShader(ps_5_0, PS(0, true)));
    }
}

technique11 Light1Tex
{
    P0 통과
    {

```

```
        SetVertexShader(CompileShader(vs_5_0, VS())); SetGeometryShader(NULL);
        SetPixelShader(CompileShader(ps_5_0, PS(1, true)));
    }
}

technique11 Light2Tex
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS())); SetGeometryShader(NULL);
        SetPixelShader(CompileShader(ps_5_0, PS(2, true)));
    }
}

technique11 Light3Tex
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS())); SetGeometryShader(NULL);
        SetPixelShader(CompileShader(ps_5_0, PS(3, true)));
    }
}
```

*Basic.fx*는 균일 매개변수 **gUseTexture**를 사용하여 텍스처링이 포함된 테크닉과 포함되지 않은 테크닉을 제공합니다. 따라서 텍스처링이 필요 없는 요소를 렌더링해야 할 경우 텍스처링이 포함되지 않은 테크닉을 선택함으로써 텍스처링 비용을 지불하지 않아도 됩니다. 마찬가지로, 사용하는 조명 수에 따라 테크닉을 선택함으로써 필요 없는 추가 조명 계산 비용을 지불하지 않도록 합니다.

아직 설명하지 않은 상수 버퍼 변수 **gTexTransform0**이 있습니다. 이 변수는 버텍스 셰이더에서 입력 텍스처 좌표를 변환하는 데 사용됩니다:

```
vout.Tex = mul(float4(vin.Tex, 0.0f, 1.0f), gTexTransform).xy;
```

텍스처 좌표는 텍스처 평면상의 2차원 점입니다. 따라서 다른 점과 마찬가지로 이들을 이동, 회전, 크기 조정할 수 있습니다. 이 데모에서는 입력 텍스처 좌표가 변경되지 않도록 단위 행렬 변환을 사용합니다. 그러나 §8.9에서 살펴보겠지만, 텍스처 좌표를 변환함으로써 일부 특수 효과를 얻을 수 있습니다. 2차원 텍스처 좌표를 4×4 행렬로 변환하려면 이를 4차원 벡터로 확장해야 합니다:

```
vin.Tex ----> float4(vin.Tex, 0.0f, 1.0f)
```

곱셈이 완료된 후, 결과 4차원 벡터는 z- 및 w-성분을 제거하여 2차원 벡터로 다시 캐스팅됩니다.

즉,

```
vout.Tex = mul(float4(vin.Tex, 0.0f, 1.0f), gTexTransform).xy;
```

8.8 주소 모드

텍스처는 상수 또는 선형 구간과 결합되어 벡터 값 함수 $T(u, v) = (r, g, b, a)$ 를 정의합니다. 즉, 주어진

텍스처 좌표 $(u, v) \in [0, 1]^2$ 에 대해 텍스처 함수 T 는 색상 (r, g, b, a) 를 반환합니다. Direct3D는 이 함수의 정의역을 네 가지 다른 방식(*주소 모드라고 함*)으로 확장할 수 있게 합니다: *랩(Wrap)*, *보더 컬러(Border Color)*, *클램프(Clamp)*, *미러(Mirror)*.

- wrap*은 정수 경계마다 이미지를 반복하여 텍스처 함수를 확장합니다([그림 8.10](#) 참조).
- border color*는 $[0, 1]^2$ 에 속하지 않는 각 (u, v) 를 프로그래머가 지정한 특정 색상으로 매핑하여 텍스처 함수를 확장합니다([그림 8.11](#) 참조).

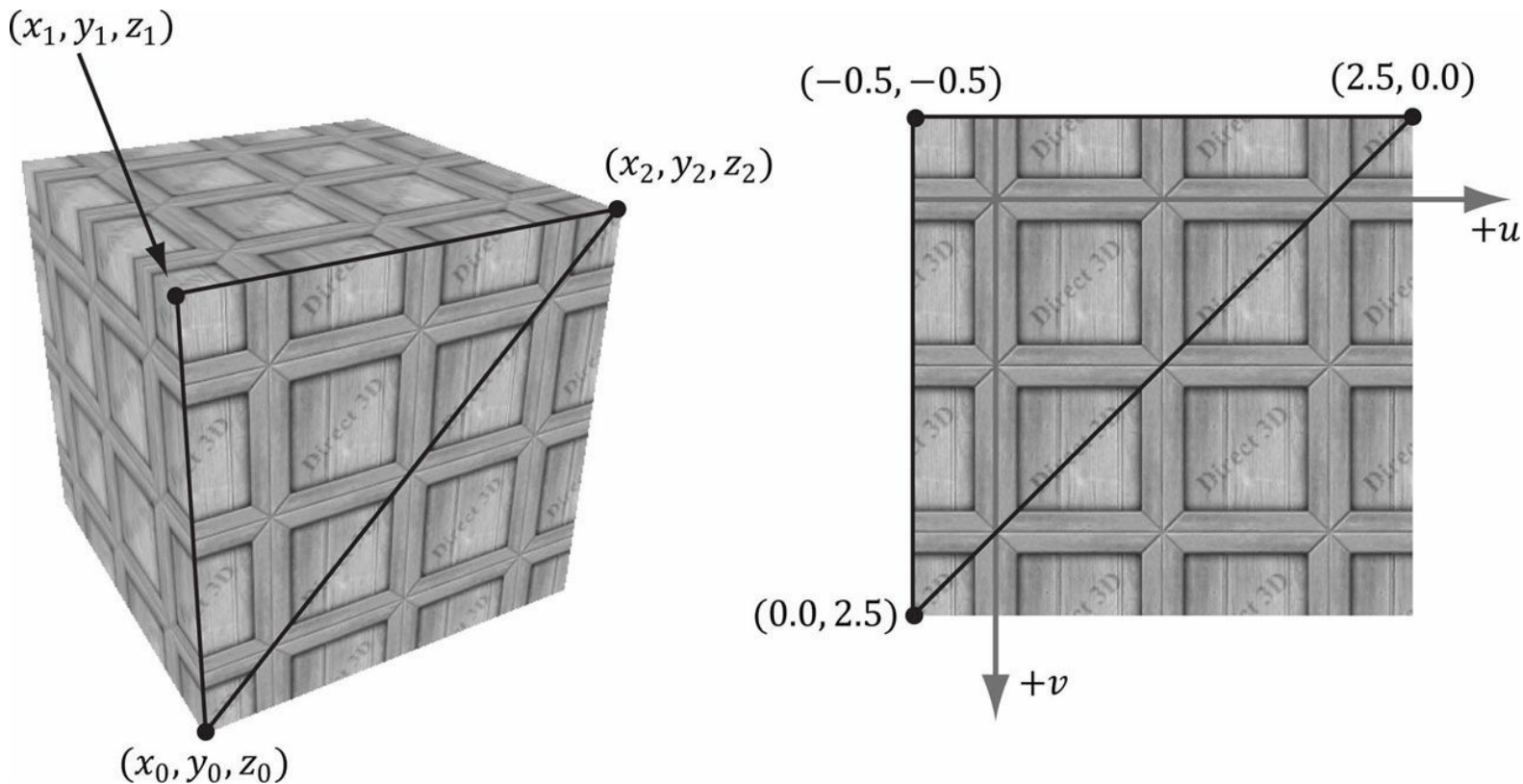


그림 8.10. 랩 주소 모드.

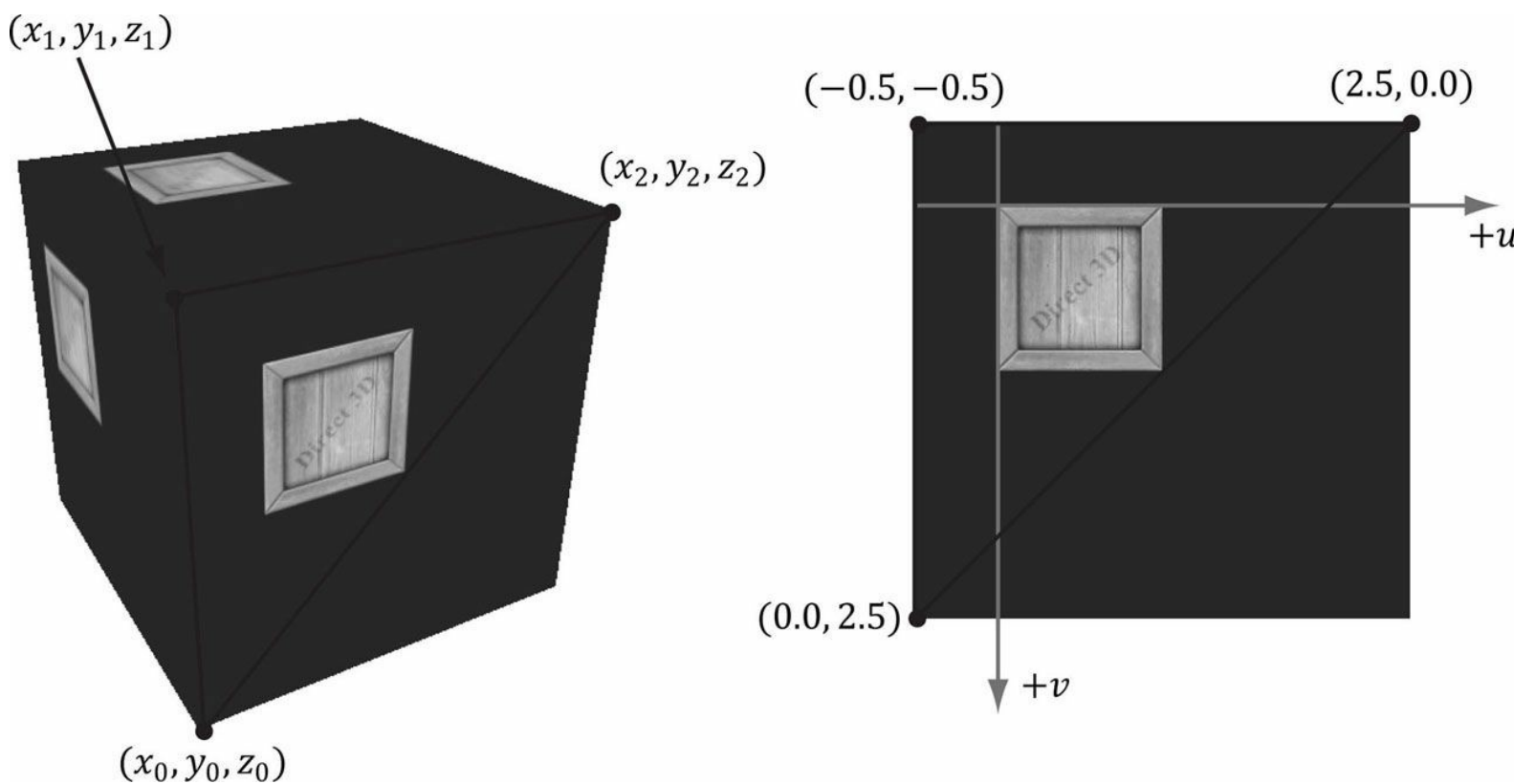


그림 8.11. 경계색 주소 모드.

3. 클램프는 텍스처 함수를 확장하여 $[0, 1]^2$ 에 속하지 않는 각 (u, v) 를 색상 $T(u_0, v_0)$ 로 매핑합니다. 여기서 (u_0, v_0) 는 $[0, 1]^2$ 에 포함된 (u, v) 에 가장 가까운 점입니다(그림 8.12 참조).

4. mirror는 모든 정수 경계점에서 이미지를 미러링하여 텍스처 함수를 확장합니다(그림 8.13 참조).

항상 주소 모드가 지정되며(기본값은 랩 모드), 따라서 $[0, 1]$ 범위를 벗어난 텍스처 좌표는 항상 정의됩니다.

랩 주소 모드는 아마도 가장 자주 사용되는 방식일 것입니다. 이 모드를 사용하면 텍스처를 특정 표면에 반복적으로 타일링할 수 있습니다. 이는

이는 추가 데이터 없이도 텍스처 해상도를 효과적으로 높여줍니다(비록 추가된 해상도가 반복적이지만). 타일링 시 텍스처가 매끄럽게 이어지는 것이 일반적으로 중요합니다. 예를 들어, 상자 텍스처는 반복이 명확히 보이기 때문에 매끄럽지 않습니다.

반복이 뚜렷이 보입니다. 그러나 그림 8.14는 2×3회 반복된 매끄러운 벽돌 텍스처를 보여줍니다.

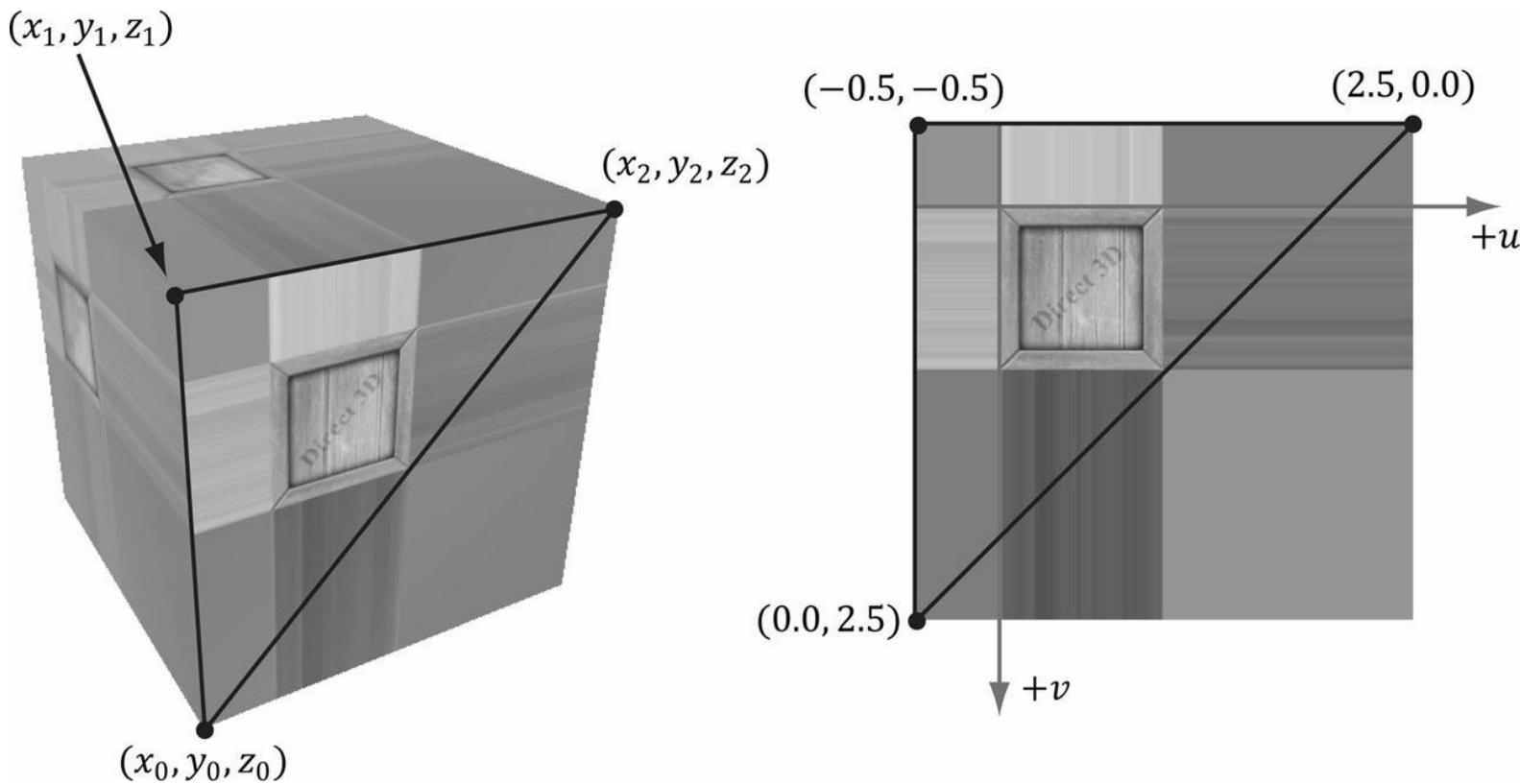


그림 8.12. 클램프 주소 모드.

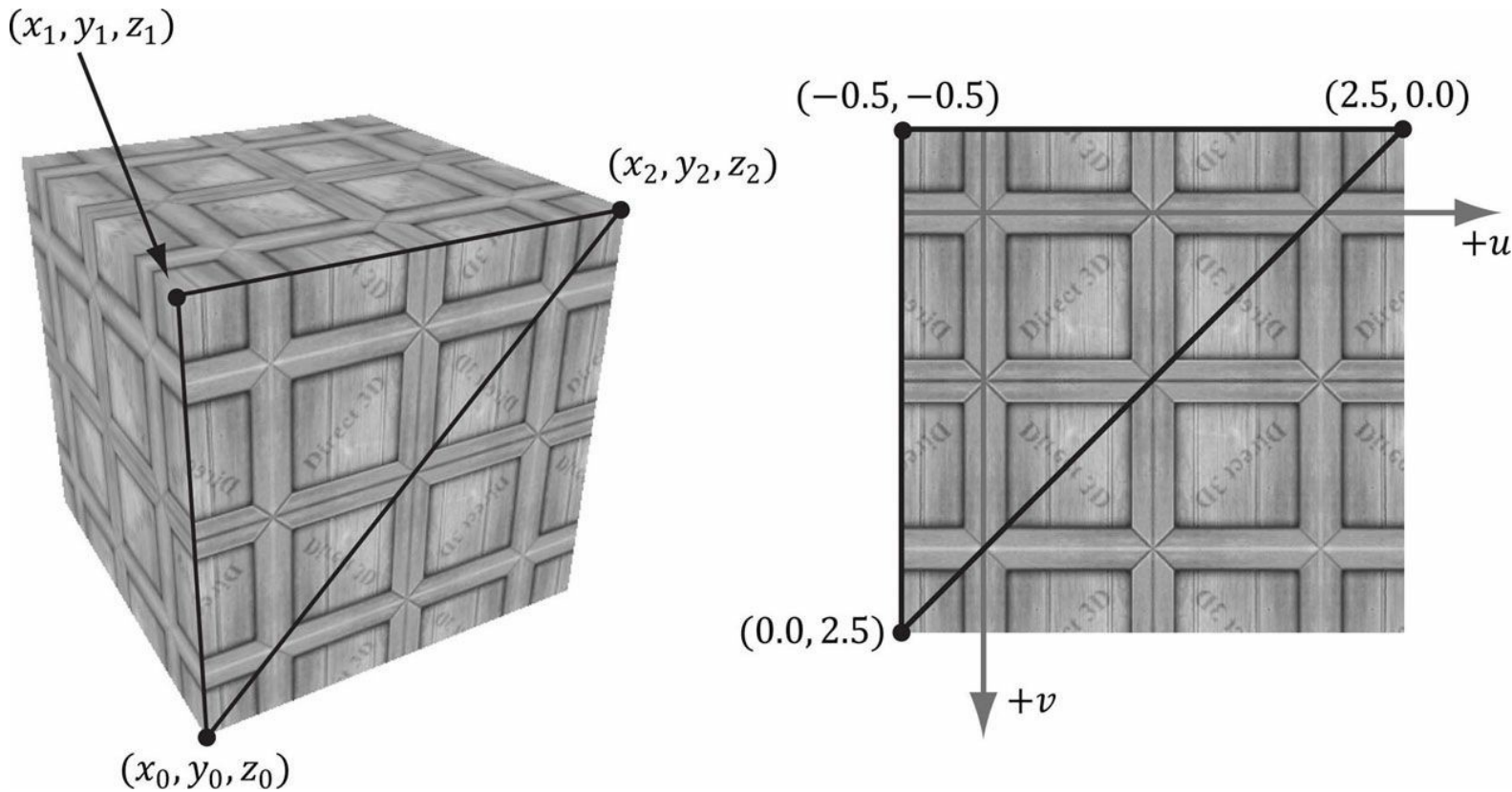


그림 8.13. 미러 주소 모드.

주소 모드는 샘플러 객체에서 지정됩니다. 다음 예제는 그림 8.10–8.13을 생성하는 데 사용되었습니다:

```
SamplerState samTriLinear
{
    Filter = MIN_MAG_MIP_LINEAR; AddressU =
    WRAP;
    AddressV = WRAP;
```

```
};
```

샘플러 상태 samTriLinear

```
{
    필터 = MIN_MAG_MIP_LINEAR; 주소U =
    BORDER;
    AddressV = BORDER;

    // 파란색 테두리 색상
    BorderColor = float4(0.0f, 0.0f, 1.0f, 1.0f);
};
```

샘플러 상태 samTriLinear

```
{
    Filter = MIN_MAG_MIP_LINEAR; AddressU =
    CLAMP;
    AddressV = CLAMP;
};
```

샘플러 상태 samTriLinear

```
{
    필터 = MIN_MAG_MIP_LINEAR; 주소U =
    MIRROR;
    AddressV = MIRROR;
};
```



Tile 2 x 3

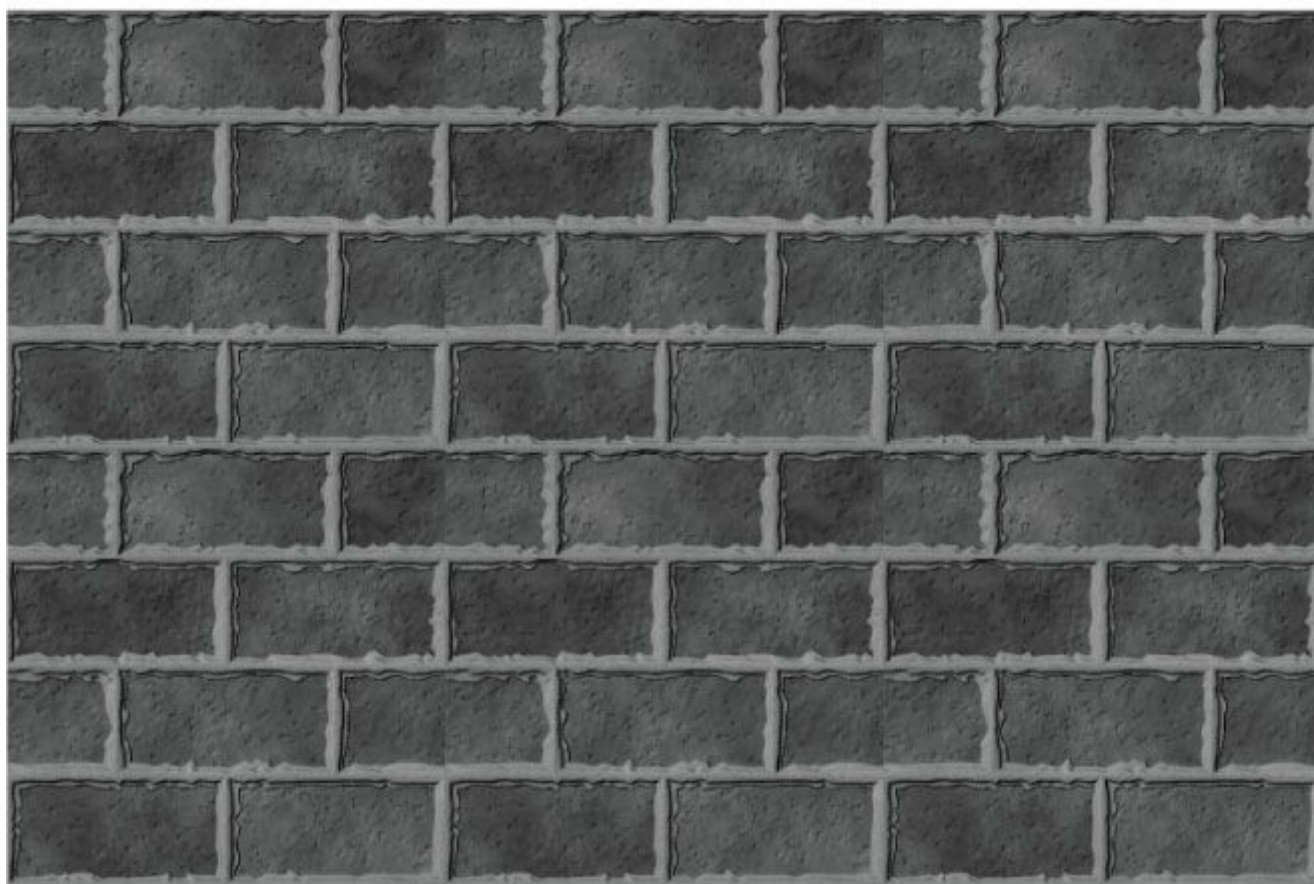


그림 8.14. 2×3 크기로 타일링된 벽돌 텍스처. 텍스처가 이음매 없이 연결되어 반복 패턴이 잘 눈에 띄지 않습니다.

u 축과 v 축 방향의 주소 모드를 독립적으로 제어할 수 있음을 확인하십시오. 독자는  에서 이를 실험해 볼 수 있습니다.

8.9 텍스처 변환

텍스처 좌표는 텍스처 평면상의 2차원 점을 나타냅니다. 따라서 다른 점과 마찬가지로 이동, 회전, 크기 조정이 가능합니다. 텍스처 변형의 몇 가지 활용 예시는 다음과 같습니다:

1. 벽을 따라 벽돌 질감이 펼쳐져 있습니다. 벽의 정점들은 현재 $[0, 1]$ 범위의 질감 좌표를 가지고 있습니다.

텍스처 좌표를 4배로 확대하여 [0, 4] 범위로 조정합니다. 이렇게 하면 벽 전체에 걸쳐 텍스처가 4x4 크기로 반복됩니다.

2. 맑은 푸른 하늘 위에 구름 텍스처가 펼쳐져 있습니다. 시간에 따라 텍스처 좌표를 변환함으로써 구름이 하늘 위를 움직이는 애니메이션이 구현됩니다.

3. 텍스처 회전은 때로 입자 효과에 유용합니다. 예를 들어 불덩이 텍스처를 시간에 따라 회전시키는 경우처럼 말이죠.

텍스처 좌표 변환은 일반적인 변환과 동일한 방식으로 수행됩니다. 변환 행렬을 지정하고 텍스처 좌표 벡터를 해당 행렬로 곱합니다. 예를 들어:

```
// 상수 버퍼 변수 float4x4 gTexMtx;
```

```
// 셰이더 프로그램 내
```

```
vOut.texC = mul(float4(vIn.texC, 0.0f, 1.0f), gTexMtx);
```

2D 텍스처 좌표를 다루고 있으므로, 우리는 첫 번째 두 좌표에 대한 변환만 고려합니다.

예를 들어, 텍스처 행렬이 z 좌표를 이동시킨다면 결과 텍스처 좌표에는 아무런 영향도 미치지 않습니다.

8.10 텍스처 처리된 언덕과 파도 데모

이 데모에서는 육지와 수면 장면에 텍스처를 적용합니다. 첫 번째 핵심 과제는 육지 표면에 잔디 텍스처를 타일링하는 것입니다. 육지 메쉬가 넓은 면적이므로 텍스처를 단순히 늘려 적용하면 각 삼각형에 할당되는 텍셀 수가 너무 적어집니다. 즉, 표면에 충분한 텍스처 해상도가 확보되지 않아 확대 아티팩트가 발생합니다. 따라서 해상도를 높이기 위해 잔디 텍스처를 지형 메시에 반복 적용합니다. 두 번째 핵심은 물 지오메트리에 시간 함수로 물 텍스처를 스크롤 적용하는 것입니다. 이 추가된 움직임이 물을 좀 더 사실적으로 보이게 합니다. [그림 8.15](#)는 데모의 스크린샷입니다.

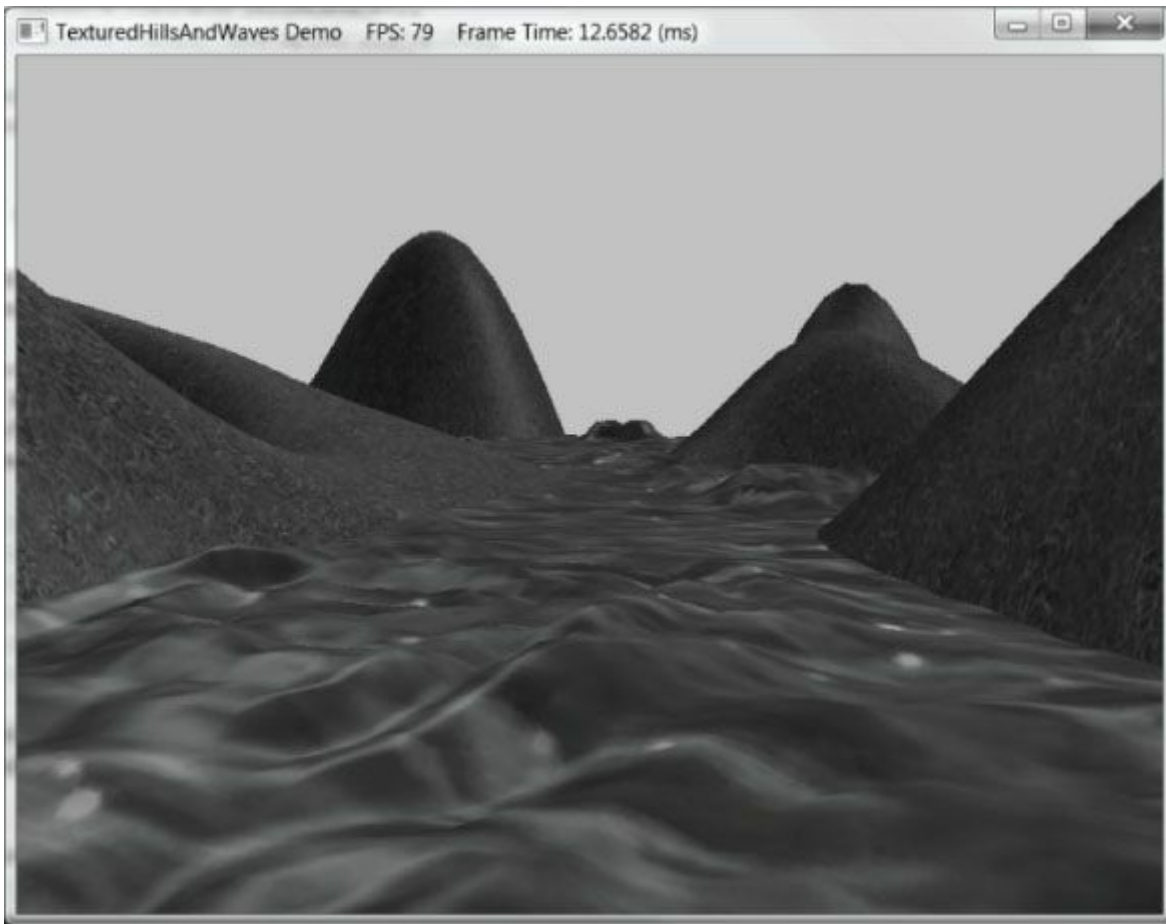


그림 8.15. "Land Tex" 데모의 스크린샷.

8.10.1 그리드 텍스처 좌표 생성

그림 8.16은 xz 평면상의 $m \times n$ 격자와 정규화된 텍스처 공간 영역 $[0, 1]$ 내의 대응 격자를 보여줍니다⁽²⁾. 그림에서 알 수 있듯이, xz 평면상의 ij 번째 격자 정점의 텍스처 좌표는 텍스처 공간 내 ij 번째 격자 정점의 좌표와 일치합니다. ij 번째 정점의 텍스처 공간 좌표는 다음과 같습니다:

$$u_{ij}=j\Delta u\ v_{ij}=$$

$$i\Delta v$$

where $\Delta u = \frac{1}{n-1}$ 그리고 $\Delta v = \frac{1}{m-1}$.

따라서 **GeometryGenerator::CreateGrid** 메서드에서 그리드의 텍스처 좌표를 생성하기 위해 다음 코드를 사용합니다:

void GeometryGenerator::CreateGrid(float width, float depth, UINT m, UINT n, MeshData& meshData)

```
{
    UINT vertexCount = m*n;
    UINT faceCount = (m-1)*(n-1)*2;

    //
    // 정점을 생성합니다.
    //

    float halfWidth = 0.5f * width; float
    halfDepth = 0.5f * depth;

    float dx = width / (n-1); float dz =
    depth / (m-1);

    float du = 1.0f / (n-1); float dv =
    1.0f / (m-1);

    meshData.Vertices.resize(vertexCount); for(UINT i = 0; i < m;
    ++i)
    {
        float z = halfDepth - i*dz; for(UINT
        j = 0; j < n; ++j)
        {
            float x = -halfWidth + j*dx;

            meshData.Vertices[i*n+j].Position = XMFLOAT3(x,
                0.0f, z);
            meshData.Vertices[i*n+j].Normal =
                XMFLOAT3(0.0f, 1.0f, 0.0f);
            meshData.Vertices[i*n+j].TangentU = XMFLOAT3(1.0f, 0.0f,
                0.0f);

            // 그리드에 텍스처를 늘림. meshData.Vertices[i*n+j].TexC.x =
            j*du; meshData.Vertices[i*n+j].TexC.y = i*dv;
        }
    }
}
```

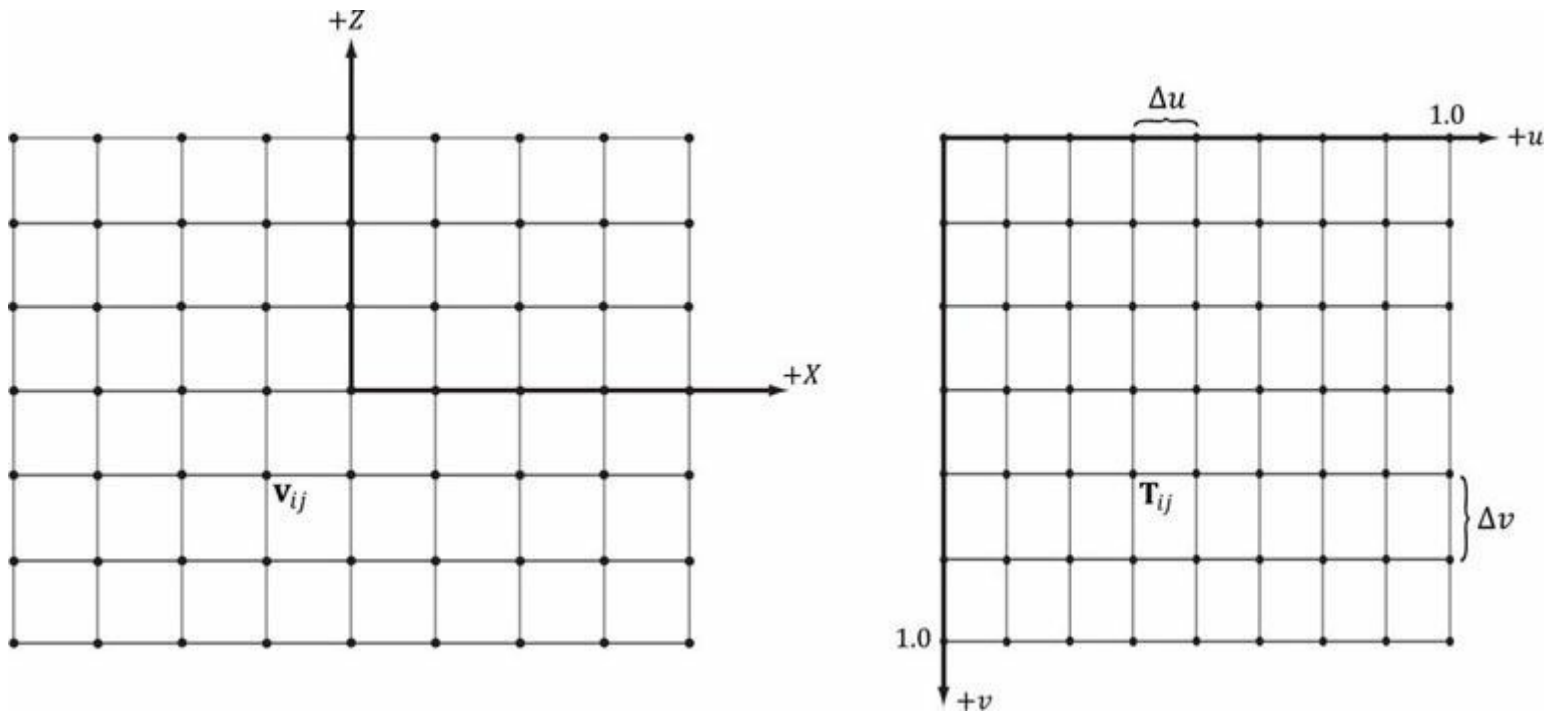


그림 8.16. xz 공간에서 격자 점 v_{ij} 의 텍스처 좌표는 uv 공간에서 i 번째 격자 점 T_{ij} 에 의해 주어집니다.

8.10.2 텍스처 타일링

우리는 지형 메시에 잔디 텍스처를 타일링하고 싶다고 말했습니다. 하지만 지금까지 계산한 텍스처 좌표는 단위 영역 $[0, 1]^{(2)}$ 에 속하므로 타일링이 발생하지 않습니다. 텍스처를 타일링하기 위해, 우리는 랩 주소 모드를 지정하고 텍스처 변환 행렬을 사용하여 텍스처 좌표를 5배로 스케일링합니다. 따라서 텍스처 좌표는 $[0, 5]^2$ 영역으로 매핑되어 텍스처가 지형 메시 표면 전체에 걸쳐 5×5 로 타일링됩니다:

```
XMMATRIX grassTexScale = XMMatrixScaling(5.0f, 5.0f, 0.0f);
XMStoreFloat4x4(&mGrassTexTransform, grassTexScale);
...
Effects::BasicFX->SetTexTransform(XMLoadFloat4x4(&mGrassTexTransform));
...
activeTech->GetPassByIndex(p)->Apply(0, md3dImmediateContext); md3dImmediateContext-
->DrawIndexed(mLandIndexCount, 0, 0);
```

8.10.3 텍스처 애니메이션

물 지오메트리 위에 물 텍스처를 스크롤하기 위해, UpdateScene 메서드에서 시간 함수로 텍스처 평면의 텍스처 좌표를 변환합니다. 각 프레임마다 변화가 작다면, 이는 부드러운 애니메이션 효과를 줍니다. 텍스처 공간 평면 주변에서 텍스처 좌표를 매끄럽게 변환할 수 있도록 랩 주소 모드와 무봉합 텍스처를 함께 사용합니다. 다음 코드는 물 텍스처의 오프셋 벡터를 계산하고 물의 텍스처 매트릭스를 구축 및 설정하는 방법을 보여줍니다:

```
// 물 텍스처 타일링
XMMATRIX wavesScale = XMMatrixScaling(5.0f, 5.0f, 0.0f);

// 시간에 따른 텍스처 이동. mWaterTexOffset.y +=
0.05f*dt; mWaterTexOffset.x += 0.1f*dt;
XMMATRIX wavesOffset = XMMatrixTranslation(mWaterTexOffset.x, mWaterTexOffset.y, 0.0f);

// 스케일과 변환을 결합합니다. XMStoreFloat4x4(&mWaterTexTransform, wavesScale*wavesOffset);

...
Effects::BasicFX->SetTexTransform(XMLoadFloat4x4(&mWaterTexTransform));
...
activeTech->GetPassByIndex(p)->Apply(0, md3dImmediateContext);
```

```
md3dImmediateContext->DrawIndexed(3*mWaves.TriangleCount(), 0, 0);
```

8.11 압축 텍스처 형식

가상 세계가 수백 개의 텍스처로 확장됨에 따라 텍스처에 대한 GPU 메모리 요구 사항은 빠르게 증가합니다(이 모든 텍스처를 신속하게 적용하기 위해 GPU 메모리에 보관해야 함을 기억하십시오). 이러한 메모리 요구 사항을 완화하기 위해 Direct3D는 압축 텍스처 형식인 BC1, BC2, BC3, BC4, BC5, BC6 및 BC7을 지원합니다:

1. BC1 (DXGI_FORMAT_BC1_UNORM): 세 개의 색상 채널과 1비트(on/off) 알파 구성 요소만 지원하는 형식을 압축해야 할 때 사용합니다.
2. BC2 (DXGI_FORMAT_BC2_UNORM): 3개의 색상 채널과 4비트 알파 구성 요소만 지원하는 형식을 압축해야 할 때 사용합니다.
3. BC3 (DXGI_FORMAT_BC3_UNORM): 3개의 색상 채널과 8비트 알파 컴포넌트를 지원하는 형식을 압축해야 하는 경우 이 형식을 사용하십시오.
4. BC4 (DXGI_FORMAT_BC4_UNORM): 하나의 색상 채널(예: 그레이스케일 이미지)을 포함하는 형식을 압축해야 하는 경우 이 형식을 사용하십시오.
5. BC5 (DXGI_FORMAT_BC5_UNORM): 두 개의 색상 채널을 지원하는 형식을 압축해야 하는 경우 이 형식을 사용하십시오.
6. BC6 (DXGI_FORMAT_BC6_UF16): 압축된 HDR(High Dynamic Range) 이미지 데이터에 이 형식을 사용하십시오.
7. BC7 (DXGI_FORMAT_BC7_UNORM): 고품질 RGBA 압축에 이 형식을 사용하십시오. 특히 이 형식은 노멀 맵 압축으로 인한 오류를 크게 줄여줍니다.

 압축된 텍스처는 렌더링 파이프라인의 셰이더 단계에 대한 입력으로만 사용할 수 있습니다.

 블록 압축 알고리즘은 4×4 픽셀 블록 단위로 작동하므로 텍스처의 크기는 4의 배수여야 합니다.

이러한 형식의 장점은 GPU 메모리에 압축된 상태로 저장할 수 있으며, 필요할 때 GPU가 즉시 압축을 해제할 수 있다는 점입니다. 압축되지 않은 이미지 데이터가 포함된 파일이 있는 경우, 로드 시에 Direct3D가 압축 형식으로 변환하도록 할 수 있습니다.

예를 들어, BMP 파일을 로드하는 다음 코드를 살펴보십시오.

```
D3DX11_IMAGE_LOAD_INFO loadInfo; loadInfo.Format =
DXGI_FORMAT_BC3_UNORM;

HR(D3DX11CreateShaderResourceViewFromFile(md3dDevice, L"Textures/darkbrick.bmp", &loadInfo,
0, &mDiffuseMapSRV, 0));

// 리소스 뷰에서 실제 2D 텍스처를 가져옵니다. ID3D11Texture2D* tex;
mDiffuseMapSRV->GetResource((ID3D11Resource**)&tex);

// 2D 텍스처의 설명을 가져옵니다. D3D11_TEXTURE2D_DESC
texDesc;
tex->GetDesc(&texDesc);
```

그림 8.17a는 디버거에서 texDesc가 어떻게 보이는지 보여줍니다. 원하는 압축 텍스처 형식을 사용하고 있음을 확인할 수 있습니다. 대신 pLoadInfo 매개변수에 null을 지정했다면, 소스 이미지의 형식이 사용됩니다(그림 8.17b). 이는 압축되지 않은 DXGI_FORMAT_R8G8B8A8_UNORM 형식입니다.

또는 압축 텍스처를 직접 저장할 수 있는 DDS(Direct Draw Surface) 형식을 사용할 수도 있습니다. 이를 위해서는

이미지 파일을 SDK 디렉터리(D:\Microsoft DirectX SDK (June 2010)\Utilities\bin\x86)에 있는 DirectX 텍스처 도구(DxTex.exe)로 불러옵니다. 그런 다음 메뉴->포맷->표면 포맷 변경으로 이동하여 DXT1, DXT2, DXT3, DXT4 또는 DXT5를 선택합니다. 파일을 DDS 파일로 저장합니다. 이 형식들은 실제로 Direct3D 9 압축 텍스처 형식이지만, DXT1은 BC1과 동일하고, DXT2와 DXT3은 BC2와 동일하며, DXT4와 DXT5는 BC3과 동일합니다. 예를 들어, 파일을 DXT1로 저장하고 D3DX11CreateShaderResourceViewFromFile을 사용하여 로드하면 텍스처의 형식은 DXGI_FORMAT_BC1_UNORM이 됩니다:

```
HR(D3DX11CreateShaderResourceViewFromFile(md3dDevice, L"Textures/darkbrickdxt1.dds", 0, 0,
&mDiffuseMapSRV, 0));
```



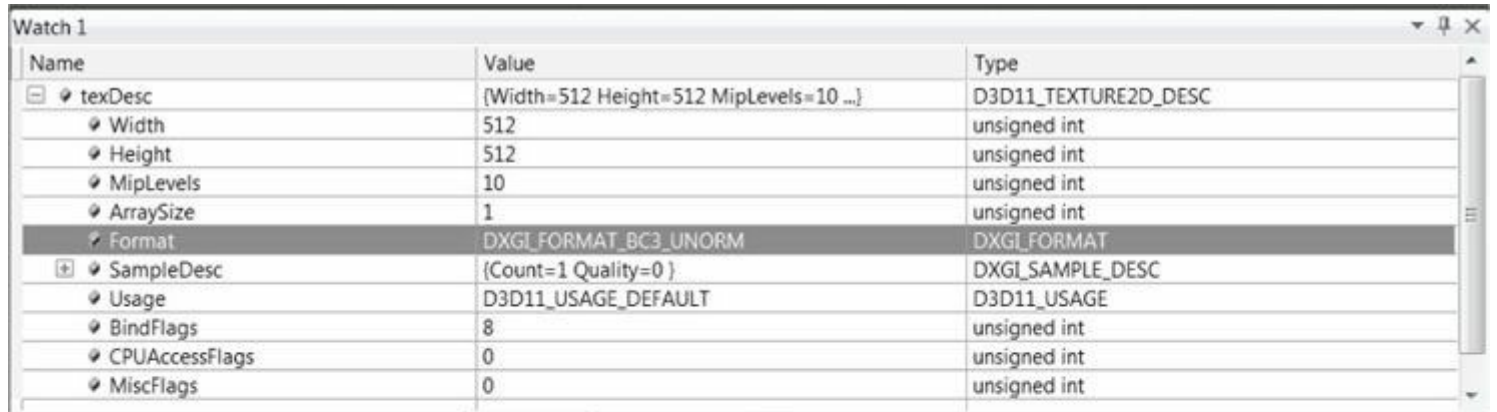
```
// 리소스 뷰에서 실제 2D 텍스처를 가져옵니다. ID3D11Texture2D* tex;
```

```
mDiffuseMapSRV->GetResource((ID3D11Resource**)&tex);
```

```
// 2D 텍스처의 설명을 가져옵니다. D3D11_TEXTURE2D_DESC
```

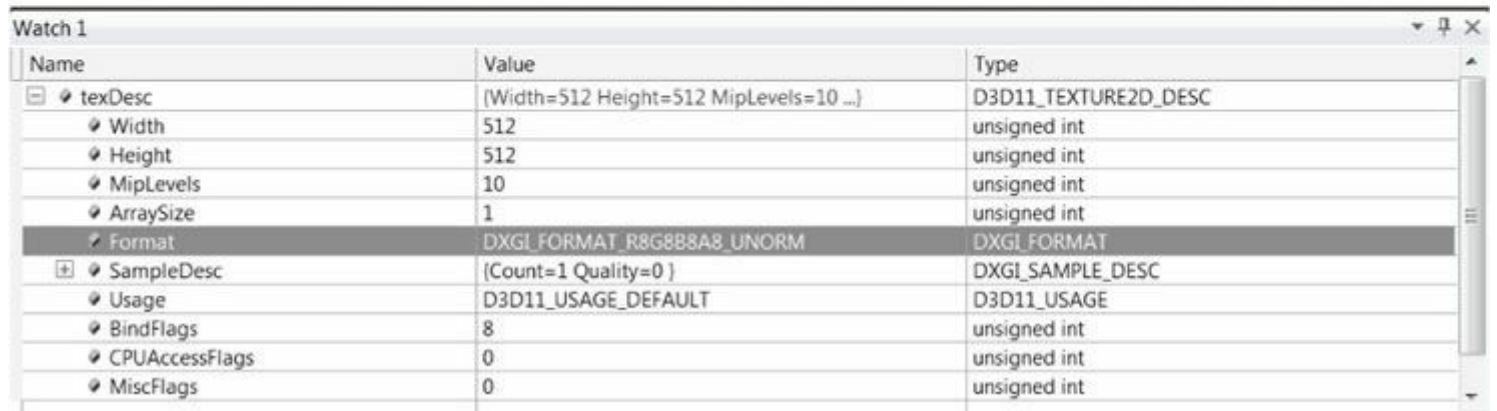
```
texDesc;
```

```
tex->GetDesc(&texDesc);
```



Name	Value	Type
texDesc	{Width=512 Height=512 MipLevels=10 ...}	D3D11_TEXTURE2D_DESC
Width	512	unsigned int
Height	512	unsigned int
MipLevels	10	unsigned int
ArraySize	1	unsigned int
Format	DXGI_FORMAT_BC3_UNORM	DXGI_FORMAT
SampleDesc	{Count=1 Quality=0 }	DXGI_SAMPLE_DESC
Usage	D3D11_USAGE_DEFAULT	D3D11_USAGE
BindFlags	8	unsigned int
CPUAccessFlags	0	unsigned int
MiscFlags	0	unsigned int

(a)



Name	Value	Type
texDesc	{Width=512 Height=512 MipLevels=10 ...}	D3D11_TEXTURE2D_DESC
Width	512	unsigned int
Height	512	unsigned int
MipLevels	10	unsigned int
ArraySize	1	unsigned int
Format	DXGI_FORMAT_R8G8B8A8_UNORM	DXGI_FORMAT
SampleDesc	{Count=1 Quality=0 }	DXGI_SAMPLE_DESC
Usage	D3D11_USAGE_DEFAULT	D3D11_USAGE
BindFlags	8	unsigned int
CPUAccessFlags	0	unsigned int
MiscFlags	0	unsigned int

(b)

그림 8.17. (a) 텍스처는 DXGI_FORMAT_BC3_UNORM 압축 형식으로 생성됩니다. (b) 텍스처는

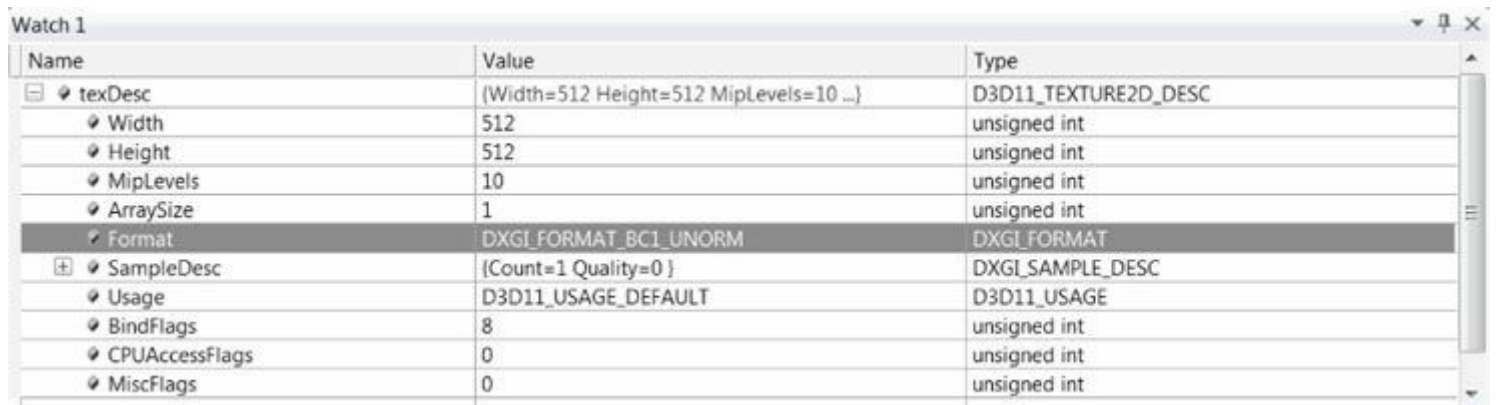
DXGI_FORMAT_R8G8B8A8_UNOR 무압축 형식으로 생성된 텍스처.

DDS 파일이 압축 형식 중 하나를 사용하는 경우 pLoadInfo 매개변수에 null을 지정할 수 있으며,

D3DX11CreateShaderResourceViewFromFile는 파일에 지정된 압축 형식을 사용합니다.

BC4 및 BC5 형식의 경우 NVIDIA 텍스처 도구(<http://code.google.com/p/nvidia-texture-tools/>)를 사용할 수 있습니다. BC6 및

BC7 형식의 경우, DirectX SDK에는 "BC6HBC7EncoderDecoder11"이라는 샘플이 있습니다. 이 프로그램을 사용하여 텍스처 파일을 BC6 또는 BC7로 변환할 수 있습니다. 이 샘플에는 전체 소스 코드가 포함되어 있어 자체 아트 파이프라인에 통합할 수 있습니다. 흥미롭게도, 이 샘플은 그래픽 하드웨어가 컴퓨터 셰이더를 지원하는 경우 GPU를 사용하여 변환을 수행합니다. 이는 CPU 구현보다 훨씬 빠른 변환 속도를 제공합니다.



Name	Value	Type
texDesc	{Width=512 Height=512 MipLevels=10 ...}	D3D11_TEXTURE2D_DESC
Width	512	unsigned int
Height	512	unsigned int
MipLevels	10	unsigned int
ArraySize	1	unsigned int
Format	DXGI_FORMAT_BC1_UNORM	DXGI_FORMAT
SampleDesc	{Count=1 Quality=0 }	DXGI_SAMPLE_DESC
Usage	D3D11_USAGE_DEFAULT	D3D11_USAGE
BindFlags	8	unsigned int
CPUAccessFlags	0	unsigned int
MiscFlags	0	unsigned int

그림 8.18. 텍스처는 DXGI_FORMAT_BC1_UNORM 형식으로 생성됩니다.

DirectX 텍스처 도구에서 맵 레벨을 생성(메뉴->포맷->맵 생성)하고

DDS 파일로 저장할 수도 있습니다. 이렇게 하면 맵 레벨이 미리 계산되어 파일과 함께 저장되므로 로드 시점에 계산할 필요가 없습니다(단순히 로드만 하면 됩니다).
텍스처를 DDS 파일로 압축 저장하는 또 다른 장점은 디스크 공간도 덜 차지한다는 점입니다.

8.12 요약

1. 텍스처 좌표는 3D 삼각형에 매핑되는 텍스처상의 삼각형을 정의하는 데 사용됩니다.
2. `D3DX11CreateShaderResourceViewFromFile` 함수를 사용하여 디스크에 저장된 이미지 파일에서 텍스처를 생성할 수 있습니다.
3. 축소, 확대 및 mip 맵 필터 샘플러 상태를 사용하여 텍스처를 필터링할 수 있습니다.
4. 주소 모드는 $[0, 1]$ 범위를 벗어난 텍스처 좌표에 대해 Direct3D가 수행해야 할 작업을 정의합니다. 예를 들어, 텍스처를 타일링, 미러링, 클램핑해야 할까요?
5. 텍스처 좌표는 다른 점들처럼 스케일링, 회전, 변환될 수 있습니다. 프레임마다 텍스처 좌표를 조금씩 변환함으로써 텍스처를 애니메이션화할 수 있습니다.
6. 압축된 Direct3D 텍스처 형식인 BC1, BC2, BC3, BC4, BC5, BC6 또는 BC7을 사용하면 상당한 양의 GPU 메모리를 절약할 수 있습니다. DirectX 텍스처 도구를 사용하여 BC1, BC2, BC3 형식의 텍스처를 생성하십시오. BC4 및 BC5의 경우 NVIDIA 텍스처 도구(<http://code.google.com/p/nvidia-texture-tools/>)를 사용할 수 있습니다. SDK의 "BC6HBC7EncoderDecoder11" 샘플을 사용하여 BC6 및 BC7 형식의 텍스처를 생성하십시오.

8.13 연습 문제

1. 텍스처 좌표를 변경하고 다양한 주소 모드 조합 및 필터링 옵션을 사용하여 "Crate" 데모를 실험해 보십시오. 특히 그림 8.7, 8.9, 8.10, 8.11, 8.12 및 8.13의 이미지를 재현해 보십시오.
2. DirectX 텍스처 도구를 사용하면 각 mip 맵 레벨을 수동으로 지정할 수 있습니다(**파일->이 표면에 열기**). [그림 8.19](#)와 같은 mip 맵 체인을 가진 DDS 파일을 생성하세요. 각 레벨마다 서로 다른 텍스트 설명이나 색상을 적용하여 각 mip 맵 레벨을 쉽게 구분할 수 있도록 합니다. 이 텍스처를 사용하여 크레이트 데모를 수정하고, 카메라가 확대/축소되어 mip 맵 레벨이 변화하는 것을 명확히 확인할 수 있도록 하십시오. 포인트 mip 맵 필터링과 선형 mip 맵 필터링을 모두 시도해 보십시오.



그림 8.19. 각 레벨을 쉽게 구분할 수 있도록 수동으로 구성한 mip 맵 체인.

3. 동일한 크기의 두 텍스처를 주어진다면, 다양한 연산을 통해 이를 결합하여 새로운 이미지를 얻을 수 있습니다. 보다 일반적으로 이는 *멀티텍스처링*이라 불리며, 여러 텍스처를 활용해 결과를 도출하는 기법입니다. 예를 들어 두 텍스처의 대응 텍셀을 더하거나, 빼거나, (성분별로) 곱할 수 있습니다. [그림 8.20](#)은 두 텍스처를 성분별로 곱해 불덩어리 같은 결과를 얻은 예시입니다. 이 연습에서는 [그림 8.20](#)의 두 소스 텍스처를 픽셀 셰이더에서 결합하여 각 큐브 면 위에 불덩어리 텍스처를 생성하도록 "Crate" 데모를 수정하십시오. (이 연습에 필요한 이미지 파일은 책의 웹사이트에서 다운로드할 수 있습니다.) 하나 이상의 텍스처를 지원하도록 *Basic.fx*를 수정해야 한다는 점에 유의하십시오.

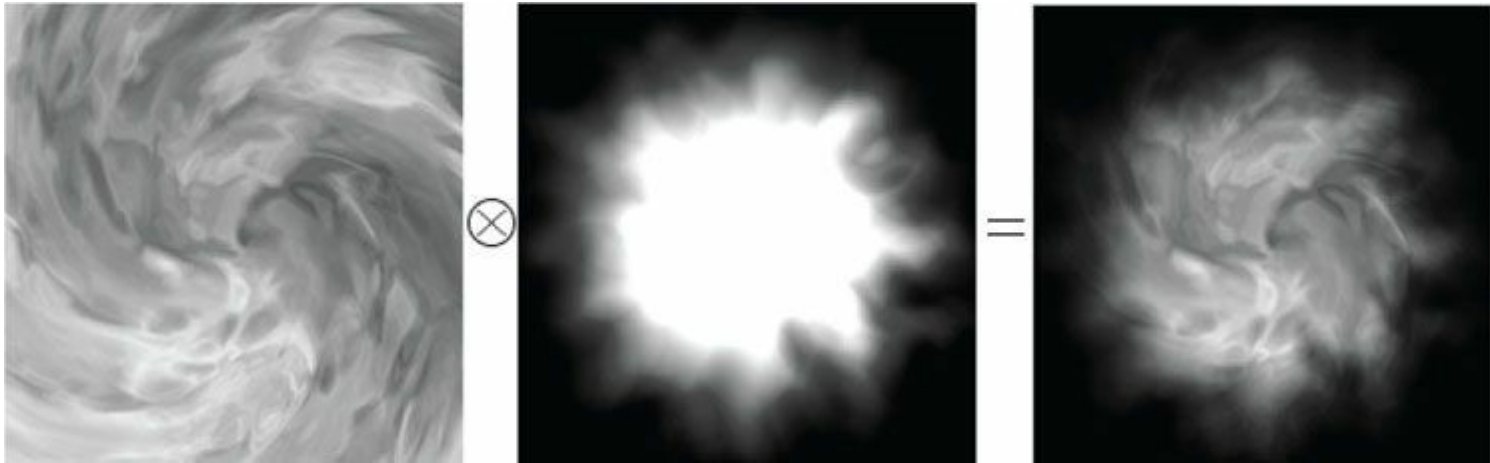


그림 8.20. 두 텍스처의 대응 텍셀을 성분별로 곱하여 새로운 텍스처를 생성하는 과정.

4. 연습문제 3의 솔루션을 수정하여 각 큐브 면에 불덩이 텍스처를 시간 함수로 회전시키십시오.
5. 이 장의 다운로드 디렉토리에는 4초(초당 30프레임) 동안 재생되도록 설계된 120프레임의 불 애니메이션이 담긴 폴더가 포함되어 있습니다. [그림 8.21](#)은 첫 30프레임을 보여줍니다. 이 애니메이션을 각 큐브 면에 재생하도록 "Crate" 데모를 수정하세요. (*힌트:* 이미지를 120개의 텍스처 객체 배열에 로드하세요. 첫 번째 프레임 텍스처를 사용한 후, 1/30초마다 다음 프레임 텍스처로 증가시키세요. 120번째 프레임 텍스처 이후에는 첫 번째 텍스처로 되돌아가 과정을 반복합니다.) 이는 플립북의 페이지를 넘기는 것과 유사하여 때때로 *페이지 플리핑*이라고 불립니다. *참고:* 개별 텍스처 애니메이션 프레임을 하나씩 처리하는 이 방법은 비효율적입니다. 모든 프레임을 하나의 텍스처 아틀라스에 넣고, 1/30초마다 텍스처 좌표를 오프셋하여 애니메이션의 다음 프레임으로 이동하는 것이 더 좋습니다. 하지만 연습을 위해 비효율적인 방법도 괜찮습니다.

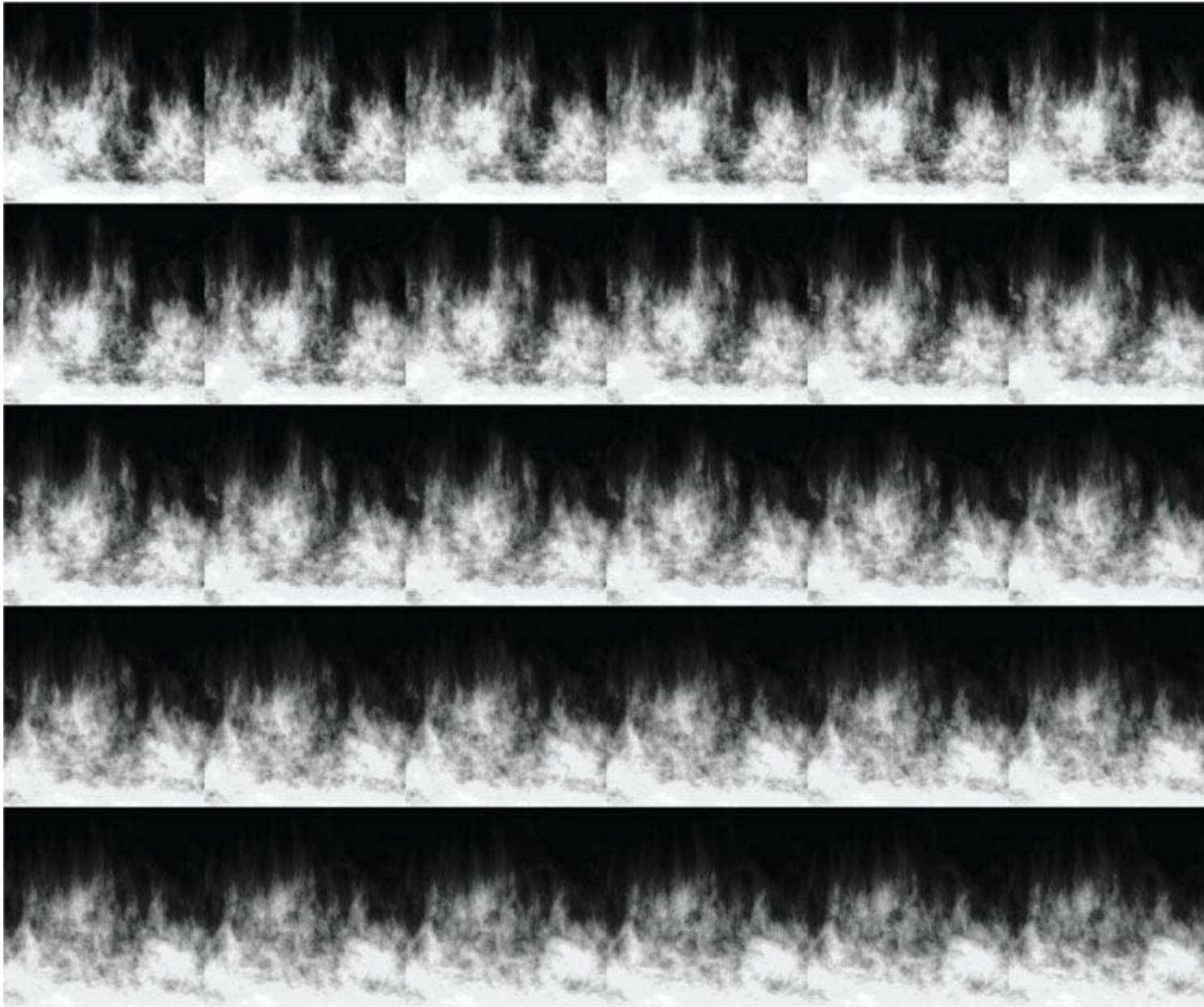


그림 8.21. 사전 계산된 불 애니메이션의 프레임.

6. $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2$ 를 각각 텍스처 좌표 $\mathbf{q}_0, \mathbf{q}_1, \mathbf{q}_2$ 를 가지는 3차원 삼각형의 꼭짓점으로 하자. §8.2에서 언급한 바와 같이, 3차원 삼각형 상의 임의의 점 $\mathbf{p}(s, t) = \mathbf{p}_0 + s(\mathbf{p}_1 - \mathbf{p}_0) + t(\mathbf{p}_2 - \mathbf{p}_0)$ 이며 여기서 $s \geq 0, t \geq 0, s + t \leq 1$ 이다. 이 점의 텍스처 좌표 (u, v) 는 동일한 s, t 매개변수로 삼각형의 꼭짓점 텍스처 좌표를 선형 보간하여 구한다:

$$(u, v) = \mathbf{q}_0 + s(\mathbf{q}_1 - \mathbf{q}_0) + t(\mathbf{q}_2 - \mathbf{q}_0)$$

(u, v) 와 $\mathbf{q}_0, \mathbf{q}_1, \mathbf{q}_2$ 가 주어졌을 때, (s, t) 를 u 와 v 에 대해 구하시오.



벡터 방정식 $(u, v) - \mathbf{q}_0 = s(\mathbf{q}_1 - \mathbf{q}_0) + t(\mathbf{q}_2 - \mathbf{q}_0)$ 를 고려하십시오.

$\mathbf{p}$ 를 u 와 v 의 함수로 표현하라; 즉, $\mathbf{p} = \mathbf{p}(u, v)$ 라는 공식이 되도록 구하라.

$\partial \mathbf{p} / \partial u$ 와 $\partial \mathbf{p} / \partial v$ 를 계산하고, 이 벡터들이 의미하는 바에 대한 기하학적 해석을 제시하십시오.

7. 이전 장의 "Lit Skull" 데모를 수정하여 바닥, 기둥, 구에 텍스처를 추가하십시오(그림 8.22). 텍스처는 본 장의 코드 디렉터리에서 찾을 수 있습니다.

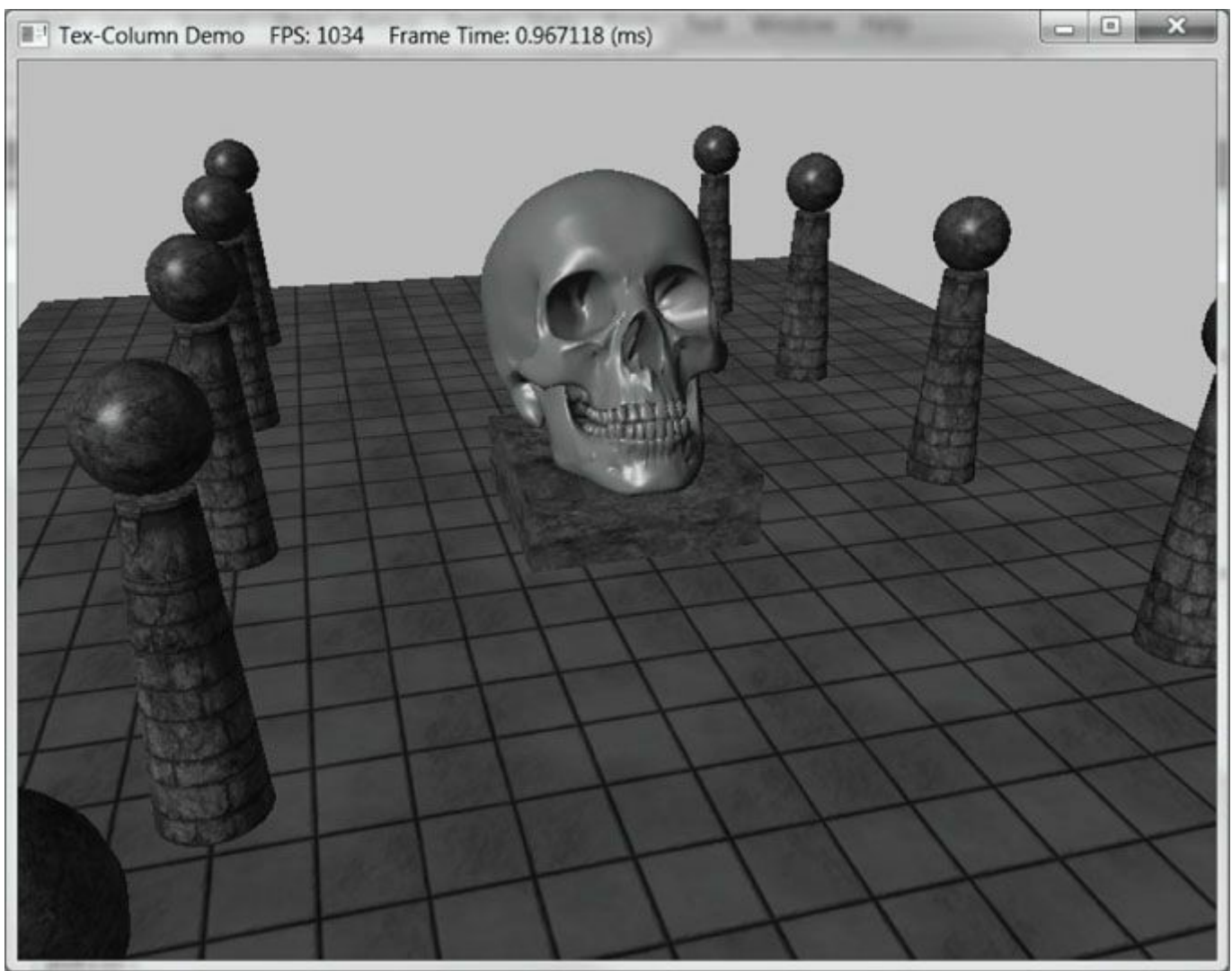


그림 8.22. 텍스처 처리된 기둥 장면.

제9장 블렌딩

그림 9.1을 살펴보자. 프레임 렌더링을 시작할 때 먼저 지형을 그린 다음 나무 상자를 그리므로, 지형과 상자 픽셀이 백 버퍼에 위치하게 된다. 그런 다음 *블렌딩*을 사용하여 백 버퍼에 수면을 그리면, 물 픽셀이 백 버퍼의 지형 및 상자 픽셀과 블렌딩되어 지형과 상자가 물 속으로 비쳐 보이게 된다. 이 장에서는 현재 래스터화 중인 픽셀(소위 *소스 픽셀*)을 백 버퍼에 미리 래스터화된 픽셀(소위 *대상 픽셀*)과 혼합(결합)할 수 있는 *블렌딩* 기법을 살펴본다. 이 기법은 물이나 유리 같은 반투명 객체를 렌더링하는 등 다양한 작업을 가능하게 한다.

논의를 위해 렌더 타겟으로 백 버퍼를 구체적으로 언급합니다. 그러나 나중에 "오프 스크린" 렌더 타겟에도 렌더링할 수 있음을 보여줄 것입니다. 블렌딩은 이러한 렌더 타겟에도 동일하게 적용되며, 대상 픽셀은 이전에 이러한 오프 스크린 렌더 타겟에 래스터화된 픽셀 값들입니다.

목표:

- 1. 블렌딩이 어떻게 작동하는지, 그리고 Direct3D에서 이를 사용하는 방법을 이해하기 위해.
- 2. Direct3D가 지원하는 다양한 블렌딩 모드에 대해 알아보기.

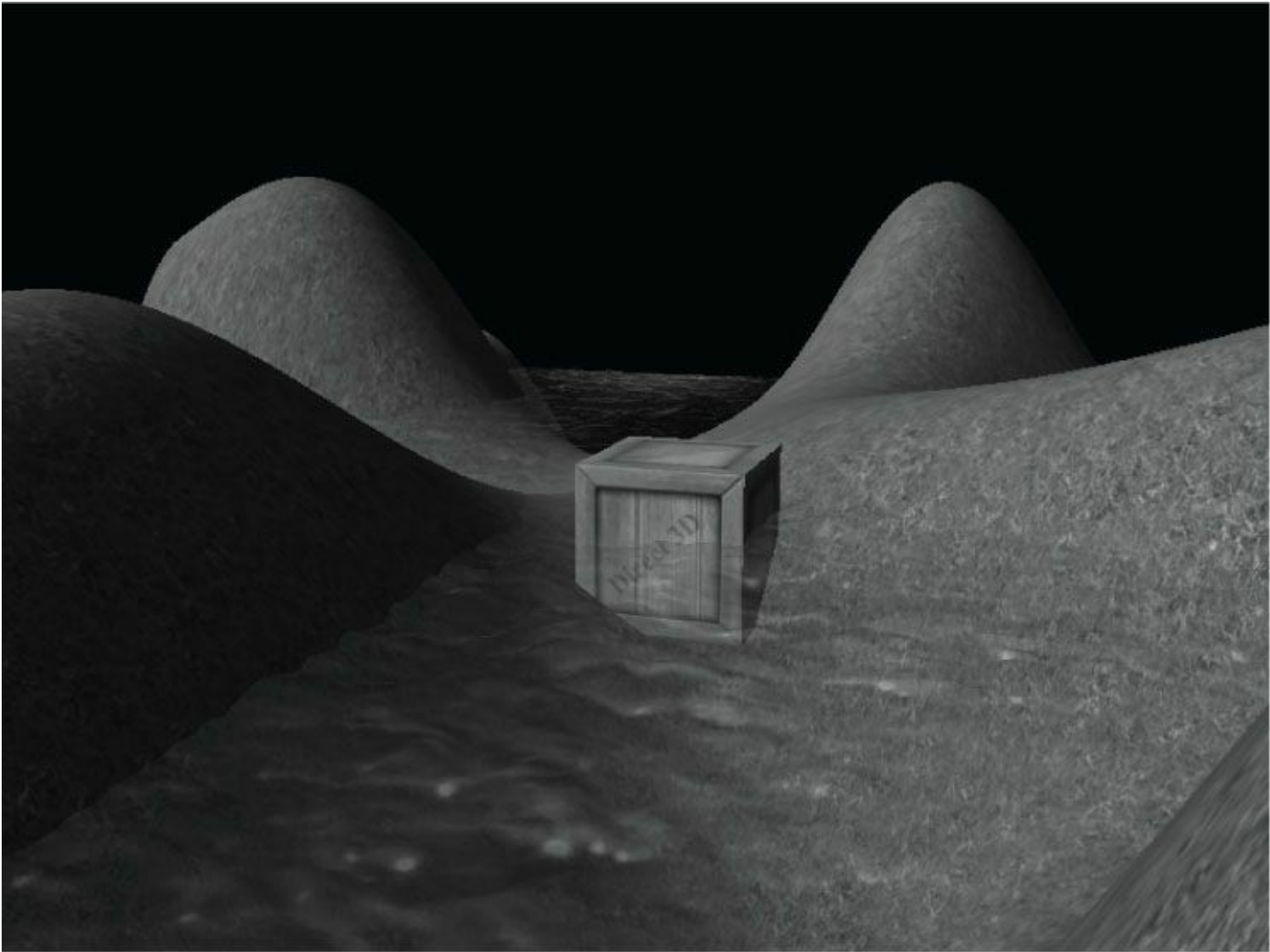


그림 9.1. 반투명한 수면.

- 3. 알파 컴포넌트를 사용하여 프리미티브의 투명도를 제어하는 방법을 알아보기.
- 4. HLSL 클립 함수를 사용하여 백 버퍼에 픽셀이 완전히 그려지는 것을 방지하는 방법을 학습합니다.

9.1 블렌딩 방정식

현재 래스터화 중인 i 번째 픽셀(소스 픽셀)에 대해 픽셀 셰이더가 출력하는 색상을 C_{src} 로 정의하고, 백 버퍼에 현재 존재하는 j 번째 픽셀(대상 픽셀)의 색상을 C_{dst} 로 정의한다. 블렌딩이 없다면, $C_{(src)}$ 은 $C_{(dst)}$ 를 덮어쓰게 될 것입니다(깊이/스텐실 테스트를 통과한다고 가정할 때). 그리고 i 번째 백 버퍼 픽셀의 새로운 색상이 됩니다. 그러나 블렌딩을 사용하면 C_{src} 와 C_{dst} 가 혼합되어 새로운 색상 C 를 생성하며, 이 C 가 $C_{(dst)}$ 를 덮어쓴다(즉, 혼합된 색상 C 가 백 버퍼의 i 번째 픽셀에 기록됨). Direct3D는 소스 및 대상 픽셀 색상을 혼합하기 위해 다음 블렌딩 방정식을 사용한다:

$$C = C_{src} \otimes F_{src} \boxplus C_{dst} \otimes F_{dst}$$

색상 F_{src} (소스 블렌드 계수) 및 F_{dst} (대상 블렌드 계수)는 §9.3에 설명된 값 중 어느 것이나 될 수 있으며, 이를 통해 원본 소스 및 대상 픽셀을 다양한 방식으로 수정하여 서로 다른 효과를 얻을 수 있습니다. $\otimes$ 연산자는 §5.3.1에 정의된 대로 색상 벡터에 대한 성분별 곱셈을 의미합니다; $\boxplus$ 연산자는 §9.2에 정의된 이진 연산자 중 하나일 수 있습니다. §9.2에 정의된 이진 연산자 중 하나일 수 있습니다.

위의 혼합 방정식은 색상의 RGB 성분에 대해서만 성립합니다. 알파 성분은 실제로 별도의 유사한 방정식으로 처리됩니다:

$$A = A_{src} F_{src} \boxplus A_{dst} F_{dst}$$

이 방정식은 본질적으로 동일하지만, 블렌드 계수와 이진 연산이 다를 수 있습니다. RGB와 알파를 분리하는 이유는 단순히 각각을 독립적으로, 따라서 다르게 처리하기 위함입니다.

Note: 알파 성분 블렌딩은 RGB 성분 블렌딩보다 훨씬 덜 자주 필요합니다. 이는 주로 백 버퍼 알파 값이 중요하지 않기 때문입니다. 백 버퍼 알파 값은 목적지 알파 값을 요구하는 알고리즘이 있는 경우에만 중요합니다.

9.2 블렌딩 연산

블렌딩 방정식에서 사용되는 이진 $\boxplus$ 연산자는 다음 중 하나일 수 있습니다:

<pre>typedef enum D3D11_BLEND_OP { D3D11_BLEND_OP_ADD = 1, D3D11_BLEND_OP_SUBTRACT = 2, D3D11_BLEND_OP_REV_SUBTRACT = 3, D3D11_BLEND_OP_MIN = 4, D3D11_BLEND_OP_MAX = 5, } D3D11_BLEND_OP;</pre>	$C = C_{src} \otimes F_{src} + C_{dst} \otimes F_{dst}$ $C = C_{dst} \otimes F_{dst} - C_{src} \otimes F_{src}$ $C = C_{src} \otimes F_{src} - C_{dst} \otimes F_{dst}$ $C = \min(C_{src}, C_{dst})$ $C = \max(C_{src}, C_{dst})$
--	---

Note: 최소/최대 연산에서는 블렌드 계수가 무시됩니다.

동일한 연산자가 알파 블렌딩 방정식에도 적용됩니다. 또한 RGB와 알파에 대해 서로 다른 연산자를 지정할 수 있습니다.

예를 들어, 두 RGB 항은 더하고 두 알파 항은 빼는 것도 가능합니다:

$$C = C_{src} \otimes F_{src} + C_{dst} \otimes F_{dst}$$
$$A = A_{dst} F_{dst} - A_{src} F_{src}$$

9.3 블렌딩 계수

소스 및 대상 블렌드 팩터에 서로 다른 조합을 설정하고 다양한 블렌드 연산자를 함께 사용함으로써 수십 가지의 서로 다른 블렌딩 효과를 얻을 수 있습니다. §9.5에서 몇 가지 조합을 예시로 보여드리겠습니다, 그 효과를 직접 느껴보려면 다른 조합들도 직접 실험해 보셔야 합니다. 다음 목록은 F_{src} 와 F_{dst} 모두에 적용되는 기본 블렌드 팩터를 설명합니다. 추가 고급 블렌드 팩터에 대해서는 SDK 문서의 **D3D11_BLEND** 열거형 유형을 참조하십시오. $C_{src} = (r_s, g_s, b_s), A_{src} = a_s$ (픽셀 셰이더에서 출력되는 RGBA 값), $C_{dst} = (r_d, g_d, b_d), A_{dst} = a_d$ (렌더 타겟에 이미 저장된 RGBA 값)이며, F 는 F_{src} 또는 F_{dst} 이고 F 는 F_{src} 또는 F_{dst} 인 경우 다음과 같습니다:

```
D3D11_BLEND_ZERO: F = (0,0,0) 및 F = 0 D3D11_BLEND_ONE: F = (1,1,1) 및 F = 1
D3D11_BLEND_SRC_COLOR: F = (r_s, g_s, b_s) D3D11_BLEND_INV_SRC_COLOR: F = (1 - r_s, 1 - g_s, 1 - b_s)
D3D11_BLEND_SRC_ALPHA: F = (a_s, a_s, a_s) 및 F = a_s
D3D11_BLEND_INV_SRC_ALPHA: F = (1 - a_s, 1 - a_s, 1 - a_s) 및 F = 1 - a_s
D3D11_BLEND_DEST_ALPHA: F = (a_d, a_d, a_d) 및 F = a_d D3D11_BLEND_INV_DEST_ALPHA: F = (1 - a_d, 1 - a_d, 1 - a_d) 및 F = 1 - a_d
D3D11_BLEND_DEST_COLOR: F = (r_d, g_d, b_d) D3D11_BLEND_INV_DEST_COLOR: F = (1 - r_d, 1 - g_d, 1 - b_d)
D3D11_BLEND_SRC_ALPHA_SAT: F = (a', a', a') F = a'
where a' = clamp(a_s, 0, 1)
D3D11_BLEND_BLEND_FACTOR: F = (r, g, b) 및 F = a, 여기서 색상 (r, g, b, a)은
```

ID3D11DeviceContext::OMSetBlendState 메서드 (§9.4)의 두 번째 매개변수에 전달됩니다. 이를 통해 블렌드 팩터 색상을 직접 지정할 수 있지만, 블렌드 상태를 변경하기 전까지는 상수로 유지됩니다.

D3D11_BLEND_INV_BLEND_FACTOR: $F = (1 - r, 1 - g, 1 - b)$ 및 $F = 1 - a$, 여기서 색상 (r, g, b, a) 는

ID3D11DeviceContext::OMSetBlendState 메서드 (§9.4)의 두 번째 매개변수로 제공됩니다. 이를 통해 사용할 블렌드 팩터 색상을 직접 지정할 수 있지만, 블렌드 상태를 변경할 때까지는 상수로 유지됩니다.

Note: 클램프 함수는 다음과 같이 정의됩니다:

$$\text{clamp}(x, a, b) = \begin{cases} x, & a \leq x \leq b \\ a, & x < a \\ b, & x > b \end{cases}$$

앞서 설명한 모든 블렌드 팩터는 RGB 블렌딩 방정식에 적용됩니다. 알파 블렌딩 방정식의 경우, **_COLOR**는 허용되지 않습니다.

9.4 블렌드 상태

블렌딩 연산자와 블렌드 계수에 대해 설명했지만, Direct3D에서 이러한 값은 어디에서 설정할까요? 이러한 블렌드 설정은 **ID3D11BlendState** 인터페이스로 제어됩니다. 해당 인터페이스는 **D3D11_BLEND_DESC** 구조체를 작성한 후 **ID3D11Device::CreateBlendState**를 호출하여 찾을 수 있습니다:

```
HRESULT ID3D11Device::CreateBlendState(
    const D3D11_BLEND_DESC *pBlendStateDesc, ID3D11BlendState
    **ppBlendState);
```

1. **pBlendStateDesc**: 생성할 블렌드 상태를 설명하는 작성된 **D3D11_BLEND_DESC** 구조체에 대한 포인터.
2. **ppBlendState**: 생성된 블렌드 상태 인터페이스에 대한 포인터를 반환합니다.

D3D11_BLEND_DESC 구조체는 다음과 같이 정의됩니다:

```
typedef struct D3D11_BLEND_DESC {
    BOOL AlphaToCoverage Enable; // 기본값: False
    BOOL IndependentBlendEnable; // 기본값: False
    D3D11_RENDER_TARGET_BLEND_DESC RenderTarget[8];
} D3D11_BLEND_DESC;
```

1.

AlphaToCoverageEnable: 알파-투-커버리지를 활성화하려면 true를 지정합니다. 이는 앞사귀나 게이트 텍스처를 렌더링할 때 유용한 멀티샘플링 기법입니다. 알파-투-커버리지를 비활성화하려면 false를 지정합니다. 알파-투-커버리지는 멀티샘플링이 활성화되어야 합니다(즉, 백 버퍼와 깊이 버퍼가 멀티샘플링으로 생성됨). 11장에서 알파-투-커버리지를 사용하는 샘플을 보 여드리겠습니다.
2.

IndependentBlendEnable: Direct3D 11은 최대 8개의 렌더 타겟에 동시에 렌더링을 지원합니다. 이 플래그가 true로 설정되면 각 렌더 타겟에 대해 서로 다른 블렌딩(다른 블렌드 계수, 다른 블렌드 연산, 블렌딩 비활성화/활성화 등)을 수행할 수 있음을 의미합니다. 이 플래그가 false로 설정되면 모든 렌더 타겟이 D3D11_BLEND_DESC::RenderTarget 배열의 첫 번째 요소에 설명된 방식으로 동일하게 블렌딩됩니다. 고급 알고리즘에는 여러 렌더 타겟이 사용되지만, 현재는 한 번에 하나의 렌더 타겟에만 렌더링한다고 가정하십시오.
3.

RenderTarget: 8개의 D3D11_RENDER_TARGET_BLEND_DESC 요소로 구성된 배열로, i번째 요소는 i번째 동시 렌더 타겟에 대한 블렌딩 방식을 설명합니다. IndependentBlendEnable이 false로 설정된 경우, 모든 렌더 타겟은 블렌딩에 RenderTarget[0]을 사용합니다.

D3D11_RENDER_TARGET_BLEND_DESC 구조체는 다음과 같이 정의됩니다:

```
typedef struct D3D11_RENDER_TARGET_BLEND_DESC {  
  
    BOOL BlendEnable; // 기본값: False  
  
    D3D11_BLEND SrcBlend; // 기본값: D3D11_BLEND_ONE  
  
    D3D11_BLEND DestBlend; // 기본값: D3D11_BLEND_ZERO  
  
    D3D11_BLEND_OP BlendOp; // 기본값: D3D11_BLEND_OP_ADD  
  
    D3D11_BLEND SrcBlendAlpha; // 기본값: D3D11_BLEND_ONE  
  
    D3D11_BLEND DestBlendAlpha; // 기본값: D3D11_BLEND_ZERO  
  
    D3D11_BLEND_OP BlendOpAlpha; // 기본값: D3D11_BLEND_OP_ADD  
  
    UINT8 렌더타겟 쓰기 마스크; // 기본값: D3D11_COLOR_WRITE_ENABLE_ALL  
  
} D3D11_RENDER_TARGET_BLEND_DESC;
```

1.

BlendEnable: 블렌딩을 활성화하려면 true를, 비활성화하려면 false를 지정합니다.
2.

SrcBlend: RGB 블렌딩에 대한 소스 블렌드 계수 F_{src} 를 지정하는 D3D11_BLEND 열거형의 멤버입니다.
3.

DestBlend: RGB 블렌딩에대한 대상 블렌드 계수 F_{dst} 를 지정하는 D3D11_BLEND 열거형의 멤버입니다.
4.

BlendOp: RGB 블렌딩 연산자를 지정하는 D3D11_BLEND_OP 열거형의 멤버입니다.
5.

SrcBlendAlpha: 알파 블렌딩을 위한 대상 블렌드 계수 F_{src} 를 지정하는 D3D11_BLEND 열거형의 멤버입니다.
6.

DestBlendAlpha: 알파 블렌딩에 대한 대상 블렌드 계수 F_{dst} 를 지정하는 D3D11_BLEND 열거형의 멤버입니다.
7.

BlendOpAlpha: 알파 블렌딩 연산자를 지정하는 D3D11_BLEND_OP 열거형의 멤버입니다.
8.

RenderTargetWriteMask : 다음 플래그 중 하나 이상의 조합입니다.

```
typedef enum D3D11_COLOR_WRITE_ENABLE {  
    D3D11_COLOR_WRITE_ENABLE_RED = 1,  
    D3D11_COLOR_WRITE_ENABLE_GREEN = 2,  
    D3D11_COLOR_WRITE_ENABLE_BLUE = 4,  
    D3D11_COLOR_WRITE_ENABLE_ALPHA = 8,  
    D3D11_COLOR_WRITE_ENABLE_ALL =  
        (D3D11_COLOR_WRITE_ENABLE_RED | D3D11_COLOR_WRITE_ENABLE_GREEN |  
         D3D11_COLOR_WRITE_ENABLE_BLUE | D3D11_COLOR_WRITE_ENABLE_ALPHA)  
} D3D11_COLOR_WRITE_ENABLE;
```

이 플래그들은 블렌딩 후 백 버퍼의 어떤 색상 채널에 쓰기가 수행되는지를 제어합니다. 예를 들어, D3D11_COLOR_WRITE_ENABLE_ALPHA를 지정하여 RGB 채널에 대한 쓰기를 비활성화하고 알파 채널에만 쓰기를 수행할 수 있습니다. 이러한 유연성은 고급 기법에 유용할 수 있습니다. 블렌딩이 비활성화된 경우, 픽셀 셰이더에서 반환된 색상이 쓰기 마스크 적용 없이 사용됩니다.

블렌드 상태 객체를 파이프라인의 출력 병합 단계에 바인딩하려면 다음을 호출합니다:

```
void ID3D11DeviceContext::OMSetBlendState( ID3D11BlendState  
    *pBlendState, const FLOAT BlendFactor,
```


UINT SampleMask);

1. **pBlendState**: 장치에서 활성화할 블렌드 상태 객체에 대한 포인터입니다.
2. **BlendFactor**: RGBA 색상 벡터를 정의하는 네 개의 부동 소수점 숫자로 구성된 배열입니다. 이 색상 벡터는 **D3D11_BLEND_BLEND_FACTOR** 또는 **D3D11_BLEND_INV_BLEND_FACTOR**가 지정될 때 블렌드 팩터로 사용됩니다.
3. **SampleMask** : 멀티샘플링은 최대 32개의 샘플을 취할 수 있습니다. 이 32비트 정수 값은 샘플을 활성화/비활성화하는 데 사용됩니다. 예를 들어, 5번째 비트를 비활성화하면 5번째 샘플이 취해지지 않습니다. 물론, 다중 샘플링을 최소 5개 이상 사용 중인 경우에만 5번째 샘플 비활성화가 영향을 미칩니다. 단일 샘플링을 사용하는 애플리케이션에서는 이 매개변수의 첫 번째 비트만 중요합니다(연습 1 참조). 일반적으로 기본값인 0xffffffff가 사용되며, 이는 애플리케이션이 취할 수 있는 샘플을 차단하지 않습니다.

다른 상태 블록과 마찬가지로 기본 블렌드 상태(블렌딩 비활성화)가 존재합니다. **OMSetBlendState**를 null로 호출하면 기본 블렌드 상태로 복원됩니다. 블렌딩은 추가적인 픽셀 단위 작업을 필요로 하므로, 필요한 경우에만 활성화하고 작업이 끝나면 반드시 비활성화해야 합니다.

다음 코드는 블렌딩 상태 생성 및 설정 예시를 보여줍니다:

```
D3D11_BLEND_DESC transparentDesc = {0};
transparentDesc.AlphaToCoverageEnable = false;
transparentDesc.IndependentBlendEnable = false;

transparentDesc.RenderTarget[0].BlendEnable = true;
transparentDesc.RenderTarget[0].SrcBlend                = D3D11_BLEND_SRC_ALPHA;
transparentDesc.RenderTarget[0].DestBlend               = D3D11_BLEND_INV_SRC_ALPHA;
transparentDesc.RenderTarget[0].BlendOp                 = D3D11_BLEND_OP_ADD;
transparentDesc.RenderTarget[0].SrcBlendAlpha           = D3D11_BLEND_ONE;
transparentDesc.RenderTarget[0].DestBlendAlpha          = D3D11_BLEND_ZERO;
transparentDesc.RenderTarget[0].BlendOpAlpha            = D3D11_BLEND_OP_ADD;
transparentDesc.RenderTarget[0].RenderTargetWriteMask = D3D11_COLOR_WRITE_ENABLE_ALL;

ID3D11BlendState* TransparentBS;
HR(device->CreateBlendState(&transparentDesc, &TransparentBS));

...

float blendFactors[] = {0.0f, 0.0f, 0.0f, 0.0f};
md3dImmediateContext->OMSetBlendState( TransparentBS,
blendFactor, 0xffffffff);
```

다른 상태 블록 인터페이스와 마찬가지로, 애플리케이션 초기화 시점에 모두 생성한 후 필요에 따라 상태 인터페이스 간에 전환해야 합니다.

블렌드 상태 객체는 효과 파일에서도 설정 및 정의할 수 있습니다:

```
BlendState blend
{
    // 첫 번째 렌더 타겟용 블렌딩 상태. BlendEnable[0] = TRUE;
    SrcBlend[0] = SRC_COLOR; DestBlend[0] =
    INV_SRC_ALPHA; BlendOp[0] = ADD;
    SrcBlendAlpha[0] = ZERO; DestBlendAlpha[0] =
    ZERO; BlendOpAlpha[0] = ADD;
    RenderTargetWriteMask[0] = 0x0F;

    // 두 번째 동시 렌더 타겟에 대한 블렌딩 상태. BlendEnable[1] = True;
    SrcBlend[1] = One; DestBlend[1] = Zero;
    BlendOp[1] = Add; SrcBlendAlpha[1] =
    Zero;
```

```
DestBlendAlpha[1] = Zero; BlendOpAlpha[1] =
Add; RenderTargetWriteMask[1] = 0x0F;
};

technique11 Tech
{
    pass P0
    {
        ...

        // 이 패스에는 "blend"를 사용합니다. SetBlendState(blend, float4(0.0f, 0.0f, 0.0f, 0.0f),
        0xffffffff);
    }
}
```

블렌드 상태 객체에 할당하는 값은 C++ 구조체에 할당하는 값과 유사하지만 점두사가 없습니다. 예를 들어, `D3D11_BLEND_SRC_COLOR` 대신 효과 코드에서 단순히 `SRC_COLOR`를 지정합니다. 또한 상태 속성에 대한 값 할당은 대소문자를 구분하지 않습니다.

9.5 예시

다음 소절에서는 특정 효과를 얻기 위해 사용되는 블렌딩 팩터 조합을 살펴봅니다. 이 예시에서는 RGB 블렌딩만 다룹니다. 알파 블렌딩은 유사한 방식으로 처리됩니다.

9.5.1 색상 쓰기 없음

원본 대상 픽셀을 정확히 그대로 유지하고, 현재 래스터화 중인 소스 픽셀로 덮어쓰거나 블렌딩하지 않으려는 경우를 가정해 보겠습니다. 예를 들어, 백 버퍼가 아닌 깊이/스텐실 버퍼에만 쓰기를 원할 때 유용합니다. 이를 위해 소스 픽셀 블렌드 팩터를 `D3D11_BLEND_ZERO`로, 대상 블렌드 팩터를 `D3D11_BLEND_ONE`로, 블렌드 연산자를 `D3D11_BLEND_OP_ADD`로 설정합니다. 이 설정에서 블렌딩 방정식은 다음과 같이 단순화됩니다:

$$\begin{aligned} C &= C_{src} \otimes F_{src} \boxplus C_{dst} \otimes F_{dst} \\ C &= C_{src} \otimes (0,0,0) + C_{dst} \otimes (1,1,1) \\ C &= C_{dst} \end{aligned}$$

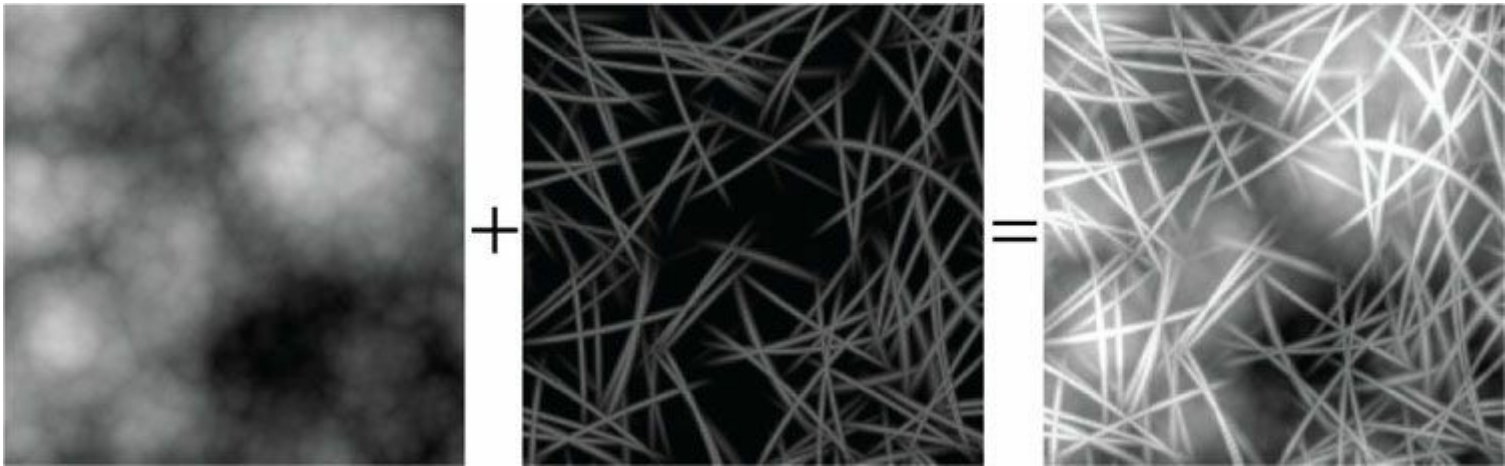


그림 9.2. 소스 및 대상 색상 추가. 색상이 더해지기 때문에 더 밝은 이미지가 생성됩니다.

이것은 인위적인 예시입니다. 동일한 기능을 구현하는 또 다른 방법은 `D3D11_RENDER_TARGET_BLEND_DESC::RenderTargetWriteMask` 멤버를 0으로 설정하여 어떤 색상 채널에도 쓰기가 이루어지지 않도록 하는 것입니다.

9.5.2 더하기/빼기

소스 픽셀과 대상 픽셀을 더하고 싶다고 가정해 보겠습니다([그림 9.2](#) 참조). 이를 위해 소스 블렌드 팩터를 **D3D11_BLEND_ONE**으로, 대상 블렌드 팩터를 **D3D11_BLEND_ONE**으로, 블렌드 연산자를 **D3D11_BLEND_OP_ADD**로 설정합니다. 이렇게 설정하면 블렌딩 방정식은 다음과 같이 단순화됩니다:

$$\begin{aligned} C &= C_{src} \otimes F_{src} \boxplus C_{dst} \otimes F_{dst} \\ C &= C_{src} \otimes (1,1,1) + C_{dst} \otimes (1,1,1) \\ C &= C_{src} + C_{dst} \end{aligned}$$

이전 블렌드 팩터를 사용하고 블렌드 연산을 **D3D11_BLEND_OP_SUBTRACT**로 대체하면 됩니다([그림 9.3](#)).

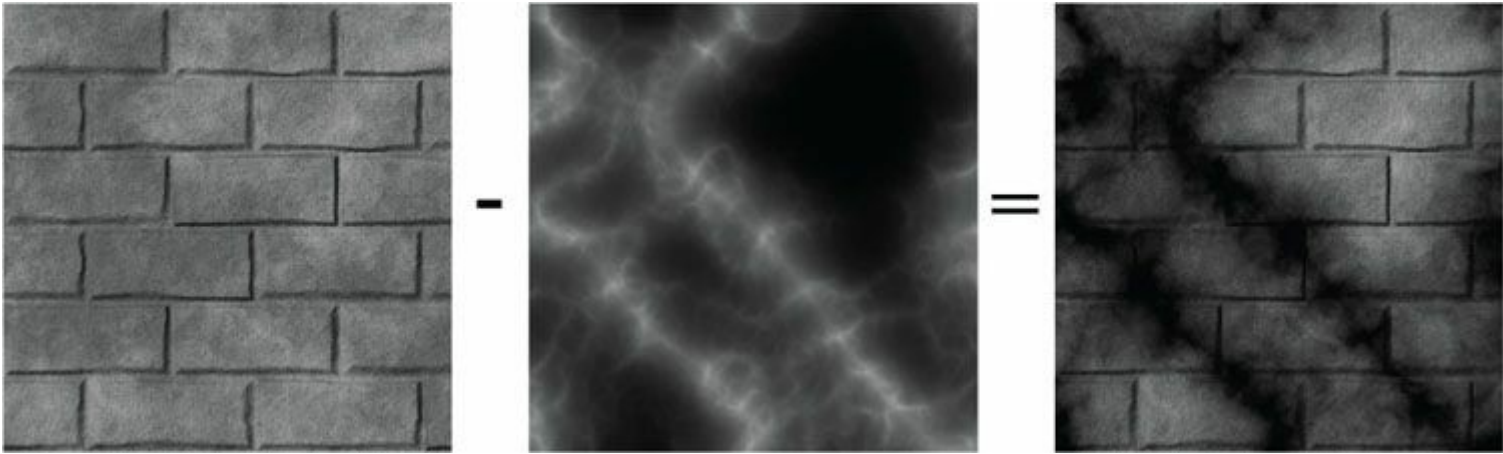


그림 9.3. 대상 색상에서 소스 색상을 빼기. 색상이 제거되므로 텍셀은 더 어두운 이미지를 생성합니다.



그림 9.4. 소스 색상과 대상 색상 곱하기.

9.5.3 곱하기

소스 픽셀을 해당 대상 픽셀과 곱하기를 원한다고 가정해 보자([그림 9.4](#) 참조). 이를 위해 소스 블렌드 팩터를 **D3D11_BLEND_ZERO**로, 대상 블렌드 팩터를 **D3D11_BLEND_SRC_COLOR**로, 블렌드 연산자를 **D3D11_BLEND_OP_ADD**로 설정한다. 이 설정으로 블렌딩 방정식은 다음과 같이 단순화된다:

$$\begin{aligned} C &= C_{src} \otimes F_{src} \boxplus C_{dst} \otimes F_{dst} \\ C &= C_{src} \otimes (0,0,0) + C_{dst} \otimes C_{src} \\ C &= C_{dst} \otimes C_{src} \end{aligned}$$

9.5.4 투명도

소스 알파 성분 a_s 를 소스 픽셀의 불투명도를 제어하는 백분율로 생각하자(예: 알파 0은 0% 불투명, 0.4는 40% 불투명, 1.0은 100% 불투명). 불투명도와 투명도의 관계는 단순히 $T = 1 - A$ 이다. 여기서 A는 불투명도, T는 투명도이다. 예를 들어, 무언가의 불투명도가 0.4라면, 그 투명도는 $1 - 0.4 = 0.6$ 이 됩니다. 이제 가정해 보겠습니다.

소스 픽셀의 불투명도에 따라 소스와 대상 픽셀을 혼합하고자 합니다. 이를 위해 소스 혼합 계수를 **D3D11_BLEND_SRC_ALPHA**로, 대상 혼합 계수를 **D3D11_BLEND_INV_SRC_ALPHA**로 설정하고 혼합 연산자를 **D3D11_BLEND_OP_ADD**로 지정합니다. 이 설정으로 혼합 방정식은 다음과 같이 단순화됩니다:

$$\begin{aligned} \mathbf{C} &= \mathbf{C}_{src} \otimes \mathbf{F}_{src} \boxplus \mathbf{C}_{dst} \otimes \mathbf{F}_{dst} \\ \mathbf{C} &= \mathbf{C}_{src} \otimes (a_s, a_s, a_s) + \mathbf{C}_{dst} \otimes (1 - a_s, 1 - a_s, 1 - a_s) \\ \mathbf{C} &= a_s \mathbf{C}_{dst} + (1 - a_s) \mathbf{C}_{src} \end{aligned}$$

예를 들어 $a_s = 0.25$ 라고 가정하면, 소스 픽셀의 불투명도가 25%임을 의미합니다. 이때 소스 픽셀과 대상 픽셀을 혼합하면 최종 색상은 소스 픽셀의 25%와 대상 픽셀(소스 픽셀 "뒤에 있는" 픽셀)의 75%가 혼합된 값이 될 것으로 예상됩니다. 소스 픽셀이 75% 투명하기 때문입니다. 위 방정식은 정확히 이를 나타냅니다:

$$\begin{aligned} \mathbf{C} &= a_s \mathbf{C}_{dst} + (1 - a_s) \mathbf{C}_{src} \\ \mathbf{C} &= 0.25 \mathbf{C}_{dst} + 0.75 \mathbf{C}_{src} \end{aligned}$$

이 블렌딩 방식을 사용하면 [그림 9.1](#)과 같은 투명 객체를 그릴 수 있습니다. 이 블렌딩 방식에서는 객체를 그리는 순서가 중요하다는 점을 유의해야 합니다. 우리는 다음과 같은 규칙을 사용합니다: *블렌딩을 사용하지 않는 객체를 먼저 그립니다. 다음으로, 블렌딩을 사용하는 객체들을 카메라로부터의 거리에 따라 정렬합니다. 마지막으로, 블렌딩을 사용하는 객체는 뒤에서 앞으로 순서로 그립니다.* 뒤에서 앞으로의 그리기 순서를 사용하는 이유는 객체가 공간적으로 그 뒤에 있는 객체와 블렌딩되도록 하기 위함입니다. 만약 객체가 투명하면 그 뒤의 장면을 볼 수 있기 때문입니다. 따라서 투명한 객체 뒤에 있는 모든 픽셀이 이미 백 버퍼에 기록되어 있어야 합니다. 그래야 투명한 소스 픽셀을 그 뒤 장면의 대상 픽셀과 블렌딩할 수 있습니다.

§9.5.1의 블렌딩 방식에서는 소스 픽셀이 백 버퍼에 쓰여지는 것을 단순히 차단하기 때문에 그리기 순서는 중요하지 않습니다. §9.5.2 및 9.5.3에서 논의된 블렌딩 방식의 경우에도, 비블렌딩 오브젝트를 먼저 그리고 블렌딩 오브젝트를 마지막에 그립니다. 이는 블렌딩을 시작하기 전에 모든 비블렌딩 지오메트리를 백 버퍼에 먼저 배치하기 위함입니다. 그러나 블렌딩을 사용하는 오브젝트를 정렬할 필요는 없습니다. 이는 연산이 교환법칙을 따르기 때문입니다. 즉, 백 버퍼 픽셀 색상 **B**로 시작한 후 해당 픽셀에 n 회 i 가 산/감산/곱셈 블렌딩을 수행할 때 순서는 중요하지 않습니다:

$$\begin{aligned} \mathbf{B}' &= \mathbf{B} + \mathbf{C}_0 + \mathbf{C}_1 + \dots + \mathbf{C}_{n-1} \\ \mathbf{B}' &= \mathbf{B} - \mathbf{C}_0 - \mathbf{C}_1 - \dots - \mathbf{C}_{n-1} \\ \mathbf{B}' &= \mathbf{B} \otimes \mathbf{C}_0 \otimes \mathbf{C}_1 \otimes \dots \otimes \mathbf{C}_{n-1} \end{aligned}$$

9.5.5 블렌딩과 깊이 버퍼

가산/감산/곱셈 혼합으로 블렌딩할 때 깊이 테스트에 문제가 발생합니다. 예시를 위해 가산 혼합으로만 설명하겠지만, 감산/곱셈 혼합에도 동일한 원리가 적용됩니다. 가산 혼합으로 객체 집합 S 를 렌더링할 때, S 내 객체들은 서로 가리지 않고 단순히 색상이 누적되도록 설계됩니다([그림 9.5](#) 참조). 따라서 S 내 객체 간에는 깊이 테스트를 수행하지 않아야 합니다. 만약 수행한다면, 뒤에서 앞으로의 렌더링 순서가 보장되지 *않을 경우* S 내 객체 중 하나가 다른 객체를 가리게 되어 깊이 테스트로 인해 픽셀 프래그먼트가 거부될 수 있습니다. 이는 해당 객체의 픽셀 색상이 블렌딩 합계에 누적되지 않음을 의미합니다. S 내 객체들 사이의 깊이 테스트는 S 내 객체 렌더링 시 깊이 버퍼 쓰기를 비활성화함으로써 무효화할 수 있습니다. 깊이 쓰기가 비활성화되면, 가산 혼합으로 그려진 S 내 객체의 깊이는 깊이 버퍼에 기록되지 않습니다. 따라서 이 객체는 깊이 테스트로 인해 뒤에 그려진 S 내 객체를 가리지 않습니다. 단, 깊이 쓰기 비활성화는 S 내 객체(가산 혼합으로 그려진 객체 집합)를 그리는 동안에만 적용됩니다. 깊이 읽기와 깊이 테스트는 여전히 활성화됩니다. 이는 블렌딩되지 않은 지오메트리(블렌딩된 지오메트리보다 먼저 그려짐)가 여전히 뒤에 있는 블렌딩된 지오메트리를 가릴 수 있도록 하기 위함입니다. 예를 들어 벽 뒤에 가산 혼합된 객체 집합이 있다면, 단단한 벽이 블렌딩된 객체를 가리기 때문에 볼 수 없습니다. 깊이 쓰기 비활성화 방법 및 보다 일반적인 깊이 테스트 설정 구성 방법은 다음 장에서 다룰 예정입니다.

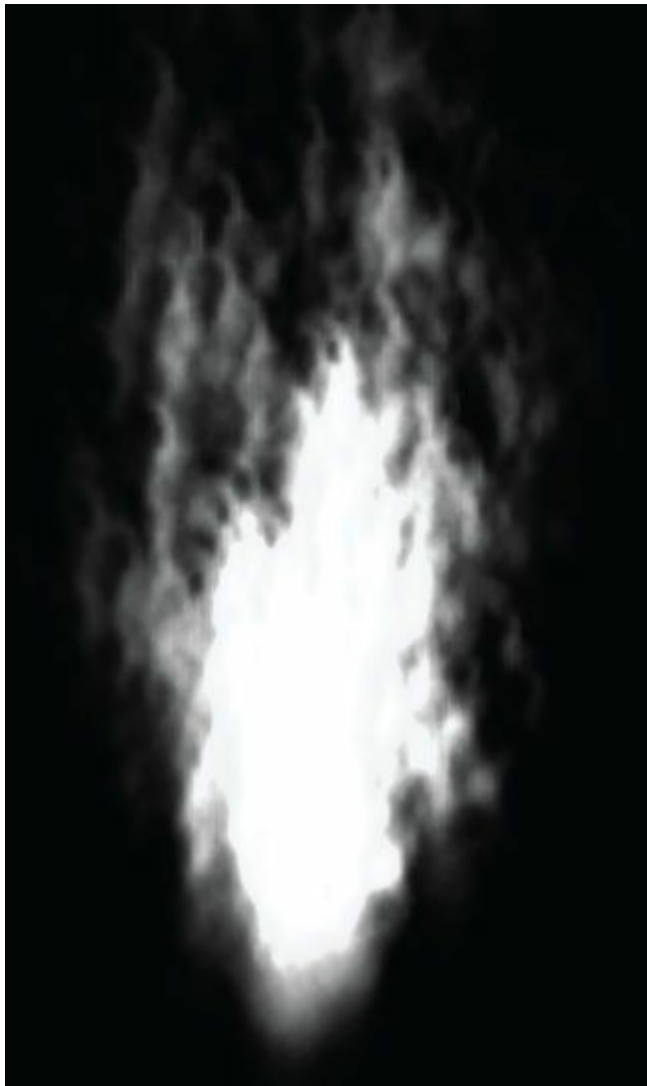
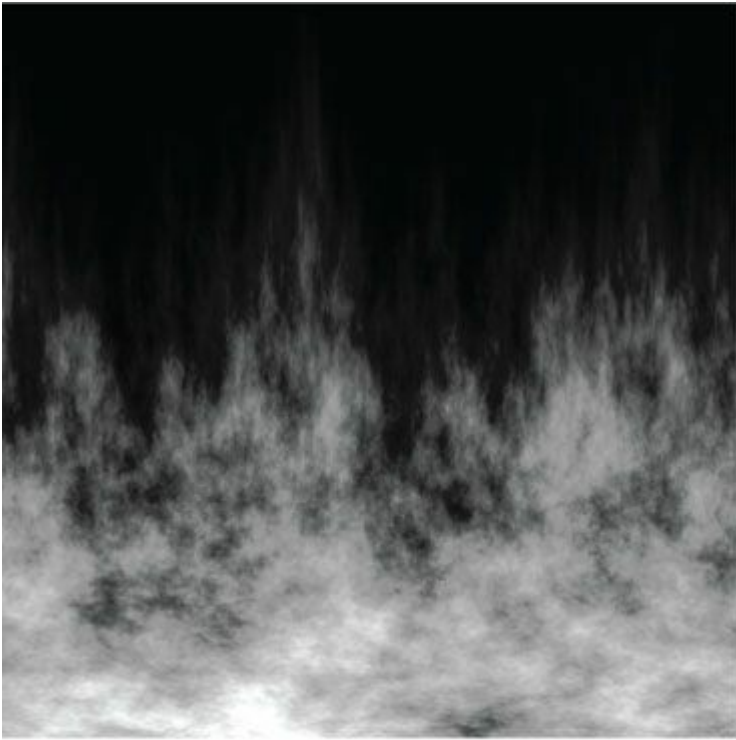


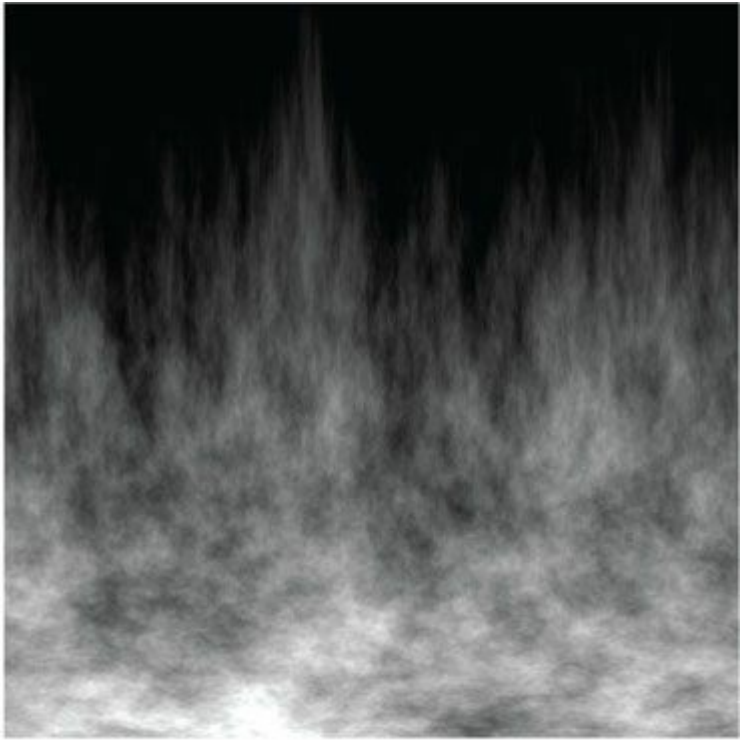
그림 9.5. 가산 혼합을 사용하면, 더 많은 입자가 겹쳐지고 합쳐지는 소스 지점 근처에서 강도가 더 강해집니다. 입자가 퍼져 나갈수록 겹쳐지고 합쳐지는 입자가 줄어들기 때문에 강도가 약해집니다.

9.6 알파 채널

§9.5.4의 예시는 RGB 블렌딩에서 소스 알파 성분을 투명도 제어에 활용할 수 있음을 보여주었습니다. 블렌딩 방정식에 사용되는 소스 색상은 픽셀 셰이더에서 비롯됩니다. 지난 장에서 살펴본 바와 같이, 우리는 확산 재질의 알파 값을 픽셀 셰이더의 알파 출력으로 반환합니다. 따라서 확산 맵의 알파 채널이 투명도 제어에 사용됩니다.



RGB Channels



Alpha Channel

그림 9.6. RGB 이미지(왼쪽)와 그레이스케일 이미지(오른쪽). 그레이스케일 이미지는 텍스처의 알파 채널에 삽입됩니다.

```
float4 PS(VertexOut pin) : SV_Target
{
    ...

    // 디퓨즈 머티리얼과 텍스처에서 알파를 가져오는 공통 처리.
    litColor.a = gMaterial.Diffuse.a * texColor.a;

    litColor를 반환합니다;
}
```

일반적으로 Adobe Photoshop과 같은 널리 사용되는 이미지 편집 소프트웨어에서 알파 채널을 추가한 후 알파 채널을 지원하는 형식(예: 32비트 .bmp 형식 또는 .dds 형식)으로 이미지를 저장할 수 있습니다. 그러나 여기서는 이전 장에서 설명한 DXTex 유틸리티 프로그램을 사용하여 알파 채널을 삽입하는 다른 방법을 보여줍니다.

먼저 두 개의 이미지가 있다고 가정합니다—RGB 컬러 이미지와 알파 채널에 삽입될 그레이스케일 이미지입니다 (그림 9.6 참조).

이제 DXTex 도구를 열고 웹 페이지의 본 장 샘플 폴더에 있는 *fire_rgb.bmp* 파일을 엽니다. fire 텍스처는 자동으로 24비트 RGB 텍스처(즉, **D3DFMT_R8G8B8**)로 로드되며, 픽셀당 8비트의 빨강, 8비트의 녹색, 8비트의 파랑을 가집니다. 알파 채널을 지원하는 형식(예: 32비트 ARGB 텍스처 형식 **D3DFMT_A8R8G8B8** 또는 **D3DFMT_DXT5**와 같은 압축 형식)으로 변경해야 합니다. 메뉴에서 'Format'을 선택한 후 'Change_Surface Format'을 선택합니다. 그림 9.7과 같은 대화 상자 가 나타나면 **DXT5** 형식을 선택하고 '확인'을 누릅니다.

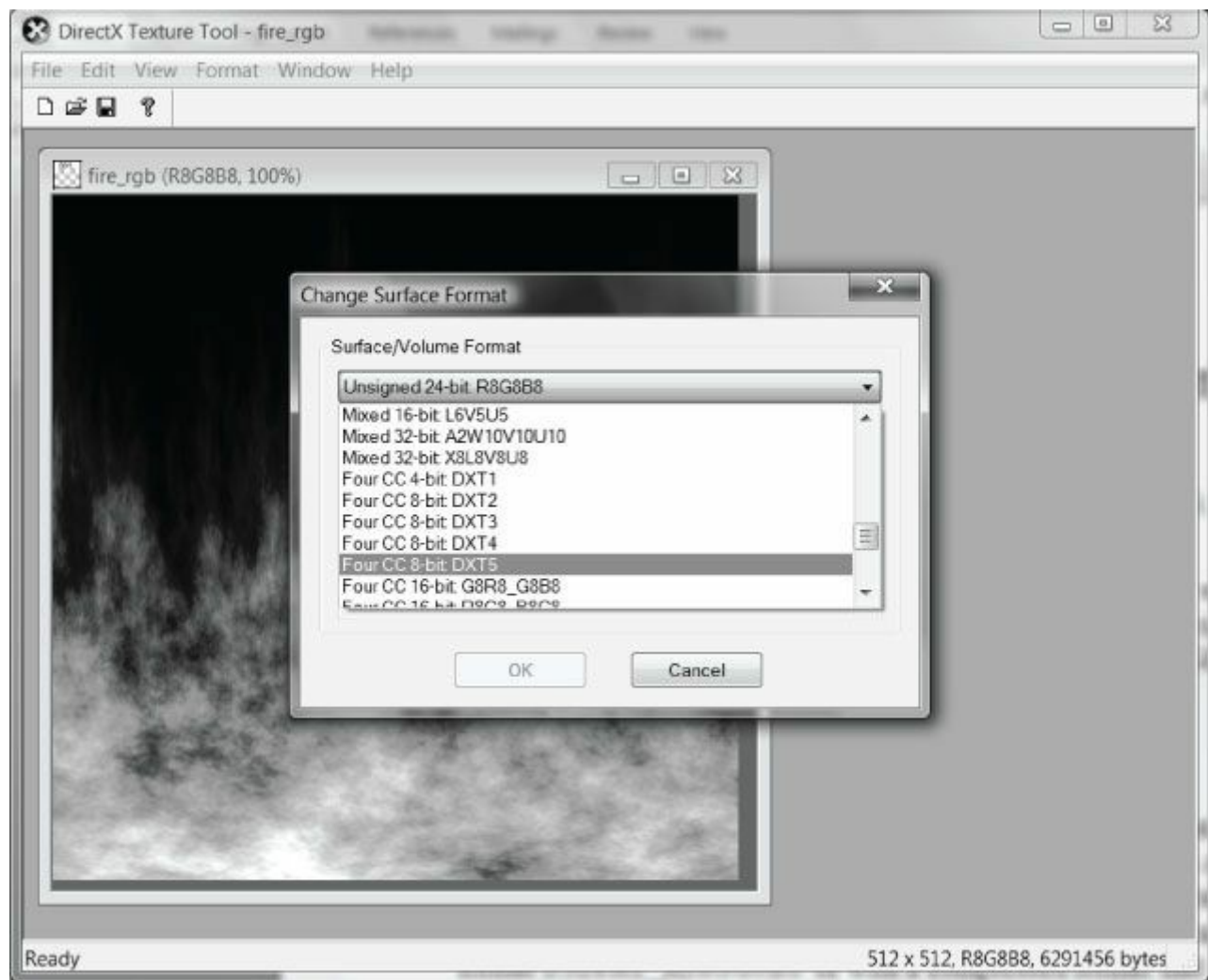


그림 9.7. 텍스처 형식 변경

이 작업은 알파 채널이 포함된 압축 텍스처를 생성합니다. 다음 작업은 알파 채널에 데이터를 로드하는 것입니다. 그림 9.6에 표시된 8비트 그레이스케일 맵을 알파 채널에 로드하겠습니다. 메뉴에서 **파일을** 선택한 후, 이 텍스처의 **알파 채널에 열기**를 선택하고, 이어서 **형식->리맵 생성**을 선택합니다. 알파 채널에 로드할 데이터가 포함된 이미지 파일을 찾아달라는 대화 상자가 나타납니다. 본 장의 데모 폴더에 있는 *fire_a.bmp* 파일을 선택합니다. 그림 9.8은 알파 채널 데이터가 삽입된 후의 프로그램 화면으로, 텍스처가 배경색과 투명하게 혼합된 상태를 보여줍니다. 배경색은 메뉴 바에서 **보기(View)**를 선택한 후 **배경색 변경(Change Background Color...)**을 선택하여 변경할 수 있습니다. 또한 **보기->알파 채널만 보기(View->Alpha Channel Only)**를 선택하면 알파 채널만 볼 수 있습니다.

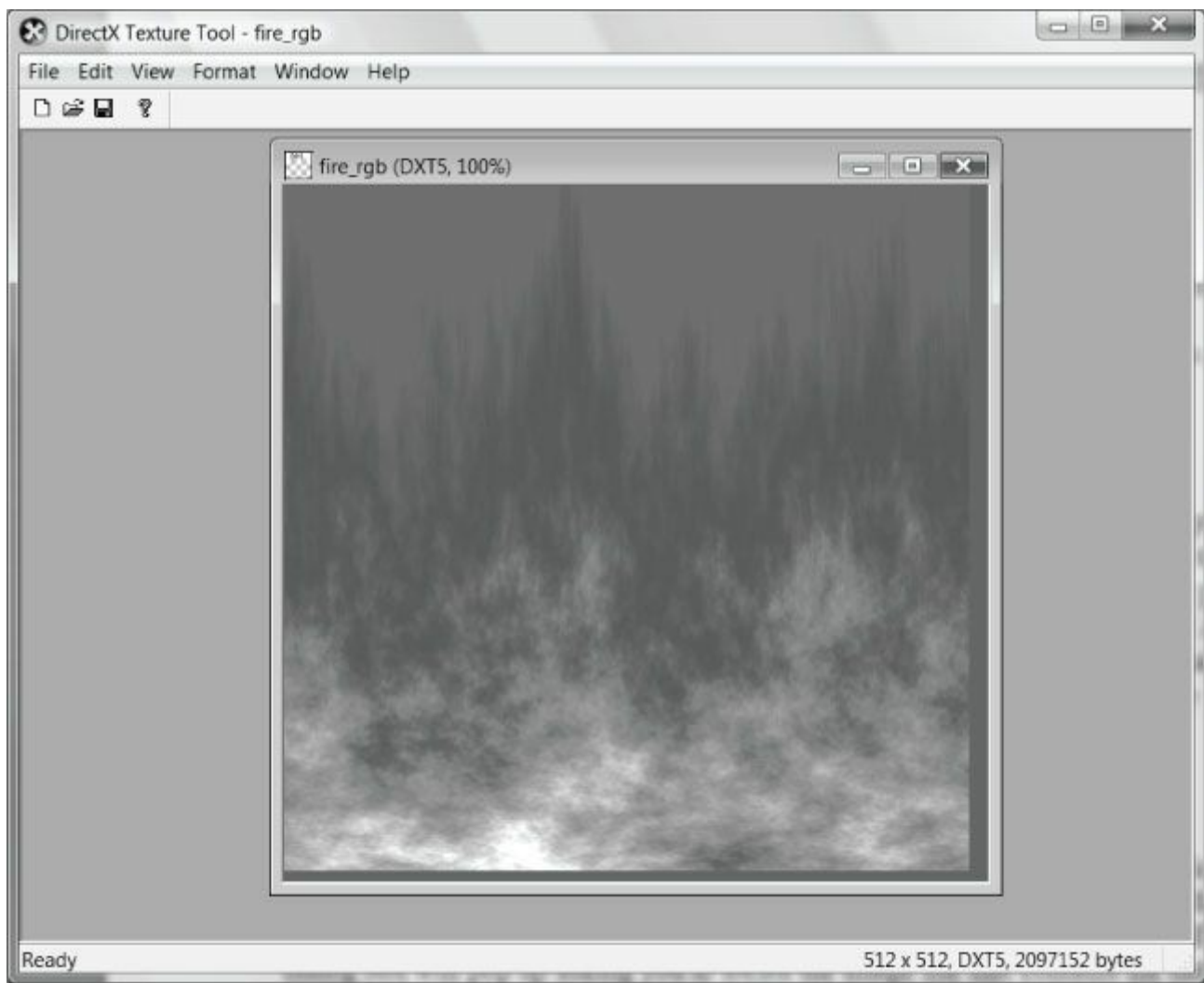


그림 9.8. 알파 채널이 적용된 결과 텍스처. 불 텍스처가 파란색 배경색에 투명하게 블렌딩되었습니다.

이제 원하는 이름(예: "fire.dds")으로 텍스처를 저장하세요.

9.7 픽셀 클리핑

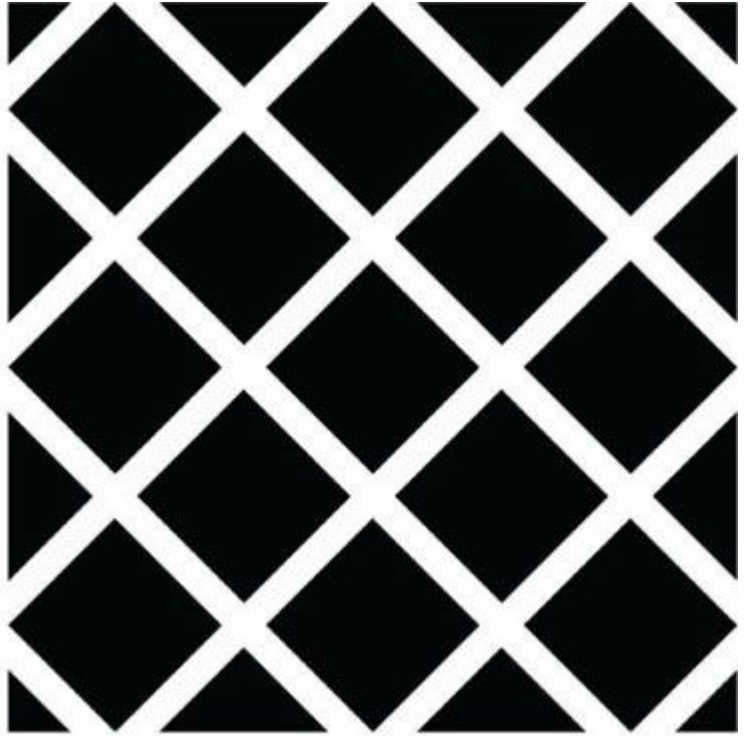
때로는 소스 픽셀을 추가 처리에서 완전히 제외하고 싶을 때가 있습니다. 이는 HLSL 내장 함수 **clip(x)**로 구현할 수 있습니다. 이 함수는 픽셀 셰이더에서만 호출 가능하며, $x < 0$ 일 경우 현재 픽셀을 추가 처리에서 제외합니다. 이 함수는 예를 들어 그림 9.9에 표시된 것과 같은 철조망 텍스처를 렌더링할 때 유용합니다. 즉, 픽셀이 완전히 불투명하거나 완전히 투명한 경우를 렌더링하는 데 유용합니다.

픽셀이 완전히 불투명하거나 완전히 투명한 경우에 유용합니다.

픽셀 셰이더에서 텍스처의 알파 성분을 추출합니다. 이 값이 0에 가까운 작은 값이라면 해당 픽셀이 완전히 투명하다는 의미이므로, 해당 픽셀을 추가 처리에서 제외합니다.



RGB Channels



Alpha Channel

그림 9.9. 알파 채널이 포함된 철조망 텍스처. 검은색 알파 값을 가진 픽셀은 클립 함수에 의해 거부되어 그려지지 않으므로, 오직 철조망만 남게 됩니다. 본질적으로 알파 채널은 텍스처에서 철조망이 아닌 픽셀을 가리기 위해 사용됩니다.

```
float4 PS(VertexOut 핀, uniform int gLightCount, uniform bool gUseTexture,
           uniform bool gAlphaClip, uniform bool gFogEnabled) : SV_Target
{
    // 보간된 법선은 비정규화될 수 있으므로 정규화합니다. 핀.NormalW = normalize(핀.NormalW);

    // toEye 벡터는 조명 계산에 사용됩니다. float3 toEye = gEye
    PosW - 핀.PosW;

    // 이 표면 점에서 눈까지의 거리를 캐시합니다. float distToEye = length(toEye);

    // 정규화합니다.
    toEye /= distToEye;

    // 기본값으로 곱셈 항등원 사용. float4 texColor =
    float4(1, 1, 1, 1); if(gUse Texture)
    {
        // 텍스처 샘플링.
        텍스처 색상 = gDiffuseMap.샘플(samAnisotropic, 핀.Tex);

        if(gAlphaClip)
        {
            // 텍스처 알파 < 0.1인 픽셀은 버림. 참고:
            // 가능한 한 빨리 이 테스트를 수행하여
            // 잠재적으로 셰이더를 조기에 종료하여
            // 나머지 셰이더 코드를 건너웁니다. clip(texColor.a -
            0.1f);
        }
    }
    ...
}
```

유니폼 부울 매개변수 **gAlphaClip**이 true일 때만 클리핑을 수행한다는 점에 유의하십시오. 이는 특정 지오메트리에 대해 클리핑을 호출하지 않으려는 경우가 있을 수 있기 때문입니다.

특정 지오메트리에 대해 클리핑을 호출하지 않으려는 경우가 있을 수 있으므로, 전용 셰이더를 통해 이를 켜고 끌 수 있어야 합니다.

블렌딩을 사용해도 동일한 결과를 얻을 수 있지만, 이 방법이 더 효율적입니다. 무엇보다도 블렌딩 계산이 필요하지 않기 때문입니다.

블렌딩은 비활성화할 수 있습니다. 또한 드로우 순서는 중요하지 않습니다. 더 나아가 픽셀 셰이더에서 픽셀을 조기에 버림으로써 나머지 픽셀 셰이더 명령어를 건너뛸 수 있습니다(버려진 픽셀에 대한 계산을 수행할 필요가 없음).

Note: 필터링으로 인해 알파 채널이 약간 흐려질 수 있으므로, 픽셀을 클리핑할 때 약간의 버퍼 공간을 남겨두어야 합니다. 예를 들어, 알파 값이 0에 가까운 픽셀은 클리핑하되, 반드시 정확히 0인 픽셀은 클리핑하지 않아도 됩니다.

그림 9.10은 "Blend" 데모의 스크린샷입니다. 이 데모는 투명도 블렌딩을 사용하여 반투명 물을 렌더링하고, 클립 테스트를 사용하여 철조망 상자를 렌더링합니다. 언급할 만한 또 다른 변경점은 이제 철조망 텍스처가 적용된 상자를 통해 볼 수 있게 되었으므로, 후면 제거를 비활성화해야 한다는 점입니다.

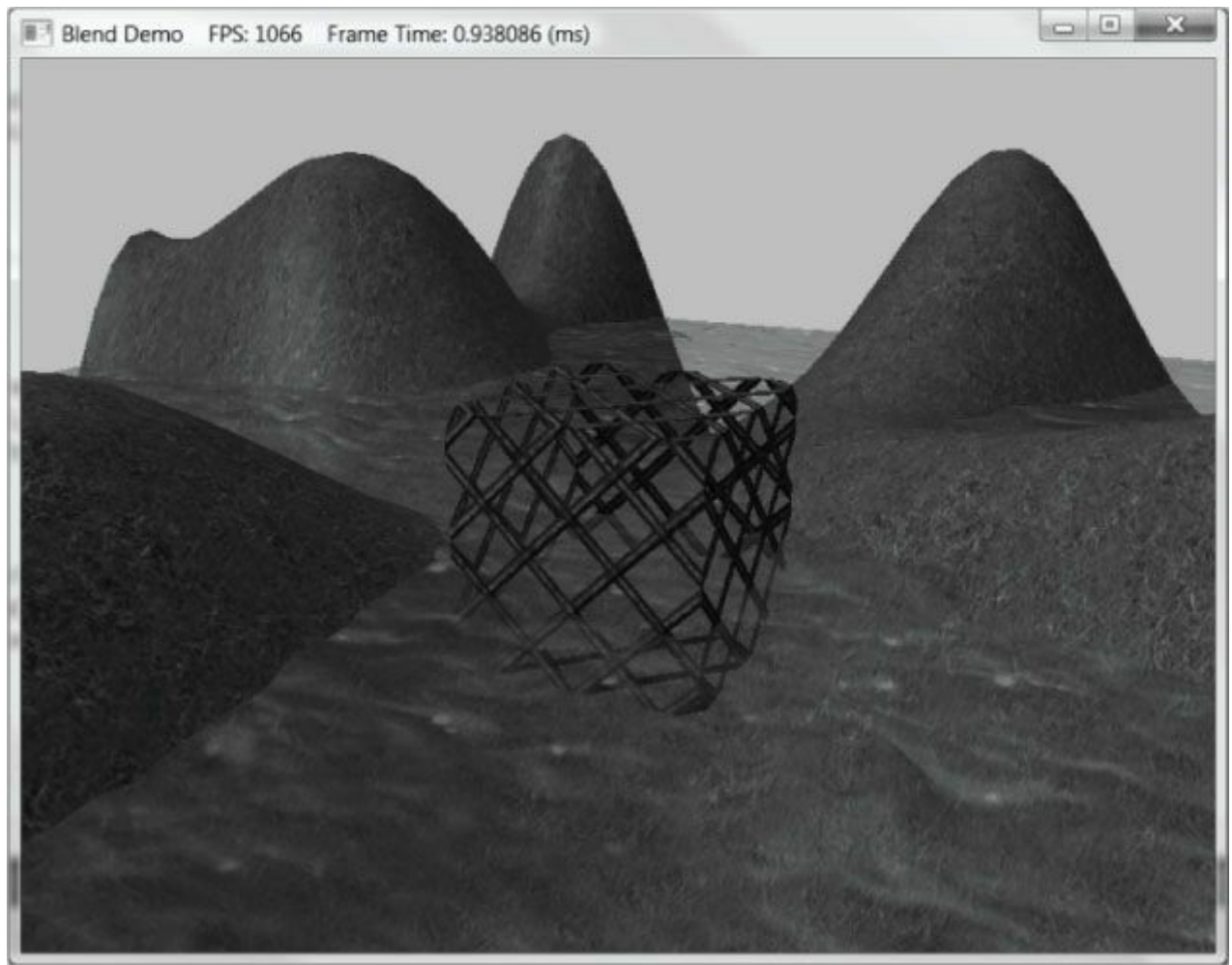


그림 9.10. "Blend" 데모의 스크린샷.

```
D3D11_RASTERIZER_DESC noCullDesc; ZeroMemory(&noCullDesc,
sizeof(D3D11_RASTERIZER_DESC)); noCullDesc.FillMode = D3D11_FILL_SOLID;
noCullDesc.CullMode = D3D11_CULL_NONE; noCullDesc.FrontCounterClockwise = false;
noCullDesc.DepthClipEnable = true;
```

```
ID3D11RasterizerState* NoCullRS;
HR(device->CreateRasterizerState(&noCullDesc, &NoCullRS));
```

...

// 펜스 텍스처에 투명한 영역이 있으므로

// 이를 통해 삼각형의 뒷면을 볼 수 있습니다.

// 따라서 이 경우에는 뒷면 제거를 원하지 않습니다. md3dImmediateContext-

>RSSetState(NoCullRS);

boxTech->GetPassByIndex(p)->Apply(0, md3dImmediateContext); md3dImmediateContext->DrawIndexed(36, 0, 0);

```
// 기본 렌더 상태 복원. md3dImmediateContext-
>RSSetState(0);
```

9.8 FOG

게임에서 특정 기상 조건을 시뮬레이션하려면 안개 효과를 구현할 수 있어야 합니다([그림 9.11](#) 참조). 안개는 명백한 목적 외에도 몇 가지 부수적인 이점을 제공합니다. 예를 들어, 먼 거리 렌더링 아티팩트를 가리고 *팝핑* 현상을 방지할 수 있습니다. 팝핑(popping)이란 카메라 이동으로 인해 원거리 평면 뒤에 있던 오브젝트가 갑자기 시야(frustum) 앞으로 들어와 보이게 되는 현상을 말합니다. 마치 장면에 갑자기 '튀어나온(pop)' 것처럼 보입니다. 원거리에 안개 층을 두면 이러한 팝핑 현상이 숨겨집니다. 참고로, 장면이 맑은 날에 설정된 경우에도 먼 거리에 미묘한 안개를 포함시키는 것이 좋습니다. 맑은 날에도 산과 같은 먼 물체는 깊이에 따라 흐릿해지고 대비가 떨어지는데, 안개를 활용하여 이러한 *대기 원근* 현상을 시뮬레이션할 수 있기 때문입니다.

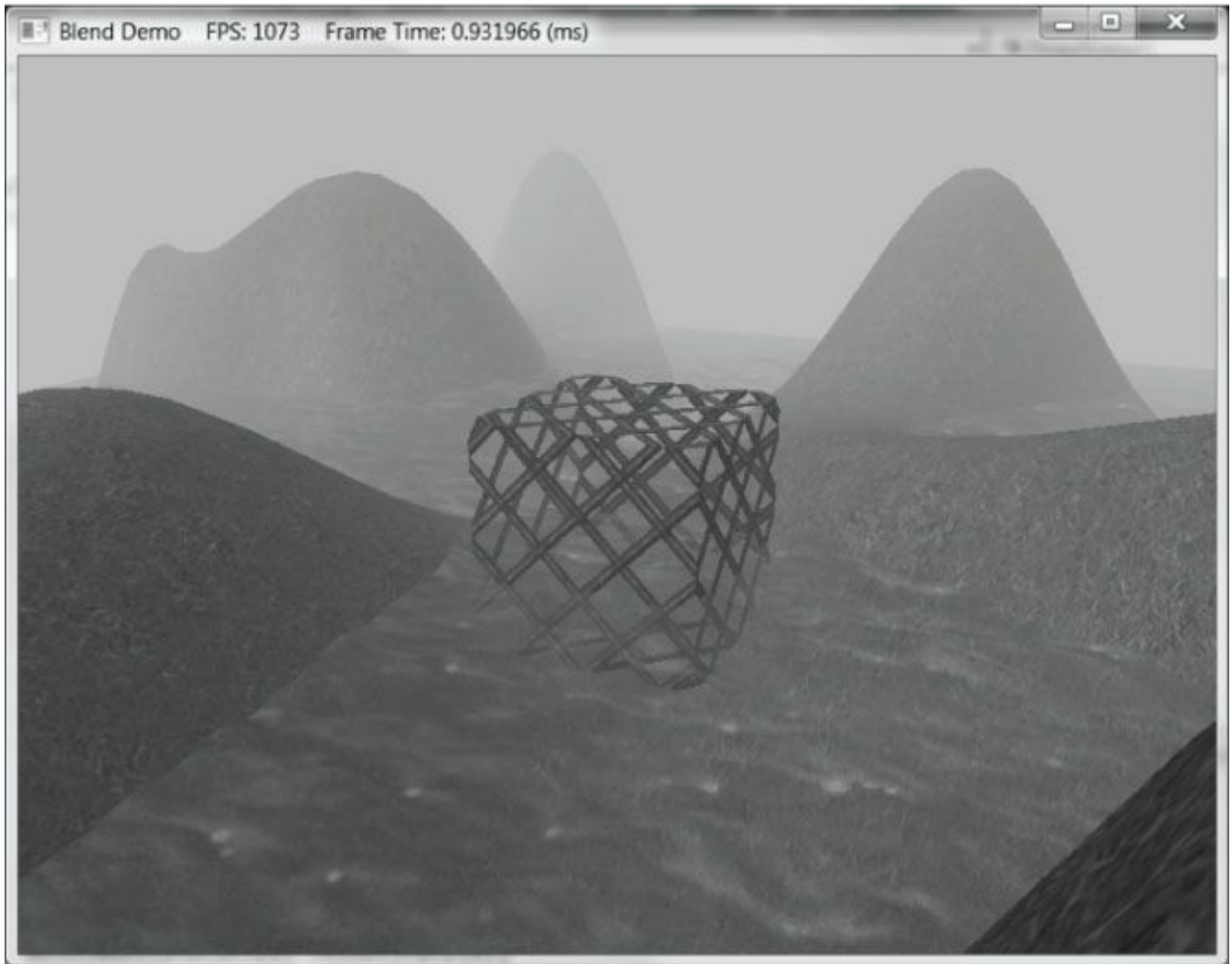


그림 9.11. 안개 효과가 적용된 "Blend" 데모 화면 캡처.

안개 구현 전략은 다음과 같습니다. 안개 색상, 카메라로부터의 안개 시작 거리, 안개 범위(즉, 안개 시작 거리부터 안개가 모든 물체를 완전히 가릴 때까지의 거리)를 지정합니다. 그런 다음 삼각형 상의 점의 색상은 해당 점의 일반 색상과 안개 색상의 가중 평균으로 계산됩니다:

$$\begin{aligned} \text{foggedColor} &= \text{litColor} + s(\text{fogColor} - \text{litColor}) \\ &= (1 - s) \cdot \text{litColor} + s \cdot \text{fogColor} \end{aligned}$$

매개변수 s 는 0에서 1 사이의 값을 가지며, 카메라 위치와 표면 점 사이의 거리에 대한 함수입니다. 표면 점과 시점 사이의 거리가 증가할수록 해당 점은 안개에 의해 점점 더 가려집니다. 매개변수 s 는 다음과 같이 정의됩니다:

$$s = \text{saturate} \left(\frac{\text{dist}(\mathbf{p}, \mathbf{E}) - \text{fogStart}}{\text{fogRange}} \right)$$

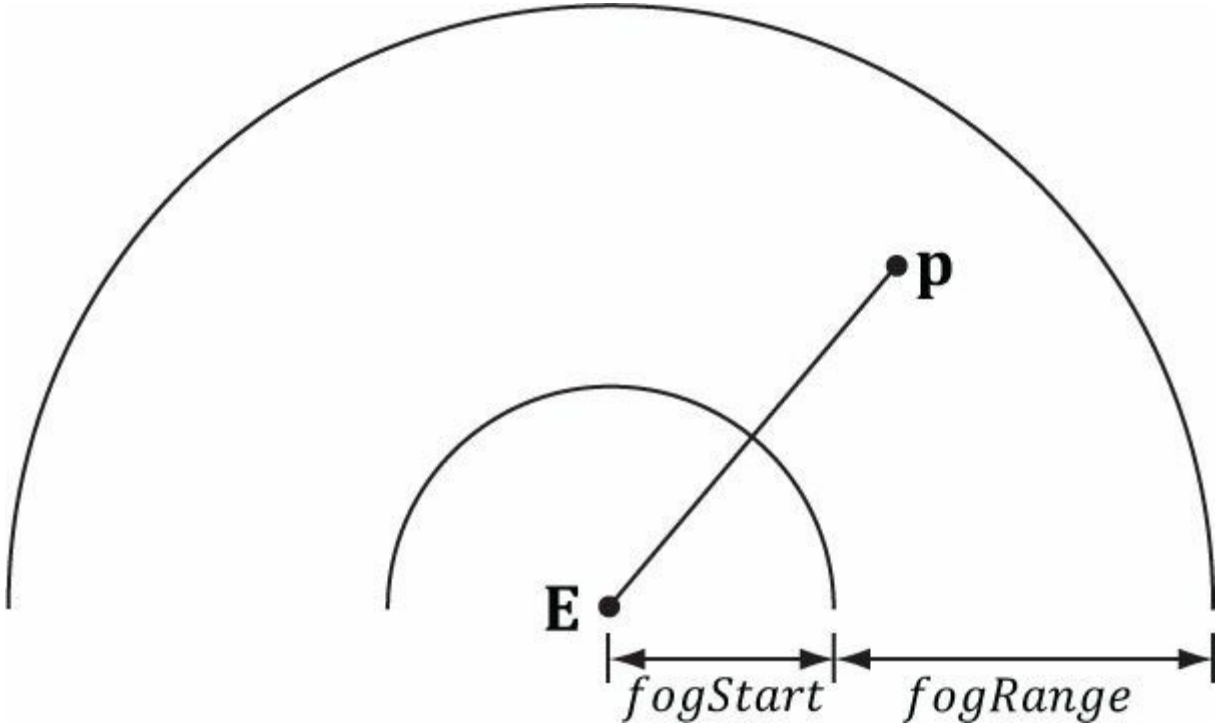


그림 9.12. 시선으로부터의 점 거리, 그리고 fogStart 및 fogRange 매개변수.

여기서 $\text{dist}(\mathbf{p}, \mathbf{E})$ 는 표면 점 $\mathbf{p}$ 와 카메라 위치 $\mathbf{E}$ 사이의 거리입니다. **saturate** 함수는 인자를 $[0, 1]$ 범위로 제한합니다:

$$\text{saturate}(x) = \begin{cases} x, & 0 \leq x \leq 1 \\ 0, & x < 0 \\ 1, & x > 1 \end{cases}$$

그림 9.13은 거리 함수로서의 s 를 나타낸 그래프입니다. $\text{dist}(\mathbf{p}, \mathbf{E}) \leq \text{fogStart}$ 일 때 $s = 0$ 이며, 안개 처리된 색상은 다음과 같이 주어집니다:

$$\text{foggedColor} = \text{litColor}$$

즉, 카메라로부터의 거리가 *fogStart*보다 작은 정점의 색상은 안개에 의해 수정되지 않습니다. 이는 "fogStart"라는 명칭에 부합하는 의미입니다. 카메라로부터의 거리가 최소한 *fogStart*만큼 떨어져야 안개가 색상에 영향을 미치기 시작합니다.

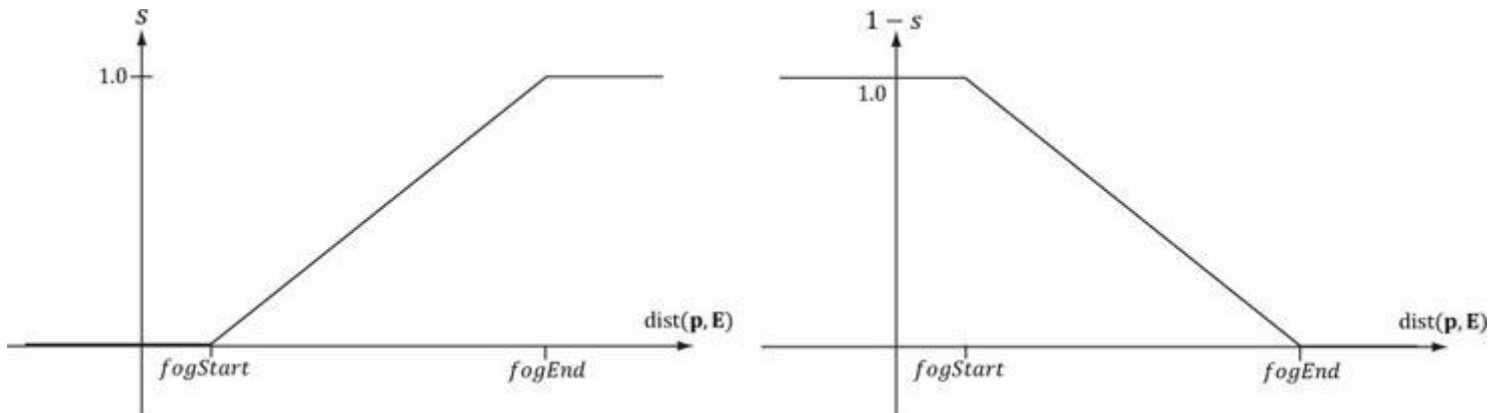


그림 9.13. (왼쪽) 거리 함수로서의 s (안개 색상 가중치) 플롯. (오른쪽) 거리 함수로서의 $1 - s$ (조명된 색상 가중치) 플롯. s 가 증가함에 따라 $(1 - s)$ 는 동일한 양만큼 감소합니다.

let $\text{fogEnd} = \text{fogStart} + \text{fogRange}$. $\text{dist}(\mathbf{p}, \mathbf{E}) \geq \text{fogEnd}$ 일 때, $s = 1$ 이며 안개 처리된 색상은 다음과 같이 주어집니다:

$$\text{흐림색상} = \text{안개색상}$$

즉, 안개는 *fogEnd* 이상 거리에서 표면 점을 완전히 가리므로 보이는 것은 오직 안개 색상뿐이다.

안개 시작점(fogStart) < 거리(**p**, **E**) < *안개 종료점(fogEnd)*일 때, 거리(**p**, **E**)가 *안개 시작점에서* 안개 종료점으로 증가함에 따라 *s*가 0에서 1로 선형적으로 증가하는 것을 볼 수 있습니다.

이는 거리가 증가함에 따라 안개 색상이 점점 더 큰 비중을 차지하고 원래 색상은 점점 더 작은 비중을 차지함을 의미합니다. 물론 이는 거리가 증가할수록 안개가 표면 점을 점점 더 가리기 때문에 당연한 결과입니다.

다음 셰이더 코드는 안개 구현 방식을 보여줍니다. 픽셀 단위에서 거리와 보간을 계산하며, 이는 조명된 색상을 계산한 후, 픽셀 단위로 거리와 보간값을 계산합니다.

```
cbuffer cbPerFrame
{
    DirectionalLight gDirLights[3]; float3 gEye
    PosW;

    // 애플리케이션이 프레임당 한 번씩 안개 매개변수를 변경할 수 있도록 허용합니다.

    // 예를 들어, 특정 시간대에만 안개를 사용할 수 있습니다. float gFogStart;
    float gFogRange; float4
    gFogColor;
};

struct VertexIn
{
    float3 PosL                : POSITION;
    float3 NormalL             : NORMAL;
    float2 텍스                : 텍스처 좌표;
};

struct VertexOut
{
    float4 PosH                : SV_POSITION;
    float3 PosW                : POSITION;
    float3 NormalW             : NORMAL;
    float2 텍스                : 텍스처 좌표;
};

VertexOut VS(VertexIn vin)
{
    VertexOut vout;

    // 월드 공간으로 변환.
    vout.PosW = mul(float4(vin.PosL, 1.0f), gWorld).xyz; vout.NormalW = mul(vin.NormalL,
(float3x3)gWorldInvTranspose);

    // 동차 클립 공간으로 변환.
    vout.PosH = mul(float4(vin.PosL, 1.0f), gWorldViewProj);

    // 삼각형 간 보간을 위한 출력 정점 속성. vout.Tex = mul(float4(vin.Tex, 0.0f, 1.0f),
gTexTransform).xy;

    return vout;
}

float4 PS(VertexOut pin, uniform int gLightCount, uniform bool
gUse Texture, uniform bool gAlphaClip, uniform bool gFogEnabled) : SV_Target
{

    // 보간된 노멀은 비정규화될 수 있으므로 정규화합니다. pin.NormalW = normalize(pin.NormalW);

    // toEye 벡터는 라이팅에 사용됩니다.
```

```

float3 toEye = gEye PosW - pin.PosW;

// 이 표면 점에서 눈까지의 거리를 캐시합니다. float distToEye = length(toEye);

// 정규화합니다.
toEye /= distToEye;

// 기본값으로 곱셈 항등원 설정. float4 texColor =
float4(1, 1, 1, 1); if(gUse Texture)
{

    // 텍스처 샘플링.
    텍스처 색상 = gDiffuseMap.샘플(samAnisotropic, pin.Tex);

    if(gAlphaClip)
    {
        // 텍스처 알파 값이 0.1 미만인 픽셀은 버립니다. 참고로
        // 이 테스트는 가능한 한 빨리 수행하여
        // 셰이더를 조기에 종료할 수 있도록 하여
        // 나머지 셰이더 코드를 건너뛸 수 있도록 합니다. clip(texColor.a
        - 0.1f);
    }
}

//
// 조명.
//

float4 litColor = texColor; if(gLightCount > 0)
{
    // 합계를 0으로 시작합니다.
    float4 ambient                = float4(0.0f, 0.0f, 0.0f, 0.0f);
    float4 확산                    = float4(0.0f, 0.0f, 0.0f, 0.0f);
    float4 spec                    = float4(0.0f, 0.0f, 0.0f, 0.0f);

    // 각 광원의 빛 기여도를 합산합니다. [unroll]
    for(int i = 0; i < gLightCount; ++i)
    {
        float4          A,          D,          S; Compute
        DirectionalLight(gMaterial, gDirLights[i],
        pin.NormalW, toEye, A, D, S);
        ambient += A; diffuse +=
        D; spec += S;
    }

    // 후기 합성으로 변조.
    litColor = texColor*(ambient + diffuse) + spec;
}
//
// 포깅
//

if( gFogEnabled )
{
    float fogLerp = saturate((distToEye - gFogStart) / gFogRange);

```

```
// 안개 색상과 조명 색상을 블렌딩합니다. litColor = lerp(litColor,
gFogColor, fogLerp);
}
```

```
// 디퓨즈 머티리얼과 텍스처에서 알파를 가져오는 공통 처리. litColor.a = gMaterial.Diffuse.a *
texColor.a;

return litColor;
}
```

Note

안개 계산에서 **distToEye** 값을 사용한다는 점을 유의하십시오. 이 값은 **toEye** 벡터를 정규화하기 위해 계산한 것과 동일합니다. 벡터를 정규화하기 위해 계산한 **distToEye** 값을 사용합니다. 덜 최적화된 구현 방식은 다음과 같았을 것입니다.

```
float3 toEye = normalize(gEyePosW - pin.PosW); float distToEye =
distance(gEyePosW, pin.PosW);
```

이는 본질적으로 **toEye** 벡터의 길이를 두 번 계산하는 것입니다. 한 번은 정규화 함수에서, 또 한 번은 거리 함수에서 계산합니다. **distance** 함수에서 다시 계산하는 것입니다.

Note

"Blend" 데모는 세 가지 렌더링 모드를 지원하며, '1', '2', '3' 키를 눌러 전환할 수 있습니다. 첫 번째 모드는 조명만 사용하여 장면을 렌더링합니다(그림 9.14 참조). 조명은 이미 다룬 주제이지만, 텍스처 없이 장면이 어떻게 보이는지 확인하는 데 유용합니다. 두 번째 모드는 조명과 텍스처링을 적용하여 장면을 렌더링합니다(그림 9.10). 세 번째 모드는 조명, 텍스처링, 안개 효과를 적용하여 장면을 렌더링합니다(그림 9.11). 이러한 렌더링 모드 전환은 효과 프레임워크 내 유니폼 매개변수를 활용해 특정 기능을 켜고 끄는 맞춤형 셰이더를 생성하는 방식을 보여줍니다. 독자는 Basic.fx 파일의 어셈블리 언어 출력 결과를 다시 살펴보는 것이 좋습니다. 또한 조명만 적용한 경우, 조명과 텍스처링을 함께 적용한 경우, 조명 텍스처링 안개를 모두 적용한 경우의 프레임 속도를 비교해 보는 것도 흥미로운 분석이 될 것입니다.

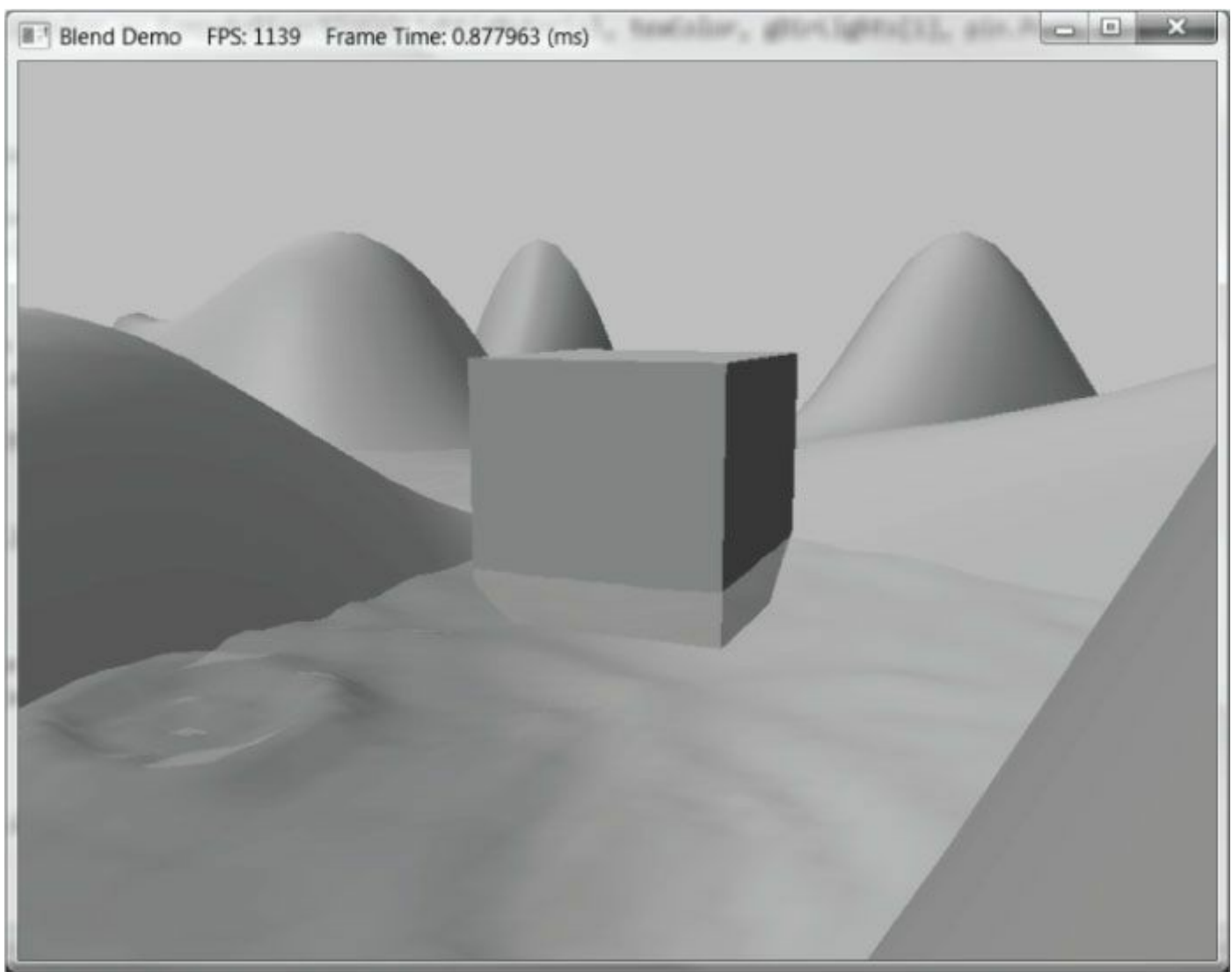


그림 9.14. 조명만 활성화된 "Blend" 데모.

9.9 요약

1. 블렌딩은 현재 래스터화 중인 픽셀(소위 소스 픽셀)을 백 버퍼에 미리 래스터화된 픽셀(소위 대상 픽셀)과 혼합(결합)하는 기술입니다. 이 기술을 통해 물이나 유리 같은 반투명 객체를 렌더링하는 등 다양한 효과를 구현할 수 있습니다.
2. 블렌딩 방정식은 다음과 같습니다.

$$\begin{aligned} C &= C_{src} \otimes F_{src} \boxplus C_{dst} \otimes F_{dst} \\ A &= A_{src} F_{src} \boxplus A_{dst} F_{dst} \end{aligned}$$

RGB 구성 요소는 알파 구성 요소와 독립적으로 블렌딩된다는 점에 유의하십시오. $\boxplus$ 이진 연산자는 **D3D11_BLEND_OP** 열거형 유형으로 정의된 연산자 중 하나일 수 있습니다.

3. F_{src} , F_{dst} , F_{src} , F_{dst} 는 블렌딩 팩터라고 불리며, 블렌딩 방정식을 사용자 정의하는 수단을 제공합니다. 이들은 **D3D11_BLEND** 열거형의 멤버일 수 있습니다. 알파 블렌딩 방정식의 경우, **_COLOR**로 끝나는 블렌딩 팩터는 *허용되지* 않습니다.
4. 소스 알파 정보는 확산 재질에서 비롯됩니다. 본 프레임워크에서 확산 재질은 텍스처 맵으로 정의되며, 텍스처의 알파 채널이 알파 정보를 저장합니다.
5. 소스 픽셀은 내장 HLSL **clip(x)** 함수를 사용하여 추가 처리에서 완전히 거부될 수 있습니다. 이 함수는 픽셀 셰이더에서만 호출할 수 있으며, $x < 0$ 일 경우 현재 픽셀을 추가 처리에서 제외합니다. 이 함수는 특히 픽셀이 완전히 불투명하거나 완전히 투명한 경우(알파 값이 0에 가까운 완전히 투명한 픽셀을 거부하는 데 사용됨) 효율적으로 렌더링하는 데 유용합니다.
6. 안개를 사용하여 다양한 기상 효과와 대기 원근법을 모델링하여 먼 렌더링 아티팩트를 숨기고 팝핑 현상을 완화합니다.

우리의 선형 안개 모델에서는 안개 색상, 카메라로부터의 안개 시작 거리, 안개 범위를 지정합니다. 삼각형 상의 점의 색상은 해당 점의 일반 색상과 안개 색상의 가중 평균으로 계산됩니다:

$$\begin{aligned}foggedColor &= litColor + s(fogColor - litColor) \\ &= (1 - s) \cdot litColor + s \cdot fogColor\end{aligned}$$

매개변수 s 는 0에서 1 사이의 값을 가지며, 카메라 위치와 표면 점 사이의 거리에 대한 함수입니다. 표면 점과 시점 사이의 거리가 증가할수록 해당 점은 안개에 의해 점점 더 가려지게 됩니다.

9.10 연습 문제

1. "투명한 물" 데모를 수정하여 물 그리기 전에 다음 줄을 사용하십시오:

```
md3dImmediateContext->OMSetBlendState( TransparentBS, blendFactors, 0xffffffff);
```

이렇게 하면 첫 번째 샘플을 비활성화하는 첫 번째 비트가 꺼집니다. 멀티샘플링을 사용하지 않기 때문에(이는 하나의 샘플로 멀티샘플링을 사용하는 것과 유사함), 이로 인해 물 픽셀이 그려지지 않게 됩니다.

2. 다양한 블렌드 연산과 블렌드 팩터 조합을 실험해 보세요.

3. "Blend" 데모를 수정하여 물을 먼저 그립니다. 결과를 설명하세요.

4. 안개 시작점($fogStart$) = 10, 안개 범위($fogRange$) = 200이라고 가정합니다. 다음 조건에서 안개 처리된 색상($foggedColor$)을 계산하세요.

```
) dist(p, E) = 160
```

```
) dist(p, E) = 110
```

```
) dist(p, E) = 60
```

```
) dist(p, E) = 30
```

5. 생성된 셰이더 어셈블리를 살펴보면, **discard** 명령어가 없는 **gAlphaClip=false** 기술과 **discard** 명령어가 있는 **gAlphaClip=true** 기술을 확인하십시오. **discard** 명령어는 HLSL **clip** 명령어에 해당합니다.

6. "Blend" 데모를 수정하여 빨간색 및 녹색 색상 채널에 대한 색상 쓰기를 비활성화하는 블렌드 렌더 상태를 생성하고 적용하십시오.

제 10장 스텐실링

스텐실 버퍼는 특수 효과를 구현하기 위해 사용할 수 있는 오프스크린 버퍼입니다. 스텐실 버퍼는 백 버퍼 및 깊이 버퍼와 동일한 해상도를 가지며, 스텐실 버퍼의 ij 번째 픽셀은 백 버퍼와 깊이 버퍼의 ij 번째 픽셀과 대응됩니다. §4.1.5에서 언급했듯이 스텐실 버퍼가 지정되면 깊이 버퍼에 연결됩니다. 이름에서 알 수 있듯이 스텐실 버퍼는 스텐실 역할을 하여 특정 픽셀 조각이 백 버퍼로 렌더링되는 것을 차단할 수 있게 합니다.

예를 들어 거울을 구현할 때, 우리는 거울 평면을 가로질러 물체를 반사해야 하지만, 거울에 반사된 모습만 그릴 수 있습니다. 스텐실 버퍼를 사용하면 거울에 그려지는 경우가 아닌 한 반사 렌더링을 차단할 수 있습니다(그림 10.1 참조).

스텐실 버퍼(및 깊이 버퍼)는 `ID3D11DepthStencilState` 인터페이스를 통해 제어됩니다. 블렌딩과 마찬가지로 이 인터페이스는 유연하고 강력한 기능 세트를 제공합니다. 스텐실 버퍼를 효과적으로 사용하는 법을 배우는 가장 좋은 방법은 기존 예제 애플리케이션을 연구하는 것입니다. 스텐실 버퍼의 몇 가지 적용 사례를 이해하면 자신의 특정 요구 사항에 맞게 이를 활용하는 방법을 더 잘 파악할 수 있을 것입니다.

목표:

1. `ID3D11DepthStencilState` 인터페이스를 사용하여 깊이 버퍼 및 스텐실 버퍼 설정을 제어하는 방법을 알아봅니다.

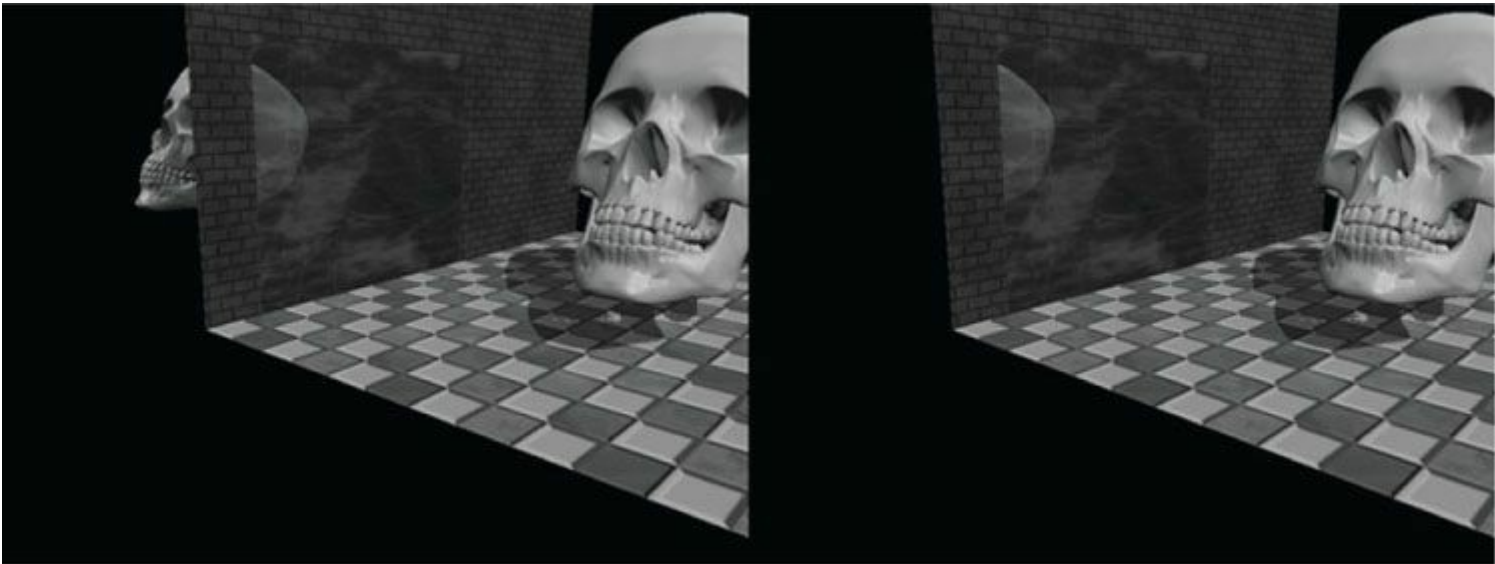


그림 10.1. (왼쪽) 반사된 두개골이 거울에 제대로 비춰집니다. 이 영역에서 깊이 테스트를 통과하지 못했기 때문에 벽 벽들을 통해 반사가 보이지 않습니다. 그러나 벽 뒤를 보면 반사가 보이므로 환상이 깨집니다(반사는 거울을 통해서만 나타나야 함). (오른쪽) 스텐실 버퍼를 사용하면 거울에 그려지지 않는 한 반사된 두개골이 렌더링되는 것을 차단할 수 있습니다.

2. 스텐실 버퍼를 사용하여 비반사 표면에 반사광이 그려지는 것을 방지하는 미래 구현 방법을 학습합니다.
3. 더블 블렌딩을 식별하고 스텐실 버퍼가 이를 방지하는 방법을 이해하기 위한.
4. 깊이 복잡성을 설명하고 장면의 깊이 복잡성을 측정할 수 있는 두 가지 방법을 기술합니다.

10.1 깊이/스텐실 형식 및 초기화

깊이/스텐실 버퍼가 텍스처임을 상기하면, 특정 데이터 형식으로 생성되어야 합니다. 깊이/스텐실 버퍼링에 사용되는 형식은 다음과 같습니다:

1. `DXGI_FORMAT_D32_FLOAT_S8X24_UINT`: 32비트 부동 소수점 깊이 버퍼를 지정하며, 스텐실 버퍼에 매핑된 8비트(부호 없는 정수)는 $[0, 255]$ 범위로 예약되고, 패딩을 위해 사용되지 않는 24비트가 포함됩니다.
2. `DXGI_FORMAT_D24_UNORM_S8_UINT`: $[0, 1]$ 범위에 매핑된 부호 없는 24비트 깊이 버퍼를 지정하며, 8비트(부호 없는 정수)는 $[0, 255]$ 범위에 매핑된 스텐실 버퍼용으로 예약됩니다.

`D3DApp` 프레임워크에서 깊이 버퍼를 생성할 때 다음과 같이 지정합니다:

depthStencilDesc.Format = DXGI_FORMAT_D24_UNORM_S8_UINT;

또한 스텔실 버퍼는 각 프레임 시작 시 특정 값으로 재설정되어야 합니다. 이는 다음 메서드로 수행됩니다(동시에 깊이 버퍼도 지웁니다):

```
void ID3D11DeviceContext::ClearDepthStencilView( ID3D11DepthStencilView
    *pDepthStencilView, UINT ClearFlags, FLOAT Depth, UINT8 Stencil);
```

- 1. **pDepthStencilView**: 지우려는 깊이/스텐실 버퍼 뷰에 대한 포인터.
- 2. **ClearFlags**: 깊이 버퍼만 지우려면 **D3D11_CLEAR_DEPTH**를 지정합니다. 스텔실 버퍼만 지우려면 **D3D11_CLEAR_STENCIL**을 지정합니다. 둘 다 지우려면 **D3D11_CLEAR_DEPTH | D3D11_CLEAR_STENCIL**을 지정합니다.
- 3. **Depth**: 깊이 버퍼의 각 픽셀에 설정할 부동 소수점 값입니다. $0 \leq x \leq 1$ 을 만족하는 부동 소수점 숫자 x 여야 합니다.
- 4. **스텐실**: 스텔실 버퍼의 각 픽셀을 설정할 정수 값입니다. $0 \leq n \leq 255$ 를 만족하는 정수 n 이여야 합니다. 우리는 이미 데모에서 매 프레임마다 이 메서드를 호출해 왔 습니다. 예를 들어:

```
void MirrorApp::DrawScene()
{
    md3dImmediateContext->ClearRenderTargetView( mRenderTargetView,
        reinterpret_cast<const float*>(&Colors::Black));

    md3dImmediateContext->ClearDepthStencilView( mDepthStencilView,
        D3D11_CLEAR_DEPTH|D3D11_CLEAR_STENCIL, 1.0f, 0);
}
```

10.2 스텔실 테스트

앞서 언급한 바와 같이, 스텔실 버퍼를 사용하여 백 버퍼의 특정 영역에 대한 렌더링을 차단할 수 있습니다. 특정 픽셀의 쓰기 작업을 차단할지 여부는 *스텐실 테스트*에 의해 결정되며, 이는 다음과 같 이 주어집니다:

```
if(StencilRef & StencilReadMask <= Value & StencilReadMask) 픽셀 허용
else
    픽셀 거부
```

스텐실 테스트는 스텔실링이 활성화된 상태에서 픽셀이 래스터화될 때(즉, 출력 병합 단계 중) 수행되며, 두 개의 피연산자를 취합니다:

- 1. 응용 프로그램에서 정의한 *스텐실 참조 값* (**StencilRef**)과 응용 프로그램에서 정의한 *마스킹 값* (**StencilReadMask**)을 AND 연산하여 결정되는 왼쪽 피연산자(LHS).
- 2. 우측(RHS) 피연산자는 테스트 중인 특정 픽셀의 스텔실 버퍼에 이미 존재하는 항목(**Value**)과 애플리케이션 정의 마스킹 값(**StencilReadMask**)을 AND 연산하여 결정됩니다.

StencilReadMask는 LHS와 RHS 모두 동일합니다. 스텔실 테스트는 애플리케이션이 선택한 *비교 함수*로 지정된 LHS와 RHS를 비교하여 참 또는 거짓 값을 반환합니다. 테스트 결과가 참 으로 평가되면(깊이 테스트도 통과한 경우) 해당 픽셀을 백 버퍼에 기록합니다. 테스트 결과가 거짓으로 평가되면 해당 픽셀이 백 버퍼에 기록됩니다. 물론, 스텔실 테스트에 실패하여 픽셀이 거부된 경우, 해당 픽셀은 깊이 버퍼에도 기록되지 않습니다.

$\leq$ 연산자는 **D3D11_COMPARISON_FUNC** 열거형에 정의된 함수 중 하나입니다:

```
typedef enum D3D11_COMPARISON_FUNC
{
    D3D11_COMPARISON_NEVER = 1,
    D3D11_COMPARISON_LESS = 2,
    D3D11_COMPARISON_EQUAL = 3,
    D3D11_COMPARISON_LESS_EQUAL = 4,
    D3D11_COMPARISON_GREATER = 5,
    D3D11_COMPARISON_NOT_EQUAL = 6,
```

```
        D3D11_COMPARISON_GREATER_EQUAL=7,  
        D3D11_COMPARISON_ALWAYS=8,  
    } D3D11_COMPARISON_FUNC;
```

1. **D3D11_COMPARISON_NEVER**: 함수는 항상 `false`를 반환합니다.
2. **D3D11_COMPARISON_LESS**: $\leq$ 연산자를 $<$ 연산자로 대체합니다.
3. **D3D11_COMPARISON_EQUAL**: $\leq$ 을 $=$ 연산자로 대체합니다.
4. **D3D11_COMPARISON_LESS_EQUAL**: $\leq$ 을 $\leq$ 연산자로 대체합니다.
5. **D3D11_COMPARISON_GREATER**: $\leq$ 을 $>$ 연산자로 대체합니다.
6. **D3D11_COMPARISON_NOT_EQUAL**: $\leq$ 을 $\neq$ 연산자로 대체합니다.
7. **D3D11_COMPARISON_GREATER_EQUAL**: $\leq$ 을 $\geq$ 연산자로 대체합니다.
8. **D3D11_COMPARISON_ALWAYS**: 함수는 항상 `true`를 반환합니다.

10.3 깊이/스텐실 상태 블록

ID3D11DepthStencilState 인터페이스를 생성하는 첫 번째 단계는 **D3D11_DEPTH_STENCIL_DESC** 인스턴스를 채우는 것입니다.

```
typedef struct D3D11_DEPTH_STENCIL_DESC {  
    BOOL  
        DepthEnable; // 기본값: True  
  
    // 기본값: D3D11_DEPTH_WRITE_MASK_ALL  
    D3D11_DEPTH_WRITE_MASK DepthWriteMask;  
  
    // 기본값: D3D11_COMPARISON_LESS  
    D3D11_COMPARISON_FUNC DepthFunc;  
  
    BOOL StencilEnable; // 기본값: False  
    UINT8  
        StencilReadMask; // 기본값: 0xff  
    UINT8 StencilWriteMask;  
  
    // 기본값: 0xff  
    D3D11_DEPTH_STENCIL_OP_DESC FrontFace;  
    D3D11_DEPTH_STENCIL_OP_DESC Back Face;  
} D3D11_DEPTH_STENCIL_DESC;
```

10.3.1 깊이 설정

1. **DepthEnable**: 깊이 버퍼링을 활성화하려면 `true`를 지정하고, 비활성화하려면 `false`를 지정합니다.

깊이 테스트가 비활성화된 경우, 드로우 순서가 중요하며, 가림 객체 뒤에 있더라도 픽셀 프래그먼트가 그려집니다 (§4.1.5 참조). 깊이 버퍼링이 비활성화된 경우, `DepthWriteMask` 설정과 관계없이 깊이가 버퍼의 요소도 업데이트되지 않습니다.

2. **DepthWriteMask** : **D3D11_DEPTH_WRITE_MASK_ZERO** 또는 **D3D11_DEPTH_WRITE_MASK_ALL** 중 하나만 지정할 수 있습니다. **DepthEnable**이 `true`로 설정된 경우, **D3D11_DEPTH_WRITE_MASK_ZERO**는 깊이 버퍼에 대한 쓰기를 비활성화하지만 깊이 테스트는 여전히 수행됩니다. **D3D11_DEPTH_WRITE_MASK_ALL**은 깊이 버퍼에 대한 쓰기를 활성화합니다. 깊이가 테스트와 스텐실 테스트가 모두 통과할 경우 새로운 깊이가 값이 기록됩니다. 특정 특수 효과를 구현하기 위해서는 깊이를 읽기 및 쓰기 제어 기능이 필수적입니다.
3. **DepthFunc**: 깊이 테스트 비교 함수를 정의하기 위해 **D3D11_COMPARISON_FUNC** 열거형의 멤버 중 하나를 지정합니다. 일반적으로 이는 항상 **D3D11_COMPARISON_LESS**이며, §4.1.5에 설명된 대로 일반적인 깊이가 테스트가 수행됩니다. 즉, 픽셀 프래그먼트는 그 깊이가 값이 백 버퍼에 쓰여진 이전 픽셀의 깊이보다 작을 경우에만 허용됩니다. 그러나 보시다시피 **Direct3D**는 필요한 경우 깊이를 테스트를 사용자 정의할 수 있도록 허용합니다.

10.3.2 스텐실 설정

1. **StencilEnable**: 스텐실 테스트를 활성화하려면 `true`를 지정하고, 비활성화하려면 `false`를 지정합니다.
2. **StencilReadMask** : 스텐실 테스트에 사용되는 스텐실 읽기 마스크:

```
if(StencilRef & StencilReadMask <= Value & StencilReadMask) 픽셀 허용  
else  
    픽셀 거부
```

기본값은 어떤 비트도 마스크하지 않습니다:

```
#define D3D11_DEFAULT_STENCIL_READ_MASK (0xff)
```

3. **StencilWriteMask** : 스텐실 버퍼가 업데이트될 때, 쓰기 마스크를 사용하여 특정 비트가 쓰여지는 것을 차단할 수 있습니다. 예를 들어, 상위 4비트가 쓰여지는 것을 방지하려면 0x0f 쓰기 마스크를 사용할 수 있습니다. 기본값은 어떤 비트도 차단하지 않습니다:

```
#define D3D11_DEFAULT_STENCIL_WRITE_MASK (0xff)
```

4. **FrontFace**: 정면 삼각형에 대한 스텐실 버퍼 작동 방식을 나타내는 완성된 **D3D11_DEPTH_STENCIL_OP_DESC** 구조체입니다.
5. **Back Face**: 후면 삼각형에 대한 스텐실 버퍼 작동 방식을 나타내는 완성된 **D3D11_DEPTH_STENCIL_OP_DESC** 구조체.

```
typedef struct D3D11_DEPTH_STENCIL_OP_DESC {  
    D3D11_STENCIL_OP StencilFailOp;                // 기본값: D3D11_STENCIL_OP_KEEP D3D11_STENCIL_OP  
    StencilDepthFailOp;                            // 기본값: D3D11_STENCIL_OP_KEEP D3D11_STENCIL_OP  
    StencilPassOp;                                // 기본값: D3D11_STENCIL_OP_KEEP  
    D3D11_COMPARISON_FUNC StencilFunc;            // 기본값: D3D11_COMPARISON_ALWAYS  
} D3D11_DEPTH_STENCIL_OP_DESC;
```

1. **StencilFailOp**: 픽셀 프래그먼트에 대해 스텐실 테스트가 실패할 때 스텐실 버퍼가 어떻게 업데이트되어야 하는지를 설명하는 **D3D11_STENCIL_OP** 열거형의 멤버입니다.
2. **StencilDepthFailOp**: **D3D11_STENCIL_OP** 열거형의 멤버로, 스텐실 테스트는 통과했으나 픽셀 프래그먼트에 대한 깊이 테스트가 실패했을 때 스텐실 버퍼가 어떻게 업데이트되어야 하는지를 설명합니다.
3. **StencilPassOp**: **D3D11_STENCIL_OP** 열거형의 멤버로, 픽셀 조각에 대해 스텐실 테스트와 깊이 테스트가 모두 통과할 때 스텐실 버퍼를 업데이트하는 방법을 설명합니다.
4. **StencilFunc**: 스텐실 테스트 비교 함수를 정의하는 **D3D11_COMPARISON_FUNC** 열거형의 멤버입니다.

```
typedef enum D3D11_STENCIL_OP {  
    D3D11_STENCIL_OP_KEEP                = 1,  
    D3D11_STENCIL_OP_ZERO                = 2,  
    D3D11_STENCIL_OP_REPLACE              = 3,  
    D3D11_STENCIL_OP_INCR_SAT = 4,  
    D3D11_STENCIL_OP_DECR_SAT = 5,  
    D3D11_STENCIL_OP_INVERT              = 6,  
    D3D11_STENCIL_OP_INCR                = 7,  
    D3D11_STENCIL_OP_DECR                = 8  
} D3D11_STENCIL_OP;
```

1. **D3D11_STENCIL_OP_KEEP**: 스텐실 버퍼를 변경하지 않도록 지정합니다. 즉, 현재 존재하는 값을 유지합니다.
2. **D3D11_STENCIL_OP_ZERO**: 스텐실 버퍼 항목을 0으로 설정하도록 지정합니다.
3. **D3D11_STENCIL_OP_REPLACE**: 스텐실 버퍼 항목을 스텐실 테스트에 사용된 스텐실 참조 값(**StencilRef**)으로 대체하도록 지정합니다. **StencilRef** 값은 깊이/스텐실 상태 블록을 렌더링 파이프라인에 바인딩할 때 설정됩니다 (§10.3.3).
4. **D3D11_STENCIL_OP_INCR_SAT**: 스텐실 버퍼 항목을 증가시킵니다. 증가된 값이 최대값(예: 8비트 스텐실 버퍼의 경우 255)을 초과하면 해당 항목을 최대값으로 클램프합니다.
5. **D3D11_STENCIL_OP_DECR_SAT**: 스텐실 버퍼 항목을 감소시킵니다. 감소된 값이 0보다 작으면 항목을 0으로 클램프합니다.
6. **D3D11_STENCIL_OP_INVERT**: 스텐실 버퍼 항목의 비트를 반전하도록 지정합니다.
7. **D3D11_STENCIL_OP_INCR**: 스텐실 버퍼 항목을 증가시킵니다. 증가된 값이 최대값(예: 8비트 스텐실 버퍼의 경우 255)을 초과하면 0으로 감싸집니다.
8. **D3D11_STENCIL_OP_DECR**: 스텐실 버퍼 항목을 감소시킵니다. 감소된 값이 0보다 작으면 허용되는 최대값으로 감싸집니다.

전면 삼각형과 후면 삼각형에 대한 스텐실 동작이 다를 수 있음을 유의하십시오. 후면 삼각형의 경우

백면 제거(back face culling)로 인해 뒤쪽을 향한 폴리곤을 렌더링하지 않는 경우 설정은 무의미합니다. 그러나  특정 그래픽 알고리즘이나 투명 지오메트리(예: 철조망 상자처럼 상자를 통해 뒤쪽이 보이는 경우)를 위해 뒤쪽을 향한 폴리곤을 렌더링해야 할 때가 있습니다. 이러한 경우 백면 설정(Back Face settings)이 적용됩니다. 설정이 중요해집니다.

10.3.3 깊이/스텐실 상태 생성 및 바인딩

D3D11_DEPTH_STENCIL_DESC 구조체를 작성한 후, 다음 메서드를 사용하여 ID3D11DepthStencilState 인터페이스에 대한 포인터를 다음과 같은 메서드로 얻을 수 있습니다:

```
HRESULT ID3D11Device::CreateDepthStencilState(
    const D3D11_DEPTH_STENCIL_DESC *pDepthStencilDesc, ID3D11DepthStencilState
    **ppDepthStencilState);
```

- 1. pDepthStencilDesc: 생성하려는 깊이/스텐실 상태 블록을 설명하는, 작성된 D3D11_DEPTH_STENCIL_DESC 구조체에 대한 포인터입니다.
- 2. ppDepthStencilState: 생성된 ID3D11DepthStencilState 인터페이스에 대한 포인터를 반환합니다.

ID3D11DepthStencilState 인터페이스가 생성되면 다음 메서드를 사용하여 파이프라인의 출력 병합 단계에 바인딩합니다:

```
void ID3D11DeviceContext::OMSetDepthStencilState( ID3D11DepthStencilState
    *pDepthStencilState, UINT StencilRef);
```

- 1. pDepthStencilState: 설정할 깊이/스텐실 상태 블록에 대한 포인터입니다.
- 2. StencilRef: 스텐실 테스트에 사용할 32비트 스텐실 참조 값.

다른 상태 그룹과 마찬가지로 기본 깊이/스텐실 상태가 존재합니다(기본적으로 스텐실링이 비활성화된 일반적인 깊이 테스트). OMSetDepthStencilState의 첫 번째 매개변수에 null을 전달하면 기본 깊이/스텐실 상태를 복원할 수 있습니다:

```
// 기본값 복원
md3dImmediateContext->OMSetDepthStencilState(0, 0);
```

10.3.4 이펙트 파일의 깊이/스텐실 상태

심도/스텐실 상태는 효과 파일에서 직접 정의하고 설정할 수도 있습니다:

```
DepthStencilState DSS
{
    DepthEnable          = true; 깊이 쓰
    기 마스크 = Zero;

    StencilEnable        = true; 스텐
    실 읽기 마스크 = 0xff; 스텐실 쓰기 마
    스크 = 0xff;

    FrontFaceStencilFunc = 항상; FrontFaceStencil Pass = 증
    가; FrontFaceStencil 실패 = 유지;

    Back FaceStencilFunc = 항상; Back
    FaceStencilPass = 증가; Back FaceStencilFail
    = 유지;
};

...
technique11 Tech
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS()));
        SetGeometryShader(NULL); SetPixelShader(CompileShader(ps_5_0,
        PS())); SetDepthStencilState(DSS, 0);
    }
}
```

깊이/스텐실 상태 객체에 할당하는 값은 C++ 구조체에 할당하는 값과 유사하지만 점두사가 없습니다. 예를 들어, D3D11_STENCIL_OP_INCR을 지정하는 대신 효과 코드에서 단순히 INCR을 지정합니다. 참고로,

소문자와 대문자를 구분하지 않습니다. 따라서 INCR은 Incr과 동일합니다.

10.4 평면 거울 구현

자연계의 많은 표면은 거울 역할을 하여 사물의 반사를 볼 수 있게 합니다. 이 섹션에서는 3D 애플리케이션을 위한 거울 시뮬레이션 방법을 설명합니다. 간결함을 위해 거울 구현 작업을 평면 표면으로만 제한합니다. 예를 들어 반짝이는 자동차도 반사를 보여줄 수 있지만, 자동차 차체는 매끄럽고 둥글며 평면이 아닙니다. 대신 반짝이는 대리석 바닥이나 벽에 걸린 거울에 비친 것과 같은 반사, 즉 평면에 놓인 거울의 반사를 렌더링합니다.

프로그래밍 방식으로 거울을 구현하려면 두 가지 문제를 해결해야 합니다. 첫째, 임의의 평면에 대해 객체를 반사하는 방법을 배워야 합니다. 그래야 반사된 모습을 정확하게 그릴 수 있습니다.

둘째, 반사된 모습은 반드시 거울에 표시되어야 합니다. 즉, 어떤 표면을 거울로 '표시'한 다음, 렌더링 과정에서 반사된 모습만 그려야 합니다.

둘째, 거울에 반사된 모습만 표시해야 합니다. 즉, 어떤 방식으로든 표면을 거울로 '표시'한 후, 렌더링 과정에서 거울에 있는 경우에만 반사된 객체를 그려야 합니다. 이 개념을 처음 소개한 그림

10.1을 다시 참고하십시오.

첫 번째 문제는 해석 기하학을 통해 쉽게 해결할 수 있으며 부록 C에서 논의됩니다. 두 번째 문제는

스텐실 버퍼를 사용하여 해결할 수 있습니다.

10.4.1 거울 개요

 반사상을 그릴 때, 광원도 거울 평면을 가로질러 반사시켜야 합니다. 그렇지 않으면 반사상의 조명이 정확하지 않을 것입니다.

그림 10.2는 물체의 반사를 그리기 위해 단순히 거울 평면을 기준으로 반사시키면 된다는 것을 보여줍니다. 그러나 이 방법은 그림 10.1에 나타난 문제를 야기합니다. 즉, 물체(이 경우 두개골)의 반사는 장면 내 또 다른 물체로 인식되며, 가리는 요소가 없다면 시야에 들어옵니다. 그러나 반사는 오직 거울을 통해서만 보여야 합니다. 이 문제는 스텐실 버퍼를 사용하여 해결할 수 있습니다. 스텐실 버퍼는 백 버퍼의 특정 영역에 대한 렌더링을 차단할 수 있기 때문입니다. 따라서 거울에 렌더링되지 않는 반사된 두개골의 렌더링을 스텐실 버퍼로 차단할 수 있습니다. 이를 달성하기 위한 단계는 다음과 같습니다:

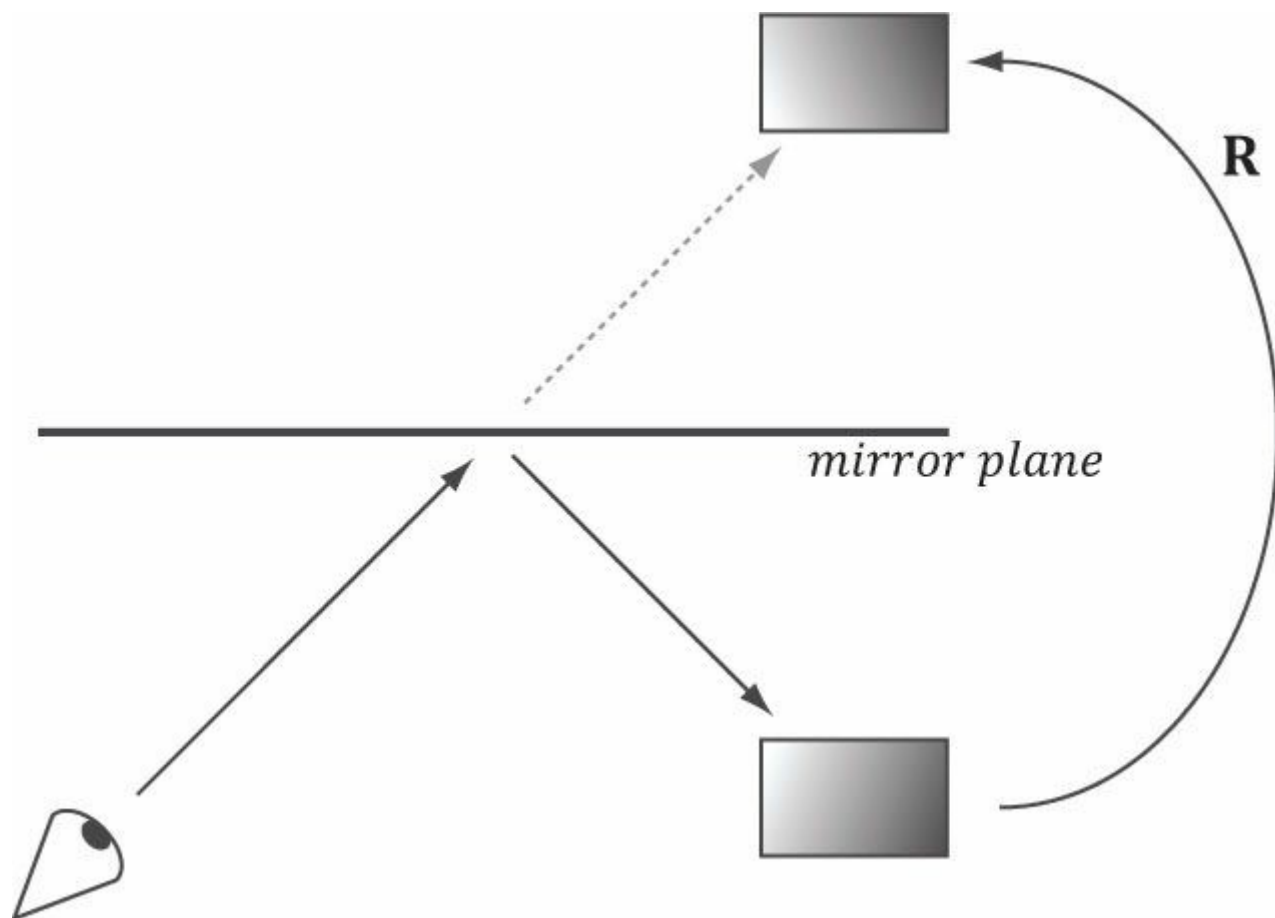
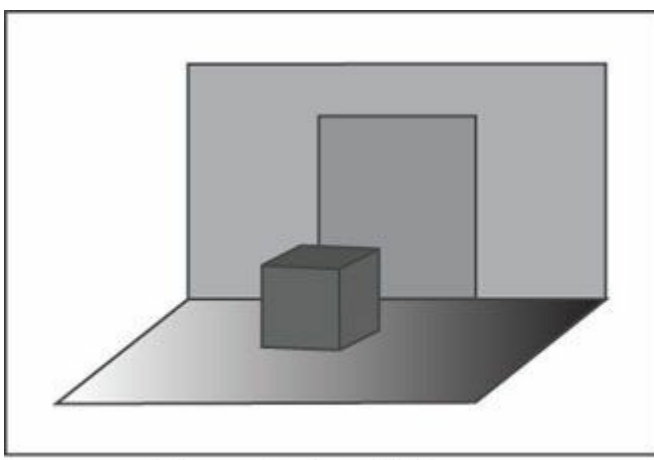
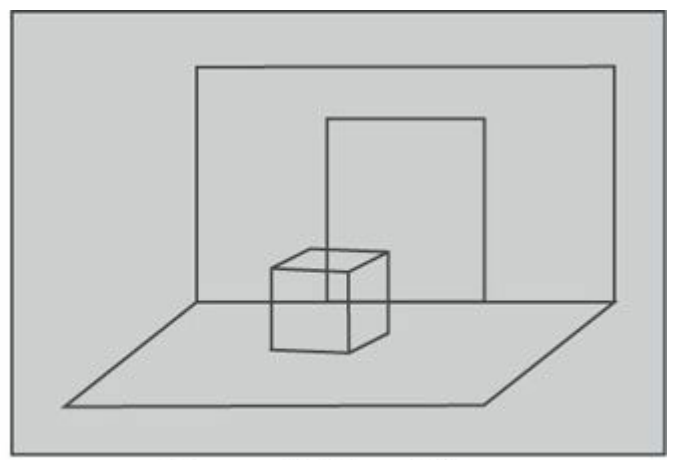


그림 10.2. 눈은 거울을 통해 상자의 반사상을 본다. 이를 시뮬레이션하기 위해 상자를 거울 평면에서 반사시키고 반사된 상자를 평소처럼 렌더링한다.



Back buffer



Stencil buffer

그림 10.3. 바닥, 벽, 두개골이 백 버퍼에 렌더링되고 스텐실 버퍼가 0으로 초기화됨(밝은 회색으로 표시됨). 스텐실 버퍼에 그려진 검은색 윤곽선은 백 버퍼 픽셀과 스텐실 버퍼 픽셀 간의 관계를 나타내며, 스텐실 버퍼에 그려진 데이터를 의미하지는 않습니다.

1. 바닥, 벽, 두개골을 백 버퍼에 평소처럼 렌더링합니다(거울 제외). 이 단계는 스텐실 버퍼를 수정하지 않습니다.
2. 스텐실 버퍼를 0으로 초기화합니다. 그림 10.3은 이 시점의 백 버퍼와 스텐실 버퍼를 보여줍니다(그림을 단순화하기 위해 두개골 대신 상자를 사용함).
3. 미러링을 스텐실 버퍼에만 렌더링합니다. 백 버퍼에 대한 컬러 쓰기를 비활성화하려면 블렌드 상태를 생성하여

```
D3D11_RENDER_TARGET_BLEND_DESC::RenderTargetWriteMask = 0;
```

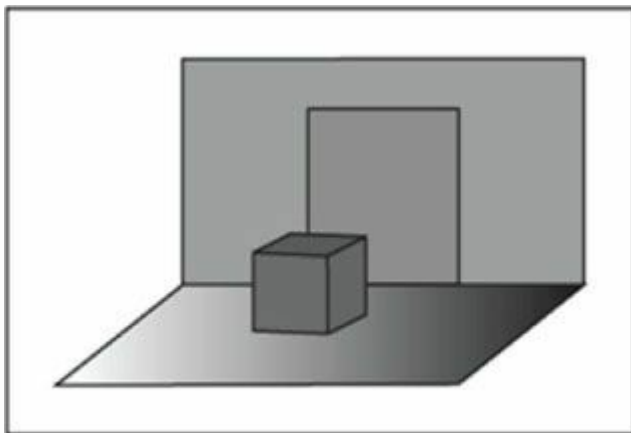
그리고 깊이 버퍼에 대한 쓰기를 비활성화하려면

```
D3D11_DEPTH_STENCIL_DESC::DepthWriteMask = D3D11_DEPTH_WRITE_MASK_ZERO;
```

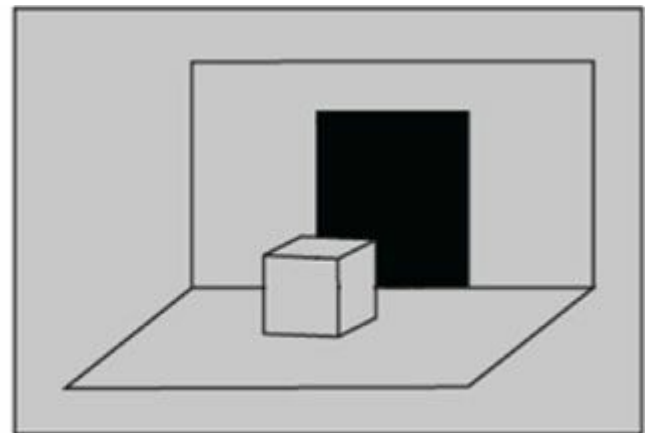
미러를 스텐실 버퍼에 렌더링할 때, 스텐실 테스트가 항상 성공하도록 설정(D3D11_COMPARISON_ALWAYS)하고 테스트 통과 시 스텐실 버퍼 항목을 1(StencilRef)로 대체(D3D11_STENCIL_OP_REPLACE)하도록 지정합니다. 깊이 테스트가 실패할 경우, 스텐실 버퍼가 변경되지 않도록 D3D11_STENCIL_OP_KEEP을 지정합니다(예를 들어 해골이 거울의 일부를 가리는 경우 발생할 수 있음). 우리는 스텐실 버퍼에만 거울을 렌더링하기 때문에, 스텐실 버퍼의 모든 픽셀은 0이 되며, 거울의 보이는 부분에 해당하는 픽셀만 1이 됩니다. 그림 10.4는 업데이트된 스텐실 버퍼를 보여줍니다. 본질적으로 우리는 스텐실 버퍼에서 거울의 보이는 픽셀을 표시하고 있습니다.

두개골을 그린 후 거울을 스텐실 버퍼에 그려야 합니다. 이렇게 하면 두개골에 가려진 거울의 픽셀이 깊이 테스트에 실패하여 스텐실 버퍼를 수정하지 않습니다. 가려진 스텐실 버퍼의 일부를 활성화하고 싶지 않습니다. 그렇지 않으면 반사가 두개골을 통해 비쳐 보일 것입니다.

가려진 스텐실 버퍼 부분을 활성화해서는 안 됩니다. 그렇지 않으면 반사가 두개골을 통해 비쳐 보일 것입니다.



Back Buffer



Stencil Buffer

그림 10.4. 미러를 스텐실 버퍼에 렌더링하는 과정으로, 본질적으로 스텐실 버퍼 내 미러의 가시 영역에 해당하는 픽셀을 표시합니다. 스텐실 버퍼의 검은색 영역은 스텐실 엔트리가 1로 설정되었음을 나타냅니다. 상자(box)에 의해 가려진 스텐실 버퍼 영역은 깊이 테스트(depth test)에 실패하므로(미러의 해당 부분이 상자 앞에 위치함) 1로 설정되지 않음을 유의하십시오.

깊이 테스트를 통과하지 못하기 때문에 1로 설정되지 않습니다(상자가 거울의 해당 부분 앞에 위치함).

4. 이제 반사된 두개골을 백 버퍼와 스텐실 버퍼에 렌더링합니다. 하지만 스텐실 테스트를 통과한 경우에만 백 버퍼에 렌더링된다는 점을 기억하세요. 이번에는 스텐실 버퍼의 값이 1과 같을 때만 스텐실 테스트가 성공하도록 설정합니다. 이는 **StencilRef** 값을 1로 설정하고 스텐실 연산자 **D3D11_COMPARISON_EQUAL**을 사용하여 수행됩니다. 이렇게 하면 반사된 해골은 해당 스텐실 버퍼 항목에 1이 있는 영역에만 렌더링됩니다. 스텐실 버퍼에서 거울의 가시 부분에 해당하는 영역만이 1을 가지는 항목이므로, 반사된 해골은 거울의 가시 부분에만 렌더링된다는 결론이 나옵니다.
5. 마지막으로, 우리는 평소처럼 거울을 백 버퍼에 렌더링합니다. 그러나 거울 뒤에 있는 두개골 반사가 비쳐 보이도록 하려면, 투명도 블렌딩을 적용하여 거울을 렌더링해야 합니다. 투명도로 거울을 렌더링하지 않으면 거울의 깊이가 반사보다 낮기 때문에 거울이 반사를 가려버립니다. 이를 구현하려면 거울에 대한 새로운 머터리얼 인스턴스를 정의하기만 하면 됩니다. 디퓨즈 컴포넌트의 알파 채널을 0.5로 설정하여 거울을 50% 불투명하게 만들고, 지난 장(§9.5.4)에서 설명한 투명도 블렌딩 상태로 거울을 렌더링합니다.

```
mMirrorMat.Ambient = XMFLOAT4(0.5f, 0.5f, 0.5f, 1.0f); mMirrorMat.Diffuse =
XMFLOAT4(1.0f, 1.0f, 1.0f, 0.5f); mMirrorMat.Specular = XMFLOAT4(0.4f, 0.4f, 0.4f,
16.0f); 이 설정은 다음과 같은 블렌딩 방정식을 제공합니다:
```

$$C = 0.5 \cdot C_{src} + 0.5 \cdot C_{dst}$$

반사된 두개골 픽셀을 백 버퍼에 기록했다고 가정하면, 색상의 50%는 거울(소스)에서, 나머지 50%는 두개골(대상)에서 비롯된 것으로 나타납니다.

10.4.2 거울 깊이/스텐실 상태 정의

앞서 설명한 알고리즘을 구현하려면 두 개의 깊이/스텐실 상태가 필요합니다. 첫 번째는 거울을 그릴 때 스텐실 버퍼에 거울 픽셀을 표시하는 데 사용됩니다. 두 번째는 반사된 두개골을 거울의 가시 영역에만 그려지도록 하는 데 사용됩니다.

```
//
// MirrorDSS 표시
//

D3D11_DEPTH_STENCIL_DESC mirrorDesc; mirrorDesc.DepthEnable = true;
mirrorDesc.DepthWriteMask = D3D11_DEPTH_WRITE_MASK_ZERO;
mirrorDesc.DepthFunc = D3D11_COMPARISON_LESS;
mirrorDesc.StencilEnable = true;
mirrorDesc.StencilReadMask = 0xff;
mirrorDesc.StencilWriteMask = 0xff;

mirrorDesc.FrontFace.StencilFailOp = D3D11_STENCIL_OP_KEEP;
mirrorDesc.FrontFace.StencilDepthFailOp = D3D11_STENCIL_OP_KEEP;
mirrorDesc.FrontFace.StencilPassOp = D3D11_STENCIL_OP_REPLACE;
mirrorDesc.FrontFace.StencilFunc = D3D11_COMPARISON_ALWAYS;

// 후면 폴리곤은 렌더링하지 않으므로 이 설정은 중요하지 않습니다. mirrorDesc.BackFace.StencilFailOp = D3D11_STENCIL_OP_KEEP;
mirrorDesc.BackFace.StencilDepthFailOp = D3D11_STENCIL_OP_KEEP;
mirrorDesc.BackFace.StencilPassOp = D3D11_STENCIL_OP_REPLACE;
mirrorDesc.BackFace.StencilFunc = D3D11_COMPARISON_ALWAYS;

ID3D11DepthStencilState* Mark MirrorDSS;
HR(device->CreateDepthStencilState(&mirrorDesc, &Mark MirrorDSS));

//
```

```
// DrawReflectionDSS
//

D3D11_DEPTH_STENCIL_DESC drawReflectionDesc; drawReflectionDesc.DepthEnable = true;
drawReflectionDesc.DepthWriteMask = D3D11_DEPTH_WRITE_MASK_ALL;
drawReflectionDesc.DepthFunc = D3D11_COMPARISON_LESS;

drawReflectionDesc.StencilEnable = true;
drawReflectionDesc.StencilReadMask = 0xff;
drawReflectionDesc.StencilWriteMask = 0xff;

drawReflectionDesc.FrontFace.StencilFailOp = D3D11_STENCIL_OP_KEEP;
drawReflectionDesc.FrontFace.StencilDepthFailOp = D3D11_STENCIL_OP_KEEP;
drawReflectionDesc.FrontFace.StencilPassOp = D3D11_STENCIL_OP_KEEP;
drawReflectionDesc.FrontFace.StencilFunc = D3D11_COMPARISON_EQUAL;

// 우리는 뒷면 폴리곤을 렌더링하지 않으므로 이러한 설정은 중요하지 않습니다. drawReflectionDesc.Back
// Face.StencilFailOp = D3D11_STENCIL_OP_KEEP;
drawReflectionDesc.Back Face.StencilDepthFailOp = D3D11_STENCIL_OP_KEEP; drawReflectionDesc.Back
// Face.StencilPassOp = D3D11_STENCIL_OP_KEEP; drawReflectionDesc.Back Face.StencilFunc =
// D3D11_COMPARISON_EQUAL;

ID3D11DepthStencilState* DrawReflectionDSS;
HR(device->CreateDepthStencilState(&drawReflectionDesc, &DrawReflectionDSS));
```

10.4.3 씬 그리기

다음 코드는 우리의 draw 메서드를 개략적으로 보여줍니다. 간결성과 명확성을 위해 상수 버퍼 값 설정과 같은 관련 없는 세부 사항은 생략했습니다(전체 세부 사항은 예제 코드를 참조하세요).

```
//
// 바닥, 벽, 해골을 백 버퍼에 평소처럼 렌더링합니다.
//
...

//
// 거울을 스텐실 버퍼에 그림니다.
// 여기서 거울의 가시 픽셀 스텐실 값을
// "거울 픽셀"을 표시합니다.
//
// 거울을 마지막에 그려야 하는 이유는
// 먼저 두개골을 깊이 버퍼에 렌더링하여
// 거울을 렌더링할 때, 두개골에 의해 가려진 거울 부분이
// 스텐실 버퍼에 렌더링되지 않도록 합니다.
// 스텐실 버퍼에 렌더링되지 않도록 합니다. 가려진 부분의 픽셀을
// 스텐실 버퍼에 가려진 픽셀을 설정해서는 안 됩니다. 그렇지 않으면
// 두개골을 통해 비쳐 보일 수 있습니다.
//

// 렌더 타겟에 쓰기 금지. md3dImmediateContext-
>OMSetBlendState(
    RenderStates::NoRenderTargetWritesBS, blendFactor, 0xffffffff);

// 가시적인 미러 픽셀을 스텐실 버퍼에 렌더링합니다.
// 이 시점에서 미러 깊이를 깊이 버퍼에 쓰지 마십시오. 그렇지 않으면
// 반사가 가려질 수 있습니다. md3dImmediateContext-
>OMSetDepthStencilState(
    RenderStates::MarkMirrorDSS, 1); pass-
>Apply(0, md3dImmediateContext);
```

```

// 미리 그리기. md3dImmediateContext->Draw(6, 24);

//
// 반사된 해골을 그립니다.
//

// 두개골을 반사시키기 위한 반사 행렬 생성.
XMVECTOR mirrorPlane = XMVectorSet(0.0f, 0.0f, 1.0f, 0.0f); // xy 평면 XMMATRIX R =
XMMatrixReflect(mirrorPlane);
XMMATRIX world = XMLoadFloat4x4(&mSkullWorld) * R;
...
// 광원도 함께 반사합니다.

// 기존 광원 방향을 캐시하고, 광원 방향을 반사합니다. XMVECTOR oldLightDirections[3];

for(int i = 0; i < 3; ++i)
{
    oldLightDirections[i] = mDirLights[i].Direction;
    XMVECTOR lightDir = XMLoadFloat3(&mDirLights[i].Direction); XMVECTOR reflectedLightDir =
    XMVector3TransformNormal(lightDir, R); XMStoreFloat3(&mDirLights[i].Direction, reflectedLightDir);
}

Effects::BasicFX->SetDirLights(mDirLights);

// 반사는 와인딩 순서를 변경하므로 시계 방향으로 컬링 처리
// 삼각형만 선택합니다 (§10.4.4 참조).
md3dImmediate Context->RSSetState(RenderStates::CullClockwise RS);

// 스텐실 값이 1인 픽셀에만 반사 렌더링.
// 보이는 거울 픽셀만 스텐실 값이 1이므로 해골은
// 거울에만 렌더링됩니다. md3dImmediate Context->OMSetDepthStencilState(
    RenderStates::DrawReflectionDSS, 1);
pass->Apply(0, md3dImmediateContext);
md3dImmediateContext->DrawIndexed(mSkullIndexCount, 0, 0);

// 기본 상태 복원. md3dImmediateContext->RSSetState(0);
md3dImmediateContext->OMSetDepthStencilState(0, 0);

// 광원 방향 복원. for(int i = 0; i < 3; ++i)
{
    mDirLights[i].Direction = oldLightDirections[i];
}

Effects::BasicFX->SetDirLights(mDirLights);

//
// 평소처럼 백 버퍼에 거울을 그립니다. 단, 반사가 비쳐 보이도록 투명하게 블렌딩합니다.
// 블렌딩하여 반사된 모습이 비치도록 합니다.
//

// 미러링
md3dImmediate Context->OMSetBlendState( RenderStates::TransparentBS, blendFactor,
    0xffffffff);
pass->Apply(0, md3dImmediateContext);
md3dImmediateContext->Draw(6, 24);

```

10.4.4 회전 방향과 반사

삼각형이 평면을 가로질러 반사될 때, 그 와인딩 순서는 반전되지 않으므로 면 법선도 반전되지 않습니다. 따라서 반사 후 바깥을 향하던 법선은 안을 향하는 법선이 됩니다(그림 10.5 참조). 이를 수정하기 위해 Direct3D에 반시계 방향 와인딩 순서의 삼각형을 전면 삼각형으로, 시계 방향 와인딩 순서의 삼각형을 후면 삼각형으로 해석하도록 지시합니다(이는 일반적인 관례와 반대입니다—§5.10.2). 이렇게 하면 반사 후 노멀 방향이 외부 방향이 되도록 효과적으로 반사됩니다. 다음 래스터라이저 상태를 설정하여 와인딩 순서 관례를 반전시킵니다:

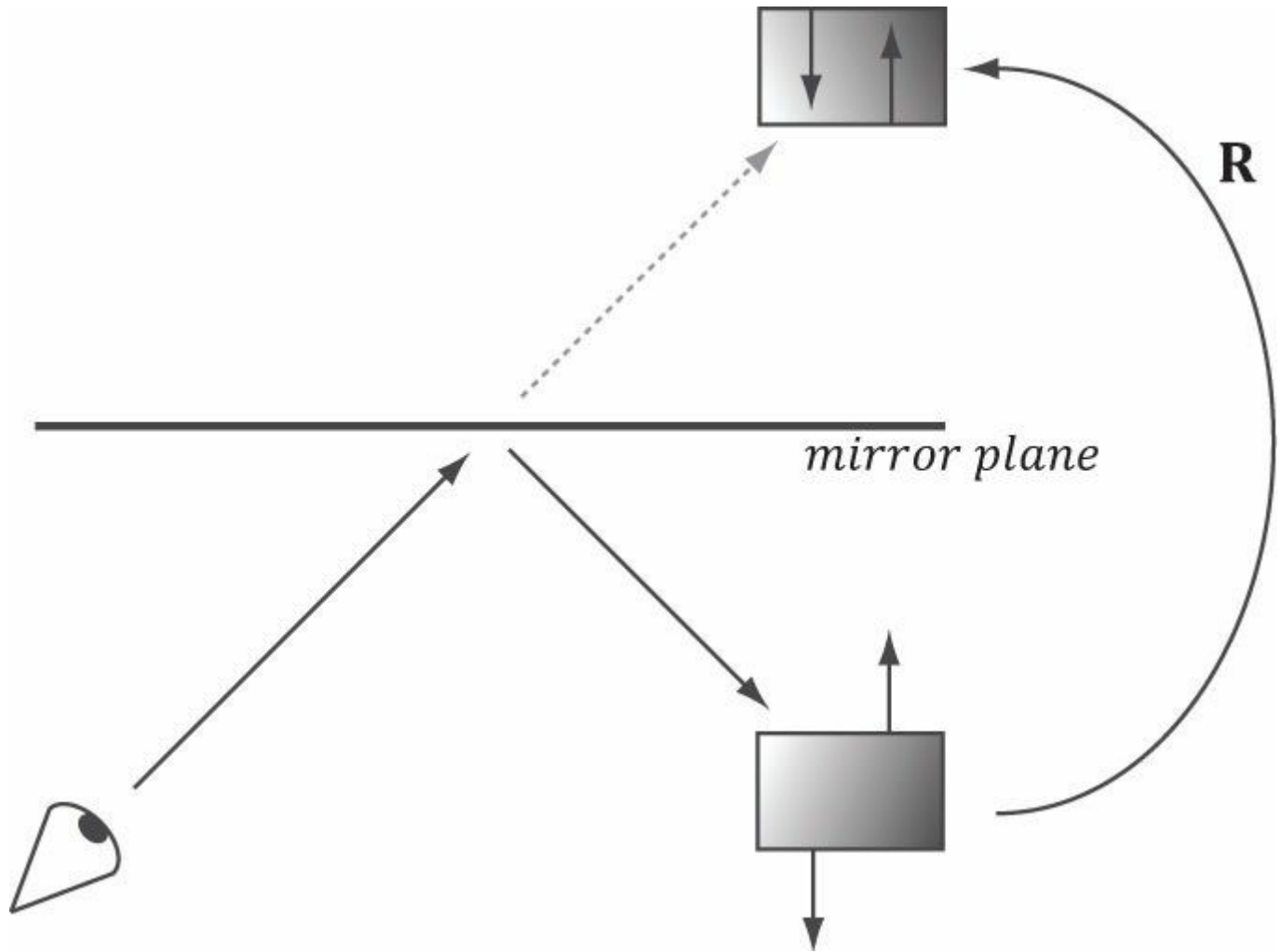


그림 10.5. 다각형 법선은 반사 시 반전되지 않아, 반사 후 내부를 향하게 됩니다.

```
// 참고: 정면 면을 CCW(반시계 방향)로 설정하여 후면 면을 여전히 켈링하도록 정의합니다.
// 후면 배면을 제거하지 않았다면, D3D11_DEPTH_STENCIL_DESC의 Back Face
// 속성에 대해 고려해야 합니다. D3D11_RASTERIZER_DESC cullClockwiseDesc;
ZeroMemory(&cullClockwiseDesc, sizeof(D3D11_RASTERIZER_DESC)); cullClockwiseDesc.FillMode =
D3D11_FILL_SOLID; cullClockwiseDesc.CullMode = D3D11_CULL_BACK;
cullClockwiseDesc.FrontCounterClockwise = true; cullClockwiseDesc.DepthClipEnable = true;

ID3D11RasterizerState* CullClockwiseRS;
HR(device->CreateRasterizerState(&cullClockwiseDesc, &CullClockwiseRS));
```

10.5 평면 그림자 구현



본 섹션의 일부 내용은 Frank D. Luna의 저서 『DirectX 9.0c를 활용한 3D 게임 프로그래밍 입문: 셰이더 접근법』(2006년, Jones and Bartlett Learning, Burlington, MA)에 수록된 내용입니다. www.jblearning.com. 허가를 받아 재인쇄합니다.

그림자는 장면 내에서 빛이 발산되는 위치를 인지하는 데 도움을 주며, 궁극적으로 장면을 더욱 사실적으로 만듭니다. 이

섹션에서는 평면 그림자, 즉 평면에 놓인 그림자를 구현하는 방법을 보여드리겠습니다([그림 10.6](#) 참조).

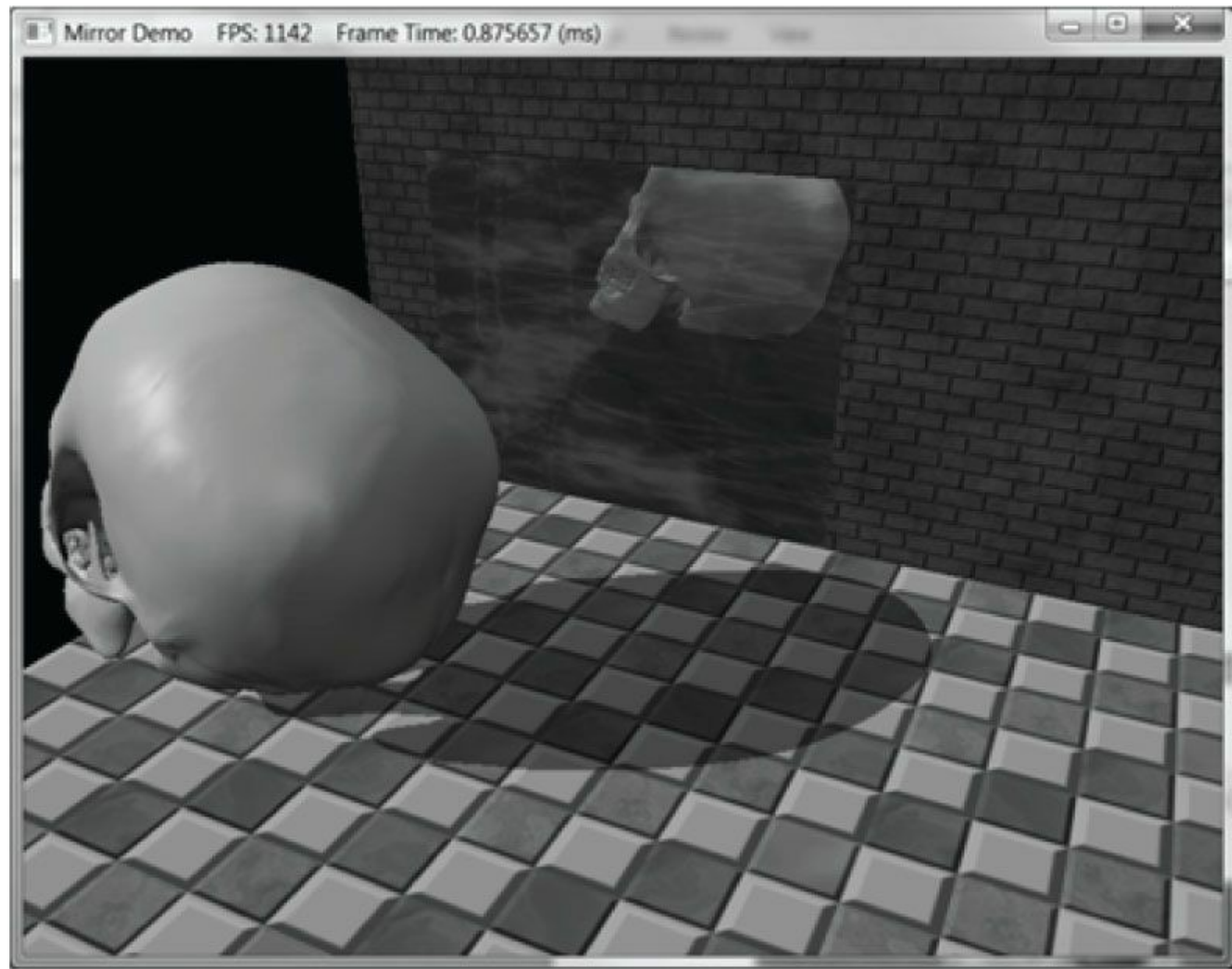


그림 10.6. "Mirror" 데모에서 주광원이 평면 그림자를 드리운 모습.

평면 그림자를 구현하려면 먼저 물체가 평면에 드리우는 그림자를 찾아 기하학적으로 모델링하여 렌더링할 수 있도록 해야 합니다. 이는 3D 수학을 활용하면 쉽게 수행할 수 있습니다. 그런 다음 그림자를 묘사하는 삼각형들을 50% 투명도의 검정색 재질로 렌더링합니다. 이렇게 그림자를 렌더링하면 "더블 블렌딩"이라는 렌더링 아티팩트가 발생할 수 있는데, 이는 몇 단락 후에 설명하겠습니다. 우리는 스텐실 버퍼를 활용하여 더블 블렌딩이 발생하지 않도록 방지합니다.

10.5.1 평행광 그림자

[그림 10.7](#)은 평행 광원에 대한 물체의 그림자를 보여줍니다. 방향 $\mathbf{L}$ 을 가진 평행 광원이 주어지면, 꼭짓점 $\mathbf{P}$ 를 통과하는 광선은 $\mathbf{r}(t) = \mathbf{p} + t\mathbf{L}$ 로 주어집니다. 광선 $\mathbf{r}(t)$ 와 그림자 평면 $(\mathbf{n}, d)$ 의 교점은 s 를 준다. (광선과 평면에 대한 자세한 내용은 부록 C를 참조할 수 있다.) 물체의 각 꼭짓점을 통해 평면으로 광선을 투사하여 찾은 교점들의 집합이 그림자의 투영 기하학을 정의한다. 꼭짓점 $\mathbf{p}$ 의 그림자 투영은 다음 식으로 주어진다.

$$\mathbf{s} = \mathbf{r}(t_s) = \mathbf{p} - \frac{\mathbf{n} \cdot \mathbf{p} + d}{\mathbf{n} \cdot \mathbf{L}} \mathbf{L} \quad (\text{식 10.1})$$

광선/평면 교차 테스트의 세부 사항은 부록 C에 설명되어 있습니다.

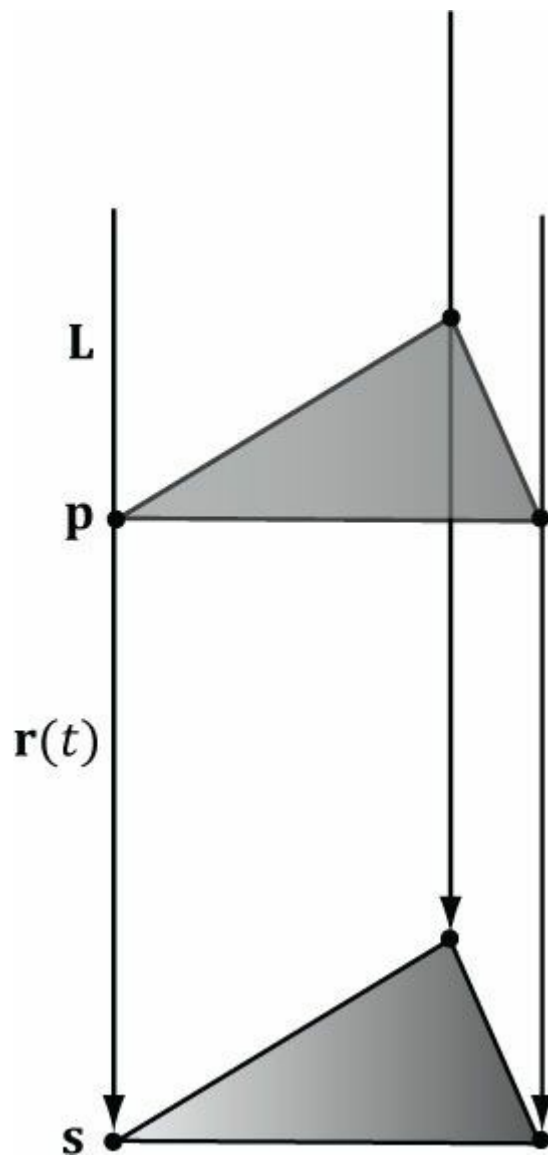


그림 10.7. 평행 광원에 대한 그림자 투영.

방정식 10.1은 행렬로 표현할 수 있다.

$$\mathbf{s}' = \begin{bmatrix} p_x & p_y & p_z & 1 \end{bmatrix} \begin{bmatrix} \mathbf{n} \cdot \mathbf{L} - L_x n_x & -L_y n_x & -L_z n_x & 0 \\ -L_x n_y & \mathbf{n} \cdot \mathbf{L} - L_y n_y & -L_z n_y & 0 \\ -L_x n_z & -L_y n_z & \mathbf{n} \cdot \mathbf{L} - L_z n_z & 0 \\ -L_x d & -L_y d & -L_z d & \mathbf{n} \cdot \mathbf{L} \end{bmatrix}$$

앞서 언급한 4×4 행렬을 방향 그림자 행렬이라 부르며 $\mathbf{S}_{dir}$ 로 표기한다. 이 행렬이 방정식 10.1과 어떻게 동등한지 확인하려면 단순히 곱셈을 수행하면 된다. 그러나 먼저, 이 방정식이 w 성분을 수정하여 $s_w = \mathbf{n} \cdot \mathbf{L}$ 이 되도록 한다는 점을 관찰하자. 따라서 시점 분할 (§5.6.3.4)이 발생할 때 s의 각 좌표는 $\mathbf{n} \cdot \mathbf{L}$ 로 나뉘게 되며, 이것이 방정식 10.1에서 $\mathbf{n} \cdot \mathbf{L}$ 로 나누는 결과를 얻는 방식이다. $\mathbf{L}$ 이 되도록 수정함을 알 수 있다. 따라서 투시 분할 (§5.6.3.4)이 발생할 때 s의 각 좌표는 $\mathbf{n} \cdot \mathbf{L}$ 로 나뉘게 되며, 이것이 행렬을 사용하여 방정식 10.1에서 $\mathbf{n} \cdot \mathbf{L}$ 로 나누는 결과를 얻는 방식이다. 이제 행렬 곱셈을 수행하여 $i \in \{1, 2,$

3}, 이어서 관점 분할을 적용하면 다음과 같이 얻어집니다.

$$\begin{aligned}
 s'_i &= \frac{(\mathbf{n} \cdot \mathbf{L}) p_i - L_i n_x p_x - L_i n_y p_y - L_i n_z p_z - L_i d}{\mathbf{n} \cdot \mathbf{L}} \\
 &= \frac{(\mathbf{n} \cdot \mathbf{L}) p_i - (\mathbf{n} \cdot \mathbf{p} + d) L_i}{\mathbf{n} \cdot \mathbf{L}} \\
 &= p_i - \frac{\mathbf{n} \cdot \mathbf{p} + d}{\mathbf{n} \cdot \mathbf{L}} L_i
 \end{aligned}$$

이는 방정식 10.1에서 s 의 i 번째 좌표와 정확히 일치하므로 $\mathbf{s} = \mathbf{s}'$ 입니다.

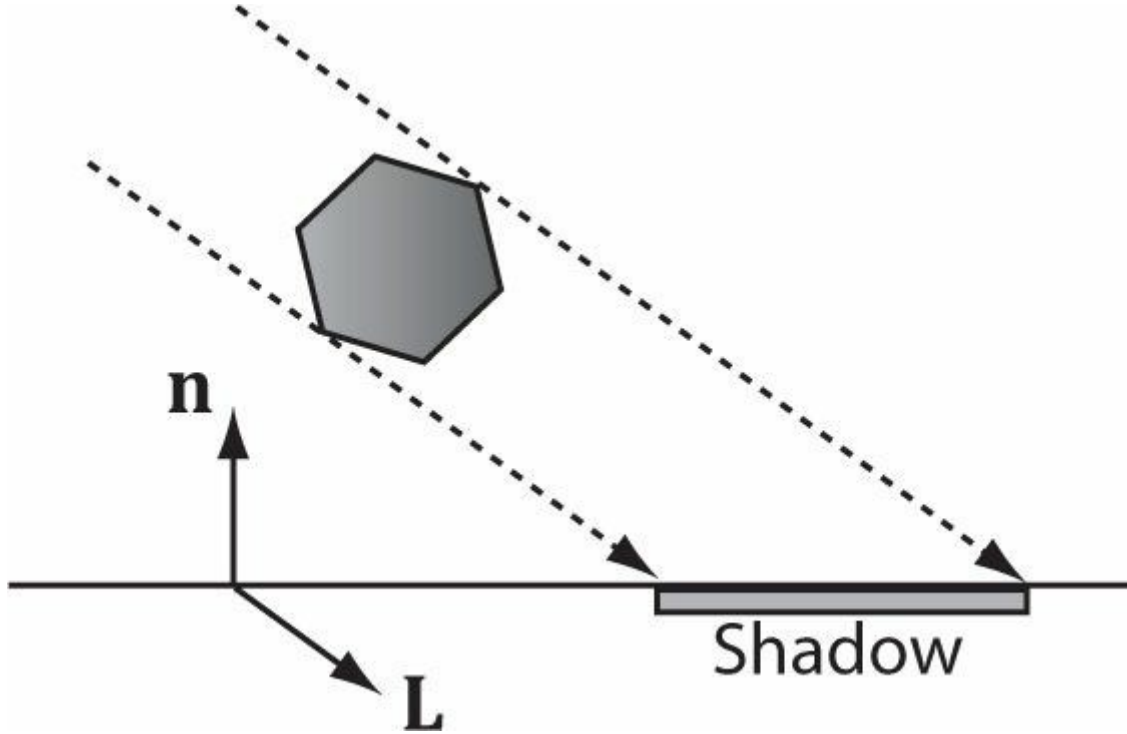


그림 10.8. $\mathbf{n} \cdot \mathbf{L} < 0$ 인 상황.

그림자 행렬을 사용하려면 이를 월드 행렬과 결합합니다. 그러나 월드 변환 후에도 기하학적 객체가 아직 그림자 평면에 투영되지 않은 상태입니다. 왜냐하면 투시 분할이 아직 수행되지 않았기 때문입니다. $s_w = \mathbf{n} \cdot \mathbf{L} < 0$ 인 경우 문제가 발생합니다. 이는 w 좌표를 음수로 만들기 때문입니다. 일반적으로 원근 투영 과정에서 z 좌표를 w 좌표로 복사하는데, 음수 w 좌표는 해당 점이 뷰 볼륨에 속하지 않아 클리핑된다는 의미입니다(클리핑은 분할 전 동차 공간에서 수행됨). 이는 평면 그림자 구현 시 문제가 됩니다. 왜냐하면 원근 분할 외에도 w 좌표를 그림자 구현에 사용하기 때문입니다. **그림 10.8**은 $\mathbf{n} \cdot \mathbf{L} < 0$ 인 유효한 상황을 보여줍니다. 그러나 그림자는 나타나지 않습니다.

이를 해결하기 위해, 광선 방향 $\mathbf{L}$ 대신 무한히 먼 빛의 근원 방향 벡터 $\mathbf{LT} = -\mathbf{L}$. $\mathbf{r}(t) = \mathbf{p} + t\mathbf{L}$ 과 $\mathbf{r}(t) = \mathbf{p} + t\mathbf{LT}$ 가 동일한 3차원 직선을 정의하며, 이 직선과 평면의 교점은 동일할 것임을 관찰하라 (교점 매개변수 $t_{(s)}$ 는 $\mathbf{LT}$ 와 $\mathbf{L}$ 사이의 부호 차이를 보상하기 위해 달라질 것이다). 따라서 $\mathbf{LT} = -\mathbf{L}$ 을 사용하면 동일한 해를 얻지만, $\mathbf{n} \cdot \mathbf{L} > 0$ 이 되어 음의 w 좌표를 피할 수 있습니다.

10.5.2 점광원의 그림자

그림 10.9는 점 $\mathbf{L}$ 의 위치로 표현되는 점광원에 대해 물체가 드리우는 그림자를 보여줍니다. 점광원에서 나오는 광선이 어떤 정점 $\mathbf{p}$ 를 통과할 때의 빔은 $\mathbf{r}(t) = \mathbf{p} + t(\mathbf{p} - \mathbf{L})$ 로 주어집니다. 광선 $\mathbf{r}(t)$ 와 그림자 평면 $(\mathbf{n}, d)$ 의 교점은 $\mathbf{s}$ 를 줍니다. 물체의 각 정점을 통해 평면으로 광선을 쏠 때 발견된 교점들의 집합이 그림자의 투영 기하학을 정의합니다. 정점 $\mathbf{p}$ 에 대해, 그 그림자 투영은 다음 식으로 주어집니다.

$$\mathbf{s} = \mathbf{r}(t_s) = \mathbf{p} - \frac{\mathbf{n} \cdot \mathbf{p} + d}{\mathbf{n} \cdot (\mathbf{p} - \mathbf{L})} (\mathbf{p} - \mathbf{L}) \quad (\text{식 10.2})$$

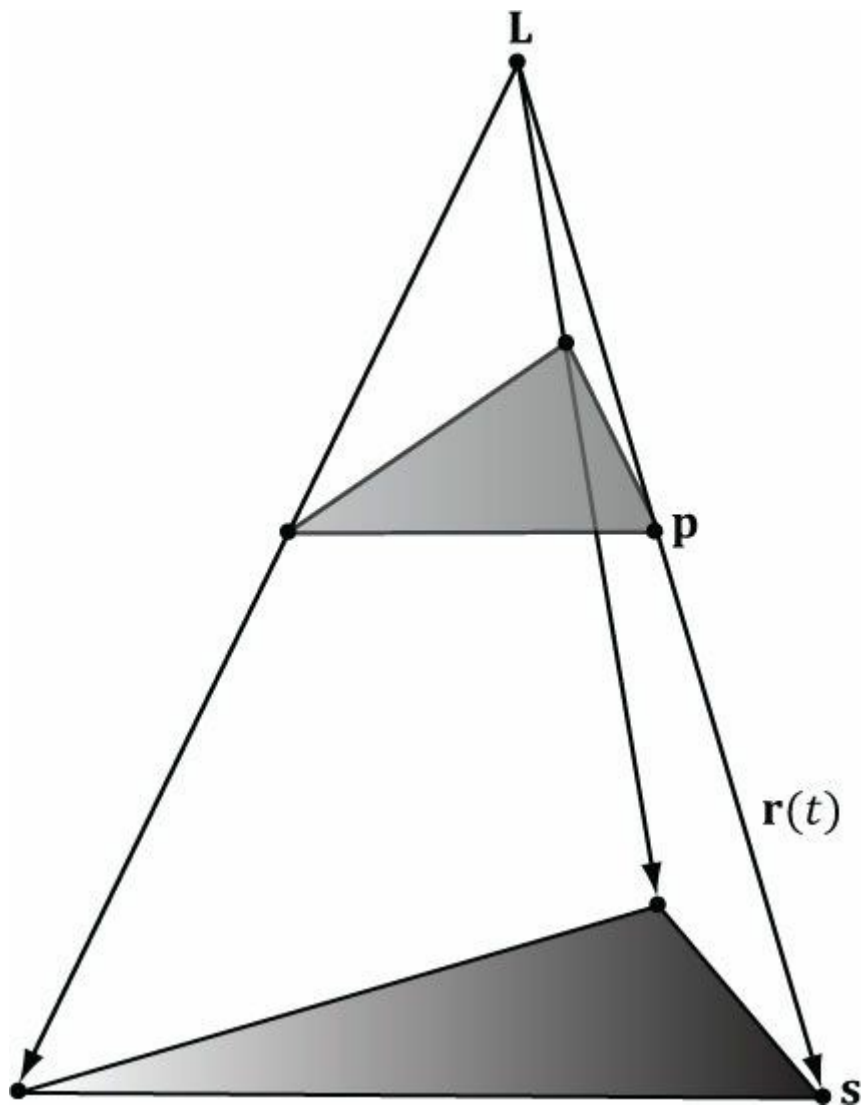


그림 10.9. 점광원에 대한 그림자 투영.

방정식 10.2는 행렬 방정식으로 다음과 같이 쓸 수 있다.

$$S_{point} = \begin{bmatrix} \mathbf{n} \cdot \mathbf{L} + d - L_x n_x & -L_y n_x & -L_z n_x & -n_x \\ -L_x n_y & \mathbf{n} \cdot \mathbf{L} + d - L_y n_y & -L_z n_y & -n_y \\ -L_x n_z & -L_y n_z & \mathbf{n} \cdot \mathbf{L} + d - L_z n_z & -n_z \\ -L_x d & -L_y d & -L_z d & \mathbf{n} \cdot \mathbf{L} \end{bmatrix}$$

이 행렬이 방정식 10.2와 어떻게 동등한지 확인하려면, 이전 절에서 수행한 것과 동일한 방식으로 곱셈을 수행하기만 하면 됩니다. 마지막 열에는 0이 없으며 다음과 같이 주어집니다:

$$\begin{aligned} s_w &= -p_x n_x - p_y n_y - p_z n_z + \mathbf{n} \cdot \mathbf{L} \\ &= -\mathbf{p} \cdot \mathbf{n} + \mathbf{n} \cdot \mathbf{L} \\ &= -\mathbf{n} \cdot (\mathbf{p} - \mathbf{L}) \end{aligned}$$

이는 방정식 10.2의 분모의 부호가 반전된 형태이지만, 분자를 부호 반전시키면 분모도 부호 반전시킬 수 있습니다.

점광원과 평행광원에서 L 의 용도가 다르다는 점에 유의하십시오. 점광원의 경우 L 을 사용하여 점광원의 위치를 정의합니다. 평행광원의 경우 L 을 사용하여 무한히 먼 광원을 향하는 방향 (즉, 평행 광선이 이동하는 반대 방향).

10.5.3 일반 그림자 행렬

동질 좌표를 사용하면 점광원과 방향광원 모두에 적용 가능한 일반적인 그림자 행렬을 생성할 수 있습니다.

- 1. $L_w=0$ 이면 **L**은 무한히 먼 광원 방향(즉, 평행 광선이 이동하는 반대 방향)을 나타냅니다.
- 2. $L_w=1$ 이면 **L**은 점광원의 위치를 나타냅니다.

그러면 정점 **p**에서 그 투영 **s**로의 변환을 다음과 같은 *그림자 행렬*로 표현합니다:

$$S = \begin{bmatrix} \mathbf{n} \cdot \mathbf{L} + dL_w - L_x n_x & -L_y n_x & -L_z n_x & -L_w n_x \\ -L_x n_y & \mathbf{n} \cdot \mathbf{L} + dL_w - L_y n_y & -L_z n_y & -L_w n_y \\ -L_x n_z & -L_y n_z & \mathbf{n} \cdot \mathbf{L} + dL_w - L_z n_z & -L_w n_z \\ -L_x d & -L_y d & -L_z d & \mathbf{n} \cdot \mathbf{L} \end{bmatrix}$$

S가 $L_w=0$ 일 때 S_{dir} 로, $L_w=1$ 일 때 S_{point} 로 축소됨을 쉽게 알 수 있다.
XNA 수학 라이브러리는 그림자를 투영할 평면을 주어 그림자 행렬을 생성하는 다음 함수를 제공합니다.

```
XMFLOATLINE XMVECTOR XMMatrixShadow( FXMVECTOR
ShadowPlane, FXMVECTOR LightPosition);
```

추가 참고 자료로 [Blinn96]과 [Möller02] 모두 평면 그림자에 대해 논의합니다.

10.5.4 스텐실 버퍼를 이용한 이중 블렌딩 방지

물체의 기하학적 구조를 평면에 평탄화하여 그림자를 표현할 때, 평탄화된 삼각형 중 두 개 이상이 겹칠 가능성이 있습니다(실제로 그럴 가능성이 높습니다). 투명도(블렌딩)를 사용하여 그림자를 렌더링할 때, 삼각형이 겹치는 영역은 여러 번 블렌딩되어 더 어둡게 보입니다. [그림 10.10](#)이 이를 보여줍니다.

이 문제는 스텐실 버퍼를 사용하여 해결할 수 있습니다.

- 1. 그림자가 렌더링될 스텐실 버퍼 픽셀이 0으로 초기화되었다고 가정합니다. 이는 우리의 미리 데모에서 사실입니다. 왜냐하면 우리는 지면 평면에만 그림자를 드리우고 있으며, 미리 스텐실 버퍼 픽셀만 수정했기 때문입니다.
- 2. 스텐실 버퍼 항목이 0인 픽셀만 허용하도록 스텐실 테스트를 설정합니다. 스텐실 테스트가 통과되면 스텐실 버퍼 값을 1로 증가시킵니다.

그림자 픽셀을 처음 렌더링할 때는 스텐실 버퍼 항목이 0이므로 스텐실 테스트가 통과됩니다. 그러나 이 픽셀을 렌더링할 때 해당 스텐실 버퍼 항목도 1로 증가시킵니다. 따라서 이미 렌더링 된 영역(스텐실 버퍼에 값 1로 표시됨)을 *덮어쓰려고* 하면 스텐실 테스트가 실패합니다. 이를 통해 동일한 픽셀을 두 번 이상 그리는 것을 방지하여 더블 블렌딩을 막습니다.

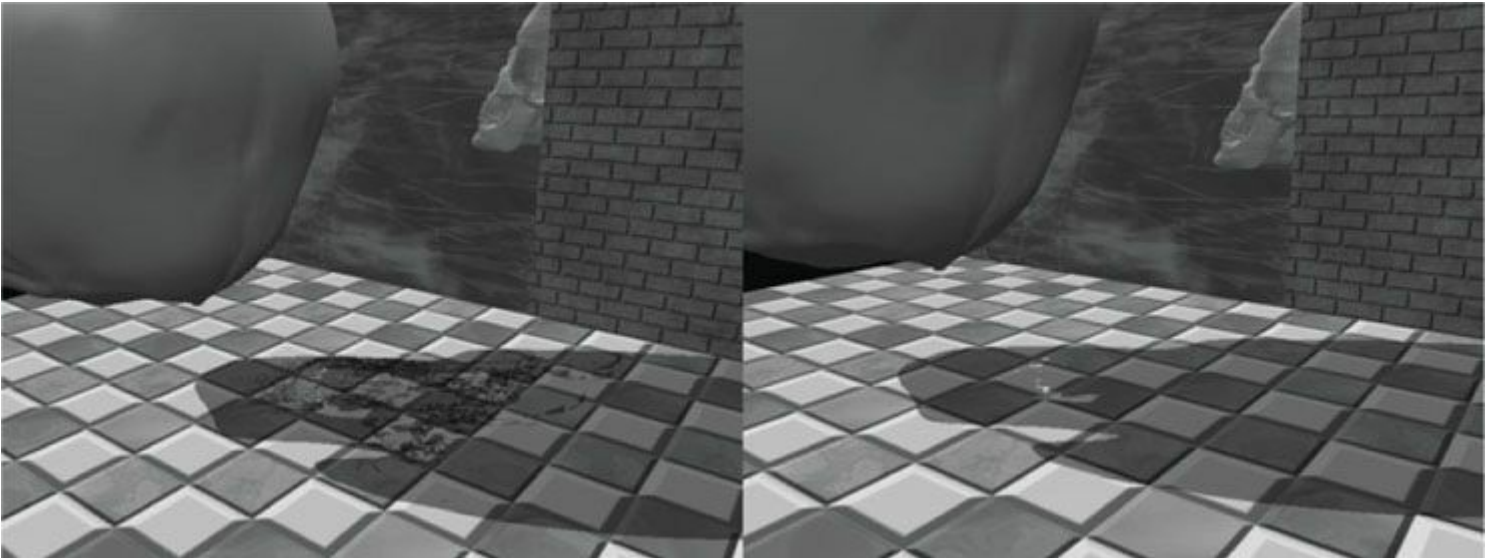


그림 10.10. 왼쪽 그림에서 그림자의 더 어두운 "여드름" 모양 영역을 주목하십시오. 이 영역들은 납작해진 두개골의 일부가 겹쳐져 "이중 블렌딩"을 일으킨 부분에 해당합니다. 오른쪽 그림은 이중 블렌딩 없이 그림자가 올바르게 렌더링된 모습을 보여줍니다.

10.5.5 그림자 코드

그림자를 채색하는 데 사용되는 그림자 재질을 정의합니다. 이는 단순히 50% 투명한 검정색 재질입니다:

```
mShadowMat.Ambient = XMFLOAT4(0.0f, 0.0f, 0.0f, 1.0f); mShadowMat.Diffuse =
XMFLOAT4(0.0f, 0.0f, 0.0f, 0.5f); mShadowMat.Specular = XMFLOAT4(0.0f, 0.0f, 0.0f,
16.0f);

더블 블렌딩을 방지하기 위해 다음과 같은 깊이/스텐실 상태 객체를 설정합니다:

D3D11_DEPTH_STENCIL_DESC noDoubleBlendDesc; noDoubleBlendDesc.DepthEnable          = true;
noDoubleBlendDesc.DepthWriteMask = D3D11_DEPTH_WRITE_MASK_ALL; noDoubleBlendDesc.DepthFunc
= D3D11_COMPARISON_LESS;
noDoubleBlendDesc.StencilEnable          = true; noDoubleBlendDesc.StencilReadMask = 0xff;
noDoubleBlendDesc.StencilWriteMask = 0xff;

noDoubleBlendDesc.FrontFace.StencilFailOp          = D3D11_STENCIL_OP_KEEP;
noDoubleBlendDesc.FrontFace.StencilDepthFailOp = D3D11_STENCIL_OP_KEEP; noDouble
BlendDesc.FrontFace.StencilPassOp = D3D11_STENCIL_OP_INCR; noDouble BlendDesc.FrontFace.StencilFunc    =
D3D11_COMPARISON_EQUAL;

// 우리는 백페이스링 폴리곤을 렌더링하지 않으므로 이러한 설정은 중요하지 않습니다. noDoubleBlendDesc.Back
Face.StencilFailOp          = D3D11_STENCIL_OP_KEEP;
noDoubleBlendDesc.Back Face.StencilDepthFailOp = D3D11_STENCIL_OP_KEEP; noDoubleBlendDesc.Back
Face.StencilPassOp = D3D11_STENCIL_OP_INCR; noDoubleBlendDesc.Back Face.StencilFunc =
D3D11_COMPARISON_EQUAL;

ID3D11DepthStencilState* NoDoubleBlendDSS;
HR(device->CreateDepthStencilState(&noDoubleBlendDesc, &NoDoubleBlendDSS));

그런 다음 다음과 같이 해골 그림자를 그립니다: 그림자 행렬을 구축하고 StencilRef 값이 0인 "no-double-blend" 깊이/스텐실 상태로 렌더링합니다:

XMVECTOR shadowPlane = XMVectorSet(0.0f, 1.0f, 0.0f, 0.0f); // xz 평면 XMVECTOR toMainLight = -
XMLoadFloat3(&mDirLights[0].Direction); XMMATRIX S = XMMatrixShadow(shadowPlane,
toMainLight); XMMATRIX shadowOffsetY = XMMatrixTranslation(0.0f, 0.001f, 0.0f);

// 객체별 상수 설정.
XMMATRIX world = XMLoadFloat4x4(&mSkullWorld)*S*shadowOffsetY; XMMATRIX worldInvTranspose =
MathHelper::InverseTranspose(world); XMMATRIX worldViewProj = world*view*proj;

Effects::BasicFX->SetWorld(world);
Effects::BasicFX->SetWorldInvTranspose(worldInvTranspose); Effects::BasicFX-
>SetWorldViewProj(worldViewProj); Effects::BasicFX->SetMaterial(mShadowMat);

md3dImmediate Context->OMSetDepthStencilState( RenderStates::NoDouble
BlendDSS, 0);
pass->Apply(0, md3dImmediateContext);
md3dImmediateContext->DrawIndexed(mSkullIndexCount, 0, 0);
```

10.6 요약

1. 스텐실 버퍼는 백 버퍼로의 특정 픽셀 프래그먼트 렌더링을 차단하는 데 사용할 수 있는 오프스크린 버퍼입니다. 스텐실 버퍼는 깊이 버퍼와 공유되므로 깊이 버퍼와 동일한 해상도를 가집니다. 유효한 깊이/스텐실 버퍼 형식은 DXGI_FORMAT_D32_FLOAT_S8X24_UINT 및 DXGI_FORMAT_D24_UNORM_S8_UINT입니다.
2. 특정 픽셀의 쓰기 작업을 차단할지 여부는 *스텐실 테스트*에 의해 결정되며, 이는 다음과 같이 주어집니다:

```
if(StencilRef & StencilReadMask <= Value & StencilReadMask) 픽셀 허용
else
    픽셀 거부
```

여기서 $\leq$ 연산자는 D3D11_COMPARISON_FUNC 열거형에 정의된 함수 중 하나입니다. StencilRef, StencilReadMask, StencilReadMask 및 비교 연산자 $\leq$ 는 모두 Direct3D 깊이/스텐실 API로 설정된 애플리케이션 정의 값입니다. Value 값은 스텐실 버퍼의 현재 값입니다.

3. ID3D11DepthStencilState 인터페이스를 생성하기 위한 첫 번째 단계는 생성하려는 깊이/스텐실 상태를 설명하는 D3D11_DEPTH_STENCIL_DESC 인스턴스를 채우는 것입니다. D3D11_DEPTH_STENCIL_DESC 구조체를 작성한 후에는 ID3D11Device::CreateDepthStencilState 메서드를 사용하여 ID3D11DepthStencilState 인터페이스에 대한 포인터를 얻을 수 있습니다. 마지막으로 ID3D11Device::OMSetDepthStencilState 메서드를 사용하여 깊이/스텐실 상태 블록을 파이프라인의 출력 병합 단계에 바인딩합니다.

10.7 연습 문제

1. 일반 그림자 행렬 S 가 $L_w = 0$ 일 때 S_{dir} 로, $L_w = 1$ 일 때 S_{point} 로 축소됨을 증명하십시오.
2.
$$s = p - \frac{n \cdot p + d}{n \cdot (n \cdot L)} (p - L) = p S_{point}$$
를 증명하십시오. 각 성분에 대해 행렬 곱셈을 수행하여 증명하십시오. §10.5.1에서 방향광에 대해 수행한 것과 같이 각 성분에 대해 행렬 곱셈을 수행하여 증명하십시오.
3. "Mirror" 데모를 수정하여 그림 10.1의 "Left" 이미지를 생성하십시오.
4. "Mirror" 데모를 수정하여 그림 10.10의 "Left" 이미지를 생성하십시오.
5. 다음과 같이 "Mirror" 데모를 수정하십시오. 먼저, 다음 깊이 설정으로 벽

을 그리십시오:

```
depthStencilDesc.DepthEnable = false; depthStencilDesc.DepthWriteMask =
D3D11_DEPTH_WRITE_MASK_ALL; depthStencilDesc.DepthFunc = D3D11_COMPARISON_LESS;
```

다음으로 벽 뒤에 두개골을 다음 깊이 설정으로 그리십시오:

```
depthStencilDesc.DepthEnable = true; depthStencilDesc.DepthWriteMask =
D3D11_DEPTH_WRITE_MASK_ALL; depthStencilDesc.DepthFunc = D3D11_COMPARISON_LESS;
```

벽이 두개골을 가리니까? 설명하십시오. 대신 다음을 사용하여 벽을 그릴 경우 어떻게 됩니까?

```
depthStencilDesc.DepthEnable = true; depthStencilDesc.DepthWriteMask =
D3D11_DEPTH_WRITE_MASK_ALL; depthStencilDesc.DepthFunc = D3D11_COMPARISON_LESS;
```

이 연습에서는 스텐실 버퍼를 사용하지 않으므로 해당 기능을 비활성화해야 합니다.

6. "Mirror" 데모를 수정하여 삼각형의 와인딩 순서 규칙을 반전시키지 마십시오. 반사된 주전자가 올바르게 렌더링됩니까?
7. 제9장의 "Blend" 데모를 수정하여 장면 중앙에 두께가 없는 원통을 그리십시오. 이 장의 디렉토리에 있는 60프레임 애니메이션 전기 번개 애니메이션을 가산 혼합(additive blending)을 사용하여 원통에 텍스처링합니다. 그림 10.11은 출력 예시를 보여줍니다. (힌트: 가산 혼합 지오메트리 렌더링 시 사용할 깊이 상태는 §9.5.5를 참조하십시오.)

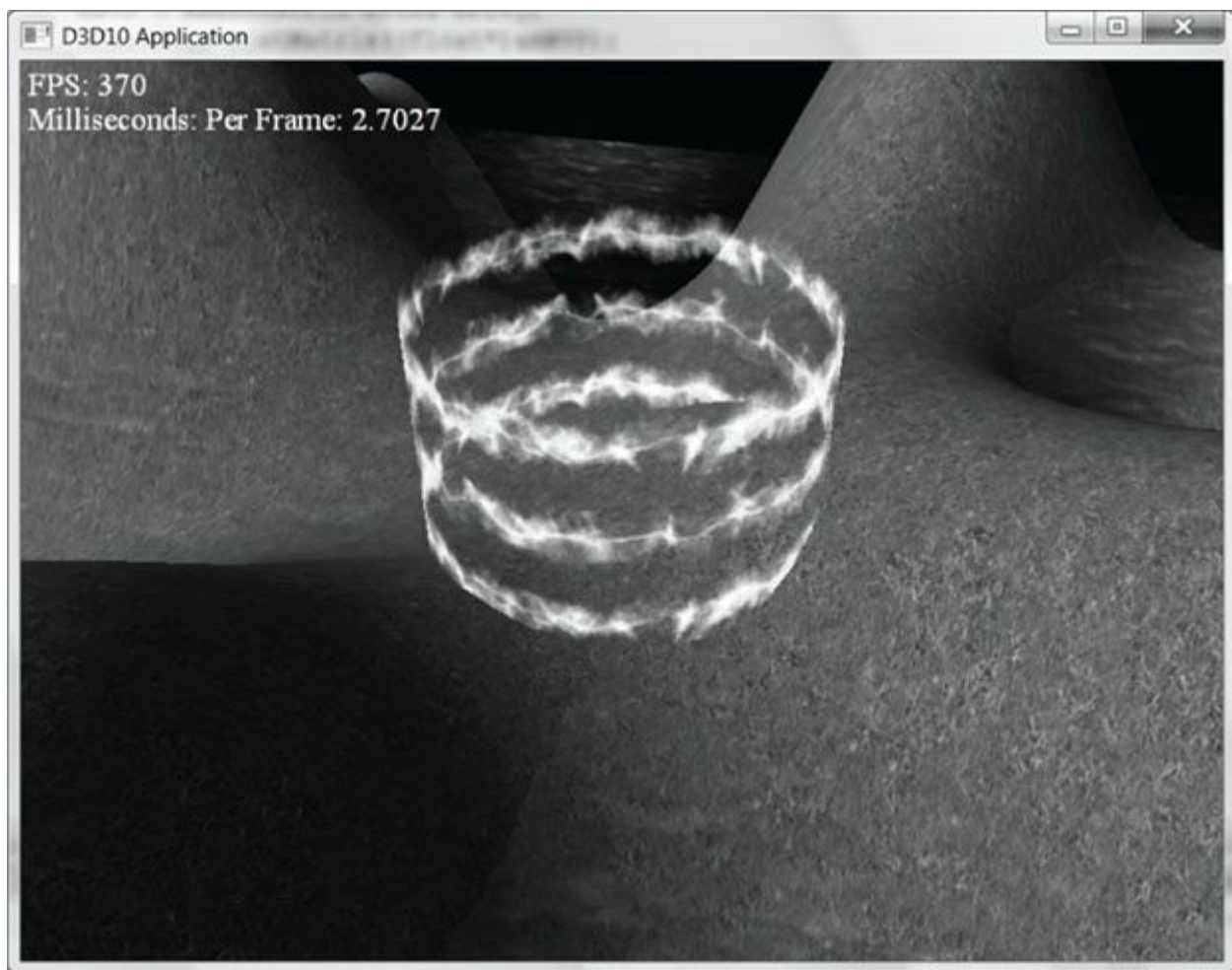


그림 10.11. 연습문제 7의 해결책 샘플 스크린샷.

8. **깊이 복잡도**는 백 버퍼의 특정 항목에 기록되기 위해 깊이 테스트를 통해 경쟁하는 픽셀 조각의 수를 의미합니다. 예를 들어, 우리가 그린 픽셀은 카메라에 더 가까운 픽셀에 의해 덮어쓰일 수 있습니다(전체 장면이 그려진 후 가장 가까운 픽셀이 실제로 결정되기 전까지 여러 번 발생할 수 있음). 그림

10.12의 픽셀은 3개의 픽셀 조각이 해당 픽셀을 놓고 경쟁하므로 깊이 복잡도가 3입니다.

잠재적으로 그래픽 카드는 각 프레임마다 한 픽셀을 여러 번 채울 수 있습니다. 이러한 *오버드로*는 성능에 영향을 미치는데, 그래픽 카드가 결국 덮어쓰여 표시되지 않는 픽셀을 처리하는 데 시간을 낭비하기 때문입니다. 따라서 성능 분석을 위해 장면의 깊이 복잡도를 측정하는 것이 유용합니다.

깊이 복잡도는 다음과 같이 측정할 수 있습니다: 장면을 렌더링하고 스텐실 버퍼를 카운터로 사용합니다. 즉, 스텐실 버퍼의 각 픽셀은 원래 0으로 초기화되며, 픽셀 프래그먼트가 처리될 때마다 **D3D11_STENCIL_OP_INCR**로 카운트를 증가시킵니다. 어떤 경우에도 모든 픽셀 조각에 대해 해당 스텐실 버퍼 항목이 항상 증가되어야 하므로, 스텐실 비교 함수 **D3D11_COMPARISON_ALWAYS**를 사용합니다. 그런 다음, 예를 들어 프레임이 그려진 후 *ij*번째 픽셀에 스텐실 버퍼에 5라는 대응 항목이 있다면, 해당 프레임 동안 그 픽셀에 대해 5개의 픽셀 프래그먼트가 처리되었음을 알 수 있습니다(즉, 픽셀의 깊이 복잡도가 5임). 깊이 복잡도를 계산할 때 기술적으로 장면을 스텐실 버퍼에 렌더링하기만 하면 된다는 점에 유의하십시오.

(스텐실 버퍼에 저장된) 깊이 복잡도를 시각화하려면 다음과 같이 진행하십시오:

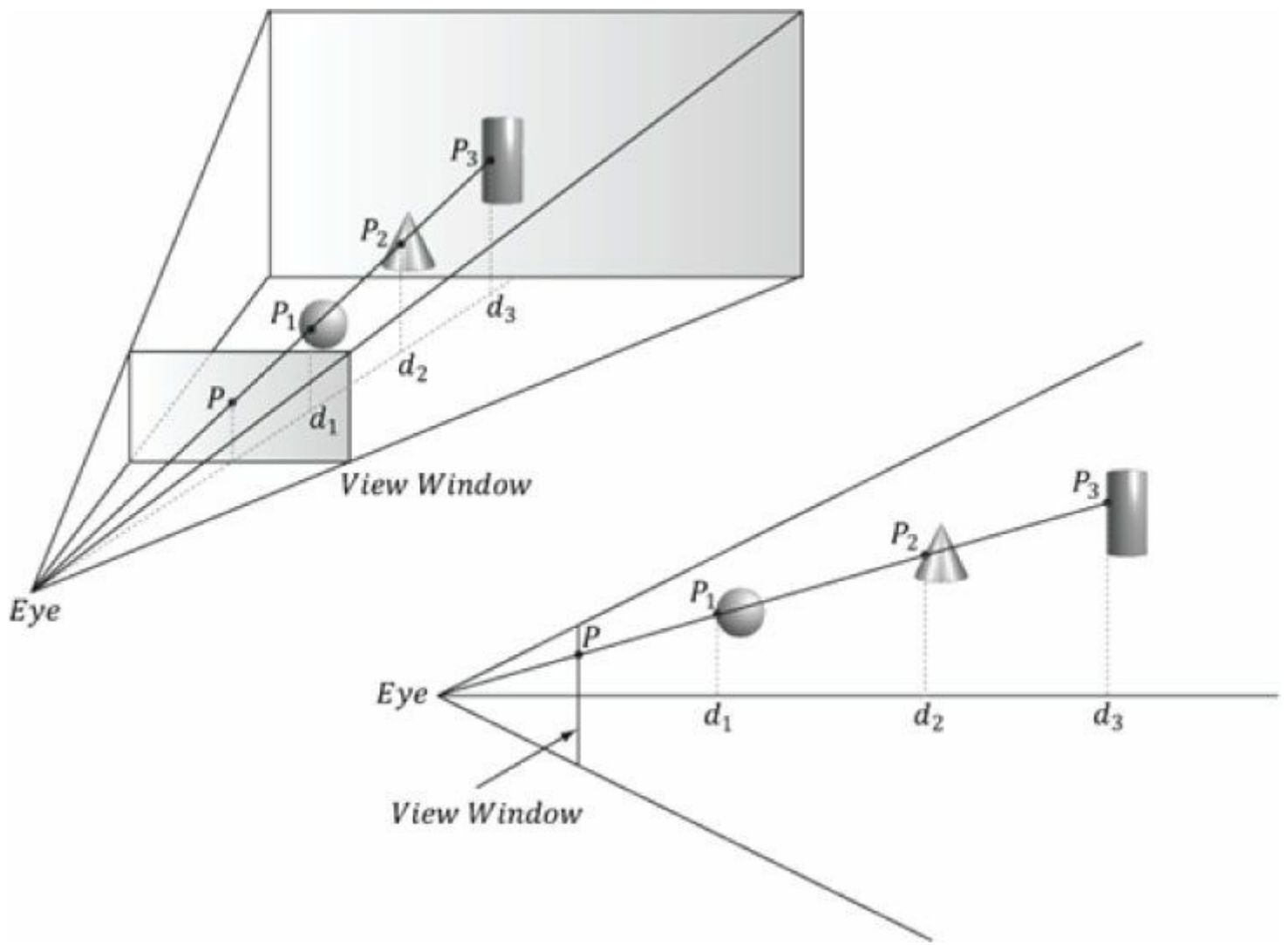


그림 10.12. 투영 창에서 단일 픽셀로 렌더링되기 위해 경쟁하는 다중 픽셀 조각들. 이 장면에서 픽셀 P의 깊이 복잡도는 3입니다.

깊이 복잡도 k 수준마다 색상 c_k 를 할당합니다. 예를 들어, 깊이 복잡도 1에는 파란색, 깊이 복잡도 2에는 녹색, 깊이 복잡도 3에는 빨간색을 할당하는 식입니다. (픽셀의 깊이 복잡도가 매우 커질 수 있는 매우 복잡한 장면에서는 각 수준마다 색상을 할당하지 않는 것이 좋습니다. 대신, 서로 다른 수준 범위에 색상을 할당할 수 있습니다. 예를 들어, 깊이 복잡도 1-5인 픽셀은 파란색, 깊이 복잡도 6-10인 픽셀은 녹색 등으로 색상을 지정합니다.)

스텐실 버퍼 작업을 **D3D11_STENCIL_OP_KEEP**로 설정하여 더 이상 수정하지 않도록 합니다. (씬 렌더링 시 깊이 복잡도를 계산할 때는 **D3DSTENCILOP_INCR**로 스텐실 버퍼를 수정하지만, 스텐실 버퍼를 시각화하는 코드를 작성할 때는 스텐실 버퍼에서 *읽기만 하면* 되며 쓰기 작업은 수행해서는 안 됩니다.)

깊이 복잡도 k 의 각 레벨에 대해:

스텐실 비교 함수를 **D3D11_COMPARISON_EQUAL**로 설정하고 스텐실 참조 값을 k 로 설정합니다.

(ii) 전체 투영 창을 덮는 색상 c_k 의 사각형을 그립니다. 이는 앞서 정의된 세트 스텐실 비교 함수와 참조 값으로 인해 깊이 복잡도가 k 인 픽셀만 채색함을 유의하십시오.

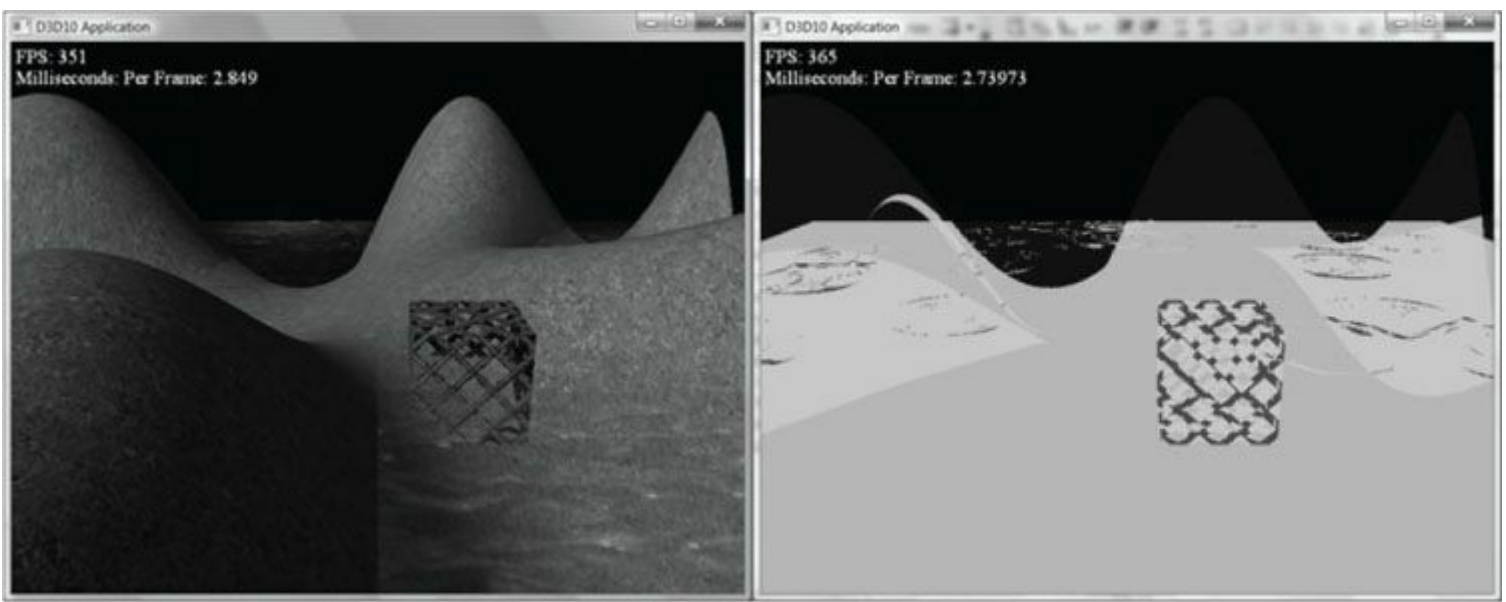


그림 10.13. 연습문제 8의 해결책 샘플 스크린샷.

이러한 설정으로 각 픽셀을 깊이 복잡도에 따라 고유하게 채색했으므로 장면의 깊이 복잡도를 쉽게 분석할 수 있습니다. 본 연습에서는 [제9장의 "Blend"](#) 데모에 사용된 장면의 깊이 복잡도를 렌더링하십시오. [그림 10.13](#)은 샘플 스크린샷을 보여줍니다.

깊이 테스트는 파이프라인의 출력 병합 단계에서 발생하며, 이는 픽셀 셰이더 단계 이후에 이루어집니다. 이는 픽셀 조각이 깊이 테스트에서 최종적으로 거부될 수 있음에도 불구하고 픽셀 셰이더를 통해 처리된다는 것을 의미합니다. 그러나 현대 하드웨어는 픽셀 셰이더 전에 깊이 테스트를 수행하는 "조기 Z 테스트"를 수행합니다. 이렇게 하면 거부된 픽셀 프래그먼트가 비용이 많이 들 수 있는 픽셀 셰이더로 처리되기 전에 버려집니다. 이 최적화를 활용하려면 블렌딩되지 않은 게임 오브젝트를 앞쪽에서 뒤쪽으로 렌더링하도록 노력해야 합니다.

Notes 카메라에 대한 역순 정렬: 이렇게 하면 가장 가까운 오브젝트가 먼저 렌더링되고, 그 뒤에 있는 오브젝트는 조기 Z-테스트에 실패하여 더 이상 처리되지 않습니다. 깊이 복잡도가 높아 과도한 오버드로우가 발생하는 장면에서는 상당한 성능 향상을 가져올 수 있습니다. Direct3D API를 통해 조기 Z-테스트를 제어할 수는 없습니다. 조기 Z-테스트 수행 가능 여부는 그래픽스 드라이버가 결정합니다. 예를 들어, 픽셀 셰이더가 픽셀 프래그먼트의 깊이 값을 수정하는 경우, 픽셀 셰이더가 깊이 값을 변경하므로 깊이 테스트 전에 픽셀 셰이더가 실행되어야 하므로 조기 Z-테스트가 불가능합니다.

Notes 픽셀 셰이더에서 픽셀의 깊이를 수정하는 기능에 대해 언급했습니다. 어떻게 작동할까요? 픽셀 셰이더는 지금까지 해왔던 것처럼 단일 색상 벡터뿐만 아니라 실제로 구조체를 출력할 수 있습니다.

```
struct PixelOut
{
    float4 color : SV_Target; float depth
    : SV_Depth;
};

PixelOut PS(VertexOut pin)
{
    PixelOut pout;

    // ... 일반적인 픽셀 작업

    pout.Color = float4(litColor, alpha);

    // 정규화된 [0, 1] 범위로 픽셀 깊이 설정 pout.depth = pin.PosH.z -
    0.05f;

    return pout;
}
```

SV_Position 요소의 *z*-좌표(`pin.PosH.z`)는 수정되지 않은 픽셀 깊이 값을 제공합니다. 특수 시스템

값 의미론 SV_Depth를 사용하여 픽셀 셰이더는 수정된 깊이 값을 출력할 수 있습니다.

9. 깊이 복잡도 시각화를 구현하는 또 다른 방법은 가산 혼합을 사용하는 것입니다. 먼저 백 버퍼를 검정색으로 지우고 깊이 테스트를 비활성화합니다. 다음으로 소스 및 대상 혼합 계수를 모두 D3D11_BLEND_ONE으로 설정하고 혼합 연산을 D3D11_BLEND_OP_ADD로 설정하여 혼합 방정식이 $C = C_{src} + C_{dst}$ 형태가 되도록 합니다. 이 공식에서는 각 픽셀에 대해 해당 픽셀에 기록된 모든 픽셀 프래그먼트의 색상을 누적한다는 점을 유의하십시오. 이제 장면 내 모든 오브젝트를 (0.05, 0.05, 0.05)와 같은 낮은 강도의 색상을 출력하는 픽셀 셰이더로 렌더링합니다. 픽셀의 오버드로우 횟수가 많을수록 이러한 낮은 강도의 색상이 더 많이 합산되어 픽셀의 밝기가 증가합니다. 예를 들어, 한 픽셀이 10번 오버드로우되었다면, 그 픽셀의 색상 강도는 (0.5, 0.5, 0.5)가 됩니다. 따라서 장면을 렌더링한 후 각 픽셀의 강도를 살펴봄으로써 장면의 깊이 복잡성에 대한 개념을 얻을 수 있습니다. 제9장의 "Blend" 데모를 테스트 장면으로 사용하여 이 버전의 깊이 복잡성 측정법을 구현해 보십시오.
10. 깊이 테스트를 통과하는 픽셀의 수를 계산하는 방법을 설명하십시오. 깊이 테스트를 통과하지 못하는 픽셀의 수를 계산하는 방법을 설명하십시오.
11. "Mirror" 데모를 수정하여 해골뿐만 아니라 바닥도 거울에 비치도록 하십시오.

제 11 장 지오메트리 셰이더

테셀레이션 단계를 사용하지 않는다고 가정할 때, 지오메트리 셰이더 단계는 버텍스 셰이더와 픽셀 셰이더 단계 사이에 위치하는 선택적 단계입니다. 버텍스 셰이더가 버텍스를 입력으로 받는 반면, 지오메트리 셰이더는 전체 프리미티브를 입력으로 받습니다. 예를 들어 삼각형 리스트를 그리는 경우, 지오메트리 셰이더 프로그램은 리스트 내 각 **삼각형 T**에 대해 실행됩니다:

```
for(UINT i = 0; i < 삼각형 수; ++i)
    OutputPrimitiveList = GeometryShader(T[i].vertexList);
```

각 삼각형의 세 정점이 지오메트리 셰이더로 입력되며, 지오메트리 셰이더는 프리미티브 목록을 출력합니다. 정점을 생성하거나 삭제할 수 없는 버텍스 셰이더와 달리, 지오메트리 셰이더의 주요 장점은 지오메트리를 생성하거나 삭제할 수 있다는 점입니다. 이를 통해 GPU에서 흥미로운 효과를 구현할 수 있습니다. 예를 들어, 입력된 프리미티브를 하나 이상의 다른 프리미티브로 확장하거나, 특정 조건에 따라 프리미티브를 출력하지 않도록 지오메트리 셰이더가 선택할 수 있습니다. 출력 프리미티브는 입력 프리미티브와 동일한 유형일 필요는 없습니다. 예를 들어, 지오메트리 셰이더의 일반적인 응용 사례는 점을 사각형(두 개의 삼각형)으로 확장하는 것입니다.

지오메트리 셰이더에서 출력되는 프리미티브는 버텍스 리스트로 정의됩니다. 지오메트리 셰이더를 떠나는 버텍스 위치는 동일한 클립 공간으로 변환되어야 합니다. 지오메트리 셰이더 단계 이후에는 동일한 클립 공간에서 프리미티브를 정의하는 버텍스 목록이 생성됩니다. 이 버텍스들은 투영(동일한 나눗셈)을 거친 후 평소와 같이 래스터라이제이션이 수행됩니다.

목표:

- 1. 지오메트리 셰이더 프로그래밍 방법을 배우기 위해.
- 2. 지오메트리 셰이더를 사용하여 빌보드를 효율적으로 구현하는 방법을 알아보기.
- 3. 자동 생성된 프리미티브 ID와 그 응용 사례를 파악하기.
- 4. 텍스처 배열을 생성하고 사용하는 방법을 알아보고, 그 유용성을 이해하기 위함입니다.
- 5. 알파 커트아웃의 앨리어싱 문제를 해결하는 데 알파-투-커버리지가 어떻게 도움이 되는지 이해하기.

11.1 지오메트리 셰이더 프로그래밍

지오메트리 셰이더 프로그래밍은 버텍스 또는 픽셀 셰이더 프로그래밍과 매우 유사하지만 몇 가지 차이점이 있습니다. 다음 코드는 일반적인 형태를 보여줍니다:

```
[maxvertexcount(N)] void 셰이더
명 (
    PrimitiveType InputVertexType InputName [NumElements], inout
    StreamOutputObject<OutputVertexType> OutputName)
{
    // 지오메트리 셰이더 본체...
}
```

먼저 단일 호출(지오메트리 셰이더는 프리미티브별로 호출됨)에 대해 지오메트리 셰이더가 출력할 최대 정점 수를 지정해야 합니다. 이는 셰이더 정의 전에 다음 속성 구문을 사용하여 최대 정점 수를 설정함으로써 수행됩니다:

```
[maxvertexcount(N)]
```

여기서 **N**은 단일 호출 시 지오메트리 셰이더가 출력할 수 있는 최대 정점 수입니다. 지오메트리 셰이더가 호출당 출력할 수 있는 정점 수는 가변적이지만, 정의된 최대값을 초과할 수 없습니다. 성능을 위해 **maxvertexcount**는 가능한 한 작게 설정해야 합니다. [NVIDIA08]에 따르면 GS가 1~20개의 스칼라를 출력할 때 최고 성능을 달성하며, 27~40개의 스칼라를 출력할 경우 성능이 50%로 떨어집니다. 호출당 스칼라 출력 수는 *maxvertexcount*와 출력 정점 유형 구조체 내 스칼라 수의 곱입니다. 이러한 제약 조건 하에서 작업하는 것은 실제로 어렵기 때문에, 우리는 최고 성능보다 낮은 수준을 충분히 좋다고 받아들이거나, 지오메트리 셰이더를 사용하지 않는 대체 구현을 선택할 수 있습니다.

구현을 선택할 수 있습니다. 그러나 대체 구현 방식에도 다른 단점이 존재할 수 있어, 여전히 지오메트리 셰이더 구현이 더 나은 선택일 수 있다는 점도 고려해야 합니다. 또한 [NVIDIA08]의 권고 사항은 2008년(1세대 지오메트리 셰이더) 기준이므로, 현재는 상황이 개선되었을 것입니다.

지오메트리 셰이더는 두 개의 매개변수를 받습니다: 입력 매개변수와 출력 매개변수입니다. (사실 더 많은 매개변수를 받을 수 있지만, 이는 특별한 주제이므로 §11.2.4를 참조하십시오.) 입력 매개변수는 항상 원시체를 정의하는 정점 배열입니다. 점은 하나의 정점, 선은 두 개의 정점, 삼각형은 세 개의 정점, 인접성을 가진 선은 네 개의 정점, 인접성을 가진 삼각형은 여섯 개의 정점으로 표현됩니다. 입력 정점의 정점 유형은 버텍스 셰이더가 반환하는 정점 유형(예: **VertexOut**)입니다. 입력 매개변수 앞에는 기하학 셰이더로 입력 되는 프리미티브의 유형을 설명하는 프리미티브 유형이 반드시 접두사로 붙어야 합니다. 다음 중 하나일 수 있습니다:

- 1. **point**: 입력 프리미티브는 점입니다.
- 2. **선**: 입력 원시체는 선(목록 또는 스트립)입니다.
- 3. **삼각형**: 입력 기본 요소는 삼각형(목록 또는 스트립)입니다.
- 4. **lineadj**: 입력 원시체는 인접성을 가진 선분(목록 또는 스트립)입니다.
- 5. **삼각형 인접**: 입력 원시 객체는 인접성을 가진 삼각형(목록 또는 스트립)입니다.

지오메트리 셰이더로 입력되는 프리미티브는 항상 완전한 프리미티브입니다(예: 선분에는 두 개의 정점, 삼각형에는 세 개의 정점). 따라서 지오메트리 셰이더는 리스트와 스트립을 구분할 필요가 없습니다. 예를 들어,

Note: *삼각형 스트립을 렌더링하는 경우에도 스트립 내 각 삼각형마다 지오메트리 셰이더가 실행되며, 각 삼각형의 세 정점이 입력으로 지오메트리 셰이더에 전달됩니다. 이는 여러 프리미티브가 공유하는 정점이 지오메트리 셰이더에서 여러 번 처리되므로 추가적인 오버헤드를 수반합니다.*

출력 매개변수는 항상 **inout** 수정자를 가집니다. 또한 출력 매개변수는 항상 스트림 유형입니다. 스트림 유형은 지오메트리 셰이더가 출력하는 지오메트리를 정의하는 정점 목록을 저장합니다. 지오메트리 셰이더는 내장 **Append** 메서드를 사용하여 나가는 스트림 목록에 정점을 추가합니다:

```
void StreamOutputObject<OutputVertexType>::Append(OutputVertexType v);

스트림 유형은 템플릿 유형으로, 템플릿 인자는 출력 정점의 정점 유형을 지정하는 데 사용됩니다(예:
```

GeoOut). 가능한 스트림 유형은 세 가지입니다:

- 1. **PointStream<OutputVertexType>**: 점 목록을 정의하는 정점 목록입니다.
- 2. **LineStream<OutputVertexType>**: 라인 스트립을 정의하는 정점 목록입니다.
- 3. **TriangleStream<OutputVertexType>**: 삼각형 스트립을 정의하는 정점 목록입니다.

지오메트리 셰이더가 출력하는 정점은 프리미티브를 형성하며, 출력 프리미티브의 유형은 스트림 유형(**PointStream**, **LineStream**, **TriangleStream**)으로 표시됩니다. 선과 삼각형의 경우 출력 프리미티브는 항상 스트립입니다. 그러나 선 및 삼각형 리스트는 내장 **RestartStrip** 메서드를 사용하여 시물레이션할 수 있습니다:

```
void StreamOutputObject<OutputVertexType>::RestartStrip();

예를 들어, 삼각형 목록을 출력하려면 출력 스트림에 정점 3개가 추가될 때마다 RestartStrip을 호출해야 합니다.
다음은 지오메트리 셰이더 시그니처의 구체적인 예시입니다:
```

```
// 예제 1: GS는 최대 4개의 정점을 출력합니다. 입력 프리미티브는 선입니다.
// 출력은 삼각형 스트립입니다.
// [maxvertexcount(4)]
void GS(line VertexOut gin[2],
        inout TriangleStream<GeoOut> triStream)
{
    // 지오메트리 셰이더 본체...
}
//
// 예제 2: GS는 최대 32개의 정점을 출력합니다. 입력 프리미티브는
// 삼각형. 출력은 삼각형 스트립입니다.
// [maxvertexcount(32)]
void GS(삼각형 VertexOut gin[3],
        inout TriangleStream<GeoOut> triStream)
{
```

```

// 지오메트리 셰이더 본체...
}
//
// 예제 3: GS는 최대 4개의 정점을 출력합니다. 입력 프리미티브는
// 은 점입니다. 출력은 삼각형 스트림입니다.
// [maxvertexcount(4)]
void GS(point VertexOut gin[1],
        inout TriangleStream<GeoOut> triStream)
{
    // 지오메트리 셰이더 본체...
}

```

다음 지오메트리 셰이더는 Append 및 RestartStrip 메서드를 보여줍니다. 삼각형을 입력으로 받아 이를 세분화(그림 11.1)한 후, 세분화된 네 개의 삼각형을 출력합니다:

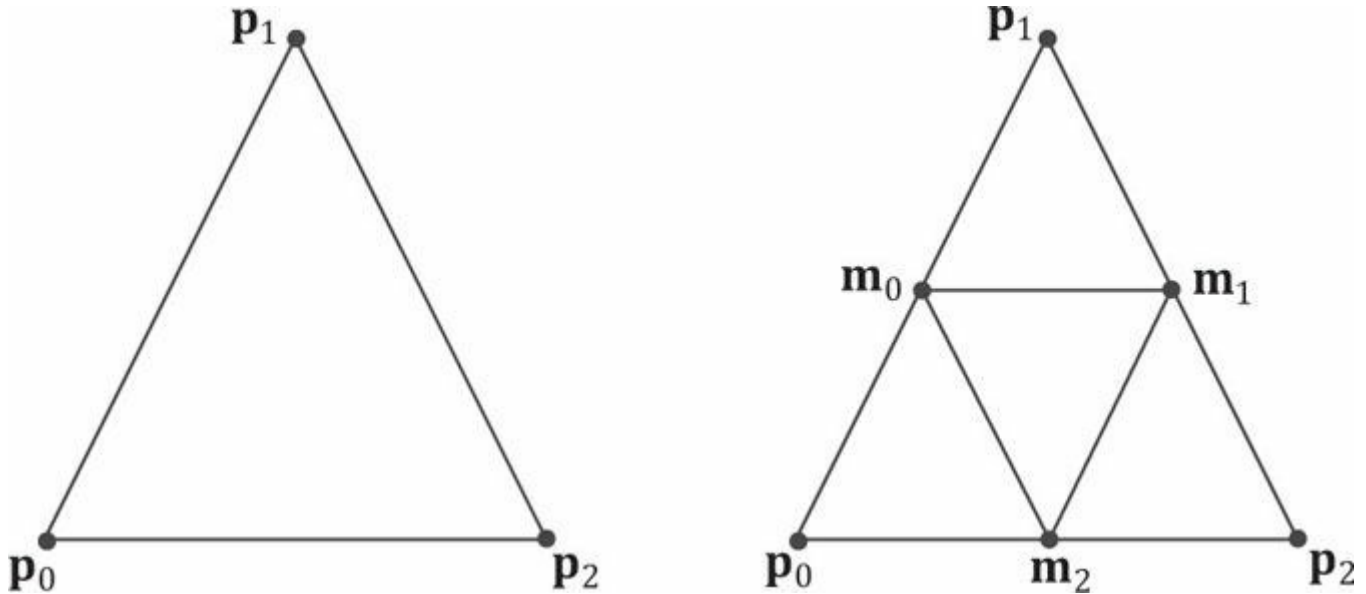


그림 11.1. 삼각형을 네 개의 동일한 크기의 삼각형으로 분할하는 모습. 세 개의 새로운 꼭짓점이 원래 삼각형의 변을 따라 위치한 중간점임을 관찰하십시오.

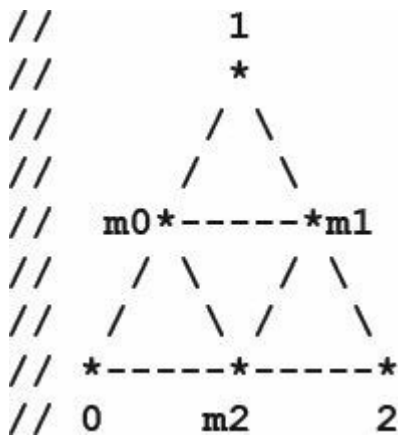
```

struct VertexOut
{
    float3 PosL      : 위치; float3
    NormalL : 법선; float2 Tex  :
    TEXCOORD;
};

struct GeoOut
{
    float4 PosH      : SV_POSITION; float3
    PosW      : POSITION; float3
    NormalW : NORMAL; float2 Tex      :
    TEXCOORD; float FogLerp : FOG;
};

void Subdivide(VertexOut inVerts[3], out VertexOut outVerts[6])
{

```



```
VertexOut m[3];
// 에지 중간점 계산.
m[0].PosL = 0.5f*(inVerts[0].PosL+inVerts[1].PosL); m[1].PosL =
0.5f*(inVerts[1].PosL+inVerts[2].PosL); m[2].PosL =
0.5f*(inVerts[2].PosL+inVerts[0].PosL);

// 단위 구에 투영 m[0].PosL =
normalize(m[0].PosL); m[1].PosL =
normalize(m[1].PosL); m[2].PosL =
normalize(m[2].PosL);

// 법선 도출. m[0].NormalL = m[0].PosL;
m[1].NormalL = m[1].PosL;
m[2].NormalL = m[2].PosL;

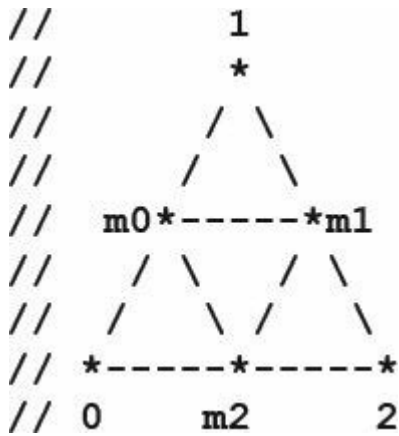
// 텍스처 좌표 보간.
m[0].Tex = 0.5f*(inVerts[0].Tex+inVerts[1].Tex); m[1].Tex =
0.5f*(inVerts[1].Tex+inVerts[2].Tex); m[2].Tex =
0.5f*(inVerts[2].Tex+inVerts[0].Tex);

outVerts[0] = inVerts[0]; outVerts[1] =
m[0]; outVerts[2] = m[2]; outVerts[3] =
m[1]; outVerts[4] = inVerts[2];
outVerts[5] = inVerts[1];
};

void OutputSubdivision(VertexOut v[6],
    inout TriangleStream<GeoOut> triStream)
{
    GeoOut gout[6];

    [unroll]
    for(int i = 0; i < 6; ++i)
    {
        // 월드 공간으로 변환.
        gout[i].PosW = mul(float4(v[i].PosL, 1.0f), gWorld).xyz; gout[i].NormalW =
        mul(v[i].NormalL,
            (float3x3)gWorldInvTranspose);

        // 동차 클립 공간으로 변환.
        gout[i].PosH = mul(float4(v[i].PosL, 1.0f), gWorldViewProj); gout[i].Tex = v[i].Tex;
    }
}
```



// 서브디비전을 두 개의 스트림으로 그릴 수 있습니다:

// 스트림 1: 하단 삼각형 3개

// 스트림 2: 상단 삼각형

```
[unroll]
for(int j = 0; j < 5; ++j)
{
    triStream.Append(gout[j]);
}
triStream.RestartStrip();

triStream.Append(gout[1]);
triStream.Append(gout[5]);
triStream.Append(gout[3]);
}
[maxvertexcount(8)]
void GS(triangle VertexOut gin[3], inout TriangleStream<GeoOut>)
{
    VertexOut v[6]; Subdivide(gin,
    v);
    OutputSubdivision(v, 삼각형 스트림);
}
```

지오메트리 셰이더가 구현되면 렌더링 시 사용하려면 효과 패스에 바인딩해야 합니다:

```
technique11 Tech
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS())); SetGeometryShader(Compile
        Shader(gs_5_0, GS())); SetPixelShader(CompileShader(ps_5_0, PS()));
    }
}
```

Note:

입력된 프리미티브에 대해, 지오메트리 셰이더는 특정 조건에 따라 이를 출력하지 않을 수 있습니다. 이러한 방식으로 지오메트리는 지오메트리 셰이더에 의해 "파괴"되며, 이는 일부 알고리즘에 유용할 수 있습니다.

Note:

지오메트리 셰이더에서 프리미티브를 완성하기에 충분한 정점을 출력하지 않으면 부분 프리미티브는 버려집니다.

11.2 트리 빌보드 데모

11.2.1 개요

나무가 멀리 있을 때 효율성을 위해 *빌보드* 기법이 사용됩니다. 즉, 완전한 3D 나무의 지오메트리를 렌더링하는 대신, 3D 나무의 그림이 그려진 사각형(쿼드)을 그 위에 칠합니다([그림 11.2](#) 참조). 멀리서 보면 빌보드가 사용되고 있다는 것을 알 수 없습니다. 그러나 핵심은 빌보드가 항상 카메라를 향하도록 하는 것입니다(그렇지 않으면 환상이 깨집니다).

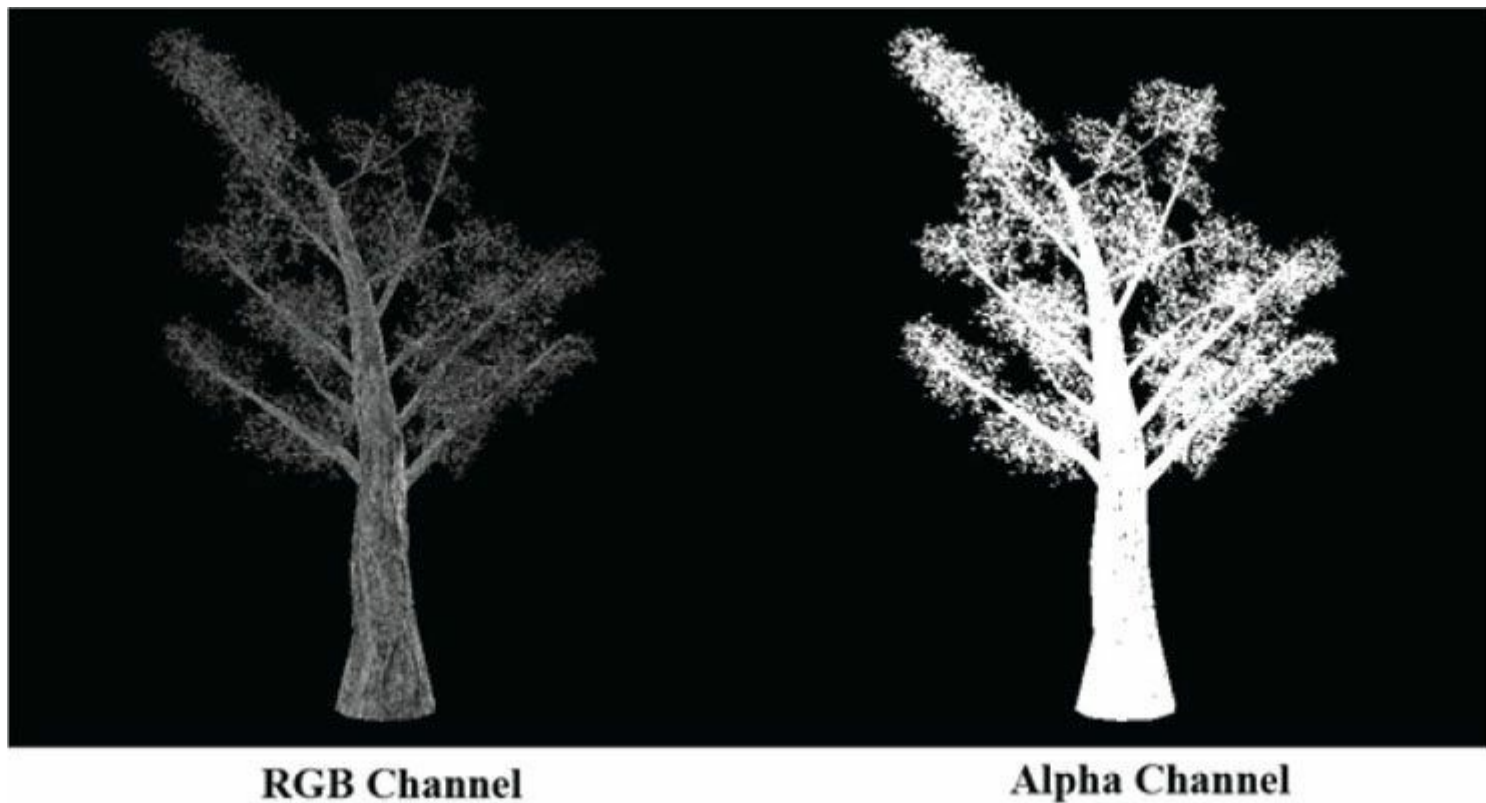


그림 11.2. 알파 채널이 포함된 나무 빌보드 텍스처.

y축이 위를 향하고 xz평면이 지면 평면이라고 가정할 때, 트리 빌보드들은 일반적으로 y축과 정렬되며 xz평면에서 카메라를 향하게 됩니다. [그림 11.3](#)은 새의 눈높이에서 본 여러 빌보드의 국소 좌표계를 보여줍니다—빌보드들이 카메라를 "바라보고" 있음을 주목하십시오.

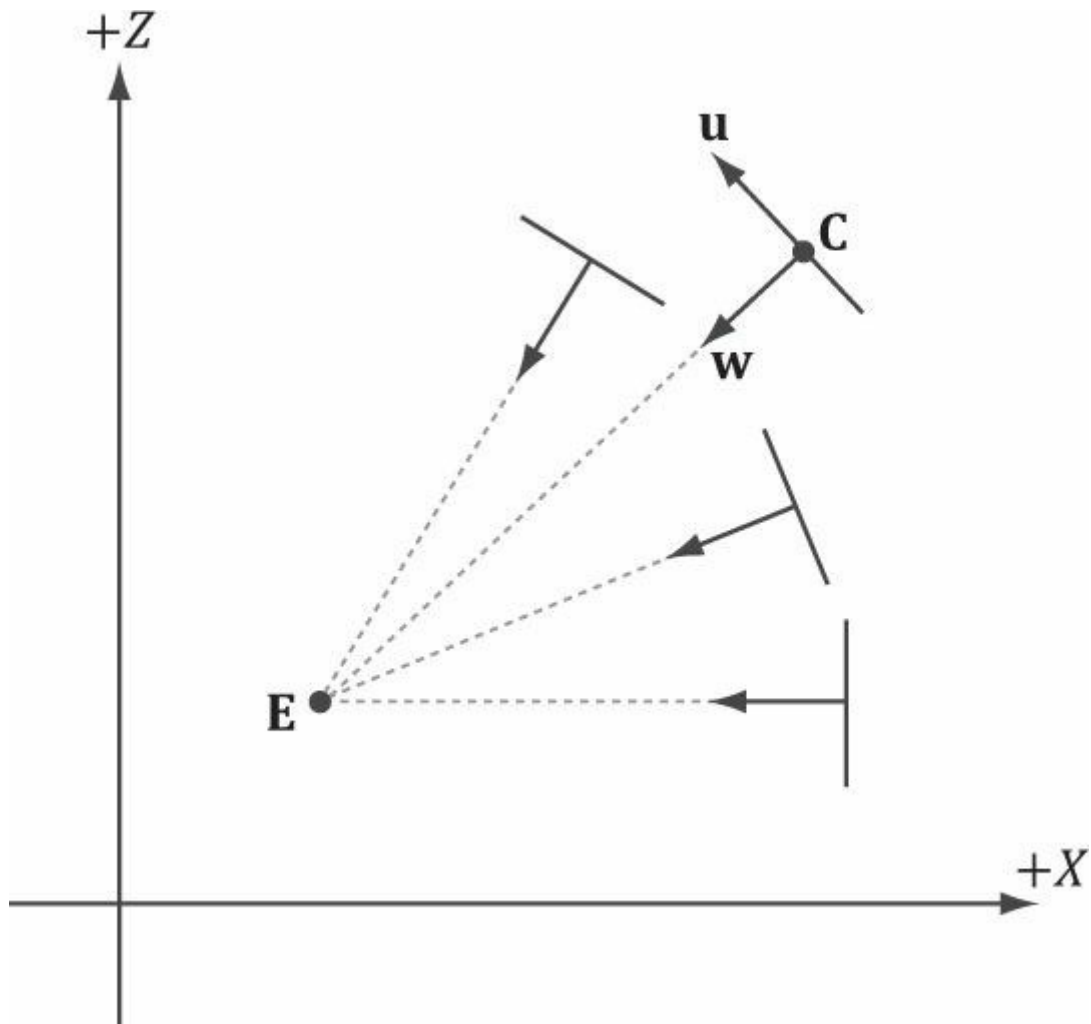


그림 11.3. 카메라를 향한 빌보드들.

따라서 월드 공간 내 빌보드의 중심 위치 $C = (C_x, C_y, C_z)$ 와 월드 공간 내 카메라 위치 $E = (E_x, E_y, E_z)$ 가 주어지면, 월드 공간에 대한 빌보드의 로컬 좌표계를 설명하기에 충분한 정보를 갖게 됩니다:

$$\mathbf{w} = \frac{(E_x - C_x, 0, E_z - C_z)}{\|(E_x - C_x, 0, E_z - C_z)\|}$$

$$\mathbf{v} = (0, 1, 0)$$

$$\mathbf{u} = \mathbf{v} \times \mathbf{w}$$

빌보드의 세계 좌표계에 대한 로컬 좌표계와 빌보드의 세계 크기가 주어지면, 빌보드 쿼드 정점은 다음과 같이 구할 수 있습니다(그림 11.4 참조):

```
v[0] = float4(gin[0].CenterW + halfWidth*right - halfHeight*up, 1.0f); v[1] = float4(gin[0].CenterW + halfWidth*right + halfHeight*up, 1.0f); v[2] = float4(gin[0].CenterW - halfWidth*right - halfHeight*up, 1.0f); v[3] = float4(gin[0].CenterW - halfWidth*right + halfHeight*up, 1.0f);
```

빌보드마다 로컬 좌표계가 다르므로 각 빌보드마다 계산해야 합니다.

이 데모에서는 육지대륙의 약간 위에 위치한 점 프리미티브(D3D11_PRIMITIVE_TOPOLOGY_POINTLIST) 목록을 구성합니다.

이 점들은 우리가 그리려는 빌보드의 중심을 나타냅니다. 지오메트리 셰이더에서 이 점들을 빌보드 쿼드로 확장할 것입니다. 또한 지오메트리 셰이더에서 빌보드의 월드 매트릭스를 계산할 것입니다. [그림 11.5](#)는 데모의 스크린샷을 보여줍니다.

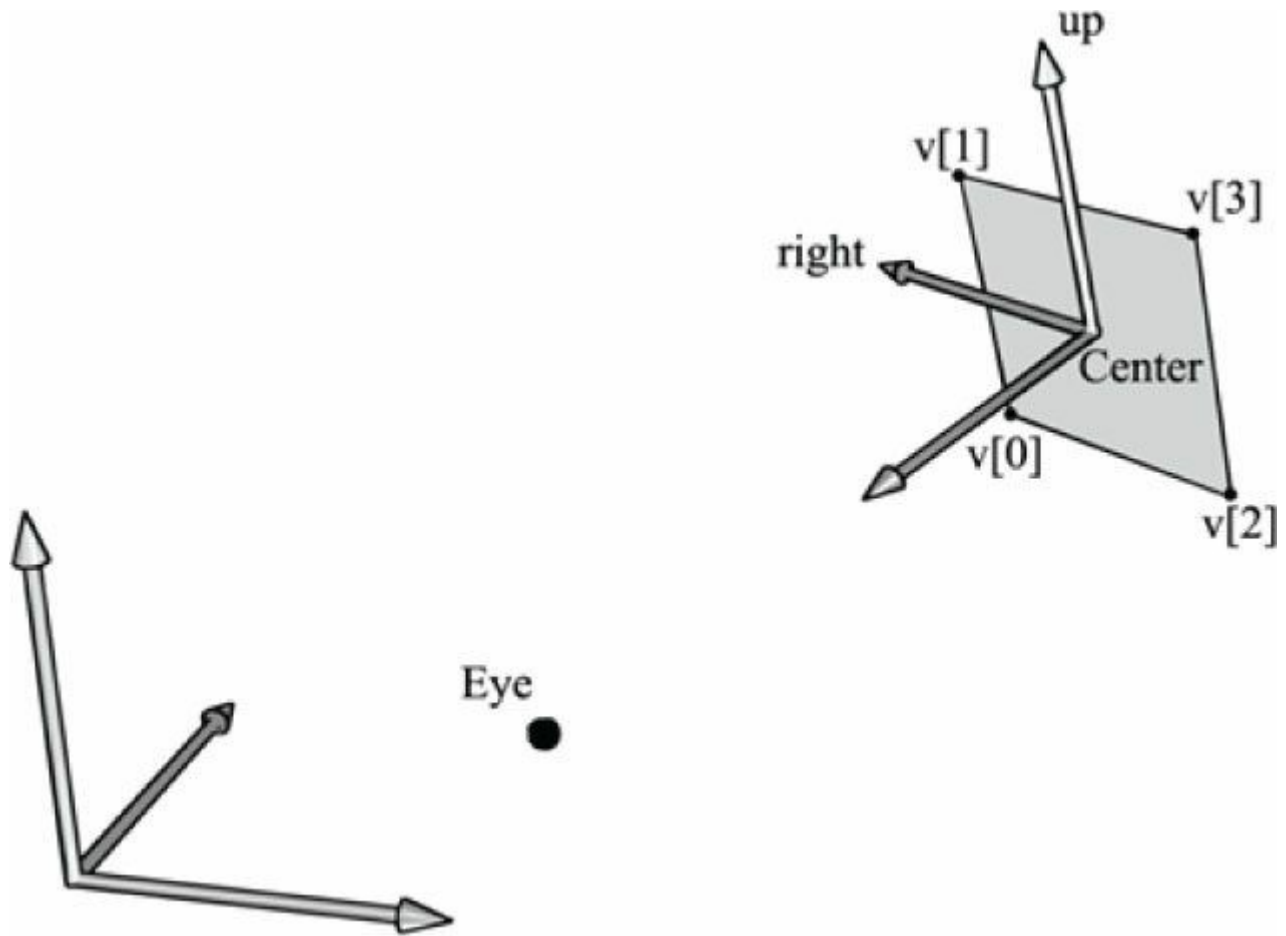


그림 11.4. 로컬 좌표계와 빌보드의 월드 크기를 사용하여 빌보드 사각형의 정점을 계산하는 과정.



그림 11.5. "트리 빌보드" 데모의 스크린샷.

그림 11.5에서 볼 수 있듯이, 이 샘플은 제9장의 "Blend" 데모를 기반으로 구축되었습니다.

빌보드의 일반적인 CPU 구현 방식은 동적 정점 버퍼에 빌보드당 네 개의 정점을 사용하는 것입니다. 그런 다음 카메라가 움직일 때마다 `ID3D11DeviceContext::Map` 메서드를 사용하여 CPU에서 정점을 업데이트함으로써 빌보드가 카메라를 향하도록 합니다. 이 접근 방식은 빌보드당 네 개의 정점을 1A 단계에 제출해야 합니다.

 동적 버텍스 버퍼를 업데이트해야 하므로 오버헤드가 발생합니다. 지오메트리 셰이더 방식을 사용하면 정적 버텍스 버퍼를 사용할 수 있습니다. 지오메트리 셰이더가 빌보드 확장을 수행하고 빌보드가 카메라를 향하도록 만들기 때문입니다. 또한 빌보드의 메모리 사용량은 매우 적습니다. 각 빌보드당 하나의 버텍스만 1A 단계에 제출하면 되기 때문입니다.

11.2.2 버텍스 구조

빌보드 포인트에는 다음과 같은 버텍스 구조를 사용합니다.

```
struct TreePointSprite
{
    XMFLOAT3 Pos; XMFLOAT2
    Size;
};

const D3D11_INPUT_ELEMENT_DESC InputLayoutDesc::TreePointSprite[2] =
{
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"SIZE", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 12,
D3D11_INPUT_PER_VERTEX_DATA, 0}
};
```

정점은 월드 공간에서 빌보드의 중심 위치를 나타내는 점을 저장합니다. 또한 빌보드의 너비/높이(월드 공간 단위로 스케일링됨)를 저장하는 크기 멤버를 포함합니다. 이는 지오메트리 셰이더가 확장 후 빌보드의 크기를 알 수 있도록 하기 위함입니다([그림 11.6](#)). 정점마다 크기를 다르게 설정함으로써 서로 다른 크기의 빌보드를 쉽게 구현할 수 있습니다.

텍스처 배열([§11.2.4](#))을 제외하면, "Tree Billboard" 데모의 다른 C++ 코드는 이제 일상적인 Direct3D 코드여야 합니다 (버텍스 버퍼 생성, 이펙트 생성, 드로우 메서드 호출 등). 따라서 이제 *TreeSprite.fx* 파일에 주목하겠습니다.

11.2.3 이펙트 파일

이것은 지오메트리 셰이더를 사용한 첫 번째 데모이므로, 전체 효과 파일을 보여드리겠습니다. 이를 통해 버텍스 셰이더, 픽셀 셰이더 및 다른 효과 오브젝트들과 어떻게 결합되는지 확인하실 수 있습니다. 이 효과는 아직 다루지 않은 몇 가지 새로운 오브젝트(`SV_PrimitiveID` 및 `Texture2DArray`)도 소개합니다. 이 항목들은 다음에 설명하겠습니다. 지금은 주로 지오메트리 셰이더 프로그램 **GS**에 집중하세요. 이 셰이더는

§11.2.1.

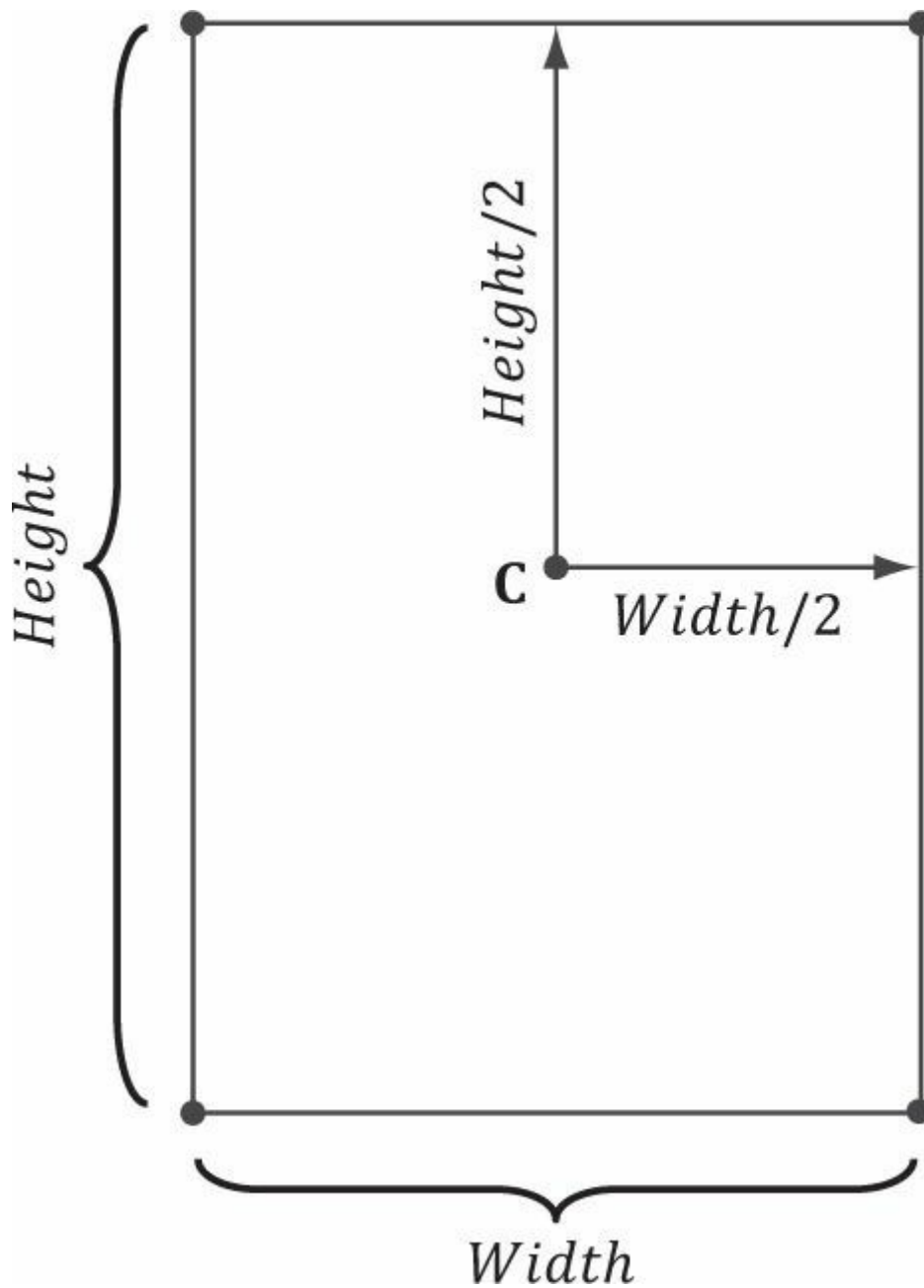


그림 11.6. 점을 사각형으로 확장하기.

```

//*****
// TreeSprite.fx by Frank Luna (C) 2011 All Rights Reserved.
//
// 지오메트리 셰이더를 사용하여 점 스프라이트를 y축에 정렬된 사각형으로 확장합니다.
// 카메라를 향하는 빌보드.
//***** #include "LightHelper.fx"

cbuffer cbPerFrame
{
    DirectionalLight gDirLights[3]; float3
    gEyePosW;

    float gFogStart; float
    gFogRange; float4
    gFogColor;
};

cbuffer cbPerObject
{
    float4x4 gViewProj;

```

```

    Material gMaterial;
};

cbuffer cbFixed
{
    //
    // 텍스처를 사각형에 늘려 적용하기 위한 텍스처 좌표 계산.
    //

    float2 gTexC[4] =
    {
        float2(0.0f, 1.0f),
        float2(0.0f, 0.0f),
        float2(1.0f, 1.0f), float2(1.0f,
        0.0f)
    };
};

// 비수치 값은 cbuffer에 추가할 수 없습니다. Texture2DArray
gTreeMapArray;

SamplerState samLinear
{
    Filter = MIN_MAG_MIP_LINEAR; AddressU = WRAP;
    AddressV = WRAP;
};

struct VertexIn
{
    float3 PosW : POSITION; float2 SizeW :
    SIZE;
};

struct VertexOut
{
    float3 CenterW : POSITION; float2 SizeW :
    SIZE;
};

struct GeoOut
{
    float4 PosH      : SV_POSITION; float3
    PosW      : POSITION; float3
    NormalW : NORMAL; float2 Tex      :
    TEXCOORD;
    uint PrimID      : SV_PrimitiveID;
};

VertexOut VS(VertexIn vin)
{
    VertexOut vout;

    // 단순히 데이터를 지오메트리 셰이더로 전달합니다.
    vout.CenterW = vin.PosW;
    vout.SizeW = vin.SizeW;

    return vout;
}

// 각 점을 쿼드(4개의 정점)로 확장하므로,
지오메트리 셰이더 호출당 출력되는 정점 수는 4개입니다. [maxvertexcount(4)]

```

```

void GS(point VertexOut gin[1], uint primID :
    SV_PrimitiveID,
    inout TriangleStream<GeoOut> triStream)
{
    //
    // 빌보드의 y축과 정렬되도록
    // 빌보드가 y축과 정렬되고
    //
    //
    float3 up = float3(0.0f, 1.0f, 0.0f);
    float3 look = gEyePosW - gin[0].CenterW;
    look.y = 0.0f; // y축 정렬, 따라서 xz 평면으로 투영 look = normalize(look);

    float3 right = cross(up, look);

    //
    // 세계 좌표계에서 삼각형 스트립 정점(사각형) 계산.
    //
    float halfWidth = 0.5f*gin[0].SizeW.x; float halfHeight =
    0.5f*gin[0].SizeW.y;

    float4 v[4];
    v[0] = float4(gin[0].CenterW + halfWidth*right - halfHeight*up, 1.0f); v[1] = float4(gin[0].CenterW +
    halfWidth*right + halfHeight*up, 1.0f); v[2] = float4(gin[0].CenterW - halfWidth*right -
    halfHeight*up, 1.0f); v[3] = float4(gin[0].CenterW - halfWidth*right + halfHeight*up, 1.0f);

    //
    // 사각형 정점을 월드 공간으로 변환하고
    // 삼각형 스트립으로 출력합니다.
    //
    GeoOut gout; [unroll]
    for(int i = 0; i < 4; ++i)
    {
        gout.PosH          = mul(v[i], gViewProj);
        gout.PosW          = v[i].xyz; gout.NormalW =
        look;
        gout.Tex = gTexC[i]; gout.PrimID
        = primID; triStream.Append(gout);
    }
}

float4 PS(GeoOut pin, uniform int gLightCount, uniform bool gUseTexure, uniform bool gAlphaClip, uniform bool gFogEnabled) : SV_Target
{
    // 보간된 법선은 비정규화될 수 있으므로 정규화합니다. pin.NormalW =
    normalize(pin.NormalW);

    // toEye 벡터는 조명 계산에 사용됩니다. float3 toEye =
    gEyePosW - pin.PosW;

    // 이 표면 점에서 눈까지의 거리를 캐시합니다. float distToEye = length(toEye);

    // 정규화.
    toEye /= distToEye;

    // 곱셈 항등원으로 기본 설정. float4 texColor =
    float4(1, 1, 1, 1); if(gUseTexure)

```

```

{
    // 텍스처 샘플링.
    float3 uvw = float3(pin.Tex, pin.PrimID%4); texColor =
    gTreeMapArray.Sample(samLinear, uvw); if(gAlphaClip)
    {
        // 텍스처 알파 < 0.05인 픽셀은 버림. 참고:
        // 가능한 한 빨리 이 테스트를 수행하여
        // 셰이더를 조기에 종료할 수 있도록 하여
        // 나머지 셰이더 코드를 건너뛸 수 있도록 합니다.
        clip(texColor.a - 0.05f);
    }
}

//
// 조명.
//

float4 litColor = texColor; if(gLightCount > 0)
{
    // 합계를 0으로 시작합니다.
    float4 ambient = float4(0.0f, 0.0f, 0.0f, 0.0f); float4 diffuse =
    float4(0.0f, 0.0f, 0.0f, 0.0f); float4 spec          = float4(0.0f,
    0.0f, 0.0f, 0.0f);
    // 각 광원의 빛 기여도를 합산합니다. [unroll]
    for(int i = 0; i < gLightCount; ++i)
    {
        float4 A, D, S; ComputeDirectionalLight(gMaterial, gDirLights[i],
        pin.NormalW, toEye, A, D,
        S);
        ambient += A;
        diffuse += D; spec+=
        S;
    }
    // 후기 합성으로 변조.
    litColor = texColor*(ambient + diffuse) + spec;
}

//
// 포깅
//

if(gFogEnabled)
{
    float fogLerp = saturate((distToEye - gFogStart) / gFogRange);
    // 안개 색상과 조명 색상을 블렌딩합니다. litColor = lerp(litColor,
    gFogColor, fogLerp);
}

// 디퓨즈 머티리얼과 텍스처에서 알파를 가져오는 공통 처리. litColor.a =
gMaterial.Diffuse.a * texColor.a;
return litColor;
}

//-----
// 테크닉--데모에 필요한 것만 정의하세요; 필요에 따라 다른
// 변형은 필요에 따라 정의하세요.
//-----

```

```

technique11 Light3
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS()));
        SetGeometryShader(CompileShader(gs_5_0, GS())); SetPixelShader(CompileShader(ps_5_0,
        PS(3, false, false, false)));
    }
}

technique11 Light3TexAlphaClip
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS()));
        SetGeometryShader(CompileShader(gs_5_0, GS())); SetPixelShader(CompileShader(ps_5_0,
        PS(3, true, true, false)));
    }
}

technique11 Light3TexAlphaClipFog
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS()));
        SetGeometryShader(CompileShader(gs_5_0, GS())); SetPixelShader(CompileShader(ps_5_0,
        PS(3, true, true, true)));
    }
}

```

11.2.4 SV_PrimitiveID

이 예제의 지오메트리 셰이더는 **SV_PrimitiveID** 의미론의 특별한 부호 없는 정수 매개변수를 사용합니다.

```

[maxvertexcount(4)]
void GS(point VertexOut gin[1],
    uint primID : SV_PrimitiveID,
    inout TriangleStream<GeoOut> triStream)

```

이 시맨틱이 지정되면 입력 어셈블러 단계에서 각 프리미티브에 대해 프리미티브 ID를 자동으로 생성하도록 지시합니다. *n개의* 프리미티브를 그리기 위해 드로우 콜이 실행될 때, 첫 번째 프리미티브는 0으로, 두 번째 프리미티브는 1로, 그리고 마지막 프리미티브는 *n-1*로 라벨링됩니다. 프리미티브 ID는 단일 드로우 콜 내에서만 고유합니다. 빌보드 예제에서 지오메트리 셰이더는 이 ID를 사용하지 않습니다(지오메트리 셰이더가 사용할 수도 있음). 대신 지오메트리 셰이더는 프라이머리 ID를 출력 버텍스에 기록하여 픽셀 셰이더 단계로 전달합니다. 픽셀 셰이더는 프라이머리 ID를 사용하여 텍스처 배열에 인덱싱하며, 이는 다음 섹션으로 이어집니다.

 지오메트리 셰이더가 없는 경우, 프라이머리 ID 매개변수를 픽셀 셰이더의 매개변수 목록에 추가할 수 있습니다.

```

float4 PS(VertexOut pin, uint primID : SV_PrimitiveID) : SV_Target
{
    // 픽셀 셰이더 본문...
}

```

그러나 지오메트리 셰이더가 존재하는 경우, 프리미티브 ID 매개변수는 지오메트리 셰이더 시그니처에 반드시 포함되어야 합니다. 그러면 지오메트리 셰이더는 프리미티브 ID를 사용하거나 픽셀 셰이더 단계로 전달할 수 있습니다(또는 둘 다).

입력 어셈블러가 버텍스 ID를 생성하도록 설정할 수도 있습니다. 이를 위해 추가 매개변수를 다음 유형으로 추가하십시오:

uint를 시맨틱 **SV_VertexID**를 가진 버텍스 셰이더 시그니처에 추가합니다.

다음 버텍스 셰이더 시그니처는 이를 구현한 예시입니다.

```

VertexOut VS(VertexIn vin, uint vertID : SV_VertexID)
{
    // 버텍스 셰이더 본체...
}

```

드로우 쿨의 경우, 드로우 쿨 내 정점들은 $0, 1, \dots, n-1$ 의 ID로 라벨링됩니다. 여기서 n 은 드로우 쿨 내 정점의 수입니다. 드로우 인덱스드 쿨의 경우, 정점 ID는 정점 인덱스 값에 대응합니다.

11.3 텍스처 배열

11.3.1 개요

텍스처 배열은 텍스처 배열을 저장합니다. C++ 코드에서 텍스처 배열은 **ID3D11Texture2D** 인터페이스(단일 텍스처에도 사용되는 동일한 인터페이스)로 표현됩니다. **ID3D11Texture2D** 객체를 생성할 때 실제로 **ArraySize**라는 속성을 설정하여 텍스처가 저장하는 텍스처 요소 수를 지정할 수 있습니다. 그러나 텍스처 생성에 D3DX를 의존해 왔기 때문에 이 데이터 멤버를 명시적으로 설정하지 않았습니다. 효과 파일에서는 텍스처 배열이 Texture2DArray 유형으로 표현됩니다:

```
Texture2DArray gTreeMapArray;
```

이제 텍스처 배열이 왜 필요한지 궁금하실 겁니다. 그냥 이렇게 하면 안 될까요?

```

Texture2D TexArray[4];
...
float4 PS(GeoOut pin) : SV_Target
{
    float4 c = TexArray[pin.PrimID%4].Sample(samLinear, pin.Tex);
}

```

이 코드는 "샘플러 배열 인덱스는 리터럴 표현식이어야 합니다"라는 오류 메시지를 발생시킵니다. 즉, 픽셀마다 배열 인덱스가 달라지는 방식을 허용하지 않습니다. 리터럴 배열 인덱스를 지정하면 이 코드는 정상 작동합니다:

```
float4 c = TexArray[2].Sample(samLinear, pin.Tex);
```

하지만 이 방법은 첫 번째 시나리오보다 유연성이 떨어집니다.

11.3.2 텍스처 배열 샘플링

빌보드 데모에서는 다음 코드로 텍스처 배열을 샘플링합니다:

```
float3 uvw = float3(pin.Tex, pin.PrimID%4); texColor =
gTreeMapArray.Sample(samLinear, uvw);
```

텍스처 배열을 사용할 때는 세 개의 텍스처 좌표가 필요합니다. 첫 번째와 두 번째 텍스처 좌표는 일반적인 2D 텍스처 좌표이며, 세 번째 텍스처 좌표는 텍스처 배열 내 인덱스 역할을 합니다. 예를 들어, 0은 배열의 첫 번째 텍스처 인덱스, 1은 두 번째 텍스처 인덱스, 2는 세 번째 텍스처 인덱스이며, 이와 같이 이어집니다.

"빌보드" 데모에서는 각각 다른 나무 텍스처를 가진 네 개의 텍스처 요소를 가진 텍스처 배열을 사용합니다([그림 11.7](#)).

그러나 드로우 호출당 4그루 이상의 나무를 그리는 경우, 프리미티브 ID가 3을 초과하게 됩니다. 따라서 프리미티브 ID를 4로 나눈 나머지($\text{pin.PrimID} \% 4$)를 취해 프리미티브 ID를 0, 1, 2 또는 3으로 매핑합니다. 이는 4개 요소를 가진 배열에 대한 유효한 배열 인덱스입니다.

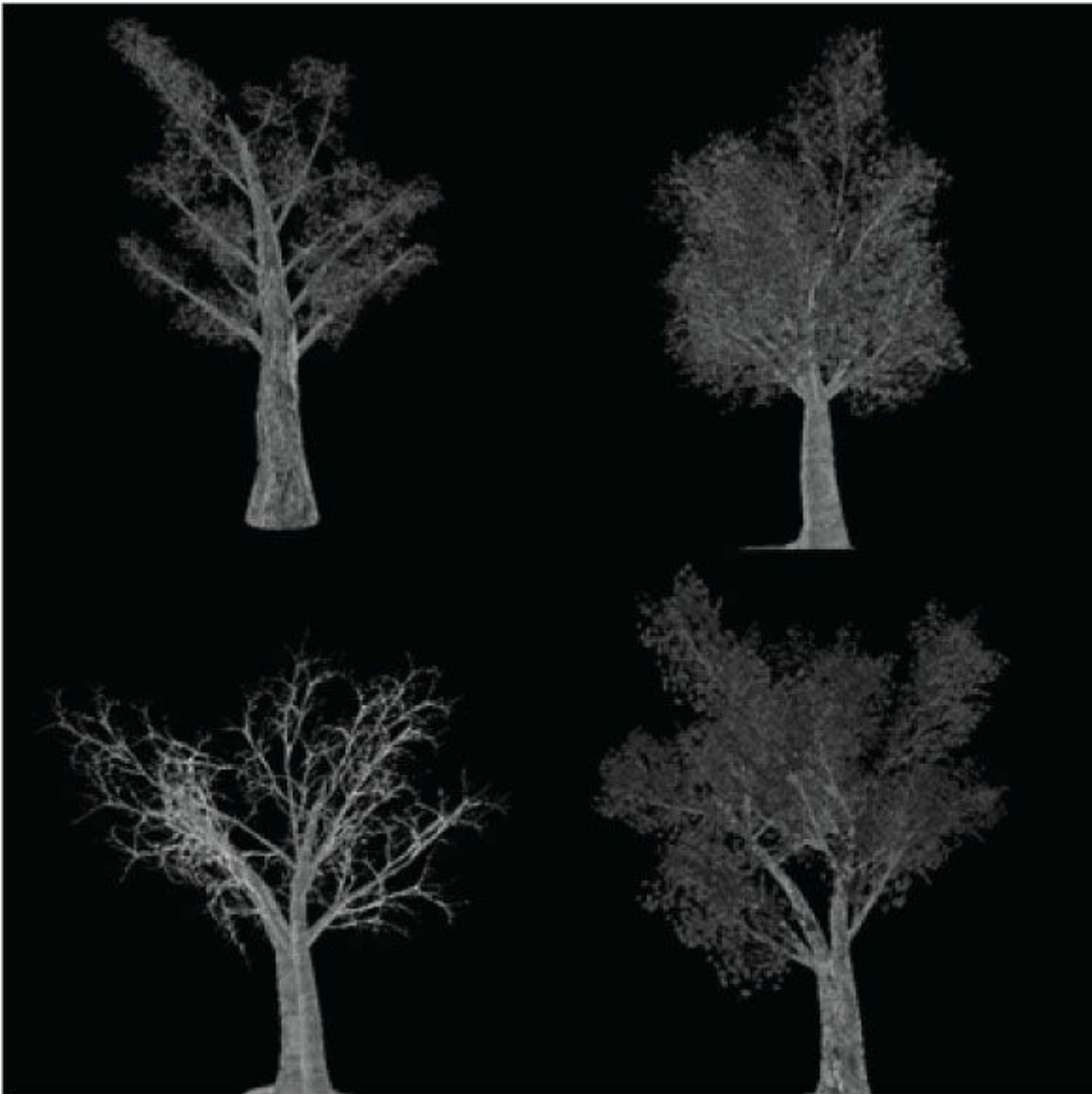


그림 11.7. 트리 빌보드 이미지.

텍스처 배열의 장점 중 하나는 서로 다른 텍스처를 가진 여러 개의 프리미티브를 하나의 드로우 호출로 그릴 수 있다는 점입니다. 일반적으로 다음과 같은 방식(의사 코드)을 사용해야 합니다:

```
SetTextureA(); DrawPrimitivesWithTextureA();
```

```
SetTextureB(); DrawPrimitivesWithTextureB();
```

```
... SetTextureZ();
```

```
DrawPrimitivesWithTextureZ();
```

각 설정 및 그리기 호출에는 일부 오버헤드가 발생합니다. 텍스처 배열을 사용하면 이를 하나의 설정 및 하나의 그리기로 줄일 수 있습니다.

호출:

```
SetTextureArray(); DrawPrimitivesWithTextureArray();
```

11.3.3 텍스처 배열 로딩

이 글을 작성하는 시점에는 파일에서 이미지 세트를 텍스처 배열로 로드하는 D3DX 함수가 존재하지 않습니다. 따라서 이 작업을 직접 수행해야 합니다. 과정은 다음과 같이 요약됩니다:

1. 파일에서 각 텍스처를 시스템 메모리에 하나씩 개별적으로 생성합니다.

2. 텍스처 배열을 생성합니다.
3. 각 개별 텍스처를 텍스처 배열의 요소로 복사합니다.
4. 텍스처 배열에 대한 셰이더 리소스 뷰를 생성합니다.

d3dUtil.h/cpp //파일명 목록으로부터 텍스처 배열을 생성하는 데 도움을 주는 헬퍼 함수를 구현했습니다. 모든 텍스처는 동일한 크기를 가져야 합니다.

```
ID3D11ShaderResourceView* d3dHelper::CreateTexture2DArraySRV( ID3D11Device* device,
    ID3D11DeviceContext* context, std::vector<std::wstring>& filenames,
    DXGI_FORMAT format, UINT
    filter,
    UINT mipFilter)
{
    //
    // 파일에서 텍스처 요소를 개별적으로 로드합니다. 이 텍스처들은
    // GPU에서 사용되지 않음(바인드 플래그 0). 파일에서 이미지 데이터만 불러오는 용도임
    // 파일에서 이미지 데이터를 로드하는 데 사용됩니다. STAGING 사용법을 사용하여
    // CPU가 리소스를 읽을 수 있도록 합니다.
    //
    UINT size = filenames.size(); std::vector<ID3D11Texture2D*>
srcTex(size); for(UINT i = 0; i < size; ++i)
    {
        D3DX11_IMAGE_LOAD_INFO loadInfo; loadInfo.Width =
D3DX11_FROM_FILE; loadInfo.Height =
D3DX11_FROM_FILE; loadInfo.Depth =
D3DX11_FROM_FILE; loadInfo.FirstMipLevel = 0;
loadInfo.MipLevels = D3DX11_FROM_FILE; loadInfo.Usage =
D3D11_USAGE_STAGING; loadInfo.BindFlags = 0;
loadInfo.CpuAccessFlags = D3D11_CPU_ACCESS_WRITE | D3D11_CPU_ACCESS_READ; loadInfo.MiscFlags = 0;
loadInfo.Format = format; loadInfo.Filter =
filter; loadInfo.MipFilter = mipFilter;
loadInfo.pSrcInfo = 0;
HR(D3DX11CreateTextureFromFile(device, filenames[i].c_str(), &loadInfo, 0,
(ID3D11Resource*)&srcTex[i], 0));
    }

    //
    // 텍스처 배열 생성. 텍스처 배열의 각 요소는
    // 배열의 각 요소는 동일한 형식/크기를 가집니다.
    //
    D3D11_TEXTURE2D_DESC texElementDesc; srcTex[0]-
>GetDesc(&texElementDesc); D3D11_TEXTURE2D_DESC
texArrayDesc;
texArrayDesc.Width                = texElementDesc.Width;
texArrayDesc.Height               = texElementDesc.Height;
texArrayDesc.MipLevels            = texElementDesc.MipLevels;
texArrayDesc.ArraySize            = size;
texArrayDesc.Format               = texElementDesc.Format;
texArrayDesc.SampleDesc.Count = 1;
    텍스처 배열 설명.샘플 설명.품질 = 0; 텍스처 배열 설명.사용법 =
D3D11_USAGE_DEFAULT;
    텍스처 배열 설명. 바인딩 플래그 = D3D11_BIND_SHADER_RESOURCE;
texArrayDesc.CPUAccessFlags      = 0;
    텍스처 배열 설명.기타 플래그 = 0;
```



```

ID3D11Texture2D* texArray = 0;
HR(device->CreateTexture2D( &texArrayDesc, 0, &texArray));

//
// 개별 텍스처 요소를 텍스처 배열에 복사합니다.
//

// 각 텍스처 요소에 대해...
for(UINT texElement = 0; texElement < size; ++texElement)
{
    // 각 mip맵 레벨마다...
    for(UINT mipLevel = 0; mipLevel < texElementDesc.MipLevels; ++mipLevel)
    {
        D3D11_MAPPED_SUBRESOURCE mappedTex2D;
        HR(context->Map(srcTex[texElement],
            mipLevel, D3D11_MAP_READ, 0, &mappedTex2D)); context-
        >UpdateSubresource(
            texArray, D3D11CalcSubresource(
                mipLevel, texElement,
                텍스엘리먼트디스크립션.미레벨스),
            0,
            mappedTex2D.pData,
            mappedTex2D.RowPitch,
            mappedTex2D.DepthPitch);
        context->Unmap(srcTex[texElement], mipLevel);
    }
}

//
// 텍스처 배열에 대한 리소스 뷰 생성.
//

D3D11_SHADER_RESOURCE_VIEW_DESC 뷰디스크립션 뷰디스크립션;
viewDesc.Format = texArrayDesc.Format;
viewDesc.ViewDimension = D3D11_SRV_DIMENSION_TEXTURE2DARRAY;
viewDesc.Texture2DArray.MostDetailedMip = 0; viewDesc.Texture2DArray.MipLevels =
texArrayDesc.MipLevels; viewDesc.Texture2DArray.FirstArraySlice = 0; viewDesc.Texture2DArray.ArraySize =
size;

ID3D11ShaderResourceView* texArraySRV = 0;
HR(device->CreateShaderResourceView(texArray, &viewDesc, &texArraySRV));

//
// 정리--리소스 뷰만 필요합니다.
// ReleaseCOM(texArray);

for(UINT i = 0; i < size; ++i) ReleaseCOM(srcTex[i]);

return texArraySRV;
}

```

ID3D11DeviceContext::UpdateSubresource 메서드는 한 서브리소스의 메모리를 다른 서브리소스로 복사합니다(매개변수에 대한 설명은 SDK 문서를 참조하십시오).

```

void ID3D11DeviceContext::UpdateSubresource( ID3D11Resource
    *pDstResource,
    UINT DstSubresource,

```

```
const D3D11_BOX *pDstBox, const void
*pSrcData,
UINT SrcRowPitch, UINT
SrcDepthPitch);
```

1. **pDstResource**: 대상 리소스 객체.
2. **DstSubresource**: 대상 리소스에서 업데이트할 서브리소스를 식별하는 인덱스 (§11.3.4 참조).
3. **pDstBox**: 업데이트할 대상 서브리소스 내 볼륨을 지정하는 **D3D11_BOX** 인스턴스 포인터; 전체 서브리소스를 업데이트하려면 null을 지정합니다.
4. **pSrcData**: 소스 데이터에 대한 포인터.
5. **SrcRowPitch**: 소스 데이터의 한 행의 바이트 크기.
6. **SrcDepthPitch**: 소스 데이터의 한 깊이 슬라이스의 바이트 크기.

2D 텍스처의 경우 **SrcDepthPitch** 매개변수가 불필요해 보일 수 있지만, 3D 텍스처 업데이트에도 동일한 방법이 사용되며, 이는 2D 텍스처의 스택으로 시각화될 수 있습니다. 트리 스프라이트의 텍스처 배열을 생성하기 위해 초기화 시점에 이전 함수를 호출합니다:

```
ID3D11ShaderResourceView* mTreeTextureMapArraySRV; mTreeTextureMapArraySRV =
d3dHelper::CreateTexture2DArraySRV(
md3dDevice, md3dImmediateContext, treeFilenames, DXGI_FORMAT_R8G8B8A8_UNORM);
```

11.3.4 텍스처 서브리소스

텍스처 어레이에 대해 논의했으니 이제 서브리소스에 대해 이야기해 보겠습니다. [그림 11.8](#)은 여러 텍스처를 가진 텍스처 어레이의 예를 보여줍니다. 각 텍스처는 차례로 자체 밍맵 체인을 가지고 있습니다. Direct3D API는 텍스처의 요소와 그 완전한 밍맵 체인을 가리키기 위해 '어레이 슬라이스'라는 용어를 사용합니다. Direct3D API는 텍스처 배열 내 특정 레벨의 모든 밍맵을 가리키는 용어로 '밍 슬라이스(mip slice)'를 사용합니다. 서브리소스는 텍스처 배열 요소 내 단일 밍맵 레벨을 가리킵니다.

텍스처 배열 인덱스와 밍맵 레벨을 지정하면 텍스처 배열 내 서브리소스에 접근할 수 있습니다. 그러나 서브리소스는 선형 인덱스로도 레이블링될 수 있습니다. Direct3D는 [그림 11.9](#)에 표시된 순서로 선형 인덱스를 사용합니다.

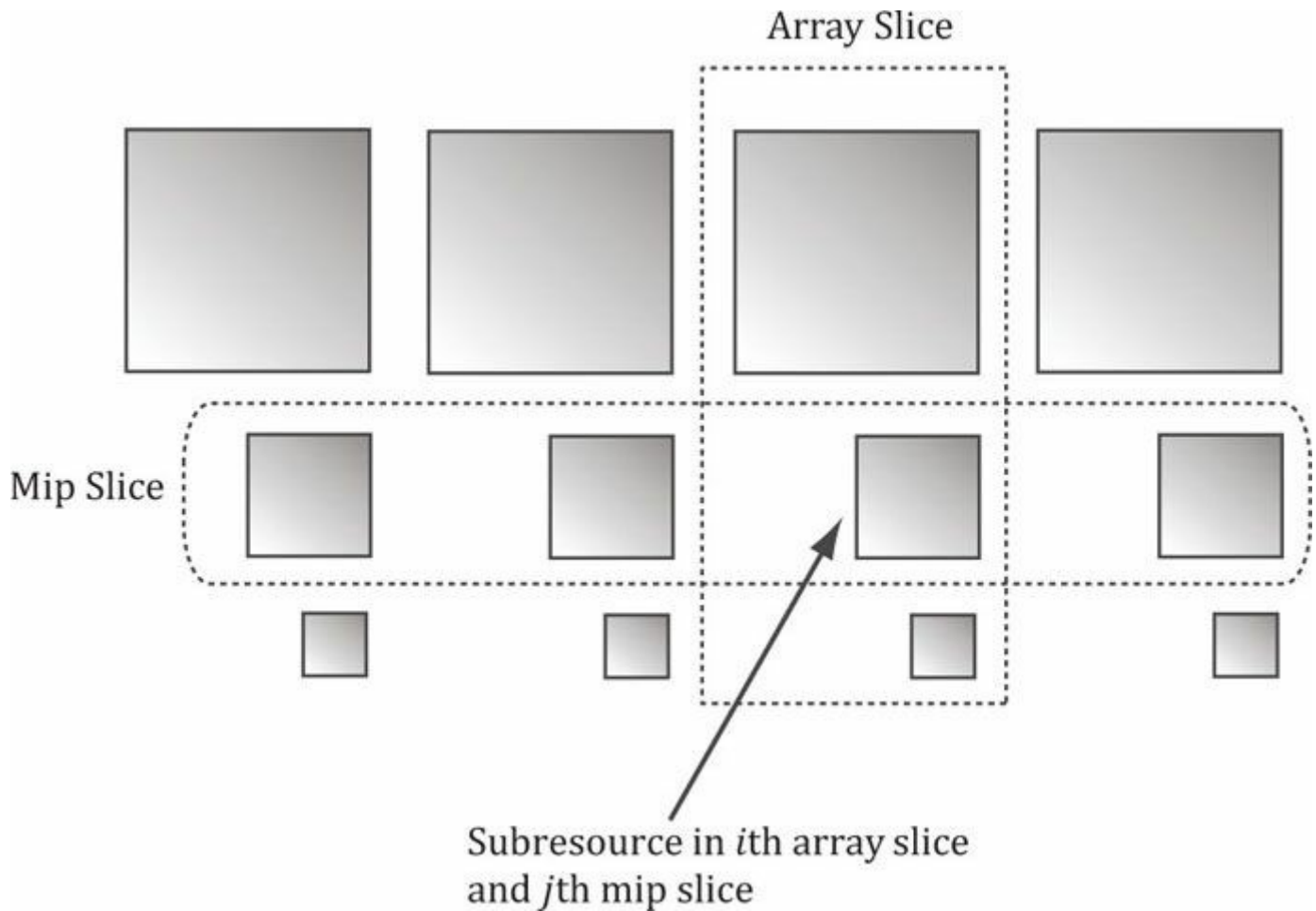


그림 11.8. 네 개의 텍스처로 구성된 텍스처 배열. 각 텍스처는 세 개의 밍맵 레벨을 가집니다.

다음 유틸리티 함수는 밍맵 레벨, 배열 인덱스, 밍맵 레벨 수를 입력으로 선형 서브리소스 인덱스를 계산하는 데 사용됩니다:

```
inline UINT D3D11CalcSubresource(
    UINT MipSlice, UINT ArraySlice, UINT MipLevels);
```

공식은 간단합니다: $k = \text{ArraySlice} * \text{MipLevels} + \text{MipSlice}$.

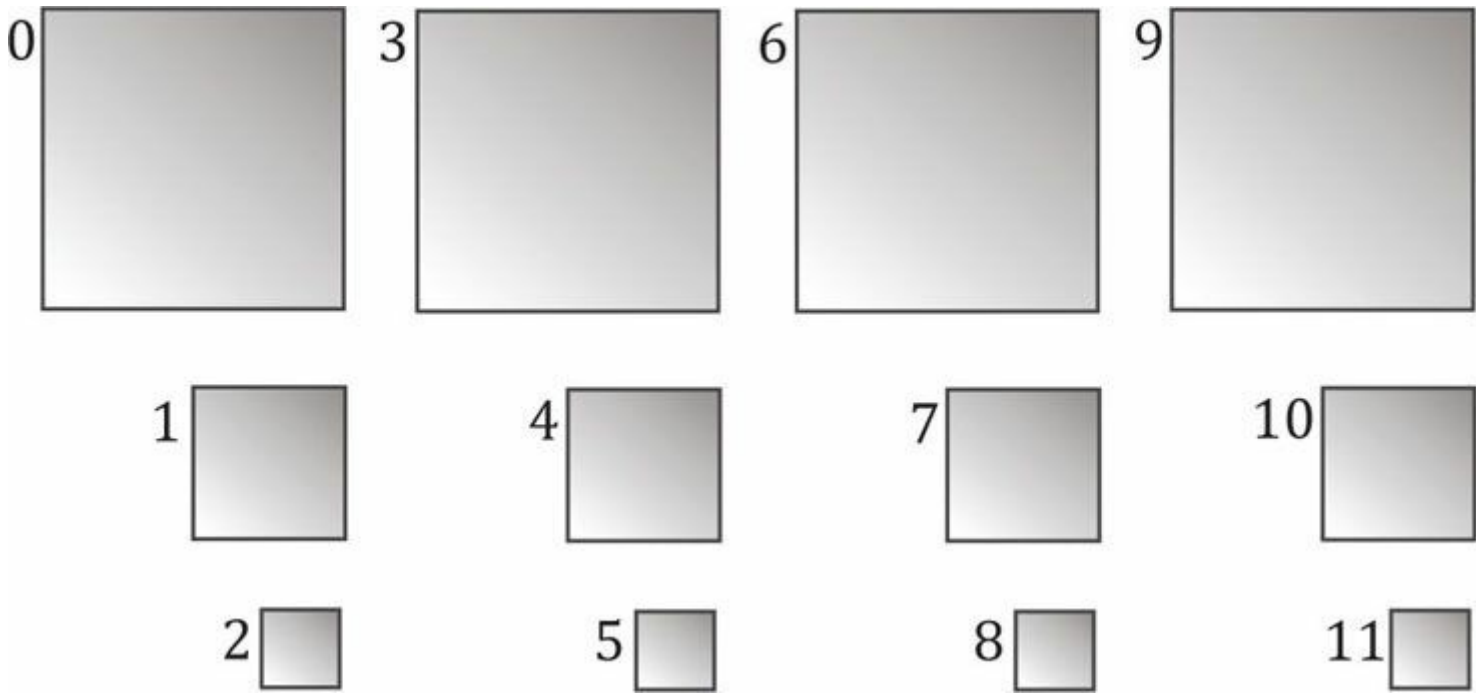


그림 11.9. 선형 인덱스로 레이블링된 텍스처 배열 내 서브리소스.

11.4 ALPHA-TO-COVERAGE

"트리 빌보드" 데모를 실행하면 특정 거리에서 나무 빌보드 오려낸 부분의 가장자리가 블록처럼 보일 수 있습니다. 이는 나무의 일부가 아닌 텍스처 픽셀을 가리기 위해 사용하는 **클립** 함수 때문입니다. 클립 함수는 픽셀을 유지하거나 거부할 뿐, 부드러운 전환이 없습니다. 시선과 빌보드 사이의 거리가 중요한 이유는 가까운 거리에서는 확대 효과가 발생하여 블록 아티팩트가 더 크게 보이고, 가까운 거리에서는 더 낮은 해상도의 mip맵 레벨이 사용되기 때문입니다.

이 사용되기 때문입니다.

이 문제를 해결하는 한 가지 방법은 알파 테스트 대신 투명도 블렌딩을 사용하는 것입니다. 선형 텍스처 필터링으로 인해 가장자리 픽셀은 투명도 블렌딩은 흰색(불투명 픽셀)에서 검정색(마스크 처리된 픽셀)으로 부드러운 전환을 위해 약간 흐리게 처리됩니다. 이로 인해 불투명 픽셀에서 마스크 처리된 픽셀로 이어지는 가장자리 부분에 부드러운 페이드 아웃 효과가 발생합니다. 불행히도 투명도 블렌딩은 정렬 및 렌더링을 후면-전면 순서로 수행해야 합니다. 소수의 트리 빌보드 정렬에 드는 오버헤드는 크지 않지만, 숲이나 초원을 렌더링할 경우 매 프레임마다 정렬을 수행해야 하므로 비용이 많이 듭니다. 더 심각한 문제는 뒤에서 앞으로 순서로 렌더링하면 막대한 오버드로우([10장 연습문제 8 참조](#))가 발생하여 성능을 크게 저하시킬 수 있다는 점입니다.

MSAA(다중 샘플링 앤티앨리어싱—[§4.1.7 참조](#))가 도움이 될 수 있다고 제안할 수 있습니다. MSAA는 블록 같은 가장자리를 부드럽게 하는 데 사용되기 때문입니다. 다각형. 실제로 도움이 될 수 있지만 문제가 있습니다. MSAA는 픽셀당 한 번씩 픽셀 중심에서 픽셀 셰이더를 실행한 후, 가시성(심도/스텐실 테스트는 서브픽셀별로 평가됨)과 커버리지(서브픽셀 중심이 다각형 내부 또는 외부에 위치하는가?)에 따라 해당 색상 정보를 서브픽셀과 공유합니다. 핵심은 *커버리지가 폴리곤 단위로 결정된다*는 점입니다. 따라서 MSAA는 알파 채널로 정의된 나무 빌보드 커트아웃의 가장자리를 감지하지 못합니다. 텍스처가 매핑된 쿼드(사각형)의 가장자리만 살펴볼 뿐입니다. 그렇다면 커버리지 계산 시 Direct3D가 알파 채널을 고려하도록 지시할 방법이 있을까요? 답은 '예'이며, 이는 *알파-투-커버리지(alpha-to-coverage)* 기법으로 이어집니다.

MSAA가 활성화되고 알파-투-커버리지가 활성화된 경우(**ID3D11BlendState**의 멤버), 하드웨어는 픽셀 셰이더가 반환하는 알파 값을 확인하고 이를 커버리지 결정에 사용합니다 [NVIDIA05]. 예를 들어 4X MSAA에서 픽셀 셰이더 알파가 0.5라면, 커버리지를 결정하기 위해 두 개의 쿼드(0.5 픽셀이 커버하는 영역)가 필요하다고 가정할 수 있습니다.

픽셀 셰이더가 반환한 알파 값을 확인하여 커버리지를 결정합니다 [NVIDIA05]. 예를 들어, 4X MSAA에서 픽셀 셰이더 알파가 0.5라면, 네 개의 서브픽셀 중 두 개가 커버된 것으로 간주하여 부드러운 경계를 생성합니다.

알파-커버리지 블렌드 상태 객체는 다음과 같이 생성됩니다:

```
D3D11_BLEND_DESC a2CDesc = {0};
a2CDesc.AlphaToCoverageEnable = true; a2CDesc.IndependentBlendEnable =
false; a2CDesc.RenderTarget[0].BlendEnable = false;
a2CDesc.RenderTarget[0].RenderTargetWriteMask =
    D3D11_COLOR_WRITE_ENABLE_ALL;
```

```
ID3D11BlendState* AlphaToCoverageBS;  
HR(device->CreateBlendState(&a2CDesc, &AlphaToCoverageBS));
```

일반적인 조연은 잎사귀나 울타리처럼 알파 마스크 처리된 텍스처를 오려낼 때는 항상 알파-투-커버리지를 사용해야 한다는 것입니다.

그러나 이를 위해서는 MSAA가 활성화되어 있어야 합니다. 참고로 데모 애플리케이션 생성자에서 다음과 같이 설정합니다:

```
mEnable4xMsaa = true;
```

이로 인해 샘플 프레임워크는 4X MSAA를 지원하는 백 버퍼와 깊이 버퍼를 생성합니다.

 "트리 빌보드" 데모에서 알파-커버리지는 각각 'r' 키와 't' 키를 눌러 켜고 끌 수 있습니다.

11.5 요약

1. 테셀레이션 단계를 사용하지 않는다고 가정할 때, 지오메트리 셰이더 단계는 버텍스 셰이더와 픽셀 셰이더 단계 사이에 위치하는 선택적 단계입니다. 지오메트리 셰이더는 입력 어셈블러를 통해 전송되는 각 프리미티브에 대해 호출됩니다. 지오메트리 셰이더는 0개, 1개 또는 그 이상의 프리미티브를 출력할 수 있습니다. 출력 프리미티브 유형은 입력 프리미티브 유형과 다를 수 있습니다. 출력 프리미티브의 정점은 지오메트리 셰이더를 떠나기 전에 동형 클립 공간으로 변환되어야 합니다. 지오메트리 셰이더에서 출력된 프리미티브는 다음으로 렌더링 파이프라인의 래스터라이제이션 단계로 진입합니다. 지오메트리 셰이더는 효과 파일, 병렬 정점 셰이더 및 픽셀 셰이더로 프로그래밍됩니다.
2. 빌보드 기법은 물체의 실제 3D 모델 대신 해당 물체의 이미지로 텍스처링된 사각형 면을 사용하는 기술입니다. 멀리 있는 물체의 경우, 시청자는 빌보드가 사용되고 있음을 구분할 수 없습니다. 빌보드의 장점은 텍스처링된 사각형 면으로 충분할 때 GPU가 전체 3D 물체를 렌더링하는 데 처리 시간을 낭비하지 않아도 된다는 점입니다. 이 기법은 카메라 근처의 나무에는 실제 3D 지오메트리를 사용하고, 멀리 있는 나무에는 빌보드를 사용하는 숲 렌더링에 유용할 수 있습니다. 빌보드 기법이 제대로 작동하려면 빌보드가 항상 카메라를 향해야 합니다. 빌보드 기법은 지오메트리 셰이더에서 효율적으로 구현할 수 있습니다.
3. 다음 예시와 같이 지오메트리 셰이더의 매개변수 목록에 **uint** 형식이며 **SV_PrimitiveID** 시맨틱을 가진 특수 매개변수를 추가할 수 있습니다:

```
[maxvertexcount(4)]  
void GS(point VertexOut gin[1],  
    uint primID : SV_PrimitiveID,  
    inout TriangleStream<GeoOut> triStream);
```

이 의미론이 지정되면 입력 어셈블러 단계에서 각 프리미티브에 대해 프리미티브 ID를 자동으로 생성하도록 지시합니다. n개의 프리미티브를 그리기 위해 드로우 콜이 실행될 때, 첫 번째 프리미티브는 0으로, 두 번째 프리미티브는 1로, 그리고 마지막 프리미티브는 n-1로 라벨링됩니다. 지오메트리 셰이더가 존재하지 않는 경우, 프리미티브 ID 매개변수를 픽셀 셰이더의 매개변수 목록에 추가할 수 있습니다. 그러나 지오메트리 셰이더가 존재하는 경우, 프리미티브 ID 매개변수는 지오메트리 셰이더 시그니처에 반드시 포함되어야 합니다. 그러면 지오메트리 셰이더는 프리미티브 ID를 사용하거나 픽셀 셰이더 단계로 전달할 수 있습니다(또는 둘 다).

4. 입력 어셈블러 단계는 버텍스 ID를 생성할 수 있습니다. 이를 위해 버텍스 셰이더 시그니처에 **uint** 유형의 추가 매개변수를 세미틱 **SV_VertexID**로 추가합니다. **드로우** 콜의 경우, 드로우 콜 내 버텍스들은 0, 1, ..., n-1의 ID로 라벨링됩니다(여기서 n은 드로우 콜의 버텍스 수입니다). **드로우 인덱스드** 콜의 경우, 버텍스 ID는 버텍스 인덱스 값에 대응합니다.
5. 텍스처 배열은 텍스처 배열을 저장합니다. C++ 코드에서 텍스처 배열은 **ID3D11Texture2D** 인터페이스(단일 텍스처에도 사용되는 동일한 인터페이스)로 표현됩니다. 효과 파일에서는 텍스처 배열이 **Texture2DArray** 유형으로 표현됩니다. 텍스처 배열을 사용할 때는 세 개의 텍스처 좌표가 필요합니다. 첫 번째와 두 번째 텍스처 좌표는 일반적인 2D 텍스처 좌표이며, 세 번째 텍스처 좌표는 텍스처 배열 내 인덱스입니다. 예를 들어, 0은 배열의 첫 번째 텍스처에 대한 인덱스이고, 1은 배열의 두 번째 텍스처에 대한 인덱스이며, 2는 배열의 세 번째 텍스처에 대한 인덱스입니다. 텍스처 배열의 장점 중 하나는 서로 다른 텍스처를 가진 여러 개의 프리미티브를 하나의 드로우 호출로 그릴 수 있다는 점입니다. 각 프리미티브는 텍스처 배열에 대한 인덱스를 가지며, 이 인덱스는 프리미티브에 적용할 텍스처를 나타냅니다.
6. 알파-투-커버리지는 하드웨어가 서브픽셀 커버리지를 결정할 때 픽셀 셰이더가 반환한 알파 값을 참조하도록 지시합니다. 이는 잎사귀나 울타리 같은 알파 마스크 처리된 커트아웃 텍스처에 부드러운 가장자리를 구현합니다. 알파-투-커버리지는 **ID3D11BlendState**로 제어됩니다. **D3D11_BLEND_DESC** 구조체를 작성할 때 **AlphaToCoverageEnable** 속성을 **true**로 설정하세요.

11.6 연습 문제

1. xz 평면에 라인 스트립으로 그려진 원을 고려하십시오. 지오메트리 셰이더를 사용하여 캡이 없는 원통으로 라인 스트립을 확장하십시오.
2. 이코사헤드론은 구의 대략적인 근사체입니다. 각 삼각형(그림 11.1)을 세분화하고 새로운 꼭짓점을 구에 투영함으로써 더 나은 근사치를 얻을 수 있습니다. (꼭짓점을 단위 구에 투영하는 것은 단순히 위치 벡터를 정규화하는 것과 같습니다. 모든 단위 벡터의 끝점이 단위 구의 표면과 일치하기 때문입니다.) 이 연습에서는 이코사헤드론을 생성하고 렌더링하세요. 지오메트리 셰이더를 사용하여 카메라와의 $거리/d$ 따라 이코사헤드론을 세분화하세요. 예를 들어, $d < 15$ 이면 원래 이코사헤드론을 두 번 세분화하고, $15 \leq d < 30$ 이면 한 번 세분화하며, $d \geq 30$ 이면 원래 이코사헤드론 그대로 렌더링합니다. 이 아이디어는 오브젝트가 카메라에 가까울 때만 많은 폴리곤을 사용한다는 것입니다. 오브젝트가 멀리 있으면 더 거친 메시로 충분하며, 필요 이상으로 많은 폴리곤을 처리하는 데 GPU 성능을 낭비할 필요가 없습니다. 그림 11.10은 세 가지 LOD 레벨을 와이어프레임 모드와 솔리드(조명 적용) 모드로 나란히 보여줍니다. 이코사헤드론의 테셀레이션에 대한 논의는 §6.12.3을 참조하십시오.
3. 간단한 폭발 효과는 삼각형의 면 법선 방향으로 시간 함수에 따라 변위시켜 시뮬레이션할 수 있습니다. 이 시뮬레이션은 지오메트리 셰이더에서 구현 가능합니다. 지오메트리 셰이더로 입력된 각 삼각형에 대해, 지오메트리 셰이더는 면 법선 $\mathbf{n}$ 을 계산한 후, 폭발 시작 이후 경과 시간 t/d 따라 세 개의 삼각형 정점 $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2$ 를 $\mathbf{n}$ 방향으로 변위시킵니다:

$$\mathbf{p}'_i = \mathbf{p}_i + t\mathbf{n} \text{ for } i = 0, 1, 2$$

면 법선 $\mathbf{n}$ 은 반드시 단위 길이가 아니어도 되며, 폭발 속도를 제어하기 위해 적절히 스케일링할 수 있습니다. 심지어 스케일을 프리미티브 ID에 따라 달라지게 하여 각 프리미티브가 서로 다른 속도로 이동하도록 할 수도 있습니다. 이 효과를 구현하기 위한 샘플 메시로 이분할되지 않은 이코사헤드론을 사용하십시오.



그림 11.10. 정이십면체의 분할 및 정점들을 단위 구면에 투영한 모습.

4. 메시의 정점 법선을 시각화하는 것은 디버깅에 유용할 수 있습니다. 메시의 정점 법선을 짧은 선분으로 렌더링하는 이펙트를 작성하세요. 이를 위해 메시의 점 프리미티브(즉, `D3D11_PRIMITIVE_TOPOLOGY_POINTLIST` 토폴로지를 가진 정점)를 입력으로 받는 지오메트리 셰이더를 구현하여 각 정점이 지오메트리 셰이더를 통과하도록 합니다. 이제 지오메트리 셰이더는 각 점을 $길이/L$ 의 선분으로 확장할 수 있습니다. 정점의 위치가 $\mathbf{p}$ 이고 법선이 $\mathbf{n}$ 이라면, 해당 정점 법선을 나타내는 선분의 두 끝점은 $\mathbf{p}$ 와 $\mathbf{p} + L\mathbf{n}$ 입니다. 구현 후 메쉬를 평소처럼 렌더링하고, 법선 벡터 시각화 기법을 적용하여 장면 위에 법선이 렌더링되도록 장면을 다시 렌더링하세요. 테스트 장면으로 "Blend" 데모를 사용하십시오.
5. 이전 연습과 유사하게, 메시의 면 법선을 짧은 선분으로 렌더링하는 이펙트를 작성하세요. 이 이펙트에서

지오메트리 셰이더는 삼각형을 입력으로 받아 노멀을 계산한 후 선분을 출력합니다.

6. 이 연습은 드로우 쿨의 경우 드로우 쿨 내 정점들이 0, 1, ..., n-1의 ID로 라벨링되며, 여기서 n은 드로우 쿨의 정점 수임을 보여줍니다. 또한 **드로우** 인덱스드 쿨의 경우 정점 ID가 정점 인덱스 값과 일치함을 보여줍니다.

다음과 같이 "트리 빌보드" 데모를 수정하십시오. 먼저 버텍스 셰이더를 다음과 같이 변경하십시오:

```
VertexOut VS(VertexIn vin, uint vertID : SV_VertexID)
{
    VertexOut vout;
    // 단순히 데이터를 지오메트리 셰이더로 전달합니다.
    vout.CenterW = vin.PosW;
    vout.SizeW = float2(2+vertID, 2+vertID); return vout;
}
```

다시 말해, 트리 빌보드의 크기는 중심 정점의 ID를 기준으로 조정됩니다. 이제 프로그램을 실행해 보세요. 16개의 빌보드를 그릴 때 크기는 2부터 17까지 다양하게 나타납니다. 이제 다음과 같이 그리기 방식을 수정해 보세요: 16개 지점을 한 번에 그리는 단일 드로우 쿨 대신, 다음과 같이 네 번으로 나누어 사용하세요:

```
md3dImmediateContext->Draw(4, 0);

md3dImmediateContext->Draw(4, 4);

md3dImmediateContext->Draw(4, 8);

md3dImmediateContext->Draw(4, 12);
```

이제 프로그램을 실행하세요. 이번에는 크기가 2부터 5까지 분포해야 합니다. 각 드로우 호출이 4개의 정점을 그리므로, 각 호출에서 정점 ID는 0부터 3까지 분포합니다. 이제 인덱스 버퍼와 네 번의 **DrawIndexed** 호출을 사용하세요. 프로그램을 실행한 후 크기가 다시 2부터 17까지 분포해야 합니다. 이는 **DrawIndexed**를 사용할 때 정점 ID가 정점 인덱스 값에 대응하기 때문입니다.

7. 다음과 같이 "Tree Billboards" 데모를 수정하십시오. 먼저 픽셀 셰이더에서 "modulo 4"를 제거하십시오:

```
float3 uvw = float3(pin.Text, pin.PrimID);
```

이제 프로그램을 실행하세요. 0부터 15까지의 프라이머리 ID를 가진 16개의 프라이머리를 그리는 중이므로, 이 ID들은 배열 범위를 벗어납니다. 그러나 범위를 벗어난 인덱스는 유효한 최대 인덱스(이 경우 3)로 클램핑되므로 오류가 발생하지 않습니다. 이제 16개 점을 한 번에 그리는 단일 드로우 호출 대신, 다음과 같이 네 번 호출하세요:

```
md3dImmediateContext->Draw(4, 0);

md3dImmediateContext->Draw(4, 4);

md3dImmediateContext->Draw(4, 8);

md3dImmediateContext->Draw(4, 12);
```

프로그램을 다시 실행합니다. 이번에는 클램핑이 발생하지 않습니다. 각 드로우 호출이 4개의 프리미티브를 그리므로, 각 드로우 호출에 대해 프리미티브 ID는 0부터 3까지의 범위를 가집니다. 따라서 프리미티브 ID를 범위를 벗어나지 않고 인덱스로 사용할 수 있습니다. 이는 프리미티브 ID "카운트"가 각 드로우 호출마다 0으로 재설정됨을 보여줍니다.

제12^장 컴퓨터 셰이더

GPU는 단일 위치 또는 연속된 위치에서 대량의 메모리를 처리하도록 최적화되어 있습니다(소위 "스트리밍 연산"); 이는 임의 메모리 접근을 위해 설계된 CPU와 대조적입니다[Boyd10]. 또한 버텍스와 픽셀이 독립적으로 처리되기 때문에 GPU는 대규모 병렬 처리를 위해 설계되었습니다. 예를 들어 NVIDIA의 "Fermi" 아키텍처는 32개의 CUDA 코어를 가진 최대 16개의 스트리밍 멀티프로세서로 지원하여 총 512개의 CUDA 코어를 제공합니다[NVIDIA09].

이 GPU 아키텍처는 그래픽을 위해 설계되었으므로 그래픽 성능이 향상되는 것은 당연합니다. 그러나 일부 비-그래픽 애플리케이션은 GPU가 제공하는 병렬 아키텍처로 인한 막대한 연산 성능의 혜택을 받습니다. 비그래픽 애플리케이션에 GPU를 사용하는 것을 *범용 GPU(GPGPU) 프로그래밍*이라고 합니다. 모든 알고리즘이 GPU 구현에 적합한 것은 아닙니다. GPU는 병렬 아키텍처를 활용하기 위해 데이터 병렬 알고리즘이 필요합니다. 즉, 유사한 연산이 수행될 대량의 데이터 요소가 필요하며, 이를 통해 요소들을 병렬로 처리할 수 있습니다. 픽셀 음영 처리와 같은 그래픽 작업이 좋은 예시입니다. 그러지는 각 픽셀 조각은 픽셀 셰이더에 의해 연산되기 때문입니다. 또 다른 예로, 이전 장에서 다룬 파동 시뮬레이션 코드를 살펴보면 업데이트 단계에서 각 격자 요소에 대해 계산을 수행하는 것을 확인할 수 있습니다. 따라서 이 역시 GPU 구현에 적합한 후보입니다. 각 격자 요소를 GPU가 병렬로 업데이트할 수 있기 때문입니다. 입자 시스템도 또 다른 예시입니다. 입자들이 서로 상호작용하지 않는다는 단순화를 적용한다면, 각 입자의 물리적 특성을 독립적으로 계산할 수 있습니다.

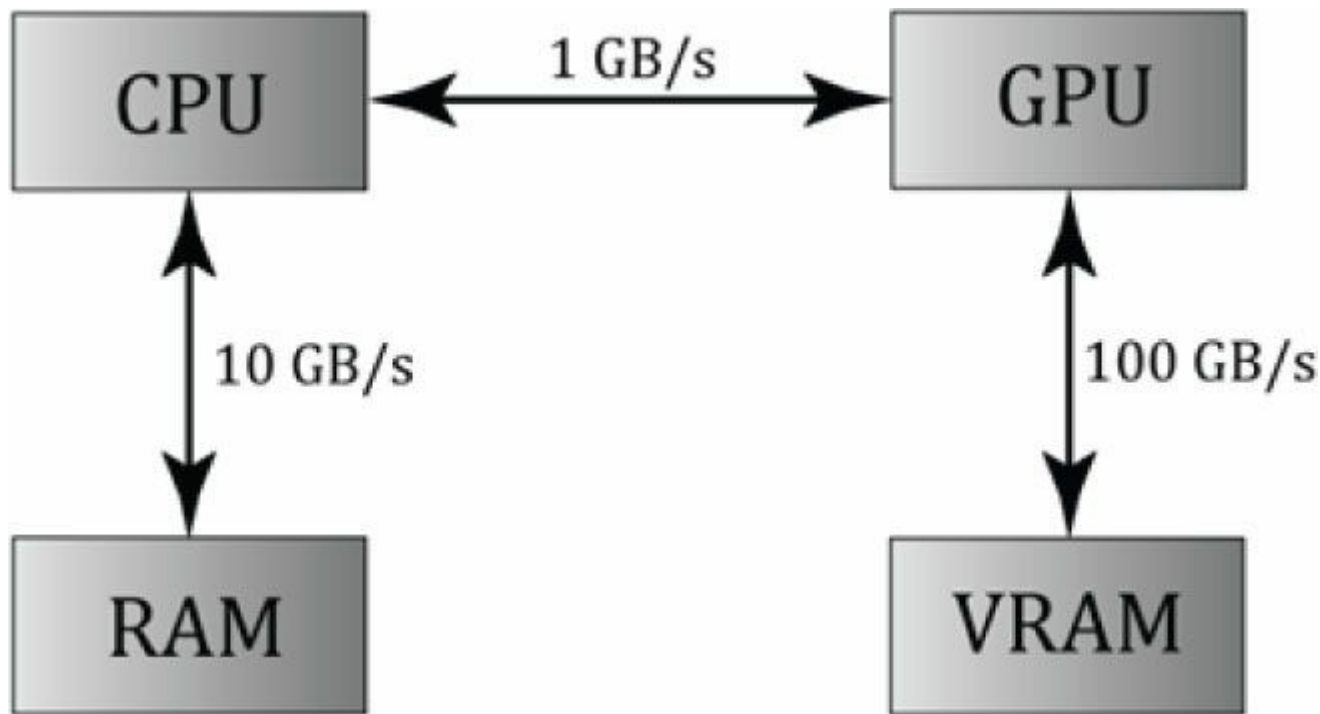


그림 12.1. [Boyd10]에서 재구성한 이미지. CPU와 RAM, CPU와 GPU, GPU와 VRAM 간의 상대적 메모리 대역폭 속도. 이 수치는 대역폭 간의 차이를 보여주기를 위한 예시적 수치일 뿐이다. CPU와 GPU 간 메모리 전송이 병목 현상임을 확인할 수 있다.

GPGPU 프로그래밍에서는 일반적으로 사용자가 계산 결과를 CPU에서 다시 액세스해야 합니다. 이는 비디오 메모리에서 시스템 메모리로 결과를 복사해야 함을 의미하며, 이는 느리지만(그림 12.1 참조) GPU에서 계산을 수행함으로써 얻는 속도 향상과 비교하면 무시할 수 있는 문제일 수 있습니다. 그래픽스의 경우, 우리는 일반적으로 계산 결과를 렌더링 파이프라인의 입력으로 사용하므로 GPU에서 CPU로의 전송이 필요하지 않습니다. 예를 들어 컴퓨터 셰이더로 텍스처를 불러 처리한 후, 해당 불러 처리된 텍스처를 셰이더 리소스 뷰에 바인딩하여 셰이더 입력으로 사용할 수 있습니다.

컴퓨터 셰이더는 렌더링 파이프라인의 직접적인 일부가 아닌, Direct3D가 제공하는 프로그래밍 가능한 셰이더입니다. 대신, 옆에 위치하며 GPU 리소스에서 읽고 GPU 리소스에 쓸 수 있습니다(그림 12.2). 본질적으로 컴퓨터 셰이더는 아무것도 그리지 않고도 데이터 병렬 알고리즘을 구현하기 위해 GPU에 접근할 수 있게 해줍니다. 앞서 언급했듯이 이는 GPGPU 프로그래밍에 유용하지만, 컴퓨터 셰이더로 구현 가능한 그래픽 효과 역시 여전히 많습니다. 따라서 그래픽 프로그래머에게도 여전히 매우 관련성이 높습니다. 또한 앞서 언급한 바와 같이 컴퓨터 셰이더는 Direct3D의 일부이므로 Direct3D 리소스에서 읽고 쓸 수 있으며, 이를 통해 컴퓨터 셰이더의 출력을 렌더링 파이프라인에 직접 바인딩할 수 있습니다.

목표:

1. 컴퓨트 셰이더 프로그래밍 방법을 학습합니다.
2. 하드웨어가 스레드 그룹과 그 내부의 스레드를 처리하는 방식에 대한 기본적인 고수준 이해를 얻기 위함.
3. 컴퓨트 셰이더의 입력으로 설정할 수 있는 Direct3D 리소스와 출력으로 설정할 수 있는 Direct3D 리소스를 파악한다.
4. 다양한 스레드 ID와 그 용도를 이해하기 위해.
5. 공유 메모리에 대해 배우고 성능 최적화에 활용하는 방법을 알아보기.
6. GPGPU 프로그래밍에 대한 더 자세한 정보를 어디서 얻을 수 있는지 알아보기.

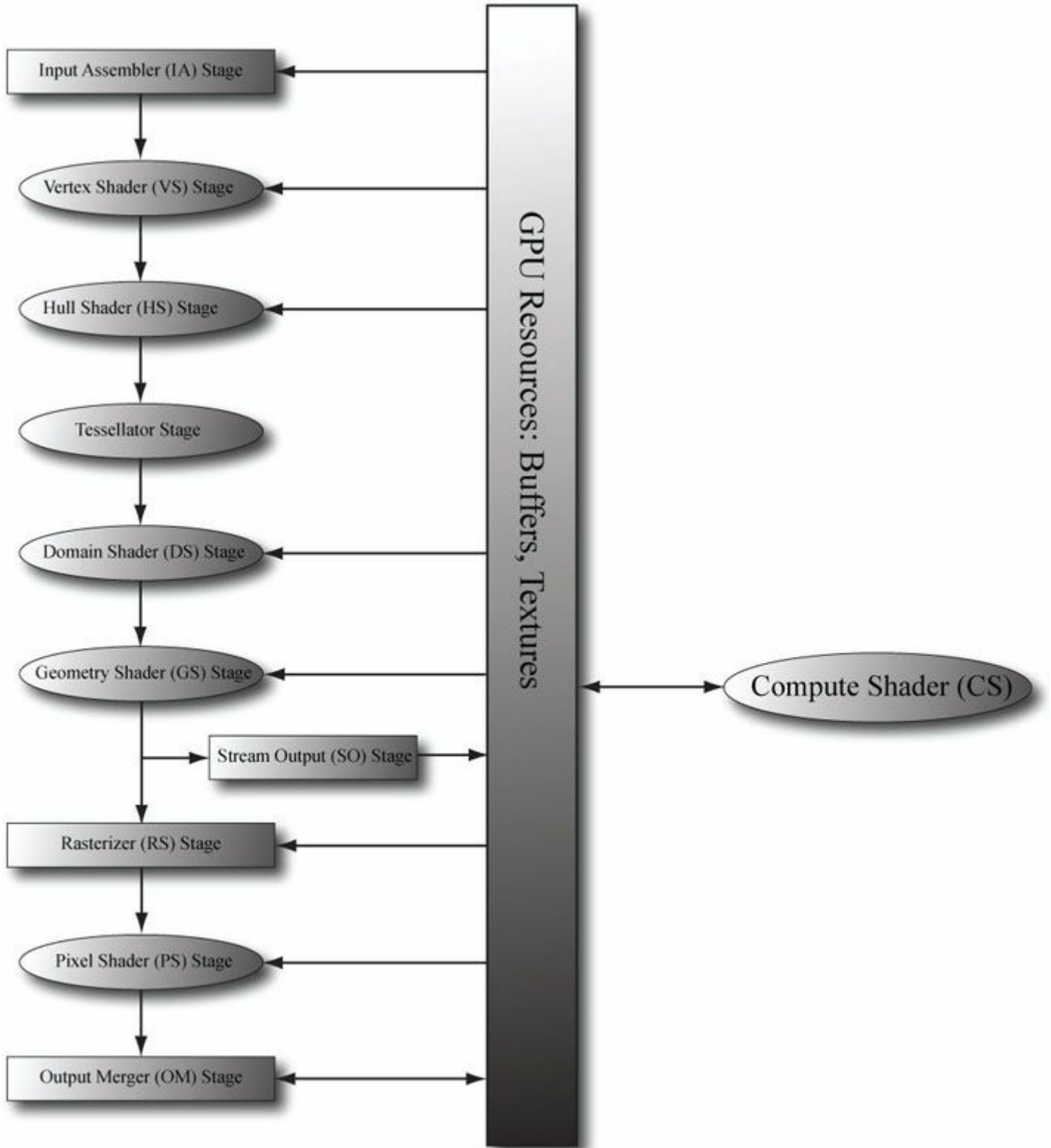


그림 12.2. 컴퓨트 셰이더는 렌더링 파이프라인의 일부가 아니라 옆에 위치합니다. 컴퓨트 셰이더는 GPU 리소스를 읽고 쓸 수 있습니다. 컴퓨트 셰이더는 그래픽 렌더링과 혼합하거나 GPGPU 프로그래밍을 위해 단독으로 사용할 수 있습니다.

12.1 스레드와 스레드 그룹

GPU 프로그래밍에서 실행을 원하는 스레드 수는 *스레드 그룹* 그리드로 분할됩니다. 스레드 그룹은 단일 멀티프로세서에서 실행됩니다. 따라서 16개의 멀티프로세서를 가진 GPU를 사용한다면, 각 멀티프로세서가 처리할 작업이 있도록 문제를 최소 16개의 스레드 그룹으로 분할해야 합니다. 성능 향상을 위해 멀티프로세서당 최소 두 개의 스레드 그룹을 할당하는 것이 좋습니다. 멀티프로세서는 스몰을 숨기기 위해 다른 그룹의 스레드 처리를 전환할 수 있기 때문입니다[Fung10] (예를 들어, 세이더가 다음 명령어로 진행하기 전에 텍스처 작업 결과를 기다려야 하는 경우 스몰이 발생할 수 있음).

각 스레드 그룹은 해당 그룹 내 모든 스레드가 접근할 수 있는 공유 메모리를 할당받습니다. 스레드는 다른 스레드 그룹의 공유 메모리에 접근할 수 없습니다. 다른 스레드 그룹의 공유 메모리에 접근할 수 없습니다. 스레드 그룹 내 스레드 간에는 동기화 작업이 가능하지만, 서로 다른 스레드 그룹 간 동기화는 불가능합니다. 실제로 서로 다른 스레드 그룹의 처리 순서를 제어할 수 없습니다. 이는 스레드 그룹이 서로 다른 멀티프로세서에서 실행될 수 있으므로 타당한 설계입니다.

스레드 그룹은 n개의 스레드로 구성됩니다. 하드웨어는 실제로 이 스레드들을 *워프*(워프당 32개 스레드)로 분할하며, 워프는 멀티프로세서에 의해 SIMD32로 처리됩니다(즉, 32개 스레드에 대해 동일한 명령어가 동시에 실행됨). 각 CUDA 코어는 하나의 스레드를 처리하며, "Fermi" 멀티프로세서는 32개의 CUDA 코어를 갖는다는 점을 상기하십시오(따라서 CUDA 코어는 SIMD "레인"과 유사함). Direct3D에서는 스레드 그룹 크기를 32의 배수가 아닌 차원으로 지정할 수 있지만, 성능상의 이유로 스레드 그룹 차원은 항상 워프 크기의 배수여야 합니다 [Fung10].

256 스레드 그룹 크기는 다양한 하드웨어에서 잘 작동할 수 있는 좋은 시작점입니다. 이후 실험을 통해 다른 크기. 그룹당 스레드 수를 변경하면 디스패치되는 그룹 수가 변경됩니다.

NVIDIA 하드웨어는 32스레드 규모의 워프(warp)를 사용합니다. ATI는 64스레드 규모의 "웨이브프론트(wavefront)"를 사용하며, Note: 스레드 그룹 크기는 항상 웨이브프론트 규모의 배수여야 한다고 권장합니다 [Bilodeau10]. 또한 워프 규모 또는 웨이브프론트 규모는 크기는 향후 하드웨어 세대에서 변경될 수 있습니다.

Direct3D에서는 스레드 그룹을 다음 메서드 호출을 통해 런칭합니다:

```
void ID3D11DeviceContext::Dispatch(
    UINT ThreadGroupCountX,
    UINT ThreadGroupCountY,
    UINT ThreadGroupCountZ);
```

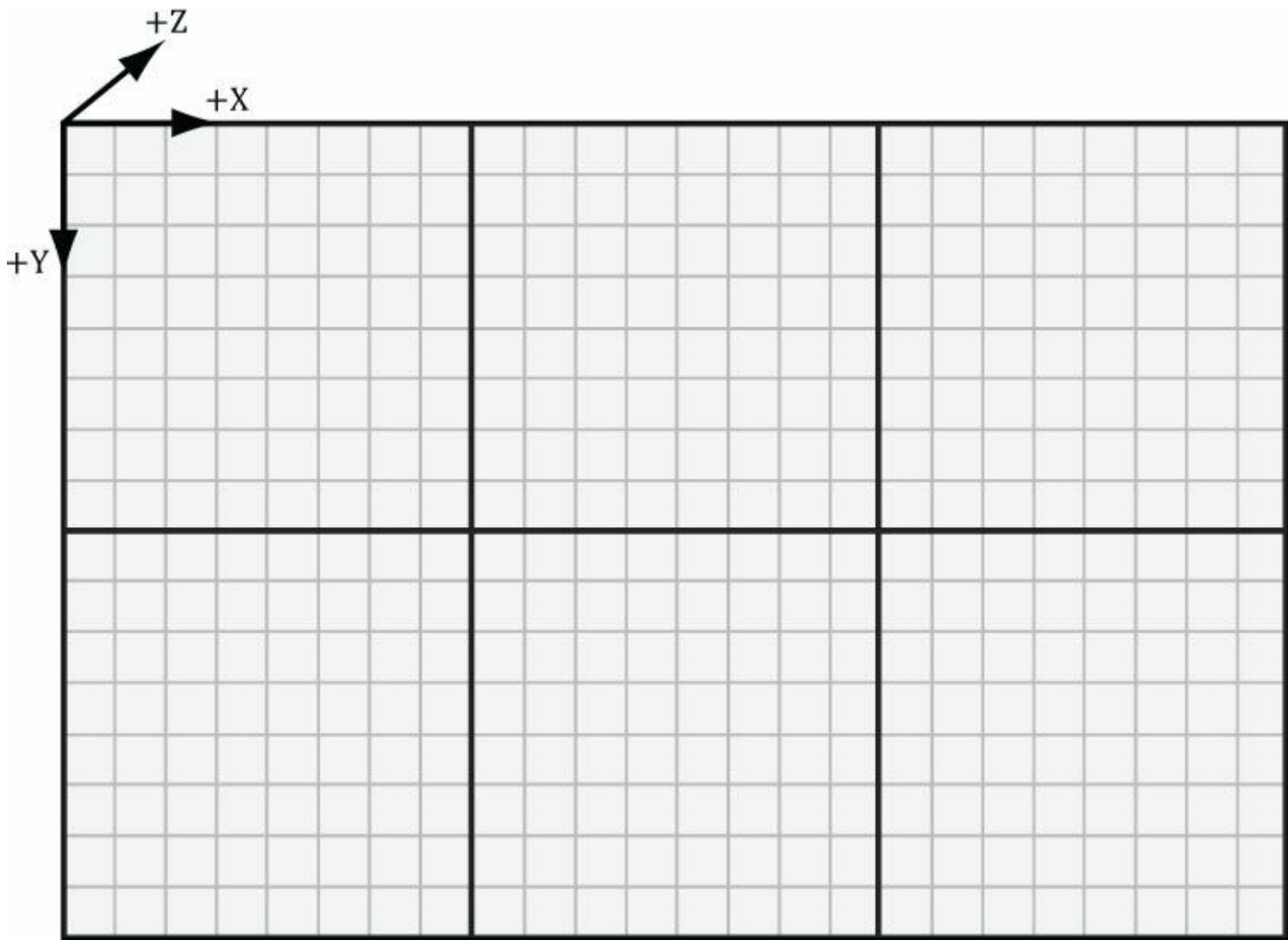


그림 12.3. 3×2 스레드 그룹 그리드 디스패치. 각 스레드 그룹은 8×8 스레드를 가집니다.

이를 통해 스레드 그룹의 3차원 그리드를 실행할 수 있지만, 이 책에서는 스레드 그룹의 2차원 그리드만 다룹니다. 다음 예제 호출은 x 방향으로 세 그룹, y 방향으로 두 그룹을 실행하여 총 $3 \times 2 = 6$ 개의 스레드 그룹을 생성합니다(그림 12.3 참조).

12.2 간단한 컴퓨트 셰이더

다음은 두 텍스처를 합산하는 간단한 컴퓨트 셰이더입니다. 모든 텍스처가 동일한 크기라고 가정합니다. 이 셰이더는 그다지 흥미롭지 않지만, 컴퓨트 셰이더 작성의 기본 구문을 보여줍니다.

```
cbuffer cbSettings
{
    // 컴퓨트 셰이더는 상수 버퍼의 값에 접근할 수 있습니다.
};

// 데이터 소스 및 출력. Texture2D
gInputA; Texture2D gInputB;
RWTexture2D<float4> gOutput;

// 스레드 그룹의 스레드 수. 그룹 내 스레드는
// 1차원, 2차원 또는 3차원 그리드 레이아웃으로 배열될 수 있습니다.
[numthreads(16, 16, 1)]
void CS(int3 dispatchThreadID : SV_DispatchThreadID) // 스레드 ID
{
    // x번째, y번째 텍셀을 합산하고 결과를 x번째, y번째 텍셀에 저장
    // gOutput의 x번째, y번째 텍셀에 저장합니다.
```

```
        gOutput[dispatchThreadID.xy] =
            gInputA[dispatchThreadID.xy] +
            gInputB[dispatchThreadID.xy];
    }

    technique11 AddTextures
    {
        pass P0
        {
            SetVertexShader(NULL); SetPixelShader(NULL);
            SetComputeShader(CompileShader(cs_5_0, CS()));
        }
    }
}
```

컴퓨터 셰이더는 다음 구성 요소로 이루어집니다:

1. 상수 버퍼를 통한 전역 변수 접근.
2. 다음 섹션에서 설명할 입력 및 출력 리소스.
3. 스레드 그룹 내 스레드 수를 3차원 스레드 그리드로 지정하는 `[numthreads(X, Y, Z)]` 속성.
4. 각 스레드에 대해 실행할 명령어가 포함된 셰이더 본체.
5. 스레드 식별 시스템 값 매개변수 (§12.4에서 설명됨).

스레드 그룹의 다양한 토폴로지를 정의할 수 있음을 주목하십시오. 예를 들어, 스레드 그룹은 X 개의 스레드로 구성된 단일 행(`[numthreads(X, 1, 1)]`)이거나 Y 개의 스레드로 구성된 단일 열(`[numthreads(1, Y, 1)]`)일 수 있습니다. $X \times Y$ 스레드로 구성된 2차원 스레드 그룹은 z -차원을 1로 설정하여 다음과 같이 만들 수 있습니다(`[numthreads(X, Y, 1)]`). 선택하는 토폴로지는 작업 중인 문제에 따라 결정됩니다. 이전 섹션에서 언급했듯이, 그룹당 총 스레드 수는 워프 크기(NVIDIA 카드 기준 32)의 배수 또는 웨이브프론트 크기(ATI 카드 기준 64)의 배수여야 합니다. 웨이브프론트 크기의 배수는 워프 크기의 배수이기도 하므로, 웨이브프론트 크기의 배수를 선택하면 두 유형의 카드 모두에 적용됩니다.

12.3 데이터 입력 및 출력 리소스

컴퓨터 셰이더에는 두 가지 유형의 리소스를 바인딩할 수 있습니다: 버퍼와 텍스처입니다. 우리는 이미 버텍스 버퍼, 인덱스 버퍼, 상수 버퍼와 같은 버퍼를 다루어 보았습니다. 효과 프레임워크를 사용하여 상수 버퍼를 설정하지만, 이들은 단순히 `D3D11_BIND_CONSTANT_BUFFER` 플러그가 설정된 `ID3D11Buffer` 인스턴스입니다. 또한 [8장에서](#) 텍스처 리소스에 대해 이미 알고 있습니다.

12.3.1 텍스처 입력

이전 섹션에서 정의한 컴퓨터 셰이더는 두 개의 입력 텍스처 리소스를 정의했습니다:

```
Texture2D gInputA; Texture2D
gInputB;
```

입력 텍스처 `gInputA`와 `gInputB`는 텍스처에 대한 `ID3D11ShaderResourceViews(SRVs)`를 생성하고 `ID3DX11EffectShaderResourceVariable` 변수를 통해 컴퓨터 셰이더에 설정함으로써 셰이더에 입력으로 바인딩됩니다. 이는 픽셀 셰이더에 셰이더 리소스 뷰를 바인딩하는 방식과 정확히 동일합니다. SRVs는 읽기 전용이라는 점에 유의하십시오.

12.3.2 텍스처 출력 및 무순서 액세스 뷰(UAV)

이전 섹션에서 정의한 컴퓨터 셰이더는 하나의 출력 리소스를 정의했습니다:

```
RWTexture2D<float4> gOutput;
```

출력(Outputs)은 특별하게 처리되며, 그 유형 앞에 "RW"라는 특수 접두사가 붙습니다. 이는 읽기-쓰기(read-write)를 의미하며, 이름에서 알 수 있듯이 컴퓨터 셰이더 내에서 이 리소스의 요소를 읽고 쓸 수 있습니다. 반면 텍스처 `gInputA`와 `gInputB`는 읽기 전용입니다. 또한 출력의 유형과 차원을 템플릿 각괄호 구문 `<float4>`로 명시해야 합니다. 출력이 `DXGI_FORMAT_R8G8_SINT`와 같은 2차원 정수형이었다면 다음과 같이 작성했을 것입니다:

```
RWTexture2D<int2> gOutput;
```

그러나 출력 리소스 바인딩은 입력과 다릅니다. 컴퓨터 셰이더에서 쓰기 작업을 수행할 리소스를 바인딩하려면

새로운 뷰 유형인 *무순접근뷰* (UAV)를 사용하여 바인딩해야 합니다. 이는 코드에서

ID3D11UnorderedAccessView 인터페이스로 표현됩니다. 이는 셰이더 리소스 뷰와 유사한 방식으로 생성됩니다. 예시는 다음과 같습니다:

```
// 참고: 압축된 형식은 UAV에 사용할 수 없습니다. 다음과 같은 오류가 발생합니다:
// ERROR: ID3D11Device::CreateTexture2D: The format (0x4d, BC3_UNORM)
// UnorderedAccessView로 바인딩하거나 형식으로 캐스팅할 수 없습니다.

// 무순접근뷰(UnorderedAccessView)로 바인딩될 수 있습니다. 따라서 이 형식은
// D3D11_BIND_UNORDERED_ACCESS를 지원하지 않습니다.
```

```
D3D11_TEXTURE2D_DESC blurredTexDesc; blurredTexDesc.Width =
width; blurredTexDesc.Height = height; blurredTexDesc.MipLevels =
1;
blurredTexDesc.ArraySize = 1; blurredTexDesc.Format = format;
blurredTexDesc.SampleDesc.Count = 1;
blurredTexDesc.SampleDesc.Quality = 0; blurredTexDesc.Usage =
D3D11_USAGE_DEFAULT;
blurredTexDesc.BindFlags = D3D11_BIND_SHADER_RESOURCE |
D3D11_BIND_UNORDERED_ACCESS;
blurredTexDesc.CPUAccessFlags = 0;
blurredTexDesc.MiscFlags = 0;
```

```
ID3D11Texture2D* blurredTex = 0;
HR(device->CreateTexture2D(>blurredTexDesc, 0, >blurredTex));
```

```
D3D11_SHADER_RESOURCE_VIEW_DESC srvDesc;
srvDesc.Format = format;
srvDesc.ViewDimension = D3D11_SRV_DIMENSION_TEXTURE2D;
srvDesc.Texture2D.MostDetailedMip = 0;
srvDesc.Texture2D.MipLevels = 1;
```

```
ID3D11ShaderResourceView* mBlurredOutputTexSRV; HR(device-
>CreateShaderResourceView(blurredTex,
>srvDesc, >mBlurredOutputTexSRV));
```

```
D3D11_UNORDERED_ACCESS_VIEW_DESC uavDesc;
uavDesc.Format = format;
uavDesc.ViewDimension = D3D11_UAV_DIMENSION_TEXTURE2D; uavDesc.Texture 2D.MipSlice =
0;
```

```
ID3D11UnorderedAccessView* mBlurredOutputTexUAV; HR(device->Create
UnorderedAccessView(blurredTex,
>uavDesc, >mBlurredOutputTexUAV));
```

```
// 뷰는 텍스처에 대한 참조를 저장하므로 우리의 참조를 해제할 수 있습니다. ReleaseCOM(blurredTex);
```

텍스처가 UAV로 바인딩될 경우 **D3D11_BIND_UNORDERED_ACCESS** 플래그로 생성되어야 함을 유의하십시오; 이전 예시에서 텍스처는 UAV와 SRV로 바인딩됩니다(동시 바인딩은 아님). 따라서 결합된 플래그 **D3D11_BIND_SHADER_RESOURCE | D3D11_BIND_UNORDERED_ACCESS**를 가집니다. 이는 흔한 경우로, 컴퓨터 셰이더로 텍스처에 대한 작업을 수행한 후(텍스처가 컴퓨터 셰이더에 UAV로 바인딩됨), 이후 해당 텍스처로 지오메트리를 텍스처링하기 위해 버텍스 또는 픽셀 셰이더에 SRV로 바인딩하기 때문입니다.

를 사용하여 **ID3D11UnorderedAccessView** 객체가 생성되면, **SetUnorderedAccessView** 메서드를 사용하여 **"RW" 컴퓨터 셰이더** 변수에 바인딩할 수 있습니다:

```
// HLSL 변수. Texture2D gInputA;
Texture2D gInputB;
RWTexture2D<float4> gOutput;
```

```
// C++ 코드.
```

```
ID3DX11EffectShaderResourceVariable* InputA;
ID3DX11EffectShaderResourceVariable* InputB;
ID3DX11EffectUnorderedAccessViewVariable* Output;
```

```
InputA = mFX->GetVariableByName("gInputA")->AsShaderResource(); InputB = mFX-
>GetVariableByName("gInputB")->AsShaderResource(); Output = mFX->GetVariableByName("gOutput")-
>AsUnorderedAccessView();
```

```
void SetInputA(ID3D11ShaderResourceView* tex)
{
    InputA->SetResource(tex);
}
```

```
void SetInputB(ID3D11ShaderResourceView* tex)
{
    InputB->SetResource(tex);
}
```

```
void SetOutput(ID3D11UnorderedAccessView* tex)
{
    Output->SetUnorderedAccessView(tex);
}
```

12.3.3 텍스처 인덱싱 및 샘플링

텍스처의 요소는 2차원 인덱스를 사용하여 접근할 수 있습니다. §12.2에서 정의된 컴퓨트 셰이더에서는 디스패치 스레드 ID(스레드 ID는 §12.4에서 설명됨)를 기반으로 텍스처를 인덱싱합니다. 각 스레드에는 고유한 디스패치 ID가 부여됩니다.

```
[numthreads(16, 16, 1)]
void CS(int3 dispatchThreadID : SV_DispatchThreadID)
{
    // xy번째 텍셀을 합산하고 결과를 gOutput의 xy번째 텍셀에 저장
    // gOutput. gOutput[dispatchThreadID.xy] =
        gInputA[dispatchThreadID.xy] +
        gInputB[dispatchThreadID.xy];
}
```

텍스처를 커버할 만큼 충분한 스레드 그룹을 디스패치했다고 가정하면(즉, 텍셀 하나당 하나의 스레드가 실행됨), 이 코드는 텍스처 이미지를 합산하여 결과값을 **텍스처 gOutput에** 저장합니다.



컴퓨트 셰이더에서 범위를 벗어난 인덱스의 동작은 명확히 정의되어 있습니다. 범위를 벗어난 읽기는 0을 반환하며, 범위를 벗어난 쓰기는 아무 작업도 수행하지 않습니다 [Boyd08].

컴퓨트 셰이더는 GPU에서 실행되므로 일반적인 GPU 도구를 사용할 수 있습니다. 특히 텍스처 필터링을 통해 텍스처를 샘플링할 수 있습니다. 그러나 두 가지 문제가 있습니다. 첫째, **Sample** 메서드를 사용할 수 없으며 대신 **SampleLevel** 메서드를 사용해야 합니다. **SampleLevel**은 텍스처의 mip 맵 레벨을 지정하는 추가적인 세 번째 매개변수를 받습니다. 0은 최상위 레벨을, 1은 두 번째 mip 레벨을 선택하며, 선형 mip 필터링이 활성화된 경우 소수값을 사용하여 두 mip 레벨 사이를 보간합니다. 반면 **Sample**은 텍스처가 화면에서 차지할 픽셀 수를 기반으로 최적의 mip 맵 레벨을 자동 선택합니다. 컴퓨트 셰이더는 직접 렌더링에 사용되지 않으므로 이와 같은 자동 선택 방식을 알지 못하므로, 컴퓨트 셰이더에서는 **SampleLevel**로 레벨을 명시적으로 지정해야 합니다. 두 번째 문제는 텍스처를 샘플링할 때 정수 인덱스 대신 [0, 1] 범위 내의 정규화된 텍스처 좌표²를 사용한다는 점입니다. 그러나 텍스처 크기(*width*, *height*)를 상수 버퍼 변수에 설정하면 정규화된 텍스처 좌표를 정수 인덱스(*x*, *y*)로부터 유도할 수 있습니다:

$$u = \frac{x}{width}$$
$$v = \frac{y}{height}$$

다음 코드는 정수 인덱스를 사용하는 컴퓨터 셰이더와, 텍스처 좌표 및

텍스처 크기가 512×512 이고 최상위 mip만 필요하다고 가정합니다:

```
//
// 버전 1: 정수 인덱스 사용.
//

cbuffer cbUpdateSettings
{
    float gWaveConstants[3];
};
Texture2D gPrevSolInput; Texture2D
gCurrSolInput;
RWTexture2D<float> gNextSolOutput;

[numthreads(16, 16, 1)]
void CS(int3 dispatchThreadID : SV_DispatchThreadID)
{
    int x = dispatchThreadID.x; int y =
dispatchThreadID.y;

    gNextSolOutput[int2(x,y)] =
        gWaveConstants[0]* gPrevSolInput[int2(x,y)].r + gWaveConstants[1]*
        gCurrSolInput[int2(x,y)].r + gWaveConstants[2]*(
            gCurrSolInput[int2(x,y+1)].r          +
            gCurrSolInput[int2(x,y-1)].r          +
            gCurrSolInput[int2(x+1,y)].r          +
            gCurrSolInput[int2(x-1,y)].r);
}
//
// 버전 2: 샘플 레벨과 텍스처 좌표 사용.
//

cbuffer cbUpdateSettings
{
    float gWaveConstants[3];
};

샘플러 상태 samPoint
{
    Filter = MIN_MAG_MIP_POINT; AddressU =
CLAMP;
    AddressV = CLAMP;
};

Texture2D gPrevSolInput; Texture2D
gCurrSolInput;
RWTexture2D<float> gNextSolOutput;

[numthreads(16, 16, 1)]
void CS(int3 dispatchThreadID : SV_DispatchThreadID)
{
    // [] 연산자 대신 SampleLevel()을 사용하는 것과 동등함. int x = dispatchThreadID.x;
    int y = dispatchThreadID.y;

    float2 c = float2(x,y)/512.0f; float2 t =
float2(x,y-1)/512.0; float2 b =
float2(x,y+1)/512.0; float2 l = float2(x-
1,y)/512.0; float2 r = float2(x+1,y)/512.0;
```

```
gNextSolOutput[int2(x,y)] = gWaveConstants[0]*gPrevSolInput.SampleLevel(samPoint, c, 0.0f).r +
gWaveConstants[1]*gCurrSolInput.SampleLevel(samPoint, c, 0.0f).r + gWaveConstants[2]*(
gCurrSolInput.SampleLevel(samPoint, b, 0.0f).r +
gCurrSolInput.SampleLevel(samPoint, t, 0.0f).r +
gCurrSolInput.SampleLevel(samPoint, r, 0.0f).r +
gCurrSolInput.SampleLevel(samPoint, 1, 0.0f).r);
}
```

12.3.4 구조화된 버퍼 리소스

다음 예제는 HLSL에서 *구조화된 버퍼*가 어떻게 정의되는지 보여줍니다:

```
struct Data
{
    float3 v1;
    float2 v2;
};
StructuredBuffer<Data> gInputA; StructuredBuffer<Data> gInputB;

RWStructuredBuffer<Data> gOutput;
```

구조체 버퍼는 단순히 동일한 유형의 요소들로 구성된 버퍼, 즉 본질적으로 배열입니다. 보시다시피, 유형은 HLSL에서 사용자 정의 구조체일 수 있습니다. 구조체 버퍼는 일반 버퍼와 마찬가지로 생성되지만, 구조체 버퍼 플래그를 지정하고 저장할 요소의 바이트 크기를 반드시 명시해야 합니다:

```
struct Data
{
    XMFLOAT3 v1;
    XMFLOAT2 v2;
};

// 셰이더에 바인딩될 버퍼 생성

// 입력 (D3D11_BIND_SHADER_RESOURCE).

D3D11_BUFFER_DESC inputDesc; inputDesc.Usage =
D3D11_USAGE_DEFAULT;
inputDesc.ByteWidth = sizeof(Data) * mNumElements; inputDesc.BindFlags =
D3D11_BIND_SHADER_RESOURCE; inputDesc.CPUAccessFlags = 0;
inputDesc.StructureByteStride = sizeof(Data);
inputDesc.MiscFlags = D3D11_RESOURCE_MISC_BUFFER_STRUCTURED;

D3D11_SUBRESOURCE_DATA vinitDataA; vinitDataA.pSysMem =
>dataA[0];

ID3D11Buffer* bufferA = 0;
HR(md3dDevice->CreateBuffer(>inputDesc, >vinitDataA, >bufferA));

// 컴퓨터 셰이더가 사용할 수 있는 읽기/쓰기 버퍼 생성

// (D3D11_BIND_UNORDERED_ACCESS)에 쓰기.
```

```
D3D11_BUFFER_DESC outputDesc; outputDesc.Usage =
D3D11_USAGE_DEFAULT;
outputDesc.ByteWidth = sizeof(Data) * mNumElements; outputDesc.BindFlags =
D3D11_BIND_UNORDERED_ACCESS; outputDesc.CPUAccessFlags = 0;
outputDesc.StructureByteStride = sizeof(Data);
outputDesc.MiscFlags = D3D11_RESOURCE_MISC_BUFFER_STRUCTURED; ID3D11Buffer* mOutputBuffer;
```

```
HR(md3dDevice->CreateBuffer(>outputDesc, 0, >mOutputBuffer));
```

구조체 버퍼를 컴퓨터 셰이더에 입력으로 바인딩하려면, 해당 버퍼에 대한 SRV를 생성한 후 **ID3DX11EffectShaderResourceVariable** 변수를 통해 컴퓨터 셰이더에 설정합니다. 마찬가지로 구조체 버퍼를 RW 출력으로 바인딩하려면, 해당 버퍼에 대한 UAV를 생성한 후 **ID3DX11EffectUnorderedAccessViewVariable** 변수를 통해 컴퓨터 셰이더에 설정합니다. 다음 코드는 구조체 버퍼에 대한 SRV 및 UAV 생성 방법을 보여줍니다:

```
D3D11_SHADER_RESOURCE_VIEW_DESC srvDesc;
srvDesc.Format = DXGI_FORMAT_UNKNOWN; srvDesc.ViewDimension =
D3D11_SRV_DIMENSION_BUFFEREX; srvDesc.BufferEx.FirstElement = 0;
srvDesc.BufferEx.Flags = 0; srvDesc.BufferEx.NumElements = mNumElements;
```

```
md3dDevice->CreateShaderResourceView(bufferA, &srvDesc, &mInputASRV); md3dDevice-
>CreateShaderResourceView(bufferB, &srvDesc, &mInputBSRV);
```

```
D3D11_UNORDERED_ACCESS_VIEW_DESC uavDesc;
uavDesc.Format = DXGI_FORMAT_UNKNOWN; uavDesc.ViewDimension =
D3D11_UAV_DIMENSION_BUFFER; uavDesc.Buffer.FirstElement = 0;
uavDesc.Buffer.Flags = 0; uavDesc.Buffer.NumElements =
mNumElements;
```

```
md3dDevice->CreateUnorderedAccessView(mOutputBuffer, &uavDesc, &mOutputUAV);
```

다음 코드 섹션은 이펙트 프레임워크를 통해 컴퓨터 셰이더에 SRV와 UAV를 설정하는 방법을 보여줍니다:

```
// HLSL 변수. struct Data
```

```
{
    float3 v1;
    float2 v2;
};
```

```
StructuredBuffer<Data> gInputA; StructuredBuffer<Data> gInputB;
RWStructuredBuffer<Data> gOutput;
```

```
// C++ 코드. ID3DX11EffectShaderResourceVariable* InputA;
ID3DX11EffectShaderResourceVariable* InputB;
ID3DX11EffectUnorderedAccessViewVariable* Output;
```

```
InputA = mFX->GetVariableByName("gInputA")->AsShaderResource(); InputB = mFX-
>GetVariableByName("gInputB")->AsShaderResource(); Output = mFX->GetVariableByName("gOutput")-
>AsUnorderedAccessView();
```

```
void SetInputA(ID3D11ShaderResourceView* srv)
{
    InputA->SetResource(srv);
}
```

```
void SetInputB(ID3D11ShaderResourceView* srv)
{
    InputB->SetResource(srv);
}
```

```
void SetOutput(ID3D11UnorderedAccessView* uav)
{
    Output->SetUnorderedAccessView(uav);
}
```


구조화된 버퍼에 대한 SRV 또는 UAV를 생성할 때 Format 속성에 **DXGI_FORMAT_UNKNOWN**을 지정하는 점에 유의하십시오. 이는 일반적으로 구조화된 버퍼가 사용자 정의 유형을 사용하므로 해당 구조가 **DXGI_FORMAT** 멤버 중 하나와 직접 대응하지 않기 때문입니다. 타입 *지정 버퍼*(*typed buffer*)라는 것이 있으며, HLSL 구문은 다음과 같습니다:

```
Buffer<float4> typedBuffer1; Buffer<float>
typedBuffer2; Buffer<int2> typedBuffer3;
```

이러한 버퍼를 생성할 때는 **D3D11_RESOURCE_MISC_BUFFER_STRUCTURED** 플래그를 지정하지 마십시오. 뷰를 생성할 때는 Format 속성에 적절한 **DXGI_FORMAT**을 반드시 지정해야 합니다. HLSL 구문은 단순히 타입과 구성 요소 수를 정의할 뿐이지만, HLSL 타입에 대응할 수 있는 **DXGI_FORMAT** 멤버는 다양합니다. 예를 들어 **typedBuffer1**의 가능한 형식은 4개 구성 요소를 가진 모든 부동 소수점 형식입니다: **DXGI_FORMAT_R32G32B32A32_FLOAT**, **DXGI_FORMAT_R16G16B16A16_FLOAT**, 및 **DXGI_FORMAT_R8G8B8A8_UNORM**. 마찬가지로 **typedBuffer3**의 가능한 형식은 **DXGI_FORMAT_R32G32_SINT**, **DXGI_FORMAT_R16G16_SINT**, **DXGI_FORMAT_R8G8_SINT**입니다.

 *원시 버퍼(raw buffer)라는 것도 있는데, 기본적으로 데이터의 바이트 배열입니다. 바이트 오프셋을 사용하며 데이터를 적절한 유형으로 캐스팅할 수 있습니다. 예를 들어 동일한 버퍼에 서로 다른 데이터 유형을 저장하는 데 유용할 수 있습니다. 이 책에서는 원시 버퍼를 사용하지 않습니다. 자세한 내용은 SDK 문서를 참조하십시오. SDK 샘플 "BasicCompute11"은 원시 버퍼를 생성하고 사용하는 방법을 보여줍니다.*

12.3.5 CS 결과를 시스템 메모리로 복사하기

일반적으로 컴퓨터 셰이더를 사용하여 텍스처를 처리할 때는 처리된 텍스처를 화면에 표시합니다. 따라서 시각적으로 결과를 확인하여 컴퓨터 셰이더의 정확성을 검증할 수 있습니다. 그러나 구조화된 버퍼 계산이나 일반적인 GPGPU 컴퓨팅에서는 결과를 전혀 표시하지 않을 수도 있습니다. 따라서 문제는 GPU 메모리(UAV를 통해 구조화 버퍼에 쓰면 해당 버퍼가 GPU 메모리에 저장됨)에서 결과를 시스템 메모리로 어떻게 가져올 것인가입니다. 일반적인 패턴은 스테이징 플래그 **D3D11_USAGE_STAGING**과 CPU 액세스 플래그 **D3D11_CPU_ACCESS_READ**를 사용하여 시스템 메모리 버퍼를 생성하는 것입니다. 그런 다음 **ID3D11DeviceContext::CopyResource** 메서드를 사용하여 GPU 리소스를 시스템 메모리 리소스로 복사할 수 있습니다. 시스템 메모리 리소스는 복사하려는 리소스와 동일한 유형 및 크기여야 합니다. 마지막으로 매핑 API를 사용하여 시스템 메모리 버퍼를 매핑하여 CPU에서 읽을 수 있습니다. 이후 데이터를 시스템 메모리 배열로 복사하여 CPU 측에서 추가 처리하거나, 데이터를 파일에 저장하거나, 기타 작업을 수행할 수 있습니다.

이 장을 위해 "VecAdd"라는 구조화 버퍼 데모를 포함시켰습니다. 이 데모는 단순히 두 구조화 버퍼에 저장된 대응하는 벡터 구성 요소들을 단순히 합산합니다:

```
struct Data
{
    float3 v1;
    float2 v2;
};

StructuredBuffer<Data> gInputA; StructuredBuffer<Data> gInputB;
RWStructuredBuffer<Data> gOutput;

[numthreads(32, 1, 1)]
void CS(int3 dtid : SV_DispatchThreadID)
{
    gOutput[dtid.x].v1 = gInputA[dtid.x].v1 + gInputB[dtid.x].v1; gOutput[dtid.x].v2 =
    gInputA[dtid.x].v2 + gInputB[dtid.x].v2;
}
```

간편함을 위해 구조체 버퍼는 32개 요소만 포함합니다. 따라서 하나의 스레드 그룹만 디스패치하면 됩니다(한 스레드 그룹이 32개 요소를 처리하기 때문). 이 데모에서 컴퓨터 셰이더가 모든 스레드에 대한 작업을 완료한 후, 결과를 시스템 메모리로 복사하여 파일에 저장합니다. 다음 코드는 시스템 메모리 버퍼 생성 방법과 GPU 결과를 CPU 메모리로 복사하는 방법을 보여줍니다:

```
// 결과를 읽기 위한 시스템 메모리 버퍼 생성

// 결과를 다시 읽을 수 있도록 버퍼의 시스템 메모리
버전을 생성합니다. D3D11_BUFFER_DESC
outputDesc;
```

```

outputDesc.Usage = D3D11_USAGE_STAGING; outputDesc.BindFlags
= 0;
outputDesc.ByteWidth = sizeof(Data) * mNumElements; outputDesc.CPUAccessFlags =
D3D11_CPU_ACCESS_READ; outputDesc.StructureByteStride = sizeof(Data);
outputDesc.MiscFlags = D3D11_RESOURCE_MISC_BUFFER_STRUCTURED;

ID3D11Buffer* mOutputDebugBuffer;
HR(md3dDevice->CreateBuffer(>outputDesc, 0, >mOutputDebugBuffer));

// ...
//
// 컴퓨트 셰이더 완료!

struct Data
{
    XMFLOAT3 v1;
    XMFLOAT2 v2;
};

// 출력 버퍼를 시스템 메모리로 복사합니다.
md3dImmediateContext->CopyResource(mOutputDebugBuffer, mOutputBuffer);

// 읽기용 데이터 매핑. D3D11_MAPPED_SUBRESOURCE
mappedData;
md3dImmediateContext->Map(mOutputDebugBuffer, 0, D3D11_MAP_READ, 0, >mappedData); Data* dataView =
reinterpret_cast<Data*>(mappedData.pData);

for(int i = 0; i < mNumElements; ++i)
{
    fout << "(" << dataView[i].v1.x << ", " << dataView[i].v1.y
    << ", " <<
    dataView[i].v1.z << ", " <<
    dataView[i].v2.x << ", " << dataView[i].v2.y << ")"
    << std::endl;
}

md3dImmediateContext->Unmap(mOutputDebugBuffer, 0); fout.close();

```

데모에서는 두 입력 버퍼를 다음과 같은 초기 데이터로 채웁니다:

```

std::vector<Data> dataA(mNumElements);
std::vector<Data> dataB(mNumElements); for(int i = 0; i <
mNumElements; ++i)
{
    dataA[i].v1 = XMFLOAT3(i, i, i); dataA[i].v2 =
    XMFLOAT2(i, 0); dataB[i].v1 = XMFLOAT3(-i, i,
    0.0f); dataB[i].v2 = XMFLOAT2(0, -i);
}

```

결과 텍스트 파일에는 다음과 같은 데이터가 포함되어 있으며, 이는 컴퓨트 셰이더가 예상대로 작동하고 있음을 확인시켜 줍니다.

```

(0, 0, 0, 0, 0)
(0, 2, 1, 1, -1)
(0, 4, 2, 2, -2)
(0, 6, 3, 3, -3)
(0, 8, 4, 4, -4)
(0, 10, 5, 5, -5)
(0, 12, 6, 6, -6)

```

(0, 14, 7, 7, -7)
(0, 16, 8, 8, -8)
(0, 18, 9, 9, -9)
(0, 20, 10, 10, -10)
(0, 22, 11, 11, -11)
(0, 24, 12, 12, -12)
(0, 26, 13, 13, -13)
(0, 28, 14, 14, -14)
(0, 30, 15, 15, -15)
(0, 32, 16, 16, -16)
(0, 34, 17, 17, -17)
(0, 36, 18, 18, -18)
(0, 38, 19, 19, -19)
(0, 40, 20, 20, -20)
(0, 42, 21, 21, -21)
(0, 44, 22, 22, -22)
(0, 46, 23, 23, -23)
(0, 48, 24, 24, -24)
(0, 50, 25, 25, -25)
(0, 52, 26, 26, -26)
(0, 54, 27, 27, -27)
(0, 56, 28, 28, -28)
(0, 58, 29, 29, -29)
(0, 60, 30, 30, -30)
(0, 62, 31, 31, -31)

그림 12.1에서 볼 수 있듯이 CPU와 GPU 메모리 간 복사 작업이 가장 느립니다. 그래픽스 작업에서는 프레임마다 이 복사 작업을 수행해서는 안 되며, 이는 성능을 크게 저하시킵니다. GPGPU 프로그래밍에서는 일반적으로 결과를 CPU로 다시 가져와야 하지만, 이는 GPGPU 프로그래밍에서 큰 문제가 되지 않습니다. 왜냐하면 GPU에서 수행함으로써 얻는 이점이

 GPU가 GPU에서 CPU로의 복사 비용을 상쇄합니다. 게다가 GPGPU의 경우 복사 빈도가 "프레임당" 보다 낮습니다. 예를 들어, 애플리케이션이 비용이 많이 드는 이미지 처리 계산을 구현하기 위해 GPGPU 프로그래밍을 사용한다고 가정해 보겠습니다. 계산이 완료된 후 결과는 CPU로 복사됩니다. 사용자가 다른 계산을 요청할 때까지 GPU는 다시 사용되지 않습니다.

12.4 스레드 식별 시스템 값

그림 12.4를 고려하십시오.

1. 각 스레드 그룹에는 시스템에 의해 ID가 할당됩니다. 이를 *그룹 ID*라고 하며 시스템 값인 **SV_GroupID**를 가집니다. $G_x \times G_y \times G_z$ 가 디스패치된 스레드 그룹의 수라면, 그룹 ID는 (0,0,0)부터 (G_x-1, G_y-1, G_z-1) 까지의 범위를 가집니다.
2. 스레드 그룹 내부에서 각 스레드에는 해당 그룹에 대해 고유한 ID가 부여됩니다. 스레드 그룹의 크기가 $X \times Y \times Z$ 인 경우, *그룹 스레드 ID*는 (0, 0, 0)부터 $(X-1, Y-1, Z-1)$ 까지의 범위를 가집니다. 그룹 스레드 ID에 대한 시스템 값 의미론은 **SV_GroupThreadID**입니다.
3. **디스패치** 호출은 스레드 그룹의 그리드를 디스패치합니다. *디스패치 스레드 ID*는 디스패치 호출로 생성된 모든 스레드에 대해 해당 스레드를 고유하게 식별합니다. 즉, 그룹 스레드 ID가 해당 스레드 그룹 내의 스레드를 고유하게 식별하는 반면, 디스패치 스레드 ID는 디스패치 호출로 디스패치된 모든 스레드 그룹의 스레드 전체 집합에 대해 해당 스레드를 고유하게 식별합니다. 스레드 그룹 크기를 **ThreadGroupSize = (X,Y,Z)**로 정의하면, 디스패치 스레드 ID는 그룹 ID와 그룹 스레드 ID로부터 다음과 같이 유도될 수 있습니다:

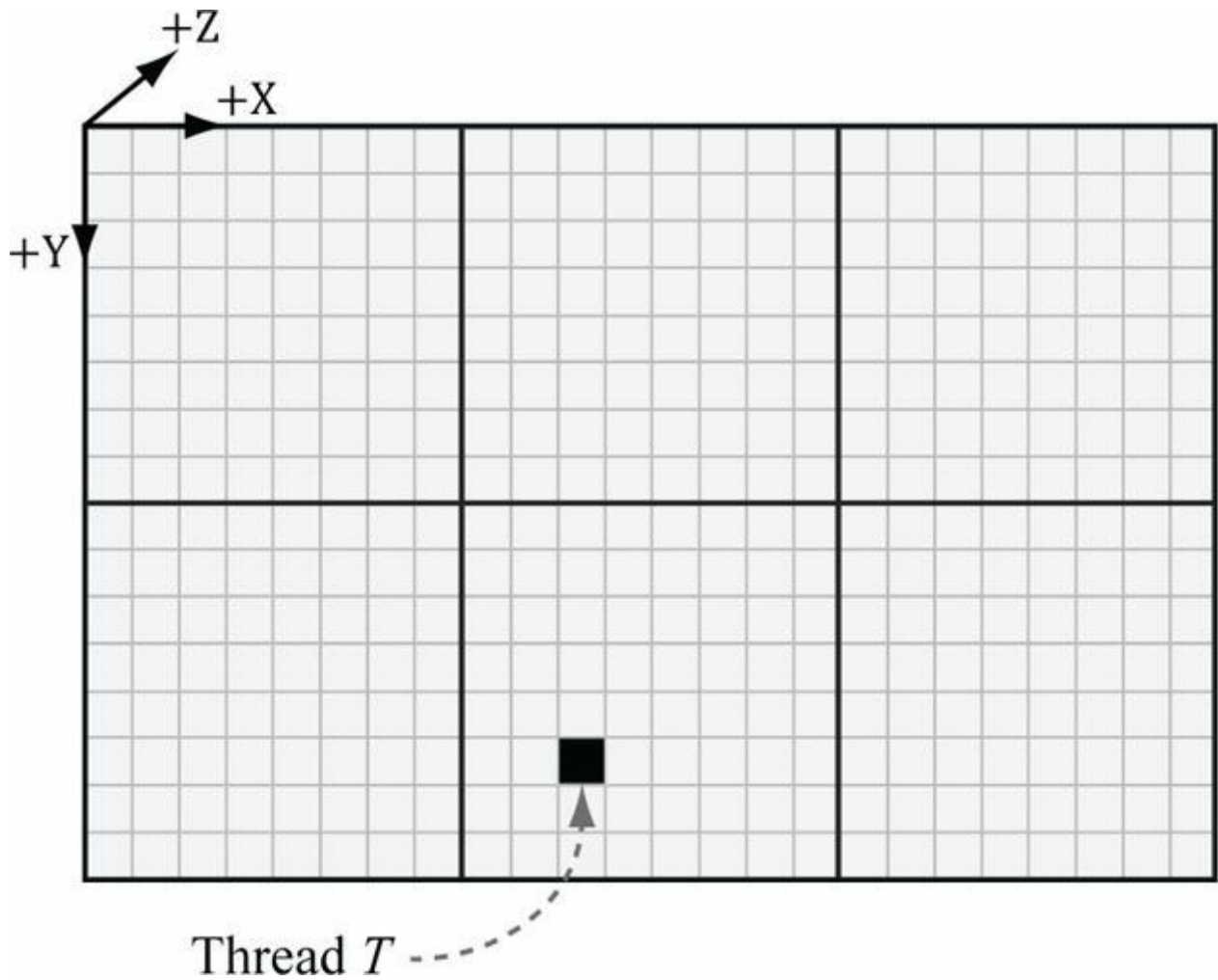


그림 12.4. 표시된 스레드 T 를 고려하자. 스레드 T 는 스레드 그룹 ID (1, 1, 0)을 가진다. 그룹 스레드 ID는 (2, 5, 0)이다. 디스패치 스레드 ID는 $(1, 1, 0) \otimes (8, 8, 0) + (2, 5, 0) = (10, 13, 0)$ 이다. 그룹 인덱스 ID는 $5 \cdot 8 + 2 = 42$ 입니다.

`dispatchThreadID.xyz = groupID.xyz * ThreadGroupSize.xyz + groupThreadID.xyz;`

디스패치 스레드 ID는 시스템 값 시맨틱 `SV_DispatchThreadID`를 가집니다. 각 스레드 그룹이 10×10 인 3×2 스레드 그룹이 디스패치되면 총 60개의 스레드가 디스패치되며 디스패치 스레드 ID는 (0, 0, 0)부터 (29, 19, 0)까지의 범위를 가집니다.

4. Direct3D는 `SV_GroupIndex` 시스템 값을 통해 그룹 스레드 ID의 선형 인덱스 버전을 제공합니다. 이는 다음과 같이 계산됩니다:

`groupIndex = groupThreadID.z*ThreadGroupSize.x*ThreadGroupSize.y +
groupThreadID.y*ThreadGroupSize.x +
groupThreadID.x;`

인덱싱 좌표 순서에 관해, 첫 번째 좌표는 x 위치(또는 열)를, 두 번째 좌표 Note: 는 y 위치(또는 행)를 나타냅니다. 이는 일반적인 행렬 표기법과 대조적입니다. 일반적인 행렬 표기법에서는 M_{ij} 가 i 번째 행과 j 번째 열의 요소를 나타냅니다.

그렇다면 왜 스레드 ID 값이 필요할까요? 컴퓨터 셰이더는 일반적으로 입력 데이터 구조를 받아 특정 데이터 구조로 출력합니다. 스레드 ID 값을 이러한 데이터 구조에 대한 인덱스로 활용할 수 있습니다.

```
Texture2D gInputA; Texture2D
gInputB;
RWTexture2D<float4> gOutput; [numthreads(16, 16,
1)]
void CS(int3 dispatchThreadID : SV_DispatchThreadID)
{
    // 디스패치 스레드 ID를 사용하여 출력 및 입력 텍스처에 인덱싱합니다.
    gOutput[dispatchThreadID.xy] =
```

```
gInputA[dispatchThreadID.xy] +
gInputB[dispatchThreadID.xy];
}
```

SV_GroupThreadID는 스레드 로컬 저장소 메모리 (§12.6)에 인덱싱하는 데 유용합니다.

12.5 버퍼 추가 및 소비

다음 구조체로 정의된 입자 버퍼가 있다고 가정합니다:

```
struct Particle
{
    float3 Position; float3
    Velocity; float3
    Acceleration;
};
```

컴퓨터 셰이더에서 입자의 위치 정보를 일정한 가속도와 속도에 따라 업데이트하고자 합니다. 또한 입자 업데이트 순서나 출력 버퍼에 기록되는 순서는 중요하지 않다고 가정합니다. 이러한 시나리오에는 구조체 버퍼를 소비하고 추가하는 방식이 이상적이며, 인덱싱을 신경 쓸 필요가 없다는 편리함을 제공합니다.

```
struct Particle
{
    float3 Position; float3
    Velocity; float3
    Acceleration;
};

float TimeStep = 1.0f / 60.0f;

ConsumeStructuredBuffer<Particle> gInput;
AppendStructuredBuffer<Particle> gOutput; [numthreads(16, 1)]
void CS()
{
    // 입력 버퍼에서 데이터 요소를 소비합니다. Particle p = gInput.Consume();

    p.Velocity += p.Acceleration*TimeStep; p.Position +=
    p.Velocity*TimeStep;

    // 정규화된 벡터를 출력 버퍼에 추가합니다. gOutput.Append(p);
}
```

데이터 요소가 한 번 소비되면 다른 스레드에서 다시 소비될 수 없습니다. 하나의 스레드는 정확히 하나의 데이터 요소만을 소비합니다. 또한, 소비 및 추가되는 요소의 순서는 알려지지 않았음을 다시 한번 강조합니다. 따라서 일반적으로 입력 버퍼의 i번째 요소가 출력 버퍼의 i번째 요소에 쓰여지는 경우는 아닙니다.

 추가 구조화 버퍼는 동적으로 확장되지 않습니다. 추가할 모든 요소를 저장할 수 있을 만큼 충분히 커야 합니다.

The 유일한 단계 필요한 단계 필요한 신청 측면 필요하다 그 만들 때 생성 UAV 무인 항공기 우리는 반드시 지정해야 합니다 D3D11_BUFFER_UAV_FLAG_APPEND 플래그를

D3D11_BUFFER_UAV_FLAG_APPEND 플래그를 지정해야 합니다:

```
D3D11_UNORDERED_ACCESS_VIEW_DESC uavDesc;
uavDesc.Format = DXGI_FORMAT_UNKNOWN; uavDesc.ViewDimension =
D3D11_UAV_DIMENSION_BUFFER; uavDesc.Buffer.FirstElement = 0;
uavDesc.Buffer.Flags = D3D11_BUFFER_UAV_FLAG_APPEND;
uavDesc.Buffer.NumElements = mNumElements;
```

12.6 공유 메모리 및 동기화

스레드 그룹에는 소위 *공유 메모리* 또는 *스레드 로컬 저장소*(SLS)의 일부가 할당됩니다. 이 메모리에 대한 접근은 빠르며 하드웨어 캐시만큼 빠르다고 생각할 수 있습니다. 컴퓨트 셰이더 코드에서 공유 메모리는 다음과 같이 선언됩니다:

```
groupshared float4 gCache[256];

배열 크기는 원하는 대로 설정할 수 있지만, 그룹 공유 메모리의 최대 크기는 32KB입니다. 공유 메모리는
스레드 그룹에 국한되므로 SV_GroupThreadID로 인덱싱됩니다. 예를 들어, 그룹 내 각 스레드가 공유 메모리의 하나의 슬롯에 접근할 수 있도록 할 수 있습니다.
다음 예에서 설명하듯이, 공유 메모리를 과도하게 사용하면 성능 문제가 발생할 수 있습니다[Fung10]. 가정해 보겠습니다.
멀티프로세서는 32kb의 공유 메모리를 지원하며, 컴퓨트 셰이더는 20kb의 공유 메모리가 필요합니다. 이는 20kb + 20kb > 40kb이므로 다른 스레드 그룹을 위한 메모리가 부족하여 단 하나의 스레드 그룹만 멀티프로세서에 배치될 수 있음을 의미합니다[Fung10]. 이는 멀티프로세서가 스레드 그룹 간 전환을 통해 지연 시간을 숨길 수 없게 하여 GPU의 병렬 처리를 제한합니다(12.1절에서 언급된 바와 같이).
= 40kb > 32kb입니다. 이는 멀티프로세서가 스레드 그룹 간 전환을 통해 지연 시간을 숨길 수 없게 하여 GPU의 병렬 처리를 제한합니다(12.1절에서 멀티프로세서당 최소 두 개의 스레드 그룹이 권장됨을 상기하십시오). 따라서 하드웨어가 기술적으로
32kb의 공유 메모리를 지원하지만, 더 적은 양을 사용함으로써 성능 향상을 달성할 수 있습니다.
공유 메모리의 일반적인 용도 중 하나는 텍스처 값을 저장하는 것이다. 블러와 같은 특정 알고리즘은 동일한 텍셀을 여러 번 가져와야 한다. 텍스처 샘플링은 실제로 가장 느린 GPU 작업 중 하나인데, 이는 메모리 대역폭과 메모리 지연 시간이 GPU의 순수 연산 성능만큼 크게 개선되지 않았기 때문이다 [Möller08].
텍스처 샘플링은 메모리 대역폭과 메모리 지연 시간이 GPU의 순수 연산 성능만큼 크게 개선되지 않아 실제로 GPU 작업 중 느린 편에 속합니다[Möller08]. 스레드 그룹은 필요한 모든 텍스처 샘플을 공유 메모리 배열에 미리 로드함으로써 중복 텍스처 가져오기를 피할 수 있습니다. 이후 알고리즘은 공유 메모리 배열에서 텍스처 샘플을 조회하는데, 이는 매우 빠릅니다. 다음의 오류가 있는 코드로 이 전략을 구현한다고 가정해 보겠습니다:
```

```
Texture2D gInput; RWTexture2D<float4> gOutput;

groupshared float4 gCache[256]; [numthreads(256, 1,

1)]
void CS(int3 groupThreadID : SV_GroupThreadID,
        int3 dispatchThreadID : SV_DispatchThreadID)
{
    // 각 스레드는 텍스처를 샘플링하고
    // 값을 공유 메모리에 저장합니다.
    gCache[groupThreadID.x] = gInput[dispatchThreadID.xy];

    // 계산 작업 수행: 공유 메모리 내 요소 접근
    // 다른 스레드가 저장한 요소:

    // 나쁨!!! 좌우 인접 스레드가
    // 텍스처 샘플링을 완료하고 공유 메모리에 저장하지 않았을 수 있음.
    float4 left = gCache[groupThreadID.x - 1]; float4 right =
    gCache[groupThreadID.x + 1];
    ...
}

이 시나리오에서는 스레드 그룹 내 모든 스레드가 동시에 완료된다는 보장이 없기 때문에 문제가 발생합니다. 따라서 스레드가 아직 초기화되지 않은 공유 메모리 요소에 접근할 수 있는데, 이는 해당 요소를 초기화하는 인접 스레드가 아직 완료되지 않았기 때문입니다. 이 문제를 해결하려면 컴퓨트 셰이더가 계속 진행하기 전에 모든 스레드가 텍스처 로딩을 공유 메모리에 완료할 때까지 기다려야 합니다. 이는 동기화 명령을 통해 달성됩니다:
```

```
Texture2D gInput; RWTexture2D<float4> gOutput;

groupshared float4 gCache[256]; [numthreads(256, 1,

1)]
void CS(int3 groupThreadID : SV_GroupThreadID,
        int3 dispatchThreadID : SV_DispatchThreadID)
{
    // 각 스레드는 텍스처를 샘플링하고
    // 값을 공유 메모리에 저장합니다.
    gCache[groupThreadID.x] = gInput[dispatchThreadID.xy];
```

```

// 그룹 내 모든 스레드가 완료될 때까지 대기.
GroupMemoryBarrierWithGroupSync();

// 이제 공유 메모리의 모든 요소를 안전하게 읽을 수 있으며
// 및 계산 작업을 수행해도 안전합니다.

float4 left = gCache[groupThreadID.x - 1]; float4 right =
gCache[groupThreadID.x + 1];
...
}

```

12.7 블러 데모

이 섹션에서는 컴퓨터 셰이더에서 블러 알고리즘을 구현하는 방법을 설명합니다. 먼저 블러링의 수학적 이론을 설명합니다. 그런 다음 데모에서 블러링할 소스 이미지를 생성하는 데 사용하는 렌더-투-텍스처 기법에 대해 논의합니다. 마지막으로 컴퓨터 셰이더 구현 코드를 검토하고 구현을 다소 까다롭게 만드는 세부 사항을 처리하는 방법을 논의합니다.

12.7.1 블러링 이론

사용하는 블러링 알고리즘은 다음과 같이 설명됩니다. 소스 이미지의 각 픽셀 P_{ij} 에 대해, 픽셀 P_{ij} 를 중심으로 한 $m \times n$ 픽셀 행렬의 가중 평균을 계산합니다([그림 12.5](#) 참조); 이 가중 평균이 블러링된 이미지의 i 번째 픽셀이 됩니다. 수학적으로,

$$Blur(P_{ij}) = \sum_{r=-a}^a \sum_{c=-b}^b w_{rc} P_{i+r,j+c} \quad \text{for} \quad \sum_{r=-a}^a \sum_{c=-b}^b w_{rc} = 1$$

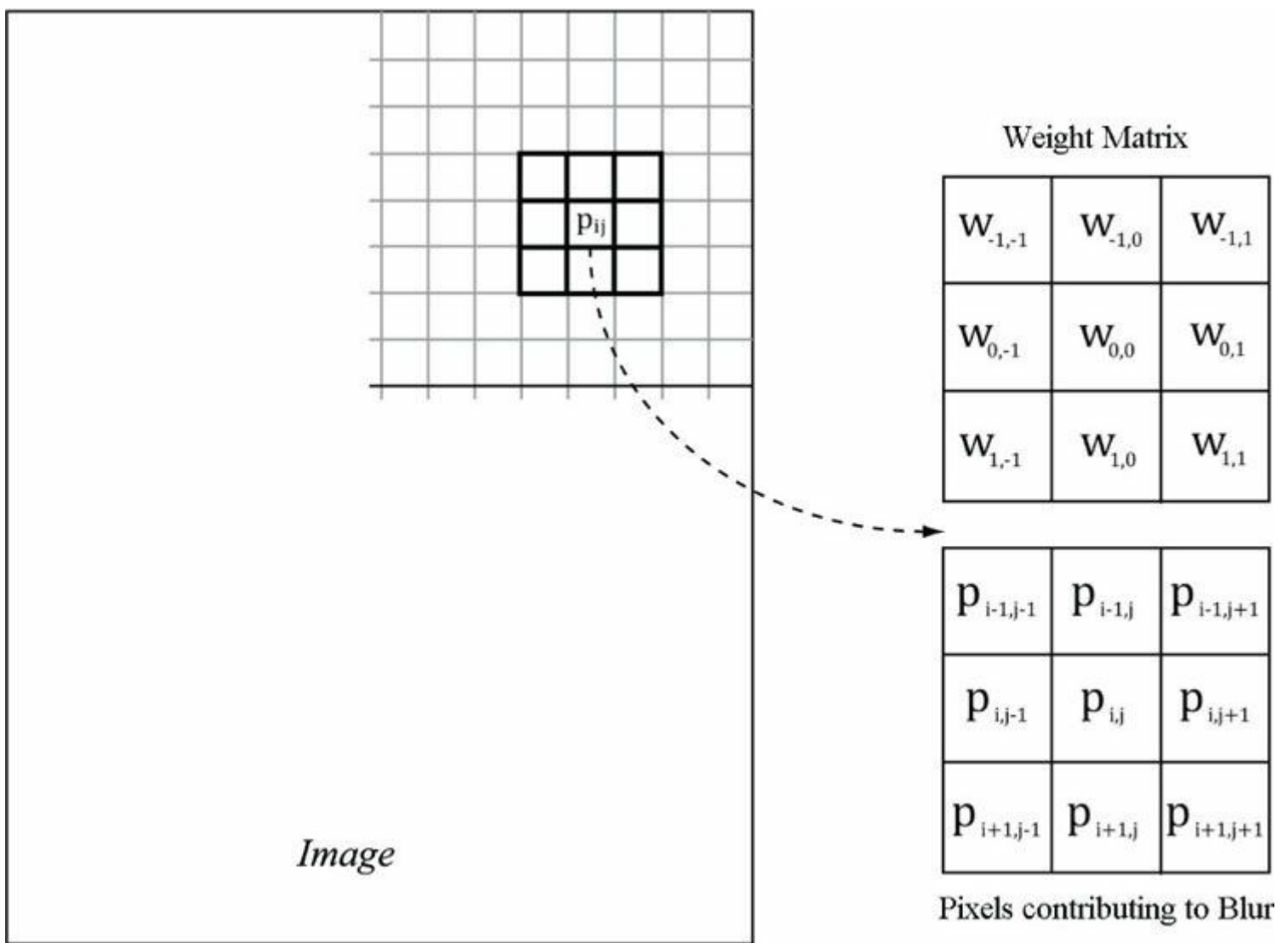


그림 12.5. 픽셀 p_{ij} 를 블러링하기 위해 해당 픽셀을 중심으로 한 $m \times n$ 픽셀 행렬의 가중 평균을 계산합니다. 이 예시에서 행렬은 정사각형 3×3 행렬이며, 블러 반경 $a = b = 1$ 입니다. 중심 가중치 w_{00} 이 픽셀 p_{ij} 와 일치함을 확인하십시오.

여기서 $m = 2a + 1$ 이고 $n = 2b + 1$ 이다. m 과 n 을 홀수로 강제함으로써 $m \times n$ 행렬이 항상 자연스러운 "중심"을 가지도록 보장한다. a 를 수직 블러 반경, b 를 수평 블러 반경이라 부른다. $a = b$ 일 경우 차원을 명시하지 않고 단순히 *블러 반경*이라 칭한다. 이 $m \times n$ 가중치 행렬을 *블러 커널*이라고 부릅니다. 또한 가중치들의 합이 반드시 1이 되어야 함을 유의하십시오. 가중치 합이 1보다 작으면 색상이 제거되어 블러 처리된 이미지가 더 어둡게 보입니다. 가중치 합이 1보다 크면 색상이 추가되어 블러 처리된 이미지가 더 밝게 보입니다.

가중치를 계산하는 방법은 여러 가지가 있으나, 그 합이 1이 되어야 합니다. 많은 이미지 편집 프로그램에서 발견되는 잘 알려진 블러 연산자는 가우시안 블러입니다.

가우시안 블러는 가우시안 함수 $G(x) = \exp\left(-\frac{x^2}{2\sigma^2}\right)$ 로부터 가중치를 얻습니다. 이 함수의 그래프는 다양한 σ 에 대해 그림 12.6에 표시되어 있습니다.

1×5 가우시안 블러(즉, 수평 방향의 1차원 블러)를 수행한다고 가정하고 $\sigma = 1$ 이라 하자. $x = -2, -1, 0, 1, 2$

에 대해 $G(x)$ 를 계산하면 다음과 같습니다:

$$G(-2) = \exp\left(-\frac{(-2)^2}{2}\right) = e^{-2}$$

$$G(-1) = \exp\left(-\frac{(-1)^2}{2}\right) = e^{-\frac{1}{2}}$$

$$G(0) = \exp(0) = 1$$

$$G(1) = \exp\left(-\frac{1^2}{2}\right) = e^{-\frac{1}{2}}$$

$$G(2) = \exp\left(-\frac{2^2}{2}\right) = e^{-2}$$

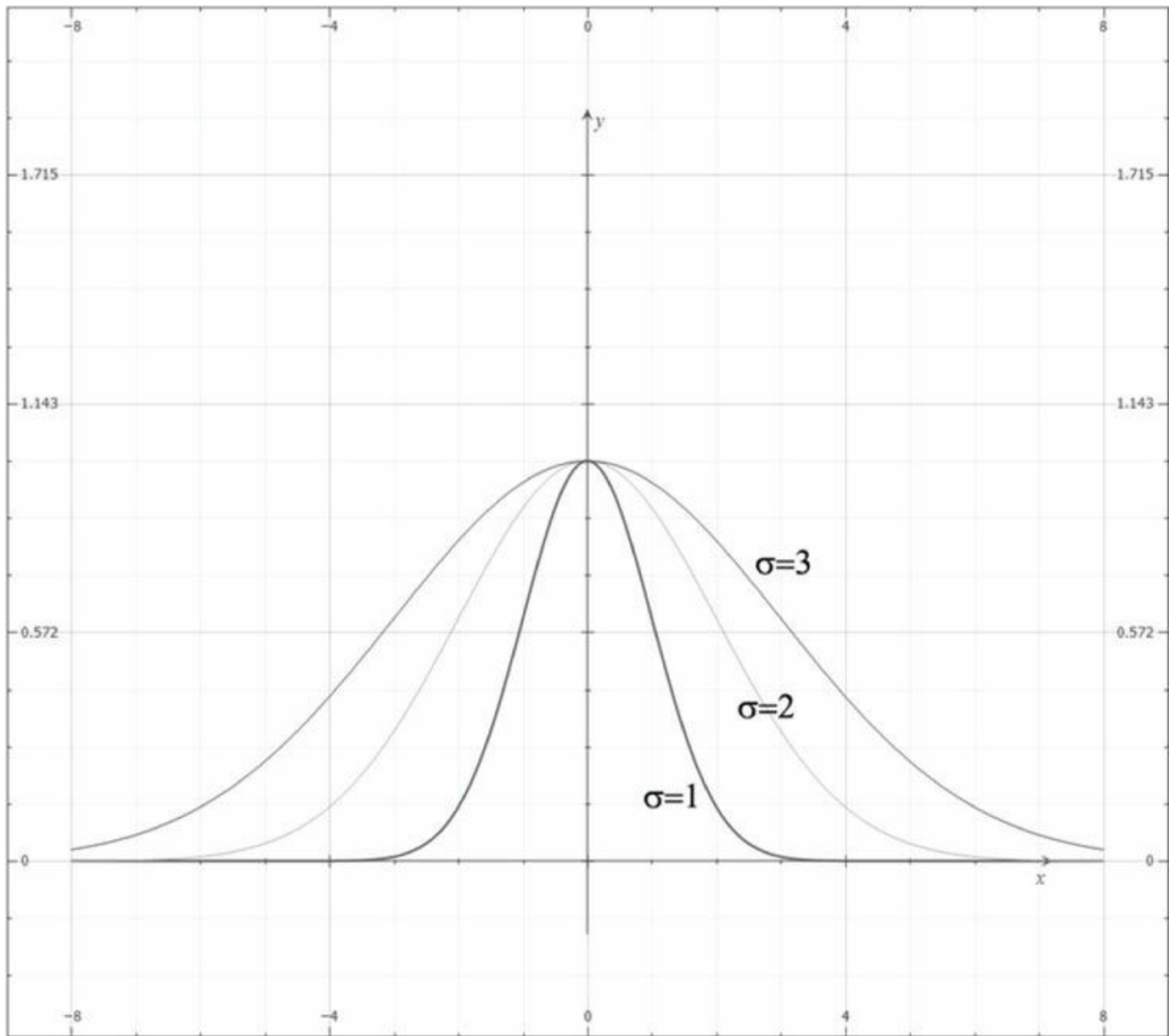


그림 12.6. $\sigma = 1, 2, 3$ 에 대한 $G(x)$ 의 플롯. σ 가 클수록 곡선이 평탄해지고 인접한 점들에 더 큰 가중치가 부여됨을 관찰할 수 있다.

그러나 이 값들은 합이 1이 되지 않으므로 가중치가 아닙니다.

$$\begin{aligned} \sum_{x=-2}^{x=2} G(x) &= G(-2) + G(-1) + G(0) + G(1) + G(2) \\ &= 1 + 2e^{-\frac{1}{2}} + 2e^{-2} \\ &\approx 2.48373 \end{aligned}$$

$\sum_{x=-2}^{x=2} G(x)$ 의 합으로 나누어 이전 방정식을 정규화하면, 가우시안 함수를 기반으로 합이 1이 되는 가중치를 얻을 수 있습니다:

$$\frac{G(-2) + G(-1) + G(0) + G(1) + G(2)}{\sum_{x=-2}^{x=2} G(x)} = 1$$

따라서 가우시안 블러 가중치는 다음과 같습니다:

$$w_{-2} = \frac{G(-2)}{\sum_{x=-2}^{x=2} G(x)} = \frac{e^{-2}}{1 + 2e^{-\frac{1}{2}} + 2e^{-2}} \approx 0.0545$$

$$w_{-1} = \frac{G(-1)}{\sum_{x=-2}^{x=2} G(x)} = \frac{e^{-\frac{1}{2}}}{1 + 2e^{-\frac{1}{2}} + 2e^{-2}} \approx 0.2442$$

$$w_0 = \frac{G(0)}{\sum_{x=-2}^{x=2} G(x)} = \frac{1}{1 + 2e^{-\frac{1}{2}} + 2e^{-2}} \approx 0.4026$$

$$w_1 = \frac{G(1)}{\sum_{x=-2}^{x=2} G(x)} = \frac{e^{-\frac{1}{2}}}{1 + 2e^{-\frac{1}{2}} + 2e^{-2}} \approx 0.2442$$

$$w_2 = \frac{G(2)}{\sum_{x=-2}^{x=2} G(x)} = \frac{e^{-2}}{1 + 2e^{-\frac{1}{2}} + 2e^{-2}} \approx 0.0545$$

가우시안 블러는 *분리*/가능하다고 알려져 있으며, 이는 다음과 같이 두 개의 1차원 블러로 분해될 수 있음을 의미합니다.

1. 입력 이미지 I 에 1차원 수평 블러를 적용합니다: $I_H = \text{Blur}_H(I)$.
2. 이전 단계의 출력에 1차원 수직 블러를 적용합니다: $\text{Blur}(I) = \text{Blur}_V(I_H)$. 더 간결하게 표현하면 다음과 같습니다:

$$\text{Blur}(I) = \text{Blur}_V(\text{Blur}_H(I))$$

블러 커널이 9×9 행렬이라고 가정하면, 2차원 블러 처리에 총 81개의 샘플이 필요했습니다. 블러를 두 개의 1차원 블러로 분리함으로써, 우리는 단지 9 + 9 = 18개의 샘플만 필요로 합니다. 일반적으로 우리는 텍스처를 블러링할 것입니다. 이 장에서 언급했듯이, 텍스처 샘플을 가져오는 것은 비용이 많이 드므로, 블러를 분리하여 텍스처 샘플을 줄이는 것은 환영할 만한 개선입니다. 블러가 분리 불가능한 경우(일부 블러 연산자는 분리 불가능함)에도 최종 이미지가 충분히 정확해 보인다면 성능 향상을 위해 단순화를 적용하는 경우가 흔합니다.

12.7.2 텍스처 렌더링

지금까지 프로그램에서는 백 버퍼에 렌더링해 왔습니다. 그런데 백 버퍼란 무엇일까요? **D3DApp** 코드를 살펴보면 백 버퍼는 스왑 체인 내의 텍스처에 불과합니다:

```
ID3D11Texture2D* back Buffer;
HR(mSwapChain->GetBuffer(0, __uuidof(ID3D11Texture2D),
    reinterpret_cast<void*>(>backBuffer)));
HR(md3dDevice->CreateRenderTargetView(back Buffer, 0,
    >mRenderTargetView));
```

Direct3D에 백 버퍼의 렌더 타겟 뷰를 렌더링 파이프라인의 OM 단계에 바인딩하여 백 버퍼로 렌더링하도록 지시합니다:

```
md3dImmediateContext->OMSetRenderTargets(1,
    >mRenderTargetView, mDepthStencilView);
```

백 버퍼의 내용은 백 버퍼가

IDXGISwapChain::Present 메서드를 통해 백 버퍼가 제시될 때 화면에 표시됩니다.

이 코드를 고려해 보면, 다른 텍스처를 생성하고, 그 텍스처에 대한 렌더 타겟 뷰를 생성한 다음

이를 렌더링 파이프라인의 OM 단계에 바인딩하는 것을 막을 수 있는 것은 없습니다. 따라서 백 버퍼 대신 이 다른 "오프스크린" 텍스처(다른 카메라를 사용할 수도 있음)에 렌더링하게 됩니다. 이 기법은 *렌더-투-오프스크린-텍스처(render-to-off-screen-texture)* 또는 간단히 *렌더-투-텍스처(render-to-texture)*로 알려져 있습니다. 유일한 차이점은 이 텍스처가 백 버퍼가 아니기 때문에 프레젠테이션 중에 화면에 표시되지 않는다는 점입니다.

따라서 렌더 투 텍스처는 화 면에 표시되지 않기 때문에 처음에는 쓸모없어 보일 수 있습니다. 그러나

텍스처 렌더링을 수행한 후, 백 버퍼를 OM 스테이지에 다시 바인딩하고 백 버퍼에 지오메트리 그리기를 재개할 수 있습니다. 텍스처 렌더링 기간 동안 생성한 텍스처로 지오메트리에 텍스처를 적용할 수 있습니다. 이 전략은 다양한 특수 효과를 구현하는 데 사용됩니다. 예를 들어, 새의 눈높이 시점에서 장면을 텍스처에 렌더링할 수 있습니다. 그런 다음 백 버퍼에 그리기 시, 화면 오른쪽 하단 모서리에 새의 눈높이 시점 텍스처를 적용한 사각형을 그려 레이어 시스템을 시뮬레이션할 수 있습니다([그림 12.7](#) 참조). 다른 렌더링-투-텍스처 기법으로는 다음과 같은 것들이 있습니다:



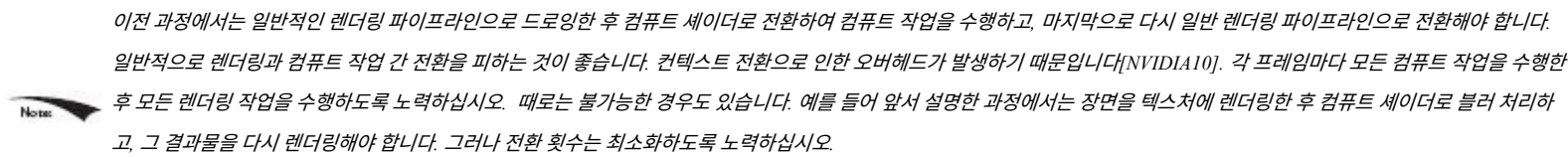
그림 12.7. 플레이어 위쪽에 새의 눈높이 시점 카메라를 배치하여 장면을 오프스크린 텍스처로 렌더링합니다. 플레이어 시점에서 장면을 백 버퍼에 그릴 때, 화면 오른쪽 하단 모서리의 사각형 영역에 텍스처를 매핑하여 레이어 지도를 표시합니다.

1. 새도 매핑
2. 화면 공간 앰비언트 오클루전
3. 큐브 맵을 활용한 동적 반사

이 데모에서는 일반적인 데모 장면인 땅과 파도를 오프스크린 텍스처에 렌더링합니다. 이 텍스처는 컴퓨터 셰이더에서 실행되는 블러링 알고리즘의 입력으로 사용됩니다. 텍스처가 블러링된 후, 블러링된 텍스처를 적용한 풀스크린 쿼드를 백 버퍼에 그려 블러링 결과를 확인하여 구현된 블러 효과를 테스트합니다. 단계는 다음과 같습니다:

1. 육지와 파도 장면을 오프스크린 텍스처에 렌더링합니다.
2. 컴퓨터 셰이더 프로그램을 사용하여 오프스크린 텍스처를 블러 처리합니다.

3. 백 버퍼를 렌더 타깃으로 복원하고, 블러 처리된 텍스처를 적용한 폴스크린 쿼드를 그림니다.



12.7.3 블러 구현 개요

우리는 블러가 분리 가능하다고 가정하므로, 블러를 수평 블러와 수직 블러라는 두 개의 1차원 블러 계산으로 분해합니다. 이를 구현하려면 두 텍스처 버퍼에 동시에 읽고 쓸 수 있어야 하므로, 두 텍스처 모두에 대한 SRV(스트림 리소스 버퍼)와 UAV(유니버설 어드레스 버퍼)가 필요합니다. 한 텍스처를 **A**, 다른 텍스처를 **B**라고 부르겠습니다. 블러링 알고리즘은 다음과 같이 진행됩니다:

- 1. SRV를 컴퓨터 셰이더의 입력으로 **A**에 바인딩합니다(이것은 수평 블러링될 입력 이미지입니다).
- 2. 컴퓨터 셰이더의 출력으로 UAV를 **B**에 바인딩합니다(이것은 블러 처리된 결과를 저장할 출력 이미지입니다).
- 3. 수평블러 연산을 수행하기 위해 스레드 그룹을 디스패치합니다. 이후 텍스처 **B**에는 수평 블러 처리된 결과 $Blur_H(I)$ 가 저장됩니다. 여기서 I 는 블러 처리 대상 이미지입니다.
- 4. SRV를 컴퓨터 셰이더의 입력으로 **B**에 바인딩합니다(이것은 다음에 수직 블러 처리가 적용될 수평 블러 처리된 이미지입니다).
- 5. 컴퓨터 셰이더의 출력으로 UAV를 **A**에 바인딩합니다(이것은 최종 블러 처리 결과를 저장할 출력 이미지입니다).
- 6. 스레드 그룹을 디스패치하여 수직블러 연산을 수행합니다. 이후 텍스처 **A**에는 최종 블러 처리 결과 $Blur(I)$ 가 저장되며, 여기서 I 는 블러 처리 대상 이미지입니다.

이 로직은 분리 가능한 블러 공식 $Blur(I) = Blur_V(Blur_H(I))$ 를 구현합니다. **텍스처 A**와 **텍스처 B** 모두 컴퓨터 셰이더에 입력 및 출력으로 사용되지만 동시에 사용되지는 않는다는 점을 유의 하십시오. (리소스를 동시에 입력과 출력으로 바인딩하는 것은 Direct3D 오류입니다.) 수평 및 수직 블러 패스를 결합하면 하나의 완전한 블러 패스가 구성됩니다. 결과 이미지는 추가 블러 패스를 수행하여 더 흐리게 처리할 수 있습니다. 원하는 수준까지 흐려질 때까지 이미지를 반복적으로 블러 처리할 수 있습니다.

본 구현에서는 이전 단계에서 입력 텍스처(블러 처리 대상 텍스처)를 **텍스처 A**로 사용합니다. 즉, *인플레이스* 블러 처리를 수행합니다. 이는 결국 입력 텍스처를 블러 처리한다는 의미입니다. 때로는 원하지 않는 경우가 있습니다. 즉, 원본의 블러 처리되지 않은 이미지와 블러 처리된 이미지 모두를 원할 수 있습니다. 이 경우 입력 텍스처의 복사본을 만들어 복사본에 블러를 적용해야 합니다.

씬을 렌더링하는 텍스처는 창 클라이언트 영역과 동일한 해상도를 가집니다. 따라서 오프셋을 재구성해야 합니다. 화면 텍스처와 블러 알고리즘에 사용되는 두 번째 텍스처 버퍼 **B**를 재설정합니다. 이는 **OnResize** 메서드에서 수행됩니다:

```
void BlurApp::OnResize()
{
    D3DApp::OnResize();

    XMMATRIX P = XMMatrixPerspectiveFovLH(0.25f*MathHelper::Pi, AspectRatio(), 1.0f, 1000.0f);
    XMStoreFloat4x4(>mProj, P);

    //
    // 클라이언트 영역 크기에 의존하는 리소스를 재생성합니다.
    //

    // 오프스크린 텍스처 크기 조정 및 뷰 재구성:
    // 렌더 타겟 뷰, SRV, UAV 빌드 오프스크린 뷰:
    BuildOffscreenViews();

    // 텍스처 B 크기 조정 및 뷰 재구성: SRV, UAV. mBlur.Init(md3dDevice,
    mClientWidth, mClientHeight, DXGI_FORMAT_R8G8B8A8_UNORM);
}
```

이 장에서 텍스처, SRV, UAV 생성 예제를 이미 살펴보았으므로, 해당 메서드 구현은 생략하겠습니다. 한 가지 주의할 점은 오프스크린 텍스처에는 추가적인 **D3D11_BIND_RENDER_TARGET** 바인딩 플래그가 적용된다는 것입니다:

```
D3D11_BIND_RENDER_TARGET |
D3D11_BIND_SHADER_RESOURCE |
```

이는 오프스크린 텍스처로 렌더링하기 때문에 해당 텍스처에 대한 렌더 타겟 뷰를 생성해야 하기 때문입니다. **mBlur** 변수는 우리가 만든 BlurFilter 헬퍼 클래스의 인스턴스입니다. 이 클래스는 텍스처 B에 대한 뷰를 캡슐화하며, 컴퓨터 셰이더에서 실제 블러 연산을 시작하는 메서드를 제공합니다. 이 메서드의 구현은 잠시 후에 살펴보겠습니다.

블러링은 비용이 많이 드는 작업이며 소요 시간은 블러링 대상 이미지 크기에 비례합니다. 종종 장면을 오프스크린 텍스처로 렌더링할 때 오프스크린 텍스처는 백 버퍼 크기의 1/4로 설정됩니다. 예를 들어 백 버퍼가 800×600 이면 오프스크린 텍스처는 400×300 이 됩니다. 이렇게 하면 오프스크린 텍스처에 그리는 속도가 빨라집니다(채울 픽셀 수가 적어짐). 또한 블러 처리 속도도 빨라집니다(블러 처리할 픽셀 수가 적어짐). 게다가 텍스처가 화면 해상도의 1/4에서 전체 화면 해상도로 확대될 때 확대 텍스처 필터에 의해 추가적인 블러링이 수행됩니다.



이미지의 너비가 w 이고 높이 h 라고 가정합니다. 다음 섹션에서 컴퓨터 셰이더를 살펴볼 때 보게 되겠지만, 수평 1차원 블러의 경우 스레드 그룹은 256개의 스레드로 구성된 수평 선분이며, 각 스레드는 이미지 내 하나의 픽셀을 블러링하는 역할을 담당합니다. 따라서 이미지 내 각 픽셀이 블러 처리되려면 x 방향으로 $\text{ceil}\left(\frac{w}{256}\right)$ 개의 스레드 그룹과 y 방향으로 h 개의 스레드 그룹을 디스패치해야 합니다. w 가 256으로 정확히 나누어지지 않으면 마지막 수평 스레드 그룹에 불필요한 스레드가 남게 됩니다([그림 12.8](#) 참조). 스레드 그룹 크기가 고정되어 있으므로 이 문제를 해결할 방법은 사실상 없습니다. 셰이더 코드에서 클램핑 검사로 범위를 벗어난 값을 처리합니다.

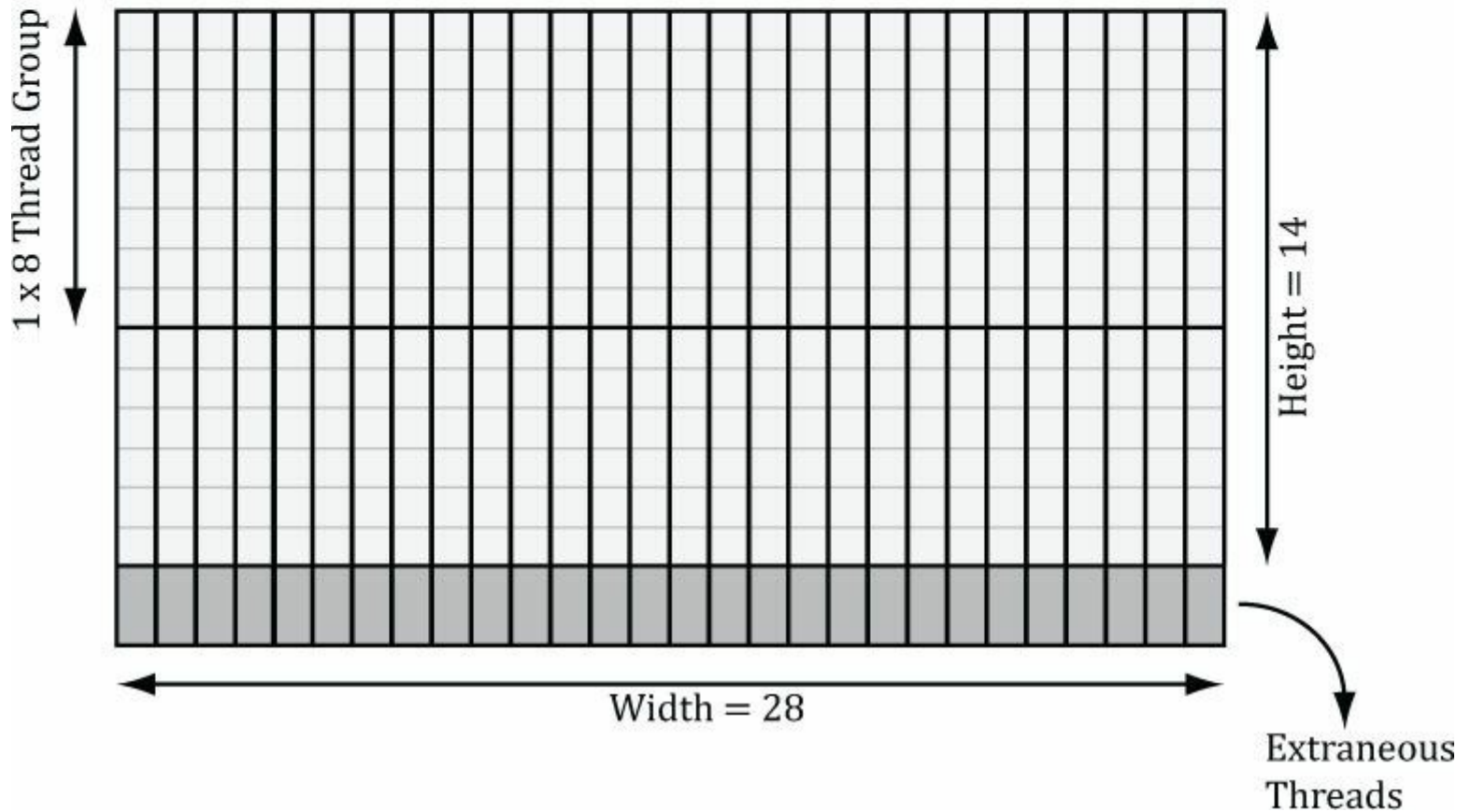
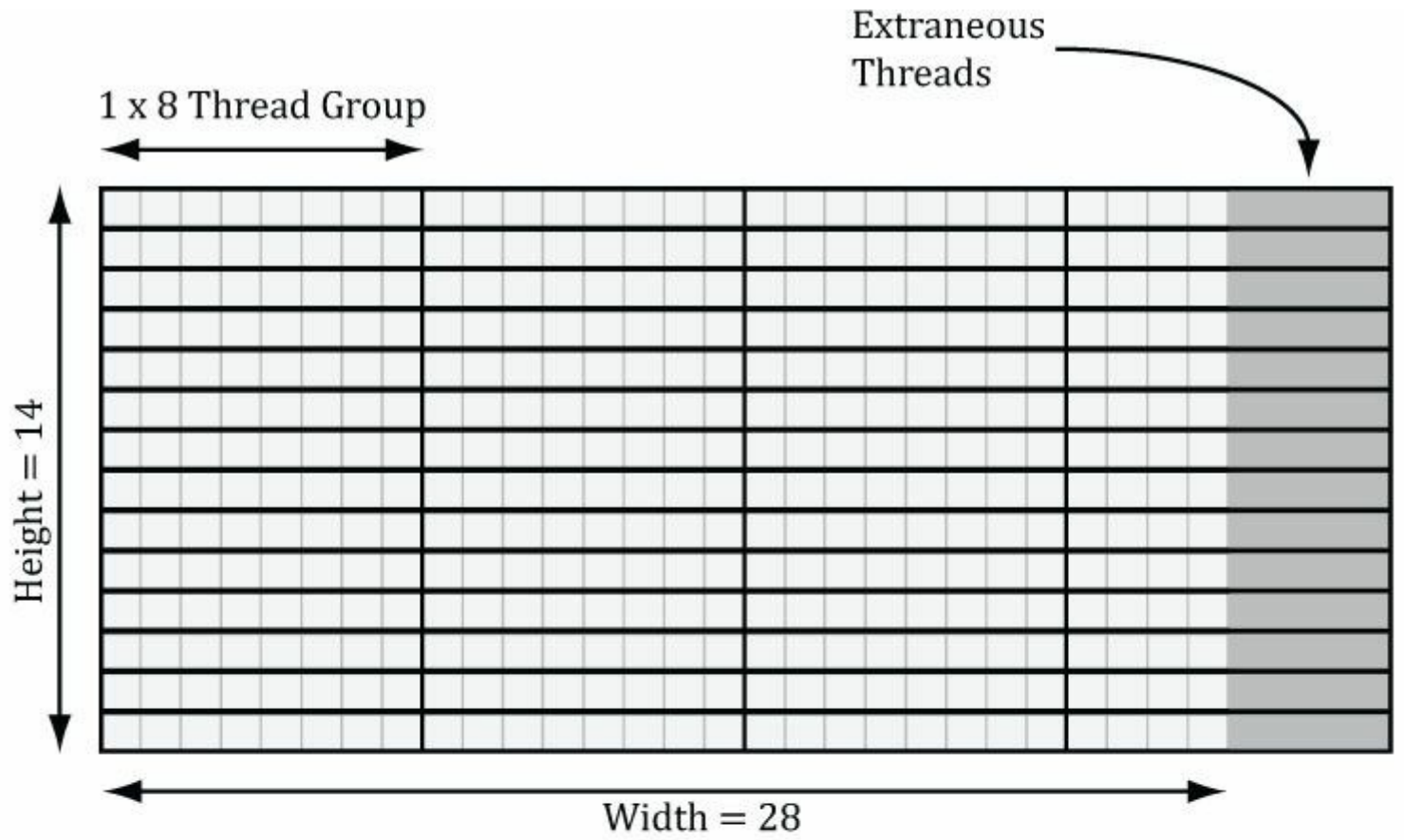


그림 12.8. 가로 스레드 그룹이 8×1 이고 세로 스레드 그룹이 1×8 인 28×14 텍스처를 고려하자($X \times Y$ 형식). 수평 패스의 경우, 모든 픽셀을 커버하기 위해 x-방향으로 $\text{ceil}\left(\frac{W}{8}\right) = \text{ceil}\left(\frac{28}{8}\right) = 4$ 개의 스레드 그룹과 y-방향으로 14개의 스레드 그룹을 디스패치해야 합니다. 28이 8의 배수가 아니기 때문에, 가장 오른쪽 스레드 그룹에서는 작업을 수행하지 않는 불필요한 스레드가 발생합니다.

수직 패스에서 모든 픽셀을 커버하기 위해 ceil 함수를 호출해야 합니다

$$\left(\frac{h}{8}\right) = \text{ceil}\left(\frac{14}{8}\right) = y \text{ 방향으로 2개의 스레드 그룹과 } x \text{ 방향으로 28개의 스레드 그룹을 분배해야 합니다.}$$

방향으로 스레드 그룹을 배치해야 합니다. 14는 8의 배수가 아니므로, 최하위 스레드 그룹에서는 아무 작업도 수행하지 않는 불필요한 스레드가 발생합니다. 스레드 그룹 크기가 256인 더 큰 텍스처에도 동일한 개념이 적용됩니다.

수직 1차원 블러의 경우도 상황은 유사합니다. 다시 말해, 스레드 그룹은 256개의 스레드로 구성된 수직 선분이며, 각 스레드는 이미지 내 하나의 픽셀을 블러링하는 역할을 담당합니다. 따라서 y-방향으로 $\text{ceil}\left(\frac{h}{256}\right)$ 개의 스레드 그룹을, w-방향으로 w개의 스레드 그룹을 분배해야 합니다. 이는 이미지의 각 픽셀이 블러링되도록 하기 위함입니다.

x-방향으로 thread 그룹을 디스패치해야 이미지의 각 픽셀이 블러 처리됩니다.

아래 코드는 각 방향별로 배포할 스레드 그룹 수를 계산하고, 실제 블러 연산을 시작합니다.

컴퓨터 셰이더:

```
void BlurFilter::BlurInPlace(ID3D11DeviceContext* dc,
    ID3D11ShaderResourceView* inputSRV, ID3D11UnorderedAccessView*
    inputUAV,
    int blurCount)
{
    //
    // 오프스크린 텍스처를 블러 처리하기 위해 컴퓨터 셰이더 실행.
    //

    for(int i = 0; i < blurCount; ++i)
    {
        // 수평 블러 패스. D3DX11_TECHNIQUE_DESC techDesc; Effects::BlurFX-
        >HorzBlurTech->GetDesc(>techDesc); for(UINT p = 0; p < techDesc.Passes;
        ++p)
        {
            Effects::BlurFX->SetInputMap(inputSRV);
            Effects::BlurFX->SetOutputMap(mBlurredOutputTexUAV); Effects::BlurFX->HorzBlurTech-
            >GetPassByIndex(p)->Apply(0, dc);

            // 한 행의 픽셀을 커버하기 위해 몇 개의 그룹을 분배해야 하는가

            // 각 그룹이 256픽셀을 커버하는 경우

            // (256은 컴퓨터 셰이더에서 정의됨).
            UINT numGroupsX = (UINT)ceilf(mWidth / 256.0f); dc-
            >Dispatch(numGroupsX, mHeight, 1);
        }

        // 정리 차원에서 입력 텍스처를 컴퓨터 셰이더에서 분리합니다. ID3D11ShaderResourceView*
        nullSRV[1] = { 0 };
        dc->CSSetShaderResources(0, 1, nullSRV);

        // 컴퓨터 셰이더에서 출력을 언바인드합니다 (다음 패스에서 이 출력을 입력으로 사용할 예정입니다).

        // 출력을 입력으로 사용할 예정, 리소스는

        // 동시에 출력과 입력이 될 수 없습니다. ID3D11UnorderedAccessView*
        nullUAV[1] = { 0 };
        dc->CSSetUnorderedAccessViews(0, 1, nullUAV, 0);

        // VERTICAL 블러 패스.
        Effects::BlurFX->VertBlurTech->GetDesc(>techDesc); for(UINT p = 0; p <
        techDesc.Passes; ++p)
        {
            Effects::BlurFX->SetInputMap(mBlurredOutputTexSRV); Effects::BlurFX-
            >SetOutputMap(inputUAV);
            Effects::BlurFX->VertBlurTech->GetPassByIndex(p)->Apply(0, dc);

            // 하나의 픽셀 열을 커버하기 위해 몇 개의 그룹을 분배해야 하는가

            // 각 그룹이 256픽셀을 커버하는 경우

            // (256은 컴퓨터 셰이더에서 정의됨).
            UINT numGroupsY = (UINT)ceilf(mHeight / 256.0f); dc-
            >Dispatch(mWidth, numGroupsY, 1);
        }
        dc->CSSetShaderResources(0, 1, nullSRV);
        dc->CSSetUnorderedAccessViews(0, 1, nullUAV, 0);
    }
}
```



```

}

// 컴퓨터 셰이더 비활성화. dc-
>CSSetShader(0, 0, 0);

}

```

12.7.4 컴퓨터 셰이더 프로그램

이 섹션에서는 실제로 블러링을 수행하는 컴퓨터 셰이더 프로그램을 살펴봅니다. 수평 블러링 사례만 논의할 것입니다. 수직 블러링 사례는 유사하지만 상황이 반전됩니다.

이전 섹션에서 언급했듯이, 스레드 그룹은 256개 스레드로 구성된 수평 선분이며 각 스레드는 이미지의 한 픽셀을 블러링합니다. 비효율적인 첫 번째 접근 방식은 블러 알고리즘을 직접 구현하는 것입니다. 즉, 각 스레드는 단순히 처리 중인 픽셀을 중심으로 한 픽셀 행렬(1차원 수평 패시드로 행렬)의 가중 평균을 수행합니다. 이 접근 방식의 문제는 동일한 텍셀을 여러 번 가져와야 한다는 점입니다([그림 12.10](#) 참조).

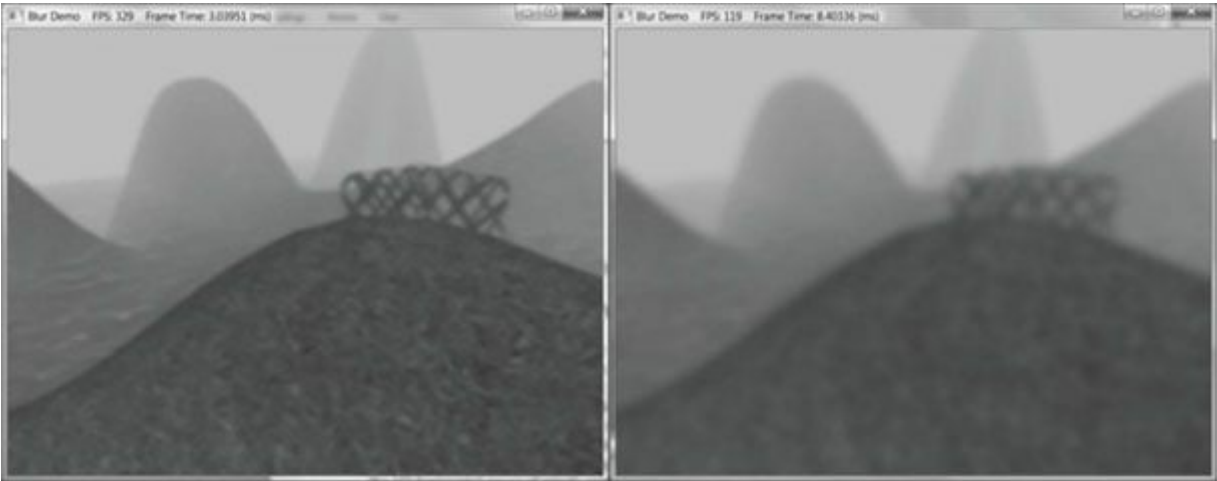


그림 12.9. (왼쪽) 이미지를 2회 블러 처리한 "Blur" 데모의 스크린샷. (오른쪽) 이미지를 8회 블러 처리한 "Blur" 데모의 스크린샷.

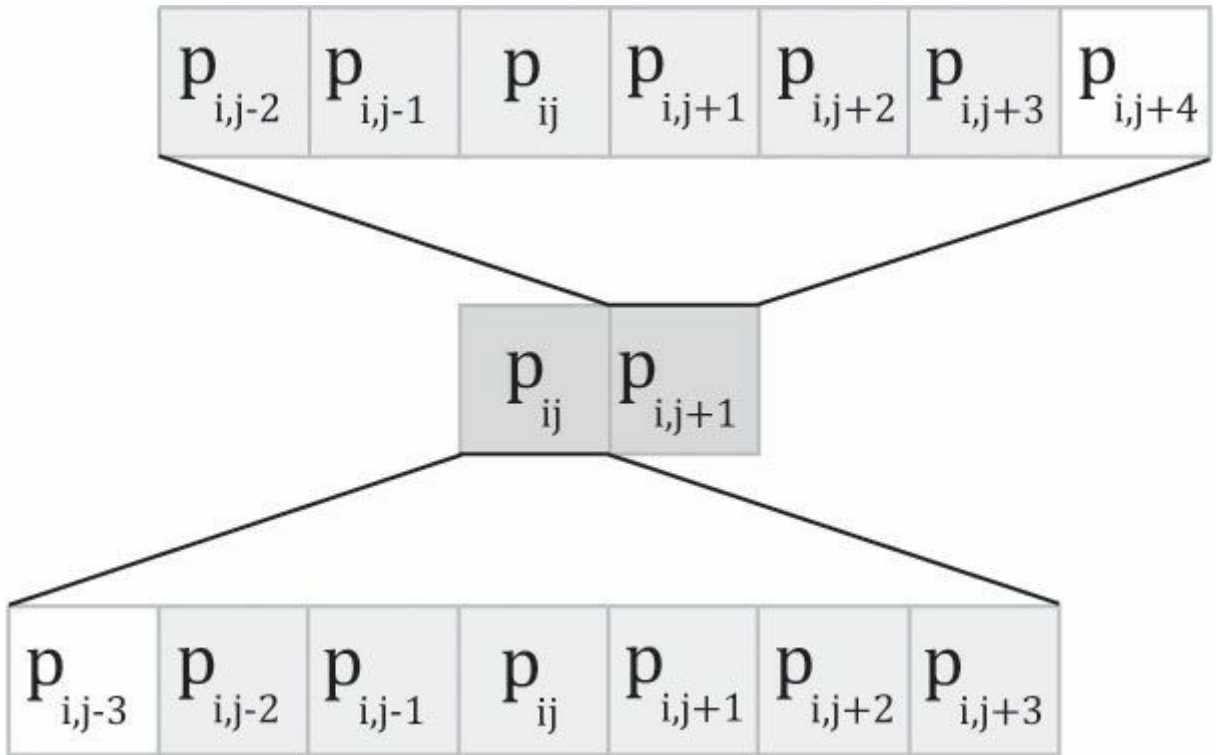


그림 12.10. 입력 이미지에서 인접한 두 픽셀만을 고려하고, 블러 커널이 1×7 이라고 가정하자. 8개의 고유 픽셀 중 6개가 두 번 샘플링됨을 관찰할 수 있다—각 픽셀마다 한 번씩이다.

§12.6에 설명된 전략을 따라 공유 메모리를 활용하여 최적화할 수 있습니다. 각 스레드는 텍셀 값을 읽어 공유 메모리에 저장합니다. 모든 스레드가 텍셀 값을 공유 메모리에 읽어들이고 후, 스레드는 블러 연산을 수행할 수 있지만 이때 텍셀을 공유 메모리에서 읽어들이는 접근 속도가 빠릅니다. 여기서 유일한 까다로운 점은

이것은 스레드 그룹 $n = 256$ 개 스레드가 블러를 수행하려면 $n + 2R$ 텍셀이 필요하다는 점입니다. 여기서 R 은 블러 반경입니다(그림 12.11). 해결책은 간단합니다. 공유 메모리에 $n + 2R$ 개의 요소를 할당하고, $2R$ 개의 스레드가 두 텍셀 값을 조회하도록 합니다. 유일한 이 방법의 까다로운 점은 공유 메모리에 인덱싱할 때 추가적인 관리가 필요하다는 것입니다. 공유 메모리의 i 번째 요소에 대응하는 i 번째 그룹 스레드 ID가 더 이상 존재하지 않기 때문입니다. 그림 12.12는 $R = 4$ 일 때 스레드와 공유 메모리의 매핑을 보여줍니다.

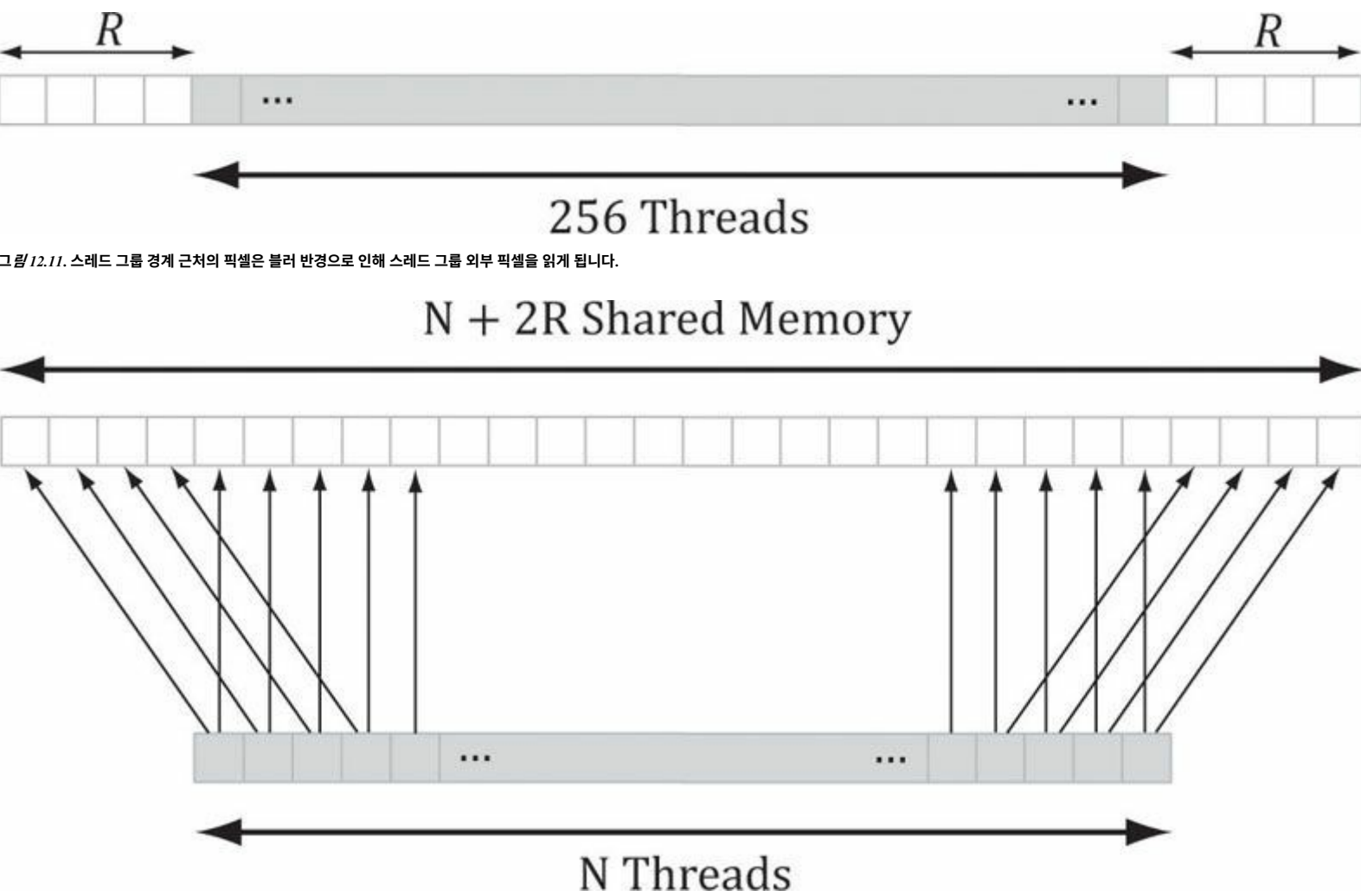


그림 12.12. 이 예시에서 $R = 4$ 입니다. 가장 왼쪽의 4개 스레드는 각각 두 개의 텍셀 값을 읽고 공유 메모리에 저장합니다. 가장 오른쪽의 4개 스레드도 각각 두 개의 텍셀 값을 읽고 공유 메모리에 저장합니다. 나머지 스레드들은 각각 하나의 텍셀 값만 읽고 공유 메모리에 저장합니다. 이렇게 하면 블러 반경 R 로 N 개의 픽셀을 블러 처리하는 데 필요한 모든 텍셀 값을 얻을 수 있습니다.

마지막으로 논의할 상황은 그림 12.13과 같이 가장 왼쪽 스레드 그룹과 가장 오른쪽 스레드 그룹이 입력 이미지의 범위를 벗어난 인덱싱을 수행할 수 있는 경우입니다. 범위를 벗어난 인덱스로부터 읽는 것은 불법이 아닙니다—0을 반환하도록 정의되어 있습니다(범위를 벗어난 인덱스로 쓰기는 오류 없이 실행됩니다). op). 그러나 경계를 벗어났을 때 0을 읽는 것은 원하지 않습니다. 이는 경계 부근의 블러에 0 색상(즉, 검정색)이 포함된다는 의미이기 때문입니다. 대신 클램프 텍스처 주소 모드와 유사한 방식을 구현하고자 합니다. 즉, 경계를 벗어난 값을 읽을 경우 경계 텍셀과 동일한 값을 반환하도록 하는 것입니다. 이는 인덱스를 클램핑하여 구현할 수 있습니다:

Leftmost threads of leftmost thread block will sample outside the image due to blur radius.

Extraneous threads are still executed and write to shared memory.

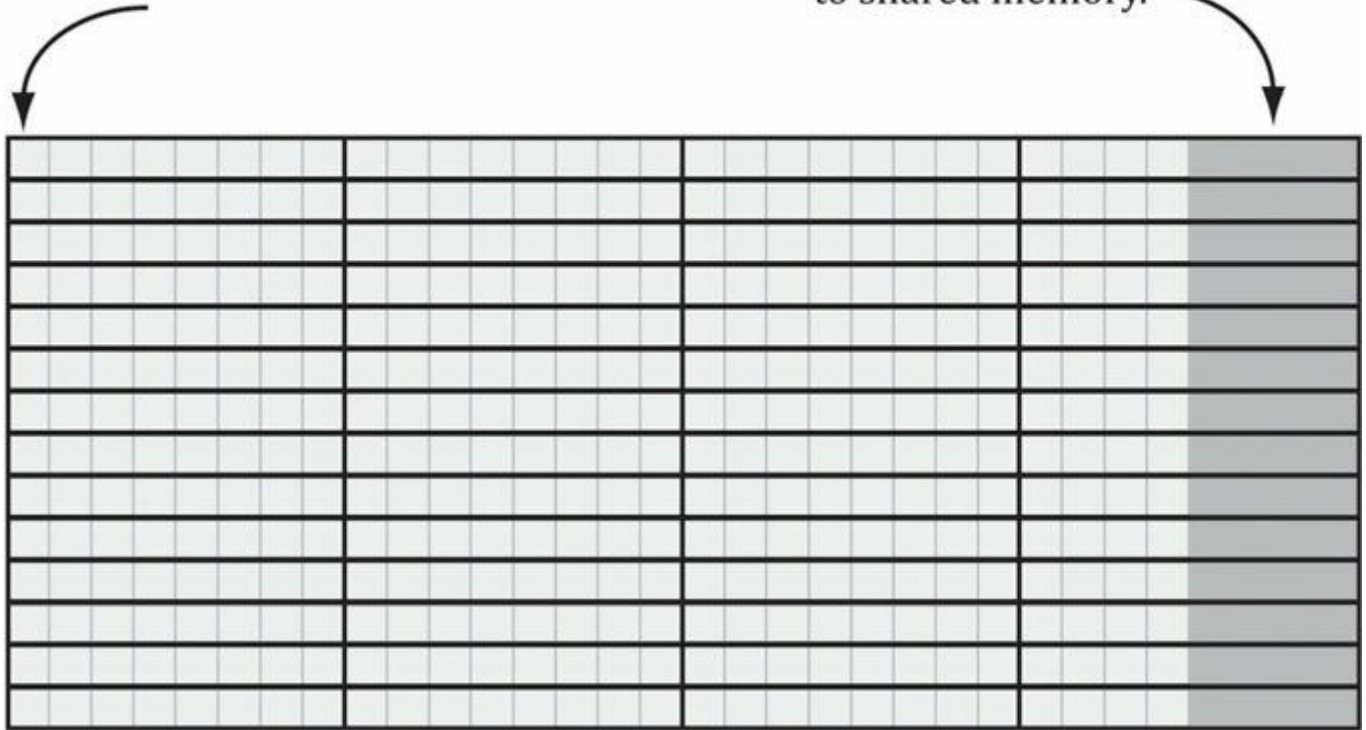


그림 12.13. 이미지 경계를 벗어난 영역을 읽을 수 있는 상황들.

```
// 왼쪽 이미지 경계에서 발생하는 범위를 벗어난 샘플을 클램프합니다. int x =
max(dispatchThreadID.x - gBlurRadius, 0); gCache[groupThreadID.x] = gInput[int2(x,
dispatchThreadID.y)];
```

```
// 이미지 오른쪽 경계에서 발생하는 범위를 벗어난 샘플을 클램프합니다. int x =
min(dispatchThreadID.x + gBlurRadius, gInput.Length.x-1);
gCache[groupThreadID.x+2*gBlurRadius] =
gInput[int2(x, dispatchThreadID.y)];
```

```
// 이미지 경계에서 발생하는 범위를 벗어난 샘플을 클램프합니다.
gCache[groupThreadID.x+gBlurRadius] =
gInput[min(dispatchThreadID.xy, gInput.Length.xy-1)];
```

전체 셰이더 코드는 아래와 같습니다.

```
//=====
// Blur.fx by Frank Luna (C) 2011 All Rights Reserved.
//
// 분리 가능한 블러를 수행하며 블러 반경은 5입니다.
//=====

cbuffer cbSettings
{
    float gWeights[11] =
    {
        0.05f, 0.05f, 0.1f, 0.1f, 0.1f, 0.2f, 0.1f, 0.1f, 0.1f, 0.05f, 0.05f,
    };
};

cbuffer cbFixed
{
    static const int gBlurRadius = 5;
};
```

```
Texture2D gInput; RWTexture2D<float4> gOutput;
```

```
#define N 256
```

```
#define CacheSize (N + 2*gBlurRadius) groupshared float4
```

```
gCache[CacheSize];
```

```
[numthreads(N, 1, 1)]
```

```
void HorzBlurCS(int3 groupThreadID : SV_GroupThreadID, int3 dispatchThreadID : SV_DispatchThreadID)
```

```
{  
    //  
    // 대역폭을 줄이기 위해 로컬 스레드 저장소를 채웁니다.  
    // N 픽셀을 블러링하려면, 블러 반경으로 인해 N + 2*BlurRadius 픽셀을 로드해야 함  
    // 블러 반경 때문입니다.  
    //  
    // 이 스레드 그룹은 N개의 스레드를 실행합니다. 추가로 필요한 2*BlurRadius  
    // 픽셀을 얻으려면, 2*BlurRadius 스레드가 추가 픽셀을 샘플링하도록 합니다.  
    if(groupThreadID.x < gBlurRadius)  
    {  
        // 이미지 경계에서 발생하는 범위를 벗어난 샘플을 제한합니다. int x =  
        max(dispatchThreadID.x - gBlurRadius, 0);  
        gCache[groupThreadID.x] = gInput[int2(x, dispatchThreadID.y)];  
    }  
    if(groupThreadID.x >= N-gBlurRadius)  
    {  
        // 이미지 경계에서 발생하는 범위를 벗어난 샘플을 클램프합니다.  
        int x = min(dispatchThreadID.x + gBlurRadius, gInput.Length.x-1);  
        gCache[groupThreadID.x+2*gBlurRadius] =  
            gInput[int2(x, dispatchThreadID.y)];  
    }  
  
    // 이미지 경계에서 발생하는 범위를 벗어난 샘플을 클램프합니다.  
    gCache[groupThreadID.x+gBlurRadius] =  
        gInput[min(dispatchThreadID.xy, gInput.Length.xy-1)];  
  
    // 모든 스레드가 완료될 때까지 대기.  
    GroupMemoryBarrierWithGroupSync();  
  
    //  
    // 이제 각 픽셀을 블러 처리합니다.  
    //  
  
    float4 blurColor = float4(0, 0, 0, 0); [unroll]  
    for(int i = -gBlurRadius; i <= gBlurRadius; ++i)  
    {  
        int k = groupThreadID.x + gBlurRadius + i;  
  
        blurColor += gWeights[i+gBlurRadius]*gCache[k];  
    }  
  
    gOutput[dispatchThreadID.xy] = blurColor;  
}
```

```
[numthreads(1, N, 1)]
```

```
void VertBlurCS(int3 groupThreadID : SV_GroupThreadID, int3 dispatchThreadID : SV_DispatchThreadID)
```

```
{  
    //  
    // 대역폭을 줄이기 위해 로컬 스레드 저장소를 채웁니다.  
    // N 픽셀을 블러 처리하려면 N + 2*BlurRadius 픽셀을 로드해야 합니다
```

```

// 블러 반경 때문입니다.
//

// 이 스레드 그룹은 N개의 스레드를 실행합니다. 추가로 필요한 2*BlurRadius
// 픽셀을 얻으려면, 2*BlurRadius 스레드가 추가 픽셀을 샘플링하도록 합니다.
if(groupThreadID.y < gBlurRadius)
{
    // 이미지 경계에서 발생하는 범위를 벗어난 샘플을 제한합니다. int y =
    max(dispatchThreadID.y - gBlurRadius, 0);
    gCache[groupThreadID.y] = gInput[int2(dispatchThreadID.x, y)];
}
if(groupThreadID.y >= N-gBlurRadius)
{
    // 이미지 경계에서 발생하는 범위를 벗어난 샘플을 클램프합니다.
    int y = min(dispatchThreadID.y + gBlurRadius, gInput.Length.y-1);
    gCache[groupThreadID.y+2*gBlurRadius] =
        gInput[int2(dispatchThreadID.x, y)];
}

// 이미지 경계에서 발생하는 범위를 벗어난 샘플을 클램프합니다.
gCache[groupThreadID.y+gBlurRadius] =
    gInput[min(dispatchThreadID.xy, gInput.Length.xy-1)];

// 모든 스레드가 완료될 때까지 대기합니다.
GroupMemoryBarrierWithGroupSync();

//

// 이제 각 픽셀을 흐리게 처리합니다.
//

float4 blurColor = float4(0, 0, 0, 0); [unroll]
for(int i = -gBlurRadius; i <= gBlurRadius; ++i)
{
    int k = groupThreadID.y + gBlurRadius + i;

    blurColor += gWeights[i+gBlurRadius]*gCache[k];
}

gOutput[dispatchThreadID.xy] = blurColor;
}
technique11 수평 블러
{
    pass P0
    {
        SetVertexShader(NULL); SetPixelShader(NULL);
        SetComputeShader(CompileShader(cs_5_0, HorzBlurCS()));
    }
}
technique11 VertBlur
{
    pass P0
    {
        SetVertexShader(NULL); SetPixelShader(NULL);
        SetComputeShader(CompileShader(cs_5_0, VertBlurCS()));
    }
}

```

마지막 줄에 대해

```
gOutput[dispatchThreadID.xy] = blurColor;
```

가장 오른쪽 스레드 그룹에서는 출력 텍스처의 요소에 대응하지 않는 불필요한 스레드가 존재할 수 있습니다([그림 12.13](#)). 즉, `dispatchThreadID.xy`가 출력 텍스처에 대한 범위를 벗어난 인덱스가 됩니다. 그러나 범위를 벗어난 쓰기는 아무 작업도 수행하지 않으므로 이 경우를 처리할 필요가 없습니다.

12.8 추가 자료

컴퓨터 셰이더 프로그래밍은 독립적인 주제이며, GPU를 활용한 컴퓨터 프로그램에 관한 여러 책이 있습니다:

1. David B. Kirk와 Wen-mei W. Hwu의 『*Programming Massively Parallel Processors: A Hands-on Approach*』.
2. *오픈CL 프로그래밍 가이드* (저자: 아프타브 문시, 베네딕트 R. 가스터, 티모시 G. 맷슨, 제임스 평, 댄 긴스버그)

CUDA와 OpenCL 같은 기술은 GPU에 접근하여 컴퓨팅 프로그램을 작성하기 위한 서로 다른 API에 불과합니다. CUDA 및 OpenCL 프로그램의 모범 사례는 Direct Compute 프로그램의 모범 사례와 동일합니다. 모든 프로그램이 동일한 하드웨어에서 실행되기 때문입니다. 이 장에서는 Direct Compute 구문의 대부분을 다루었으므로, CUDA 또는 OpenCL 프로그램을 Direct Compute로 포팅하는 데 어려움이 없을 것입니다.

척 월본(Chuck Walbourn)은 다양한 Direct Compute 프레젠테이션 링크로 구성된 블로그 페이지를 게시했습니다:

<http://blogs.msdn.com/b/chuckw/archive/2010/07/14/directcompute.aspx>

또한 Microsoft의 Channel 9에는 DirectCompute 프로그래밍에 관한 일련의 강의 동영상이 있습니다:

<http://channel9.msdn.com/tags/DirectCompute-Lecture-Series/>

마지막으로, NVIDIA는 CUDA 교육에 관한 전체 섹션을 제공합니다:

<http://developer.nvidia.com/cuda-training>

특히 일리노이 대학교의 CUDA 프로그래밍에 관한 전체 강의 동영상이 있으며, 이를 적극 추천합니다. 다시 한번 강조하건대, CUDA는 GPU의 컴퓨팅 기능에 접근하기 위한 또 다른 API일 뿐입니다. 구문을 이해한 후에는 GPU 컴퓨팅의 어려운 부분은 효율적인 프로그램을 작성하는 방법을 배우는 것입니다. 이러한 CUDA 강의를 공부하면 GPU 하드웨어가 어떻게 작동하는지 더 잘 이해하게 되어 최적의 코드를 작성할 수 있습니다.

12.9 요약

1. `ID3D11DeviceContext::Dispatch` API 호출은 스레드 그룹 그리드를 디스패치합니다. 각 스레드 그룹은 스레드의 3차원 그리드입니다. 스레드 그룹당 스레드 수는 컴퓨터 셰이더의 `[numthreads(x,y,z)]` 속성으로 지정됩니다. 성능상의 이유로 총 스레드 수는 워프 크기(NVIDIA 하드웨어 기준 32) 또는 웨이브프론트 크기(ATI 하드웨어 기준 64)의 배수여야 합니다.
2. 병렬 처리를 보장하려면 멀티프로세서당 최소 두 개의 스레드 그룹이 디스패치되어야 합니다. 따라서 하드웨어에 16개의 멀티프로세서가 있다면, 멀티프로세서가 항상 작업을 수행할 수 있도록 최소 32개의 스레드 그룹이 디스패치되어야 합니다. 향후 하드웨어는 더 많은 멀티프로세서를 갖출 가능성이 있으므로, 프로그램이 미래 하드웨어에 잘 확장되도록 스레드 그룹 수는 더 높아야 합니다.
3. 스레드 그룹이 멀티프로세서에 할당되면, NVIDIA 하드웨어에서 해당 스레드 그룹 내 스레드들은 32개 스레드로 구성된 워프로 분할됩니다. 이후 멀티프로세서는 SIMD 방식(즉, 워프 내 각 스레드에 동일한 명령어를 실행)으로 한 번에 하나의 스레드 워프를 처리합니다. 예를 들어 텍스처 메모리를 가져오는 과정에서 워프가 정지되면, 멀티프로세서는 신속하게 전환하여 다른 워프의 명령어를 실행함으로써 이 지연 시간을 숨길 수 있습니다. 이를 통해 멀티프로세서는 항상 바쁘게 작동합니다. 스레드 그룹 크기를 워프 크기의 배수로 설정하라는 권장 사항의 이유를 알 수 있습니다. 그렇지 않으면 스레드 그룹이 워프로 분할될 때 아무 작업도 수행하지 않는 스레드가 포함된 워프가 발생할 수 있기 때문입니다.
4. 컴퓨터 셰이더는 텍스처에 대한 SRV(스트림 리소스 뷰)를 생성하고 이를 컴퓨터 셰이더에 바인딩함으로써 텍스처 리소스에 입력으로 접근할 수 있습니다. 읽기-쓰기 텍스처(RWTexture)는 컴퓨터 셰이더가 읽고 쓰기 출력을 할 수 있는 텍스처입니다. 컴퓨터 셰이더에 읽기/쓰기용 텍스처를 설정하려면 해당 텍스처에 대한 UAV(무순서 액세스 뷰)를 생성하여 컴퓨터 셰이더에 바인딩합니다. 텍스처 요소는 [] 연산자 표기법으로 인덱싱하거나, 텍스처 좌표와 샘플러 상태를 통해 SampleLevel 메서드로 샘플링할 수 있습니다.
5. 구조체 버퍼는 배열처럼 모두 동일한 유형의 요소로 구성된 버퍼입니다. 유형은 예를 들어 구조체로 정의된 사용자 정의 유형일 수 있습니다.

구조체로 정의된 사용자 정의 유형일 수 있습니다. 읽기 전용 구조체 버퍼는 HLSL에서 다음과 같이 정의됩니다:

```
StructuredBuffer<DataType> gInputA;
```

HLSL에서 읽기-쓰기 구조체 버퍼는 다음과 같이 정의됩니다:

```
RWStructuredBuffer<DataType> gOutput;
```

읽기 전용 버퍼 리소스는 구조화된 버퍼에 대한 SRV를 생성하고 컴퓨트 셰이더에 바인딩함으로써 컴퓨트 셰이더가 입력으로 접근할 수 있습니다. 읽기-쓰기 버퍼 리소스는 구조화된 버퍼에 대한 UAV를 생성하고 컴퓨트 셰이더에 바인딩함으로써 컴퓨트 셰이더가 읽고 쓸 수 있습니다.

6. 다양한 스레드 ID가 시스템 값을 통해 컴퓨트 셰이더로 전달됩니다. 이러한 ID는 리소스와 공유 메모리에 인덱싱하는 데 자주 사용됩니다.

7. HLSL에서 구조체 버퍼의 소비 및 추가(Append)는 다음과 같이 정의됩니다:

```
ConsumeStructuredBuffer<DataType> gInput;  
AppendStructuredBuffer<DataType> gOutput;
```

소비 및 추가 구조화 버퍼는 데이터 요소가 처리되고 출력 버퍼에 쓰이는 순서를 신경 쓰지 않을 때 유용합니다. 인덱싱 구문을 피할 수 있기 때문입니다. 추가 버퍼는 자동으로 확장되지 않으며, 추가할 모든 데이터 요소를 저장할 수 있을 만큼 충분히 커야 합니다.

8. 스레드 그룹에는 공유 메모리 또는 스레드 로컬 저장소라고 불리는 영역이 할당됩니다. 이 메모리 접근은 빠르며 하드웨어 캐시만큼 빠르다고 생각할 수 있습니다. 이 공유 메모리 캐시는 최적화에 유용하거나 알고리즘 구현에 필요할 수 있습니다. 컴퓨트 셰이더 코드에서 공유 메모리는 다음과 같이 선언됩니다:

```
groupshared float4 gCache[N];
```

배열 크기 N은 원하는 대로 설정할 수 있지만, 그룹 공유 메모리의 최대 크기는 32KB입니다. 멀티프로세서가 공유 메모리 최대 32KB를 지원한다고 가정할 때, 성능을 위해 스레드 그룹은 16KB 이상의 공유 메모리를 사용해서는 안 됩니다. 그렇지 않으면 단일 멀티프로세서에 두 개의 스레드 그룹을 배치하는 것이 불가능합니다.

9. 가능한 경우 컴퓨팅 처리와 렌더링 간 전환을 피하십시오. 전환에는 오버헤드가 발생합니다. 일반적으로 각 프레임에 대해 모든 컴퓨팅 작업을 먼저 수행한 후 모든 렌더링 작업을 수행하십시오.

12.10 연습문제

1. [1,10] 범위의 무작위 크기를 가진 64개의 3D 벡터로 구성된 구조체 버퍼를 입력으로 받는 컴퓨트 셰이더를 작성하세요. 이 셰이더는 벡터의 길이를 계산하여 결과를 부동소수점 버퍼에 출력합니다. 결과를 CPU 메모리로 복사하고 파일로 저장하세요. 모든 길이가 [1,10] 범위에 포함되는지 확인하세요.
2. 유형 지정 버퍼를 사용하여 이전 연습을 다시 수행하십시오. 즉, 입력 버퍼에는 **Buffer<float3>**를, 출력 버퍼에는 **Buffer<float>**를 사용하십시오.
3. 이전 연습에서는 벡터 정규화 순서를 고려하지 않았다고 가정합니다. Append 및 Consume 버퍼를 사용하여 연습 1을 다시 수행하십시오.
4. 양방향 블러 기법을 연구하고 컴퓨트 셰이더에 구현하세요. 양방향 블러를 사용하여 "Blur" 데모를 다시 실행하세요.
5. 지금까지 데모에서는 *Waves.h/.cpp*의 **Waves** 클래스를 사용해 CPU에서 2차원 파동 방정식을 구현했습니다. 이를 GPU로 이식하세요. 이전, 현재, 다음 높이 해법을 저장하기 위해 부동소수점 텍스처를 사용하세요:

```
Texture2D gPrevSolInput; Texture2D  
gCurrSolInput;  
RWTexture2D<float> gCurrSolOutput; RWTexture2D<float>  
gNextSolOutput;
```

컴퓨트 셰이더를 사용하여 파동 업데이트 계산을 수행합니다. 별도의 컴퓨트 셰이더를 사용하여 물을 교란시켜 파도를 생성할 수 있습니다. 그리드 높이를 업데이트한 후, 파도 텍스처와 동일한 버텍스 해상도(즉, 각 그리드 버텍스에 대응하는 텍셀이 존재하도록)를 가진 삼각형 그리드를 렌더링하고, 현재 파도 솔루션 텍스처를 새로운 "waves" 버텍스 셰이더에 바인딩합니다. 그런 다음 버텍스 셰이더에서 솔루션 텍스처를 샘플링하여 높이 오프셋(이를 *변위 매핑*이라고 함)을 적용하고 법선을 추정할 수 있습니다.

```

VertexOut VS(VertexIn vin)
{
    VertexOut vout;

    // 변환되지 않은 좌표로 변위 맵 샘플링

    // [0,1]^2 텍스처 좌표로 변환되지 않은 상태로 샘플링.
    vin.PosL.y = gDisplacementMap.SampleLevel( samDisplacement,
        vin.Tex, 0.0f).r;

    // 유한 차분법을 사용하여 노멀 추정. float du =
    gDisplacementMap.TexelSize.x; float dv =
    gDisplacementMap.TexelSize.y; float l =
    gDisplacementMap.SampleLevel(
        samDisplacement, vin.Tex-float2(du, 0.0f), 0.0f ).r; float r =
    gDisplacementMap.SampleLevel(
        samDisplacement, vin.Tex+float2(du, 0.0f), 0.0f ).r; float t =
    gDisplacementMap.SampleLevel(
        samDisplacement, vin.Tex-float2(0.0f, dv), 0.0f ).r; float b =
    gDisplacementMap.SampleLevel(
        samDisplacement, vin.Tex+float2(0.0f, dv), 0.0f ).r; vin.NormalL = normalize(float3(-r+l,
        2.0f*gGridSpatialStep, b-t));

    // 월드 공간으로 변환
    vout.PosW = mul(float4(vin.PosL, 1.0f), gWorld).xyz; vout.NormalW = mul(vin.NormalL,
    (float3x3)gWorldInvTranspose);

    // 동차 클립 공간으로 변환.
    vout.PosH = mul(float4(vin.PosL, 1.0f), gWorldViewProj);

    // 삼각형 간 보간을 위한 정점 속성 출력. vout.Tex = mul(float4(vin.Tex, 0.0f, 1.0f),
    gTexTransform).xy;

    return vout;
}

```

성능 결과(프레임당 시간)를 릴리스 모드에서 512×512 그리드 포인트를 사용하는 CPU 구현과 비교하십시오.

CHAPTER 13 컴퓨터 셰이더

테셀레이션 단계는 지오메트리 테셀레이션에 관여하는 렌더링 파이프라인의 세 단계를 의미합니다. 간단히 말해, 테셀레이션은 지오메트리를 더 작은 삼각형으로 분할한 후 새로 생성된 정점들을 어떤 방식으로든 오프셋하는 것을 말합니다. 삼각형 수를 늘리는 목적은 메시에 디테일을 추가하기 위함입니다. 그렇다면 처음부터 디테일이 풍부한 고폴리곤 메시를 생성해버리는 게 낫지 않을까요? 테셀레이션이 필요한 세 가지 이유는 다음과 같습니다.

- 1. GPU 기반 동적 LOD. 카메라와의 거리 및 기타 요소에 따라 메시의 디테일을 동적으로 조정할 수 있습니다. 예를 들어, 메시가 매우 멀리 떨어져 있다면 고해상도 버전을 렌더링하는 것은 낭비입니다. 어차피 그 디테일을 모두 볼 수 없기 때문입니다. 오브젝트가 카메라에 가까워질수록 테셀레이션을 지속적으로 증가시켜 오브젝트의 디테일을 높일 수 있습니다.
- 2. 물리 및 애니메이션 효율성. 저폴리곤 메시에서 물리 및 애니메이션 계산을 수행한 후 고폴리곤 버전으로 테셀레이션할 수 있습니다. 이는 물리 및 애니메이션 계산을 낮은 빈도로 수행함으로써 연산 성능을 절약합니다.
- 3. 메모리 절약. 낮은 폴리곤 메시를 메모리(디스크, RAM, VRAM)에 저장한 후 GPU가 실시간으로 높은 폴리곤 버전으로 테셀레이션하도록 할 수 있습니다.

그림 13.1은 테셀레이션 단계가 버텍스 셰이더와 지오메트리 셰이더 사이에 위치함을 보여줍니다. 이 단계들은 선택적이며, 지금까지 이 책에서는 사용하지 않았습니다.

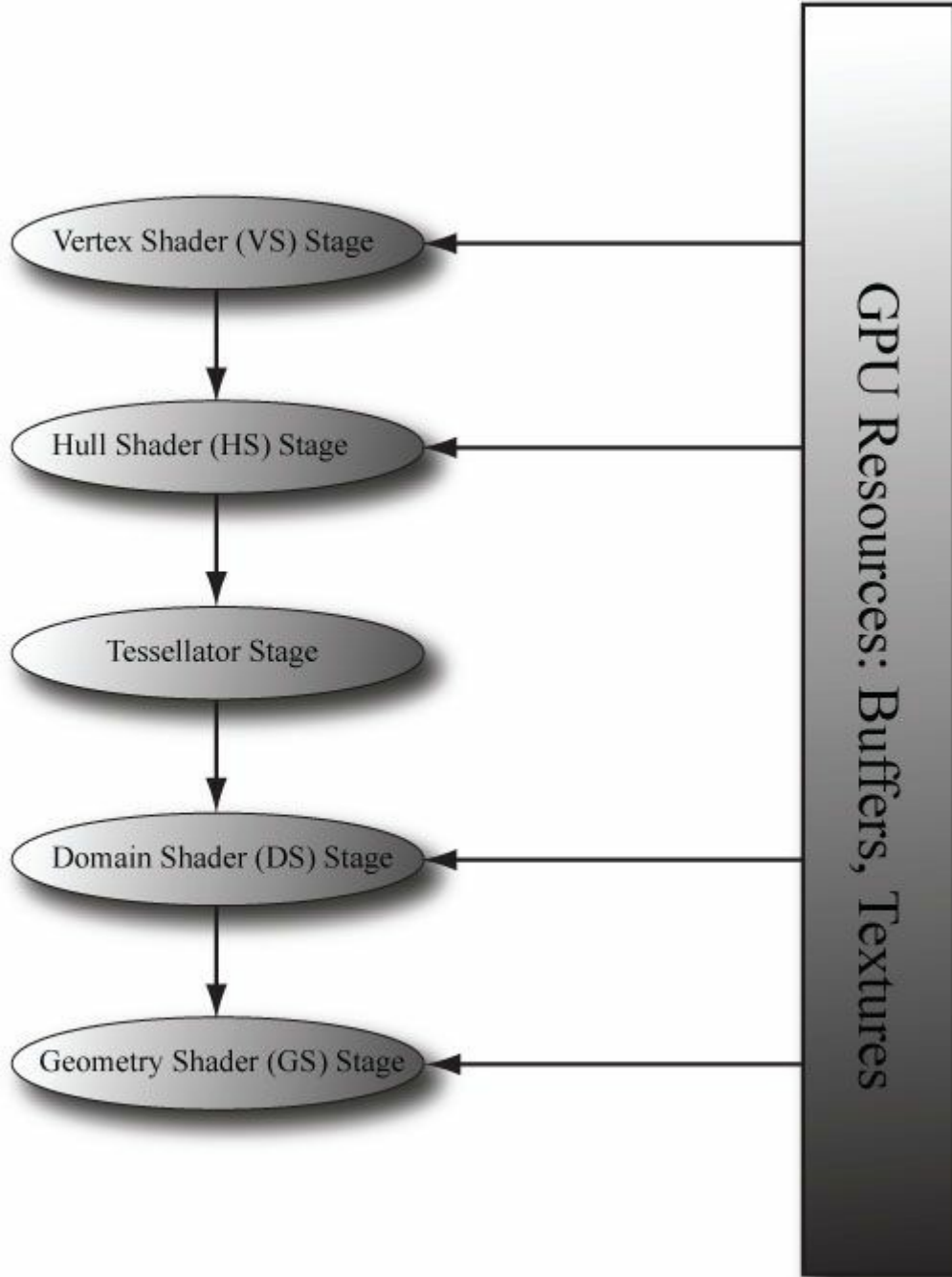


그림 13.1. 테셀레이션 단계를 보여주는 렌더링 파이프라인의 일부.

목표:

1. 테셀레이션에 사용되는 패치 프리미티브 유형을 파악한다.
2. 각 테셀레이션 단계의 기능과 해당 단계의 예상 입력 및 출력에 대한 이해를 얻기 위함.
3. 헐 셰이더 및 도메인 셰이더 프로그램을 작성하여ジオ메트리를 테셀레이션할 수 있도록 한다.
4. 테셀레이션 시점을 결정하는 다양한 전략에 익숙해지고, 하드웨어 테셀레이션과 관련된 성능 고려 사항을 숙지한다.
5. 베지어 곡선과 표면의 수학적 원리를 학습하고, 이를 테셀레이션 단계에 구현하는 방법을 익힌다.

13.1 테셀레이션 프리미티브 유형

테셀레이션을 위한 렌더링 시, 삼각형을 IA 단계에 제출하지 않습니다. 대신 여러 제어점을 가진 패치를 제출합니다. Direct3D는 1~32개의 제어점을 가진 패치를 지원하며, 이는 다음의 기본 유형으로 설명됩니다:

```

D3D11_PRIMITIVE_1_CONTROL_POINT_PATCH=8,
D3D11_PRIMITIVE_2_CONTROL_POINT_PATCH=9,
  
```

```
D3D11_PRIMITIVE_3_CONTROL_POINT_PATCH = 10,
D3D11_PRIMITIVE_4_CONTROL_POINT_PATCH = 11,
.
.
.
D3D11_PRIMITIVE_31_CONTROL_POINT_PATCH = 38,
D3D11_PRIMITIVE_32_CONTROL_POINT_PATCH = 39
```

삼각형은 세 개의 제어점을 가진 삼각형 패치(D3D11_PRIMITIVE_3_CONTROL_POINT_PATCH)로 생각할 수 있으므로, 기존 삼각형 메시를 그대로 테셀레이션 처리하도록 제출할 수 있습니다. 간단한 사각형 패치는 네 개의 제어점(D3D11_PRIMITIVE_4_CONTROL_POINT_PATCH)으로 제출할 수 있습니다. 이러한 패치는 테셀레이션 단계에서 결국 삼각형으로 테셀레이션됩니다.

단계에서 삼각형으로 테셀레이션됩니다.

그렇다면 더 많은 제어점을 가진 패치는 어떻게 될까요? 제어점의 개념은

특정 종류의 수학적 곡선 및 표면 구성에서 비롯됩니다. Adobe Illustrator와 같은 드로잉 프로그램에서 베지어 곡선을 다뤄본 적이 있다면, 제어점을 통해 곡선의 모양을 조정한다는 것을 알 것입니다. 베지어 곡선의 수학은 베지어 표면으로 일반화될 수 있습니다. 예를 들어, 9개 또는 16개의 제어점을 사용하여 모양을 만드는 베지어 쿼드 패치를 생성할 수 있습니다. 제어점의 수가 증가할수록 패치 모양을 자유롭게 조정할 수 있는 범위가 넓어집니다. 따라서 이러한 모든 제어형 프리미티브의 목적은 이러한 종류의 곡면 지원을 제공하는 데 있습니다. 본 장에서는 베지어 쿼드 패치에 대한 설명과 데모를 제공합니다.

13.1.1 테셀레이션과 버텍스 셰이더

패치 제어점을 렌더링 파이프라인에 제출하기 때문에 제어점이 정점 셰이더를 통과하게 됩니다. 따라서 테셀레이션이 활성화되면 정점 셰이더는 사실상 "제어점을 위한 정점 셰이더"가 되며, 테셀레이션이 시작되기 전에 필요한 모든 제어점 작업을 수행할 수 있습니다. 일반적으로 애니메이션이나 물리 계산을 기하학이 테셀레이션되기 전 낮은 주파수에서 정점 셰이더에서 수행합니다.

13.2 헐 셰이더

다음 하위 섹션에서는 헐 셰이더를 살펴보겠습니다. 이 셰이더는 실제로 두 개의 셰이더로 구성됩니다:

- 1. 상수 헐 셰이더
- 2. 제어점 헐 셰이더

13.2.1 상수 헐 셰이더

이 *상수 헐 셰이더*는 패치별로 평가되며, 메쉬의 소위 *테셀레이션 팩터*(tessellation factors)를 출력하는 역할을 합니다. 테셀레이션 팩터는 테셀레이션 단계에 패치를 얼마나 테셀레이션할지 지시합니다. 다음은 4개의 제어점을 가진 *사각형 패치*의 예시이며, 이를 균일하게 3회 테셀레이션합니다.

```
struct PatchTess
{
    float EdgeTess[4] : SV_TessFactor;
    float InsideTess[2] : SV_InsideTessFactor;

    // 패치별로 연관시키고자 하는 추가 정보.
};

PatchTess ConstantHS(InputPatch<VertexOut, 4> patch, uint patchID :
    SV_PrimitiveID)
{
    PatchTess pt;

    // 패치를 균일하게 3회 테셀레이션합니다.

    pt.EdgeTess[0] = 3; // 왼쪽 가장자리 pt.EdgeTess[1] =
    3; // 위쪽 가장자리 pt.EdgeTess[2] = 3; // 오른쪽 가장
    자리 pt.EdgeTess[3] = 3; // 아래쪽 가장자리
```

```
pt.InsideTess[0] = 3; // u축 (열 pt.InsideTess[1] = 3; // v축 (행)
```

```
return pt;
```

상수 헬 셰이더는 패치의 모든 제어점을 입력으로 받으며, 이는 **InputPatch<VertexOut, 4>** 유형으로 정의됩니다. 제어점은 먼저 버텍스 셰이더를 통과하므로, 그 유형은 버텍스 셰이더의 출력 유형인 **VertexOut**에 의해 결정됩니다. 이 예시에서 패치는 네 개의 제어점을 가지므로, **InputPatch**의 두 번째 템플릿 매개변수로 4를 지정합니다. 시스템은 또한 필요 시 사용할 수 있는 패치 ID 값을 **SV_PrimitiveID** 시맨틱을 통해 제공합니다. 이 ID는 드로우 콜 내 패치를 고유하게 식별합니다. 상수 헬 셰이더는 테셀레이션 팩터를 출력해야 하며, 테셀레이션 팩터는 패치의 토폴로지에 따라 달라집니다.

테셀레이션 계수(**SV_TessFactor** 및 **SV_InsideTessFactor**) 외에도, 상수 헬 셰이더에서 다른 패치 정보  를 출력할 수 있습니다. 도메인 셰이더는 상수 헬 셰이더의 출력을 입력으로 받아들이며, 이러한 추가 패치 정보를 활용할 수 있습니다.

사각 패치 테셀레이션은 두 부분으로 구성됩니다:

1. 네 개의 가장자리 테셀레이션 계수는 각 가장자리의 테셀레이션 정도를 제어합니다.
2. 두 개의 내부 테셀레이션 계수는 사각 패치를 어떻게 테셀레이션할지 나타냅니다(사각형의 가로 치수에 대한 테셀레이션 계수 하나와 세로 치수에 대한 테셀레이션 계수 하나).

그림 13.2는 테셀레이션 계수가 서로 다를 때 얻을 수 있는 다양한 사각 패치 구성 예시를 보여줍니다.

가장자리 및 내부 테셀레이션 계수의 작동 방식을 완전히 이해할 때까지 이 그림들을 연구하십시오.

*삼각형 패치*의 테셀레이션 역시 두 부분으로 구성됩니다:

1. 세 개의 모서리 테셀레이션 계수는 각 모서리를 따라 테셀레이션할 정도를 제어합니다.
2. 하나의 내부 테셀레이션 계수는 삼각형 패치를 얼마나 테셀레이션할지를 나타냅니다.

그림 13.3은 테셀레이션 계수가 서로 다를 때 얻을 수 있는 다양한 삼각형 패치 구성 예시를 보여줍니다.

Direct3D 11 하드웨어가 지원하는 최대 테셀레이션 계수는 64입니다. 모든 테셀레이션 계수가 0인 경우 패치는

추가 처리에서 제외됩니다. 이를 통해 패치 단위로 프러스텀 컬링 및 백페이스 컬링과 같은 최적화를 구현할 수 있습니다.

1. 패치가 시야각에 의해 가려지지 않는다면, 해당 패치를 추가 처리에서 제외할 수 있습니다(만약 패치 분할을 수행했다면, 분할된 삼각형들은 삼각형 클리핑 단계에서 제외될 것입니다).
2. 패치가 백페이스 상태라면 추가 처리에서 제외할 수 있습니다(테셀레이션이 수행된 경우, 테셀레이션된 삼각형은 래스터화 과정의 백페이스 컬링 단계에서 제외됩니다).

자연스럽게 제기되는 질문은 테셀레이션의 적정 수준입니다. 테셀레이션의 기본 개념은 메시에 디테일을 추가하는 것이지만, 사용자가 인지할 수 없는 디테일을 불필요하게 추가해서는 안 됩니다. 테셀레이션 수준을 결정하는 데 흔히 사용되는 지표는 다음과 같습니다:

1. 카메라로부터의 거리: 물체가 시선으로부터 멀어질수록 우리는 미세한 디테일을 덜 인지하게 됩니다. 따라서 물체가 멀리 있을 때는 저폴리 버전을 렌더링하고, 시선에 가까워질수록 더 많은 테셀레이션을 적용할 수 있습니다.
2. 화면 영역 커버리지: 물체가 화면에서 차지하는 픽셀 수를 추정할 수 있습니다. 이 수치가 작다면 물체의 저폴리 버전을 렌더링할 수 있습니다. 화면 영역 커버리지가 증가함에 따라 더 많은 테셀레이션을 적용할 수 있습니다.
3. 방향성: 삼각형의 방향성을 고려하여 실루엣 경계선을 따라 위치한 삼각형은 다른 삼각형보다 더 정교하게 처리됩니다.
4. 거칠기: 디테일이 많은 거친 표면은 매끄러운 표면보다 더 많은 테셀레이션이 필요합니다. 표면 텍스처를 분석하여 거칠기 값을 사전 계산할 수 있으며, 이를 테셀레이션 정도 결정에 활용할 수 있습니다.

[Story10]은 다음과 같은 성능 관련 조언을 제시합니다:

1. 테셀레이션 계수가 1인 경우(기본적으로 테셀레이션을 수행하지 않는다는 의미), 해당 패치를 테셀레이션 없이 렌더링하는 것을 고려하십시오. 테셀레이션 단계에서 아무 작업도 수행하지 않으면서 GPU 오버헤드를 낭비하게 될 수 있기 때문입니다.
2. GPU 구현과 관련된 성능상의 이유로, 삼각형이 너무 작아 8픽셀 미만을 차지하는 테셀레이션을 수행하지 마십시오.
3. 테셀레이션을 사용하는 드로우 콜을 일괄 처리하십시오(즉, 드로우 콜 사이에 테셀레이션을 켜고 끄는 것은 비용이 많이 듭니다).

13.2.2 제어점 헬 셰이더

제어점 헬 셰이더는 여러 제어점을 입력으로 받고 여러 제어점을 출력으로 낸다. 제어점 헬 셰이더는 출력되는 각 제어점마다 한 번씩 호출된다. 헬 셰이더의 한 가지 용도는 표면 표현을 변경하는 것이다. 예를 들어 일반적인

삼각형(파이프라인에 3개의 제어점으로 제출됨)을 큐빅 베지어 삼각형 패치(10개의 제어점을 가진 패치)로 변경하는 것입니다. 예를 들어, 메쉬가 일반적인 삼각형(3개 제어점)으로 모델링되었다고 가정해 보겠습니다. 헬 셰이더를 사용하여 삼각형을 10개 제어점을 가진 고차 큐빅 베지어 삼각형 패치로 확장할 수 있습니다. 그런 다음 추가 제어점을 통해 디테일을 추가하고 삼각형 패치를 원하는 수준까지 테셀레이션할 수 있습니다. 이 전략은 소위 *N-패치 방식* 또는 *PN 삼각형* 방식[Vlachos01]이라 불리며, 아트 파이프라인을 수정하지 않고도 테셀레이션을 통해 기존 삼각형 메쉬를 개선할 수 있어 편리합니다. 첫 번째 데모에서는 단순한 *통과* 셰이더를 구현할 예정이며, 여기서 제어점은 수정 없이 그대로 전달됩니다.

 *드라이버는 패스스루 셰이더를 감지하고 최적화할 수 있습니다 [Bilodeau10b].*

```
struct HullOut
{
    float3 PosL : POSITION;
};

[domain("quad")] [partitioning("integer")]
[outputtopology("triangle_ccw")] [outputcontrolpoints(4)]
[patchconstantfunc("ConstantHS")]
[maxtessfactor(64.0f)]
HullOut HS(InputPatch<VertexOut, 4> p,
           uint i : SV_OutputControlPointID, uint patchId :
           SV_PrimitiveID)
{
    HullOut hout; hout.PosL =
    p[i].PosL; return hout;
}
```

헬 셰이더는 **InputPatch** 매개변수를 통해 패치의 모든 제어점을 입력받습니다. 시스템 값 **SV_OutputControlPointID**는 헬 셰이더가 작업 중인 출력 제어점을 식별하는 인덱스를 제공합니
다. 입력 패치의 제어점 수가 출력 제어점 수와 일치할 필요는 *없습니다*. 예를 들어, 입력 패치는 4개의 제어점을 가질 수 있으며
출력 패치에는 16개의 제어점이 있을 수 있습니다. 추가 제어점은 4개의 입력 제어점에서 파생될 수 있습니다.
제어점 헬 셰이더는 여러 속성을 도입합니다:

- 1. **domain**: 패치 유형. 유효한 인수는 **삼각형(tri)**, **사각형(quad)**, 또는 **등고선(isoline)**입니다.
- 2. **partitioning**: 테셀레이션의 세분화 모드를 지정합니다.

teger: 새 정점은 테셀레이션 계수의 정수 값에서만 추가/제거됩니다. 테셀레이션 계수의 소수 부분은 무시됩니다. 이는 메쉬의 테셀레이션 레벨이 변경될 때 눈에 띄는 "팝핑" 현상을 유발합니다.
소수점 테셀레이션(**fractional_even/fractional_odd**): 정수 테셀레이션 계수 값에서만 새 정점이 추가/제거되지만, 테셀레이션 계수의 소수점 부분을 기준으로 점진적으로 "슬라이드"됩니다. 이는
정수 단계에서 갑작스럽게 전환하기보다 테셀레이션을 통해 메쉬의 거친 버전에서 정밀한 버전으로 부드럽게 전환하고자 할 때 유용합니다. 정수 테셀레이션과 분수 테셀레이션의 차이는 애니메
이션으로 이해하는 것이 가장 효과적이므로, 본 장 말미의 연습 문제에서 직접 차이를 확인해 보시기 바랍니다.

- 3. **outputtopology**: 세분화를 통해 생성된 삼각형의 감김 순서.
- iangle_ccw**: 시계 방향 순회 순서. **iangle_ccw**: 반시계 방향 순회 순서.

ne: 선 테셀레이션을 위한 옵션.

- 4. **출력 제어점**: 헬 셰이더가 실행되는 횟수로, 매번 하나의 제어점을 출력합니다. 시스템 값 **SV_OutputControlPointID**는 헬 셰이더가 작업 중인 출력 제어점을 식별하는 인덱스를 제공합니다.
- 5. **patchconstantfunc**: 상수 헬 셰이더 함수 이름을 지정하는 문자열입니다.
- 6. **maxtessfactor**: 셰이더가 사용하는 최대 테셀레이션 계수를 드라이버에 알려주는 힌트입니다. 하드웨어가 이 상한값을 알면 테셀레이션에 필요한 리소스 양을 파악할 수 있으므로 잠재적으
로 하드웨어 최적화를 가능하게 합니다. Direct3D 11 하드웨어가 지원하는 최대 테셀레이션 계수는 64입니다.

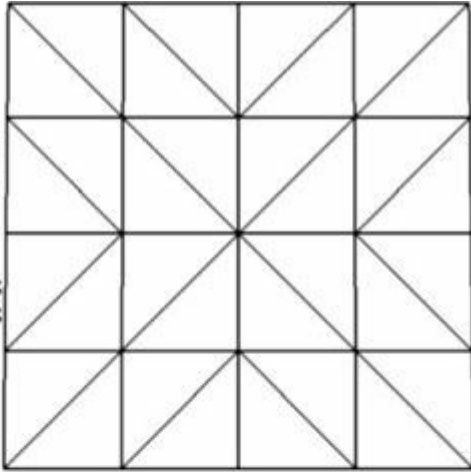
13.3 테셀레이션 단계

프로그래머는 테셀레이션 단계를 제어할 수 없습니다. 이 단계는 하드웨어가 전담하며, 상수 헵 셰이더 프로그램에서 출력된 테셀레이션 계수를 기반으로 패치를 테셀레이션합니다. 다음 그림들은 테셀레이션 계수에 따른 다양한 세분화 방식을 보여줍니다.

13.3.1 쿼드 패치 테셀레이션 예시

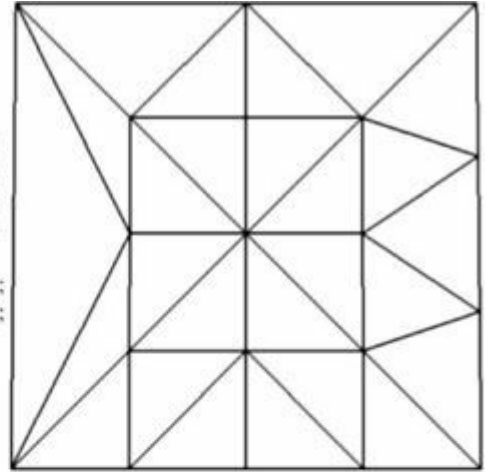
```
pt.EdgeTess[0] = 4;  
pt.EdgeTess[1] = 4;  
pt.EdgeTess[2] = 4;  
pt.EdgeTess[3] = 4;
```

```
pt.InsideTess[0] = 4;  
pt.InsideTess[1] = 4;
```



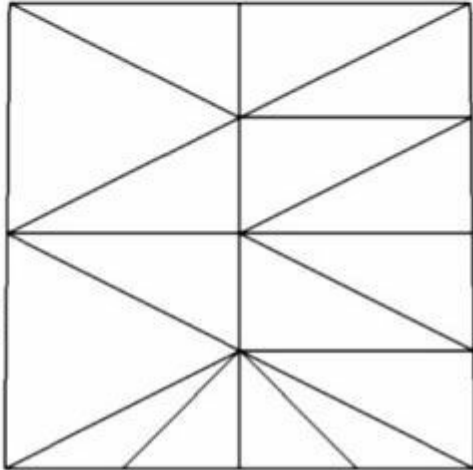
```
pt.EdgeTess[0] = 1;  
pt.EdgeTess[1] = 2;  
pt.EdgeTess[2] = 3;  
pt.EdgeTess[3] = 4;
```

```
pt.InsideTess[0] = 4;  
pt.InsideTess[1] = 4;
```



```
pt.EdgeTess[0] = 2;  
pt.EdgeTess[1] = 2;  
pt.EdgeTess[2] = 4;  
pt.EdgeTess[3] = 4;
```

```
pt.InsideTess[0] = 2;  
pt.InsideTess[1] = 4;
```



```
pt.EdgeTess[0] = 4;  
pt.EdgeTess[1] = 4;  
pt.EdgeTess[2] = 4;  
pt.EdgeTess[3] = 4;
```

```
pt.InsideTess[0] = 3;  
pt.InsideTess[1] = 3;
```

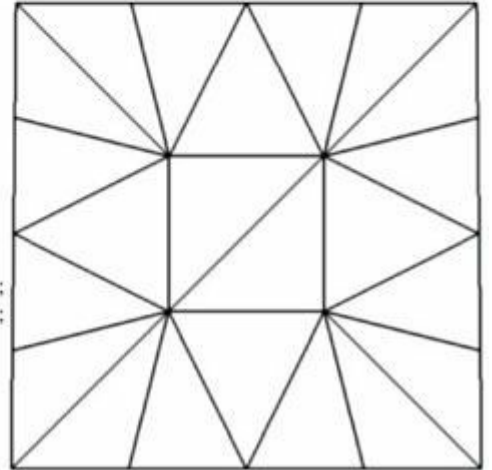
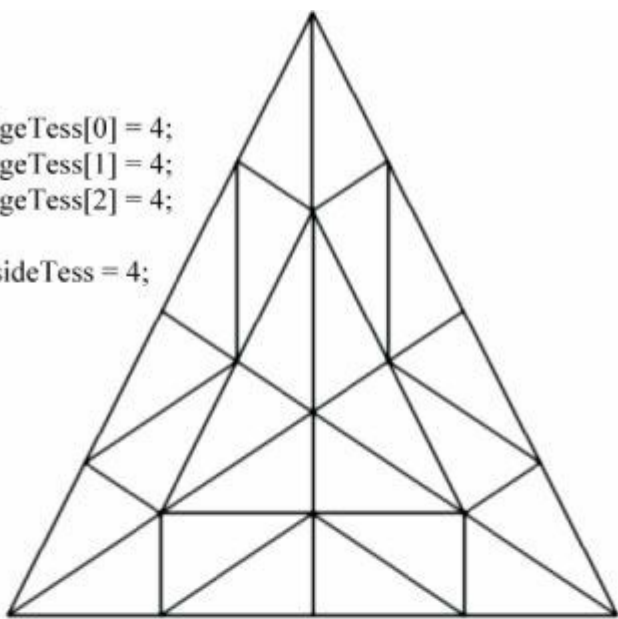


그림 13.2. 모서리와 내부 테셀레이션 계수를 기반으로 한 사각형 세분화.

13.3.2 삼각형 패치 테셀레이션 예시

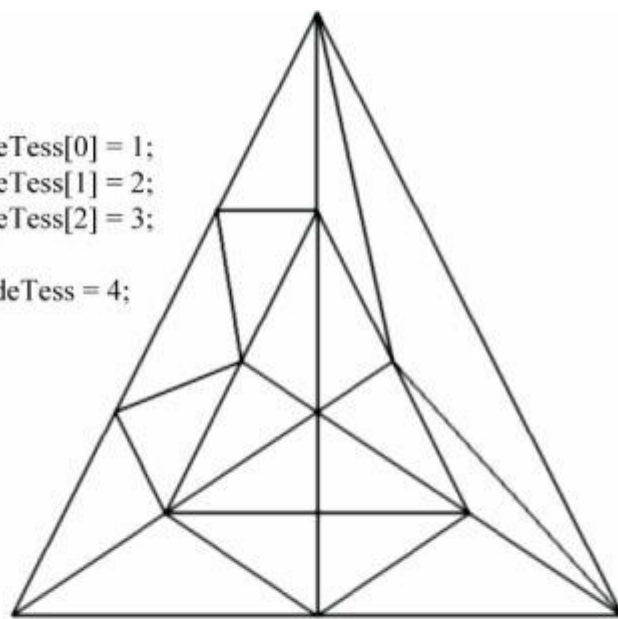
```
pt.EdgeTess[0] = 4;
pt.EdgeTess[1] = 4;
pt.EdgeTess[2] = 4;

pt.InsideTess = 4;
```



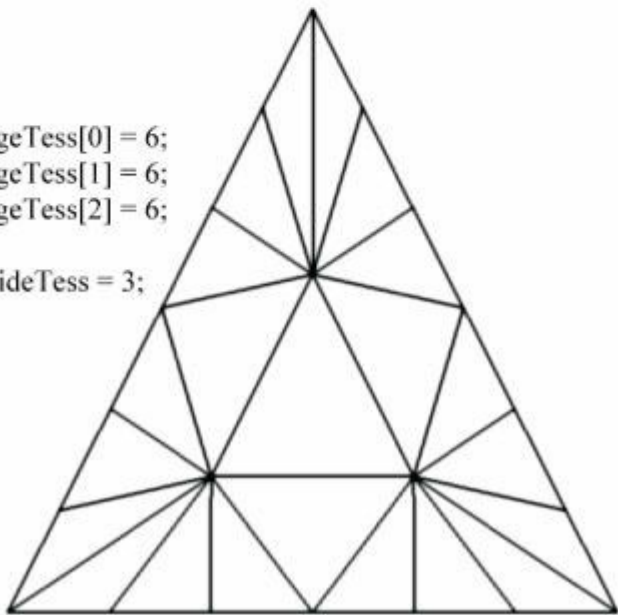
```
pt.EdgeTess[0] = 1;
pt.EdgeTess[1] = 2;
pt.EdgeTess[2] = 3;

pt.InsideTess = 4;
```



```
pt.EdgeTess[0] = 6;
pt.EdgeTess[1] = 6;
pt.EdgeTess[2] = 6;

pt.InsideTess = 3;
```



```
pt.EdgeTess[0] = 6;
pt.EdgeTess[1] = 12;
pt.EdgeTess[2] = 3;

pt.InsideTess = 1;
```

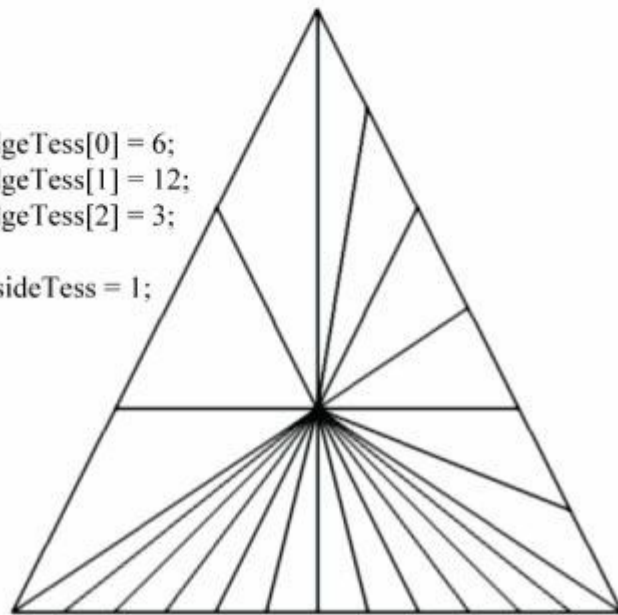


그림 13.3. 가장자리 및 내부 테셀레이션 계수를 기반으로 한 삼각형 세분화.

13.4 도메인 셰이더

테셀레이션 단계는 새로 생성된 모든 정점과 삼각형을 출력합니다. 도메인 셰이더는 테셀레이션 단계에서 생성된 각 정점에 대해 호출됩니다. 테셀레이션이 활성화된 상태에서는, 버텍스 셰이더가 각 제어점에 대한 버텍스 셰이더 역할을 하는 반면, 헬 셰이더는 본질적으로 테셀레이션된 패치에 대한 버텍스 셰이더 역할을 합니다. 특히, 테셀레이션된 패치의 정점들을 동차 클립 공간으로 투영하는 작업이 바로 여기서 수행됩니다.

사각 패치의 경우, 도메인 셰이더는 테셀레이션 계수(및 패치당 출력하는 기타 정보)를 입력으로 받습니다.

상수 헬 셰이더, 테셀레이션된 정점 위치의 매개변수 (u, v) 좌표, 그리고 제어점 헬 셰이더에서 출력된 모든 패치 제어점입니다. 도메인 셰이더는 실제 테셀레이션된 정점 위치를 제공하지 않으며,

대신 패치 도메인 공간에서 이 점들의 매개변수 (u, v) 좌표(그림 13.4)를 제공합니다. 이러한 파라메트릭 좌표와 제어점을 사용하여 실제 3D 정점 위치를 도출하는 것은 사용자에게 달려 있습니다. 다음 코드에서는 이 작업을 이차원 보간법(텍스처 선형 필터링과 동일한 방식으로 작동)을 통해 수행합니다.

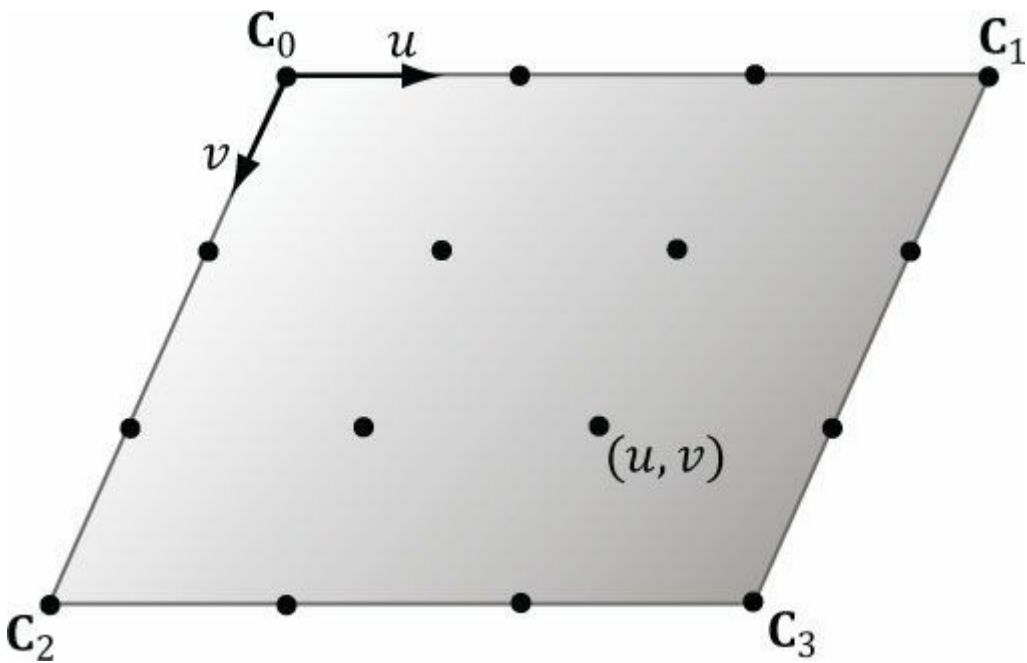


그림 13.4. 4개의 제어점을 가진 쿼드 패치의 테셀레이션으로 정규화된 uv 공간에서 $[0, 1]^2$ 범위의 좌표를 가진 16개의 정점을 생성합니다.

```
struct DomainOut
{
    float4 PosH : SV_POSITION;
};

// 도메인 셰이더는 테셀레이터가 생성한 모든 버텍스에 대해 호출됩니다.
// 이는 테셀레이션 후의 버텍스 셰이더와 유사합니다. [domain("quad")]
DomainOut DS(PatchTess patchTess, float2 uv :
    SV_DomainLocation,
    const OutputPatch<HullOut, 4> quad)
{
    DomainOut dout;

    // 이차 보간.
    float3 v1 = lerp(quad[0].PosL, quad[1].PosL, uv.x); float3 v2 =
    lerp(quad[2].PosL, quad[3].PosL, uv.x); float3 p = lerp(v1, v2, uv.y);

    dout.PosH = mul(float4(p, 1.0f), gWorldViewProj); return dout;
}
```

 **Note:** 그림 13.4에서 볼 수 있듯이, 사각형 패치의 제어점 순서는 행별로 정렬됩니다.

삼각형 패치의 도메인 셰이더는 유사하지만, 매개변수 (u, v) 값 대신 정점의 부리센트릭 (u, v, w) 좌표가 입력됩니다(부리센트릭 좌표에 대한 설명은 "부록 C," §C.3 참조). 삼각형 패치에 대해 중점 좌표를 출력하는 이유는 아마도 베지어 삼각형 패치가 중점 좌표로 정의되기 때문일 것입니다.

13.5 사각형 패치 테셀레이션

이 장의 데모 중 하나에서는 사각형 패치를 렌더링 파이프라인에 제출하고, 카메라로부터의 거리를 기준으로 테셀레이션한 후, 생성된 정점들을 과거 데모에서 "언덕" 표현에 사용했던 것과 유사한 수학적 함수로 변위시킵니다.

4개의 제어점을 저장하는 우리의 정점 버퍼는 다음과 같이 생성됩니다.

```
void BasicTessellation::BuildQuadPatchBuffer()
```



```

{
    D3D11_BUFFER_DESC vbd;
    vbd.Usage = D3D11_USAGE_IMMUTABLE;
    vbd.ByteWidth = sizeof(XMFLOAT3) * 4; vbd.BindFlags =
    D3D11_BIND_VERTEX_BUFFER; vbd.CPUAccessFlags = 0;
    vbd.MiscFlags = 0;

    XMFLOAT3 vertices[4] =
    {
        XMFLOAT3(-10.0f, 0.0f, +10.0f),
        XMFLOAT3(+10.0f, 0.0f, +10.0f),
        XMFLOAT3(-10.0f, 0.0f, -10.0f),
        XMFLOAT3(+10.0f, 0.0f, -10.0f)
    };

    D3D11_SUBRESOURCE_DATA vinitData; vinitData.pSysMem
    = vertices;
    HR(md3dDevice->CreateBuffer(&vbd, &vinitData, &mQuadPatchVB));
}

```

쿼드 패치를 IA 단계에 제출하는 것은 다음과 같이 간단히 수행됩니다.

```

md3dImmediateContext->IASetPrimitiveTopology( D3D11_PRIMITIVE_TOPOLOGY_4_CONTROL_POINT_PATCHLIST);

UINT stride = sizeof(Vertex::Pos); UINT offset = 0;

md3dImmediateContext->IASetVertexBuffers( 0, 1,
    &mQuadPatchVB, &stride, &offset);

md3dImmediateContext->Draw(4, 0);

```

이제 선체 셰이더에 주목하겠습니다. 선체 셰이더는 §13.2.1과 §13.2.2에서 보여드린 것과 유사하지만, 이제 테셀레이션 계수를 시선으로부터의 거리에 따라 결정한다는 점이 다릅니다. 이 방식의 핵심은 멀리 있는 메쉬에는 저폴리곤 메쉬를 사용하고, 메쉬가 시점에 가까워질수록 테셀레이션(그리고 삼각형 개수)을 증가시키는 것입니다([그림 13.5](#) 참조). 힐 셰이더는 단순히 패스트루 셰이더입니다.

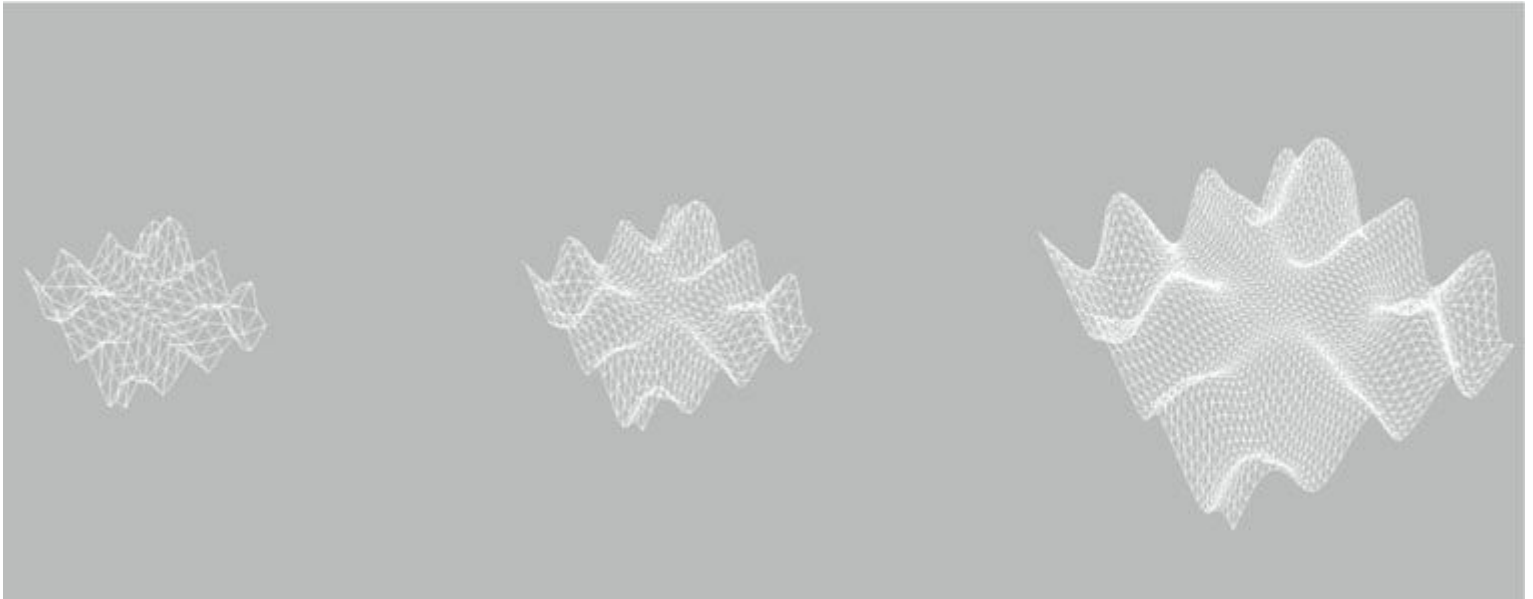


그림 13.5. 시점과의 거리가 가까워질수록 메쉬의 테셀레이션이 증가합니다.

```

struct VertexIn
{
    float3 PosL : POSITION;

```



```

};

struct VertexOut
{
    float3 PosL : POSITION;

};

VertexOut VS(VertexIn vin)
{
    VertexOut vout; vout.PosL =
    vin.PosL; return vout;
}

struct PatchTess
{
    float EdgeTess[4]   : SV_TessFactor;
    float InsideTess[2] : SV_InsideTessFactor;
};

PatchTess ConstantHS(InputPatch<VertexOut, 4> 패치, uint 패치ID : SV_PrimitiveID)
{
    PatchTess pt;

    // 패치의 월드 공간 중심점 찾기.
    float3 centerL = 0.25f*(patch[0].PosL + patch[1].PosL + patch[2].PosL + patch[3].PosL); float3 centerW = mul(float4(centerL,
    1.0f), gWorld).xyz;

    float d = distance(centerW, gEyePosW);

    // 눈으로부터의 거리에 따라 패치를 테셀레이션하여
    // 테셀레이션 값은 d >= d1일 때 0, d <= d0일 때 64입니다. 구간
    // [d0, d1]은 테셀레이션이 이루어지는 범위를 정의합니다.

    const float d0 = 20.0f; const float d1
    = 100.0f;
    float tess = 64.0f * saturate((d1 - d) / (d1 - d0));

    // 패치를 균일하게 테셀레이션합니다.

    pt.EdgeTess[0]   =   tess;
    pt.EdgeTess[1]   =   tess;
    pt.EdgeTess[2]   =   tess;
    pt.EdgeTess[3] = tess;

    pt.InsideTess[0] = tess; pt.InsideTess[1] = tess;

    return pt;
}

struct HullOut
{
    float3 PosL : POSITION;
};

[domain("quad")] [partitioning("integer")]
[outputtopology("triangle_cw")] [outputcontrolpoints(4)]
[patchconstantfunc("ConstantHS")]
[maxtessfactor(64.0f)]
HullOut HS(InputPatch<VertexOut, 4> p,

```

```

uint i : SV_OutputControlPointID, uint patchId :
SV_PrimitiveID)

{

    HullOut hout; hout.PosL =

    p[i].PosL; return hout;

}

```

단순한 테셀레이션만으로는 디테일을 추가하기에 충분하지 않습니다. 새로 생성된 삼각형들은 단순히 세분화된 패치 위에 놓여 있기 때문입니다. 모델링 중인 오브젝트의 형태를 더 잘 근사화하기 위해 추가된 점점들을 어떤 방식으로든 오프셋해야 합니다. 이는 도메인 셰이더에서 수행됩니다. 이 데모에서는 §6.10에서 소개한 "hills" 함수를 사용하여 y좌표를 오프셋합니다.

```

struct DomainOut
{
    float4 PosH : SV_POSITION;
};

// 도메인 셰이더는 테셀레이터가 생성한 모든 버텍스에 대해 호출됩니다.
// 테셀레이션 이후의 버텍스 셰이더와 유사합니다. [domain("quad")]
DomainOut DS(PatchTess patchTess,
              float2 uv : SV_DomainLocation,
              const OutputPatch<HullOut, 4> quad)
{
    DomainOut dout;

    // 이차 보간법.
    float3 v1 = lerp(quad[0].PosL, quad[1].PosL, 1-uv.y); float3 v2 =
    lerp(quad[3].PosL, quad[2].PosL, 1-uv.y); float3 p = lerp(v1, v2, uv.x);

    // 변위 매핑
    p.y = 0.3f*(p.z*sin(p.x) + p.x*cos(p.z));

    dout.PosH = mul(float4(p, 1.0f), gWorldViewProj); return dout;
}

float4 PS(DomainOut pin) : SV_Target
{
    return float4(1.0f, 1.0f, 1.0f, 1.0f);
}

technique11 Tess
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS()));
        SetHullShader(CompileShader(hs_5_0, HS()));
        SetDomainShader(CompileShader(ds_5_0, DS()));
        SetPixelShader(CompileShader(ps_5_0, PS()));
    }
}

```

13.6 3차 베지어 쿼드 패치

이 섹션에서는 제어점의 수가 더 많은 표면이 어떻게 구성되는지 보여주기 위해 3차 베지어 쿼드 패치에 대해 설명합니다.

표면을 다루기 전에 베지어 곡선부터 시작하는 것이 도움이 됩니다.

13.6.1 베지어 곡선

세 개의 비공선점 $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2$ 를 고려하자. 이들을 제어점이라 부른다. 이 세 제어점은 다음과 같은 방식으로 베지어 곡선을 정의한다. 곡선상의 점 $\mathbf{p}(t)$ 는 먼저 t 에 따라 $\mathbf{p}_0$ 과 $\mathbf{p}_1$ 사이를 선형 보간하고, t 에 따라 $\mathbf{p}_1$ 과 $\mathbf{p}_2$ 사이를 선형 보간하여 중간점들을 구한다:

$$\mathbf{p}_0^1 = (1-t)\mathbf{p}_0 + t\mathbf{p}_1$$

$$\mathbf{p}_1^1 = (1-t)\mathbf{p}_1 + t\mathbf{p}_2$$

그러면 $\mathbf{p}(t)$ 는 t 에 대해 $\mathbf{p}_0^1$ 와 $\mathbf{p}_1^1$ 사이를 선형 보간하여 구한다:

$$\begin{aligned}\mathbf{p}(t) &= (1-t)\mathbf{p}_0^1 + t\mathbf{p}_1^1 \\ &= (1-t)((1-t)\mathbf{p}_0 + t\mathbf{p}_1) + t((1-t)\mathbf{p}_1 + t\mathbf{p}_2) \\ &= (1-t)^2\mathbf{p}_0 + 2(1-t)t\mathbf{p}_1 + t^2\mathbf{p}_2\end{aligned}$$

다시 말해, 이러한 반복 보간을 통한 구성은 2차(2차) 베지어 곡선의 매개변수 공식으로 이어집니다:

$$\mathbf{p}(t) = (1-t)^2\mathbf{p}_0 + 2(1-t)t\mathbf{p}_1 + t^2\mathbf{p}_2$$

비슷한 방식으로, 네 개의 제어점 $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$ 은 3차 베지어 곡선을 정의하며, 곡선상의 점 $\mathbf{p}(t)$ 는 반복 보간을 통해 다시 구해집니다. 그림 13.6은 이 상황을 보여줍니다. 먼저, 주어진 네 제어점을 따라 각 선분별로 선형 보간을 수행하여 세 개의 1세대 중간점을 얻습니다:

$$\mathbf{p}_0^1 = (1-t)\mathbf{p}_0 + t\mathbf{p}_1$$

$$\mathbf{p}_1^1 = (1-t)\mathbf{p}_1 + t\mathbf{p}_2$$

$$\mathbf{p}_2^1 = (1-t)\mathbf{p}_2 + t\mathbf{p}_3$$

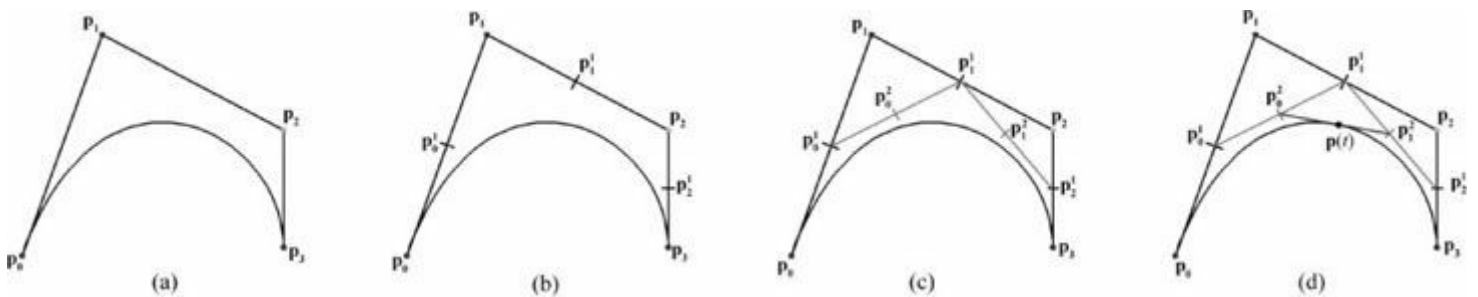


그림 13.6. 3차 베지어 곡선 상에서 반복 선형 보간으로 정의된 점들. 그림은 $t = 0.5$ 를 사용함. (a) 4개의 제어점과 이들이 정의하는 곡선. (b) 제어점 사이를 선형 보간하여 1세대 중간점 계산. (c) 1세대 중간점들 사이를 선형 보간하여 2세대 중간점들을 구합니다. (d) 2세대 중간점들 사이를 선형 보간하여 곡선상의 점을 구합니다.

다음으로, 이렇게 정의된 1세대 중간점들을 각 선분(line segment)을 따라 선형 보간하여 두 개의 2세대 중간점들을 얻습니다:

$$\begin{aligned}\mathbf{p}_0^2 &= (1-t)\mathbf{p}_0^1 + t\mathbf{p}_1^1 \\ &= (1-t)^2\mathbf{p}_0 + 2(1-t)t\mathbf{p}_1 + t^2\mathbf{p}_2\end{aligned}$$

$$\begin{aligned}\mathbf{p}_1^2 &= (1-t)\mathbf{p}_1^1 + t\mathbf{p}_2^1 \\ &= (1-t)^2\mathbf{p}_1 + 2(1-t)t\mathbf{p}_2 + t^2\mathbf{p}_3\end{aligned}$$

마지막으로, $\mathbf{p}(t)$ 는 이 마지막 세대의 중간점들 사이에서 선형 보간을 통해 구해집니다:

$$\begin{aligned}\mathbf{p}(t) &= (1-t)\mathbf{p}_0^2 + t\mathbf{p}_1^2 \\ &= (1-t)\left(\left(1-t\right)^2\mathbf{p}_0 + 2(1-t)t\mathbf{p}_1 + t^2\mathbf{p}_2\right) + t\left(\left(1-t\right)^2\mathbf{p}_1 + 2(1-t)t\mathbf{p}_2 + t^2\mathbf{p}_3\right)\end{aligned}$$

이는 3차 베지어 곡선의 매개변수 공식으로 단순화됩니다:

$$\mathbf{p}(t) = (1-t)^3\mathbf{p}_0 + 3t(1-t)^2\mathbf{p}_1 + 3t^2(1-t)\mathbf{p}_2 + t^3\mathbf{p}_3 \quad (\text{식 13.1})$$

일반적으로 사람들은 3차 곡선에서 멈추는데, 이는 곡선을 제어하기에 충분한 부드러움과 자유도를 제공하기 때문입니다. 그러나 동일한 반복 보간 재귀 패턴을 사용하여 더 높은 차원의 곡선으로 계속 진행할 수 있습니다.

n 차 베지어 곡선의 공식은 *베르슈타인 기저 함수*를 사용하여 표현할 수 있으며, 이는 다음과 같이 정의됩니다:

$$B_i^n(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i}$$

3차 곡선의 경우, 번스타인 기저 함수는 다음과 같습니다:

$$\begin{aligned}B_0^3(t) &= \frac{3!}{0!(3-0)!} t^0 (1-t)^{3-0} = (1-t)^3 \\ B_1^3(t) &= \frac{3!}{1!(3-1)!} t^1 (1-t)^{3-1} = 3t(1-t)^2 \\ B_2^3(t) &= \frac{3!}{2!(3-2)!} t^2 (1-t)^{3-2} = 3t^2(1-t) \\ B_3^3(t) &= \frac{3!}{3!(3-3)!} t^3 (1-t)^{3-3} = t^3\end{aligned}$$

이 값들을 방정식 13.1의 계수와 비교해 보십시오. 따라서 3차 베지어 곡선은 다음과 같이 쓸 수 있습니다:

$$\mathbf{p}(t) = \sum_{j=0}^3 B_j^3(t)\mathbf{p}_j = B_0^3(t)\mathbf{p}_0 + B_1^3(t)\mathbf{p}_1 + B_2^3(t)\mathbf{p}_2 + B_3^3(t)\mathbf{p}_3$$

3차 베르나르트 기저 함수의 미분은 지수 법칙과 곱 법칙을 적용하여 구할 수 있다:

$$\begin{aligned}B_0^{3'}(t) &= -3(1-t)^2 \\ B_1^{3'}(t) &= 3(1-t)^2 - 6t(1-t) \\ B_2^{3'}(t) &= 6t(1-t) - 3t^2 \\ B_3^{3'}(t) &= 3t^2\end{aligned}$$

그리고 3차 베지어 곡선의 미분은 다음과 같습니다:

$$\mathbf{p}'(t) = \sum_{j=0}^3 B_j^{3'}(t)\mathbf{p}_j = B_0^{3'}(t)\mathbf{p}_0 + B_1^{3'}(t)\mathbf{p}_1 + B_2^{3'}(t)\mathbf{p}_2 + B_3^{3'}(t)\mathbf{p}_3$$

미분은 곡선을 따라 접선 벡터를 계산하는 데 유용합니다.



온라인에는 제어점을 설정하고 조작하여 곡선의 모양을 대화식으로 확인할 수 있는 베지어 곡선 애플릿이 있습니다.

13.6.2 3차 베지어 곡면

본 절에서는 [그림 13.7](#)을 참조하십시오. 4×4 제어점 패치를 고려합니다. 따라서 각 행은 3차 베지어 곡선을 정의하는 데 사용될 수 있는 4개의 제어점을 포함합니다. i 번째 행의 베지어 곡선은 다음과 같이 주어집니다:

$$\mathbf{q}_i(u) = \sum_{j=0}^3 B_j^3(u) \mathbf{p}_{i,j}$$

이 베지어 곡선들을 각각 u_0 에서 평가하면, 각 곡선을 따라 하나씩 총 4개의 점이 있는 "열"을 얻게 됩니다. 이 4개의 점을 사용하여 u_0 에서 *베지어 표면*에 놓이는 또 다른 베지어 곡선을 정의할 수 있습니다

$$\mathbf{p}(v) = \sum_{i=0}^3 B_i^3(v) \mathbf{q}_i(u_0)$$

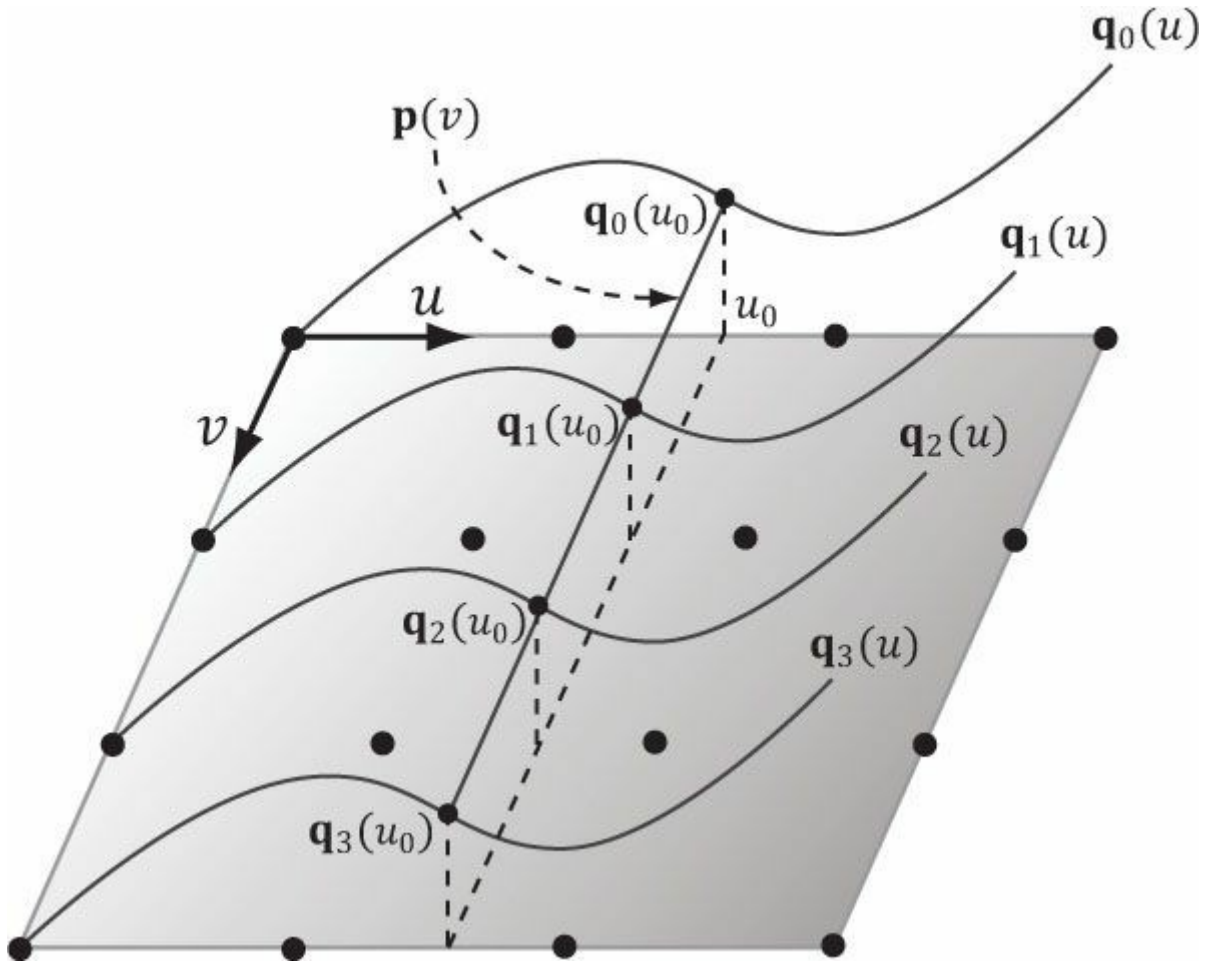


그림 13.7. 베지어 곡면 구성. 이 그림을 이해하기 쉽게 하기 위해 일부 단순화가 이루어졌습니다—제어점들이 모두 평면에 위치하지는 않으며, 그림이 암시하는 것처럼 모든 $\mathbf{q}_i(u)$ 가 동일할 필요는 없습니다(각 행의 제어점이 동일하여 동일한 곡선을 생성할 때만 동일해집니다). 또한 $\mathbf{p}(v)$ 는 일반적으로 직선이 아니라 3차 베지어 곡선입니다.

이제 u 도 변하게 하면, 3차 베지어 곡면(cubic Bézier surface)을 이루는 일련의 3차 베지어 곡선(cubic Bézier curves)을 스윕해 낼 수 있습니다.

$$\begin{aligned}\mathbf{p}(u, v) &= \sum_{i=0}^3 B_i^3(v) \mathbf{q}_i(u) \\ &= \sum_{i=0}^3 B_i^3(v) \sum_{j=0}^3 B_j^3(u) \mathbf{p}_{i,j}\end{aligned}$$

베지어 곡면의 편미분은 접선 벡터와 법선 벡터를 계산하는 데 유용합니다.

$$\begin{aligned}\frac{\partial \mathbf{p}}{\partial u}(u, v) &= \sum_{i=0}^3 B_i^3(v) \sum_{j=0}^3 \frac{\partial B_j^3}{\partial u}(u) \mathbf{p}_{i,j} \\ \frac{\partial \mathbf{p}}{\partial v}(u, v) &= \sum_{i=0}^3 \frac{\partial B_i^3}{\partial v}(v) \sum_{j=0}^3 B_j^3(u) \mathbf{p}_{i,j}\end{aligned}$$

13.6.3 3차 베지어 곡면 계산 코드

이 섹션에서는 3차 베지어 표면을 평가하는 코드를 제공합니다. 이어지는 코드를 이해하는 데 도움이 되도록 함께 기호를 전개해 보겠습니다.

$$\begin{aligned}\mathbf{q}_0(u) &= B_0^3(u) \mathbf{p}_{0,0} + B_1^3(u) \mathbf{p}_{0,1} + B_2^3(u) \mathbf{p}_{0,2} + B_3^3(u) \mathbf{p}_{0,3} \\ \mathbf{q}_1(u) &= B_0^3(u) \mathbf{p}_{1,0} + B_1^3(u) \mathbf{p}_{1,1} + B_2^3(u) \mathbf{p}_{1,2} + B_3^3(u) \mathbf{p}_{1,3} \\ \mathbf{q}_2(u) &= B_0^3(u) \mathbf{p}_{2,0} + B_1^3(u) \mathbf{p}_{2,1} + B_2^3(u) \mathbf{p}_{2,2} + B_3^3(u) \mathbf{p}_{2,3} \\ \mathbf{q}_3(u) &= B_0^3(u) \mathbf{p}_{3,0} + B_1^3(u) \mathbf{p}_{3,1} + B_2^3(u) \mathbf{p}_{3,2} + B_3^3(u) \mathbf{p}_{3,3} \\ \mathbf{p}(u, v) &= B_0^3(v) \mathbf{q}_0(u) + B_1^3(v) \mathbf{q}_1(u) + B_2^3(v) \mathbf{q}_2(u) + B_3^3(v) \mathbf{q}_3(u) \\ &= B_0^3(v) \left[B_0^3(u) \mathbf{p}_{0,0} + B_1^3(u) \mathbf{p}_{0,1} + B_2^3(u) \mathbf{p}_{0,2} + B_3^3(u) \mathbf{p}_{0,3} \right] \\ &\quad + B_1^3(v) \left[B_0^3(u) \mathbf{p}_{1,0} + B_1^3(u) \mathbf{p}_{1,1} + B_2^3(u) \mathbf{p}_{1,2} + B_3^3(u) \mathbf{p}_{1,3} \right] \\ &\quad + B_2^3(v) \left[B_0^3(u) \mathbf{p}_{2,0} + B_1^3(u) \mathbf{p}_{2,1} + B_2^3(u) \mathbf{p}_{2,2} + B_3^3(u) \mathbf{p}_{2,3} \right] \\ &\quad + B_3^3(v) \left[B_0^3(u) \mathbf{p}_{3,0} + B_1^3(u) \mathbf{p}_{3,1} + B_2^3(u) \mathbf{p}_{3,2} + B_3^3(u) \mathbf{p}_{3,3} \right]\end{aligned}$$

다음 코드는 방금 제시된 공식에 직접 대응합니다.

```
float4 BernsteinBasis(float t)
{
    float invT = 1.0f - t;

    return float4(invT * invT * invT, // B_0^3(t) = (1-t)^3
                  3.0f * t * invT * invT, // B_1^3(t) = 3t(1-t)^2
                  3.0f * t * t * invT, // B_2^3(t) = 3t^2(1-t)
                  t * t * t); // B_3^3(t) = t^3

float4 dBernsteinBasis(float t)
{
    float invT = 1.0f - t;
```

```
return float4(
    -3 * invT * invT, // B_0^{3'}(t) = -3(1-t)^2
    3 * invT * invT - 6 * t * invT, // B_1^{3'}(t) = 3(1-t)^2 - 6t(1-t)
    3 * t * t, // B_2^{3'}(t) = 6t(1-t) - 3t^2
    3 * t * t); // B_3^{3'}(t) = 3t^2
}
```

float3 CubicBezierSum(const OutputPatch<HullOut, 16> bezpatch, float4 basisU, float4 basisV)

```
{
    float3 sum = float3(0.0f, 0.0f, 0.0f);
    sum = basisV.x * (basisU.x*bezpatch[0].PosL +
                     basisU.y*bezpatch[1].PosL +
                     basisU.z*bezpatch[2].PosL +
                     basisU.w*bezpatch[3].PosL);

    sum += basisV.y * (basisU.x*bezpatch[4].PosL +
                      basisU.y*bezpatch[5].PosL +
                      basisU.z*bezpatch[6].PosL +
                      basisU.w*bezpatch[7].PosL);

    sum += basisV.z * (basisU.x*bezpatch[8].PosL +
                      basisU.y*bezpatch[9].PosL +
                      basisU.z*bezpatch[10].PosL +
                      basisU.w*bezpatch[11].PosL);

    sum += basisV.w * (basisU.x*bezpatch[12].PosL +
                      basisU.y*bezpatch[13].PosL +
                      basisU.z*bezpatch[14].PosL +
                      basisU.w*bezpatch[15].PosL);

    return sum;
}
```

$p(\mathcal{U}, \mathcal{V})$ 를 평가하고 편미분을 계산하기 위해 이전 함수들은 다음과 같이 활용할 수 있습니다:

```
float4 basisU = BernsteinBasis(uv.x); float4 basisV =
BernsteinBasis(uv.y);

// p(u, v)
float3 p = CubicBezierSum(bezPatch, basisU, basisV);

float4 dBasisU = dBernsteinBasis(uv.x); float4 dBasisV =
dBernsteinBasis(uv.y);

// \frac{\partial \mathbf{p}}{\partial v}(\mathcal{U}, \mathcal{V})

float3 dpdu = CubicBezierSum(bezPatch, dbasisU, basisV);

// \frac{\partial \mathbf{p}}{\partial v}(\mathcal{U}, \mathcal{V})

float3 dpdv = CubicBezierSum(bezPatch, basisU, dbasisV);
```

 평가된 기저 함수 값을 CubicBezierSum에 전달하는 것을 주목하십시오. 이를 통해 CubicBezierSum을 사용하여 $p(u, v)$ 와 편미분 모두 평가하는 데 사용할 수 있게 합니다. 합계 형태는 동일하며, 유일한 차이는 기저 함수뿐입니다.

13.6.4 패치 기하 구조 정의

16개의 제어점을 저장하는 우리의 버텍스 버퍼는 다음과 같이 생성됩니다:

```

void BasicTessellation::BuildQuadPatchBuffer()
{
    D3D11_BUFFER_DESC vbd;
    vbd.Usage = D3D11_USAGE_IMMUTABLE;
    vbd.ByteWidth = sizeof(XMFLOAT3) * 16; vbd.BindFlags =
    D3D11_BIND_VERTEX_BUFFER; vbd.CPUAccessFlags = 0;
    vbd.MiscFlags = 0; XMFLOAT3
    vertices[16] =
    {

        // 행 0
        XMFLOAT3(-10.0f, -10.0f, +15.0f),
        XMFLOAT3(-5.0f,      0.0f,      +15.0f),
        XMFLOAT3(+5.0f,      0.0f,      +15.0f),
        XMFLOAT3(+10.0f, 0.0f, +15.0f),

        // 행 1
        XMFLOAT3(-15.0f, 0.0f, +5.0f),
        XMFLOAT3(-5.0f, 0.0f, +5.0f), XMFLOAT3(+5.0f, 20.0f,
        +5.0f), XMFLOAT3(+15.0f, 0.0f, +5.0f),

        // 2행
        XMFLOAT3(-15.0f, 0.0f, -5.0f),
        XMFLOAT3(-5.0f, 0.0f, -5.0f),
        XMFLOAT3(+5.0f, 0.0f, -5.0f),
        XMFLOAT3(+15.0f, 0.0f, -5.0f),

        // 3행
        XMFLOAT3(-10.0f, 10.0f, -15.0f),
        XMFLOAT3(-5.0f, 0.0f, -15.0f),
        XMFLOAT3(+5.0f, 0.0f, -15.0f), XMFLOAT3(+25.0f, 10.0f,
        -15.0f)
    };

    D3D11_SUBRESOURCE_DATA vinitData; vinitData.pSysMem
    = vertices;
    HR(md3dDevice->CreateBuffer(&vbd, &vinitData, &mQuadPatchVB));
}

```

 제어점이 균일한 격자를 형성하기 위해 반드시 균등한 간격을 유지해야 한다는 제한은 없습니다.

쿼드 패치를 IA 단계로 제출하는 것은 다음과 같이 간단히 수행됩니다.

```

md3dImmediateContext->IASetPrimitiveTopology(
    D3D11_PRIMITIVE_TOPOLOGY_16_CONTROL_POINT_PATCHLIST);

UINT stride = sizeof(Vertex::Pos); UINT offset = 0;

md3dImmediateContext->IASetVertexBuffers( 0, 1,
    &mQuadPatchVB, &stride, &offset);

md3dImmediateContext->Draw(16, 0);

```

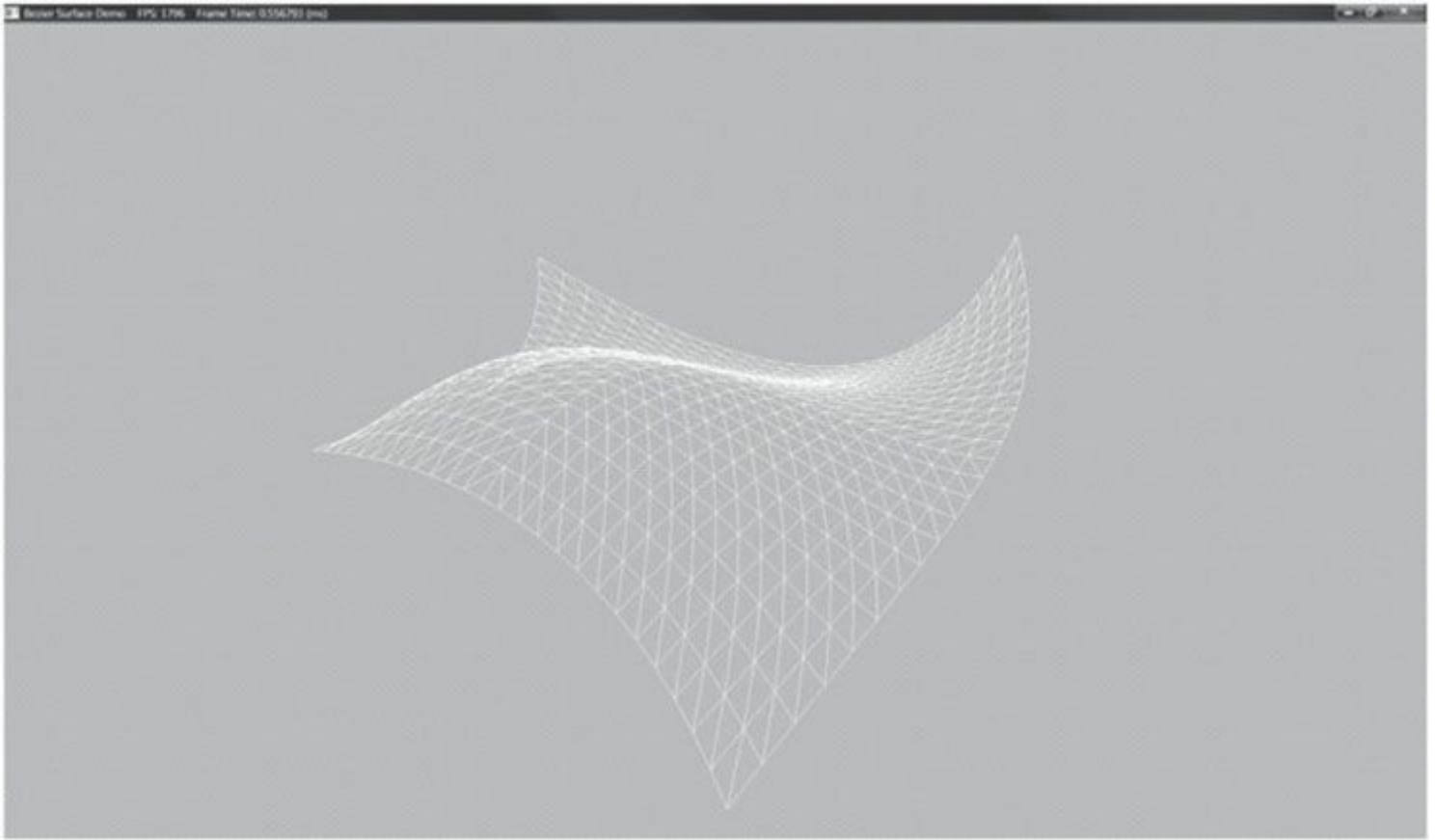



그림 13.8. 베지어 표면 데모의 스크린샷.

13.7 요약

1. 테셀레이션 단계는 렌더링 파이프라인의 선택적 단계입니다. 이 단계는 헬 셰이더, 테셀레이터, 도메인 셰이더로 구성됩니다. 헬 셰이더와 도메인 셰이더는 프로그래밍 가능하며, 테셀레이터는 하드웨어에 의해 완전히 제어됩니다.
2. 하드웨어 테셀레이션은 메모리 이점을 제공합니다. 저폴리곤(low-poly) 자산을 저장한 후 테셀레이션을 통해 실시간으로 디테일을 추가할 수 있기 때문입니다. 또한 애니메이션이나 물리 계산과 같은 연산은 테셀레이션 전 저폴리곤 메시 주파수에서 수행될 수 있습니다. 마지막으로, 연속적인 LOD 알고리즘을 이제 GPU에서 완전히 구현할 수 있게 되었습니다. 하드웨어 테셀레이션이 등장하기 전에는 항상 CPU에서 구현해야 했습니다.
3. 새로운 프리미티브 유형은 테셀레이션을 통해서만 렌더링 파이프라인에 제어점을 제출하는 데 사용됩니다. Direct3D 11은 1개에서 32개 사이의 제어점을 지원하며, 이는 열거형 유형 `D3D11_PRIMITIVE_1_CONTROL_POINT_PATCH...` `D3D11_PRIMITIVE_32_CONTROL_POINT_PATCH`로 표현됩니다.
4. 테셀레이션에서는 버텍스 셰이더가 제어점을 입력으로 받아 일반적으로 제어점별로 애니메이션을 적용하거나 물리 계산을 수행합니다. 점별로 애니메이션을 적용하거나 물리 계산을 수행합니다. 헬 셰이더는 상수 헬 셰이더와 제어점 헬 셰이더로 구성됩니다. 상수 헬 셰이더는 패치별로 작동하여 패치의 테셀레이션 계수를 출력합니다. 이 계수는 테셀레이터에게 패치를 얼마나 테셀레이션할지 지시하며, 기타 선택적 패치별 데이터도 포함할 수 있습니다. 제어점 헬 셰이더는 여러 제어점을 입력으로 받아 여러 제어점을 출력합니다. 제어점 선체 셰이더는 출력된 제어점 하나당 한 번씩 호출됩니다. 일반적으로 제어점 선체 셰이더는 입력 패치의 표면 표현을 변경합니다. 예를 들어, 이 단계는 세 개의 제어점을 가진 삼각형을 입력으로 받아 열 개의 제어점을 가진 베지어 삼각형 표면 패치를 출력할 수 있습니다.
5. 도메인 셰이더는 테셀레이션 단계에서 생성된 각 버텍스에 대해 호출됩니다. 버텍스 셰이더가 각 제어점에 대해 버텍스 셰이더 역할을 하는 반면, 테셀레이션이 활성화된 상태에서는 헬 셰이더가 본질적으로 테셀레이션된 패치 버텍스에 대한 버텍스 셰이더 역할을 합니다. 특히, 테셀레이션된 패치의 버텍스를 동질 클립 공간으로 투영하고 기타 버텍스별 작업을 수행하는 곳이 바로 여기입니다.
6. 테셀레이션을 적용하지 않을 객체(예: 테셀레이션 계수가 1에 가까운 경우)는 오버헤드가 발생하므로 테셀레이션 단계를 활성화한 상태로 렌더링하지 마십시오. 삼각형 크기가 8픽셀보다 작아질 정도로 과도한 테셀레이션을 피하십시오. 프레임 내 테셀레이션 켜고 끄기를 방지하려면 테셀레이션된 모든 객체를 함께 렌더링하십시오. 허울 셰이더에서 후면 제거(back face culling)와 시야 절단(frustum culling)을 사용하여 테셀레이션된 패치 중 보이지 않는 부분을 제거하십시오.
7. 베지어 곡선과 표면은 매끄러운 표면을 설명하는 매개변수 방정식입니다. 제어점을 통해 "형태"를 조정합니다. 베지어 표면은 매끄러운 표면을 직접 그릴 수 있게 해줄 뿐만 아니라, PN 삼각형 및 캣멀-클라크 근사법과 같은 많은 인기 있는 하드웨어 테셀레이션 알고리즘에 사용됩니다.

13.8 연습 문제

1. "기본 테셀레이션" 데모를 재현하되, 사각 패치 대신 삼각 패치를 테셀레이션하세요.
2. 거리 기반 테셀레이션으로 정육면체를 구로 분할하십시오.
3. "기본 테셀레이션" 데모를 수정하여 평평한 사각형을 고정 테셀레이션하도록 하십시오. 테셀레이션 요소가 작동하는 방식을 이해할 때까지 다양한 모서리/내부 테셀레이션 요소를 실험해 보십시오.
4. 분수 테셀레이션을 탐구하십시오. 즉, "기본 테셀레이션" 데모에서 다음을 시도하십시오:

[partitioning("fractional_even")] [partitioning("fractional_odd")]

5. 2차 베지어 곡선에 대한 번스타인 기저 함수 $B_0^2(t), B_1^2(t), B_2^2(t)$ 를 계산하고, 그 미분값을 구하세요. $B_0^{2'}(t), B_1^{2'}(t), B_2^{2'}(t)$. 2차 베지어 곡면의 매개변수 방정식을 유도하세요.
6. 제어점을 변경하여 베지어 곡면을 변형시키는 "베지어 패치" 데모를 실험해 보십시오.
7. "베지어 패치" 데모를 수정하여 9개의 제어점을 가진 2차 베지어 표면을 사용하도록 하십시오.
8. "베지어 패치" 데모를 수정하여 베지어 표면에 명암을 표현하십시오. 도메인 셰이더에서 정점 법선을 계산해야 합니다. 정점 위치에서의 법선은 해당 위치에서 편미분의 외적 연산을 통해 구할 수 있습니다.
9. 베지어 삼각형 패치를 연구하고 구현하십시오.

파트 3 주제

이 부분에서는 Direct3D를 적용하여 여러 3D 애플리케이션을 구현하는 데 중점을 두고, 지형 렌더링, 하늘 렌더링, 메시 작업, 캐릭터 애니메이션, 파티클 시스템, 피킹, 환경 매핑 및 노멀 맵핑과 같은 기술을 시연합니다.

매핑, 색도 매핑 등의 기법을 시연합니다. 이 부분의 각 장에 대한 간략한 설명은 다음과 같습니다.

제14장, 1인치 카메라 구축: 이 장에서는 1인치 게임에서 기대하는 대로 동작하는 카메라 시스템을 설계하는 방법을 보여줍니다. 키보드와 마우스 입력을 통해 카메라를 제어하는 방법을 설명합니다.

제15장, 인스턴싱과 프러스텀 컬링: 인스턴싱은 동일한 지오메트리를 서로 다른 속성(예: 장면 내 서로 다른 위치와 색상)으로 여러 번 그리는 작업을 최적화하는 하드웨어 지원 기술입니다. 프러스텀 컬링은 가상 카메라의 시야 범위 밖에 완전히 위치한 객체 전체를 렌더링 파이프라인에 제출되지 않도록 제거하는 최적화 기법입니다. 또한 메시의 바운딩 박스와 구를 계산하는 방법도 보여줍니다.

제16장, 피킹: 이 장에서는 사용자가 마우스로 선택한 특정 3D 객체(또는 3D 프리미티브)를 결정하는 방법을 보여줍니다. 피킹은 사용자가 마우스로 3D 세계와 상호작용하는 3D 게임 및 애플리케이션에서 종종 필수적입니다.

17장, 큐브 매핑: 이 장에서는 환경 매핑을 사용하여 임의의 메시에 환경을 반영하는 방법을 보여줍니다. 또한 환경 맵을 사용하여 스카이 스피어에 텍스처를 적용합니다.

제18장, 노멀 매핑과 변위 매핑: 이 장에서는 노멀 맵을 사용하여 상세한 실시간 조명 효과를 구현하는 방법을 보여줍니다. 또한 테셀레이션과 변위 맵을 결합하여 오브젝트의 기하학적 디테일을 높이는 방법을 설명합니다.

제19장, 지형 렌더링: 이 장에서는 높이맵, 테셀레이션, 멀티 텍스처링 기법을 활용하여 3D 지형을 생성, 텍스처링, 조명 처리 및 렌더링하는 방법을 보여줍니다. 또한 카메라를 지형 위에서 부드럽게 "건계" 하는 방법을 설명합니다.

제20장, 파티클 시스템과 스트림-아웃: 이 장에서는 유사한 방식으로 행동하는 수많은 작은 입자로 구성된 시스템을 모델링하는 방법을 배웁니다. 예를 들어, 파티클 시스템은 눈과 비가 내리는 모습, 불과 연기, 로켓의 비행 흔적, 스프링클러, 분수 등을 모델링하는 데 사용할 수 있습니다.

제21장, 색도 매핑: 색도 매핑은 임의의 기하학적 구조(평면 그림자에 국한되지 않음)에 그림자를 투사하는 실시간 그림자 기술입니다. 또한 투영 텍스처링의 작동 원리를 학습합니다.

제22장, 앰비언트 오클루전: 조명은 장면을 사실적으로 보이게 하는 데 중요한 역할을 합니다. 이 장에서는 장면 내 특정 지점이 들어오는 빛으로부터 얼마나 가려지는지 추정하여 조명 방정식의 앰비언트 항을 개선합니다.

제23장, 메쉬: 이 장에서는 복잡한 모델을 파일에서 불러오는 방법과 메쉬 작업을 위한 간단한 클래스 집합을 설계하는 방법을 보여줍니다.

제24장, 쿼터니언: 이 장에서는 쿼터니언이라는 수학적 객체를 연구합니다. 단위 쿼터니언이 회전을 나타내며 간단한 방법으로 보간될 수 있음을 보여줌으로써 회전 보간 방법을 제시합니다. 회전을 보간할 수 있게 되면 3D 애니메이션을 제작할 수 있습니다.

제25장, 캐릭터 애니메이션: 이 장에서는 "메쉬" 장을 확장하여 파일에서 복잡한 모델을 로드하지만, 애니메이션 데이터가 포함된 모델을 다룹니다. 캐릭터 애니메이션의 이론을 다루고, 복잡한 건기 애니메이션을 가진 전형적인 인간 게임 캐릭터를 애니메이션화하는 방법을 보여줍니다.

이 짧은 장에서는 1인칭 게임에서 기대하는 대로 동작하는 카메라 시스템을 설계합니다. 이 카메라 시스템은 지금까지 데모에서 사용해 온 궤도 카메라 시스템을 대체할 것입니다.

목표:

1. 시점 공간 변환의 수학적 원리를 복습한다.
2. 1인칭 카메라의 일반적인 기능을 식별할 수 있도록 한다.
3. 1인칭 카메라 구현 방법을 학습한다.

14.1 뷰 변환 복습

뷰 스페이스는 [그림 14.1](#)과 같이 카메라에 부착된 좌표계입니다. 카메라는 원점에 위치하여 양의 z 축을 따라 아래를 향하고, x 축은 카메라 오른쪽을 향하며, y 축은 카메라 위를 향합니다. 렌더링 파이프라인의 후속 단계에서는 장면의 정점을 월드 스페이스에 상대적으로 설명하기보다 카메라 좌표계에 상대적으로 설명하는 것이 편리합니다. 월드 공간에서 뷰 공간으로의 좌표 변환을 *뷰 변환*이라고 하며, 이에 대응하는 행렬을 *뷰 행렬*이라고 합니다.

$\mathbf{Q}_w = (Q_x, Q_y, Q_z, 1)$ 일 때, $\mathbf{u}_w = (u_x, u_y, u_z, 0)$, $\mathbf{v}_w = (v_x, v_y, v_z, 0)$, $\mathbf{w}_w = (w_x, w_y, w_z, 0)$ 가 각각 시점 공간의 원점, x -, y -, z -축을 각각 세계 좌표계에 대한 동차 좌표로 나타낸다면, §3.4.3에서 알 수 있듯이 뷰 공간에서 세계 공간으로의 좌표 변환 행렬은 다음과 같습니다:

$$\mathbf{W} = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ w_x & w_y & w_z & 0 \\ Q_x & Q_y & Q_z & 1 \end{bmatrix}$$

그러나 이는 우리가 원하는 변환이 아닙니다. 우리는 월드 공간에서 뷰 공간으로의 역변환을 원합니다. 하지만 §3.4.5에서 본 바와 같이 역변환은 단순히 역행렬로 주어집니다. 따라서 $\mathbf{W}^{-1}$ 은 월드 공간에서 뷰 공간으로 변환합니다.

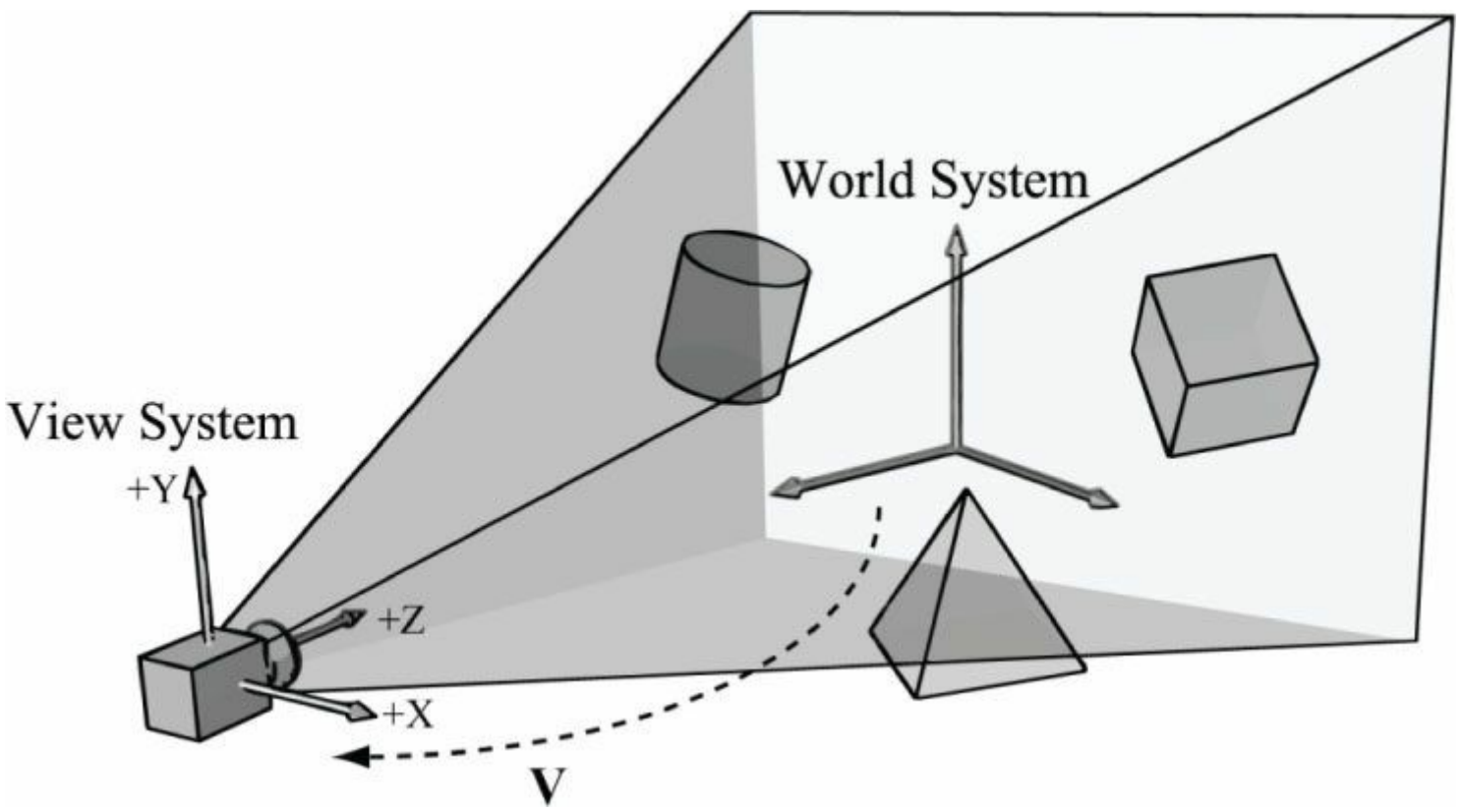


그림 14.1. 카메라 좌표계. 카메라 자체 좌표계에 대해, 카메라는 원점에 위치하며 양의 z축 방향을 향하고 있다.

일반적으로 세계 좌표계와 뷰 좌표계는 위치와 방향만 다르기 때문에, $\mathbf{W} = \mathbf{RT}$ (즉, 세계 행렬을 회전과 평행 이동으로 분해할 수 있음)라는 형태가 직관적으로 이해됩니다. 이 형태는 역행렬을 더 쉽게 계산할 수 있게 합니다:

$$\mathbf{V} = \mathbf{W}^{-1} = (\mathbf{RT})^{-1} = \mathbf{T}^{-1} \mathbf{R}^{-1} = \mathbf{T}^{-1} \mathbf{R}^T$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -Q_x & -Q_y & -Q_z & 1 \end{bmatrix} \begin{bmatrix} u_x & v_x & w_x & 0 \\ u_y & v_y & w_y & 0 \\ u_z & v_z & w_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} u_x & v_x & w_x & 0 \\ u_y & v_y & w_y & 0 \\ u_z & v_z & w_z & 0 \\ -Q \cdot \mathbf{u} & -Q \cdot \mathbf{v} & -Q \cdot \mathbf{w} & 1 \end{bmatrix}$$

따라서 뷰 행렬은 다음과 같은 형태를 가집니다:

$$\mathbf{V} = \begin{bmatrix} u_x & v_x & w_x & 0 \\ u_y & v_y & w_y & 0 \\ u_z & v_z & w_z & 0 \\ -Q \cdot \mathbf{u} & -Q \cdot \mathbf{v} & -Q \cdot \mathbf{w} & 1 \end{bmatrix} \quad (\text{eq. 14.1})$$

모든 좌표계 변환과 마찬가지로, 장면 내의 어떤 것도 이동시키지 않습니다. 좌표가 변하는 이유는 세계 좌표계 참조 대신 카메라 공간 좌표계 참조를 사용하기 때문입니다.

14.2 카메라 클래스

카메라 관련 코드를 캡슐화하기 위해 **Camera** 클래스를 정의하고 구현합니다. 카메라 클래스의 데이터는 두 가지 핵심 정보를 저장합니다. 카메라의 *위치*, *오른쪽*, *위쪽*, *바라보는* 벡터로, 각각 월드 좌표계에서 뷰 공간 좌표계의 원점, x축, y축, z축을 정의하며, 프러스텀의 속성을 나타냅니다. 카메라 렌즈를

카메라는 시야(시야각 및 근거리/원거리 평면)를 정의하는 원뿔형 시야를 갖습니다. 대부분의 메서드는 단순합니다(예: 기본 접근 메서드). 메서드와 데이터 멤버 개요는 아래 주석을 참조하십시오.
다음 섹션에서 선택된 메서드를 검토합니다.

```
class Camera
{
public:
    Camera();
    ~Camera();

    // 월드 카메라 위치 가져오기/설정. XMVECTOR
    GetPositionXM()const; XMFLOAT3 GetPosition()const;
    void SetPosition(float x, float y, float z); void
    SetPosition(const XMFLOAT3& v);

    // 카메라 기준 벡터 가져오기. XMVECTOR
    GetRightXM()const; XMFLOAT3 GetRight()const;
    XMVECTOR GetUpXM()const; XMFLOAT3
    GetUp()const; XMVECTOR GetLook XM()const;

    XMFLOAT3 GetLook()const;
    // 프러스텀 속성 가져오기. float
    GetNearZ()const; float
    GetFarZ()const; float
    GetAspect()const; float
    GetFovY()const; float
    GetFovX()const;

    // 뷰 공간 좌표계의 근거리 및 원거리 평면 치수 가져오기. float
    GetNearWindowWidth()const;
    float GetNearWindowHeight()const; float
    GetFarWindowWidth()const; float
    GetFarWindowHeight()const;

    // 프러스텀 설정.
    void SetLens(float fovY, float aspect, float zn, float zf);

    // Look At 매개변수를 통해 카메라 공간 정의.
    void Look At(FXMVECTOR pos, FXMVECTOR target, FXMVECTOR worldUp); void Look At(const
    XMFLOAT3& pos, const XMFLOAT3& target,
        const XMFLOAT3& up);

    // 뷰/프로젝션 행렬을 가져옵니다.
    XMMATRIX View()const; XMMATRIX
    Proj()const; XMMATRIX ViewProj()const;

    // 카메라를 거리 d만큼 측면 이동/걷기. void Strafe(float d);
    void Walk(float d);

    // 카메라 회전. void Pitch(float
    angle);
    void RotateY(float angle);

    // 카메라 위치/방향 수정 후 호출
    // 프레임마다 뷰 매트릭스를 재구성합니다. void
    UpdateViewMatrix();

private:
```

```
// 월드 공간을 기준으로 한 카메라 좌표계. XMFLOAT3 mPosition; // 뷰 공간 원점
XMFLOAT3 mRight;           // 뷰 공간 x축 XMFLOAT3
mUp;                       // 뷰 공간 y축 XMFLOAT3
mLook;                     // 뷰 공간 z축

// 캐시 프러스텀 속성. float mNearZ;
float mFarZ; float
mAspect; float
mFovY;
float mNearWindowHeight; float
mFarWindowHeight;

// 뷰/프로젝션 행렬 캐시. XMFLOAT4X4
mView; XMFLOAT4X4 mProj;

};
```

 Camera.h/Camera.cpp 파일은 Common 디렉터리에 있습니다.

14.3 선택된 메서드 구현

카메라 클래스의 많은 메서드는 사소한 `get/set` 메서드이므로 여기서는 생략하겠습니다. 그러나 이 섹션에서는 몇 가지 중요한 메서드를 살펴보겠습니다.

14.3.1 XMVECTOR 반환 변형

먼저, 많은 "get" 메서드에 대해 **XMVECTOR** 반환 변형을 제공한다는 점을 언급하고자 합니다. 이는 단순히 편의성을 위한 것으로, 클라이언트 코드가 **XMVECTOR**가 필요한 경우 변환할 필요가 없도록 하기 위함입니다:

```
XMVECTOR Camera::GetPositionXM()const
{
    return XMLoadFloat3(&mPosition);
}

XMFLOAT3 Camera::GetPosition()const
{
    mPosition 반환;
}
```

14.3.2 SetLens

프러스텀은 카메라의 렌즈로 생각할 수 있습니다. 시야를 제어하기 때문입니다. 프러스텀 속성을 캐싱하고 **SetLens** 메서드를 사용하여 투영 행렬을 구축합니다:

```
void Camera::SetLens(float fovY, float aspect, float zn, float zf)
{
    // 속성 캐시 mFovY = fovY;
    mAspect = aspect;

    mNearZ = zn; mFarZ =
    zf;

    mNearWindowHeight = 2.0f * mNearZ * tanf(0.5f*mFovY); mFarWindowHeight = 2.0f * mFarZ
    * tanf(0.5f*mFovY);

    XMATRIX P = XMMatrixPerspectiveFovLH(mFovY, mAspect, mNearZ, mFarZ);
```

```
XMStoreFloat4x4(&mProj, P);
```

```
}
```

14.3.3 파생된 프러스텀 정보

방금 살펴본 바와 같이, 우리는 수직 시야각을 캐싱하지만, 추가로 수평 시야각을 도출하는 메서드를 제공합니다. 또한, 가끔 유용하게 알 수 있는 근거리 및 원거리 평면에서의 프러스텀 너비와 높이를 반환하는 메서드도 제공합니다. 이러한 메서드들의 구현은 단순히 삼각법이며, 방정식을 따라가는 데 어려움이 있다면 §5.6.3을 복습하십시오:

```
float Camera::GetFovX()const
{
    // 0.5f는 수평 시야각의 절반을 반환합니다.
    float halfWidth = 0.5f*GetNearWindowWidth(); return
    2.0f*atan(halfWidth / mNearZ);
}

float Camera::GetNearWindowWidth()const
{
    return mAspect * mNearWindowHeight;
}

float Camera::GetNearWindowHeight()const
{
    return mNearWindowHeight;
}

float Camera::GetFarWindowWidth()const
{
    return mAspect * mFarWindowHeight;
}

float Camera::GetFarWindowHeight()const
{
    return mFarWindowHeight;
}
```

14.3.4 카메라 변환

1인칭 카메라의 경우 충돌 감지를 무시하고 다음을 수행할 수 있어야 합니다:

1. 카메라를 바라보는 방향 벡터(look vector)를 따라 전진 및 후진 이동. 카메라 위치를 바라보는 방향 벡터 방향으로 이동(translate)하여 구현할 수 있습니다.
2. 오른쪽 벡터를 따라 카메라를 이동시켜 좌우로 이동합니다. 이는 카메라 위치를 오른쪽 벡터 방향으로 이동시켜 구현할 수 있습니다.
3. 카메라를 오른쪽 벡터 주위로 회전시켜 위아래를 바라보게 합니다. 이는 **XMMatrixRotationAxis** 함수를 사용하여 카메라의 바라보기 벡터와 위쪽 벡터를 오른쪽 벡터 주위로 회전시켜 구현할 수 있습니다.
4. 카메라를 월드 y축(y축이 월드의 "위" 방향에 해당한다고 가정) 벡터 주위로 회전시켜 좌우를 바라보게 합니다. 이는 **XMMatrixRotationY** 함수를 사용하여 모든 기본 벡터를 월드 y축 주위로 회전시켜 구현할 수 있습니다.

```
void Camera::Walk(float d)
{
    // mPosition += d*mLook
    XMVECTOR s = XMVectorReplicate(d); XMVECTOR l =
    XMLoadFloat3(&mLook); XMVECTOR p =
    XMLoadFloat3(&mPosition);
    XMStoreFloat3(&mPosition, XMVectorMultiplyAdd(s, l, p));
}

void Camera::Strafe(float d)
{
    // mPosition += d*mRight
    XMVECTOR s = XMVectorReplicate(d); XMVECTOR r =
    XMLoadFloat3(&mRight); XMVECTOR p =
    XMLoadFloat3(&mPosition);
    XMStoreFloat3(&mPosition, XMVectorMultiplyAdd(s, r, p));
}
```



```

// mPosition += d*mRight
XMVECTOR s = XMVectorReplicate(d); XMVECTOR r =
XMLoadFloat3(&mRight); XMVECTOR p =
XMLoadFloat3(&mPosition);
XMStoreFloat3(&mPosition, XMVectorMultiplyAdd(s, r, p));
}

void Camera::Pitch(float angle)
{

    // 오른쪽 벡터를 중심으로 벡터를 위로 회전합니다.

    XMMATRIX R = XMMatrixRotationAxis(XMLoadFloat3(&mRight), angle);

    XMStoreFloat3(&mUp, XMVector3TransformNormal(XMLoadFloat3(&mUp), R)); XMStoreFloat3(&mLook,
    XMVector3TransformNormal(XMLoadFloat3(&mLook), R));
}
void Camera::RotateY(float angle)
{

    // 세계 y축을 중심으로 기준 벡터를 회전합니다. XMMATRIX R =

    XMMatrixRotationY(angle);

    XMStoreFloat3(&mRight, XMVector3TransformNormal( XMLoadFloat3(&mRight), R)); XMStoreFloat3(&mUp,
    XMVector3TransformNormal(XMLoadFloat3(&mUp), R)); XMStoreFloat3(&mLook,
    XMVector3TransformNormal(XMLoadFloat3(&mLook), R));
}

```

14.3.5 뷰 매트릭스 구축

UpdateViewMatrix 메서드의 첫 번째 부분은 카메라의 오른쪽, 위쪽, 그리고 바라보는 방향 벡터를 *재정규화합니다*. 즉, 이 벡터들이 서로 직교하고 길이가 1이 되도록 보장합니다. 여러 번의 회전 후에는 수치 오류가 누적되어 이 벡터들이 직교하지 않게 될 수 있기 때문에 이 작업이 필요합니다. 이런 상황이 발생하면 벡터들은 더 이상 직교 좌표계를 나타내지 않고 왜곡된 좌표계를 나타내게 되는데, 이는 우리가 원하는 바가 아닙니다. 이 메서드의 두 번째 부분은 카메라 벡터들을 방정식 14.1에 대입하여 뷰 변환 행렬을 계산합니다.

```

void Camera:: UpdateViewMatrix()
{
    XMVECTOR R = XMLoadFloat3(&mRight); XMVECTOR U =
XMLoadFloat3(&mUp); XMVECTOR L = XMLoadFloat3(&mLook);
XMVECTOR P = XMLoadFloat3(&mPosition);

    //

    // 오른쪽, 위쪽, 바라보는 방향 벡터를 직교 정규화합니다.
    //

    // 시선 벡터의 길이를 1로 만듭니다. L =
XMVector3Normalize(L);

    // 수정된 새로운 "위쪽" 벡터를 계산하고 정규화합니다. U =
XMVector3Normalize(XMVector3Cross(L, R));

    // 새로 보정된 "오른쪽" 벡터를 계산합니다. U와 L은
    // 이미 직교 정규화되어 있으므로 외적 정규화가 필요 없음.
    // ||up × look|| = ||up|| ||look|| sin90° = 1 R =
XMVector3Cross(U, L);

    //

    // 뷰 매트릭스 항목을 채웁니다.
    //
    float x = -XMVectorGetX(XMVector3Dot(P, R));

```

```
float y = -XMVectorGetX(XMVector3Dot(P, U)); float z = -
XMVectorGetX(XMVector3Dot(P, L));

XMStoreFloat3(&mRight, R); XMStoreFloat3(&mUp, U);
XMStoreFloat3(&mLook, L);

mView(0,0)    =    mRight.x;
mView(1,0)    =    mRight.y;
mView(2,0)    =    mRight.z;
mView(3,0) = x;

mView(0,1)    =    mUp.x;
mView(1,1)    =    mUp.y;
mView(2,1)    =    mUp.z;
mView(3,1) = y;

mView(0,2)    =    mLook.x;
mView(1,2)    =    mLook.y;
mView(2,2)    =    mLook.z;
mView(3,2) = z;

mView(0,3) = 0.0f;
mView(1,3) = 0.0f;
mView(2,3) = 0.0f;
mView(3,3) = 1.0f;
}
```

14.4 카메라 데모 주석

이제 응용 프로그램 클래스에서 궤도 카메라 시스템과 관련된 모든 기존 변수(**mPhi**, **mTheta**, **mRadius**, **mView**, **mProj** 등)를 제거할 수 있습니다. 다음 멤버 변수를 추가하겠습니다:

카메라 mCam;

창 크기가 변경될 때 더 이상 투영 행렬을 명시적으로 재구성하지 않고, 대신 **Camera**

클래스의 **SetLens** 메서드에 위임합니다:

```
void CameraApp::OnResize()
{
    D3DApp::OnResize();

    mCam.SetLens(0.25f*MathHelper::Pi, AspectRatio(), 1.0f, 1000.0f);
}
```

UpdateScene 메서드에서 키보드 입력을 처리하여 카메라를 이동합니다:

```
void CameraApp::UpdateScene(float dt)
{
    //
    // 카메라 제어.
    //
    if(GetAsyncKeyState('W') & 0x8000)
        mCam.Walk(10.0f*dt);

    if(GetAsyncKeyState('S') & 0x8000)
        mCam.Walk(-10.0f*dt);

    if(GetAsyncKeyState('A') & 0x8000)
        mCam.Strafe(-10.0f*dt);
}
```

```
if(GetAsyncKeyState('D') & 0x8000)
    mCam.Strafe(10.0f*dt);
```

OnMouseMove 메서드에서 카메라의 시선 방향을 회전시킵니다:

```
void CameraApp::OnMouseMove(WPARAM btnState, int x, int y)
{
    if((btnState & MK_LBUTTON) != 0)
    {
        // 각 픽셀을 1/4도 단위로 대응시킵니다. float dx = XMConvertToRadians(
        0.25f*static_cast<float>(x - mLastMousePos.x)); float dy =
        XMConvertToRadians( 0.25f*static_cast<float>(y - mLastMousePos.y));

        mCam.Pitch(dy); mCam.Rotate Y(dx);
    }

    mLastMousePos.x = x; mLastMousePos.y = y;
}
```

마지막으로 렌더링을 위해 뷰 매트릭스와 투영 매트릭스는 카메라 인스턴스에서 접근할 수 있습니다:

```
mCam.UpdateViewMatrix();
```

```
XMMATRIX 뷰          = mCam.View(); XMMATRIX
proj                = mCam.Proj(); XMMATRIX
viewProj = mCam.ViewProj();
```

14.5 요약

1. 카메라 좌표계는 위치와 방향을 지정하여 정의합니다. 위치는 세계 좌표계에 대한 위치 벡터로 지정되며, 방향은 세계 좌표계에 대한 세 개의 직교 정규화 벡터(오른쪽, 위쪽, 바라보는 방향 벡터)로 지정됩니다. 카메라를 이동하는 것은 세계 좌표계에 대한 카메라 좌표계를 이동하는 것과 같습니다.

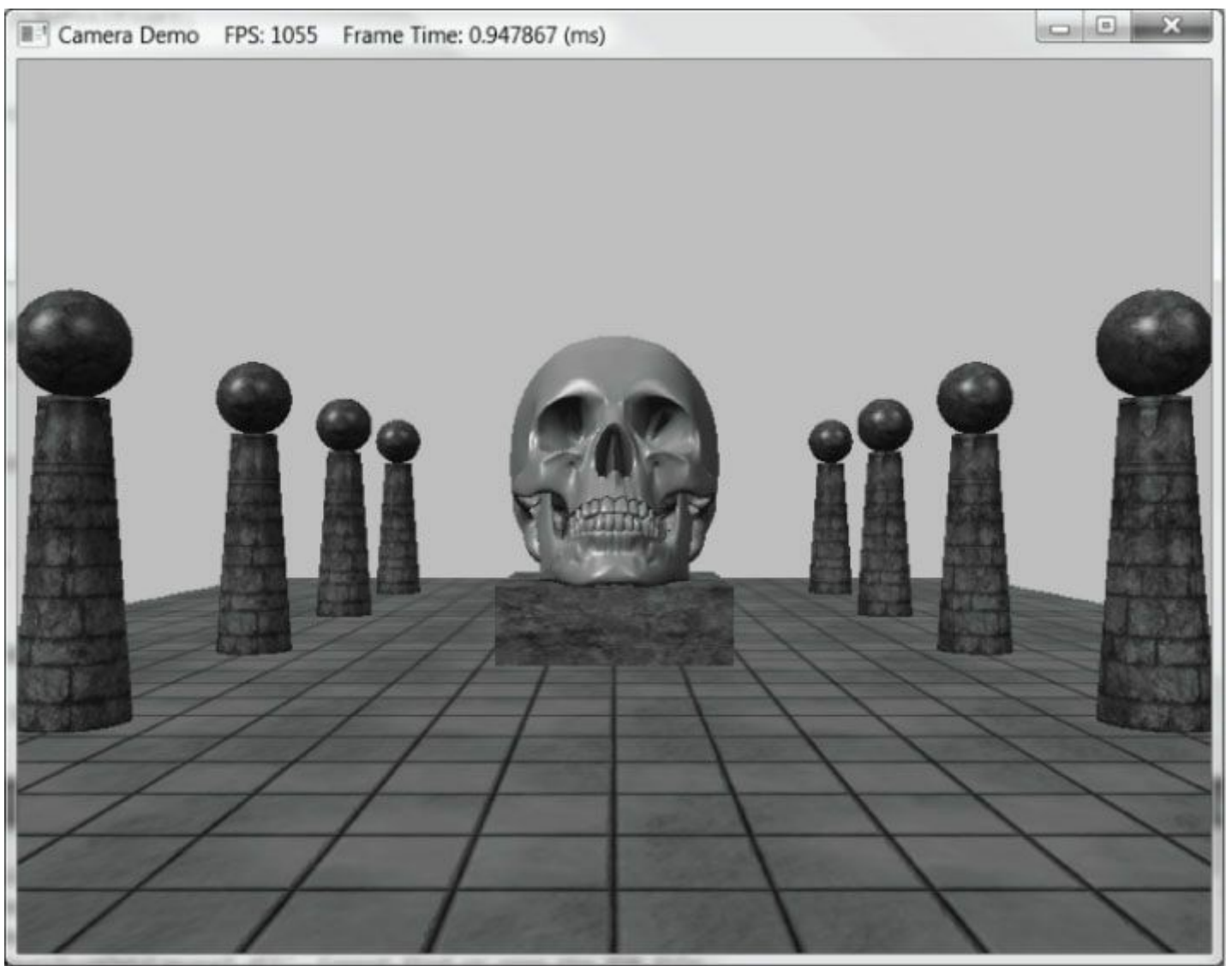


그림 14.2. 카메라 데모 화면 캡처. 'W', 'S', 'A', 'D' 키를 각각 앞으로 이동, 뒤로 이동, 좌측 이동, 우측 이동에 사용합니다. 마우스 왼쪽 버튼을 누른 상태에서 마우스를 움직여 다양한 방향으로 "조준"할 수 있습니다.

2. 카메라 클래스에 투영 관련 양을 포함시켰습니다. 왜냐하면 원근 투영 행렬은 시야각과 근거리 평면 및 원거리 평면을 제어함으로써 카메라의 '렌즈' 역할을 한다고 볼 수 있기 때문입니다.
3. 전진 및 후진은 카메라의 시선 벡터 방향으로 카메라 위치를 이동시키는 것으로 간단히 구현할 수 있습니다. 좌우로 이동하는 스트래핑은 카메라의 오른쪽 벡터 방향으로 카메라 위치를 이동시키는 것으로 간단히 구현할 수 있습니다. 위아래로 보는 것은 카메라의 시선 벡터와 위쪽 벡터를 오른쪽 벡터 주변으로 회전시켜 달성할 수 있습니다. 좌우로 보는 것은 모든 기본 벡터를 월드 좌표계의 y축 주변으로 회전시켜 구현할 수 있습니다.

14.6 연습문제

1. 세계 좌표계에서 세계 공간 축과 원점을 다음과 같이 정의한다: $\mathbf{i} = (1, 0, 0)$, $\mathbf{j} = (0, 1, 0)$, $\mathbf{k} = (0, 0, 1)$, $\mathbf{O} = (0, 0, 0)$. 또한 세계 좌표계에서 뷰 공간 축과 원점을 다음과 같이 정의한다: $\mathbf{u} = (u_x, u_y, u_z)$, $\mathbf{v} = (v_x, v_y, v_z)$, $\mathbf{w} = (w_x, w_y, w_z)$, 그리고 $\mathbf{Q} = (Q_x, Q_y, Q_z)$ 라고 할 때, 내적(dot product)을 사용하여 뷰 매트릭스 형태를 유도하십시오.

$$\mathbf{V} = \begin{bmatrix} u_x & v_x & w_x & 0 \\ u_y & v_y & w_y & 0 \\ u_z & v_z & w_z & 0 \\ -\mathbf{Q} \cdot \mathbf{u} & -\mathbf{Q} \cdot \mathbf{v} & -\mathbf{Q} \cdot \mathbf{w} & 1 \end{bmatrix}$$

내적을 사용하여 뷰 매트릭스 형태를 유도하십시오. (기억하세요, 월드 공간에서 뷰 공간으로의 좌표 변환 매트릭스를 구하려면 단순히

뷰 스페이스에 상대적인 좌표로 표현된 월드 스페이스 축과 원점. 이후 이 좌표들은 뷰 행렬의 행이 됩니다.)

2. 카메라 데모를 수정하여 "롤(roll)"을 지원하도록 합니다. 이는 카메라가 시선 벡터(look vector)를 중심으로 회전하는 기능입니다. 항공기 게임에 유용할 수 있습니다.

인스턴싱

챕터 15 및 프러스텀 컬링

이 장에서는 인스턴싱과 프러스텀 컬링이라는 두 가지 간단한 주제를 다룹니다. 인스턴싱은 동일한 객체를 장면 내에서 여러 번 그리는 것을 의미합니다. 최소한의 API 오버헤드로 인스턴싱을 구현할 수 있게 해주는 Direct3D 기능을 설명합니다. 프러스텀 컬링은 간단한 테스트를 통해 시야 프러스텀 밖에 있는 삼각형 그룹 전체를 추가 처리에서 제외하는 것을 의미합니다.

목표:

- 1. 하드웨어 인스턴싱 구현 방법을 학습한다.
- 2. 바운딩 볼륨의 개념, 유용성, 생성 방법 및 사용법을 익힌다.
- 3. 프러스텀 컬링 구현 방법을 알아본다.

15.1 하드웨어 인스턴싱

인스턴싱은 동일한 오브젝트를 장면 내에서 여러 번 그리는 것을 의미하지만, 위치, 방향, 크기, 재질, 텍스처가 서로 다릅니다(예: 나무 오브젝트를 여러 번 재사용하여 숲을 구성할 수 있음). 각 인스턴스에 대해 정점 및 인덱스 데이터를 복제하는 것은 비효율적입니다. 대신 오브젝트의 로컬 공간을 기준으로 한 단일 기하 구조(즉, 정점 및 인덱스 목록) 사본을 저장합니다. 그런 다음 오브젝트를 여러 번 렌더링하지만, 다양성이 필요할 경우 매번 다른 월드 매트릭스와 다른 재질을 적용합니다.

이 전략은 메모리를 절약하지만 여전히 객체별 API 오버헤드가 필요합니다. 즉, 각 객체마다 고유한 머티리얼과 월드 매트릭스를 설정하고 드로우 명령을 호출해야 합니다. Direct3D 10 이후 버전은 Direct3D 9에 존재하던 많은 API 오버헤드를 최소화하도록 재설계되었지만, 여전히 일부 오버헤드는 존재합니다. 따라서 Direct3D는 이러한 추가 API 오버헤드 없이 인스턴싱을 구현할 수 있는 메커니즘을 제공하며, 이를 *하드웨어 인스턴싱*이라고 부릅니다.

API 오버헤드에 대한 우려는 왜 생겼을까? Direct3D 9 애플리케이션은 API 오버헤드로 인해 CPU에 의존하는 경우가 흔했다(이는 GPU가 아닌 CPU가 병목 현상의 원인이었음을 의미한다). 그 이유는 레벨 디자이너들이 고유한 재질과 텍스처를 가진 많은 오브젝트를 그리는 것을 선호하는데, 이는 각각에 대한 상태 변경과 드로우 콜을 필요로 하기 때문이다.

객체. 각 API 호출에 대한 CPU 오버헤드가 높을 경우, 실시간 렌더링 속도를 유지하기 위해 장면은 수천 개의 드로우 콜로 제한될 수 있습니다. 그래픽스 엔진은 드로우 콜 수를 최소화하기 위해 배치 기법([Wloka03] 참조)을 사용합니다. 하드웨어 인스턴싱은 텍스처 배열과 결합되는 경우가 많으며, API가 배치 수행을 지원하는 한 측면입니다.

15.1.1 버텍스 셰이더

하드웨어 인스턴싱은 버텍스 및 인덱스 데이터 외에도 인스턴스화된 데이터를 입력 어셈블러로 스트리밍하는 방식으로 작동합니다. 이후 하드웨어에 메쉬의 N 개 인스턴스를 렌더링하도록 지시합니다. 각 인스턴스가 렌더링될 때 버텍스 셰이더는 입력 *버텍스*와 현재 렌더링 중인 인스턴스의 인스턴스화된 데이터에 접근할 수 있습니다:

```
struct VertexIn
{
    float3 PosL : POSITION; float3 NormalL : NORMAL; float2 Tex : TEXCOORD;
    row_major float4x4 World : WORLD; float4 Color : COLOR;
    uint 인스턴스ID : SV_InstanceID;
};
```

인스턴스화된 데이터를 굵게 표시했습니다. 인스턴스마다 하나의 월드 매트릭스(각 인스턴스가 장면 내에서 고유한 위치에 배치되도록)와

인스턴스별 색상: 각 인스턴스가 고유한 색상을 가집니다. 시스템은 또한 시스템 값 식별자 **SV_InstanceID**를 제공합니다. 예를 들어, 첫 번째 인스턴스의 정점은 ID 0을, 두 번째 인스턴스의 정점은 ID 1을 가지며, 이처럼 순차적으로 할당됩니다. 인스턴스 ID의 한 가지 용도는 텍스처 배열에 대한 인덱스로 사용하여 각 인스턴스에 고유한 텍스처를 적용하는 것입니다. 버텍스 셰이더는 상수 버퍼의 월드 매트릭스 대신 인스턴스화된 월드 매트릭스를 월드 변환에 사용합니다. 또한 각 인스턴스에 고유한 색상을 부여하기 위해 인스턴스 색상을 픽셀 셰이더로 전달합니다:

```
struct VertexOut
{
    float4 PosH : SV_POSITION; float3
    PosW : POSITION; float3 NormalW :
    NORMAL; float2 Tex : TEXCOORD;
    float4 Color : COLOR;
};

VertexOut VS(VertexIn vin)
{
    VertexOut vout;

    // 월드 공간으로 변환.
    vout.PosW = mul(float4(vin.PosL, 1.0f), vin.World).xyz; vout.NormalW = mul(vin.NormalL,
(float3x3)vin.World);

    // 동차 클립 공간으로 변환.
    vout.PosH = mul(float4(vout.PosW, 1.0f), gViewProj);

    // 삼각형 간 보간을 위한 정점 속성 출력. vout.Tex = mul(float4(vin.Tex, 0.0f, 1.0f),
gTexTransform).xy; vout.Color = vin.Color;

    return vout;
}
```

픽셀 셰이더는 지난 장에서 사용한 것과 유사하지만, 인스턴스 색상으로 엠비언트와 디퓨즈 항을 변조한다는 점이 다릅니다:

```
ambient += A*pin.Color; diffuse +=
D*pin.Color;
```

15.1.2 인스턴스화된 데이터 스트리밍

그렇다면 인스턴스화된 데이터를 입력 어셈블러 단계로 어떻게 스트리밍할까요? 우리는 입력 레이아웃을 통해 버텍스 데이터를 스트리밍하므로, Direct3D도 동일한 메커니즘을 사용하여 인스턴스화된 데이터를 스트리밍합니다. [6장에서](#) 처음 설명한 **D3D11_INPUT_ELEMENT_DESC** 구조체를 떠올려 보세요:

```
typedef struct D3D11_INPUT_ELEMENT_DESC { LPCSTR
    SemanticName;
    UINT SemanticIndex; DXGI_FORMAT
    Format; UINT InputSlot;
    UINT 정렬바이트오프셋; D3D11_INPUT_CLASSIFICATION 입력슬롯분
    류; UINT 인스턴스데이터단계속도;
} D3D11_INPUT_ELEMENT_DESC;
```

마지막 두 멤버는 인스턴싱과 관련이 있습니다:

1. **InputSlotClass**: 입력 요소가 버텍스 요소로 스트리밍되는지 인스턴싱된 요소로 스트리밍되는지 지정합니다. 다음 두 값 중 하나를 지정하십시오:
D3D11_INPUT_PER_VERTEX_DATA: 입력 요소가 버텍스별로 스트리밍됩니다.
D3D11_INPUT_PER_INSTANCE_DATA: 인스턴스별로 입력 요소가 스트리밍됩니다.
2. **InstanceDataStepRate**: 인스턴스화된 데이터 요소당 그릴 인스턴스 수를 지정합니다. 예를 들어, 6개의 인스턴스를 그리려 하지만 인스턴스화된 색상 배열(빨강, 녹색, 파랑)만 3개 제공된 경우, 스텝 비율을 2로 설정하면 첫 두 인스턴스는 빨간색으로, 다음 두 인스턴스는 녹색으로, 마지막 두 인스턴스는 파란색으로 그려집니다. 각 인스턴스화된 데이터 요소와 각 인스턴스 사이에 일대일 대응 관계가 있을 경우, 인스턴스화된 데이터 요소는 빨간색으로, 다음 인스턴스화된 데이터 요소는 녹색으로, 마지막 두 인스턴스화된 데이터 요소는 파란색으로 그려집니다.

인스턴스가 빨간색으로, 두 번째 두 인스턴스가 녹색으로, 마지막 두 인스턴스가 파란색으로 그려집니다. 각 인스턴스 데이터 요소와 각 인스턴스 사이에 일대일 대응 관계가 있다면, 스텝 비율은 1. 정점 데이터의 경우, 단계 비율을 0으로 지정하십시오.

데모에서는 월드 매트릭스와 색상을 인스턴스별로 스트리밍합니다. 입력 레이아웃 정의는 다음과 같습니다:

```
const D3D11_INPUT_ELEMENT_DESC InputLayoutDesc::InstancedBasic32[8] =
{
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0, D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12, D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 24, D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"WORLD", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 1, 0, D3D11_INPUT_PER_INSTANCE_DATA, 1},
    {"WORLD", 1, DXGI_FORMAT_R32G32B32A32_FLOAT, 1, 16, D3D11_INPUT_PER_INSTANCE_DATA, 1},
    {"WORLD", 2, DXGI_FORMAT_R32G32B32A32_FLOAT, 1, 32, D3D11_INPUT_PER_INSTANCE_DATA, 1},
    {"WORLD", 3, DXGI_FORMAT_R32G32B32A32_FLOAT, 1, 48, D3D11_INPUT_PER_INSTANCE_DATA, 1},
    {"COLOR", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 1, 64, D3D11_INPUT_PER_INSTANCE_DATA, 1}
};
```

정점 데이터는 입력 슬롯 0에서, 인스턴스화된 데이터는 입력 슬롯 1에서 가져옵니다. 두 개의 버퍼를 사용합니다: 첫 번째는 정점 데이터를 포함하는 일반적인 정점 버퍼이고, 두 번째는 인스턴스화된 데이터를 포함하는 인스턴스 버퍼입니다. 그런 다음 두 버퍼를 모두 IA 단계에 바인딩합니다:

```
struct Basic32
{
    XMFLOAT3 Pos;
    XMFLOAT3 Normal;
    XMFLOAT2 Tex;
};

struct 인스턴스화된데이터
{
    XMFLOAT4X4 월드;
    XMFLOAT4 컬러;
};

UINT stride[2] = {sizeof(Vertex::Basic32), sizeof(InstancedData)}; UINT offset[2] = {0,0};

ID3D11Buffer* vbs[2] = {mSkullVB, mInstancedBuffer}; md3dImmediateContext->IASetVertexBuffers(0, 2, vbs,
stride, offset); md3dImmediateContext->IASetInputLayout(InputLayouts::InstancedBasic32);
```

15.1.3 인스턴스화된 데이터 그리기

인스턴스화된 데이터를 그리기 위해 **DrawIndexedInstanced** 드로우 호출을 사용합니다:

```
void ID3D11DeviceContext::DrawIndexedInstanced( UINT
    IndexCountPerInstance,
    UINT InstanceCount, UINT
    StartIndexLocation, INT
    BaseVertexLocation,
    UINT StartInstanceLocation
);
```

- 1. **인덱스** 카운트 당 인스턴스: 하나의 인스턴스를 정의하는 이 드로우 콜에서 사용될 인덱스 수입니다. 이는 인덱스 버퍼의 모든 인덱스를 포함할 필요는 없습니다. 즉, 인덱스의 연속적인 부분 집합을 드로우할 수 있습니다.
- 2. **InstanceCount**: 드로우할 인스턴스의 수.
- 3. **StartIndexLocation**: 인덱스 버퍼 내 인덱스 읽기 시작 지점을 표시하는 요소에 대한 인덱스.
- 4. **BaseVertexLocation**: 정점 데이터를 가져오기 전에 이 드로우 콜에서 사용되는 인덱스에 추가될 정수 값입니다.
- 5. **StartInstanceLocation**: 인스턴스 버퍼 내 요소에 대한 인덱스로, 인스턴스화된 데이터 읽기를 시작할 시작점을 표시합니다.

다음은 호출 예시입니다:

```
md3dImmediateContext->DrawIndexedInstanced( mSkullIndexCount, // 해골 메시의 인덱스 수
mVisibleObjectCount, // 그릴 인스턴스 수
0, 0, 0);
```

인덱스 버퍼를 사용하지 않는 인스턴스화된 데이터는 다음과 같은 변형으로 그릴 수 있습니다:

```
void ID3D11DeviceContext::DrawInstanced( UINT
VertexCountPerInstance,
UINT InstanceCount, UINT
StartVertexLocation,
UINT StartInstanceLocation
);
```

15.1.4 인스턴스화된 버퍼 생성

인스턴스화된 데이터를 포함하는 버퍼는 다른 **ID3D11Buffer**와 마찬가지로 생성됩니다. 본 데모에서는 모든 인스턴스화된 데이터의 시스템 메모리 복사본을 저장하고, 인스턴스화된 버퍼를 동적으로 만듭니다. 그런 다음 매 프레임마다, 가시적인 인스턴스의 인스턴스화된 데이터를 버퍼로 복사합니다(이는 프러스트럼 컬링과 관련이 있습니다. §15.3 참조). 인스턴스화된 데이터는 변경될 수 있도록 동적 버퍼에 저장되는 경우가 많습니다. 예를 들어, 오브젝트를 이동시키고 싶다면 월드 매트릭스를 업데이트해야 하는데, 동적 버퍼를 사용하면 이를 쉽게 수행할 수 있습니다.

```
void InstancingAndCullingApp::BuildInstancedBuffer()
{
    const int n = 5; mInstancedData.resize(n*n*n);

    float width = 200.0f; float
height = 200.0f; float depth =
200.0f;

    float x = -0.5f*width; float y = -
0.5f*height; float z = -0.5f*depth;
float dx = width / (n-1); float dy =
height / (n-1); float dz = depth / (n-
1); for(int k = 0; k < n; ++k)
{
    for(int i = 0; i < n; ++i)
    {
        for(int j = 0; j < n; ++j)
        {
            // 3D 그리드에 따라 인스턴스화된 위치.
            mInstancedData[k*n*n + i*n + j].World = XMFLOAT4X4( 1.0f, 0.0f, 0.0f, 0.0f,
0.0f, 1.0f, 0.0f, 0.0f,
0.0f, 0.0f, 1.0f, 0.0f,
x+j*dx, y+i*dy, z+k*dz, 1.0f);

            // 무작위 색상.
            mInstancedData[k*n*n + i*n + j].Color.x =
MathHelper::RandF(0.0f, 1.0f);
            mInstancedData[k*n*n + i*n + j].Color.y =
MathHelper::RandF(0.0f, 1.0f);
            mInstancedData[k*n*n + i*n + j].Color.z =
MathHelper::RandF(0.0f, 1.0f);
            mInstancedData[k*n*n + i*n + j].Color.w = 1.0f;
        }
    }
}
```

```

D3D11_BUFFER_DESC vbd;
vbd.Usage = D3D11_USAGE_DYNAMIC;
vbd.ByteWidth = sizeof(InstanceData) * mInstanceData.size(); vbd.BindFlags =
D3D11_BIND_VERTEX_BUFFER; vbd.CPUAccessFlags = D3D11_CPU_ACCESS_WRITE;
vbd.MiscFlags = 0;
vbd.StructureByteStride = 0;

HR(md3dDevice->CreateBuffer(&vbd, 0, &mInstanceBuffer));
}

```

15.2 바운딩 볼륨과 프러스텀

프러스텀 컬링을 구현하기 위해서는 프러스텀과 다양한 바운딩 볼륨의 수학적 표현에 익숙해져야 합니다. 바운딩 볼륨은 객체의 부피를 근사하는 기본적인 기하학적 객체입니다([그림 15.1](#) 참조). 그 대가로 바운딩 볼륨은 객체를 근사할 뿐이지만, 그 형태는 간단한 수학적 표현을 가지므로 다루기 쉽습니다.

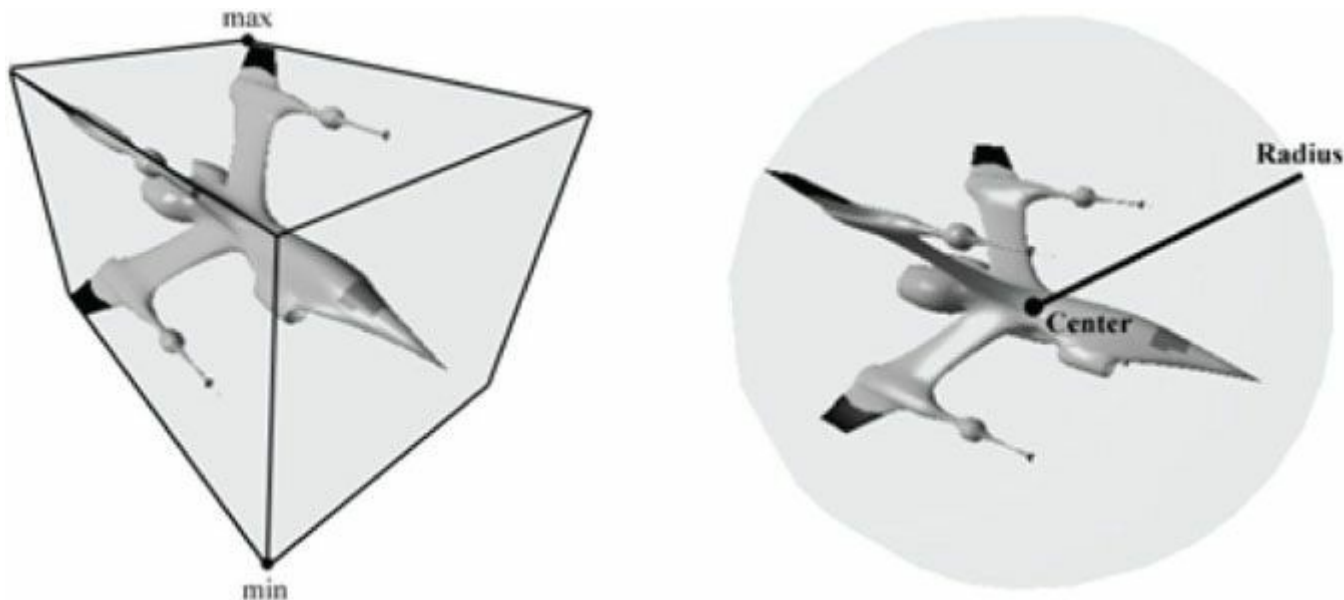


그림 15.1. AABB와 경계 구체와 함께 렌더링된 메시.

15.2.1 XNA 충돌

에서는 의 *xnacollision.h/.cpp* 유틸리티 라이브러리를 사용합니다. 해당 파일은 *Microsoft DirectX SDK (June 2010)\Samples\C++\Misc\Collision 0* 있습니다. 이 파일들은 광선/삼각형 교차, 광선/박스 교차, 박스/박스 교차, 박스/평면 교차, 박스/프러스텀, 구체/프러스텀 등 일반적인 기하학적 원시체 교차 테스트에 대한 빠른 XNA 수학 구현을 제공합니다. 연습 3에서는 이러한 파일들을 탐색하여 제공되는 내용을 익히도록 합니다.

15.2.2 박스

메시의 **축 정렬 경계 상자**(AABB)는 메시를 밀착하여 둘러싸는 상자로, 그 면은 주요 축과 평행합니다. AABB는 최소점 $\mathbf{v}_{min}$ 과 최대점 $\mathbf{v}_{max}$ 로 표현할 수 있습니다([그림 15.2](#) 참조). 최소점 $\mathbf{v}_{(min)}$ 은 메시의 모든 정점을 검색하여 최소 x-, y-, z-좌표를 찾는 방식으로 구하며, 최대점 $\mathbf{v}_{max}$ 는 메시의 모든 정점을 검색하여 최대 x-, y-, z-좌표를 찾는 방식으로 구합니다.

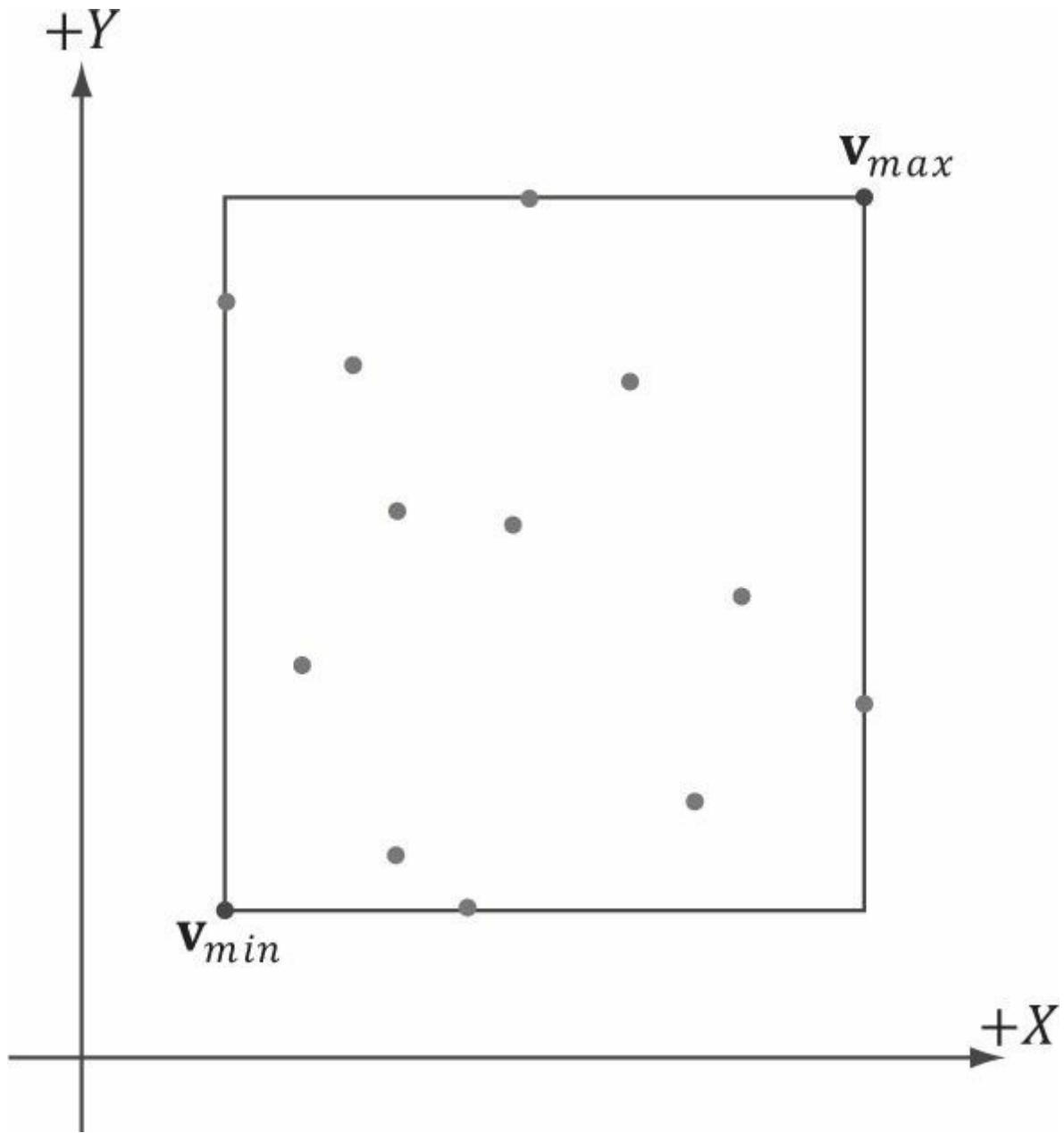


그림 15.2. 최소점과 최대점 표현을 사용한 점 집합의 Δ_{ABB} .

```
XMVECTOR vMin = XMLoadFloat3(&vMinf3); XMVECTOR
vMax = XMLoadFloat3(&vMaxf3); std::vector<Vertex::Basic3D>
vertices(vcount);
```

```

for(UINT i = 0; i < vcount; ++i)
{
    fin >> vertices[i].Pos.x >> vertices[i].Pos.y >> vertices[i].Pos.z;
    fin >> 정점[i].법선.x >> 정점[i].법선.y >> 정점[i].법선.z;

    XMVECTOR P = XMLoadFloat3(&vertices[i].Pos); vMin =
    XMVectorMin(vMin, P);
    vMax = XMVectorMax(vMax, P);
}

```

```

// 최소/최대 표현을 중심 및 범위 표현으로 변환. XMStoreFloat3(&mSkullBox.Center, 0.5f*(vMin+vMax));
XMStoreFloat3(&mSkullBox.Extents, 0.5f*(vMax-vMin));

```

XMVectorMin 및 XMVectorMax 함수는 다음 벡터를 반환합니다.

$$\min(\mathbf{u}, \mathbf{v}) = (\min(u_x, v_x), \min(u_y, v_y), \min(u_z, v_z), \min(u_w, v_w))$$

$$\max(\mathbf{u}, \mathbf{v}) = (\max(u_x, v_x), \max(u_y, v_y), \max(u_z, v_z), \max(u_w, v_w))$$

15.2.2.1 회전과 축 정렬 경계 상자

[그림 15.4](#)는 한 좌표계에서 축 정렬된 상자가 다른 좌표계에서는 축 정렬되지 않을 수 있음을 보여줍니다. 특히, 로컬 공간에서 메시의 AABB를 계산하면 월드 공간에서 방향성 경계 상자(OBB)로 변환됩니다. 그러나 메시의 로컬 공간으로 항상 변환하여 상자가 축 정렬되는 그곳에서 교차 연산을 수행할 수 있습니다.

대안으로, 월드 공간에서 AABB를 재계산할 수 있지만, 이는 실제 부피에 대한 근사치가 더 나빠지는 "더 두꺼운" 박스를 생성할 수 있습니다

([그림 15.5](#) 참조).

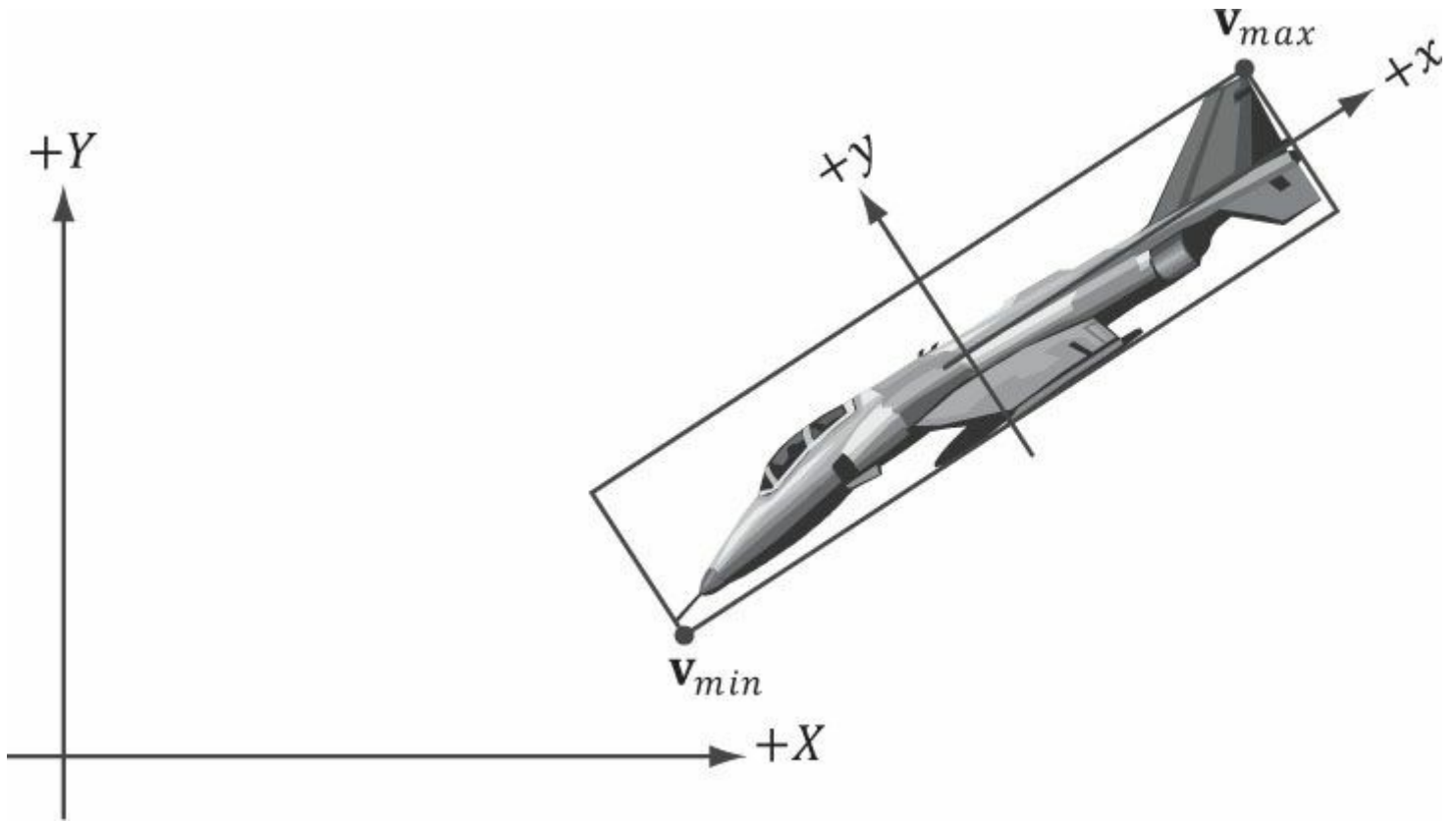


그림 15.4. 바운딩 박스는 xy-프레임과 축 정렬되지만 XY-프레임과는 그렇지 않습니다.

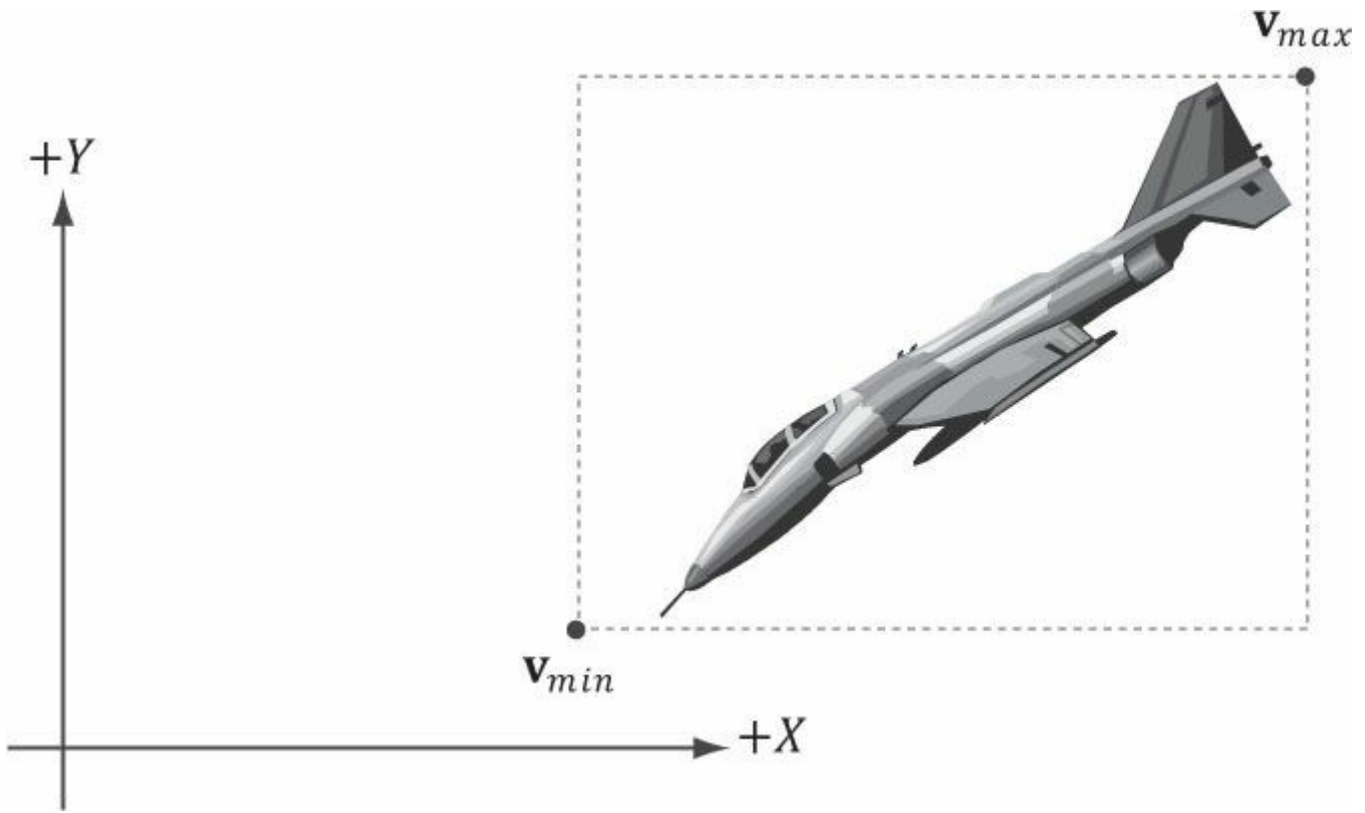


그림 15.5. 바운딩 박스가 XY 프레임과 축 정렬됨.

또 다른 대안은 축 정렬 바운딩 박스를 포기하고 방향성 바운딩 박스만 사용하는 것입니다. 이 경우 박스의 방향을 월드 공간에 상대적으로 유지합니다. XNA 충돌 라이브러리는 방향성 바운딩 박스를 표현하기 위해 다음과 같은 구조체를 제공합니다.

```
_DECLSPEC_ALIGN_16_struct OrientedBox
{
    XMFLOAT3 Center;           // 박스의 중심.
    XMFLOAT3 Extents;          // 중심에서 각 변까지의 거리. XMFLOAT4 Orientation; // 회
                               // 전을 나타내는 단위 쿼터니언
                               // (박스 -> 월드).
};
```

이 장에서는 회전/방향성을 표현하기 위한 쿼터니언에 대해 언급할 것입니다. 간단히 말해, 단위  쿼터니언은 회전 행렬과 마찬가지로 회전을 표현할 수 있습니다. 쿼터니언은 24장에서 다룹니다. 지금은

단순히 회전 행렬처럼 회전을 표현하는 것으로 생각하시면 됩니다.

XNA 충돌 라이브러리 함수를 사용해 AABB와 OBB도 생성할 수 있습니다:

```
VOID ComputeBoundingBoxFromPoints(AxisAlignedBox* pOut, UINT Count, const
    XMFLOAT3* pPoints, UINT Stride);
VOID ComputeBoundingOrientedBoxFromPoints(OrientedBox* pOut, UINT Count, const
    XMFLOAT3* pPoints, UINT Stride);
```

정점 구조가 다음과 같다면:

```
struct Basic32
{
    XMFLOAT3 Pos;
    XMFLOAT3 Normal;
    XMFLOAT2 Tex;
};
```

그리고 메쉬를 구성하는 정점 배열이 있습니다:

```
std::vector<Vertex::Basic32> vertices;
```

그런 다음 다음과 같이 이 함수를 호출합니다:

```
AxisAlignedBox box; ComputeBoundingAxisAlignedBoxFromPoints(&box,
    vertices.size(), &vertices[0].Pos, sizeof(Vertex::Basic32));
```

스트라이드는 다음 위치 요소로 이동하기 위해 건너뛴 바이트 수를 나타냅니다.

메쉬의 경계 볼륨을 계산하려면, `std::vector` 등에 저장된 것과 같은 정점 목록의 시스템 메모리 복사본이 필요합니다. 이는 CPU가 렌더링용으로 생성된 정점 버퍼에서 직접 읽을 수 없기 때문입니다. CPU는 `D3D11_USAGE_STAGING` 리소스만 읽을 수 있습니다. 따라서 애플리케이션에서는 이러한 작업(픽킹(16장), 충돌 감지 등)을 위해 시스템 메모리 복사본을 유지하는 것이 일반적입니다.

15.2.3 구체

메쉬의 경계 구체는 메쉬를 가장 밀착하여 둘러싼 구체입니다. 경계 구체는 중심점과 반지름으로 표현할 수 있습니다. 메쉬의 경계 구체를 계산하는 한 가지 방법은 먼저 AABB를 계산한 후, AABB의 중심을 경계 구체의 중심으로 삼는 것입니다.

$$\mathbf{c} = 0.5(\mathbf{v}_{min} + \mathbf{v}_{max})$$

반지름은 메쉬 내 임의의 정점 $\mathbf{p}$ 와 중심 $\mathbf{c}$ 사이의 최대 거리로 설정됩니다.

$$r = \max\{||\mathbf{c} - \mathbf{p}|| : \mathbf{p} \in \text{mesh}\}$$

로컬 공간에서 메쉬의 경계 구체(bounding sphere)를 계산한다고 가정합니다. 월드 변환 후, 스케일링으로 인해 경계 구체가 메쉬를 밀착하여 둘러싸지 못할 수 있습니다. 따라서 반지름을 이에 맞게 재조정해야 합니다. 비균일한 스케일링을 보정하기 위해, 변환된 메쉬를 완전히 감싸도록 반지름을 가장 큰 스케일링 계수로 스케일링해야 합니다. 또 다른 가능한 전략은 모든 메쉬를 게임 월드의 동일한 스케일로 모델링하여 스케일링 자체를 피하는 것이다. 이렇게 하면 모델이 애플리케이션에 로드된 후 재조정할 필요가 없다.

XNA 충돌 라이브러리 함수를 사용해 경계 구체도 계산할 수 있습니다:

```
VOID ComputeBoundingSphereFromPoints(Sphere* pOut, UINT Count, const
    XMFLOAT3* pPoints, UINT Stride);
```

15.2.4 프러스텀

우리는 제5장에서 원뿔형 절두체에 대해 잘 알고 있습니다. 원뿔형 절두체를 수학적으로 정의하는 한 가지 방법은 여섯 개의 평면—좌우 평면, 상하 평면, 근원 평면—의 교차로 표현하는 것입니다. 여섯 개의 절두체 평면은 모두 안쪽을 향한다고 가정합니다(그림 15.6 참조).

이 여섯 평면 표현 방식은 절두체와 경계 볼륨 교차 테스트를 쉽게 수행할 수 있게 합니다.

15.2.4.1 프러스텀 평면 구성

좌표계 공간에서 원뿔 단면 평면을 구성하는 쉬운 방법은 원점을 중심으로 양의 z 축을 따라 아래를 향하는 표준 형태를 취하는 것입니다. 여기서 근거리 및 원거리 평면은 z 축을 따라의 거리로 간단히 지정되며, 좌우 평면은 대칭이고 원점을 통과합니다(다시 그림 15.6 참조). 상하 평면 역시 대칭이고 원점을 통과합니다. 따라서 뷰 공간에서 프러스텀을 표현하기 위해 완전한 평면 방정식을 저장할 필요조차 없습니다. 상단/하단/좌측/우측 평면의 기울기, 그리고 근거리 및 원거리 평면의 z 거리를 알기만 하면 됩니다. XNA 충돌 라이브러리는 프러스텀을 표현하기 위해 다음과 같은 구조를 제공합니다:

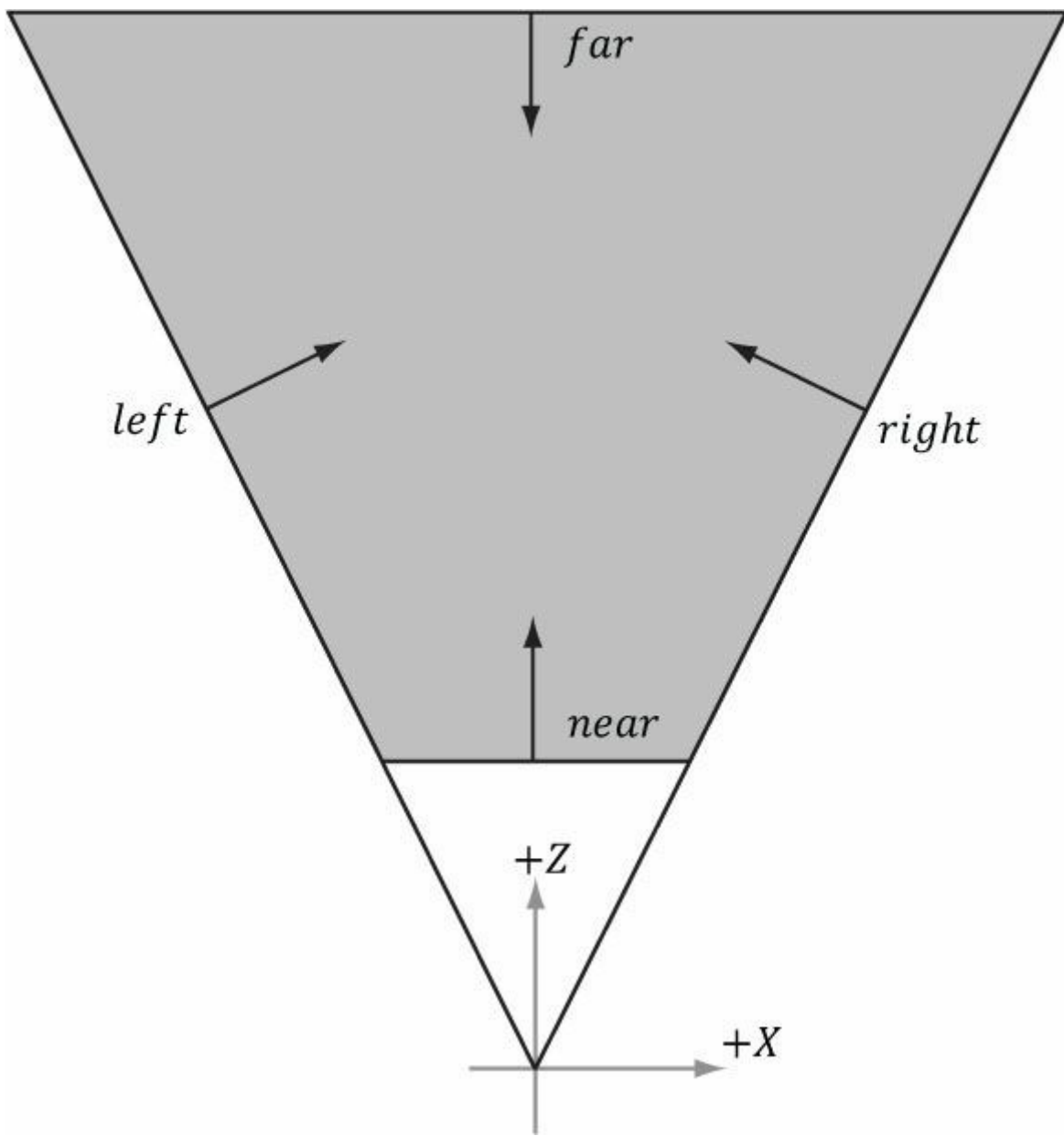


그림 15.6. 프러스텀 평면의 양의 반공간 교차로 정의되는 프러스텀 부피.

```

DECLSPEC_ALIGN_16_struct Frustum
{
    XMFLOAT3 Origin;           // 프러스텀(및 투영)의 원점. XMFLOAT4 Orientation; // 회전
                               // 나타내는 단위 쿼터니언.

    FLOAT RightSlope;          // 양의 X 경사 (X/Z). FLOAT
    LeftSlope;                 // 음의 X 기울기.
    FLOAT TopSlope;            // 양의 Y 기울기 (Y/Z). FLOAT
    BottomSlope;               // 음의 Y 기울기.
    FLOAT Near, Far;           // 근평면과 원평면의 Z값.
};

```

프러스텀의 로컬 공간(예: 카메라의 뷰 공간)에서 원점은 0이 되며, 오리엔테이션은 동일 변환(회전 없음)을 나타냅니다.

원점 위치와 방향 쿼터니언을 지정하여 원뿔의 위치와 방향을 설정할 수 있습니다.

카메라의 시야각(Frustum Vertical Field of View), 종횡비(Aspect Ratio), 근거리 평면(Near Plane) 및 원거리 평면(Far Plane)을 캐싱하면, 약간의 수학적 노력으로 뷰 공간(View Space) 내의 프러스텀 평면 방정식(Frustum Plane Equations)을 결정할 수 있습니다.

뷰 공간에서의 프러스텀 평면 방정식을 간단한 수학적 노력으로 구할 수 있습니다. 그러나 뷰 공간에서의 프러스텀 평면 방정식을 투영 행렬로부터 여러 방법으로 유도하는 것도 가능합니다 ([Lengyel02] 또는 [Möller08]에서 두 가지 다른 방법을 참조하십시오). XNA 충돌 라이브러리는 다음과 같은 전략을 채택합니다. NDC 공간에서 뷰 프러스텀은 $[-1,1] \times [-1,1] \times [0,1]$ 박스로 왜곡되었습니다. 따라서 뷰 프러스텀의 8개 모서리는 단순히 다음과 같습니다:

```

// 동차 공간에서의 투영 프러스텀 모서리. static XMVECTOR HomogenousPoints[6] =

```



```
{

    { 1.0f, 0.0f, 1.0f, 1.0f }, // 오른쪽 (원근면)

    { -1.0f, 0.0f, 1.0f, 1.0f }, // 왼쪽

    { 0.0f, 1.0f, 1.0f, 1.0f }, // 상단

    { 0.0f, -1.0f, 1.0f, 1.0f }, // 아래


    { 0.0f, 0.0f, 0.0f, 1.0f }, // 근거리

    { 0.0f, 0.0f, 1.0f, 1.0f }    // 멀리

};

투영 행렬의 역행렬을 계산하고(동일형 분할도 역변환) 8개의 모서리를 NDC 공간에서 뷰 공간으로 다시 변환할 수 있습니다. 뷰 공간에서 원뿔의 8개 모서리를 얻으면 간단한 수학을 통해 평면 방정식을 계산합니다(뷰 공간에서 원뿔은 원점에 위치하며 축과 정렬되어 있으므로 이 과정은 간단합니다). 다음 XNA 충돌 코드는 투영 행렬로부터 뷰 공간의 프러스텀을 계산합니다:
```

```
//-----
// 투영 행렬로부터 프러스텀을 생성합니다. 행렬은
// 투영만 포함할 수 있으며, 회전, 평행 이동 또는 크기 조정
//는 생성된 프러스텀이 부정확해지게 합니다.
//-----
VOID ComputeFrustumFromProjection(Frustum* pOut, XMMATRIX* pProjection)
{
    XMASSERT(pOut);
    XMASSERT(pProjection);

    // 동차 좌표계에서의 투영 프러스텀의 모서리 점들. static XMVECTOR
    HomogenousPoints[6] =
    {
        { 1.0f, 0.0f, 1.0f, 1.0f }, // 오른쪽 (원근면)

        { -1.0f, 0.0f, 1.0f, 1.0f }, // 왼쪽

        { 0.0f, 1.0f, 1.0f, 1.0f }, // 상단

        { 0.0f, -1.0f, 1.0f, 1.0f }, // 아래


        { 0.0f, 0.0f, 0.0f, 1.0f }, // 근거리
        { 0.0f, 0.0f, 1.0f, 1.0f }    // far
    };

    XMVECTOR Determinant;
    XMMATRIX matInverse = XMMatrixInverse(&Determinant, *pProjection);

    // 월드 공간에서 프러스텀 코너 계산. XMVECTOR Points[6];

    for(INT i = 0; i < 6; i++)
    {
        // 점 변환.
        Points[i] = XMVector4Transform(HomogenousPoints[i], matInverse);
    }

    pOut->Origin = XMFLOAT3(0.0f, 0.0f, 0.0f);
    pOut->Orientation = XMFLOAT4(0.0f, 0.0f, 0.0f, 1.0f);

    // 기울기 계산.
    Points[0] = Points[0] * XMVectorReciprocal(XMVectorSplatZ(Points[0])); Points[1] = Points[1] *
    XMVectorReciprocal(XMVectorSplatZ(Points[1]));      Points[2] = Points[2] *
    XMVectorReciprocal(XMVectorSplatZ(Points[2]));      Points[3] = Points[3] *
    XMVectorReciprocal(XMVectorSplatZ(Points[3]));

    pOut->RightSlope = XMVectorGetX(Points[0]); pOut->LeftSlope =
    XMVectorGetX(Points[1]); pOut->TopSlope =
    XMVectorGetY(Points[2]); pOut->BottomSlope =
    XMVectorGetY(Points[3]);
}
```

```
// 근점과 원점 계산.
Points[4] = Points[4] * XMVectorReciprocal(XMVectorSplatW(Points[4])); Points[5] = Points[5] *
XMVectorReciprocal(XMVectorSplatW(Points[5]));

pOut->Near = XMVectorGetZ(Points[4]); pOut->Far =
XMVectorGetZ(Points[5]);

return;
}
```

15.2.4.2 프러스텀/구체 교차

프러스텀 컬링을 위해 수행해야 할 테스트 중 하나는 프러스텀/구체 교차 테스트입니다. 이는 구체가 프러스텀과 교차하는지 여부를 알려줍니다. 프러스텀을 단순한 경계가 아닌 부피로 취급하기 때문에, 프러스텀 내부에 완전히 포함된 구체도 교차로 간주된다는 점에 유의하십시오. 프러스텀을 여섯 개의 안쪽을 향한 평면으로 모델링하기 때문에, 프러스텀/구체 테스트는 다음과 같이 표현할 수 있습니다: 구체가 프러스텀 *평면 L*의 음의 반공간에 존재하는 평면 *L* 존재한다면, 구체가 프러스텀 외부 전체에 위치한다고 결론지을 수 있습니다. 그러한 평면이 존재하지 않는다면, 구체가 프러스텀과 교차한다고 결론지을 수 있습니다.

따라서 원뿔체/구체 교차 검사는 여섯 개의 구체/평면 검사로 환원됩니다. [그림 15.7](#)은 구체/평면 교차 검사의 설정을 보여줍니다. 테스트, 구의 중심점을 c , 반지름을 r *O/r*하자. 그러면 구의 중심에서 평면까지의 부호 있는 거리는 $k = \mathbf{n} \cdot \mathbf{c} + d$ 이다(부록 C). $|k| \leq r$ 이면 구가 평면과 교차한다. $k < -r$ 이면 구가 평면 뒤에 위치한다. $k > r$ 이면 구가 평면 앞에 위치하며, 구는 평면의 양의 반공간과 교차한다. 원뿔/구 교차 테스트 목적상, 구가 평면 앞에 위치할 경우 평면이 정의하는 양의 반공간과 교차하므로 이를 교차로 간주한다.

XNA 충돌 라이브러리는 구체가 프러스텀과 교차하는지 테스트하기 위한 다음 함수를 제공합니다. 구체와 프러스텀은 테스트가 유효하려면 구와 프러스텀이 동일한 좌표계에 있어야 합니다.

```
// 반환 값: 0 = 교차 없음,
//          1 = 교차,
//          2 = A가 B 안에 완전히 포함됨 INT
IntersectSphereFrustum(
    const Sphere* pVolumeA, const Frustum*
    pVolumeB);
```

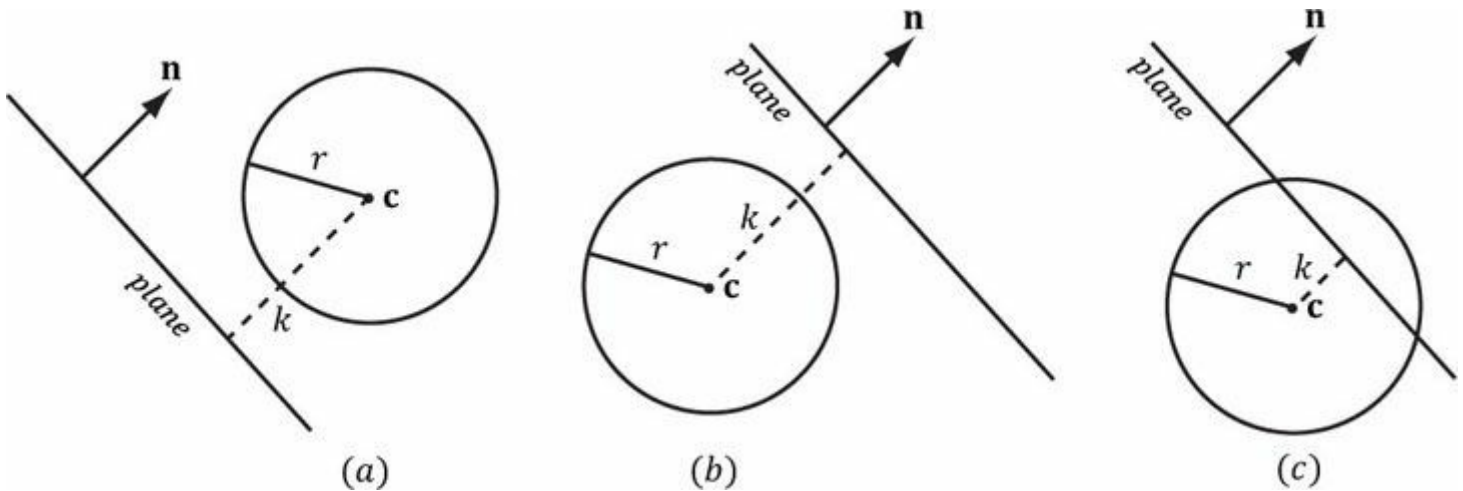


그림 15.7. 구체/평면 교차. (a) $k > r$ 이고 구체가 평면의 양반공간과 교차함. (b) $k < -r$ 이고 구체가 평면의 음반공간에 완전히 위치함. (c) $|k| \leq r$ 이고 구체가 평면과 교차함.

15.2.4.3 프러스텀/AABB 교차

프러스텀/AABB 교차 테스트는 프러스텀/구 테스트와 동일한 전략을 따릅니다. 프러스텀을 여섯 개의 안쪽을 향한 평면으로 모델링하기 때문에, 프러스텀/AABB 테스트는 다음과 같이 표현할 수 있습니다: 상자(box)가 *평면 L*의 음의 반공간에 존재하는 프러스텀 평면 *L* 존재한다면, 상자가 프러스텀 외부 전체에 위치한다고 결론지을 수 있습니다. 그러한 평면이 존재하지 않는다면, 박스가 프러스텀과 교차한다고 결론지을 수 있습니다.

따라서 원뿔/AABB 교차 검사는 여섯 개의 AABB/평면 검사로 환원됩니다. AABB/평면 검사의 알고리즘은 다음과 같습니다. 찾기
상자 대각선 벡터 $\mathbf{v} = \overrightarrow{PQ}$ 는 상자 중심을 통과하며 평면 **법선 n**과 가장 잘 정렬된다. [그림 15.8](#)에서, (a) 점 P 가 평면 앞에 있으면 점 Q 도 평면 앞에 있어야 한다; (b) 점 Q 가 평면 뒤에 있으면 점 P 도 평면 뒤에 있어야 한다; (c) P 가 평면 뒤에 있고 Q 가 평면 앞에 있다면, 박스는 평면과 교차한다.

평면 법선 벡터 $\mathbf{n}$ 과 가장 잘 정렬된 PQ 를 찾는 것은 다음 코드로 수행할 수 있습니다:

```
// 각 좌표축 x, y, z에 대해... for(int j = 0; j < 3; ++j)
```

```
{
    // PQ가 해당 축의 평면 법선과 같은 방향을 가리키도록
    // if(planeNormal[j] >= 0.0f)
    {
        P[j] = box.minPt[j];
        Q[j] = box.maxPt[j];
    }
    else
    {
        P[j] = box.maxPt[j];
        Q[j] = box.minPt[j];
    }
}
```

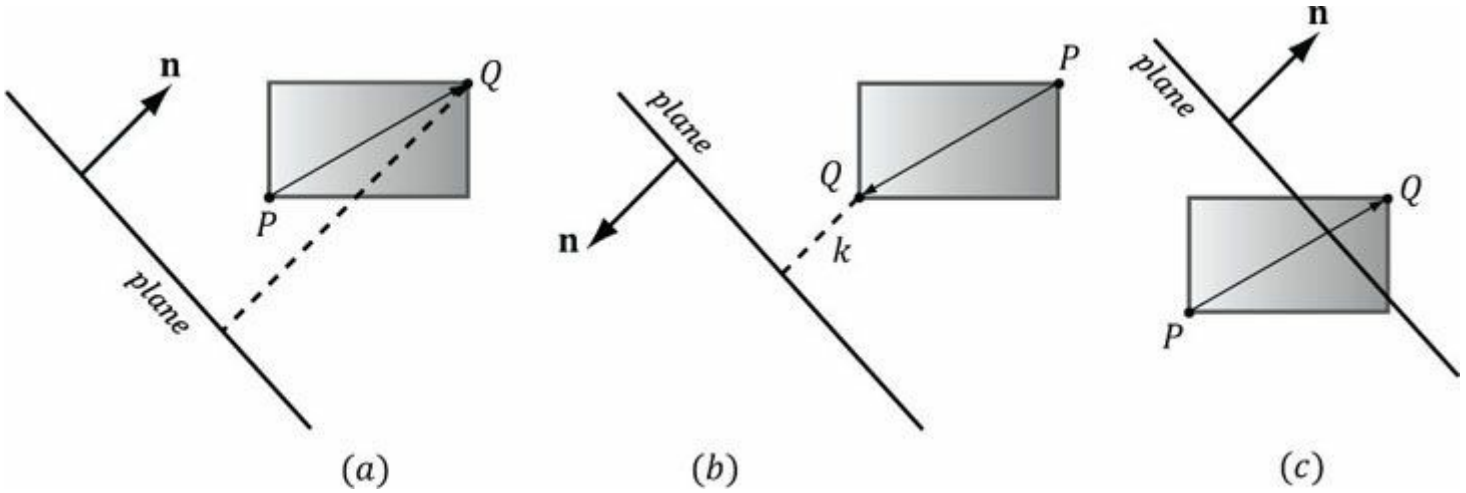


그림 15.8. AABB/평면 교차 테스트. 대각선 $\overrightarrow{PQ}$ 은 항상 평면 법선과 가장 방향이 일치하는 대각선이다.

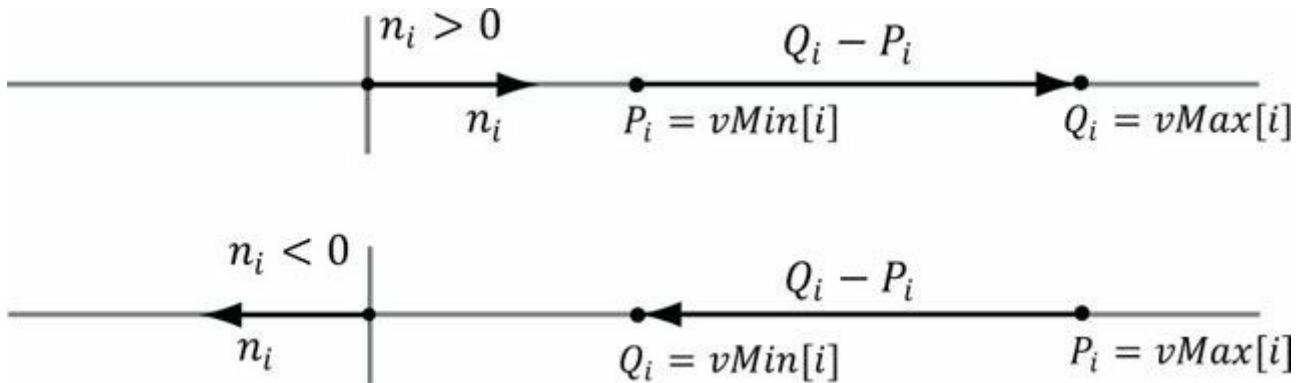


그림 15.9. (상단) i 번째 축을 따라 노멀 성분이 양수이므로, 평면 노멀 좌표 n_i 와 동일한 부호를 가지도록 $Q_i - P_i$ 와 동일 부호가 되도록 $P_i = vMin[i]$ 및 $Q_i = vMax[i]$ 를 선택합니다. (아래) i 번째 축을 따라가는 법선 성분이 음수이므로, $Q_i - P_i$ 가 평면 법선 좌표 n_i 와 같은 부호를 가지도록 $P_i = vMax[i]$ 및 $Q_i = vMin[i]$ 를 선택한다.

이 코드는 한 번에 한 차원씩만 살펴보고, $Q_i - P_i$ 가 평면 법선 좌표 n_i 와 동일한 부호를 가지도록 P_i 와 Q_i 를 선택합니다(그림 15.9).

XNA 충돌 라이브러리는 AABB가 프러스텀과 교차하는지 테스트하기 위한 다음 함수를 제공합니다. 테스트가 의미 있으려면 AABB와 프러스텀은 테스트가 유효하려면 동일한 좌표계에 있어야 합니다.

```
// 반환 값: 0 = 교차 없음,
//          1 = 교차점,
//          2 = A가 B 안에 완전히 포함됨 INT
IntersectAxisAlignedBoxFrustum(
    const AxisAlignedBox* pVolumeA, const Frustum*
    pVolumeB);
```

15.3 프러스텀 컬링

5장에서 살펴본 바와 같이, 하드웨어는 클리핑 단계에서 뷰 프러스텀 외부에 위치한 삼각형을 자동으로 제거합니다. 그러나 수백만 개의 삼각형이 존재할 경우, 모든 삼각형은 여전히 드로우 호출(API 오버헤드 발생)을 통해 렌더링 파이프라인에 제출되며, 모든 삼각형은 버텍스 셰이더를 거쳐 테셀레이션 단계와 지오메트리 셰이더를 통과한 후 클리핑 단계에서 제거됩니다. 이는 명백히 비효율적인 낭비입니다.

프러스텀 컬링의 개념은 애플리케이션 코드가 삼각형 단위가 아닌 더 높은 수준에서 삼각형 그룹을 컬링하도록 하는 것입니다.

그림 15.10은 간단한 예를 보여줍니다. 우리는 장면 내 각 오브젝트 주위에 구나 상자과 같은 경계 볼륨을 생성합니다. 경계 볼륨이 프러스텀과 교차하지 않으면, 해당 오브젝트(수천 개의 삼각형을 포함할 수 있음)를 Direct3D에 제출하여 렌더링할 필요가 없습니다. 이는 저렴한 CPU 테스트를 대가로 GPU가 보이지 않는 지오메트리에 대한 불필요한 계산을 수행하지 않도록 합니다. 시야각 90°의 카메라와 무한히 먼 원근면을 가정할 때, 카메라 프러스텀은 월드 볼륨의 1/6만을 차지하므로, 오브젝트가 장면 전체에 고르게 분포되어 있다고 가정하면 월드 오브젝트의 5/6를 프러스텀 컬링할 수 있습니다. 실제 카메라들은 90°보다 작은 시야각과 유한한 원근면을 사용하므로, 장면 오브젝트의 5/6보다 더 많은 부분을 제거할 수 있습니다.

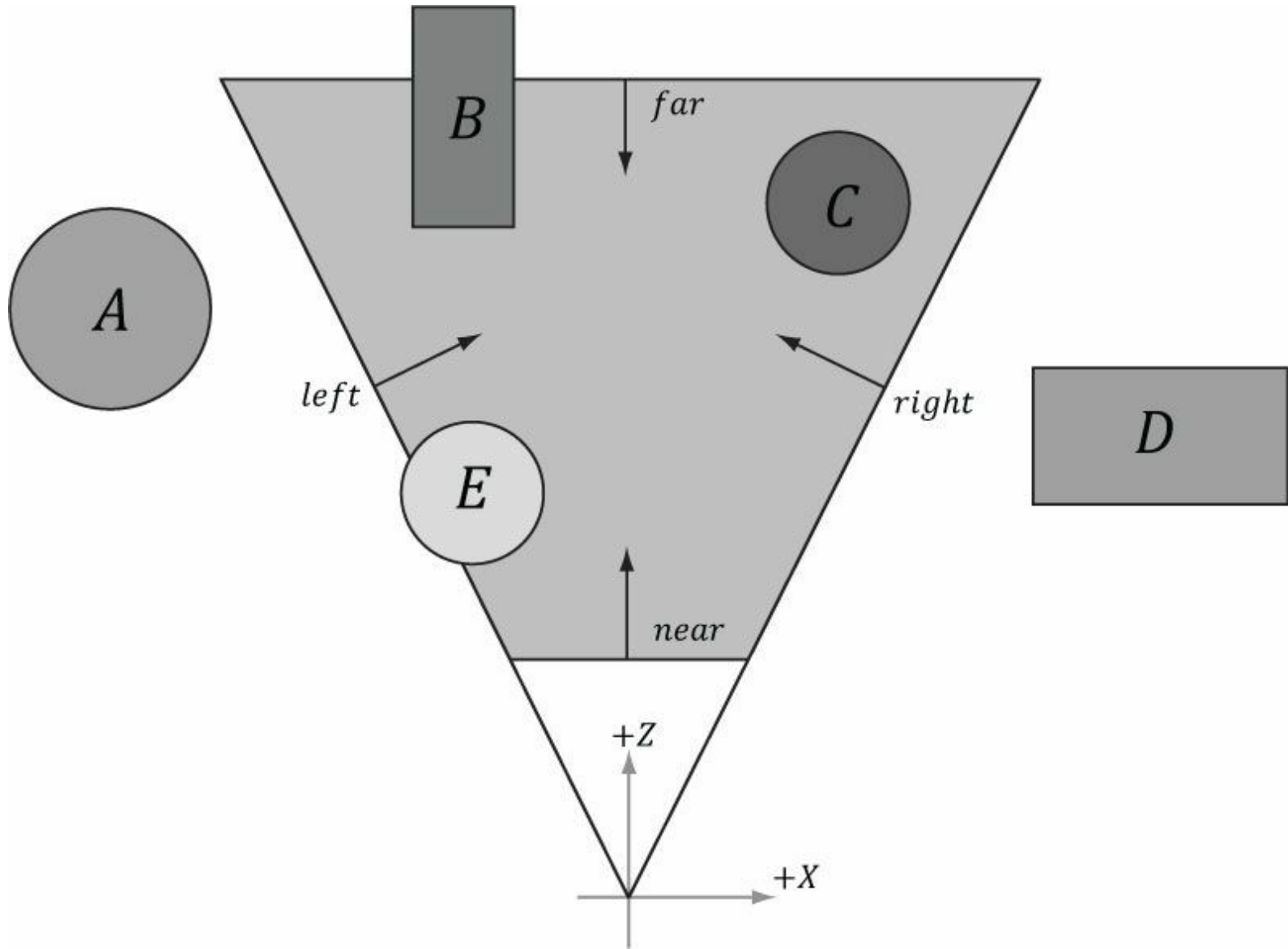


그림 15.10. 부피 A와 D로 경계지어진 객체들은 원뿔 절두체 밖에 완전히 위치하므로 그림 필요가 없습니다. 부피 C에 해당하는 객체는 원뿔 절두체 안에 완전히 위치하므로 그려야 합니다. 부피 B와 E로 경계지어진 객체들은 부분적으로 원뿔 절두체 밖에 있고 부분적으로 안에 있습니다. 이 객체들을 그려야 하며, 하드웨어가 원뿔 절두체 밖에 있는 삼각형을 클리핑하도록 해야 합니다.

데모에서는 인스턴싱을 사용하여 5×5×5 크기의 해골 메시 그리드를 렌더링합니다(그림 15.11 참조). 두개골 메시의 AABB를 로컬 공간에서 계산합니다. UpdateScene 메서드에서 모든 인스턴스에 대해 프러스텀 컬링을 수행합니다. 인스턴스가 프러스텀과 교차하면 동적 인스턴스 버퍼의 다음 사용 가능한 슬롯에 추가하고 `mVisibleObjectCount` 카운터를 증가시킵니다. 이렇게 하면 동적 인스턴스 버퍼의 앞부분에 모든 가시적 인스턴스의 데이터가 포함됩니다. (물론 인스턴스 버퍼 크기는 모든 인스턴스가 가시적일 경우를 대비해 인스턴스 수에 맞춰 설정됩니다.) 두개골 메시의 AABB가 로컬 공간에 있으므로, 교차 테스트를 수행하려면 뷰 프러스텀을 각 인스턴스의 로컬 공간으로 변환해야 합니다. 예를 들어 AABB를 월드 공간으로 변환하고 프러스텀도 월드 공간으로 변환하는 등 다른 공간을 사용할 수도 있습니다. 프러스텀 컬링 업데이트

코드는 다음과 같습니다:

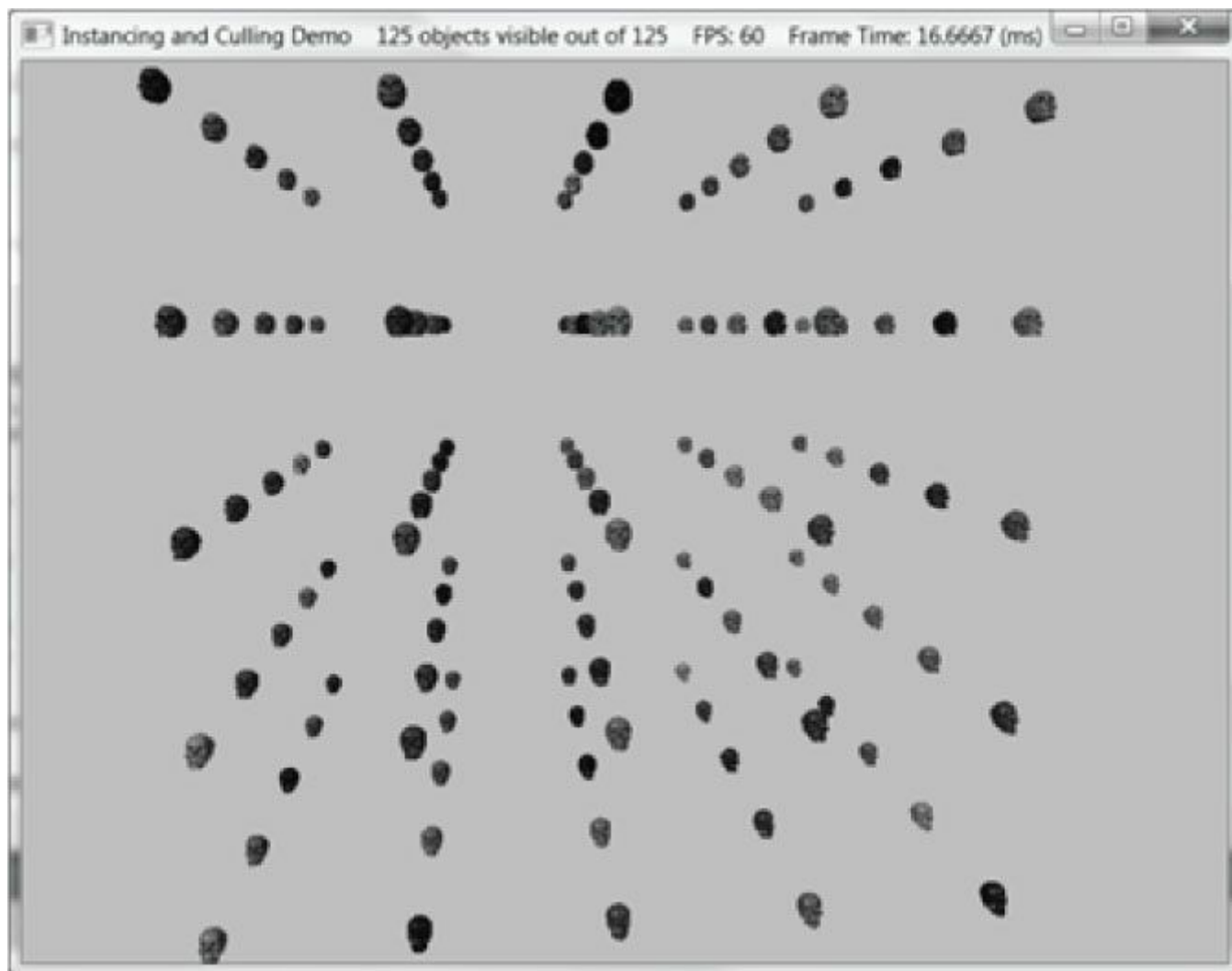


그림 15.11. "인스턴싱과 컬링" 데모의 스크린샷.

```
mCam.UpdateViewMatrix(); mVisibleObjectCount = 0;

XMVECTOR detView = XMMatrixDeterminant(mCam.View()); XMMATRIX invView =
XMMatrixInverse(&detView, mCam.View());

// 인스턴스화된 버퍼에 쓰기 위해 매핑합니다.
D3D11_MAPPED_SUBRESOURCE mappedData;
md3dImmediateContext->Map(mInstancedBuffer, 0,
    D3D11_MAP_WRITE_DISCARD, 0, &mappedData);

InstancedData* dataView = reinterpret_cast<InstancedData*>(mappedData.pData); for(UINT i = 0; i <
mInstancedData.size(); ++i)
{
    XMMATRIX W = XMLoadFloat4x4(&mInstancedData[i].World); XMMATRIX invWorld =
    XMMatrixInverse(&XMMatrixDeterminant(W), W);

    // 뷰 공간을 객체의 로컬 공간으로 변환.
    XMMATRIX toLocal = XMMatrixMultiply(invView, invWorld);

    // 행렬을 개별 요소로 분해합니다. XMVECTOR scale;
    XMVECTOR rotQuat; XMVECTOR
    translation;
    XMMatrixDecompose(&scale, &rotQuat, &translation, toLocal);
```

```
// 카메라 프러스텀을 뷰 공간에서 객체의
// 로컬 공간으로 변환합니다.
XNA::Frustum localspaceFrustum; XNA::TransformFrustum(&localspaceFrustum,
&mCamFrustum,

XMVectorGetX(scale), rotQuat, translation);

// 로컬 공간에서 박스/프러스텀 교차 테스트를 수행합니다. if(XNA::IntersectAxisAlignedBoxFrustum(&mSkullBox, &localspaceFrustum)
!= 0)
{
    // 인스턴스 데이터를 가시 객체의 동적 뷰 배열에 기록합니다.
    dataView[mVisibleObjectCount++] = mInstancedData[i];
}
}

md3dImmediateContext->Unmap(mInstancedBuffer, 0);
```

인스턴스 버퍼에 모든 인스턴스를 수용할 공간이 있음에도 불구하고, 0부터 **mVisibleObjectCount-1**까지의 인스턴스에 해당하는 가시적 인스턴스만 렌더링합니다.

```
md3dImmediateContext->DrawIndexedInstanced( mSkullIndexCount, // 해골 메시의 인덱스
수 mVisibleObjectCount, // 그릴 인스턴스 수
0, 0, 0);
```

그림 15.12는 프러스텀 컬링을 활성화한 경우와 비활성화한 경우의 성능 차이를 보여줍니다. 프러스텀 컬링을 사용하면 렌더링 파이프라인에 처리할 인스턴스를 11개만 제출합니다. 프러스텀 컬링을 사용하지 않으면 렌더링 파이프라인에 처리할 모든 125개 인스턴스를 제출합니다. 가시적인 장면은 동일하지만, 프러스텀 컬링을 비활성화하면 클리핑 단계에서 결국 버려지는 100개 이상의 해골 메쉬를 렌더링하는 데 컴퓨팅 자원을 낭비하게 됩니다. 각 해골은 약 60K개의 삼각형을 가지므로, 처리해야 할 많은 정점과 해골당 클리핑해야 할 많은 삼각형이 발생합니다. 하나의 프러스텀/AABB 테스트를 수행함으로써, 60K개의 삼각형이 그래픽스 파이프라인으로 전송되는 것 자체를 차단할 수 있습니다. 이것이 프러스텀 컬링의 장점이며, 초당 프레임 수(FPS)에서 차이를 확인할 수 있습니다.

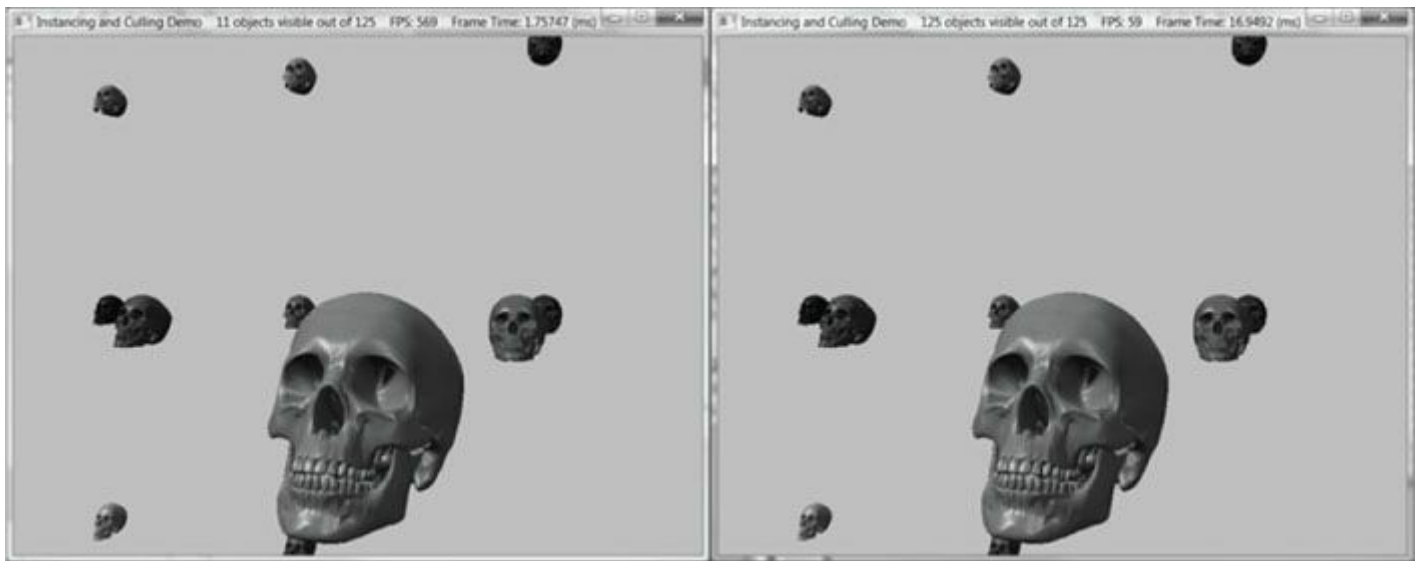


그림 15.12. (왼쪽) 프러스텀 컬링이 활성화된 상태로, 125개 인스턴스 중 11개가 가시 영역에 있으며 프레임 렌더링에 약 1.75ms 소요됨. (오른쪽) 프러스텀 컬링이 꺼진 상태에서 125개의 인스턴스 전체를 렌더링하며, 프레임 렌더링에 약 16.94ms가 소요됩니다. 이는 프러스텀 컬링이 활성화되었을 때보다 9.68배 더 긴 시간입니다.

15.4 요약

1. 인스턴싱은 동일한 오브젝트를 장면 내에서 여러 번 렌더링하지만 서로 다른 위치, 방향, 크기, 재질 및 텍스처로 표현하는 기법을 의미합니다. 메모리 절약을 위해 하나의 메시만 생성하고, 서로 다른 월드 매트릭스, 재질 및 텍스처를 적용하여 Direct3D에 다중 드로우 콜을 제출합니다. 리소스 변경을 발행하는 API 오버헤드를 피하기 위해, 인스턴스별 데이터를 입력 어셈블러로 스트리밍하고 Direct3D가 **ID3D11DeviceContext::DrawIndexedInstanced** 메서드로 여러 인스턴스를 그리도록 할 수 있습니다. 각 인스턴스는 입력 어셈블러 단계에 바인딩된 인스턴스화 버퍼에서 연관된 인스턴스화 데이터를 획득합니다.

2. 바운딩 볼륨은 객체의 부피를 근사하는 기본 기하학적 객체입니다. 바운딩 볼륨은 객체를 근사할 뿐이지만, 그 형태는 간단한 수학적 표현을 가지므로 작업하기 쉽다는 장점이 있습니다. 바운딩 볼륨의 예로는 구, 축 정렬 바운딩 박스(AABB), 방향성 바운딩 박스(OBB) 등이 있습니다. xnacollision.h/cpp 라이브러리에는 바운딩 볼륨을 표현하는 구조체와 이를 변환하고 다양한 교차 테스트를 계산하는 함수가 포함되어 있습니다.
3. GPU는 클리핑 단계에서 시야 원뿔(viewing frustum) 밖에 있는 삼각형을 자동으로 제거합니다. 그러나 클리핑된 삼각형은 여전히 드로우 콜(API 오버헤드가 발생함)을 통해 렌더링 파이프라인에 제출되며, 모든 삼각형은 버텍스 셰이더를 거쳐 테셀레이션 단계와 지오메트리 셰이더를 통과한 후 클리핑 단계에서 제거됩니다. 이러한 비효율성을 해결하기 위해 프러스텀 컬링(frustum culling)을 구현할 수 있습니다. 이 아이디어는 장면 내 각 오브젝트 주위에 구나 박스와 같은 경계 볼륨을 구축하는 것입니다. 경계 볼륨이 프러스텀과 교차하지 않으면, 해당 오브젝트(수천 개의 삼각형을 포함할 수 있음)를 Direct3D에 드로잉을 위해 제출할 필요가 없습니다. 이는 저렴한 CPU 테스트 비용을 대가로, GPU가 보이지 않는 지오메트리에 대한 낭비적인 계산을 수행하지 않도록 합니다.

15.5 연습 문제

1. "인스턴싱과 컬링" 데모를 수정하여 경계 상자 대신 경계 구체를 사용하도록 하십시오.
2. NDC 공간에서의 평면 방정식은 매우 단순한 형태를 띕니다. 시야 원뿔 내부의 모든 점은 다음과 같이 제한됩니다:

$$\begin{aligned} -1 &\leq x_{ndc} \leq 1 \\ -1 &\leq y_{ndc} \leq 1 \\ 0 &\leq z_{ndc} \leq 1 \end{aligned}$$

특히, NDC 공간에서 좌측 평면 방정식은 $x = -1$ 로, 우측 평면 방정식은 $x = 1$ 로 주어집니다. 투시 분할 전의 균일 클립 공간에서, 뷰 프러스텀 내부의 모든 점은 다음과 같이 제한됩니다:

$$\begin{aligned} -w &\leq x_h \leq w \\ -w &\leq y_h \leq w \\ 0 &\leq z_h \leq w \end{aligned}$$

여기서 왼쪽 평면은 $w = -x_h$ 로 정의되고, 오른쪽 평면은 $w = x_h$ 로 정의된다. $\mathbf{M} = \mathbf{VP}$ 를 뷰-프로젝션 행렬 곱으로 정의하고, $\mathbf{v} = (x, y, z, 1)$ 을 프러스텀 내 월드 공간의 점으로 정의한다. $(x_h, y_h, z_h, w) = \mathbf{vM} = (\mathbf{v} \cdot \mathbf{M}_{*,1}, \mathbf{v} \cdot \mathbf{M}_{*,2}, \mathbf{v} \cdot \mathbf{M}_{*,3}, \mathbf{v} \cdot \mathbf{M}_{*,4})$ 를 고려하면, 세계 좌표계에서 안쪽을 향하는 프러스텀 평면은 다음과 같이 주어짐을 알 수 있다:

좌측	$0 = \mathbf{p} \cdot (\mathbf{M}_{*,1} + \mathbf{M}_{*,4})$
오른쪽	$0 = \mathbf{p} \cdot (\mathbf{M}_{*,4} - \mathbf{M}_{*,1})$
아래	$0 = \mathbf{p} \cdot (\mathbf{M}_{*,2} + \mathbf{M}_{*,4})$
위	$0 = \mathbf{p} \cdot (\mathbf{M}_{*,4} - \mathbf{M}_{*,2})$
근접	$0 = \mathbf{p} \cdot \mathbf{M}_{*,3}$
먼	$0 = \mathbf{p} \cdot (\mathbf{M}_{*,4} - \mathbf{M}_{*,3})$

(a) 우리는 안쪽을 향하는 법선을 요구합니다. 이는 절두체 내부의 점이 평면으로부터 양의 거리를 가짐을 의미합니다. 즉, 절두체 내부의 점 $\mathbf{p}$ 에 대해 $\mathbf{n} \cdot \mathbf{p} + d \geq 0$ 이 성립합니다.



(b) $v_w = 1$ 임을 유의하십시오. 따라서 앞서 제시된 내적 공식들은 다음과 같은 형태의 평면 방정식을 생성합니다. $A_x + B_y + C_z + D = 0$.

(c) 계산된 평면 법선 벡터는 단위 길이가 아닙니다. 평면을 정규화하는 방법은 부록 C를 참조하십시오.

3. XNA 수학 라이브러리를 사용한 교차 테스트를 위한 함수들을 익히려면 `xnacollision.h` 헤더 파일을 살펴보십시오.
4. OBB는 중심점 C , 상자 방향을 정의하는 세 개의 직교 정규 축 벡터 $\mathbf{r}_0, \mathbf{r}_1, \mathbf{r}_2$, 그리고 상자 중심에서 상자 측면까지의 거리를 나타내는 상자 축 $\mathbf{r}_0, \mathbf{r}_1, \mathbf{r}_2$ 를 따라 각각의 세 개의 확장 길이 a_0, a_1, a_2 로 정의할 수 있습니다.

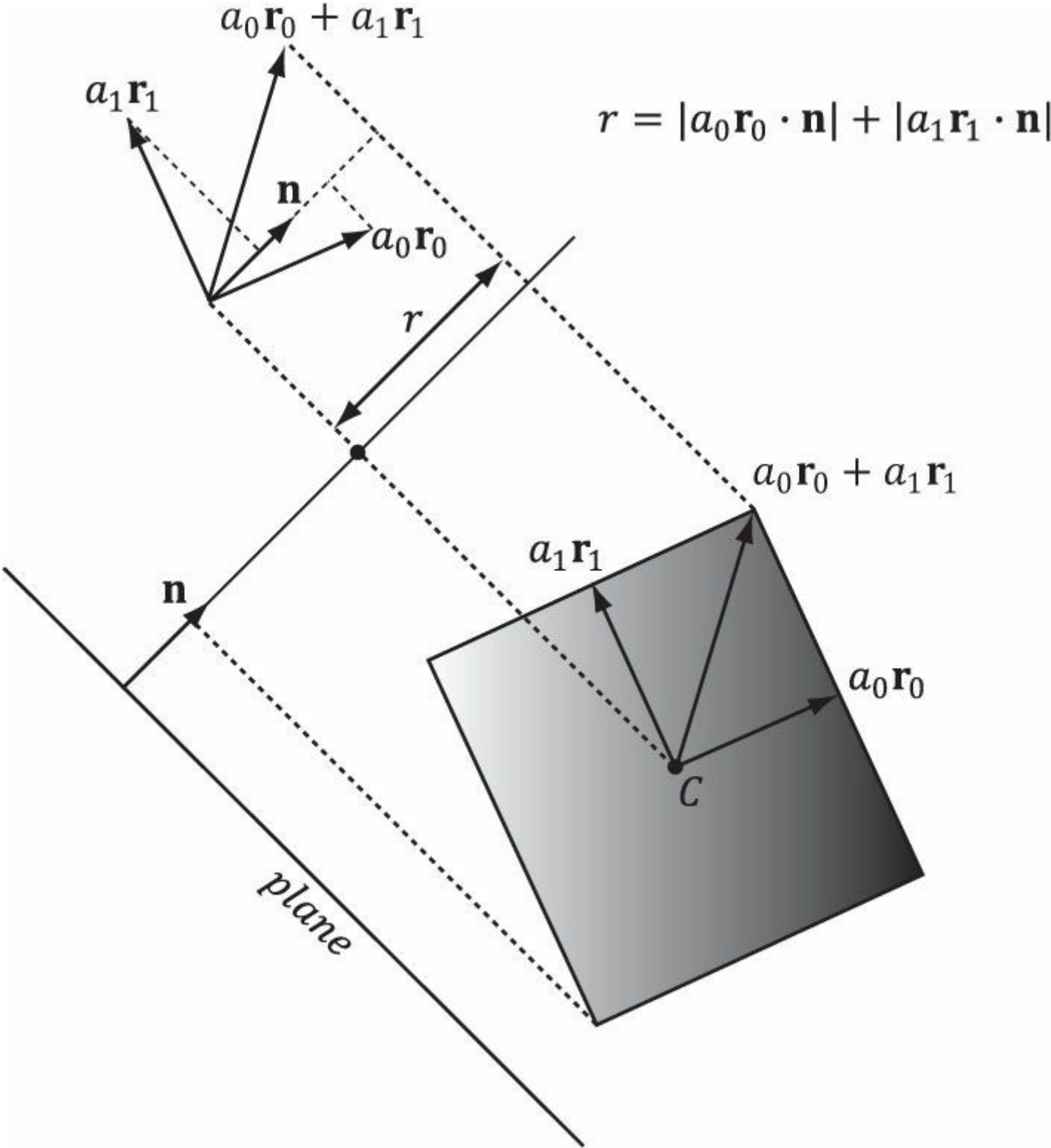


그림 15.13. 평면/OBB 교차 설정.

그림 15.13(2차원 상황을 보여줌)을 고려하면, 법선 벡터로 정의된 축에 투영된 OBB의 "그림자"는 $2r$ 임을 알 수 있습니다. 여기서

$$r = |a_0 \mathbf{r}_0 \cdot \mathbf{n}| + |a_1 \mathbf{r}_1 \cdot \mathbf{n}| + |a_2 \mathbf{r}_2 \cdot \mathbf{n}|$$

r에 대한 이전 공식에서, 왜 단순히 $r = (a_0r_0 + a_1r_1 + a_2r_2) \cdot \mathbf{n}$ 을 계산하는 대신 절대값을 취해야 하는지 설명하십시오. 평면과 OBB의 교차 테스트를 유도하여 OBB가 평면 앞에 있는지, 뒤

에 있는지, 아니면 평면과 교차하는지 판단하십시오.

AABB는 OBB의 특수한 경우이므로, 이 테스트는 AABB에도 적용됩니다. 그러나 AABB의 경우 r에 대한 공식은 단순화됩니다.

AABB의 경우 r에 대한 단순화된 공식을 구하십시오.

제16장 (P)ICKING

이 장에서는 사용자가 마우스 커서로 선택한 3D 객체(또는 프리미티브)를 결정하는 문제를 다룹니다(그림 16.1 참조). 즉, 마우스 커서의 2D 화면 좌표를 주어졌을 때, 그 지점에 투영된 3D 객체를 결정할 수 있을까요? 이 문제를 해결하기 위해, 어떤 의미에서는 역방향으로 작업해야 합니다. 즉, 일반적으로 3D 공간에서 화면 공간으로 변환하지만, 여기서는 화면 공간에서 다시 3D 공간으로 변환합니다. 물론, 이미 약간의 문제가 있습니다: 2D 화면 점은 고유한 3D 점 하나에 대응하지 않습니다(즉, 여러 개의 3D 점이 동일한 2D 투영 창 점에 투영될 수 있음—그림 16.2 참조). 따라서 실제로 선택된 객체가 무엇인지 결정하는 데 약간의 모호성이 존재합니다. 그러나 이는 그리 큰 문제가 아닙니다. 카메라에 가장 가까운 객체가 대개 우리가 원하는 객체이기 때문입니다.

그림 16.3을 고려해 보자. 이 그림은 시야 원뿔을 보여준다. 여기서 p 는 클릭된 화면 점에 대응하는 투영 창상의 점이다.

이제 눈 위치에서 시작하여 p 를 통과하는 *피킹 광선*을 쏘면 p 를 둘러싼 물체, 즉 이 예시에서는 원통과 교차함을 알 수 있습니다. 따라서 우리의 전략은 다음과 같습니다: 피킹 광선을 계산한 후 장면 내 각 물체를 순회하며 광선이 교차하는지 테스트합니다. 레이가 교차하는 오브젝트가 사용자가 선택한 오브젝트입니다. 앞서 언급했듯이, 오브젝트들이 레이 경로상에 있지만 깊이 값이 서로 다른 경우(예: 아무것도 선택되지 않은 경우) 레이가 여러 장면 오브젝트와 교차할 수 있습니다. 이 경우 카메라에 가장 가까운 교차 오브젝트를 선택된 오브젝트로 취하면 됩니다.

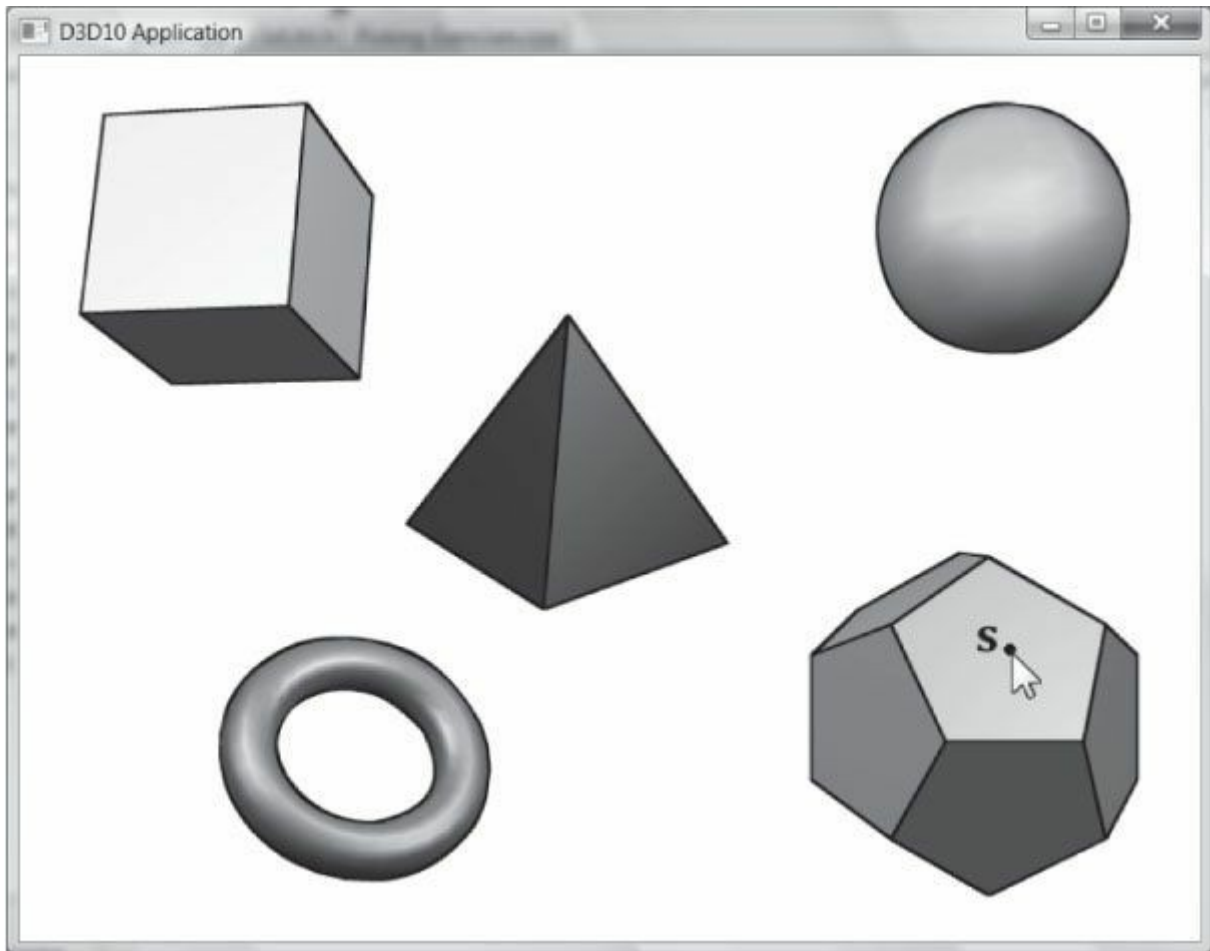


그림 16.1. 사용자가 정이십면체를 선택하는 모습.

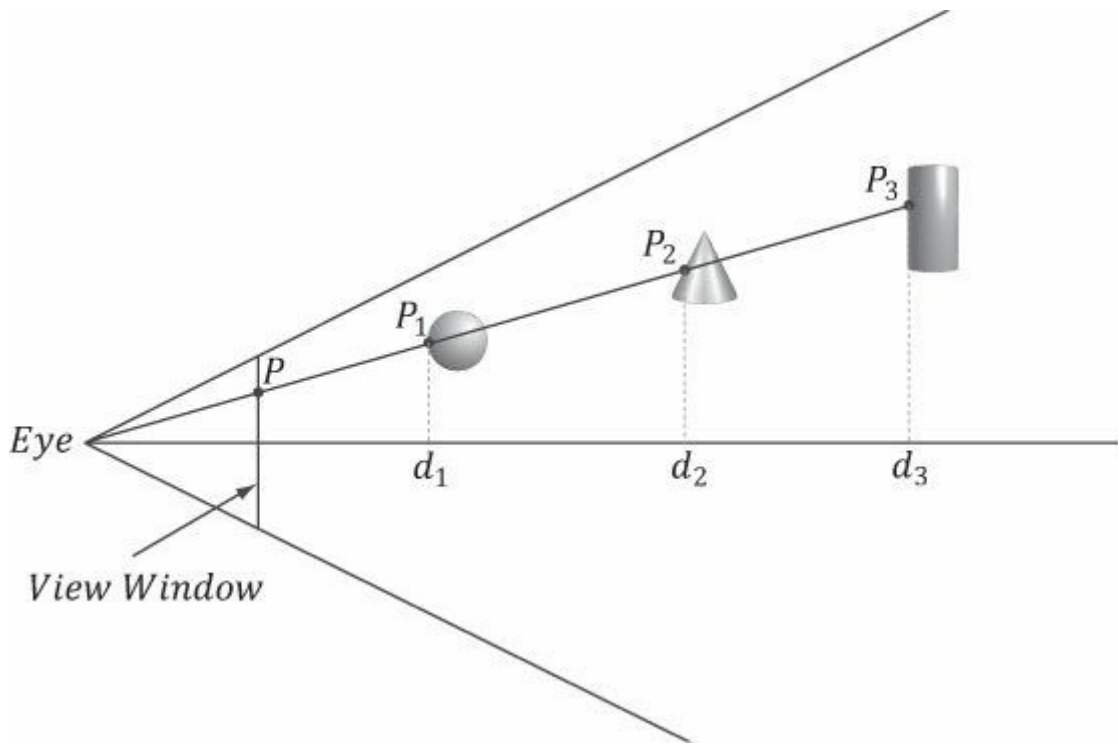


그림 16.2. 원뿔의 측면도. 3차원 공간의 여러 점이 투영 창 내의 한 점으로 투영될 수 있음을 관찰하십시오.

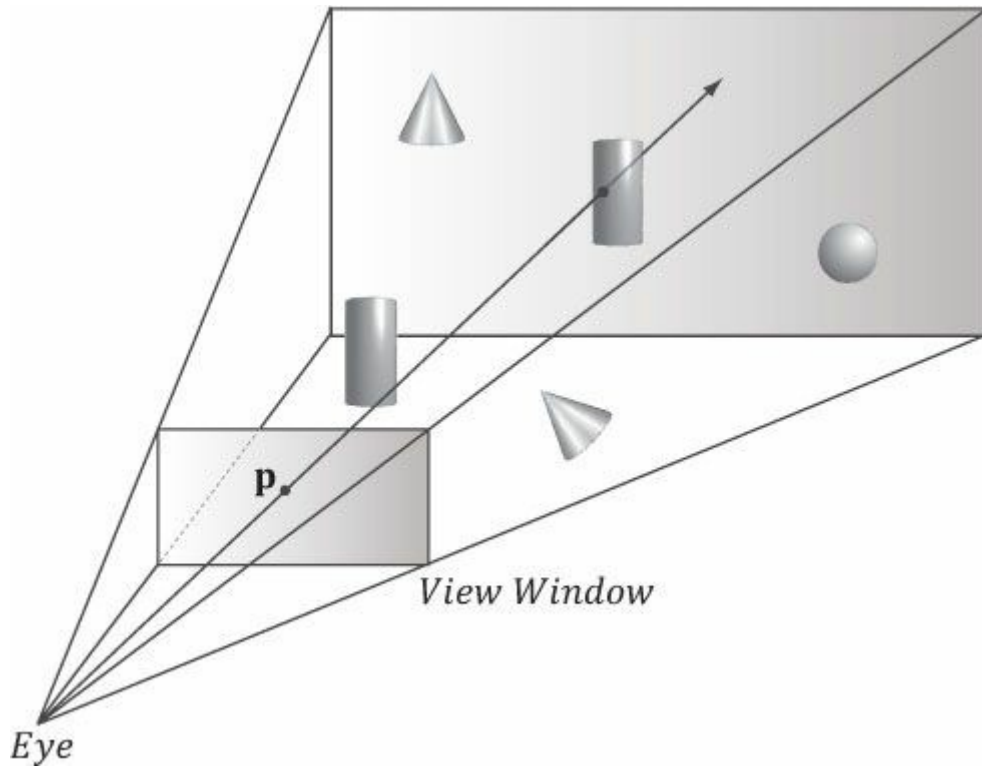


그림 16.3. p 를 통과하는 광선은 p 를 둘러싼 사물의 투영과 교차한다. 투영 창에 투영된 점 p 는 클릭된 화면 점 s 에 대응함을 유의하라.

목표:

1. 픽킹 알고리즘 구현 방법과 작동 원리를 학습한다. 픽킹을 다음 네 단계로 분해한다.

클릭된 화면 점 s 를 주어졌을 때, 투영 창 상의 대응하는 점을 찾아 p 라고 부릅니다.

뷰 공간에서 픽킹 광선을 계산합니다. 즉, 뷰 공간의 원점에서 시작하여 p 를 관통하는 광선입니다. 픽킹 광선과 해당 광선으로 테스트할 모델들을 동일한 공간으로 변환합니다.

픽킹 광선이 교차하는 오브젝트를 결정합니다. 카메라로부터 가장 가까운 교차 오브젝트가 픽킹된 화면 오브젝트에 해당합니다.

객체에 해당한다.

16.1 화면에서 투영 창으로의 변환

첫 번째 작업은 클릭된 화면 좌표를 정규화된 장치 좌표(Normalized Device Coordinates, §5.4.3.3 참조)로 변환하는 것입니다. 뷰포트 행렬은 정규화된 장치 좌표의 정점을 화면 공간으로 변환하며, 아래와 같습니다:

$$\mathbf{M} = \begin{bmatrix} \frac{Width}{2} & 0 & 0 & 0 \\ 0 & -\frac{Height}{2} & 0 & 0 \\ 0 & 0 & MaxDepth - MinDepth & 0 \\ TopLeftX + \frac{Width}{2} & TopLeftY + \frac{Height}{2} & MinDepth & 1 \end{bmatrix}$$

뷰포트 행렬의 변수는 `D3D11_VIEWPORT` 구조체의 변수를 참조합니다:

```
typedef struct D3D11_VIEWPORT { FLOAT
    TopLeftX;
    FLOAT TopLeftY;
    FLOAT Width; FLOAT
    Height; FLOAT
    MinDepth; FLOAT
    MaxDepth;
} D3D11_VIEWPORT;
```

일반적으로 게임에서 뷰포트는 백버퍼 전체이며 깊이 버퍼 범위는 0부터 1까지입니다. 따라서 $TopLeftX = 0$, $TopLeftY = 0$, $MinDepth = 0$, $MaxDepth = 1$, $Width = w$, $Height = h$ 가 됩니다. 여기서 w 와 h 는 각각 백버퍼의 너비와 높이입니다. 실제로 이 경우라면 뷰포트 행렬은 다음과 같이 단순화됩니다:

$$\mathbf{M} = \begin{bmatrix} w/2 & 0 & 0 & 0 \\ 0 & -h/2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ w/2 & h/2 & 0 & 1 \end{bmatrix}$$

이제 $\mathbf{p}_{ndc} = (x_{ndc}, y_{ndc}, z_{ndc}, 1)$ 을 정규화된 장치 공간(즉, $-1 \leq x_{ndc} \leq 1$, $-1 \leq y_{ndc} \leq 1$, $0 \leq z_{ndc} \leq 1$) 내의 점이라 하자. $\mathbf{p}_{ndc}$ 를 화면 공간으로 변환하면 다음과 같습니다:

$$\begin{bmatrix} x_{ndc} & y_{ndc} & z_{ndc} & 1 \end{bmatrix} \begin{bmatrix} w/2 & 0 & 0 & 0 \\ 0 & -h/2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ w/2 & h/2 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \frac{x_{ndc}w + w}{2}, \frac{-y_{ndc}h + h}{2}, z_{ndc}, 1 \end{bmatrix}$$

z_{ndc} 좌표는 깊이 버퍼에서만 사용되며, 피킹을 위한 깊이 좌표는 고려 대상이 아닙니다. $\mathbf{p}_{ndc}$ 에 대응하는 2차원 화면 점 $\mathbf{p}_s = (x_s, y_s)$ 는 변환된 x 및 y 좌표입니다:

$$x_s = \frac{x_{nac}w + w}{2}$$

$$y_s = \frac{-y_{nac}h + h}{2}$$

이전 방정식은 정규화된 장치 점 $\mathbf{p}_{ndc}$ 와 뷰포트 크기를 기준으로 화면 점 $\mathbf{p}_s$ 를 제공합니다. 그러나 피킹 상황에서는 초기 상태로 화면 점 $\mathbf{p}_s$ 와 뷰포트 크기가 주어지며, 우리는 $\mathbf{p}_{(ndc)}$ 를 구하고자 합니다. $\mathbf{p}_{ndc}$ 에 대해 이전 방정식을 풀면 다음과 같습니다:

$$x_{nac} = \frac{2x_s}{w} - 1$$

$$y_{nac} = -\frac{2y_s}{h} + 1$$

이제 클릭된 점을 NDC 공간에서 얻었습니다. 하지만 피킹 광선을 발사하려면 뷰 공간에서의 화면 점이 필요합니다. §5.6.3.3에서 회상하듯, 우리는 뷰 공간에서 투영된 점을 x좌표를 $중형h/r$ 로 나누어 NDC 공간으로 매핑했습니다:

$$-r \leq x' \leq r$$

$$-1 \leq x'/r \leq 1$$

따라서 뷰 공간으로 되돌아가려면 NDC 공간의 x 좌표에 중형비를 곱하기만 하면 됩니다. 뷰 공간에서 클릭된 점은 다음과 같습니다:

$$x_v = r \left(\frac{2s_x}{w} - 1 \right)$$

$$y_v = -\frac{2s_y}{h} + 1$$

Note: 뷰 공간에서 투영된 y좌표는 NDC 공간에서 동일합니다. 이는 뷰 공간에서 투영 창의 높이를 $[-1, 1]$ 구간을 덮도록 설정했기 때문입니다.

이제 §5.6.3.1에서 회상하듯, 투영 창은 원점으로부터 거리 $d = \cot\left(\frac{\alpha}{2}\right)$ 에 위치하며, 여기서 α 는 수직 시야각입니다. 시야각. 따라서 피킹 광선을 투영 창상의 점 (x_v, y_v, d) 를 통과하도록 쏠 수 있습니다. 그러나 이를 위해서는 더 간단한 방법은 [그림 16.4](#)에서 다음과 같이 관찰하는 것이다:

$$d = \cot\left(\frac{\alpha}{2}\right)$$

$$x'_v = \frac{x_v}{d} = \frac{x_v}{\cot\left(\frac{\alpha}{2}\right)} = x_v \cdot \tan\left(\frac{\alpha}{2}\right) = \left(\frac{2s_x}{w} - 1\right) r \tan\left(\frac{\alpha}{2}\right)$$

$$y'_v = \frac{y_v}{d} = \frac{y_v}{\cot\left(\frac{\alpha}{2}\right)} = y_v \cdot \tan\left(\frac{\alpha}{2}\right) = \left(-\frac{2s_y}{h} + 1\right) \tan\left(\frac{\alpha}{2}\right)$$

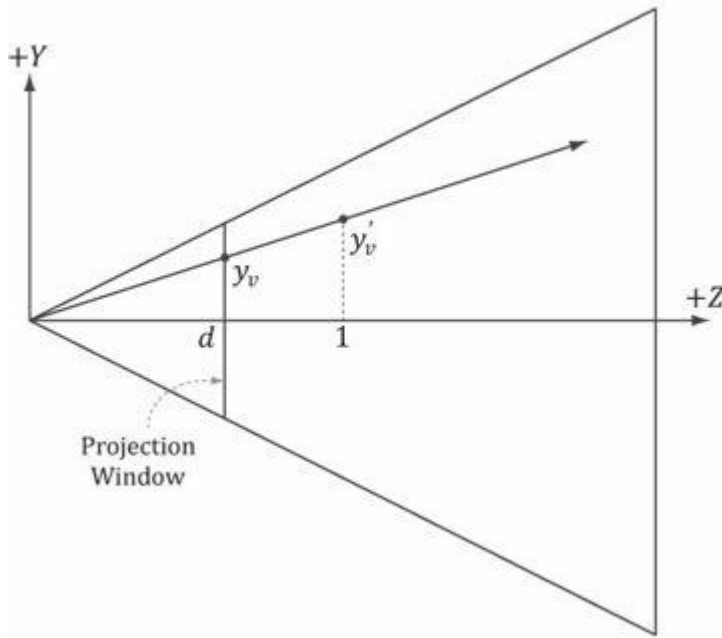


그림 16.4. 유사 삼각형에 의해,

$$\frac{y_v}{d} = \frac{y'_v}{1} \text{ 그리고 } \frac{x_v}{d} = \frac{x'_v}{1}.$$

투영 행렬에서 $\mathbf{P}_{00} = \frac{1}{r \tan(\frac{\alpha}{2})}$ 및 $\mathbf{P}_{11} = \frac{1}{\tan(\frac{\alpha}{2})}$ 임을 상기하면, 이를 다음과 같이 다시 쓸 수 있습니다:

$$x'_v = \left(\frac{2s_x}{w} - 1 \right) / \mathbf{P}_{00}$$

$$y'_v = \left(-\frac{2s_y}{h} + 1 \right) / \mathbf{P}_{11}$$

따라서 피킹 광선을 점 $(x'_v, y'_v, 1)$ 을 통과하도록 발사할 수 있습니다. 이는 점 (x_v, y_v, d) 를 통과하는 광선과 동일한 결과를 산출합니다. 뷰 공간에서 피킹 광선을 계산하는 코드는 다음과 같습니다:

```
void PickingApp::Pick(int sx, int sy)
{
    XMATRIX P = mCam.Proj();

    // 뷰 공간에서 피킹 광선 계산.
    float vx = (+2.0f*sx/mClientWidth - 1.0f)/P(0,0); float vy = (-
    2.0f*sy/mClientHeight + 1.0f)/P(1,1);

    // 뷰 공간에서의 광선 정의.
    XMVECTOR rayOrigin = XMVectorSet(0.0f, 0.0f, 0.0f, 1.0f); XMVECTOR rayDir =
    XMVectorSet(vx, vy, 1.0f, 0.0f);
```

시점 공간에서 눈이 시점 공간의 원점에 위치하기 때문에 광선이 시점 공간의 원점에서 시작됨에 유의하십시오.

16.2 월드/로컬 공간 픽킹 레이

지금까지 뷰 공간에서의 피킹 광선을 다루었으나, 이는 오브젝트 역시 뷰 공간에 있을 때만 유용합니다. 뷰 행렬이 지오메트리를 월드 공간에서 뷰 공간으로 변환하므로, 뷰 행렬의 역행렬은 지오메트리를 뷰 공간에서 월드 공간으로 변환합니다. $\mathbf{r}_v(t) = \mathbf{q} + t\mathbf{u}$ 가 뷰 공간 피킹 광선이고 $\mathbf{v}$ 가 뷰 행렬이라면, 월드 공간 피킹 광선은 다음과 같이 주어집니다:

$$\begin{aligned}\mathbf{r}_w(t) &= \mathbf{q}\mathbf{V}^{-1} + t\mathbf{u}\mathbf{V}^{-1} \\ &= \mathbf{q}_w + t\mathbf{u}_w\end{aligned}$$

레이 원점 **q**는 점으로 변환되며(즉, $q_w = 1$), 레이 방향 **u**는 벡터로 변환됩니다(즉, $u_w = 0$). 월드 스페이스 픽킹 레이는 일부 객체가 월드 스페이스에 정의된 상황에서 유용할 수 있습니다. 그러나 대부분의 경우 객체의 기하학적 구조는 해당 객체 자체의 로컬 공간을 기준으로 정의됩니다. 따라서 레이/객체 교차 검사를 수행하려면 레이를 객체의 로컬 공간으로 변환해야 합니다. 대부분의 경우, 객체의 기하 구조는 객체 자체의 로컬 공간을 기준으로 정의됩니다. 따라서 광선/객체 교차 검사를 수행하려면 광선을 객체의 로컬 공간으로 변환해야 합니다. 객체의 월드 행렬을 $\mathbf{W}$ 라고 할 때, 행렬 $\mathbf{W}^{(-1)}$ 은 기하 구조를 월드 공간에서 객체의 로컬 공간으로 변환합니다. 따라서 로컬 공간 픽킹 광선은 다음과 같습니다:

$$\mathbf{r}_L(t) = \mathbf{q}_w\mathbf{W}^{-1} + t\mathbf{u}_w\mathbf{W}^{-1}$$

일반적으로 장면 내 각 객체는 고유한 국소 공간을 가집니다. 따라서 교차 검사를 수행하려면 광선을 장면 내 각 객체의 국소 공간으로 변환해야 합니다. 메쉬를 월드 공간으로 변환한 후 교차 검사를 수행하라고 제안할 수도 있습니다. 그러나 이는 너무 많은 비용이 듭니다. 메쉬에는 수천 개의 정점이 포함될 수 있으며, 이 모든 정점을 월드 공간으로 변환해야 합니다. 레이를 오브젝트의 로컬 공간으로 변환하는 것이 훨씬 효율적입니다. 다음 코드는 뷰 공간에서 객체의 로컬 공간으로 피킹 광선을 변환하는 방법을 보여줍니다:

```
// 레이를 메시의 로컬 공간으로 변환
XMMATRIX V = mCam.View();
XMMATRIX invView = XMMatrixInverse(&XMMatrixDeterminant(V), V); XMMATRIX W =
XMLoadFloat4x4(&mMeshWorld);

XMMATRIX invWorld = XMMatrixInverse(&XMMatrixDeterminant(W), W); XMMATRIX toLocal =
XMMatrixMultiply(invView, invWorld);

레이 원점 = XMVector3TransformCoord(레이 원점, toLocal); 레이 방향 =
XMVector3TransformNormal(레이 방향, toLocal);

// 교차 테스트를 위해 광선 방향을 단위 길이로 만듭니다. rayDir =
XMVector3Normalize(rayDir);
```

`XMVector3TransformCoord` 및 `XMVector3TransformNormal` 함수는 3D 벡터를 매개변수로 받지만, `XMVector3TransformCoord` 함수의 경우 4번째 성분에 대해 암묵적으로 $w = 1$ 이 적용된다는 점에 유의하십시오. 반면 `XMVector3TransformNormal` 함수는 4번째 성분에 대해 암묵적으로 $w = 0$ 으로 간주됩니다. 따라서 `XMVector3TransformCoord`는 점 변환에, `XMVector3TransformNormal`은 벡터 변환에 사용할 수 있습니다.

16.3 레이/메쉬 교차

픽킹 레이와 메쉬가 동일한 공간에 존재하면, 픽킹 레이가 메쉬와 교차하는지 확인하기 위해 교차 테스트를 수행할 수 있습니다. 다음 코드는 메쉬 내 각 삼각형을 순회하며 레이/삼각형 교차 테스트를 수행합니다. 레이가 삼각형 중 하나와 교차하면, 해당 삼각형이 속한 메쉬를 맞춘 것입니다. 그렇지 않으면 광선이 메시를 통과한 것입니다. 일반적으로 가장 가까운 삼각형 교차점을 원합니다. 광선에 대해 삼각형들이 겹치는 경우 광선이 여러 메시 삼각형과 교차할 수 있기 때문입니다.

```
// 메쉬의 바운딩 박스에 충돌했다면,
// 메쉬 삼각형을 선택했을 수 있으므로, 광선/삼각형 검사를 수행합니다.
//
// 바운딩 박스에 충돌하지 않았다면 메시에 충돌할 수 없으므로
// 레이/삼각형 테스트를 수행하는 노력을 낭비하지 마십시오.

// 아직 아무것도 선택하지 않은 것으로 가정하고 초기값을 -1로 설정합니다.
mPickedTriangle = -1;
float tmin = 0.0f; if(XNA::IntersectRayAxisAlignedBox(rayOrigin, rayDir, &mMeshBox,
&tmin))
{
    // 가장 가까운 레이/삼각형 교차점을 찾습니다. tmin =
    MathHelper::Infinity;
```

```
for(UINT i = 0; i < mMeshIndices.size()/3; ++i)
{

    // 이 삼각형의 인덱스.
    UINT i0 = mMeshIndices[i*3+0]; UINT i1 =
    mMeshIndices[i*3+1];    UINT    i2    =
    mMeshIndices[i*3+2];

    // 이 삼각형의 정점들.
    XMVECTOR v0 = XMLoadFloat3(&mMeshVertices[i0].Pos); XMVECTOR v1 =
    XMLoadFloat3(&mMeshVertices[i1].Pos);    XMVECTOR    v2    =
    XMLoadFloat3(&mMeshVertices[i2].Pos);

    // 가장 가까운 교차점을 찾기 위해 모든 삼각형을 반복해야 합니다.

    // 가장 가까운 교차점을 찾기 위해 모든
    삼각형을 반복해야 합니다. float t = 0.0f;

    if(XNA::IntersectRayTriangle(rayOrigin, rayDir, v0, v1, v2, &t))
    {
        if(t < tmin)
        {
            // 새로 선택된 가장 가까운 삼각형입니다. tmin = t;

            mPickedTriangle = i;
        }
    }
}
```

픽킹을 수행하기 위해, 우리는 메시 지오메트리(정점과 인덱스)의 시스템 메모리 복사본을 유지합니다. 이는 읽기용 정적 정점/인덱스 버퍼에 접근할 수 없기 때문입니다. 픽킹이나 충돌 감지와 같은 작업을 위해 지오메트리의 시스템 메모리 복사본을 저장하는 것은 일반적입니다. 때로는 메모리와 연산을 절약하기 위해 이러한 목적을 위해 메시의 단순화된 버전을 저장하기도 합니다.

16.3.1 레이/AABB 교차

먼저 XNA 충돌 라이브러리 함수 `XNA::IntersectRayAxisAlignedBox`를 사용하여 광선이 메시의 바운딩 박스와 교차하는지 확인합니다. 이는 프러스텀 컬링 최적화와 유사합니다. 씬 내 모든 삼각형에 대해 광선 교차 테스트를 수행하면 계산 시간이 누적됩니다. 선택 광선 근처에 있지 않은 메시라도 광선이 메시지를 통과하지 않는다고 결론 내리기 위해 각 삼각형을 반복 처리해야 하므로 이는 낭비적이고 비효율적입니다. 일반적인 전략은 메쉬를 구나 박스와 같은 단순한 경계 볼륨으로 근사화하는 것입니다. 그런 다음 광선을 메쉬와 교차시키는 대신, 먼저 광선을 경계 볼륨과 교차시킵니다. 광선이 경계 볼륨을 통과한다면, 광선은 필연적으로 삼각형 메쉬를 통과하므로 추가 계산이 필요 없습니다. 광선이 경계 볼륨과 교차한다면, 더 정밀한 광선/메쉬 테스트를 수행합니다. 장면 내 대부분의 경계 볼륨을 광선이 통과할 것이라고 가정하면, 이로 인해 수많은 광선/삼각형 교차 테스트를 줄일 수 있습니다. `XNA::IntersectRayAxisAlignedBox` 함수는 광선이 박스와 교차할 경우 `true`를, 그렇지 않을 경우 `false`를 반환합니다. 프로토타입은 다음과 같습니다:

```
BOOL IntersectRayAxisAlignedBox(    FXMVECTOR Origin, // 광선
                                    원점
                                    FXMVECTOR Direction, // 광선 방향 (단위 길이여야 함) const AxisAlignedBox* pVolume,
                                    // 박스
                                    FLOAT* pDist); // 광선 교차 매개변수
```

광선 $\mathbf{r}(t) = \mathbf{q} + t\mathbf{u}$ 가 주어졌을 때, 마지막 매개변수는 실제 교점 $\mathbf{p}$ 를 산출하는 광선 매개변수 t_0 을 출력한다:

$$\mathbf{p} = \mathbf{r}(t_0) = \mathbf{q} + t_0\mathbf{u}$$

16.3.2 광선/구체 교차

XNA 충돌 라이브러리에는 광선/구체 교차 테스트도 제공됩니다:

```
BOOL IntersectRaySphere( FXMVECTOR
                          Origin, FXMVECTOR Direction,
                          const Sphere* pVolume,
```


Float* pDist);

이러한 테스트의 예시를 보여주기 위해, 광선/구체 교차 테스트를 유도하는 방법을 설명한다. 중심 $\mathbf{c}$, 반지름 r 을 가진 구체 표면의 점 $\mathbf{p}$ 는 다음 방정식을 만족한다:

$$\|\mathbf{p} - \mathbf{c}\| = r$$

$\mathbf{r}(t) = \mathbf{q} + t\mathbf{u}$ 를 광선이라 하자. 우리는 $\mathbf{r}(t_1)$ 과 $\mathbf{r}(t_2)$ 가 구 방정식을 만족하도록 $t_{(1)}$ 과 $t_{(2)}$ 를 구하고자 한다 (즉, 교차점을 생성하는 광선 상의 매개변수 t_1 와 $t_{(2)}$ 가 교차점을 생성하는 광선 상의 값임).

$$r = \|\mathbf{r}(t) - \mathbf{c}\|$$

$$r^2 = (\mathbf{r}(t) - \mathbf{c}) \times (\mathbf{r}(t) - \mathbf{c})$$

$$r^2 = (\mathbf{q} + t\mathbf{u} - \mathbf{c}) \times (\mathbf{q} + t\mathbf{u} - \mathbf{c})$$

$$r^2 = (\mathbf{q} - \mathbf{c} + t\mathbf{u}) \times (\mathbf{q} - \mathbf{c} + t\mathbf{u})$$

기호상의 편의를 위해 $\mathbf{m} = \mathbf{q} - \mathbf{c}$ 라 하자.

$$(\mathbf{m} + t\mathbf{u}) \cdot (\mathbf{m} + t\mathbf{u}) = r^2$$

$$\mathbf{m} \cdot \mathbf{m} + 2t\mathbf{m} \cdot \mathbf{u} + t^2\mathbf{u} \cdot \mathbf{u} = r^2$$

$$t^2\mathbf{u} \cdot \mathbf{u} + 2t\mathbf{m} \cdot \mathbf{u} + \mathbf{m} \cdot \mathbf{m} - r^2 = 0$$

이는 다음의 이차방정식이다:

$$a = \mathbf{u} \cdot \mathbf{u}$$

$$b = 2(\mathbf{m} \cdot \mathbf{u})$$

$$c = \mathbf{m} \cdot \mathbf{m} - r^2$$

광선 방향이 단위 길이일 때, $\mathbf{a} = \mathbf{u} \cdot \mathbf{u} = 1$. 해에 허수 성분이 있으면 광선이 구를 빗나간다. 두 실수 해가 같으면 광선이 구에 접하는 한 점을 교차한다. 두 실수 해가 다르면 광선이 구의 두 점을 관통한다. 음수 해는 광선 "뒤쪽"의 교점을 나타낸다. 가장 작은 양수 해가 가장 가까운 교점 매개변수를 제공한다.

16.3.3 광선/삼각형 교차

광선/삼각형 교차 테스트를 수행하기 위해 XNA 충돌 라이브러리 함수 `XNA::IntersectRayTriangle`을 사용합니다:

```
bool IntersectRayTriangle( FXMVECTOR Origin, // 광선 원점
    FXMVECTOR Direction, // 광선 방향 (단위 길이)
    FXMVECTOR V0, // 삼각형 꼭짓점 v0
    CXMVECTOR V1, // 삼각형 꼭짓점 v1
    CXMVECTOR V2, // 삼각형 꼭짓점 v2
    float* pDist); // 광선 교차 매개변수
```

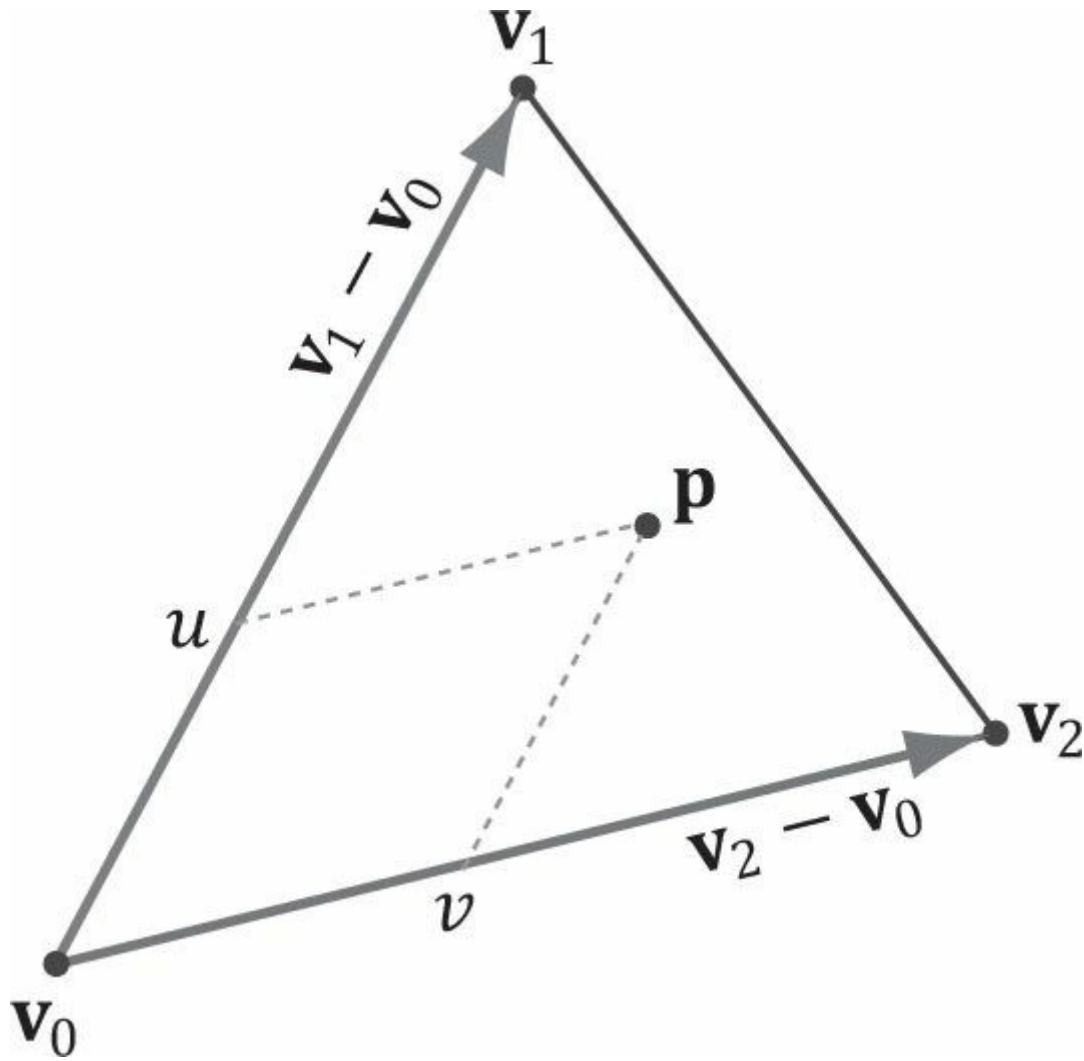


그림 16.5. 삼각형 평면상의 점 p 는 원점이 v_0 이고 축이 $v_1 - v_0$ 및 $v_2 - v_0$ 인 비스듬한 좌표계에 대해 좌표 (u, v) 를 가진다.

$\mathbf{r}(t) = \mathbf{q} + t\mathbf{u}$ 를 광선이라 하고, $\mathbf{T}(u, v) = \mathbf{v}_0 + u(\mathbf{v}_1 - \mathbf{v}_0) + v(\mathbf{v}_2 - \mathbf{v}_0)$ ($u \geq 0, v \geq 0, u + v \leq 1$)를 삼각형이라 하자(그림 16.5 참조).

우리는 $\mathbf{r}(t) = \mathbf{T}(u, v)$ (즉, 광선과 삼각형이 교차하는 점)을 만족하는 t, u, v 를 동시에 구하고자 한다:

$$\mathbf{r}(t) = \mathbf{T}(u, v)$$

$$\mathbf{q} + t\mathbf{u} = \mathbf{v}_0 + u(\mathbf{v}_1 - \mathbf{v}_0) + v(\mathbf{v}_2 - \mathbf{v}_0)$$

$$-t\mathbf{u} + u(\mathbf{v}_1 - \mathbf{v}_0) + v(\mathbf{v}_2 - \mathbf{v}_0) = \mathbf{q} - \mathbf{v}_0$$

기호상의 편의를 위해, $\mathbf{e}_1 = \mathbf{v}_1 - \mathbf{v}_0$, $\mathbf{e}_2 = \mathbf{v}_2 - \mathbf{v}_0$ 및 $\mathbf{m} = \mathbf{q} - \mathbf{v}_0$ 이라 하자:

$$-t\mathbf{u} + u\mathbf{e}_1 + v\mathbf{e}_2 = \mathbf{m}$$

$$\begin{bmatrix} \uparrow & \uparrow & \uparrow \\ -\mathbf{u} & \mathbf{e}_1 & \mathbf{e}_2 \\ \downarrow & \downarrow & \downarrow \end{bmatrix} \begin{bmatrix} t \\ u \\ v \end{bmatrix} = \begin{bmatrix} \uparrow \\ \mathbf{m} \\ \downarrow \end{bmatrix}$$

행렬 방정식 $\mathbf{Ax} = \mathbf{b}$ 를 고려하자. 여기서 $\mathbf{A}$ 는 가역 행렬이다. 그러면 크라머의 법칙에 따라 $x_i = \det \mathbf{A}_i / \det \mathbf{A}$ 가 성립한다. 여기서 $\mathbf{A}_i$ 는 $\mathbf{A}$ 의 i 번째 열 벡터를 $\mathbf{b}$ 와 교환하여 구한다. 따라서,

$$t = \det \begin{bmatrix} \uparrow & \uparrow & \uparrow \\ \mathbf{m} & \mathbf{e}_1 & \mathbf{e}_2 \\ \downarrow & \downarrow & \downarrow \end{bmatrix} / \det \begin{bmatrix} \uparrow & \uparrow & \uparrow \\ -\mathbf{u} & \mathbf{e}_1 & \mathbf{e}_2 \\ \downarrow & \downarrow & \downarrow \end{bmatrix}$$

$$u = \det \begin{bmatrix} \uparrow & \uparrow & \uparrow \\ -\mathbf{u} & \mathbf{m} & \mathbf{e}_2 \\ \downarrow & \downarrow & \downarrow \end{bmatrix} / \det \begin{bmatrix} \uparrow & \uparrow & \uparrow \\ -\mathbf{u} & \mathbf{e}_1 & \mathbf{e}_2 \\ \downarrow & \downarrow & \downarrow \end{bmatrix}$$

$$v = \det \begin{bmatrix} \uparrow & \uparrow & \uparrow \\ -\mathbf{u} & \mathbf{e}_1 & \mathbf{m} \\ \downarrow & \downarrow & \downarrow \end{bmatrix} / \det \begin{bmatrix} \uparrow & \uparrow & \uparrow \\ -\mathbf{u} & \mathbf{e}_1 & \mathbf{e}_2 \\ \downarrow & \downarrow & \downarrow \end{bmatrix}$$

다음 사실을 이용하면

$$\det \begin{bmatrix} \uparrow & \uparrow & \uparrow \\ \mathbf{a} & \mathbf{b} & \mathbf{c} \\ \downarrow & \downarrow & \downarrow \end{bmatrix} = \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$$

이를 다음과 같이 재구성할 수 있습니다:

$$t = -\mathbf{m} \cdot (\mathbf{e}_1 \times \mathbf{e}_2) / \mathbf{u} \cdot (\mathbf{e}_1 \times \mathbf{e}_2)$$

$$u = \mathbf{u} \cdot (\mathbf{m} \times \mathbf{e}_2) / \mathbf{u} \cdot (\mathbf{e}_1 \times \mathbf{e}_2)$$

$$v = \mathbf{u} \cdot (\mathbf{e}_1 \times \mathbf{m}) / \mathbf{u} \cdot (\mathbf{e}_1 \times \mathbf{e}_2)$$

계산을 조금 최적화하기 위해, 행렬에서 열을 교환할 때마다 행렬식의 부호가 바뀐다는 사실을 활용할 수 있습니다:

$$t = \mathbf{e}_2 \cdot (\mathbf{m} \times \mathbf{e}_1) / \mathbf{e}_1 \cdot (\mathbf{u} \times \mathbf{e}_2)$$

$$u = \mathbf{m} \cdot (\mathbf{u} \times \mathbf{e}_2) / \mathbf{e}_1 \cdot (\mathbf{u} \times \mathbf{e}_2)$$

$$v = \mathbf{u} \cdot (\mathbf{m} \times \mathbf{e}_1) / \mathbf{e}_1 \cdot (\mathbf{u} \times \mathbf{e}_2)$$

계산에서 재사용할 수 있는 공통 교차곱을 주목하십시오: $\mathbf{m} \times \mathbf{e}_1$ 및 $\mathbf{u} \times \mathbf{e}_2$.

16.4 데모 애플리케이션

이 장의 데모는 자동차 메쉬를 렌더링하며, 사용자가 마우스 오른쪽 버튼을 눌러 삼각형을 선택할 수 있게 합니다. 우리의 광선/삼각형 교차 루프 (§16.3)에서 선택된 삼각형의 인덱스를 변수 `mPickedTriangle`에 캐시합니다. 선택된 삼각형의 인덱스를 알게 되면, 선택된 삼각형을 강조 표시하는 재질을 사용하여 이 삼각형을 다시 그릴 수 있습니다(그림 16.6 참조):



그림 16.6. 선택된 삼각형이 녹색으로 강조 표시됨.

```
// 선택된 삼각형만 다른 머티리얼로 다시 그려
// if(mPickedTriangle != -1)
{
    // 깊이 테스트를 <에서 <=로 변경하여 동일한 삼각형을 두 번 그렸을 때도 깊이 테스트를 통과하도록 합니다.
    // 삼각형을 두 번 그릴 경우에도 깊이 테스트를 통과하도록 합니다. 이는
    // 선택된 삼각형을 다른
    // 재그리므로 발생합니다. <=을 사용하지 않으면 삼각형은
    // 두 번째로 그리려고 할 때 깊이 테스트에 실패하게 됩니다.

    md3dImmediateContext->OMSetDepthStencilState(
        RenderStates::LessEqualDSS, 0);
    Effects::BasicFX->SetMaterial(mPickedTriangleMat);
    activeMeshTech->GetPassByIndex(p)->Apply(0, md3dImmediateContext);

    // 삼각형 하나만 그리기—3개의 인덱스. 선택된 삼각형에 오프셋 적용
    // 삼각형에 오프셋 적용.
    md3dImmediateContext->DrawIndexed(3, 3*mPickedTriangle, 0);

    // 기본값 복원
    md3dImmediateContext->OMSetDepthStencilState(0, 0);
}
```

Note: 'I' 키를 길게 누르면 와이어프레임 모드로 메시를 볼 수 있습니다.

16.5 요약

1. 피킹(picking)은 사용자가 마우스로 클릭한 화면에 표시된 2D 투영 객체에 대응하는 3D 객체를 결정하는 기술입니다.
2. 픽킹 레이는 뷰 공간의 원점을 기점으로 투영 창상의 지점을 통과하는 레이를 발사하여 찾습니다.

클릭된 화면 점에 대응하는 점입니다.

3. 광선 $\mathbf{r}(t) = \mathbf{q} + t\mathbf{u}$ 는 변환 행렬을 사용하여 그 원점 $\mathbf{q}$ 와 방향 $\mathbf{u}$ 를 변환함으로써 변환할 수 있습니다. 원점은 점($w = 1$)으로 변환되고 방향은 벡터($w = 0$)로 처리된다는 점에 유의하십시오.
4. 광선이 오브젝트와 교차했는지 테스트하기 위해, 오브젝트 내 모든 삼각형에 대해 광선/삼각형 교차 테스트를 수행합니다. 광선이 삼각형 중 하나와 교차하면, 해당 삼각형이 속한 메쉬를 맞춘 것입니다. 그렇지 않으면 광선이 메쉬를 빗나간 것입니다. 일반적으로 가장 가까운 삼각형 교차점을 원합니다. 광선에 대해 삼각형들이 겹치는 경우, 광선이 여러 메쉬 삼각형과 교차할 수 있기 때문입니다.
5. 레이/메쉬 교차 테스트의 성능 최적화 기법은 먼저 레이와 메쉬를 근사하는 바운딩 볼륨 간의 교차 테스트를 수행하는 것이다. 레이 바운딩 볼륨이 볼륨을 통과하지 못하면, 레이 바운딩 볼륨이 볼륨을 통과하지 못하므로 추가 계산이 필요 없다. 레이 바운딩 볼륨과 교차하면, 더 정밀한 레이/메쉬 테스트를 수행한다. 장면 내 대부분의 경계 볼륨을 광선이 통과할 것이라고 가정하면, 이로 인해 많은 광선/삼각형 교차 테스트를 줄일 수 있습니다.

16.6 연습 문제

1. "Picking" 데모를 수정하여 메시에 AABB 대신 바운딩 스피어를 사용하도록 하십시오.
2. 레이/AABB 교차 테스트를 수행하는 알고리즘을 연구하세요.
3. 장면에 수천 개의 오브젝트가 존재하더라도, 선택을 위해 수천 번의 광선/경계 볼륨 테스트를 수행해야 합니다. 옥트리(*octree*)를 연구하고, 이를 활용하여 광선/경계 볼륨 교차 테스트를 줄이는 방법을 설명하세요. 참고로, 동일한 일반적인 전략은 프러스텀 컬링을 위한 프러스텀/경계 볼륨 교차 테스트 감소에도 적용됩니다.

챕터 17 큐브 매핑

이 장에서는 기본적으로 특별한 방식으로 해석되는 여섯 개의 텍스처 배열인 큐브 맵을 연구합니다. 큐브 매핑을 사용하면 하늘 텍스처링이나 모델 반사 표현을 쉽게 구현할 수 있습니다.

목표:

- 1. 큐브 맵이 무엇인지, 그리고 HLSL 코드에서 이를 샘플링하는 방법을 학습합니다.
- 2. DirectX 텍스처 툴을 사용하여 큐브 맵을 생성하는 방법을 알아봅니다.
- 3. 큐브 맵을 사용하여 반사를 모델링하는 방법을 알아보기 위해.
- 4. 큐브 맵을 사용하여 구에 텍스처를 적용해 하늘과 먼 산을 시뮬레이션하는 방법을 이해하기 위해.

17.1 큐브 매핑

큐브 매핑의 개념은 여섯 개의 텍스처를 저장하고 이를 특정 좌표계를 중심으로 축 정렬된 큐브의 면으로 시각화하는 것입니다. 큐브 텍스처가 축 정렬되어 있기 때문에 각 면은 세 개의 주요 축 방향과 대응됩니다. 따라서 면과 교차하는 축 방향($\pm X$, $\pm Y$, $\pm Z$)을 기반으로 큐브 맵의 특정 면을 참조하는 것이 자연스럽습니다. 큐브 맵 면을 식별하기 위해 DirectX는 `D3D11_TEXTURECUBE_FACE` 열거형 유형을 제공합니다:

```
typedef enum D3D11_TEXTURECUBE_FACE {
    D3D11_TEXTURECUBE_FACE_POSITIVE_X=0,
    D3D11_TEXTURECUBE_FACE_NEGATIVE_X=1,
    D3D11_TEXTURECUBE_FACE_POSITIVE_Y=2,
    D3D11_TEXTURECUBE_FACE_NEGATIVE_Y=3,
    D3D11_TEXTURECUBE_FACE_POSITIVE_Z=4,
    D3D11_TEXTURECUBE_FACE_NEGATIVE_Z=5
} D3D11_TEXTURECUBE_FACE;
```

큐브 맵은 6개의 요소로 구성된 텍스처 배열에 저장됩니다:

- 1. 인덱스 0은 +X 면을 가리킵니다.
- 2. 인덱스 1은 -X 면을 가리킵니다.
- 3. 인덱스 2는 +Y 면을 가리킵니다.
- 4. 인덱스 3은 -Y 면을 가리킵니다.
- 5. 인덱스 4는 +Z 면을 나타냅니다.
- 6. 인덱스 5는 -Z 면을 가리킵니다.

2D 텍스처링과 달리, 더 이상 2D 텍스처 좌표로 텍셀을 식별할 수 없습니다. 큐브 맵에서 텍셀을 식별하기 위해 우리는 3D 텍스처 좌표를 사용하며, 이는 원점을 기점으로 하는 3D *조각* 벡터 v 를 정의합니다. 벡터 v 가 큐브 맵에서 교차하는 텍셀([그림 17.1](#) 참조)은 v 의 3차원 좌표에 해당하는 텍셀입니다. [제8장에서](#) 논의된 텍스처 필터링 개념은 v 가 텍셀 샘플 사이의 지점과 교차하는 경우에도 적용됩니다.

 *록업 벡터의 크기는 중요하지 않으며 방향만 중요합니다. 방향은 같지만 크기가 다른 두 벡터는 큐브 맵에서 동일한 점을 샘플링합니다.*

HLSL에서 큐브 텍스처는 `TextureCube` 유형으로 표현됩니다. 다음 코드 조각은 큐브 맵을 샘플링하는 방법을 보여줍니다:

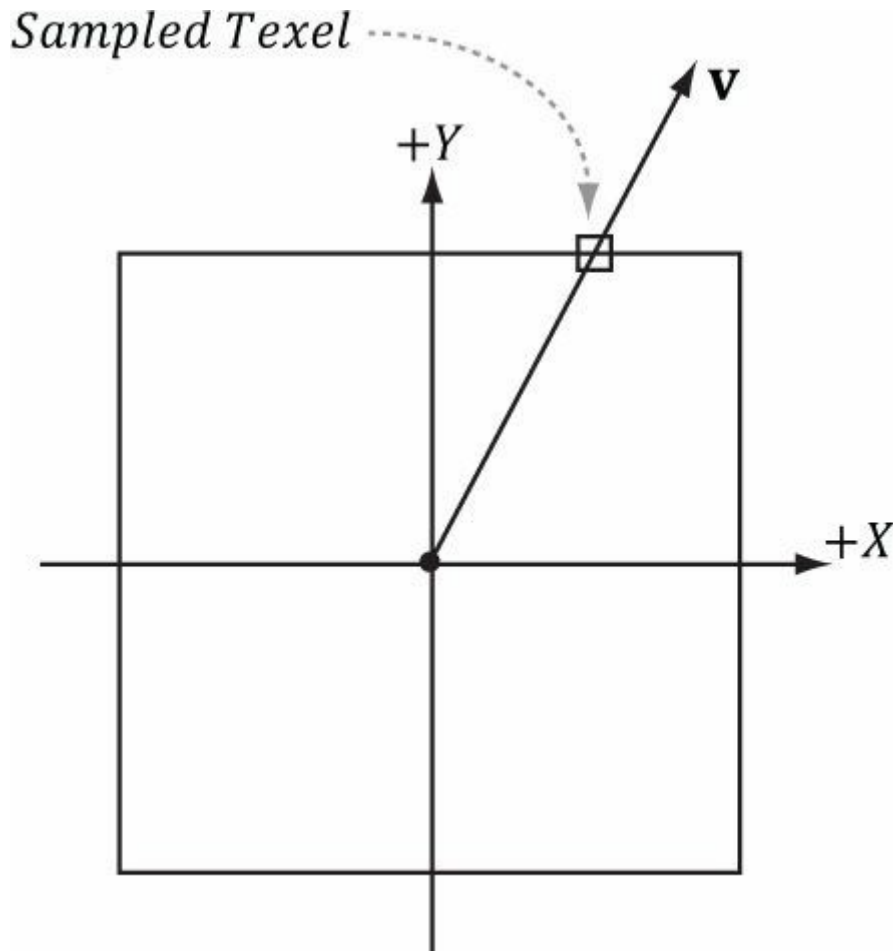


그림 17.1. 단순화를 위해 2차원으로 설명합니다. 3차원에서는 정사각형이 정육면체가 됩니다. 정사각형은 특정 좌표계에 대해 중심이 맞고 축이 정렬된 큐브 맵을 나타냅니다. 원점에서 벡터 v 를 씁니다. v 가 교차하는 텍셀이 샘플링된 텍셀입니다. 이 그림에서 v 는 $+Y$ 축에 해당하는 정육면체 면과 교차합니다.

```
TextureCube gCubeMap; SamplerState
gTriLinearSam
{
    Filter = MIN_MAG_MIP_LINEAR; AddressU =
    Wrap;
    AddressV = Wrap;
};

...

// 픽셀 셰이더에서
float3 v = float3(x,y,z); // 일부 조희 벡터
float4 color = gCubeMap.Sample(gTriLinearSam, v);
```

룩업 벡터는 큐브 맵이 상대적인 공간과 동일한 공간에 있어야 합니다. 예를 들어, 큐브 맵이 월드 공간에 상대적 이라면(즉, 큐브 면이 월드 공간 축과 축 정렬됨), 룩업 벡터는 월드 공간 좌표를 가져야 합니다.

17.2 환경 맵

큐브 맵의 주요 응용 분야는 **환경 매핑**입니다. 기본 개념은 장면 내 특정 객체/0의 중심에 카메라를 배치하고 수직 및 수평 방향 모두 90° 시야각을 설정하는 것입니다. 그런 다음 카메라가 양의 x축, 음의 x축, 양의 y축, 음의 y축, 양의 z축, 음의 z축 방향을 향해 내려다보게 하여, 이 여섯 가지 시점 각각에서 장면(객체 0 제외)의 사진을 촬영합니다. 시야각이 90°이므로 이 여섯 장의 이미지는 객체/0의 시점에서 주변 환경 전체를 포착하게 됩니다(그림 17.2 참조). 이후 주변 환경의 이 여섯 장의 이미지를 큐브 맵에 저장하는데, 이 때문에 환경 맵이라는 이름이 붙었습니다. 즉, **환경 맵**은 큐브 면에

주변 환경 이미지를 저장하는 큐브 맵입니다.

이전 설명은 환경 매핑을 사용할 각 객체에 대해 환경 맵을 생성해야 함을 시사합니다.

그러나 환경 맵은 종종 먼 "배경" 정보만 캡처하는 데 사용됩니다. 따라서 많은 근접 오브젝트가 동일한 환경 맵을 공유할 수 있습니다. 이 장의 데모([그림 17.3](#))에서 모든 구체와 해골은 [그림 17.2](#)에 표시된 환경 맵을 공유합니다. 이 환경 맵은 장면의 로컬 기둥이나 바닥을 캡처하지 않으며, 먼 산과 하늘(즉, 장면 배경)만 캡처한다는 점에 유의하십시오. 배경 환경 맵은 어떤 의미에서는 불완전하지만, 실제로는 잘 작동합니다.

환경 맵 이미지를 생성하기 위해 카메라가 내려다본 축 방향이 월드 공간 축이었다면,

환경 맵은 월드 공간을 기준으로 생성된다고 합니다. 물론 다른 방향(예: 오브젝트의 로컬 공간)에서 환경을 캡처할 수도 있습니다. 그러나 록업 벡터 좌표는 큐브 맵이 기준을 두는 공간에 있어야 합니다.

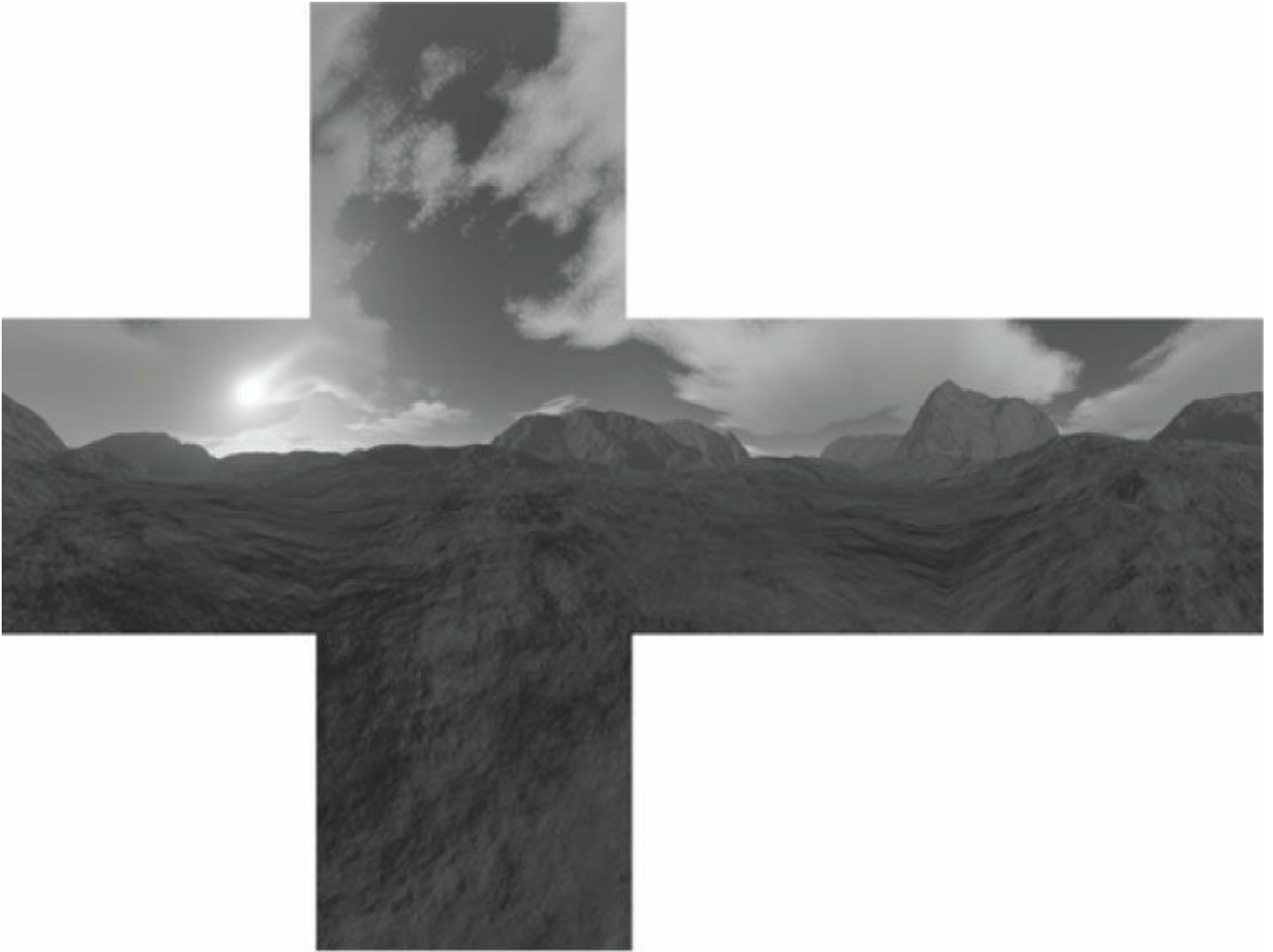


그림 17.2. 큐브 맵을 "펼친" 후의 환경 맵 예시. 이 여섯 면을 3D 상자로 다시 접었다고 상상하고, 그 상자의 중심에 서 있다고 상상해 보십시오. 어느 방향으로 보더라도 주변 환경이 보입니다.

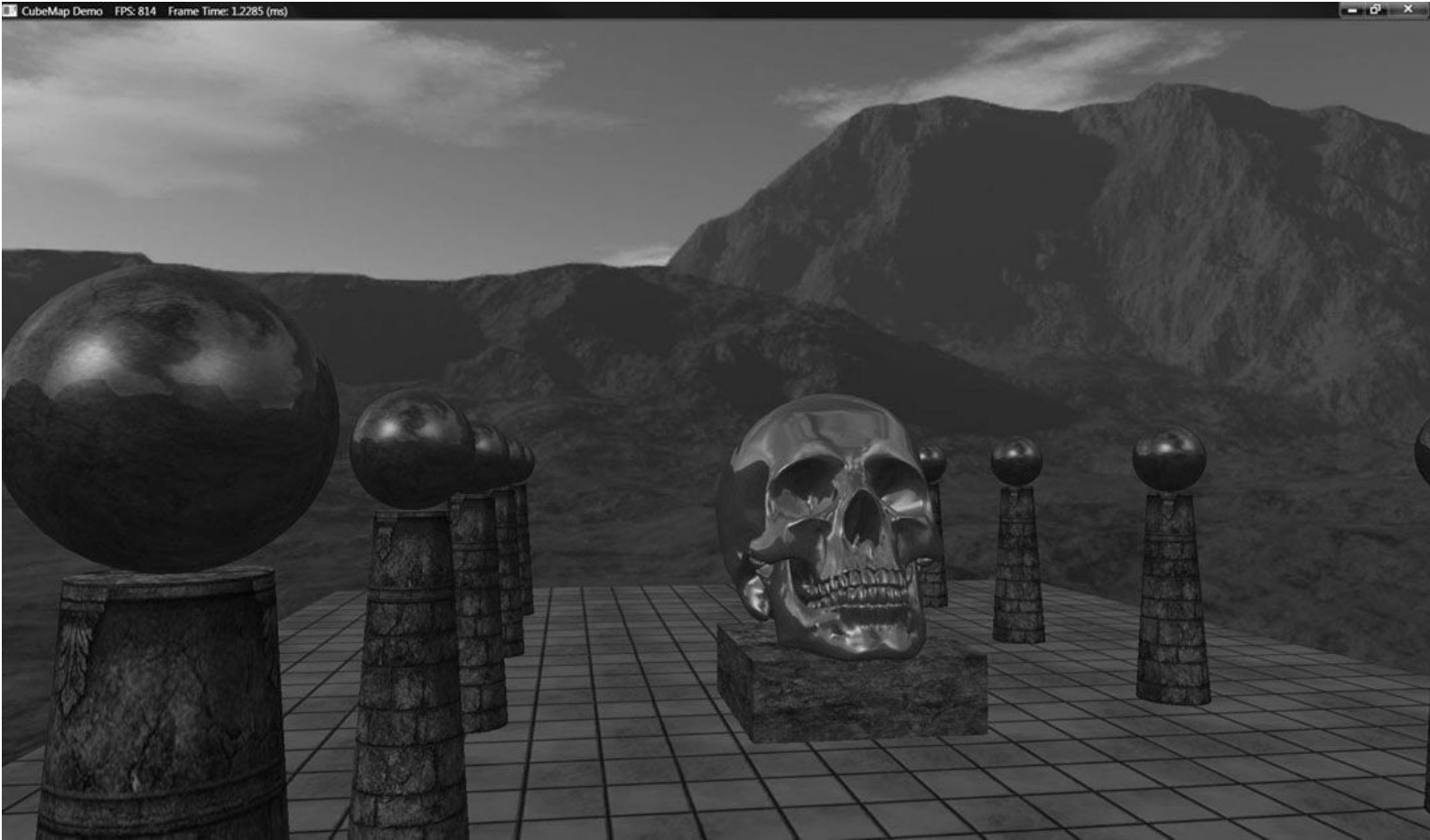


그림 17.3. "큐브 맵" 데모의 스크린샷.

환경 맵의 여섯 장 이미지는 Direct3D 프로그램에서 반드시 촬영된 것은 아니지만, 그렇게 할 수도 있습니다(17.5절에서 이를 수행하는 방법을 살펴보겠습니다). 큐브 맵은 텍스처 데이터만 저장하기 때문에, 그 내용은 종종 아티스트가 미리 생성합니다(지금까지 사용해 온 2D 텍스처와 마찬가지로). 따라서 큐브 맵 이미지를 계산하기 위해 실시간 렌더링을 사용할 필요가 없습니다. 즉, 3D 월드 에디터에서 장면을 생성한 후 에디터 내에서 큐브 맵의 여섯 면 이미지를 미리 렌더링할 수 있습니다. 야외 환경 맵의 경우 *Terragen* 프로그램(<http://www.planetside.co.uk>)이 흔히 사용되며(개인용 무료), 사실적인 야외 장면을 생성할 수 있습니다. 본서에서 제작한 환경 맵(예: 그림 17.2)은 *Terragen*으로 제작되었습니다.

*Terragen*을 사용해 보기로 선택했다면, **카메라 설정** 대화 상자로 이동하여 **줌** 계수를 1.0 Note 으로 설정하여 90° 시야각을 달성해야 합니다. 또한 수직 및 수평 시야각이 동일하도록 출력 이미지 크기를 동일하게 설정해야 합니다. 즉, 90°입니다.

수평 시야각이 동일하게 90°가 되도록 하십시오.

거기 있 있 멋진 *Terragen* 스크립트가 있습니다 웹사이트 웹

Note (https://developer.valvesoftware.com/wiki/Skybox_%282D%29_with_Terragen) 카메라

위치, 그리고 90° 시야각으로 주변 여섯 개의 이미지를 렌더링합니다.

프로그램을 사용하여 6개의 큐브 맵 이미지를 생성한 후에는, 이 6개를 모두 저장하는 큐브 맵 텍스처를 만들어야 합니다. 지금까지 사용해 온 DDS 텍스처 이미지 형식은 큐브 맵을 쉽게 지원하지며, DirectX 텍스처 도구를 사용하여 6개의 텍스처로 큐브 맵을 생성할 수 있습니다. DirectX 텍스처 도구(DirectX SDK 포함: C:\DXSDK\June10\Utilities\bin\x86)를 열고 먼저 파일 메뉴에서 **새 텍스처**를 선택합니다. 나타나는 대화 상자(그림 17.4)에서 텍스처 유형으로 **큐브맵 텍스처**를 선택하고, 여섯 개의 이미지 크기와 일치하는 크기를 입력한 후 표면 형식을 선택합니다 (DXT1과 같이 알파 채널이 없는 압축 형식을 사용하세요. 고해상도 큐브맵은 6개의 텍스처를 저장해야 하므로 메모리를 많이 소모할 수 있습니다).

이제 빈 큐브 맵이 생성되었습니다. **보기** 메뉴로 이동하여 **큐브 맵 면**을 선택한 후, 창에 표시할 축을 따라 면을 선택하세요 (그림 17.5). (모든 면은 초기 상태에서 비어 있습니다.) 시작할 면을 아무거나 선택한 후, 파일 메뉴로 이동하여 **'이 큐브맵 면에 열기'**를 선택하세요. 그러면 현재 선택된 큐브맵 면에 로드할 파일을 묻는 대화 상자가 열립니다. 해당 큐브맵 면에 해당하는 이미지를 선택하세요. 나머지 다섯 개의 큐브 맵 면에 대해서도 동일한 과정을 반복하여 각 면에 원하는 이미지를 삽입합니다. 완료 후 큐브 맵이 저장된 DDS 파일을 저장합니다.

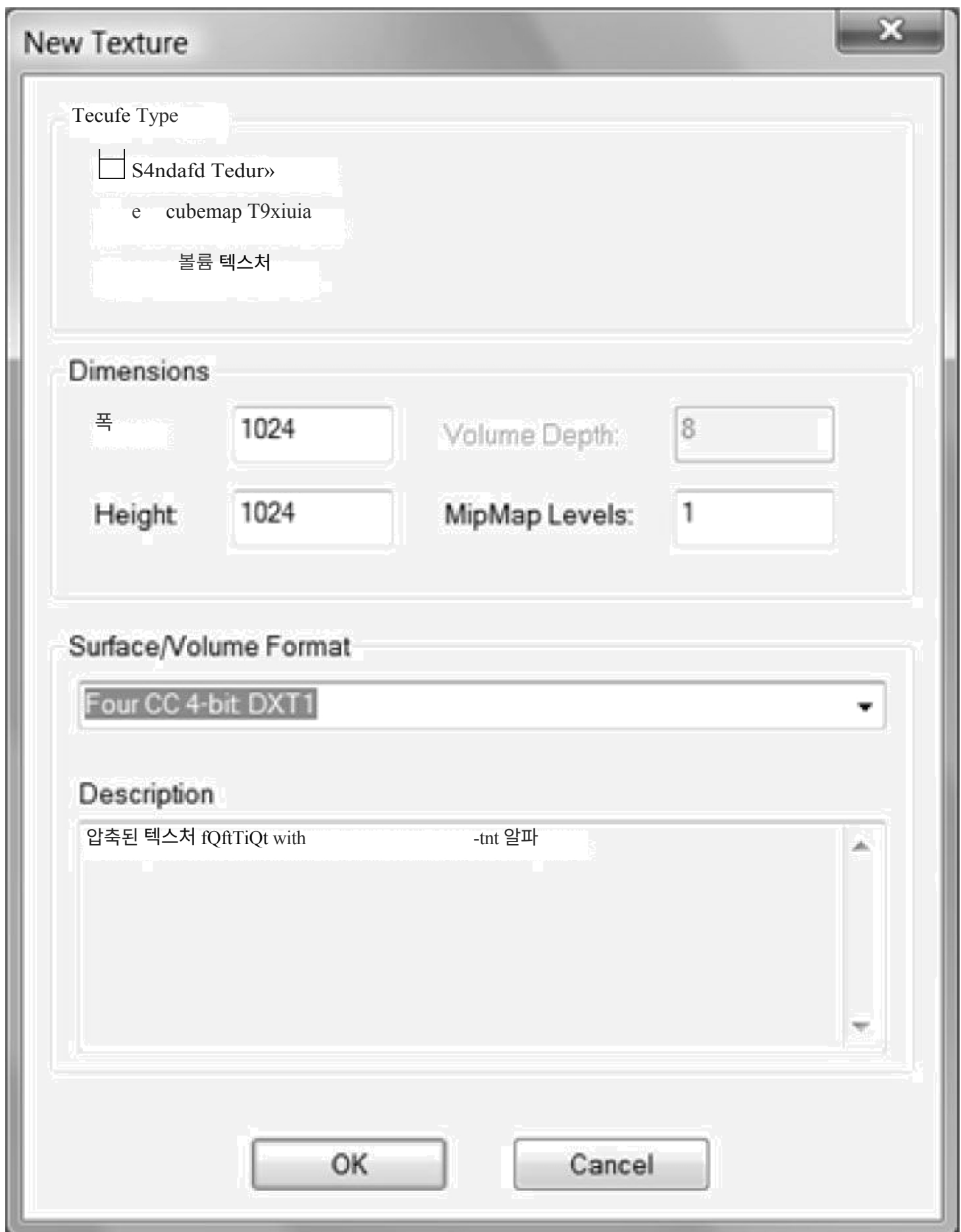


그림 17.4. DirectX 텍스처 도구로 새 큐브 텍스처 생성하기.

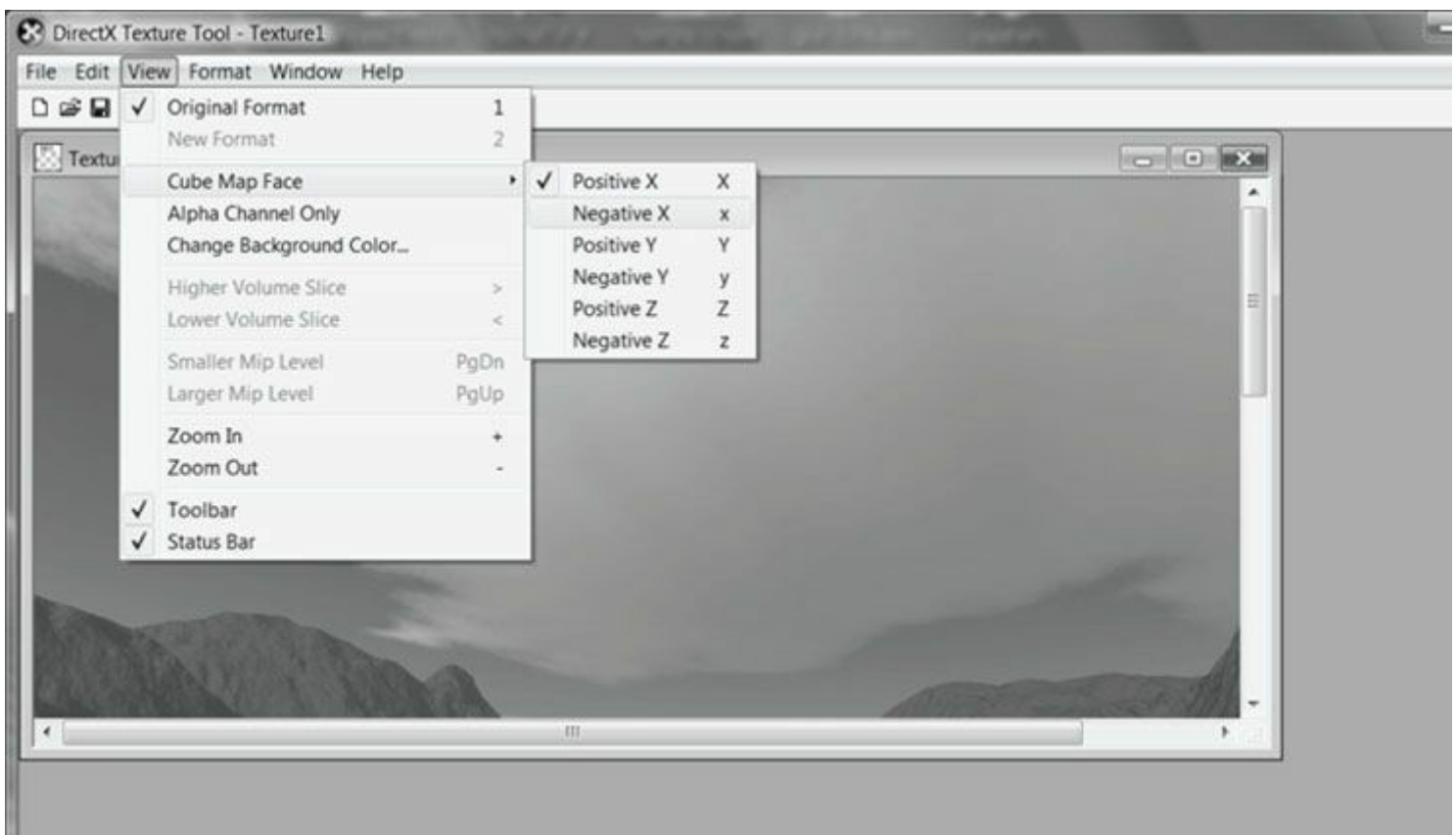


그림 17.5. DirectX 텍스처 도구에서 볼 큐브 맵의 면 선택.

NVIDIA 제공 Photoshop 플러그인 용 저장을 위한 .DDS 및 큐브맵 저장 Photoshop에서;
<http://developer.nvidia.com/nvidia-texture-tools-adobe-photoshop> 참조

17.2.1 Direct3D에서 큐브맵 로드 및 사용

편리하게도 `ID3DX11CreateShaderResourceViewFromFile` 함수는 큐브 맵을 저장한 DDS 파일을

`ID3D11Texture2D` 객체로 로드하고 이에 대한 셰이더 리소스 뷰를 생성할 수 있습니다:

```
ID3D11ShaderResourceView* mCubeMapSRV;
HR(D3DX11CreateShaderResourceViewFromFile(device,
cubemapFilename.c_str(), 0, 0, &mCubeMapSRV, 0));
```

`ID3D11Texture2D` 객체는 큐브 맵의 여섯 가지 텍스처를 텍스처 배열로 저장합니다.

큐브 텍스처에 대한 `ID3D11ShaderResourceView*`를 획득한 후,

`ID3DX11EffectShaderResourceVariable::SetResource` 메서드를 사용하여 효과 파일에서 `TextureCube` 변수에 설정할 수 있습니다:

```
// .fx 변수 TextureCube gCubeMap;

// .cpp 코드 ID3DX11EffectShaderResourceVariable* CubeMap;

CubeMap = mFX->GetVariableByName("gCubeMap")->AsShaderResource();

...

CubeMap->SetResource(cubemap);
```

17.3 하늘 텍스처링

환경 맵을 사용하여 하늘에 텍스처를 적용할 수 있습니다. 전체 장면을 둘러싸는 타원체를 생성합니다(타원체를 사용하면 더 평평하고 자연스러운 하늘 표면을 만들 수 있습니다). 지평선 너머 먼 산과 하늘의 환상을 만들기 위해,

그림 17.6에 표시된 방법으로 환경 맵을 사용하여 타원체에 텍스처를 적용합니다. 이렇게 하면 환경 맵이 타원체 표면에 투영됩니다.

우리는 하늘 타원체가 무한히 멀리 떨어져 있다고 가정합니다(즉, 월드 스페이스 중심으로 위치하지만 반경이 무한함). 따라서 카메라가 세계 공간에서 어떻게 움직이든, 우리는 하늘 타원체 표면으로부터 가까워지거나 멀어지는 것처럼 보이지 않습니다. 이 무한히 먼 하늘을 구현하기 위해, 우리는 단순히 세계 공간에서 카메라를 중심으로 하늘 타원체를 정렬하여 항상 카메라를 중심으로 위치하도록 합니다. 결과적으로 카메라가 움직여도 우리는 타원체 표면에 가까워지지 않습니다. 만약 이렇게 하지 않고 카메라가 하늘 표면에 가까워지도록 방치한다면, 하늘을 시뮬레이션하는 데 사용한 속임수가 드러나면서 전체 환상이 무너질 것이다.

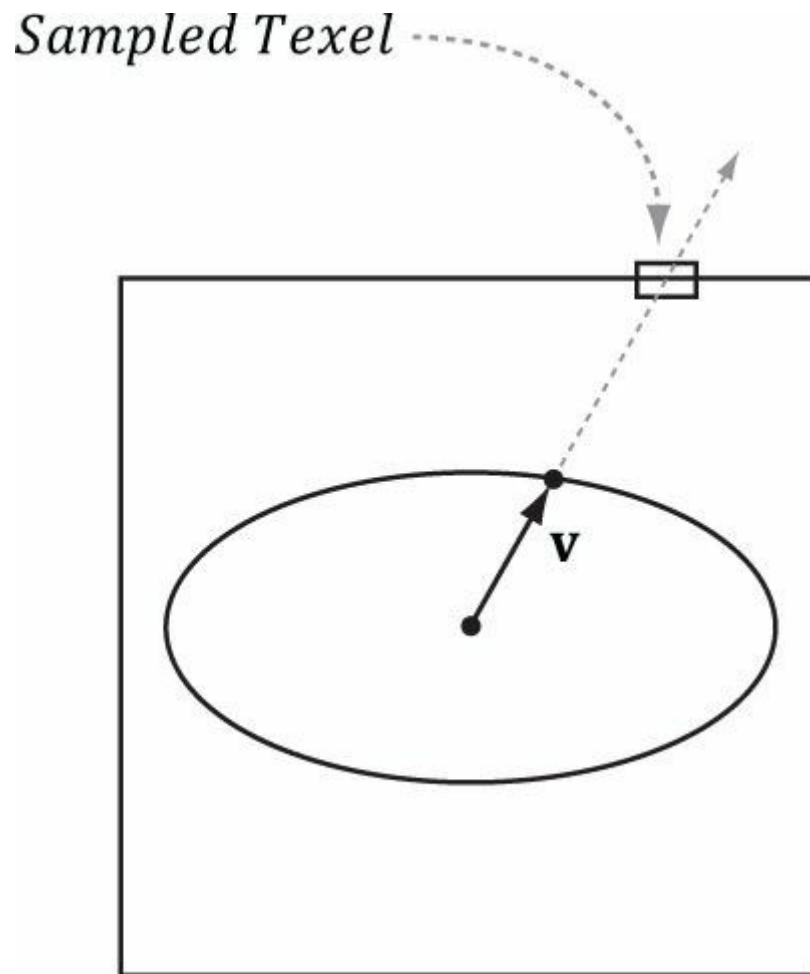


그림 17.6. 단순화를 위해 2D로 설명합니다. 3D에서는 정사각형이 정육면체가 되고 타원은 타원체가 됩니다. 하늘과 환경 맵이 동일한 원점을 중심으로 배치되었다고 가정합니다. 그러면 타원체 표면의 특정 지점에 텍스처를 적용하기 위해, 원점에서 표면 지점까지의 벡터를 큐브 맵 조회 벡터로 사용합니다. 이는 큐브 맵을 타원체에 투영합니다.

하늘 효과 파일은 다음과 같습니다:

```
//=====
// Sky.fx by Frank Luna (C) 2011 All Rights Reserved.
//
// 스카이 돔 셰이딩에 사용되는 이팩트.
//=====

cbuffer cbPerFrame
{
    float4x4 gWorldViewProj;
};

// 비슷자 값은 cbuffer에 추가할 수 없습니다. TextureCube gCubeMap;

SamplerState samTriLinearSam
{
    Filter = MIN_MAG_MIP_LINEAR; AddressU =
    Wrap;
    AddressV = Wrap;
```

```

};

struct VertexIn
{
    float3 PosL : POSITION;
};

struct VertexOut
{
    float4 PosH : SV_POSITION; float3 PosL :
    POSITION;
};

VertexOut VS(VertexIn vin)
{
    VertexOut vout;

    // z/w = 1이 되도록 z = w 설정 (즉, 스카이드롬이 항상 원근 평면에 위치). vout.PosH =
    mul(float4(vin.PosL, 1.0f), gWorldViewProj).xyww;

    // 로컬 버텍스 위치를 큐브맵 조회 벡터로 사용. vout.PosL = vin.PosL;

    return vout;
}

float4 PS(VertexOut pin) : SV_Target
{
    return gCubeMap.Sample(samTriLinearSam, pin.PosL);
}

RasterizerState NoCull
{
    CullMode = None;
};

DepthStencilState LessEqualDSS
{
    // 깊이 함수가 LESS_EQUAL인지, 단순히 LESS가 아닌지 확인합니다.
    // 그렇지 않으면 z = 1 (NDC)에서의 정규화된 깊이 값이
    // 깊이 버퍼가 1로 초기화된 경우 깊이 테스트에 실패합니다. DepthFunc =
    LESS_EQUAL;
};

technique11 SkyTech
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS()));
        SetGeometryShader(NULL); SetPixelShader(CompileShader(ps_5_0,
        PS()));

        SetRasterizerState(NoCull);
        SetDepthStencilState(LessEqualDSS, 0);
    }
}

```

과거에는 애플리케이션이 먼저 하늘을 그린 후 이를 렌더 타겟과 깊이/스텐실 버퍼를 지우는 대신 사용했습니다. 그러나 "ATI Radeon HD 2000 프로그래밍 가이드" (http://developer.amd.com/media/gpu_assets/ATI_Radeon_HD_2000_programming_guide.pdf)는 이제 다음과 같은 이유로 이를 권장하지 않습니다. 첫째, 하드웨어 내부 깊이 최적화가 제대로 작동하려면 깊이/스텐실 버퍼를 명시적으로 지워야 합니다. 렌더 타겟도 상황은 비슷합니다. 둘째, 일반적으로 하늘의 대부분은 건물이나 지형 같은 다른 지오메트리에 의해 가려집니다. 따라서 하늘을 먼저 그리면

17.4 반사 모델링

이전 섹션에서 설명한 바와 같이, 환경 맵은 하늘 텍스처링에 효과적입니다. 그러나 환경 맵의 또 다른 주요 용도는 임의의 객체에 대한 반사 모델링입니다(이 기법에서는 환경 맵의 이미지만 반사됩니다). [그림 17.7](#)은 환경 맵을 사용한 반사 구현 방식을 보여줍니다. 표면은 거울처럼 작동합니다: 시선은 **p**를 바라보며 **p**에 반사된 환경을 인식합니다.

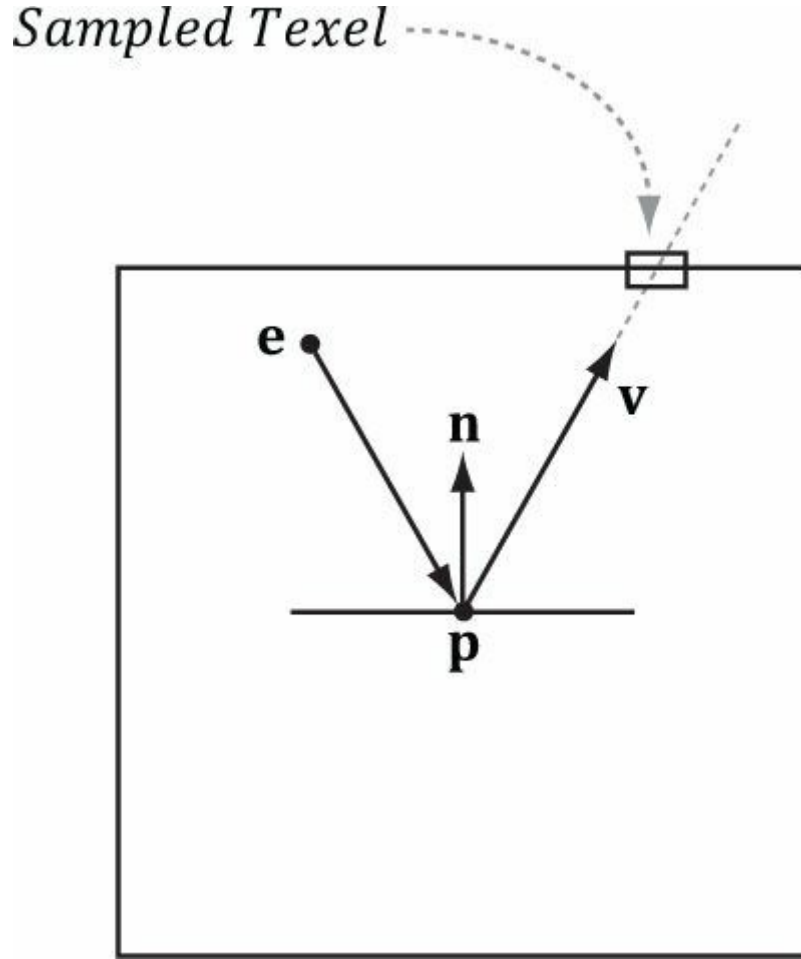


그림 17.7. 여기서 **e**는 시점(eye point)이고, **n**은 점 **p**에서의 표면 법선 벡터이다. 표면상의 점 **p**에 매핑되는 텍셀은 반사 벡터 **v**(즉, **v**를 조희 벡터로 사용)에 의해 결정된다. 이 **v**는 **e**에서 **p**로 향하는 벡터를 표면 주위로 반사한 것이다. 이러한 방식으로 시점은 반사된 환경을 보게 된다.

픽셀별로 반사 벡터를 계산한 후 이를 사용하여 환경 맵을 샘플링합니다:

```
litColor = texColor*(ambient + diffuse) + spec; if(gReflectionEnabled)
{
    float3 incident = -toEye;
    float3 reflectionVector = reflect(incident, pin.NormalW); float4 reflectionColor =
    gCubeMap.Sample(
    samAnisotropic, reflectionVector);

    litColor += gMaterial.Reflect*reflectionColor;
}
```

일반적으로 픽셀의 색상은 반사된 색상만으로 완전히 결정되지 않습니다(거울만이 100% 반사합니다). 따라서 우리는 조명 방정식을 수정하여 반사 항 $\mathbf{m}_R \otimes \mathbf{c}_{(R)}$ 을 포함시킵니다. 여기서 $\mathbf{c}_R$ 은 환경 맵에서 샘플링된 색상이며, $\mathbf{m}_R$ 은 표면이 $\mathbf{c}_R$ 의 어느 정도를 눈으로 반사하는지를 나타내는 애플리케이션 제어 재료 값입니다. 예를 들어, 표면이 단지

적색 빛을 반사한다면 $\mathbf{m}_R = (1, 0, 0)$ 으로 설정하여 환경 맵의 적색 빛만 눈에 도달하도록 할 수 있습니다.

재료 구조에는 지금까지 사용하지 않았던 반사 속성이 이미 존재합니다:

```
struct Material
{
    float4 Ambient; float4
    Diffuse;
    float4 Specular; // w = SpecPower float4
    Reflect;
};
```

조명 방정식에 반사 항을 추가할 때 발생하는 한 가지 문제는 과도한 채도입니다. 반사 항을 추가하면 픽셀에 더 많은 색상이 더해져 너무 밝아질 수 있습니다. 기본적으로 반사 항으로부터 추가 색상을 더한다면, 균형을 맞추기 위해 다른 항 중 하나에서 색상을 빼야 합니다. 이는 일반적으로 주변광 및 확산 재료 계수를 축소하여 표면에서 반사되는 주변광과 확산광의 양을 줄이는 방식으로 수행됩니다. 다른 접근법은 환경 맵에서 샘플링한 색상과 일반 조명 픽셀 색상 $\mathbf{s}$ 를 평균화하는 것입니다:

$$\mathbf{f} = t\mathbf{c}_R + (1 - t)\mathbf{s} \quad (0 \leq t \leq 1)$$

이러한 방식으로, 환경 맵에서 샘플링한 색상을 *가중치* t 로 추가할 때 균형을 유지하기 위해 일반 조명 픽셀 색상에서 동일한 양의 색상을 제거합니다. 따라서 여기서 매개변수 t 는 표면의 반사율을 제어합니다.

그림 17.8은 환경 매핑을 통한 반사가 평평한 표면에서는 제대로 작동하지 않음을 보여줍니다. 이는 반사 벡터는 위치를 포함하지 않으므로 전체 상황을 설명하지 못합니다. 우리는 실제로 반사 광선과 환경 맵과의 교차점을 필요로 합니다. 광선은 위치와 방향을 가지지만, 벡터는 방향만 가집니다. 그림에서 볼 수 있듯이, 두 반사 광선 $\mathbf{r}(t) = \mathbf{p} + t\mathbf{v}$ 와 $\mathbf{r}'(t) = \mathbf{p}' + t\mathbf{v}$ 는 큐브 맵의 서로 다른 텍셀과 교차하므로 다른 색상으로 표현되어야 합니다. 그러나 두 광선이 동일한 방향 벡터 $\mathbf{v}$ 를 가지며, 이 방향 벡터 $\mathbf{v}$ 가 큐브 맵 조회에만 사용되기 때문에, 시점이 각각 $\mathbf{e}$ 와 $\mathbf{e}'$ 에 있을 때 동일한 텍셀이 $\mathbf{p}$ 와 $\mathbf{p}'$ 에 매핑됩니다. 평평한 물체의 경우 환경 매핑의 이러한 결함이 매우 두드러집니다. 반면 곡면에서는 표면의 곡률로 인해 반사 벡터가 달라지기 때문에 환경 매핑의 이러한 단점이 거의 눈에 띄지 않습니다. 이 문제에 대한 근사적 해결책은 [Brennan02]를 참조하십시오.

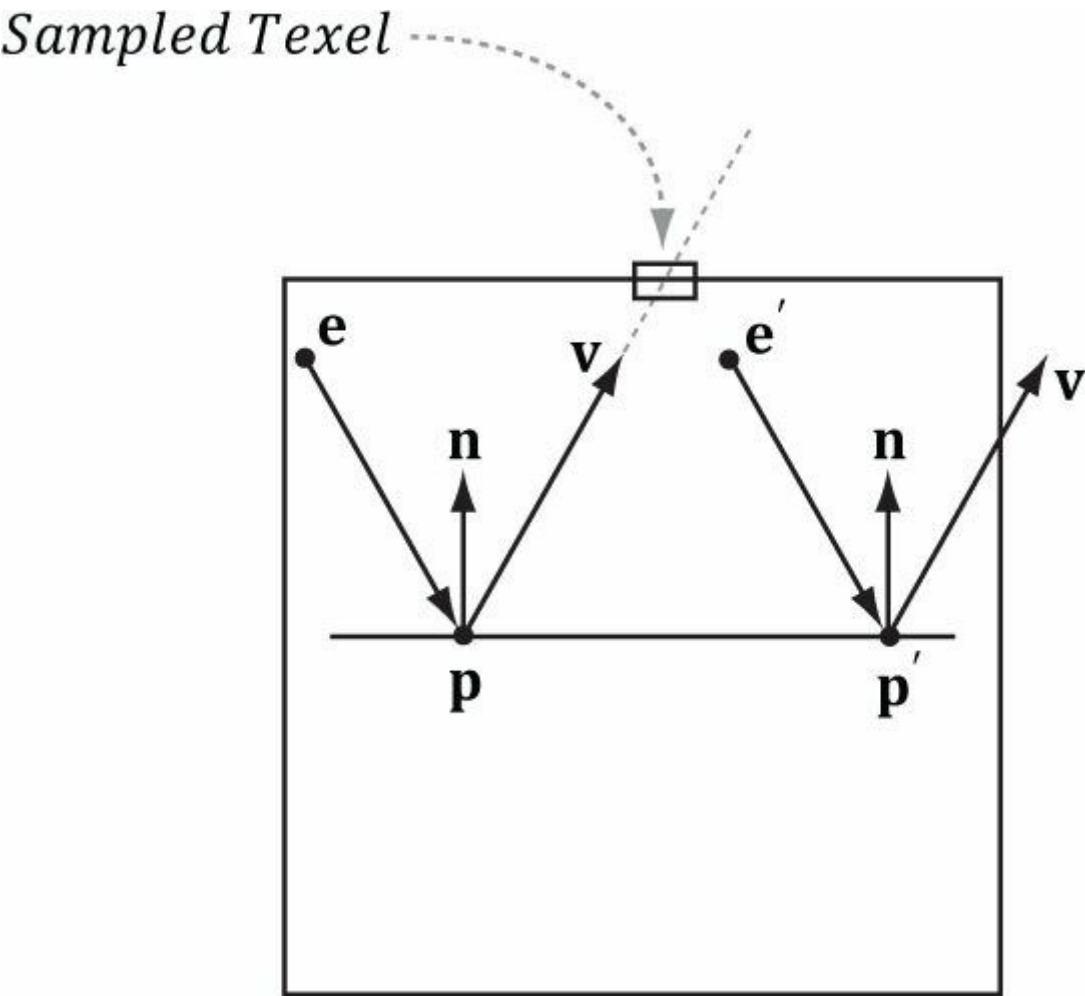


그림 17.8. 눈이 각각 위치 $\mathbf{e}$ 와 $\mathbf{e}'$ 에 있을 때 두 다른 점 $\mathbf{p}$ 와 $\mathbf{p}'$ 에 대응하는 반사 벡터.

17.5 동적 큐브 맵

지금까지 정적 큐브 맵에 대해 설명했습니다. 정적 큐브 맵은 미리 생성되어 고정된 이미지를 저장합니다. 이는 많은 상황에서 효과적이며 상대적으로 비용이 적게 듭니다. 그러나 장면 내에서 움직이는 애니메이션 객체를 원한다고 가정해 보겠습니다. 미리 생성된 큐브 맵으로는 이러한 애니메이션 객체를 포착할 수 없으므로, 애니메이션 객체를 반사할 수 없습니다. 이 한계를 극복하기 위해 런타임에 큐브 맵을 생성할 수 있습니다. 즉, 매 프레임마다 큐브 맵의 원점이 될 카메라 위치를 장면 내에 설정하고, 각 좌표축 방향을 따라 *큐브 맵의 여섯 면에 장면을 렌더링합니다*([그림 17.9](#) 참조). 큐브 맵이 매 프레임마다 재구성되므로 환경 내 애니메이션 오브젝트를 포착할 수 있으며, 반사 효과 역시 애니메이션 처리됩니다([그림 17.10](#) 참조).

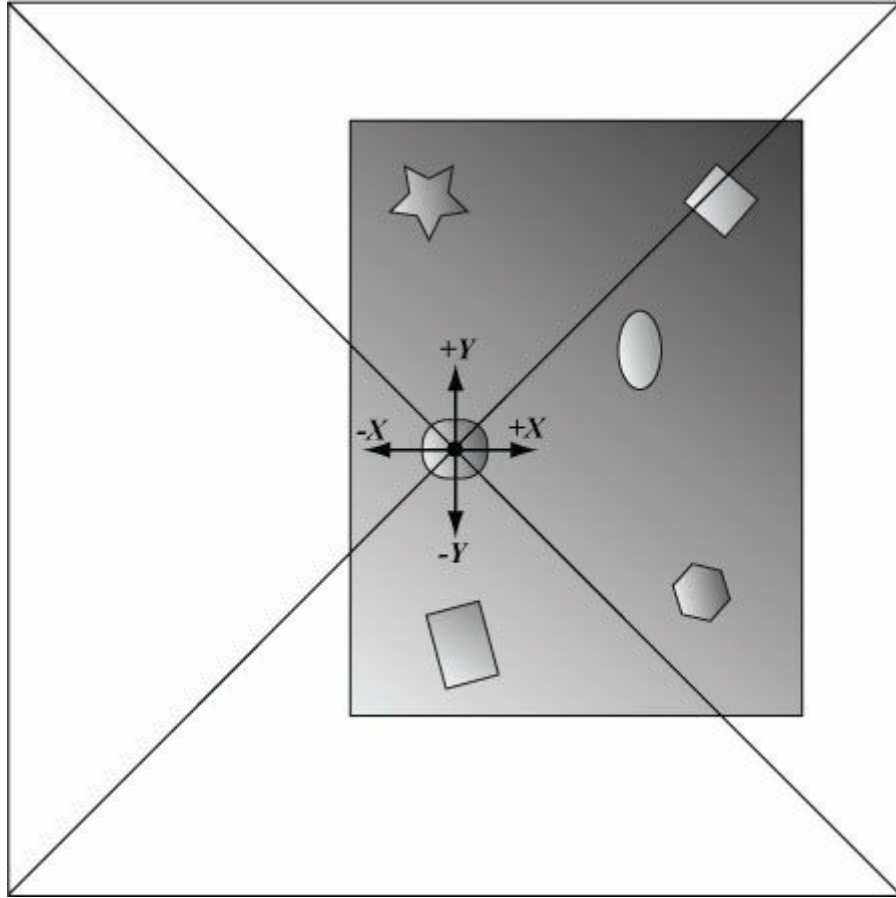


그림 17.9. 카메라를 장면 내 위치 θ 에 배치하여 동적 큐브맵을 생성하려는 대상에 대해 중심을 맞췄습니다. 시야각 90° 로 각 좌표축 방향을 따라 장면을 여섯 번 렌더링하여 주변 환경 전체의 이미지를 캡처합니다.



그림 17.10. 동적 반사를 보여주는 "Dynamic CubeMap" 데모의 스크린샷. 해골이 중앙 구체 주위를 공전하며, 구체에 비친 해골의 반사도 이에 따라 애니메이션됩니다. 또한 큐브 맵을 직접 그리기 때문에 기둥, 구체, 바닥과 같은 주변 객체들의 반사도 모델링할 수 있습니다.

큐브 맵을 동적으로 렌더링하는 것은 비용이 많이 듭니다. 장면을 6개의 렌더 타겟에 렌더링해야 하기 때문입니다! 따라서 장면에서 필요한 동적 큐브 맵의 수를 최소화하도록 노력하세요. 예를 들어, 동적 반사만 사용하는 것이 좋습니다.

Note: *장면에서 강조하거나 부각시키고 싶은 주요 오브젝트에는 동적 큐브 맵을 사용하세요. 반면 동적 반사가 눈에 띄지 않거나 없어도 무방한 덜 중요한 오브젝트에는 정적 큐브 맵을 적용하세요. 일반적으로 동적 큐브 맵에는 필 레이트를 절약하기 위해 256×256과 같은 저해상도 큐브 맵이 사용됩니다.*

17.5.1 큐브 맵 및 렌더 타겟 뷰 구축

큐브 맵 텍스처를 생성하려면 여섯 개의 요소(각 면마다 하나씩)로 구성된 텍스처 배열을 만들고, `D3D11_TEXTURE2D_DESC::MiscFlags`에 `D3D11_RESOURCE_MISC_TEXTURECUBE`를 지정하여 Direct3D가 텍스처 배열을 큐브 맵으로 해석하도록 합니다.

```
static const int CubeMapSize = 256;

//
// 큐브맵은 6개의 요소를 가진 특수한 텍스처 배열입니다. 우리는
// 이 텍스처 배열을 렌더 타겟으로 바인딩하여 큐브 면에 그리기 위해 사용하며,
// 또한 셰이더 리소스로도 사용하므로 픽셀 셰이더에서 활용할 수 있습니다.
//

D3D11_TEXTURE2D_DESC texDesc;
texDesc.Width      = CubeMapSize;
texDesc.Height     = CubeMapSize;
texDesc.MipLevels = 0;
texDesc.ArraySize = 6;
texDesc.SampleDesc.Count = 1;
texDesc.SampleDesc.Quality = 0;
텍스처 설명.포맷 = DXGI_FORMAT_R8G8B8A8_UNORM; 텍스처 설명.사용법 =
D3D11_USAGE_DEFAULT;
texDesc.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;
```

```
texDesc.CPUAccessFlags = 0;
텍스처 설명: 기타 플래그 = D3D11_RESOURCE_MISC_GENERATE_MIPS | D3D11_RESOURCE_MISC_TEXTURECUBE;
```

```
ID3D11Texture2D* cubeTex = 0;
HR(md3dDevice->CreateTexture2D(&texDesc, 0, &cubeTex));
```

또한 D3D11_RESOURCE_MISC_GENERATE_MIPS 플래그를 지정합니다. 이 플래그를 사용하면 큐브 맵에 대해 **ID3D11DeviceContext::GenerateMips** 메서드를 호출할 수 있으며, 이를 통해 하드웨어가 텍스처의 하위 mip 맵 레벨을 생성하도록 합니다. 이는 텍스처의 최상위 mip 맵 레벨에만 렌더링하기 때문에 필요하며, 따라서 하위 mip 맵 레벨은 우리가 직접 생성하거나 GenerateMips 메서드를 사용하지 않는 한 정의되지 않습니다.

다음으로, 큐브 맵 텍스처 배열의 각 요소에 대한 렌더 타겟 뷰를 생성해야 합니다. 이를 통해 각 요소에 렌더링할 수 있습니다. 큐브 맵 면. 이는 다음과 같이 수행됩니다:

```
//
// 각 큐브 맵 면에 대한 렌더 타겟 뷰 생성
// (즉, 텍스처 배열의 각 요소).
//

ID3D11RenderTargetView* mDynamicCubeMapRTV[6];

D3D11_RENDER_TARGET_VIEW_DESC rtvDesc;
rtvDesc.Format = texDesc.Format;
rtvDesc.ViewDimension = D3D11_RTV_DIMENSION_TEXTURE2DARRAY; rtvDesc.Texture2DArray.MipSlice = 0;

// 하나의 배열 요소만 뷰로 생성합니다.
rtvDesc.Texture2DArray.ArraySize = 1;

for(int i = 0; i < 6; ++i)
{
    // i번째 요소에 대한 렌더 타겟 뷰 생성.
    rtvDesc.Texture2DArray.FirstArraySlice = i; HR(md3dDevice->CreateRenderTargetView(
        cubeTex, &rtvDesc, &mDynamicCubeMapRTV[i]));
}
```

마지막으로, 큐브 맵 텍스처를 위한 셰이더 리소스 뷰를 생성합니다. 이를 통해 큐브 맵이 구축된 후 픽셀 셰이더에서 샘플링하여 반사 효과를 렌더링할 수 있습니다.

```
//

// 큐브 맵에 대한 셰이더 리소스 뷰 생성.
//

ID3D11ShaderResourceView* mDynamicCubeMapSRV;

D3D11_SHADER_RESOURCE_VIEW_DESC srvDesc;
srvDesc.Format = texDesc.Format;
srvDesc.ViewDimension = D3D11_SRV_DIMENSION_TEXTURECUBE;
srvDesc.TextureCube.MostDetailedMip = 0;
srvDesc.TextureCube.MipLevels = -1;

HR(md3dDevice->CreateShaderResourceView( cubeTex, &srvDesc,
    &mDynamicCubeMapSRV));

// 뷰가 참조를 저장합니다. ReleaseCOM(cubeTex);
```

17.5.2 뎀스 버퍼 및 뷰포트 구축

일반적으로 큐브 맵 면은 메인 백 버퍼와 다른 해상도를 가집니다. 따라서 큐브 맵 면에 렌더링하려면 큐브 맵 면의 해상도와 일치하는 크기의 깊이 버퍼가 필요합니다. 이는 다음 코드로 수행됩니다(지금쯤이면 익숙할 것입니다):

```
static const int CubeMapSize = 256; ID3D11DepthStencilView*
mDynamicCubeMapDSV;

D3D11_TEXTURE2D_DESC depthTexDesc; depthTexDesc.Width =
CubeMapSize; depthTexDesc.Height = CubeMapSize;
depthTexDesc.MipLevels = 1;
depthTexDesc.ArraySize = 1;
depthTexDesc.SampleDesc.Count = 1;
depthTexDesc.SampleDesc.Quality = 0; depthTexDesc.Format =
DXGI_FORMAT_D32_FLOAT; depthTexDesc.Usage = D3D11_USAGE_DEFAULT;
depthTexDesc.BindFlags = D3D11_BIND_DEPTH_STENCIL;
depthTexDesc.CPUAccessFlags = 0;
depthTexDesc.MiscFlags = 0;

ID3D11Texture2D* depthTex = 0;
HR(md3dDevice->CreateTexture2D(&depthTexDesc, 0, &depthTex));

// 전체 버퍼에 대한 깊이 스텔실 뷰 생성. D3D11_DEPTH_STENCIL_VIEW_DESC
dsvDesc;
dsvDesc.Format = depthTexDesc.Format; dsvDesc.Flags = 0;
dsvDesc.ViewDimension = D3D11_DSV_DIMENSION_TEXTURE2D; dsvDesc.Texture2D.MipSlice = 0;
HR(md3dDevice->CreateDepthStencilView(depthTex, &dsvDesc,
&mDynamicCubeMapDSV));

// 뷰는 참조를 저장합니다. ReleaseCOM(depthTex);
```

또한 큐브 맵 면은 메인 백 버퍼와 다른 해상도를 가지므로, 큐브 맵 면을 커버하는 새로운 뷰포트를 정의해야 합니다:

```
D3D11_VIEWPORT mCubeMapViewport;
mCubeMapViewport.TopLeftX = 0.0f; mCubeMapViewport.TopLeftY =
0.0f; mCubeMapViewport.Width = (float)CubeMapSize;
mCubeMapViewport.Height = (float)CubeMapSize;
mCubeMapViewport.MinDepth = 0.0f; mCubeMapViewport.MaxDepth =
1.0f;
```

17.5.3 큐브 맵 카메라 설정

큐브 맵을 생성하는 아이디어는 장면 내 어떤 객체 o 의 중심에 카메라를 배치하고 수직 및 수평 모두 90° 시야각을 설정하는 것임을 상기하십시오. 그런 다음 카메라가 양의 x 축, 음의 x 축, 양의 y 축, 음의 y 축, 양의 z 축, 음의 z 축 방향을 차례로 바라보게 하여, 이 여섯 시점 각각에서 장면(객체 o 제외)의 사진을 촬영합니다. 이를 용이하게 하기 위해, 주어진 위치 (x, y, z) 를 중심으로 각 면마다 하나씩 총 여섯 개의 카메라를 생성합니다:

```
Camera mCubeMapCamera[6];

void DynamicCubeMapApp::BuildCubeFaceCamera(float x, float y, float z)
{
    // 지정된 위치에 대한 큐브 맵 생성. XMFLOAT3 center(x, y, z);
    XMFLOAT3 worldUp(0.0f, 1.0f, 0.0f);

    // 각 좌표축을 따라 바라봄. XMFLOAT3 targets[6] =
    {
        XMFLOAT3(x+1.0f, y, z), // +X
```

```

        XMFLOAT3(x-1.0f, y, z), // -X XMFLOAT3(x, y+1.0f, z), //
        +Y XMFLOAT3(x, y-1.0f, z), // -Y XMFLOAT3(x, y,
        z+1.0f), // +Z XMFLOAT3(x, y, z-1.0f) // -Z
    };

    // +Y/-Y 방향을 제외한 모든 방향에 대해 세계 상향 벡터 (0,1,0)을 사용합니다.

    // 이 경우 +Y 또는 -Y 방향을 내려다보는 것이므로

    // 다른 "위" 벡터가 필요합니다.

    XMFLOAT3 ups[6] =
    {
        XMFLOAT3(0.0f,    1.0f,    0.0f), //  +X
        XMFLOAT3(0.0f,    1.0f,    0.0f), //  -X
        XMFLOAT3(0.0f,    0.0f,   -1.0f), //  +Y
        XMFLOAT3(0.0f,    0.0f,    +1.0f), //  -Y
        XMFLOAT3(0.0f,    1.0f,    0.0f), //  +Z
        XMFLOAT3(0.0f, 1.0f, 0.0f) // -Z
    };

    for(int i = 0; i < 6; ++i)
    {
        mCubeMapCamera[i].Look At(center, targets[i], ups[i]);
        mCubeMapCamera[i].SetLens(0.5f*XM_PI, 1.0f, 0.1f, 1000.0f);
        mCubeMapCamera[i].UpdateViewMatrix();
    }
}

```

17.5.4 큐브 맵에 그리기

마지막으로, 우리의 **DrawScene** 메서드는 먼저 큐브 맵을 생성하는 기준이 되는 중앙 구체(center sphere)를 제외한 장면을 큐브 맵에 렌더링한 후, 중앙 구체에 동적 큐브 맵을 적용하여 평소와 같이 장면을 렌더링합니다.

```

void DynamicCubeMapApp::DrawScene()
{
    ID3D11RenderTargetView* renderTargets[1];

    // 각 큐브 맵 면에 렌더링하여 큐브 맵 생성. md3dImmediateContext->RSSetViewports(1,
    &mCubeMapViewport); for(int i = 0; i < 6; ++i)
    {
        // 큐브 맵 면과 깊이 버퍼를 지움. md3dImmediateContext-
        >ClearRenderTargetView(
            mDynamicCubeMapRTV[i], reinterpret_cast<const
            float*>(&Colors::Silver));
        md3dImmediateContext->ClearDepthStencilView( mDynamicCubeMapDSV,
            D3D11_CLEAR_DEPTH|D3D11_CLEAR_STENCIL, 1.0f, 0);

        // 큐브 맵 면을 렌더 타깃으로 바인딩. renderTargets[0] =
        mDynamicCubeMapRTV[i]; md3dImmediateContext-
        >OMSetRenderTargets(
            1, renderTargets, mDynamicCubeMapDSV);

        // 중앙 구체를 제외한 장면을

        // 중앙 구체 제외하고 이 큐브 맵 면에 장면을 렌더링합니다.
        DrawScene(mCubeMapCamera[i], false);
    }

    // 이전 뷰포트와 렌더 타깃 복원. md3dImmediateContext->RSSetViewports(1,
    &mScreenViewport); renderTargets[0] = mRenderTargetView;
}

```

```

md3dImmediateContext->OMSetRenderTargets( 1, renderTargets,
    mDepthStencilView);

// 하드웨어가 큐브 맵의 낮은 mip 맵 레벨을 생성하도록 합니다. md3dImmediateContext-
>GenerateMips(mDynamicCubeMapSRV);

// 이제 중앙 구체를 포함하여 평소처럼 장면을 렌더링합니다. md3dImmediateContext-
>ClearRenderTargetView(
    mRenderTargetView,
    reinterpret_cast<const float*>(&Colors::Silver));
md3dImmediateContext->ClearDepthStencilView(
    mDepthStencilView, D3D11_CLEAR_DEPTH|D3D11_CLEAR_STENCIL,
    1.0f, 0);

DrawScene(mCam, true); HR(mSwapChain-

>Present(0, 0));
}

```

이 코드는 오버로드된 **DrawScene** 메서드를 활용하여 실제 객체를 그립니다. 이 오버로드된 메서드는 **카메라** 매개변수와 중심 구체 그리기 여부를 나타내는 **부울** 플래그를 받습니다.

```
void DrawScene(const Camera& camera, bool drawCenterSphere);
```

이전 코드에서 볼 수 있듯이, 우리는 각 큐브 맵 면에 해당 면의 카메라를 사용하여 6번 장면을 그렸으며, 이때 중앙 구체는 큐브 맵에 그리지 않았습니니다. 마지막으로, 이번에는 중앙 구체를 그리고 일반적인 "플레이어" 카메라를 사용하여 장면을 다시 그렸습니다. 중앙 구체가 그려질 때는 방금 생성한 큐브 맵을 사용하므로 애니메이션이 정확하게 반영됩니다.

17.6 지오메트리 셰이더를 활용한 동적 큐브 맵

이전 섹션에서는 큐브 맵을 생성하기 위해 장면을 6번 다시 그렸습니다—큐브 맵의 각 면마다 한 번씩입니다. 드로우 콜은 무료가 아니므로 이를 최소화하기 위해 노력해야 합니다. Direct3D 10 샘플 플인 "CubeMapGS"가 있습니다. 이 샘플은 지오메트리 셰이더를 사용하여 장면을 단 한 번만 렌더링함으로써 큐브 맵을 생성합니다. (Direct3D 11은 10의 상위 집합이므로, 이 방식은 Direct3D 11에서도 동일하게 작동합니다.) 이 섹션에서는 이 샘플의 주요 작동 원리를 살펴보겠습니다.

먼저, 개별 면 텍스처가 아닌 *전체* 텍스처 배열에 대한 렌더 타겟 뷰를 생성합니다:

```

// 6면 렌더 타겟 뷰 생성 D3D10_RENDER_TARGET_VIEW_DESC DescRT;

DescRT.Format = dstex.Format;
DescRT.ViewDimension = D3D10_RTV_DIMENSION_TEXTURE2DARRAY; DescRT.Texture2DArray.FirstArraySlice = 0;
DescRT.Texture2DArray.ArraySize = 6;
DescRT.Texture2DArray.MipSlice = 0; V_RETURN(pd3dDevice-
>CreateRenderTargetView(
    g_pEnvMap, &DescRT, &g_pEnvMapRTV));

```

또한 이 기법은 깊이 버퍼의 큐브 맵(각 면마다 하나씩)이 필요합니다. 깊이 스텐실 뷰를 통해 깊이 버퍼의 *전체* 텍스처 배열에 접근하는 방법은 다음과 같습니다:

```

// 전체 큐브에 대한 깊이 스텐실 뷰 생성
D3D10_DEPTH_STENCIL_VIEW_DESC DescDS;
DescDS.Format = DXGI_FORMAT_D32_FLOAT;
DescDS.ViewDimension = D3D10_DSV_DIMENSION_TEXTURE2DARRAY; DescDS.Texture2DArray.FirstArraySlice =
0;
DescDS.Texture2DArray.ArraySize = 6;
DescDS.Texture2DArray.MipSlice = 0; V_RETURN(pd3dDevice-
>CreateDepthStencilView(
    g_pEnvMapDepth, &DescDS, &g_pEnvMapDSV));

```

그런 다음 이 렌더 타겟과 깊이 스텐실 뷰를 파이프라인의 OM 단계에 바인딩합니다:

```
ID3D10RenderTargetView* aRTViews[ 1 ] = { g_pEnvMapRTV }; pd3dDevice->OMSetRenderTargets(sizeof(aRTViews)/sizeof(aRTViews[0]),
aRTViews, g_pEnvMapDSV);
```

즉, 렌더 타겟 배열에 대한 뷰 하나와 깊이 스텐실 버퍼 배열에 대한 뷰 하나를 OM 단계에 바인딩했으며, 각 배열 슬라이스에 동시에 렌더링할 예정입니다.

이제 장면은 한 번 렌더링되며, 상수 버퍼에는 각 큐브맵 면의 대응 방향을 바라보는 6개의 뷰 매트릭스 배열이 존재합니다.

면)을 바라보는 뷰 매트릭스)가 상수 버퍼에 저장됩니다. 지오메트리 셰이더는 입력된 삼각형을 여섯 번 복제하고, 각 삼각형을 여섯 개의 렌더 타겟 어레이 슬라이스 중 하나에 할당합니다. 삼각형을 렌더 타겟 어레이 슬라이스에 할당하는 작업은 시스템 값 `SV_RenderTargetArrayIndex`를 설정하여 수행됩니다. 이 시스템 값은 정수 인덱스 값으로, 지오메트리 셰이더의 출력으로만 설정되어 해당 프리미티브가 렌더링될 렌더 타겟 어레이 슬라이스의 인덱스를 지정합니다. 이 시스템 값은 렌더 타겟 뷰가 실제로 배열 리소스에 대한 뷰인 경우에만 사용할 수 있습니다.

```
struct PS_CUBEMAP_IN
{
    float4 Pos : SV_POSITION; // 투영 좌표 float2 Tex : TEXCOORD0;
                                // 텍스처 좌표 uint
    RTIndex : SV_RenderTargetArrayIndex;
};

[maxvertexcount(18)]
void GS_CubeMap(triangle GS_CUBEMAP_IN input[3], inout
TriangleStream<PS_CUBEMAP_IN> CubeMapStream)
{
    // 각 삼각형에 대해 for(int f = 0; f <
    6; ++f)
    {
        // 화면 좌표 계산 PS_CUBEMAP_IN output;

        // i번째 삼각형을 i번째 렌더 타겟에 할당 output.RTIndex = f;

        // 삼각형 내 각 정점에 대해 for(int v = 0; v < 3;
        v++)
        {
            // i번째 큐브 면의 뷰 공간으로 변환. output.Pos = mul(input[v].Pos,
            g_mViewCM[f]);

            // 동형 클립 공간으로 변환. output.Pos = mul(output.Pos,
            mProj);

            output.Tex = input[v].Tex;
            CubeMapStream.Append(output);
        }
        CubeMapStream.RestartStrip();
    }
}
```

따라서 우리는 장면을 여섯 번이 아닌 단 한 번만 렌더링함으로써 각 큐브맵 면에 장면을 렌더링했음을 알 수 있습니다.



이 샘플의 주요 아이디어를 요약했지만, 세부 사항을 보완하려면 전체 소스 코드가 포함된 "CubeMapGS" Direct3D 10 샘플을 참조하십시오.

이 전략은 흥미롭고 동시 렌더 타겟 및 `SV_RenderTargetArrayIndex` 시스템 값을 보여줍니다. 그러나 반드시 최선의 방법은 아닙니다. 이 방법을 매력적이지 않게 만드는 두 가지 문제가 있습니다.

1. 지오메트리 셰이더를 사용하여 대량의 데이터를 출력합니다. 제11장에서 살펴본 바와 같이 지오메트리 셰이더는 대량의 데이터 출력 시 비효율적으로 동작합니다. 따라서 이 목적으로 지오메트리 셰이더를 사용하면 성능 저하를 초래할 수 있습니다.
2. 일반적인 장면에서 삼각형은 하나의 큐브 맵 면 이상과 겹치지 않습니다(다시 그림 17.9 참조). 따라서 삼각형을 복제하여 6개 면 중 5개 면에 의해 잘릴 때 각 큐브 면에 렌더링하는 행위는 비효율적입니다. 물론,

5.4절의 데모는 단순화를 위해 전체 장면을 각 큐브 맵 면에 렌더링합니다. 그러나 실제 애플리케이션(데모가 아닌)에서는 프러스텀 컬링([15장](#))을 사용하여 특정 큐브 맵 면에 보이는 오브젝트만 렌더링합니다. 오브젝트 수준에서의 프러스텀 컬링은 지오메트리 셰이더 구현으로 수행할 수 없습니다.

반면 이 전략이 효과적인 상황은 장면을 둘러싼 메쉬를 렌더링할 때입니다. 예를 들어, 구름이 움직이고 시간대에 따라 하늘 색이 변하는 동적 하늘 시스템을 가졌해 보겠습니다. 하늘이 변화하기 때문에 미리 구워진 큐브 맵 텍스처로 하늘을 표현할 수 없으므로 동적 큐브 맵을 사용해야 합니다. 하늘 메쉬가 전체 장면을 둘러싸고 있기 때문에 *여섯 개의* 큐브 맵 면 *모두에 의해 가시됩니다*. 따라서 앞서 언급한 두 번째 항목은 적용되지 않으며, 지오메트리 셰이더 사용이 성능에 지나치게 영향을 주지 않는다면 드로우 쿨을 여섯 번에서 한 번으로 줄여 지오메트리 셰이더 방식이 유리할 수 있습니다.

17.7 요약

1. 큐브 맵은 육면체의 면으로 시각화되는 여섯 개의 텍스처로 구성됩니다. Direct3D 11에서는 큐브 맵을 ID3D11Texture2D 인터페이스로, 추가 플래그 **D3D11_RESOURCE_MISC_TEXTURECUBE**를 지닌 텍스처 배열로 표현할 수 있습니다. HLSL에서는 큐브 맵이 TextureCube 유형으로 표현됩니다. 큐브 맵 내 텍셀을 식별하기 위해 3D 텍스처 좌표를 사용합니다. 이는 큐브 맵 중심에서 시작하는 3차원 *조화* 벡터 **v**를 정의합니다. **v**가 교차하는 큐브 맵의 텍셀은 **v**의 3차원 좌표에 대응하는 텍셀입니다.
2. 환경 맵은 한 지점 주변 환경을 여섯 개의 이미지로 캡처합니다. 이 이미지들은 큐브 맵에 저장될 수 있습니다. 환경 맵을 사용하면 하늘 텍스처링이나 반사 효과 구현이 용이합니다.
3. 큐브 맵은 DirectX 텍스처 툴을 사용하여 여섯 개의 개별 이미지로 생성할 수 있습니다. 이후 큐브 맵은 DDS 이미지 형식으로 파일에 저장됩니다. 큐브 맵은 여섯 개의 2D 텍스처를 저장하므로 많은 메모리를 소모할 수 있으므로 압축된 DDS 형식을 사용해야 합니다. **D3DX11CreateShaderResourceViewFromFile** 함수는 큐브 맵을 저장한 DDS 파일을 로드하고 이에 대한 셰이더 리소스 뷰를 생성할 수 있습니다.
4. 사전 구운 큐브 맵은 움직이는 오브젝트나 큐브 맵 생성 시 존재하지 않았던 장면 내 오브젝트를 포착하지 못합니다. 이러한 한계를 극복하기 위해 런타임에 큐브 맵을 생성할 수 있습니다. 즉, 매 프레임마다 큐브 맵의 원점이 될 카메라 위치를 설정하고, 각 좌표축 방향에 따라 *6개의 큐브 맵 면에 장면을 렌더링합니다*. 큐브 맵이 매 프레임마다 재구성되므로 애니메이션 오브젝트와 환경 내 모든 오브젝트를 포착합니다. 동적 큐브 맵은 비용이 많이 들므로 주요 오브젝트에만 사용을 최소화해야 합니다.
5. 렌더 타겟 뷰를 텍스처 배열에 바인딩하여 OM 단계로 보낼 수 있습니다. 또한 텍스처 배열의 각 슬라이스에 동시에 렌더링할 수 있습니다. 삼각형을 렌더 타겟 배열 슬라이스에 할당하려면 시스템 값 **SV_RenderTargetArrayIndex**를 설정합니다. 텍스처 배열에 대한 렌더 타겟 뷰와 **SV_RenderTargetArrayIndex** 시스템 값을 함께 사용하면 장면을 여섯 번 렌더링하는 대신 한 번만 렌더링하여 큐브 맵을 동적으로 생성할 수 있습니다. 그러나 이 전략이 프러스텀 컬링을 적용하여 장면을 여섯 번 렌더링하는 것보다 항상 유리하지는 않을 수 있습니다.

17.8 연습 문제

1. "큐브 맵" 데모에서 다양한 **Material::Reflect** 값을 실험해 보십시오. 또한 원통과 상자를 반사성 있게 만들어 보십시오.
2. 환경을 포착한 6개의 이미지를 찾으세요(온라인에서 큐브 맵 이미지를 찾거나 *Terragen* 같은 프로그램을 사용해 생성). 그런 다음 텍스처 툴을 사용해 §17.2에 설명된 과정을 거쳐 자신만의 큐브 맵을 만드세요. "Cube Map" 데모에서 큐브 맵을 테스트해 보세요.
3. *유전체*는 빛을 굴절시키는 투명 재료입니다([그림 17.11](#) 참조). 광선이 유전체에 부딪히면 *스넬의 굴절 법칙*에 따라 일부 빛은 반사되고 일부는 굴절됩니다. 굴절률 n_1 과 n_2 는 빛이 얼마나 휘어지는지를 결정합니다:

Cube Map

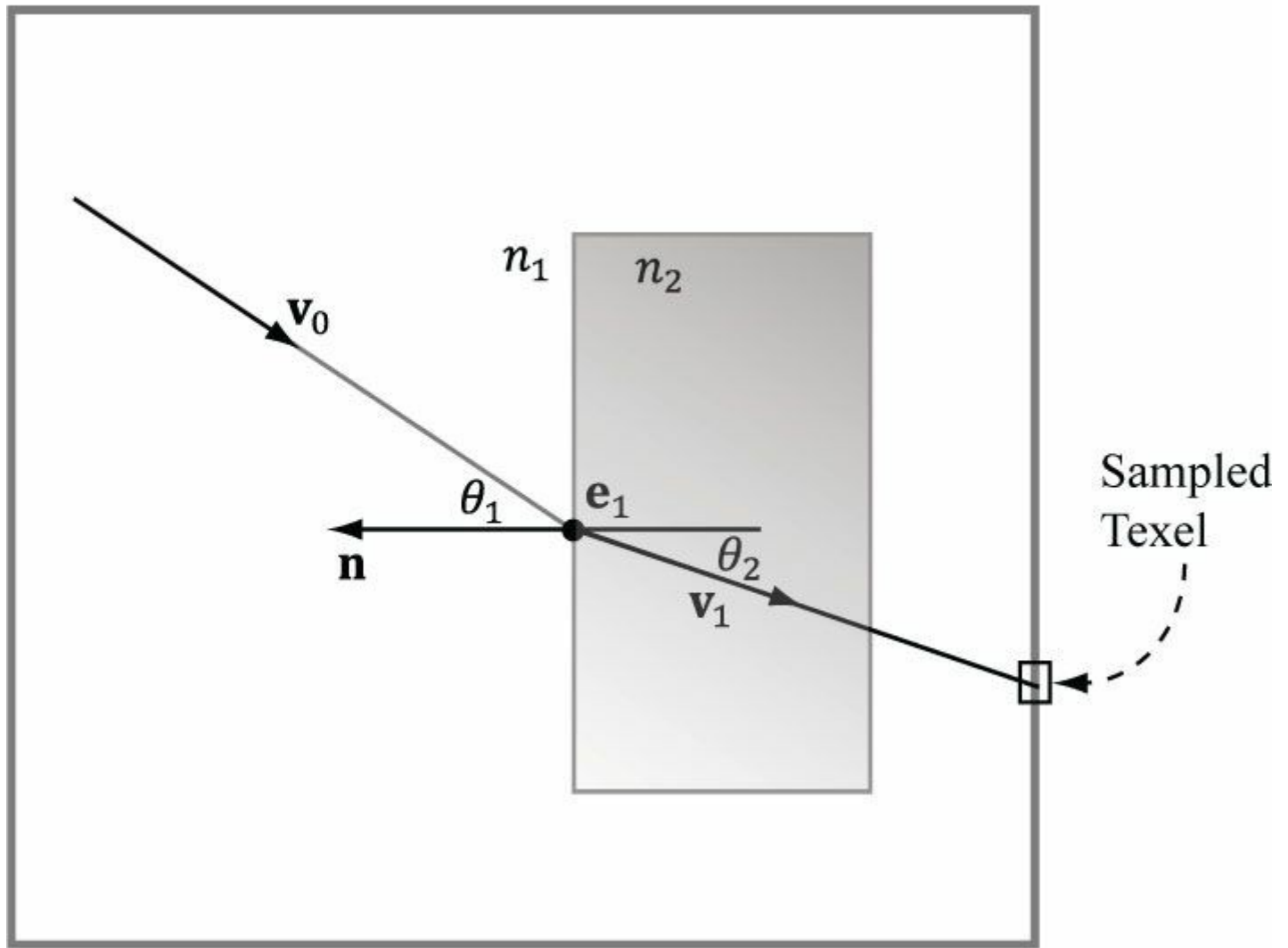


그림 17.11. 입사 벡터 v_0 굴절률 n_1 을 가진 매질을 통과한다. 광선은 굴절률 n_2 을 가진 투명 물질에 부딪혀 벡터 v_1 으로 굴절된다. 굴절된 벡터 v_1 을 큐브 맵 조회에 사용합니다. 이는 알파 블렌딩 투명도와 거의 유사하지만, 알파 블렌딩 투명도는 입사 벡터를 굴절시키지 않습니다.

$n_1 = n_2$ 이면 $\theta_1 = \theta_2$ (굴절 없음).

$n_2 > n_1$ 이면, $\theta_2 < \theta_1$ (광선이 법선 방향으로 굴절).

$n_1 > n_2$ 이면, $\theta_2 > \theta_1$ (광선이 법선 방향에서 멀어지며 굴절).

따라서 그림 17.11에서 $n_2 > n_1$ 입니다. 이는 광선이 블록에 진입할 때 법선 방향으로 굴절되기 때문입니다. 물리적으로는 광선이 블록을 떠날 때 다시 굴절되지만, 실시간 그래픽스에서는 일반적으로 첫 번째 굴절만 모델링됩니다. HLSL은 굴절 벡터를 계산하기 위한 내장 함수 **refract**를 제공합니다:

```
float3 refract(float3 incident, float3 normal, float eta);
```

입사 벡터는 들어오는 광선 벡터(그림 17.11의 v_0)이며, 법선 벡터는 외부 표면 법선(그림 17.11의 n)입니다. 세 번째 매개변수는 굴절률 비율 n_1/n_2 입니다. 진공의 굴절률은 1.0입니다. 다른 물질의 굴절률 예시: 물—1.33, 유리—1.51. 이 연습에서는 "Cube Map" 데모를 수정하여 반사 대신 굴절을 구현하세요(그림 17.12 참조). **Material::Reflect** 값을 조정해야 할 수 있습니다. $\eta = 1.0$, $\eta = 0.95$, $\eta = 0.9$ 값을 차례로 적용해 보십시오.



그림 17.12. 반사 대신 굴절을 적용한 "큐브 데모".

노멀 매핑과 변위 매핑

제18장 노멀 매핑

8장에서 텍스처 매핑을 소개했는데, 이를 통해 이미지의 미세한 디테일을 삼각형에 매핑할 수 있었습니다. 그러나 노멀 벡터는 여전히 거친 버텍스 수준에서 정의되고 삼각형 전체에 보간됩니다. 이 장의 일부에서는 더 높은 해상도로 표면 노멀을 지정하는 널리 쓰이는 방법을 살펴봅니다. 표면 노멀을 더 높은 해상도로 지정하면 조명 디테일이 증가하지만, 메시 지오메트리 디테일은 변하지 않습니다. 변위 매핑을 테셀레이션과 결합하면 메시의 디테일을 높일 수 있습니다.

목표:

- 1. 법선 매핑이 필요한 이유를 이해한다.
- 2. 법선 맵이 어떻게 저장되는지 알아보기.
- 3. 법선 맵을 생성하는 방법을 학습한다.
- 4. 노멀 맵에서 노멀 벡터가 3D 삼각형의 객체 공간 좌표계에 대해 상대적으로 어떻게 지정되는지, 그리고 이를 어떻게 연관시키는지 학습합니다.
- 5. 버텍스 셰이더와 픽셀 셰이더에서 노멀 매핑을 구현하는 방법을 알아봅니다.
- 6. 변위 맵과 테셀레이션을 결합하여 메시 디테일을 향상시키는 방법을 알아보기.

18.1 동기 부여

그림 18.1을 살펴보자. 원뿔 모양 기둥의 스페큘러 하이라이트가 자연스럽게 않게 보인다—벽돌 텍스처의 울퉁불퉁함과 비교해 지나치게 매끄럽게 보인다. 이는 기본 메시 지오메트리가 매끄럽기 때문이며, 우리는 단순히 울퉁불퉁한 벽돌 이미지를 매끄러운 원통형 표면에 적용했을 뿐이다. 그러나 조명 계산은 텍스처 이미지가 아닌 메시 지오메트리(특히 보간된 정점 법선)를 기반으로 수행됩니다. 따라서 조명은 텍스처와 완전히 일치하지 않습니다.

이상적으로는 메시 지오메트리를 충분히 세분화하여 벽돌의 실제 요철과 틈새를 모델링할 수 있도록 해야 합니다. 기본 기하 구조를 유지해야 합니다. 그러면 조명과 텍스처링을 일관되게 만들 수 있습니다. 하드웨어 테셀레이션이 이 부분에 도움이 될 수 있지만, 테셀레이터가 생성한 정점의 법선을 지정하는 방법이 여전히 필요합니다(보간된 법선을 사용해도 법선 해상도가 증가하지 않음).

또 다른 가능한 해결책은 조명 디테일을 텍스처에 직접 베이킹하는 것입니다. 그러나 조명이 텍셀 색상은 고정된 채로 남아 있기 때문입니다.



그림 18.1. 부드러운 스페큘러 하이라이트.



그림 18.2. 울퉁불퉁한 스페큘러 하이라이트.

따라서 우리의 목표는 텍스처 맵에 나타나는 미세한 디테일이 조명에서도 드러나도록 동적 조명을 구현하는 방법을 찾는 것이다. 텍스처가 미세한 디테일을 제공해 주므로, 이 문제에 대한 해결책으로 텍스처 매핑을 고려하는 것은 자연스러운 일이다. [그림 18.2](#)는 [그림 18.1](#)과 동일한 장면에 노멀 매핑을 적용한 모습이다. 이제 동적 조명이 벽돌 텍스처와 훨씬 더 일관성을 갖는 것을 확인할 수 있다.

18.2 노멀 맵

노멀 맵은 텍스처이지만, 각 텍셀에 RGB 데이터를 저장하는 대신 압축된 x좌표, y좌표, z좌표를 각각 빨간색, 녹색, 파란색 성분에 저장합니다. 이 좌표들은 법선 벡터를 정의하므로, 노멀 맵은 각 픽셀에 법선 벡터를 저장합니다. [그림 18.3](#)은 노멀 맵을 시각화하는 방법을 보여주는 예시입니다.

설명 편의를 위해 각 색상 구성요소에 1바이트를 할당하는 24비트 이미지 형식을 가정합니다. 따라서 각 색상 성분은 0~255 범위를 가집니다. (알파 성분이 사용되지 않거나 높이맵이나 스페큘러 맵과 같은 다른 스칼라 값을 저장하는 32비트 형식을 사용할 수도 있습니다. 또한 압축이 필요 없는 부동 소수점 형식을 사용할 수도 있지만, 이는 더 많은 메모리를 필요로 합니다.)

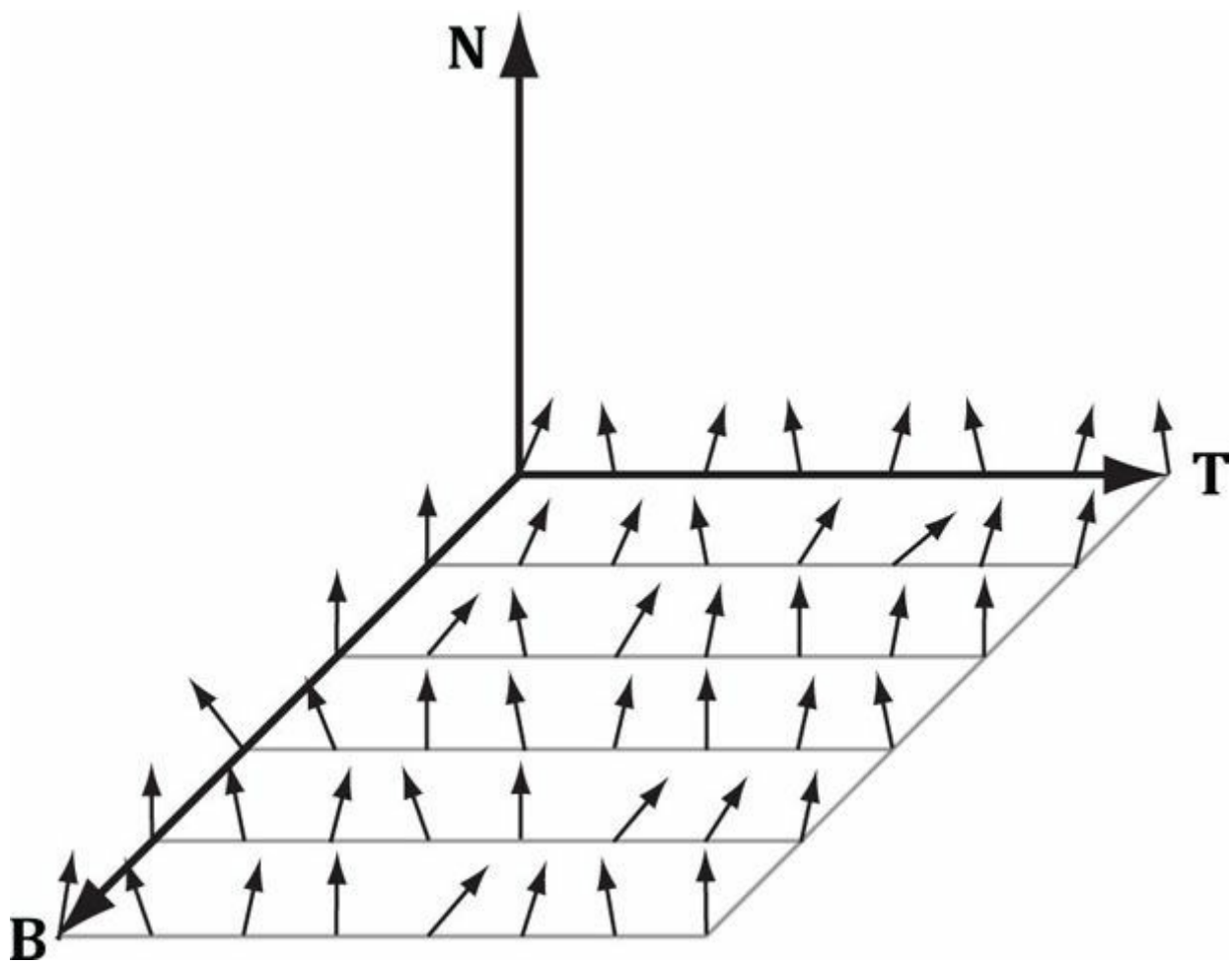


그림 18.3. 벡터 T(x축), B(y축), N(z축)으로 정의된 텍스처 공간 좌표계에 대한 노멀 맵에 저장된 노멀. T 벡터는 텍스처 이미지에 대해 오른쪽 수평 방향으로, B 벡터는 텍스처 이미지에 대해 아래쪽 수직 방향으로, N은 텍스처 평면에 직교합니다.

Note 그림 18.3에서 볼 수 있듯이, 벡터들은 일반적으로 대부분 z축과 정렬되어 있습니다. 즉, z좌표가 가장 큰 크기를 가집니다. 결과적으로 노멀 맵은 컬러 이미지로 볼 때 대부분 파란색으로 나타납니다. 이는 z좌표가 블루 채널에 저장되며 가장 큰 크기를 가지기 때문에 이 색상이 지배적이기 때문입니다.

그렇다면 단위 벡터를 이 형식으로 어떻게 압축할까요? 먼저 단위 벡터의 각 좌표는 항상 $[-1, 1]$ 범위에 있음을 유의하세요. 이 범위를 $[0, 1]$ 로 이동 및 스케일링하고 255를 곱한 후 소수점을 잘라내면 결과는 0–

255. 즉, x 가 $[-1, 1]$ 범위의 좌표라면, $f(x)$ 의 정수 부분은 0–255 범위의 정수이며, 여기서 f 는 다음과 같이 정의된다.

$$f(x) = (0.5x + 0.5) \cdot 255$$

따라서 24비트 이미지에 단위 벡터를 저장하려면 각 좌표에 f 를 적용한 후 해당 좌표를 텍스처 맵의 대응하는 색상 채널에 기록하면 됩니다.

다음 질문은 압축 과정을 역으로 수행하는 방법, 즉 0–255 범위 내의 압축된 텍스처 좌표가 주어졌을 때

$[-1, 1]$ 구간에서 함수 f 의 진정한 값을 어떻게 복원할 수 있을까? 답은 단순히 함수 f 를 역함수로 만드는 것이다. 잠시 생각해보면 다음과 같음을 알 수 있다:

$$f^{-1}(x) = \frac{2x}{255} - 1$$

즉, x 가 0–255 범위의 정수라면, $f^{-1}(x)$ 는 $[-1, 1]$ 범위의 부동 소수점 숫자가 됩니다.

이미지를 노멀 맵으로 변환하는 작업은 포토샵 플러그인을 사용할 것으로 직접 압축 과정을 수행할 필요는 없습니다.

그러나 픽셀 셰이더에서 노멀 맵을 샘플링할 때는 압축을 해제하기 위해 역과정의 일부를 수행해야 합니다. 다음과 같은 셰이더에서 노멀 맵을 샘플링할 때

```
float3 normalT = gNormalMap.Sample(gTriLinearSam, pin.Tex);
```

색상 벡터 **normalT**는 정규화된 성분 (r, g, b)을 가지며, $0 \leq r, g, b \leq 1$ 입니다.

따라서 이 메서드는 이미 압축 해제 작업의 일부(즉, 정수를 255로 나누는 연산)를 수행해 주었습니다.

0~255 범위의 값을 부동 소수점 구간 [0, 1]로 변환합니다. 각 성분을 [0, 1]에서 [-1, 1]로 이동 및 스케일링하여 변환을 완료합니다. 이를 위해 함수 $g: [0, 1] \rightarrow [-1, 1]$ 을 다음과 같이 정의합니다:

$$g(x) = 2x - 1$$

코드에서는 이 함수를 각 색상 구성 요소에 다음과 같이 적용합니다:

```
// 각 성분을 [0,1]에서 [-1,1]로 압축 해제합니다. normalT = 2.0f*normalT - 1.0f;
```

이것이 작동하는 이유는 스칼라 1.0이 벡터 (1, 1, 1)로 확장되어 표현식이 의미 있게 되고, 각 성분별로 처리되기 때문입니다.

포토샵 플러그인은 <http://developer.nvidia.com/nvidia-texture-tools-adobe-photoshop>에서 이용 가능합니다. 노멀 맵 생성을 위한 다른 도구로는  및 <http://www.crazybump.com/> 등이 있습니다. 또한 고해상도 메시로부터 노멀 맵을 생성할 수 있는 도구도 있습니다(http://www.nvidia.com/object/melody_home.html 참조). 고해상도 메시로부터 노멀 맵을 생성할 수 있는 도구들도 있습니다(http://www.nvidia.com/object/melody_home.html 참조).

 노멀 맵 저장에 압축 텍스처 형식을 사용하려면 최상의 품질을 위해 BC7(DXGI_FORMAT_BC7_UNORM) 형식을 사용하십시오. 이 형식은 노멀 맵 압축으로 인한 오류를 크게 줄여줍니다. BC6 및 BC7 형식에 대해서는 DirectX SDK에 "BC6HBC7EncoderDecoder11"이라는 샘플이 있습니다. 이 프로그램을 사용하면 텍스처 파일을 BC6 또는 BC7로 변환할 수 있습니다.

18.3 텍스처/탄젠트 공간

3차원 텍스처 매핑된 삼각형을 고려해 보자. 논의를 위해 텍스처 매핑에 왜곡이 없다고 가정하자. 즉, 텍스처 삼각형을 3차원 삼각형에 매핑하는 데는 강제 변환(이동과 회전)만 필요하다는 뜻이다. 이제 텍스처가 데칼과 같다고 가정해 보자. 따라서 데칼을 들어 올려 이동시키고 회전시켜 3D 삼각형 위에 배치한다. 따라서 [그림 18.4](#)는 텍스처 공간 축이 3D 삼각형과 어떻게 관련되는지 보여준다. 이 축들은 삼각형에 접하며 삼각형의 평면에 놓여 있다. 삼각형의 텍스처 좌표는 당연히 텍스처 공간 좌표계에 상대적이다. 삼각형 면 법선 **N**을 포함하면 삼각형 평면 내에 3D *TBN* 기반을 얻게 되는데, 이를 *텍스처 공간* 또는 *접선 공간*이라 부른다. 접선 공간은 일반적으로 삼각형마다 다르다는 점에 유의하십시오([그림 18.5](#) 참조).

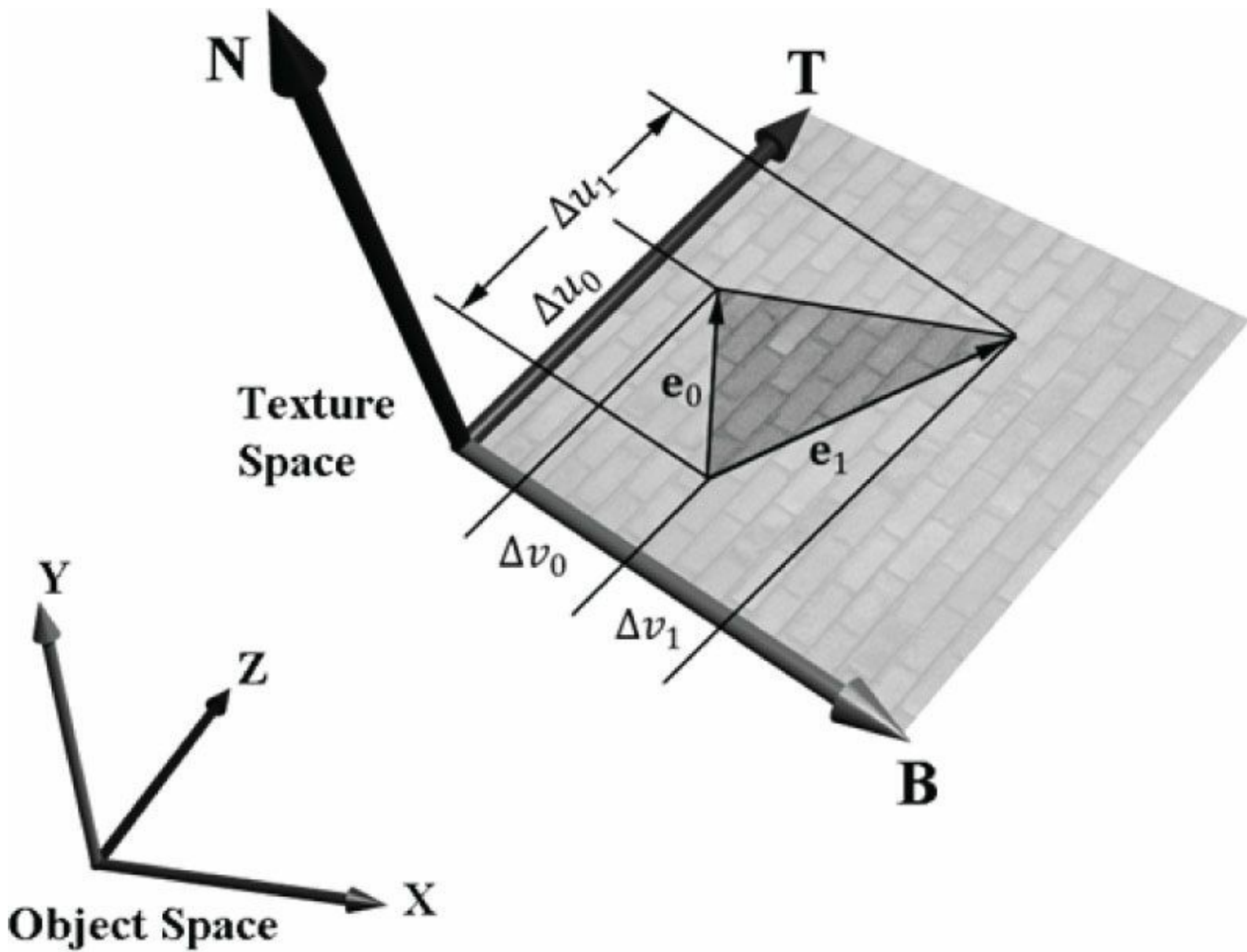


그림 18.4. 삼각형의 텍스처 공간과 객체 공간 간의 관계. 3차원 접선 벡터 **T**는 텍스처링 좌표계의 **u**축 방향을 향하며, 3차원 접선 벡터 **B**는 텍스처링 좌표계의 **v**축 방향을 향합니다.

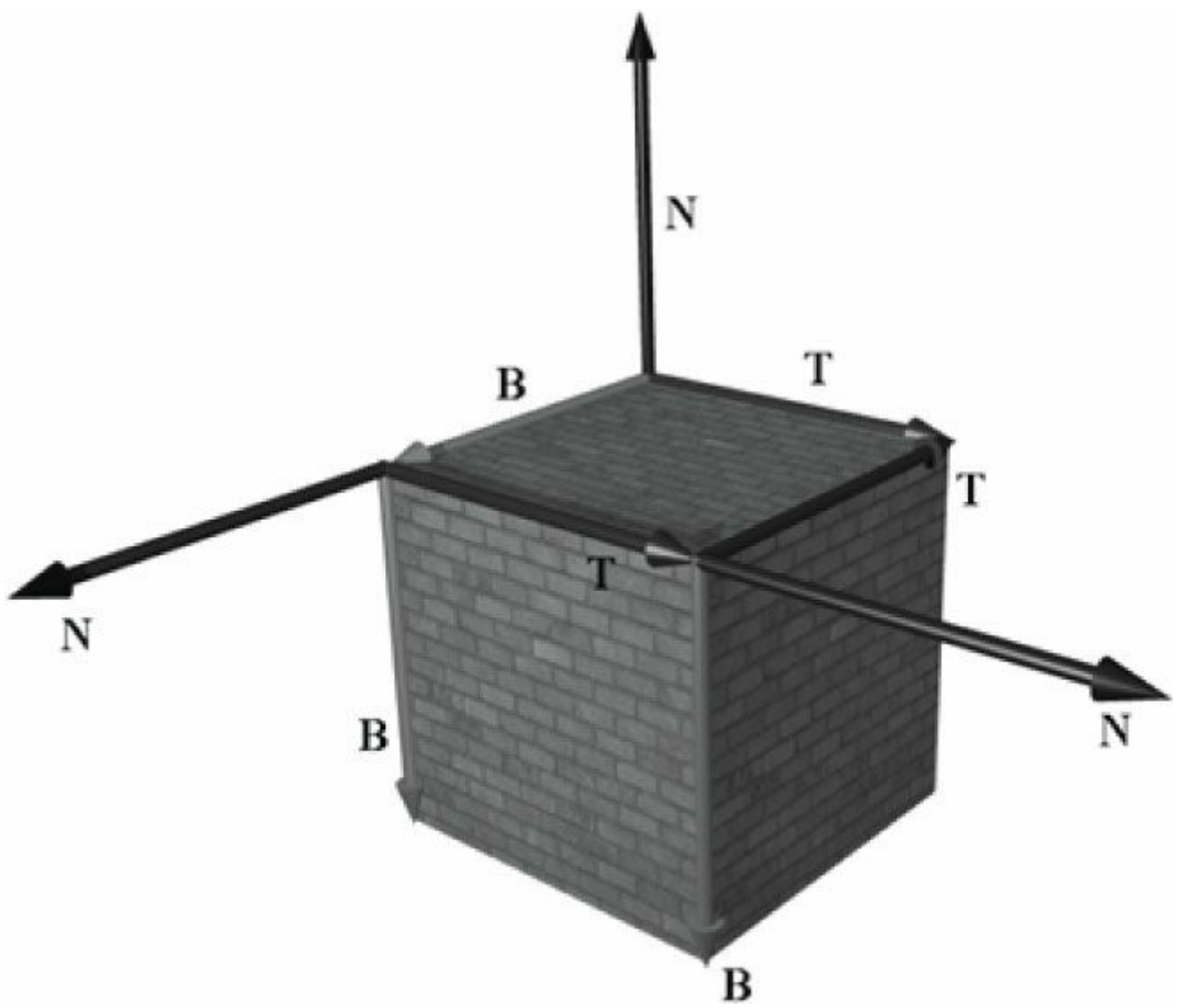


그림 18.5. 상자의 각 면마다 텍스처 공간이 다릅니다.

그림 18.3에서 볼 수 있듯이, 노멀 맵의 법선 벡터는 텍스처 공간을 기준으로 정의됩니다. 그러나 우리의 조명은 월드 공간에서 정의됩니다. 조명을 수행하려면 법선 벡터와 조명이 동일한 공간에 있어야 합니다. 따라서 첫 번째 단계는 접선 공간 좌표계를 삼각형 정점이 상대적인 오브젝트 공간 좌표계와 연관시키는 것입니다. 오브젝트 공간에 도달하면 월드 행렬을 사용하여 오브젝트 공간에서 월드 공간으로 이동할 수 있습니다(자세한 내용은 다음 섹션에서 다룹니다). 텍스처 평면에서 텍스처 공간 축(즉, T축과 B축)에 상대적인 삼각형을 정의하는 세 꼭짓점 $v_0, v_{(1)}, v_{(2)}$ 와 해당 텍스처 좌표 $(u_{(0)}, v_{(0)}), (u_{(1)}, v_{(1)}), (u_{(2)}, v_{(2)})$ 를 가지는 3차원 삼각형의 세 꼭짓점을 정의한다고 합시다. 이 삼각형은 텍스처 평면에서 텍스처 공간 축(즉, T축과 B축)에 대해 정의됩니다. $e_0 = v_1 - v_0$ 및 $e_1 = v_2 - v_0$ 을 3차원 삼각형의 두 변 벡터로 정의하며, 이에 대응하는 텍스처 삼각형 변 벡터는 $(\Delta u_0, \Delta v_0) = (u_1 - u_0, v_1 - v_0)$ 및 $(\Delta u_1, \Delta v_1) = (u_2 - u_0, v_2 - v_0)$ 를 가진다. 그림 18.4에서 명확히 알 수 있듯이

$$e_0 = \Delta u_0 \mathbf{T} + \Delta v_0 \mathbf{B} \quad e_1 = \Delta u_1 \mathbf{T} + \Delta v_1 \mathbf{B}$$

벡터를 물체 공간에 대한 상대 좌표로 표현하면, 행렬 방정식은 다음과 같습니다:

$$\begin{bmatrix} e_{0,x} & e_{0,y} & e_{0,z} \\ e_{1,x} & e_{1,y} & e_{1,z} \end{bmatrix} = \begin{bmatrix} \Delta u_0 & \Delta v_0 \\ \Delta u_1 & \Delta v_1 \end{bmatrix} \begin{bmatrix} T_x & T_y & T_z \\ B_x & B_y & B_z \end{bmatrix}$$

삼각형 꼭짓점의 객체 공간 좌표는 알려져 있으므로, 모서리 벡터의 객체 공간 좌표도 알 수 있습니다. 따라서 행렬

$$\begin{bmatrix} e_{0,x} & e_{0,y} & e_{0,z} \\ e_{1,x} & e_{1,y} & e_{1,z} \end{bmatrix}$$

는 알려져 있다. 마찬가지로 텍스처 좌표도 알고 있으므로 행렬

$$\begin{bmatrix} \Delta u_0 & \Delta v_0 \\ \Delta u_1 & \Delta v_1 \end{bmatrix}$$

도 알려져 있습니다. T와 B의 객체 공간 좌표를 구하면 다음과 같습니다:

$$\begin{bmatrix} T_x & T_y & T_z \\ B_x & B_y & B_z \end{bmatrix} = \begin{bmatrix} \Delta u_0 & \Delta v_0 \\ \Delta u_1 & \Delta v_1 \end{bmatrix}^{-1} \begin{bmatrix} e_{0,x} & e_{0,y} & e_{0,z} \\ e_{1,x} & e_{1,y} & e_{1,z} \end{bmatrix}$$

$$= \frac{1}{\Delta u_0 \Delta v_1 - \Delta v_0 \Delta u_1} \begin{bmatrix} \Delta v_1 & -\Delta v_0 \\ -\Delta u_1 & \Delta u_0 \end{bmatrix} \begin{bmatrix} e_{0,x} & e_{0,y} & e_{0,z} \\ e_{1,x} & e_{1,y} & e_{1,z} \end{bmatrix}$$

위에서 우리는 행렬의 역행렬이

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

다음과 같이 주어진다는 사실을 이용했습니다:

$$\mathbf{A}^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

벡터 T와 B는 일반적으로 객체 공간에서 단위 길이가 아니며, 텍스처 왜곡이 존재할 경우 직교 정규화되지도 않습니다.

T, B, N 벡터는 각각 *접선* 벡터, *이접선 벡터*(또는 *이접선*), *법선* 벡터로 흔히 불립니다.

18.4 정점 접선 공간

이전 섹션에서는 삼각형당 접공간을 유도했습니다. 그러나 이 텍스처 공간을 노멀 매핑에 사용하면 삼각형 면 전체에 걸쳐 접공간이 일정하기 때문에 삼각형 모양의 외관이 나타납니다. 따라서 정점당 접벡터를 지정하고, 정점 노멀에서 했던 것과 동일한 평균화 기법을 적용하여 매끄러운 표면을 근사화합니다:

1. 메쉬 내 임의의 정점 **v**에 대한 접선 벡터 **T**는 해당 정점 **v**를 공유하는 메쉬 내 모든 삼각형의 접선 벡터를 평균하여 구한다.
2. 메쉬 내 임의의 정점 **v**에 대한 이접선 벡터 **B**는 해당 정점 **v**를 공유하는 메쉬 내 모든 삼각형의 이접선 벡터들을 평균하여 구합니다.

일반적으로 평균화 후 TBN-기저(TBN-bases)는 벡터들이 서로 직교하고 길이가 1이 되도록 직교 정규화(orthonormalization)해야 합니다. 이는 일반적으로 그램-슈미트 절차(Gram-Schmidt procedure)를 사용하여 수행됩니다. 임의의 삼각형 메쉬에 대해 정점별 접공간(tangent space)을 구축하는 코드는 웹에서 확인할 수 있습니다: <http://www.terathon.com/code/tangent.html>.

본 시스템에서는 이접선 벡터 **B**를 메모리에 직접 저장하지 않습니다. 대신 **B**가 필요할 때 **B** = **N** × **T**를 계산합니다.

여기서 **N**은 일반적인 평균화된 정점 법선 벡터입니다. 따라서 우리의 정점 구조는 다음과 같습니다:

```
namespace Vertex
{
    struct NormalMap
    {
        XMFLOAT3 Pos;
        XMFLOAT3 Normal;
        XMFLOAT2 Tex;
        XMFLOAT3 접선U;
    };
}
```

}

기억하시다시피, **GeometryGenerator**로 생성된 절차적 메쉬는 텍스처 공간의 u 축에 대응하는 접선 벡터 $\mathbf{T}$ 를 계산합니다. 각 꼭지점에서 접선 벡터 $\mathbf{T}$ 의 객체 공간 좌표는 박스 및 그리드 메쉬의 경우 쉽게 지정할 수 있습니다([그림 18.5](#) 참조). 원통 및 구체의 경우, 각 꼭지점의 접선 벡터 $\mathbf{T}$ 는 원통/구체의 두 변수 벡터 함수 $\mathbf{P}(u,v)$ 를 형성하고 $\partial \mathbf{P} / \partial u$ 를 계산함으로써 구할 수 있습니다. 여기서 매개변수 u 는 u -텍스처 좌표로도 사용됩니다.

18.5 접선 공간과 객체 공간 간의 변환

지금까지 메쉬의 각 정점에서 직교 정규화된 TBN 기저를 얻었습니다. 또한 메쉬의 객체 공간에 대한 TBN 벡터의 좌표도 확보했습니다. 따라서 객체 공간 좌표계에 대한 TBN 기저의 좌표를 알게 되었으므로, 다음 행렬을 사용하여 접선 공간에서 객체 공간으로 좌표를 변환할 수 있습니다:

$$\mathbf{M}_{object} = \begin{bmatrix} T_x & T_y & T_z \\ B_x & B_y & B_z \\ N_x & N_y & N_z \end{bmatrix}$$

이 행렬은 직교 행렬이므로 역행렬은 전치 행렬과 동일합니다. 따라서 대상 공간에서 접선 공간으로의 좌표 변환 행렬은 다음과 같습니다:

$$\mathbf{M}_{tangent} = \mathbf{M}_{object}^{-1} = \mathbf{M}_{object}^T = \begin{bmatrix} T_x & B_x & N_x \\ T_y & B_y & N_y \\ T_z & B_z & N_z \end{bmatrix}$$

셰이더 프로그램에서는 실제로 조명 처리를 위해 법선 벡터를 접선 공간에서 세계 공간으로 변환해야 합니다. 한 가지 방법은 법선을 먼저 접선 공간에서 객체 공간으로 변환한 후, 세계 행렬을 사용하여 객체 공간에서 세계 공간으로 변환하는 것입니다:

$$n_{world} = (M_{tangent} M_{object}) n_{world}$$

그러나 행렬 곱셈은 결합법칙을 따르므로 다음과 같이 수행할 수 있습니다:

$$n_{world} = M_{tangent} (M_{object} n_{world})$$

또한 다음 사항에 유의하세요:

$$\mathbf{M}_{object} \mathbf{M}_{world} = \begin{bmatrix} \leftarrow \mathbf{T} \rightarrow \\ \leftarrow \mathbf{B} \rightarrow \\ \leftarrow \mathbf{N} \rightarrow \end{bmatrix} \mathbf{M}_{world} = \begin{bmatrix} \leftarrow \mathbf{T}' \rightarrow \\ \leftarrow \mathbf{B}' \rightarrow \\ \leftarrow \mathbf{N}' \rightarrow \end{bmatrix} = \begin{bmatrix} T'_x & T'_y & T'_z \\ B'_x & B'_y & B'_z \\ N'_x & N'_y & N'_z \end{bmatrix}$$

여기서 $\mathbf{T}' = \mathbf{T} \cdot \mathbf{M}_{world}$, $\mathbf{B}' = \mathbf{B} \cdot \mathbf{M}_{world}$, $\mathbf{N}' = \mathbf{N} \cdot \mathbf{M}_{world}$ 이다. 따라서 접공간에서 세계 공간으로 직접 이동하려면, 접공간 기저를 세계 좌표계로 표현하기만 하면 되는데, 이는 TBN-기저를 객체 공간 좌표계에서 세계 공간 좌표계로 변환함으로써 달성할 수 있다.

 우리는 벡터(점이 아닌)의 변환에만 관심이 있습니다. 따라서 3×3 행렬만 필요합니다. 어떤 행렬의 네 번째 행은 평행 이동을 위한 것이지만, 우리는 벡터를 이동시키지 않습니다.

18.6 노멀 매핑 셰이더 코드

노멀 매핑의 일반적인 과정을 요약하면 다음과 같습니다:

1. 일부 아트 프로그램이나 유틸리티 프로그램에서 원하는 노멀 맵을 생성하고 이미지 파일로 저장합니다. 프로그램 초기화 시 이 파일들로부터 2D 텍스처를 생성합니다.
2. 각 삼각형에 대해 접선 벡터 **T**를 계산합니다. 메쉬 내 각 정점 **v**에 대해, 해당 정점 **v**를 공유하는 메쉬 내 모든 삼각형의 접선 벡터를 평균화하여 정점별 접선 벡터를 구합니다. (본 데모에서는 단순한 기하학을 사용하므로 접선 벡터를 직접 지정할 수 있지만, 3D 모델링 프로그램에서 생성된 임의의 삼각형 메쉬를 사용할 경우 이 평균화 과정이 필요합니다.)
3. 버텍스 셰이더에서 버텍스 법선 벡터와 접선 벡터를 월드 공간으로 변환하고 결과를 픽셀 셰이더로 출력합니다.
4. 보간된 접선 벡터와 법선 벡터를 사용하여 삼각형 표면의 각 픽셀 지점에 TBN 베이스를 구축합니다. 이 베이스를 활용해 노멀 맵에서 샘플링된 법선 벡터를 접선 공간에서 월드 공간으로 변환합니다. 이렇게 하면 노멀 맵에서 얻은 월드 공간 법선 벡터를 일반적인 조명 계산에 사용할 수 있습니다.

노멀 매핑 구현을 지원하기 위해 *lighthelper.fx*에 다음 함수를 추가했습니다:

```
//-----
// 노멀 맵 샘플을 월드 공간으로 변환합니다.
//-----
float3 NormalSampleToWorldSpace(float3 normalMapSample, float3 unitNormalW,
                                float3 tangentW)
{
    // 각 성분을 [0,1]에서 [-1,1]로 압축 해제합니다. float3 normalT =
    2.0f*normalMapSample - 1.0f;

    // 직교 기저를 구성합니다. float3 N =
    unitNormalW;
    float3 T = 정규화(접선W - 점곱(접선W, N)*N); float3 B = 교차(N, T);

    float3x3 TBN = float3x3(T, B, N);

    // 접선 공간에서 세계 공간으로 변환. float3 bumpedNormalW =
    mul(normalT, TBN);

    return bumpedNormalW;
}
```

이 함수는 픽셀 셰이더에서 다음과 같이 사용됩니다:

```
float3 normalMapSample = gNormalMap.Sample(samLinear, pin.Tex).rgb; float3 bumpedNormalW =
NormalSampleToWorldSpace(
    normalMapSample,
    pin.NormalW, pin.TangentW);
```

다음 두 줄이 명확하지 않을 수 있습니다:

```
float3 N = unitNormalW;
float3 T = 정규화(접선W - 점곱(접선W, N)*N);
```

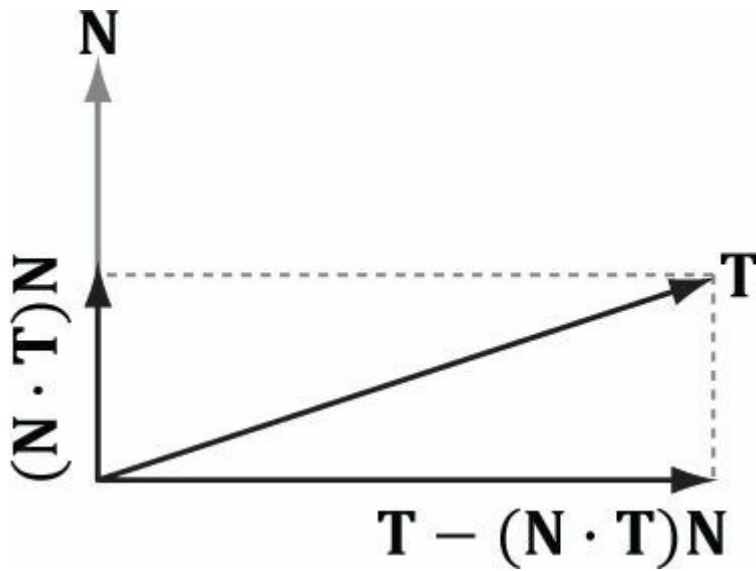


그림 18.6. $\|N\| = 1$ 이므로, $\text{proj}_N(T) = (T \cdot N)N$ 이다. 벡터 $T - \text{proj}_N(T)$ 는 T 중 N 에 직교하는 부분이다.

보간 후 접선 벡터와 법선 벡터가 직교 정규화되지 않을 수 있습니다. 이 코드는 **T가 N에** 대해 직교 정규화되도록 보장합니다. (그림 18.6 참조). 단, **unitNormalW**가 정규화되었다는 가정이 있음을 유의하십시오.

노멀 맵에서 노멀을 추출한 후, 이를 "범프 노멀"이라 부릅니다. 이후 모든 후속 계산(예: 조명, 큐브 매핑)에 이 범프 노멀을 사용합니다. (예: 조명, 큐브 매핑). 노멀 매핑과 관련된 부분을 굵게 표시하여 전체 노멀 매핑 효과를 아래에 완전하게 보여줍니다.

```
#include "LightHelper.fx" cbuffer

cbPerFrame
{
    DirectionalLight gDirLights[3]; float3
    gEyePosW;

    float gFogStart; float
    gFogRange; float4
    gFogColor;
};

cbuffer cbPerObject
{
    float4x4 gWorld;
    float4x4 gWorldInvTranspose; float4x4
    gWorldViewProj; float4x4 gTexTransform;
    Material gMaterial;
};

// 숫자가 아닌 값은 cbuffer에 추가할 수 없습니다. Texture2D
gDiffuseMap;
Texture 2D gNormalMap;
TextureCube gCubeMap;

샘플러 상태 samLinear
{
    Filter = MIN_MAG_MIP_LINEAR; AddressU =
    WRAP;
    AddressV = WRAP;
};

struct VertexIn
{
    float3 PosL           : POSITION;
```

```

float3 NormalL : NORMAL; float2 Tex :
TEXCOORD; float3 TangentL :
TANGENT;

};

struct VertexOut
{
    float4 PosH          : SV_POSITION; float3
    PosW                  : POSITION; float3
    NormalW               : NORMAL; float3
    TangentW : TANGENT; float2 Tex    : 텍스처
    좌표;
};

VertexOut VS(VertexIn vin)
{
    VertexOut vout;

    // 월드 공간으로 변환.
    vout.PosW = mul(float4(vin.PosL, 1.0f), gWorld).xyz; vout.NormalW = mul(vin.NormalL,
(float3x3)gWorldInvTranspose); vout.TangentW = mul(vin.TangentL, (float3x3)gWorld);

    // 동차 클립 공간으로 변환.
    vout.PosH = mul(float4(vin.PosL, 1.0f), gWorldViewProj);

    // 삼각형 간 보간을 위한 정점 속성 출력. vout.Tex = mul(float4(vin.Tex, 0.0f, 1.0f),
gTexTransform).xy;

    return vout;
}

float4 PS(VertexOut pin, uniform int
gLightCount, uniform bool
gUseTexure, uniform bool
gAlphaClip, uniform bool
gFogEnabled,
uniform bool gReflectionEnabled) : SV_Target
{
    // 보간된 법선은 비정규화될 수 있으므로 정규화합니다. pin.NormalW =
normalize(pin.NormalW);

    // toEye 벡터는 조명 계산에 사용됩니다. float3 toEye =
gEyePosW - pin.PosW;

    // 이 표면 지점에서 눈까지의 거리를 캐시합니다. float distToEye = length(toEye);

    // 정규화합니다.
    toEye /= distToEye;

    // 기본값으로 곱셈 항등원 설정. float4 texColor =
float4(1, 1, 1, 1);

    if(gUseTexure)
    {
        // 텍스처 샘플링.
        텍스처 색상 = gDiffuseMap.샘플링(samLinear, pin.Tex);

        if(gAlphaClip)
        {
            // 텍스처 알파 < 0.1인 픽셀은 버림. 참고:
            // 이 테스트를 가능한 한 빨리 수행하여

```

```

// 셰이더를 조기에 종료할 수 있도록 하여
// 나머지 셰이더 코드를 건너뛸 수 있도록 합니다. clip(texColor.a - 0.1f);

}
}

//
// 노멀 매핑
//

float3 normalMapSample = gNormalMap.Sample(samLinear, pin.Tex).rgb; float3 bumpedNormalW =
NormalSample ToWorldSpace( normalMapSample, pin.NormalW, pin.TangentW);

//
// 조명.
//

float4 litColor = texColor; if(gLightCount > 0)
{
    // 합계를 0으로 시작합니다.
    float4 ambient = float4(0.0f, 0.0f, 0.0f, 0.0f); float4 diffuse =
    float4(0.0f, 0.0f, 0.0f, 0.0f); float4 spec = float4(0.0f, 0.0f, 0.0f,
    0.0f);

    // 각 광원의 빛 기여도를 합산합니다. [unroll]
    for(int i = 0; i < gLightCount; ++i)
    {
        float4 A, D, S; ComputeDirectionalLight(gMaterial,
        gDirLights[i], bumpedNormalW, toEye, A, D, S);

        ambient += A;
        diffuse += D; spec+=
        S;
    }
    litColor = texColor*(ambient + diffuse) + spec;

    if(gReflectionEnabled)
    {
        float3 incident = -toEye;
        float3 reflectionVector = reflect( incident,
        bumpedNormalW);
        float4 reflectionColor = gCubeMap.Sample( samLinear,
        reflectionVector);

        litColor += gMaterial.Reflect * reflectionColor;
    }
}

//
// 포깅
//

if(gFogEnabled)
{
    float fogLerp = saturate((distToEye - gFogStart) / gFogRange);

    // 안개 색상과 조명 색상을 블렌딩합니다. litColor = lerp(litColor,
    gFogColor, fogLerp);
}

```

```

}

// 디퓨즈 머티리얼과 텍스처에서 알파를 가져오는 공통 처리. litColor.a =
gMaterial.Diffuse.a * texColor.a;

return litColor;
}

```

18.7 변위 매핑

이제 노멀 매핑에 대한 이해를 바탕으로 테셀레이션 및 변위 매핑과 결합하여 효과를 향상시킬 수 있습니다. 그 이유는 노멀 매핑이 조명 디테일만 개선할 뿐 실제 지오메트리의 디테일을 개선하지 않기 때문입니다. 따라서 노멀 매핑은 일종의 조명 기법이라고 할 수 있습니다([그림 18.7](#) 참조).

변위 매핑의 개념은 *높이맵*(*heightmap*)이라 불리는 추가 맵을 활용하는 것입니다. 이 맵은 지형의 융기나 함몰을 묘사하며, 표면. 즉, 노멀 맵이 각 픽셀에 대해 노멀 벡터 (x, y, z) 를 생성하기 위해 세 개의 색상 채널을 사용하는 반면, 하이트맵은 각 픽셀에 높이 값 h 를 생성하기 위해 단일 색상 채널을 사용합니다. 시각적으로 높이맵은 단일 색상 채널로 인해 회색조 이미지로 보이며, 각 픽셀은 높이 값으로 해석됩니다. 이는 기본적으로 2차원 스칼라 필드 $h = f(x, z)$ 의 이산적 표현입니다. 메쉬를 테셀레이션할 때 도메인 셰이더에서 높이맵을 샘플링하여 정점들을 노멀 벡터 방향으로 오프셋함으로써 메쉬에 기하학적 디테일을 추가합니다([그림 18.8](#) 참조).

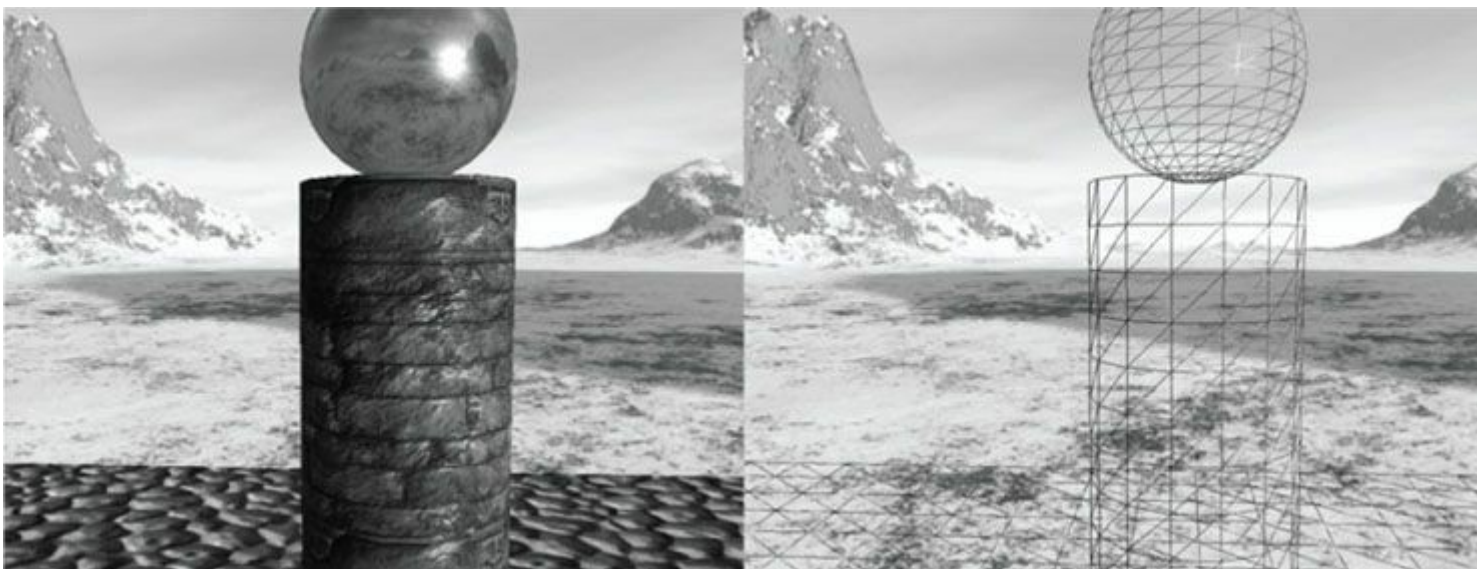


그림 18.7. 노멀 맵은 요철과 틸새의 착시 효과를 만들어 내지만, 와이어프레임을 보면 지오메트리가 매끄럽다는 것을 알 수 있습니다.

테셀레이션 기하학은 삼각형을 추가하지만, 그 자체로는 디테일을 추가하지 않습니다. 즉, 삼각형을 여러 번 세분화하면 원래 삼각형 평면에 놓인 삼각형이 더 많아질 뿐입니다. 디테일(예: 돌출부와 홈)을 추가하려면 테셀레이션된 정점을 어떤 방식으로든 오프셋해야 합니다. 높이맵은 테셀레이션된 정점을 변위시키는 데 사용할 수 있는 입력 소스 중 하나입니다. 일반적으로 정점 위치 $\mathbf{p}$ 를 변위시키기 위해 다음 공식이 사용되며, 이때 외부 표면 법선 벡터 $\mathbf{n}$ 을 변위 방향으로 활용합니다:

$$\mathbf{p}' = \mathbf{p} + s(\mathbf{h} - 1)\mathbf{n}$$

스칼라 $h \in [0, 1]$ 은 높이맵에서 얻은 스칼라 값입니다. $[0, 1]$ 구간을 $[-1, 0]$ 로 변환하기 위해 h 에서 1을 뺍니다. 메쉬에서 표면 법선이 바깥을 향하기 때문에, 이는 "*바깥쪽*"이 아닌 "*안쪽*"으로 *변위함*을 의미합니다. 이는 일반적인 관례입니다.

관례입니다. 일반적으로 지오메트리를 "튀어나오게" 하는 것보다 "안으로 튀어 들어가게" 하는 것이 더 편리하기 때문입니다. 스칼라 s 는 전체 높이를 특정 월드 공간 값으로 스케일링하는 배율 요소입니다. 하이트맵의 높이 값 h 는 정규화된 범위에 있으며, 0은 최저 높이(가장 안쪽으로 변위)를, 1은 최고 높이(변위 없음)를 의미합니다. 최대 5 월드 오프셋을 원한다고 가정해 보겠습니다.

유닛. 그런 다음 $s = 5$ 로 설정하여 $s(h - 1) \in [-5, 0]$ 이 되도록 합니다. 높이맵을 노멀맵의 알파 채널에 저장하는 것이 일반적입니다.

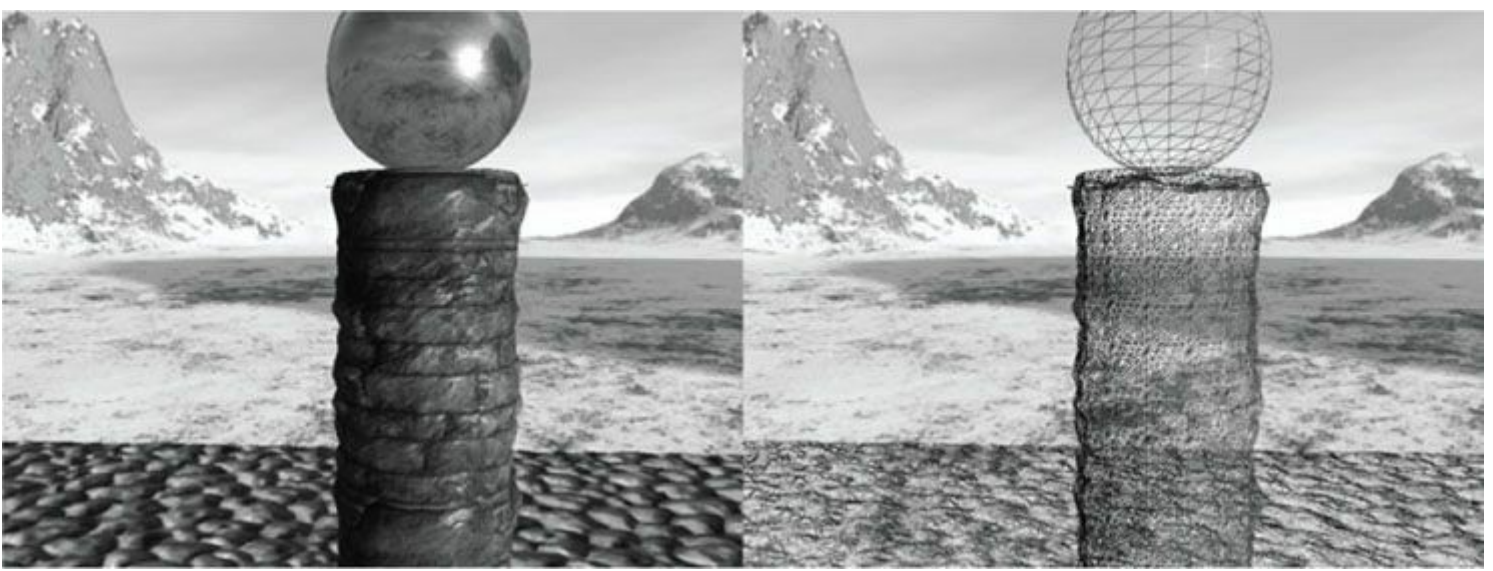


그림 18.8. 테셀레이션 단계에서는 지오메트리의 작은 돌출부와 홈을 모델링할 수 있도록 추가 삼각형을 생성합니다. 변위 맵은 정점들을 변위시켜 실제 지오메트리의 돌출부와 홈을 생성합니다.

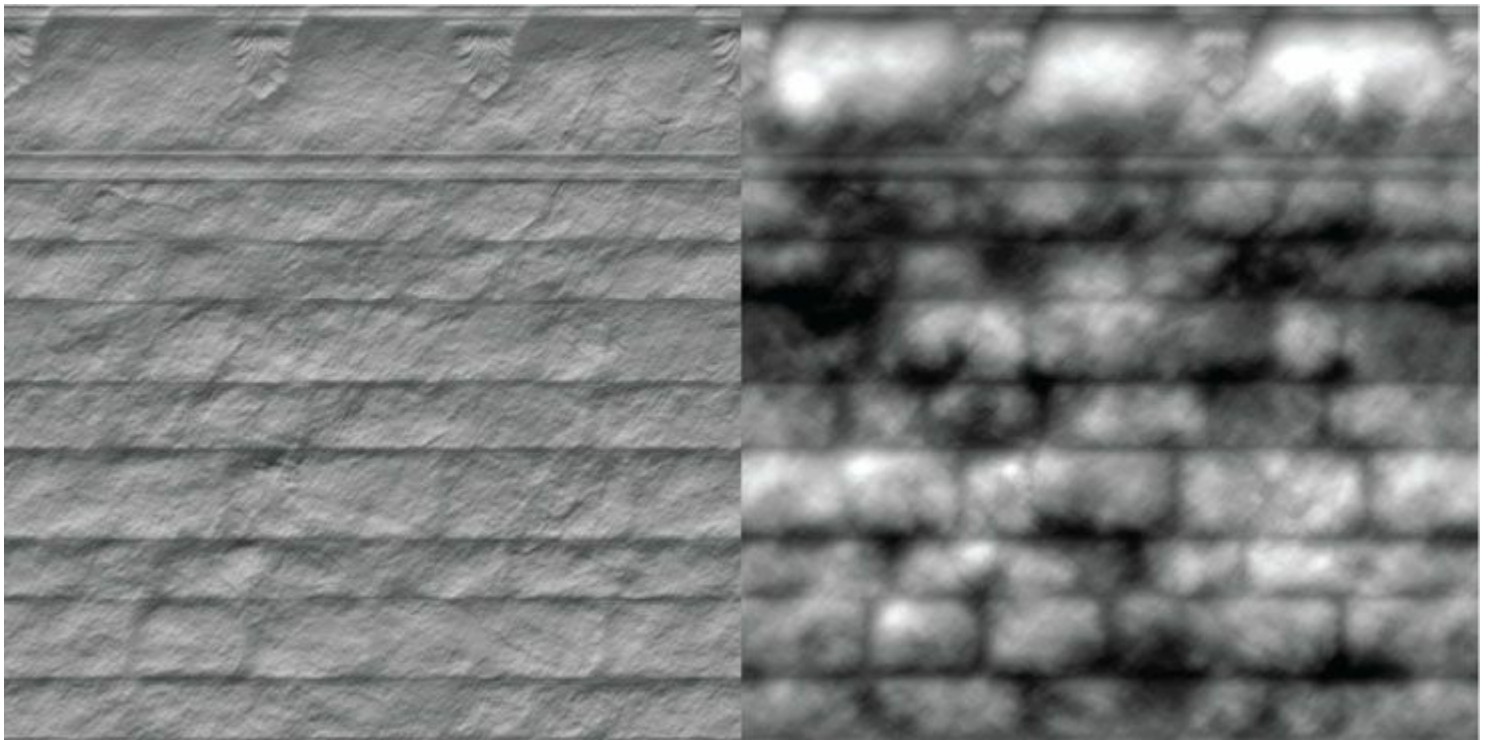


그림 18.9. (왼쪽) 노멀 맵. (오른쪽) 노멀 맵의 알파 채널에 저장된 하이트맵. 흰색 값은 가장 높은 높이를, 검은색 값은 가장 낮은 높이를 나타냅니다. 회색 값은 중간 높이를 나타냅니다.

하이맵 생성은 예술적 작업이며, 텍스처 아티스트는 이를 직접 그리거나 생성 도구를 사용할 수 있습니다(예: <http://www.crazybump.com/>).

18.8 변위 매핑 셰이더 코드

변위 매핑 코드의 대부분은 버텍스 셰이더, 헬 셰이더, 도메인 셰이더에서 발생합니다. 픽셀 셰이더는 노멀 매핑에 사용한 것과 동일합니다.

18.8.1 프리미티브 유형

렌더링에 변위 매핑을 통합하려면 테셀레이션을 지원해야 합니다. 이를 통해 기하학적 해상도를 변위 맵의 해상도에 더 잘 맞도록 조정할 수 있습니다. 기존 삼각형 메시에 테셀레이션을 매우 쉽게 통합하려면, `D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST` 대신 `D3D11_PRIMITIVE_TOPOLOGY_3_CONTROL_POINT_PATCHLIST` 프라이머리 유형으로 메쉬를 그리면 됩니다. 이렇게 하면 삼각형의 세 정점이 삼각형 패치의 3개 제어점으로 해석되어 각 삼각형을 테셀레이션할 수 있습니다.

18.8.2 버텍스 셰이더

테셀레이션을 다룰 때 각 삼각형을 얼마나 세분화할지 결정해야 합니다. 우리는 간단한 거리 측정법을 사용하여 테셀레이션 정도를 결정할 것입니다. 삼각형이 시점에 가까울수록 더 많은 테셀레이션을 받게 됩니다. 버텍스 셰이더는 각 버텍스에서 테셀레이션 계수를 계산하여 이 계수를 쉐이더로 전달함으로써 이 거리 계산에 도움을 줍니다.

거리 계산을 제어하기 위해 상수 버퍼에 다음 변수를 도입합니다. 이 변수들에 설정하는 값은 변수들은 장면에 크게 좌우됩니다(장면 크기와 원하는 테셀레이션 정도에 따라):

```
cbuffer cbPerFrame
{
    ...
    float gMaxTessDistance; float
    gMinTessDistance; float
    gMinTessFactor; float
    gMaxTessFactor;

};
```

- 1. **gMaxTessDistance**: 최대 테셀레이션이 달성되는 시점의 시선으로부터의 거리.
- 2. **gMinTessDistance**: 최소 테셀레이션이 달성되는 시점의 시점으로부터의 거리.
- 3. **gMinTessFactor**: 최소 테셀레이션 횟수. 예를 들어, 삼각형이 아무리 멀리 떨어져 있어도 최소 3회 이상 테셀레이션되도록 설정할 수 있습니다.
- 4. **gMaxTessFactor**: 최대 테셀레이션 횟수. 예를 들어, 추가 디테일이 필요하지 않으므로 6회 이상 테셀레이션할 필요가 없다고 판단할 수 있습니다. 필요한 테셀레이션 횟수는 테셀레이션 전 입력 메시의 삼각형 수에 따라 달라집니다. 또한 [13장에서](#) 언급한 조언을 기억하세요. 8픽셀 미만을 커버하는 삼각형은 비효율적입니다.

눈으로부터의 거리가 감소함에 따라 테셀레이션을 증가시킬 때 **gMaxTessDistance < gMinTessDistance**임을 관찰하십시오. 이러한 변수를 사용하여 거리에 따른 테셀레이션 정도를 결정하는 거리의 선형 함수를 만들 수 있습니다(이 함수는 함수는 §13.5에서 사용된 것과 유사합니다).

```
struct VertexIn
{
    float3 PosL      : 위치; float3
    NormalL : 법선; float2 Tex :
    TEXCOORD; float3 TangentL :
    TANGENT;

};

struct VertexOut
{
    float3 PosW      : POSITION; float3
    NormalW          : NORMAL; float3
    TangentW         : TANGENT; float2
    Tex              : 텍스처 좌표; float 테
    스처 팩터 : 테스처;

};
```

정점출력 VS(정점입력 vin)

```
{
    VertexOut vout;

    // 월드 공간으로 변환.
    vout.PosW      = mul(float4(vin.PosL, 1.0f), gWorld).xyz; vout.NormalW = mul(vin.NormalL,
(float3x3)gWorldInvTranspose); vout.TangentW = mul(vin.TangentL, (float3x3)gWorld);

    // 삼각형 간 보간을 위한 버텍스 속성 출력. vout.Tex = mul(float4(vin.Tex, 0.0f, 1.0f),
gTexTransform).xy;

    float d = distance(vout.PosW, gEyePosW);
    // 정규화된 테셀레이션 계수.
    // 테셀레이션은
    // d >= gMinTessDistance이면 0이고
```

```

// d <= gMaxTessDistance이면 1입니다.
float tess = saturate((gMinTessDistance - d) /
    (gMinTessDistance - gMaxTessDistance));

// [0,1]을 [gMinTessFactor, gMaxTessFactor]로 재조정합니다. vout.TessFactor =
gMinTessFactor +
    tess*(gMaxTessFactor-gMinTessFactor);

return vout;
}

```

18.8.3 헬 셰이더

상수 헬 셰이더는 패치별로 평가되며, 메시의 소위 테셀레이션 계수를 출력하는 역할을 합니다. 테셀레이션 계수는 패치를 얼마나 테셀레이션할지 테셀레이션 단계에 지시합니다. 테셀레이션 계수 결정 작업의 대부분은 버텍스 셰이더에서 수행되었지만, 상수 헬 셰이더가 처리해야 할 작업이 조금 남아 있습니다. 구체적으로, 에지 테셀레이션 계수는 버텍스 테셀레이션 계수의 평균을 계산하여 구합니다. 내부 테셀레이션 계수의 경우, 첫 번째 에지 테셀레이션 계수를 임의로 할당합니다.

```

struct PatchTess
{
    float EdgeTess[3] : SV_TessFactor;
    float InsideTess : SV_InsideTessFactor;
};

PatchTess PatchHS(InputPatch<VertexOut,3> patch, uint patchID :
    SV_PrimitiveID)
{
    PatchTess pt;

    // 가장자리를 따라 평균 정점 테셀레이션 계수.
    // 테셀레이션 계수 계산을
    // 에지를 공유하는 삼각형이 동일한 테셀레이션 계수를 가지지 않도록
    // 동일한 테셀레이션 계수를 가지도록 하는 것이 중요합니다.
    // 그렇지 않으면 틈이 생길 수 있습니다.
    pt.EdgeTess[0] = 0.5f*(patch[1].TessFactor + patch[2].TessFactor); pt.EdgeTess[1] =
    0.5f*(patch[2].TessFactor + patch[0].TessFactor); pt.EdgeTess[2] = 0.5f*(patch[0].TessFactor
    + patch[1].TessFactor);

    // 내부 테셀레이션을 위한 에지 테셀레이션 계수를 선택합니다. pt.InsideTess = pt.EdgeTess[0];

    return pt;
}

```

하나 이상의 삼각형이 공유하는 변은 동일한 테셀레이션 계수를 가져야 합니다. 그렇지 않으면 메시에 균열이 발생할 수 있습니다(그림 18.10 참조). 공유 변이 동일한 테셀레이션 계수를 가지도록 보장하는 한 가지 방법은 이전 코드에서 수행한 것처럼 변의 속성만을 기반으로 테셀레이션 계수를 계산하는 것입니다. 테셀레이션 계수를 계산하지 말아야 할 방법의 예로, 눈과 삼각형 중심점 사이의 거리를 살펴 내부 테셀레이션 계수를 계산한다고 가정해 보겠습니다. 그런 다음 내부 테셀레이션 계수를 가장자리 테셀레이션 계수로 전파합니다. 인접한 두 삼각형이 서로 다른 테셀레이션 계수를 가지면, 그들의 가장자리도 서로 다른 테셀레이션 계수를 가지게 되어 T자형 교차점을 생성하게 됩니다. 이는 변위 매핑 후 균열로 이어질 수 있습니다.